

# SEU CÓDIGO, EXPLICADO

Java, PostgreSQL e a interface web

Uma leitura completa do sistema de aeroporto

|                  |                          |                        |
|------------------|--------------------------|------------------------|
| 67 arquivos Java | 325 métodos/construtores | 4.314 linhas           |
| Domínio e DTO    | Chamadas e retornos      | Condições e transações |

Guia de estudo em português, com código numerado, explicações próximas das instruções, mapas de dependências e caminhos completos das operações.

Leitura do projeto em 07 de outubro de 2026. Código apresentado como está nos arquivos: os números de linha não são de uma versão simplificada.

Use o sumário clicável ou os marcadores do leitor de PDF para chegar a qualquer classe.

# Sumário clicável

|                                              |           |
|----------------------------------------------|-----------|
| <b>Sumário clicável</b>                      | <b>2</b>  |
| <b>Como ler este guia</b>                    | <b>9</b>  |
| <b>Java: palavras, símbolos e tipos</b>      | <b>10</b> |
| <b>Mapa das camadas e conexões</b>           | <b>15</b> |
| Mapa visual dos dois caminhos . . . . .      | 16        |
| <b>Operações completas, de ponta a ponta</b> | <b>17</b> |
| <b>Leitura comentada: convenções</b>         | <b>19</b> |
| <b>Java: entrada e configuração</b>          | <b>20</b> |
| Main . . . . .                               | 20        |
| Configuracao . . . . .                       | 31        |
| <b>Java: domínio e validações</b>            | <b>34</b> |
| CadastroPassageiro . . . . .                 | 34        |
| Dados . . . . .                              | 36        |
| Funcionario . . . . .                        | 41        |
| MapaAssentos . . . . .                       | 42        |
| Passageiro . . . . .                         | 45        |
| PedidoTrecho . . . . .                       | 47        |
| Pessoa . . . . .                             | 48        |
| PrimeiroAcesso . . . . .                     | 52        |
| ProgramacaoVoo . . . . .                     | 53        |
| Sessao . . . . .                             | 55        |
| SituacaoReserva . . . . .                    | 56        |
| SituacaoVoo . . . . .                        | 57        |
| Tarifa . . . . .                             | 59        |

|                                       |           |
|---------------------------------------|-----------|
| TipoCadastro . . . . .                | 60        |
| Validacao . . . . .                   | 65        |
| <b>Java: DTOs de consulta</b>         | <b>69</b> |
| AssentoResumo . . . . .               | 69        |
| ReservaResumo . . . . .               | 70        |
| TipoRelatorio . . . . .               | 72        |
| VooResumo . . . . .                   | 76        |
| <b>Java: contratos de repositório</b> | <b>77</b> |
| ConsultasRepositorio . . . . .        | 77        |
| PassageiroRepositorio . . . . .       | 79        |
| UsuarioRepositorio . . . . .          | 81        |
| <b>Java: infraestrutura JDBC</b>      | <b>83</b> |
| AuditoriaJdbc . . . . .               | 83        |
| AutorizacaoJdbc . . . . .             | 85        |
| BancoDados . . . . .                  | 90        |
| BloqueiosJdbc . . . . .               | 95        |
| CadastroJdbc . . . . .                | 98        |
| ConsultasJdbc . . . . .               | 101       |
| FinanceiroJdbc . . . . .              | 105       |
| InicializacaoJdbc . . . . .           | 108       |
| OperacaoJdbc . . . . .                | 112       |
| OperacaoSql . . . . .                 | 118       |
| PassageiroJdbc . . . . .              | 119       |
| PlanejamentoJdbc . . . . .            | 124       |
| ReservaJdbc . . . . .                 | 131       |
| Sql . . . . .                         | 144       |
| UsuarioJdbc . . . . .                 | 151       |

|                                              |            |
|----------------------------------------------|------------|
| <b>Java: serviços e regras</b>               | <b>154</b> |
| AdministracaoServico . . . . .               | 154        |
| ArquivosServico . . . . .                    | 160        |
| AtendimentoServico . . . . .                 | 166        |
| AutenticacaoServico . . . . .                | 180        |
| CadastroServico . . . . .                    | 183        |
| ConsultaServico . . . . .                    | 191        |
| DadosFicticiosServico . . . . .              | 194        |
| FinanceiroServico . . . . .                  | 200        |
| InicializacaoServico . . . . .               | 209        |
| OperacaoServico . . . . .                    | 211        |
| PassageiroServico . . . . .                  | 223        |
| PlanejamentoServico . . . . .                | 229        |
| RelatoriosCompletosServico . . . . .         | 237        |
| RelatorioServico . . . . .                   | 245        |
| ReservaServico . . . . .                     | 247        |
| VooServico . . . . .                         | 252        |
| <b>Java: terminal e menus</b>                | <b>257</b> |
| Terminal . . . . .                           | 257        |
| MenuPrincipal . . . . .                      | 264        |
| MenuOperacional . . . . .                    | 273        |
| <b>Java: senhas, exceções e exportadores</b> | <b>286</b> |
| PersistenciaException . . . . .              | 286        |
| RegraNegocioException . . . . .              | 288        |
| ExportadorCsv . . . . .                      | 289        |
| ExportadorTxt . . . . .                      | 291        |
| ExportadorVoos . . . . .                     | 293        |

|                                                  |            |
|--------------------------------------------------|------------|
| Senhas . . . . .                                 | 294        |
| <b>Java: testes e preparação</b>                 | <b>298</b> |
| UnidadeTeste . . . . .                           | 298        |
| IntegracaoTeste . . . . .                        | 301        |
| PrepararBancoTeste . . . . .                     | 346        |
| <b>Web: como a interface e a API se conectam</b> | <b>348</b> |
| web/main.jsx . . . . .                           | 349        |
| web/App.jsx . . . . .                            | 349        |
| web/api.js . . . . .                             | 349        |
| web/model.js . . . . .                           | 351        |
| web/Dashboard.jsx . . . . .                      | 351        |
| web/Views.jsx . . . . .                          | 352        |
| web/forms.jsx . . . . .                          | 352        |
| web/components.jsx . . . . .                     | 353        |
| web/styles.css . . . . .                         | 353        |
| server/index.mjs . . . . .                       | 353        |
| server/app.mjs . . . . .                         | 353        |
| server/database.mjs . . . . .                    | 354        |
| server/operations.mjs . . . . .                  | 354        |
| server/validation.mjs . . . . .                  | 355        |
| server/errors.mjs . . . . .                      | 355        |
| server/demo.mjs . . . . .                        | 355        |
| <b>Banco PostgreSQL, arquivos e ferramentas</b>  | <b>356</b> |
| sql/criar_banco.sql . . . . .                    | 356        |
| sql/web.sql . . . . .                            | 358        |
| sql/migracoes/001_cadastros_ativos.sql . . . . . | 359        |
| As palavras SQL usadas pelo código . . . . .     | 359        |

|                                                 |     |
|-------------------------------------------------|-----|
| Arquivos que completam a execução . . . . .     | 360 |
| Como os testes demonstram as conexões . . . . . | 360 |

|                                                     |            |
|-----------------------------------------------------|------------|
| <b>Índice de métodos e conferência de cobertura</b> | <b>361</b> |
|-----------------------------------------------------|------------|

|                                 |     |
|---------------------------------|-----|
| MenuOperacional . . . . .       | 361 |
| MenuPrincipal . . . . .         | 361 |
| Terminal . . . . .              | 362 |
| Configuracao . . . . .          | 362 |
| CadastroPassageiro . . . . .    | 362 |
| Dados . . . . .                 | 363 |
| Funcionario . . . . .           | 363 |
| MapaAssentos . . . . .          | 363 |
| Passageiro . . . . .            | 363 |
| PedidoTrecho . . . . .          | 364 |
| Pessoa . . . . .                | 364 |
| PrimeiroAcesso . . . . .        | 364 |
| ProgramacaoVoo . . . . .        | 364 |
| Sessao . . . . .                | 365 |
| SituacaoReserva . . . . .       | 365 |
| SituacaoVoo . . . . .           | 365 |
| Tarifa . . . . .                | 365 |
| TipoCadastro . . . . .          | 365 |
| Validacao . . . . .             | 365 |
| AssentoResumo . . . . .         | 366 |
| ReservaResumo . . . . .         | 366 |
| TipoRelatorio . . . . .         | 366 |
| VooResumo . . . . .             | 366 |
| PersistenciaException . . . . . | 366 |

|                                 |     |
|---------------------------------|-----|
| RegraNegocioException . . . . . | 367 |
| AuditoriaJdbc . . . . .         | 367 |
| AutorizacaoJdbc . . . . .       | 367 |
| BancoDados . . . . .            | 367 |
| BloqueiosJdbc . . . . .         | 368 |
| CadastroJdbc . . . . .          | 368 |
| ConsultasJdbc . . . . .         | 368 |
| FinanceiroJdbc . . . . .        | 368 |
| InicializacaoJdbc . . . . .     | 369 |
| OperacaoJdbc . . . . .          | 369 |
| OperacaoSql . . . . .           | 369 |
| PassageiroJdbc . . . . .        | 369 |
| PlanejamentoJdbc . . . . .      | 370 |
| ReservaJdbc . . . . .           | 370 |
| Sql . . . . .                   | 370 |
| UsuarioJdbc . . . . .           | 371 |
| Main . . . . .                  | 371 |
| ExportadorCsv . . . . .         | 371 |
| ExportadorTxt . . . . .         | 371 |
| ExportadorVoos . . . . .        | 372 |
| ConsultasRepositorio . . . . .  | 372 |
| PassageiroRepositorio . . . . . | 372 |
| UsuarioRepositorio . . . . .    | 372 |
| Senhas . . . . .                | 373 |
| AdministracaoServico . . . . .  | 373 |
| ArquivosServico . . . . .       | 373 |
| AtendimentoServico . . . . .    | 373 |
| AutenticacaoServico . . . . .   | 374 |

|                                                |            |
|------------------------------------------------|------------|
| CadastroServico . . . . .                      | 374        |
| ConsultaServico . . . . .                      | 374        |
| DadosFicticiosServico . . . . .                | 375        |
| FinanceiroServico . . . . .                    | 375        |
| InicializacaoServico . . . . .                 | 375        |
| OperacaoServico . . . . .                      | 376        |
| PassageiroServico . . . . .                    | 376        |
| PlanejamentoServico . . . . .                  | 376        |
| RelatoriosCompletoServico . . . . .            | 377        |
| RelatorioServico . . . . .                     | 377        |
| ReservaServico . . . . .                       | 377        |
| VooServico . . . . .                           | 378        |
| IntegracaoTeste . . . . .                      | 378        |
| UnidadeTeste . . . . .                         | 379        |
| PrepararBancoTeste . . . . .                   | 379        |
| <b>Roteiro de estudo e referências</b>         | <b>380</b> |
| Documentação oficial para aprofundar . . . . . | 380        |



# Como ler este guia

## O que foi incluído

Este guia acompanha os 67 arquivos Java existentes no projeto: código de produção, os dois programas de teste e o auxiliar de preparação do banco. As 4.314 linhas físicas e os 325 métodos/construtores declarados aparecem na leitura comentada. Os números de linha são os do arquivo original, sem reformatar o programa. As explicações cobrem também a estrutura web, o SQL e os scripts que conectam as aplicações.

## O significado de "linha por linha"

Cada linha original é exibida. Linhas com decisões, chamadas, atribuições e retornos recebem comentários próximos do código. Linhas vazias e chaves de fechamento são explicadas pela convenção de blocos, sem repetir a mesma frase centenas de vezes. Imports usam uma explicação comum. Uma instrução idêntica já explicada pode ter uma referência à primeira ocorrência; uma condição ou argumento diferente continua recebendo análise própria.

## Ordem de estudo

Leia primeiro a sintaxe e o mapa do projeto. Depois acompanhe Main, Terminal e MenuPrincipal para saber quem inicia as operações. Leia o caminho de uma reserva antes de entrar no JDBC. Consulte o capítulo detalhado de cada classe pelo sumário. Os testes devem ser lidos por último: eles usam o mesmo código para demonstrar sucessos, recusas e concorrência.

## Limite da leitura estática

As relações de chamadas foram obtidas das declarações e das chamadas presentes no código. Elas mostram o caminho que pode executar; condições decidem qual caminho realmente executa. Uma chamada por interface pode chegar à implementação JDBC. Lambdas, referências de método e bibliotecas exigem considerar o contexto. Esta documentação não representa uma execução nova dos testes.

## Versão documentada

Retrato dos arquivos em 07/10/2026. A conferência final registra tamanho, quantidade de métodos e SHA-256 de cada arquivo. Se você editar o código depois, os números de linha podem mudar. As descrições tratam das regras implementadas neste trabalho acadêmico, inclusive pagamentos simulados; não afirmam que todas são exigências oficiais de uma companhia aérea.

# Java: palavras, símbolos e tipos

## Classe, objeto e arquivo

class declara um tipo: descreve dados e comportamentos. Pessoa é uma classe; uma instância concreta de Passageiro representa uma pessoa com aquele papel. new Passageiro(...) cria uma instância e chama o construtor. O arquivo .java é texto-fonte; javac produz arquivos .class com bytecode; a JVM executa esse bytecode. Uma classe não começa a executar apenas porque existe no diretório.

## package e import

package br.edu.aeroporto.dominio declara o nome completo e organiza o espaço de nomes. O diretório acompanha esse nome. import permite escrever LocalDate em vez de java.time.LocalDate. import não cria um objeto nem chama um método. O \* importa nomes públicos de um pacote; não percorre todos os subpacotes. O ponto em pacote é organização; o ponto em objeto.metodo() acessa um membro.

## public, private, protected e acesso padrão

public permite acesso onde o tipo estiver acessível. private restringe o membro à classe que o declara, incluindo o acesso permitido entre tipos aninhados da mesma estrutura. protected também permite uso por subclasses, respeitando as regras de acesso, e no próprio pacote. Sem modificador, o acesso é do pacote. Um método private ainda pode chamar métodos public de outras classes; o modificador controla quem pode chamar aquele método.

## O que private long significa

Em private long id;, private controla o acesso e long é o tipo do campo. O id guarda um inteiro de 64 bits. Em private long criar(...), long é o tipo do valor que o método devolve: deve existir um retorno compatível, a menos que a execução termine lançando exceção. Em private final long id;, final impede atribuir outro valor ao campo depois de sua inicialização. private não criptografa os dados.

## Exemplo real: Pessoa.id

Pessoa declara private final long id. O construtor recebe long id, exige que seja positivo e executa this.id = id. this.id é o campo do objeto; id é o parâmetro local. O método public long id() { return id; } permite consultar esse número sem oferecer uma atribuição direta de outro id. Esse método não consulta o PostgreSQL: apenas lê o campo já carregado.

## long e Long; int e Integer

long é primitivo, ocupa 64 bits e não pode ser null; sua faixa vai de -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807. Long é o tipo objeto correspondente e pode representar ausência com null: Terminal.idOpcional usa isso. int é primitivo de 32 bits, adequado a índices e contagens pequenas. Literais 0L e 1L têm tipo long. A conversão automática entre primitivo e objeto chama-se boxing/unboxing; tentar desempacotar Long null gera NullPointerException. Um id de banco pode ser maior que a faixa de int.

## String, char, char[] e boolean

String representa texto e é imutável. "A" é String; 'A' é char. char[] é um vetor mutável, usado para senhas e limpo com Arrays.fill. boolean admite true ou false. Boolean pode ainda ser null; Boolean.TRUE.equals(valor) evita chamar equals em uma referência nula. O programa pode imprimir texto, mas println não o salva no banco.

## BigDecimal e dinheiro

BigDecimal preserva uma representação decimal apropriada às regras de valor e escala. new BigDecimal("0.10") recebe texto decimal e evita a aproximação de um literal double. add, subtract, multiply e setScale devolvem resultados; não alteram o objeto original. compareTo compara o valor numérico; equals também considera a escala. Dados.valor verifica sinal, casas e precisão antes de padronizar a escala.

## static e final

static associa o membro à classe. `Dados.exigir(...)` é chamado sem criar `Dados`. O `Logger` de `Main` é static final: existe uma referência compartilhada que não será trocada. final em classe impede herança; final em método impede sobrescrita; final em variável impede uma nova atribuição. Uma referência final para uma lista não torna o conteúdo da lista imutável. `List.copyOf` e cópias defensivas resolvem outros aspectos desse problema.

## Construtor, this e super

Um construtor tem o nome da classe e não declara tipo de retorno. Ele inicializa a instância. `this` aponta para a instância atual; `this.banco` = banco guarda a dependência recebida. `super(...)` chama o construtor da superclasse: `Passageiro` e `Funcionario` inicializam os campos herdados de `Pessoa` por esse caminho. `new` não garante persistência: inserir no banco é uma operação separada.

## abstract, extends, implements e @Override

abstract permite definir um contrato incompleto e impede instanciar diretamente a classe. `Pessoa` exige `identificacao()`; cada subclasse fornece sua versão. extends expressa herança. implements afirma que uma classe cumpre uma interface, como `PassageiroJdbc` cumpre `PassageiroRepositorio`. @Override pede ao compilador que confira a sobrescrita. Uma chamada a `pessoa.identificacao()` escolhe a implementação do objeto concreto em tempo de execução.

## Interface e implementação

Uma interface declara operações e seus tipos de entrada/saída. `PassageiroRepositorio` descreve o acesso a passageiros; `PassageiroJdbc` implementa com SQL. O serviço pode guardar a dependência como interface. O `Main` escolhe a implementação usando `new PassageiroJdbc()`. Não há framework de injeção neste caminho: o próprio programa conecta os objetos pelos construtores.

## record e seu construtor compacto

record declara um portador de dados e gera campos privados finais, acessores, equals, hashCode e toString. Em `CadastroPassageiro`, `nome()` é um acessor gerado. O construtor compacto pode normalizar os parâmetros antes da atribuição implícita aos campos. Esses acessores gerados não têm linhas de método no fonte e, por isso, não entram na contagem de 325 declarações. Um record é superficialmente imutável: coleções internas precisam de cópia quando necessário.

## enum

enum define um conjunto fechado de constantes. `SituacaoVoo.PROGRAMADO` não é um texto digitado livremente. O enum pode ter campos, construtor e métodos. `TipoCadastro` e `TipoRelatorio` usam constantes com metadados. `name()` devolve o nome literal da constante; `values()` devolve um vetor com as constantes; `valueOf` aceita somente um nome válido.

## Assinatura, parâmetros e argumentos

A assinatura identifica o método pelo nome e tipos de parâmetros; o tipo de retorno faz parte da declaração, mas não permite distinguir sobrecargas sozinho. Parâmetro é a variável declarada entre parênteses; argumento é o valor fornecido na chamada. Java passa os valores por cópia: para objetos, copia a referência. Alterar uma lista recebida pode alterar o mesmo objeto; reatribuir o parâmetro a outra lista não troca a variável do chamador.

## return, void, null e exceção

`return expr`; encerra aquela execução do método ou da lambda e entrega `expr` ao chamador. `return`; encerra um método void. void indica ausência de valor de retorno; não indica ausência de efeito. null é ausência de referência. Nos callbacks genéricos usados em transações sem resultado, `return null` satisfaz `OperacaoSql<T>`; a operação externa pode ser void. `throw` encerra o caminho normal e procura um catch compatível; não é um retorno de sucesso.

## Variáveis locais e var

Uma variável local existe no bloco em que foi declarada. `var` pede ao compilador para inferir o tipo a partir do inicializador; o tipo permanece estático depois disso. `var resultado = new ArrayList<>()` ainda é uma lista com tipo definido. Campos podem ter valores padrão ao nascer o objeto; variáveis locais precisam de atribuição antes do uso. Nomes como `c` e `r` são curtos: no JDBC do projeto, normalmente significam `Connection` e `ResultSet`.

## Blocos, pontuação e comentários

{ abre um bloco e } fecha esse bloco. ; encerra muitas instruções. () delimita parâmetros, argumentos ou expressões de controle; [] representa vetor ou índice; <> delimita parâmetros de tipo. // comenta até o fim da linha; /\* ... \*/ comenta um trecho; /\*\* ... \*/ documenta uma declaração. Comentários não executam. Uma linha física pode conter várias instruções, por isso a leitura comentada pode trazer mais de uma observação para o mesmo número.

## Atribuição e comparação

= atribui. == compara valores primitivos ou identidade de referências. != nega essa igualdade. String e BigDecimal normalmente exigem equals/compareTo para conteúdo. >, >=, < e <= são relações ordenadas. += e ++ atualizam a variável. ! nega uma condição. && exige ambos e para ao encontrar false; || aceita um e para ao encontrar true. Esse curto-circuito evita, por exemplo, acessar campos depois de um teste de null que já encerrou a decisão.

## if, else, switch e operador ternário

if executa seu ramo quando a condição é true; else atende o caso false. ifs encadeados têm ordem significativa. switch escolhe por valor; case identifica um ramo, default trata os demais. Com ->, o ramo não cai automaticamente no seguinte. condição ? A : B escolhe uma expressão e produz um valor; somente a alternativa escolhida é avaliada. Não se deve unificar condições diferentes só porque usam os mesmos operadores.

## for, while, do/while, break e continue

for tradicional controla inicialização, teste e atualização; índices de listas começam em zero. for (var item : lista) percorre elementos. while verifica antes de cada volta. do/while executa ao menos uma vez. break encerra o laço ou switch aplicável; continue pula para a próxima volta. while(true) de Terminal fica ativo até return ou uma exceção de fim de entrada.

## List, Set, Map e vetores

List conserva sequência e admite acesso por índice; ArrayList é a implementação mutável usada no rascunho. Set evita elementos duplicados; HashSet controla assentos e perfis repetidos. Map associa chave a valor; LinkedHashMap preserva ordem de inserção e é usado para linhas SQL. Map<String,Object> perde parte da verificação específica de tipos, por isso Dados faz extrações e validações. Um vetor tem tamanho fixo; uma matriz Java é um vetor de vetores.

## Generics, T, <?> e varargs

<T> declara um parâmetro de tipo e conecta entrada/saída sem fixar uma classe concreta. Sql.listar usa o Mapeador<T> para produzir List<T>. Enum<?> aceita uma constante de enum cujo tipo exato não é conhecido naquele ponto. <T extends Enum<T>> restringe a escolha genérica de MenuOperacional a enums. Object... parametros é varargs: a chamada fornece vários valores e o método recebe um vetor Object[].

## Optional

Optional<T> expressa um resultado presente ou ausente. Sql.unico devolve Optional.empty() para zero linhas, Optional.of(resultado) para uma e lança SQLException para mais de uma. orElse fornece um padrão; orElseThrow lança se não houver valor. Optional não é uma consulta por si: aqui é o resultado de uma consulta já executada. A presença de Optional não transforma erros de banco em ausência.

## Lambda, referência de método e streams

c -> { ... } cria uma função para ser chamada depois pelo receptor; BancoDados entrega a Connection a ela. r -> r.getLong(1) converte uma linha SQL. Sql::linha é uma referência a método compatível com o contrato de mapeamento. stream().filter/map/sorted organiza um processamento; uma operação terminal como toList, count ou collect produz o resultado. sorted antes de locks reduz diferenças na ordem de bloqueio.

## Tempo: LocalDate, Instant, Clock e ZoneId

LocalDate representa data sem hora/fuso. Instant representa um ponto da linha do tempo. ZoneId fornece as regras para apresentar esse ponto como horário local; OffsetDateTime possui deslocamento explícito. Duration compara tempo transcorrido; Period calcula diferenças de calendário. Clock é uma dependência substituível, usada para tornar testes determinísticos. truncatedTo(MICROS) alinha a precisão à persistência PostgreSQL.

## try, catch, finally, throws e recursos

try delimita uma operação que pode falhar; catch trata exceções compatíveis. finally roda quando o bloco termina normalmente ou por exceção, ressaltando interrupções abruptas do processo. throws declara que uma exceção pode escapar. try (Connection c = ...) usa AutoCloseable e fecha recursos automaticamente. addSuppressed preserva uma falha secundária de fechamento/rollback sem apagar a falha principal.

## JDBC e SQL

JDBC é a API Java para falar com o banco por um driver. Connection representa a conexão. PreparedStatement prepara SQL com marcadores ? e setObject liga os argumentos em ordem começando por 1. ResultSet percorre linhas; next avança, getLong(1) lê a primeira coluna. executeQuery retorna linhas; executeUpdate retorna quantidade afetada. O projeto usa JDBC e SQL explícito, sem JPA ou Hibernate.

## Transação, commit, rollback e locks

setAutoCommit(false) agrupa comandos. commit confirma; rollback desfaz alterações daquela transação. BancoDados retorna o resultado somente depois do commit. FOR UPDATE bloqueia linhas para alteração; FOR SHARE protege leituras contra alterações incompatíveis; advisory locks usam chaves escolhidas pela aplicação. Ordenar locks reduz deadlocks, mas não substitui restrições de unicidade e validação do banco.

## Igualdade, hash e toString

equals e hashCode permitem usar objetos em coleções de forma coerente. Pessoa usa seu id; duas instâncias que representam o mesmo id são iguais mesmo com papéis diferentes. toString produz uma representação textual. Configuracao sobrescreve essa representação para não incluir a senha. Arrays.equals compara vetores; deepEquals examina o conteúdo de uma matriz.

## byte, conversão e literais

byte é inteiro de 8 bits com sinal, de -128 a 127. byte[] guarda os bytes de sal/hash. O cast (byte) 0 especifica o tipo usado para limpar esse vetor. (SQLException) erro.getCause() converte a referência quando o contrato garante aquela causa. 600\_000 é o mesmo número que 600000: o underscore facilita leitura. charAt(i) - '0' converte um caractere decimal em seu valor numérico; isso pressupõe dígito válido.

## instanceof e variável de padrão

valor instanceof Number numero testa o tipo e, quando verdadeiro, disponibiliza numero já com esse tipo. Pessoa.equals usa outro instanceof Pessoa pessoa e depois compara ids. O && permite usar a variável somente pelo caminho em que o teste de tipo foi verdadeiro. instanceof não cria outra pessoa; reconhece o tipo do objeto existente.

## Escapes, regex e text blocks

Uma barra invertida em literal Java inicia escape. Para entregar uma barra à expressão regular, normalmente o fonte precisa de duas. Regex [0-9]{11} exige onze dígitos; + exige uma ou mais ocorrências, ? torna opcional, \* admite zero ou mais e classes entre colchetes escolhem caracteres. matches testa o texto inteiro; replaceAll substitui trechos correspondentes. split divide pelo padrão, como o cifrão escapado do hash. Três aspas abrem/fecham um text block de várias linhas.

## Locale, arredondamento e formatação

Locale.ROOT torna conversões de caixa independentes do idioma configurado na máquina. String.format/printf usam marcadores como %s (texto), %d (inteiro) e %n (quebra de linha da plataforma). Math.ceil arredonda para cima na contagem de comissários. RoundingMode define a regra de arredondamento decimal quando solicitado. Posição de lista é base zero; número de coluna JDBC é base um.

## Senha, hash e representação

Senhas gera sal aleatório de 16 bytes e deriva 256 bits usando PBKDF2WithHmacSHA256. O campo ITERACOES define 600.000 rodadas na criação. A String armazenada contém identificador, iterações, sal e hash separados por \$. Base64 representa bytes como texto; não é o algoritmo que protege a senha. Conferir deriva de novo e compara bytes. O código limpa buffers derivados e PBEKeySpec; o chamador limpa o char[] recebido nos serviços.

## Bibliotecas e ferramentas

O JDK contém compilador e ferramentas de desenvolvimento; a JVM executa o bytecode. Um .jar reúne classes/recursos; classpath informa onde procurá-los. O pom.xml oferece uma configuração Maven, mas os scripts PowerShell também compilam diretamente. Node executa JavaScript no servidor, npm instala dependências e Vite constrói a interface React. Java e JavaScript são linguagens diferentes, apesar dos nomes.

# Mapa das camadas e conexões

## Dois caminhos no mesmo projeto

Terminal -> menus Java -> serviços Java -> JDBC -> schema aeroporto. Navegador -> componentes React -> fetch /api -> Express/Node -> driver pg -> schema aerohub. Os dois caminhos usam PostgreSQL, mas têm modelos e regras próprias. A interface web não chama MenuPrincipal nem os serviços Java. Compartilhar o servidor de banco não torna as aplicações uma única cadeia de chamadas.

## Main e cli

Main é o ponto de entrada. cli significa interface de linha de comando: Terminal lê/converte entradas; MenuPrincipal organiza opções; MenuOperacional encaminha módulos. Os menus apresentam mensagens e pedem dados. Regras críticas continuam nos serviços/JDBC, pois uma regra apenas no menu poderia ser contornada por outro chamador.

## config

Configuracao carrega URL, usuário, senha e fuso a partir do arquivo e das variáveis de ambiente. É configuração de infraestrutura, não cadastro de passageiro. O arquivo .example documenta o formato. Credenciais reais não fazem parte deste guia.

## dominio

O domínio representa conceitos e invariantes do problema: Pessoa, Passageiro, Sessao, Tarifa, estados e pedidos de voo/reserva. Validacao e Dados oferecem validações/conversões. Não é uma camada que executa automaticamente antes de todo SQL: só atua onde o código a chama. Algumas regras também estão nos serviços, nos módulos JDBC e nas restrições SQL.

## dto

DTO é um objeto de transferência de dados. VooResumo e ReservaResumo agrupam resultados para mostrar ao operador, sem exigir que o menu conheça o ResultSet. AssentoResumo calcula disponibilidade a partir dos dados recebidos. Neste projeto TipoRelatorio está no pacote dto, mas contém um catálogo de consultas SQL: seu papel é mais amplo que um DTO puramente composto de campos.

## repositorio e infraestrutura/jdbc

As interfaces UsuarioRepositorio, PassageiroRepositorio e ConsultasRepositorio separam contratos da implementação. UsuarioJdbc, PassageiroJdbc e ConsultasJdbc cumprem esses contratos. Os demais módulos JDBC implementam operações diretamente. BancoDados administra conexão/transação; Sql administra statements e mapeamento. Não existe aqui uma interface repositório para cada tabela.

## servico

Os serviços expõem operações do sistema, verificam autorização, coordenam transações e registram auditoria. Um método pode devolver id, DTO, lista ou nada. Algumas consultas e atualizações SQL são feitas diretamente no serviço; outras são delegadas a um módulo JDBC. A organização real mistura esses dois estilos, e a leitura mostra onde cada regra está.

## seguranca, execucao e relatorio

Senhas concentra hash/verificação. RegraNegocioException representa recusas previstas; PersistenciaException traduz falhas JDBC em mensagens. ExportadorVoos é o contrato de exportação; ExportadorTxt e ExportadorCsv são estratégias. RelatorioServico escolhe a estratégia fornecida pelo chamador; relatórios completos usam outras consultas e exportação em lotes.

## Injeção por construtor e sentido das dependências

new PassageiroServico(banco, new PassageiroJdbc(), auditoria, autorizacao, relógio) conecta objetos existentes. O construtor guarda referências com this.campo = parametro. Quando o menu chama o serviço, ele usa essas mesmas referências. Não precisa reinstanciar o banco por operação; BancoDados abre uma Connection para a execução. Dependências recebidas tornam possível trocar implementações e relógios em testes.

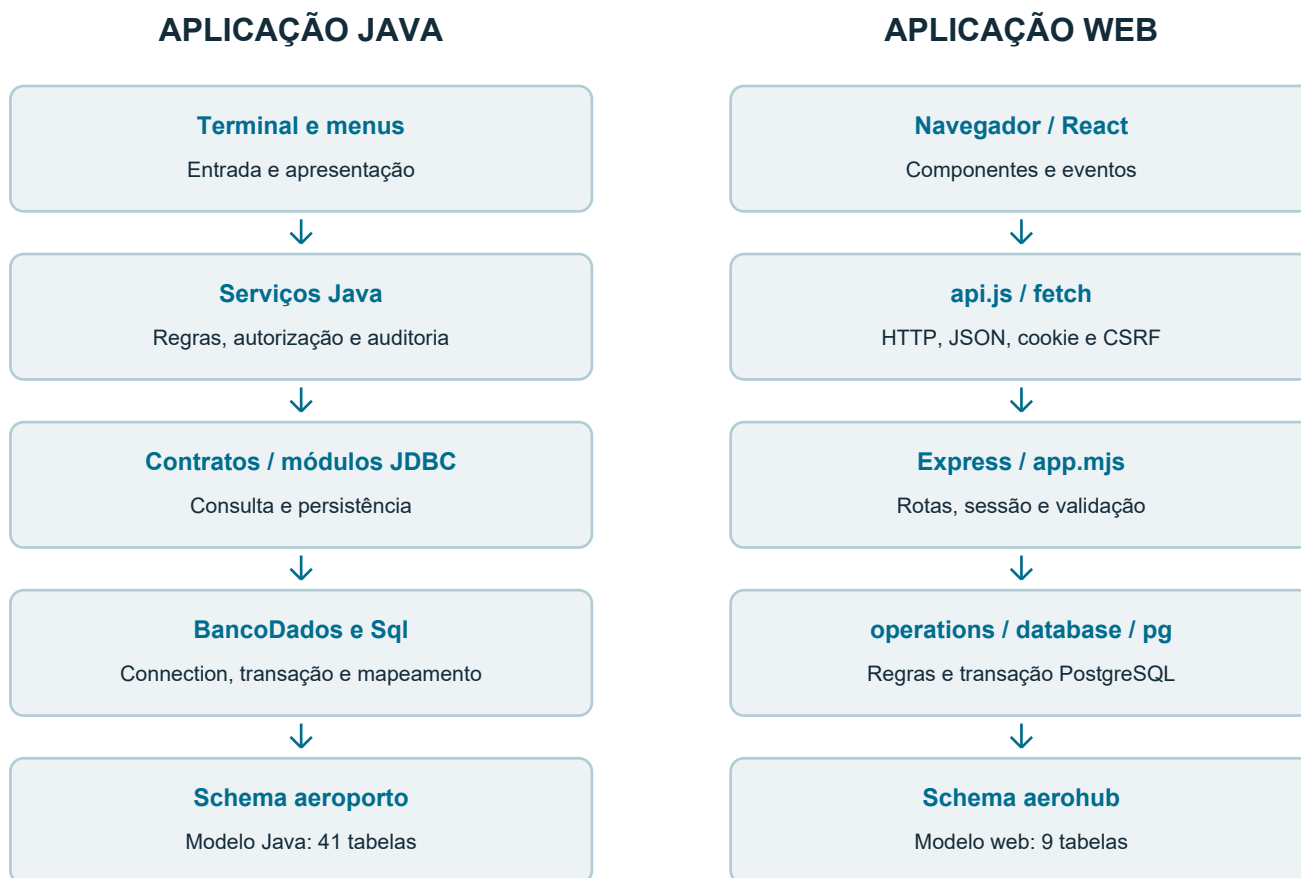
## Classe não é tabela

Pessoa é classe e pessoa é tabela, mas o vínculo não é automático. O SQL seleciona/insere a linha; o mapeador constrói o objeto. VooResumo pode juntar dados de várias tabelas. Sessao é um retrato da autenticação e não tem uma tabela correspondente com todos aqueles campos. Nem toda tabela possui uma classe própria.

## Quem chama quem

Um serviço não chama o menu para devolver dados: return entrega o resultado ao ponto da chamada que estava aguardando. Uma exceção sobe pelos chamadores até um catch. No sucesso, o menu formata o retorno; numa recusa, captura a mensagem e permite outra opção. A leitura de cada método informa chamadas observadas e possíveis chamadores identificados no fonte.

## Mapa visual dos dois caminhos



PostgreSQL é a plataforma comum; os schemas mantêm modelos distintos.

Chamadas descem no desenho; resultados e falhas voltam aos chamadores.



# Operações completas, de ponta a ponta

## Inicialização da aplicação

executar.ps1 prepara o classpath e inicia br.edu.aeroporto.Main. --ajuda sai antes de carregar configuração. No caminho comum, Main configura logs, abre Terminal, carrega Configuracao, instancia dependências e verifica migração. --migrar e --diagnostico executam operações específicas e retornam. --inicializar cria o primeiro acesso quando ainda não há usuário. Sem usuário, o caminho comum apenas orienta a inicialização.

## Login e autorização

Main.entrar lê login e char[] da senha. AutenticacaoServico.entrar normaliza login, busca usuário com UsuarioRepositorio, compara a senha por Senhas e registra o resultado. Uma Sessao carrega usuário, funcionário e perfis. Main abre os menus após sucesso; três recusas devolvem null. A autorização das operações é reconsultada por AutorizacaoJdbc: uma Sessao antiga não concede eternamente uma permissão removida.

## Cadastrar passageiro

MenuPrincipal.cadastrarPassageiro lê nome/CPF/nascimento/contatos e cria CadastroPassageiro. O construtor normaliza campos. PassageiroServico.cadastrar exige permissão, valida nascimento contra Clock e usa BancoDados.transacao. PassageiroJdbc insere pessoa e seu papel passageiro, cria o objeto e AuditoriaJdbc registra a ação. BancoDados confirma e devolve Passageiro ao menu, que imprime nome/id/código. Se qualquer etapa falhar antes de commit, a transação desfaz as inserções.

## Planejar voo

MenuOperacional.programacao reúne companhia, rota, aeronave, número e seis horários. ProgramacaoVoo exige ids, formato e janelas; valida antes e depois do truncamento para microssegundos. PlanejamentoServico verifica autorização e usa transação repetível. PlanejamentoJdbc confere frota/continuidade/margem, insere voo, agenda e inventário. O long retornado é o id do voo persistido, não uma quantidade de assentos.

## Rascunho e reserva

O rascunho de MenuOperacional é uma ArrayList<PedidoTrecho>; adicionar/editar/remover nessa tela modifica memória. Gravar chama ReservaServico.reservar. O serviço valida o grupo e prazo, e ReservaJdbc bloqueia voos, verifica percurso por passageiro, tarifas, descontos e assentos, bloqueia inventário e insere reserva/itens/ocupações. O id retorna depois do commit. Se o segundo trecho falhar, o primeiro não fica vendido parcialmente.

## Prazos, pagamento e confirmação

Uma reserva Java inicialmente pendente precisa satisfazer prazo e cobertura financeira. FinanceiroServico registra pagamentos simulados com chave de idempotência; FinanceiroJdbc calcula devido/recebido e confirma quando as condições forem atendidas. Confirmar emite bilhetes e recalcula o cabeçalho. Solicitar reembolso já compromete parte do recebido disponível. Expirar libera ocupação e preserva histórico; não apaga a reserva para fingir que nunca existiu.

## Check-in, bagagem e embarque

AtendimentoServico trabalha sobre o item da reserva. Check-in exige condições do item/voo e janela ajustada pelo atraso; valida também documentos quando aplicável. O cartão contém código e assento. Despacho identifica cada bagagem e calcula excesso incremental pelas regras implementadas. Rastreo segue transições permitidas. Embarque usa código do cartão, confere estado/janela/portão e marca atendimento. O uso do item e o avanço do voo preservam registros para consulta.

## Cancelamento e remarcação

ReservaServico coordena locks do voo e reserva antes de modificar o item. ReservaJdbc distingue cancelamento voluntário das condições de cancelamento da companhia; calcula direitos/multas e invalida bilhete/check-in/ocupação. Remarcação cria a nova situação de trecho dentro da operação coordenada e mantém a referência histórica. Não é equivalente a editar apenas o número do voo numa tela.

## Consultas e retorno de DTO

MenuPrincipal.consultarVoos fornece data, página, tamanho e fuso. ConsultaServico exige acesso e usa BancoDados.consultar. ConsultasJdbc executa JOINS e mapeia cada ResultSet em VooResumo. Sql.listar devolve uma lista copiada. O menu converte Instant para fusos dos aeroportos e imprime. O DTO não abre conexão para calcular atrasado: compara os campos que já recebeu.

## Exportação e importação

No relatório simples, o menu escolhe ExportadorTxt/Csv e RelatorioServico grava a página consultada. A exportação completa dos relatórios gerais percorre lotes de 500. CSV precisa tratar delimitadores/aspas e células que poderiam ser fórmulas. ArquivosServico importa TAB com cabeçalho e limites definidos, valida os registros e só então grava o lote transacionalmente. Sua cópia com SHA-256 é uma cópia de arquivo, não um backup completo do PostgreSQL.

## Falha, rollback e retorno

Considere banco.transacao(c -> { inserir pessoa; inserir papel; auditar; return passageiro; }). O callback termina e fornece o objeto a BancoDados. BancoDados ainda precisa executar commit antes de devolvê-lo ao serviço. Se houver SQLException ou RuntimeException, executa rollback e relança; a SQLException externa vira PersistenciaException. O menu trata a falha. return dentro de uma lambda retorna da lambda, não de Main.

## Concorrência e idempotência

Duas pessoas podem tentar o mesmo assento ao mesmo tempo. Verificar apenas em memória não resolve: locks, revalidação e restrição única no banco atuam juntos. transacaoRepetivel tenta novamente a operação completa apenas para SQLSTATE 40P01/40001, até três tentativas totais. Chaves de idempotência evitam duplicar determinados pagamentos/reembolsos repetidos. Não significa que toda chamada de serviço seja automaticamente idempotente.

# Leitura comentada: convenções

## Código e comentário

O bloco claro contém a linha original e o número à esquerda. Linhas longas continuam visualmente embaixo, sem ganhar outro número: ainda são a mesma linha do arquivo. O comentário logo abaixo explica a instrução. Uma única linha pode conter várias ações; cada decisão relevante é conservada.

## Entradas e saídas

Cada método começa com uma ficha de propósito, parâmetros, tipo de retorno e conexões. O código mantém a sequência real. O comentário do retorno diferencia o valor devolvido dos efeitos já executados. Construtores e métodos gerados de records são descritos pelas convenções de Java.

## Repetições

Imports, comentários, linhas vazias e chaves obedecem às explicações de sintaxe. Instruções exatamente iguais podem apontar para a primeira ocorrência. Uma regra com condição, mensagem ou parâmetro diferente não é deduplicada. A presença da mesma chamada em dois ramos não torna os ramos equivalentes.

## Chamadores identificados

A lista de chamadores é uma ajuda de navegação baseada no fonte, não uma prova de que todos executam em toda sessão. Interfaces, métodos gerados, callbacks, cadeias complexas e funções de bibliotecas podem não resolver para uma classe do projeto. Os nomes de chamadas observadas são preservados para você acompanhar esses casos.

## Sem código oculto

Todos os arquivos Java estão no índice de cobertura ao final. Não foram descartados testes, utilitários ou construtores pequenos. Acessores gerados pelo compilador não possuem linha própria no fonte; seu comportamento foi explicado no capítulo de records.

# Java: entrada e configuração

## Main

src/main/java/br/edu/aeroporto/Main.java

Ponto de entrada da aplicação Java. Monta os objetos, carrega a configuração, escolhe ajuda/migração/diagnóstico/primeiro acesso, autentica e entrega o controle aos menus.

176 linhas | 5 métodos/construtores declarados | SHA-256: 535b2289cbf1647c...

```
1 package br.edu.aeroporto;
    Declara o pacote deste arquivo: br.edu.aeroporto. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.cli.MenuPrincipal;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Apresenta as opções 0 a 18, coordena consultas básicas e encaminha os módulos completos ao MenuOperacional. Captura erros para manter a sessão no menu.

4 import br.edu.aeroporto.cli.Terminal;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Converte a entrada de texto em valores Java e repete a pergunta enquanto a entrada não for válida. Também detecta fim da entrada e fecha o Scanner.

5 import br.edu.aeroporto.cli.MenuOperacional;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Lê os parâmetros de cada operação e chama o serviço correspondente. Contém o rascunho de reservas em ArrayList e os menus financeiros, operacionais e de arquivos.

6 import br.edu.aeroporto.config.Configuracao;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Record com URL JDBC, usuário, senha e fuso. Carrega o arquivo local e aplica substituições por variáveis de ambiente; evita mostrar a senha em toString.

7 import br.edu.aeroporto.dominio.PrimeiroAcesso;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Dados necessários para criar o administrador e o aeroporto-base. Valida nomes, login e o formato do ICAO.

8 import br.edu.aeroporto.dominio.Sessao;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Identifica o usuário autenticado, seu funcionário e os perfis recebidos no login. Copia os perfis; autorização é reconsultada no banco em cada operação.

9 import br.edu.aeroporto.execcao.PersistenciaException;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Traduz SQLSTATE do JDBC em mensagens de aplicação e preserva a SQLException como causa para diagnóstico e repetição controlada.

10 import br.edu.aeroporto.execcao.RegraNegocioException;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Exceção não verificada para uma operação recusada pelas regras. A mensagem pode ser apresentada ao usuário.

11 import br.edu.aeroporto.infraestrutura.jdbc.AuditoriaJdbc;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Insere eventos e define variáveis locais de transação com autor/motivo usados pelo histórico de voo no PostgreSQL.

12 import br.edu.aeroporto.infraestrutura.jdbc.AutorizacaoJdbc;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Confere novamente usuário, pessoa, contrato e perfis atuais no banco. Assim uma Sessao antiga não conserva acesso depois do bloqueio do usuário.

13 import br.edu.aeroporto.infraestrutura.jdbc.BancoDados;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Abre e fecha a conexão JDBC; executa consultas/transações com commit ou rollback; pode repetir a transação inteira em deadlock ou falha de serialização.

14 import br.edu.aeroporto.infraestrutura.jdbc.ConsultasJdbc;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Implementa ConsultasRepositorio. Converte linhas SQL nos DTOs tipados para que os menus não precisem entender ResultSet.
```

```
15 import br.edu.aeroporto.infraestrutura.jdbc.InicializacaoJdbc;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Cria Brasil/localidades/cargo/pessoa/funcionário/usuário/perfis em primeiro acesso e impede dois administradores iniciais concorrentes.

```
16 import br.edu.aeroporto.infraestrutura.jdbc.PassageiroJdbc;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Implementa PassageiroRepositorio. Faz as escritas em pessoa, passageiro, documentos e contatos usando a conexão fornecida pelo serviço.

```
17 import br.edu.aeroporto.infraestrutura.jdbc.UsuarioJdbc;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Implementa UsuarioRepositorio usando PostgreSQL. Filtra usuário/pessoa ativos e vigência do contrato; reúne funcionário, perfis e hash.

```
18 import br.edu.aeroporto.servico.AutenticacaoServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Valida login/senha, obtém a Sessao e registra acesso; limpa o char[] da senha no finally.

```
19 import br.edu.aeroporto.servico.ConsultaServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Valida filtros, limites e permissões das consultas básicas e converte um dia/fuso em intervalo de instantes.

```
20 import br.edu.aeroporto.servico.InicializacaoServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Valida senha e nascimento, gera hash e chama a criação transacional do primeiro acesso com auditoria.

```
21 import br.edu.aeroporto.servico.PassageiroServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Aplica validações, permissões e auditoria antes de usar o contrato de persistência dos passageiros.

```
22 import br.edu.aeroporto.servico.RelatorioServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Exporta apenas a lista de voos fornecida pelo menu, usando uma implementação de ExportadorVoos e arquivo temporário.

```
23 import br.edu.aeroporto.servico.CadastroServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Valida o cadastro conforme seu tipo e protege operações/históricos durante edição ou inativação.

```
24 import br.edu.aeroporto.servico.AdministracaoServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Administra contas, perfis, bloqueios e alteração de senha; preserva pelo menos um administrador ativo.

```
25 import br.edu.aeroporto.servico.PlanejamentoServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Coordena programação, tarifas, bloqueios, reprogramação, horários e troca de aeronave com auditoria e atualização das agendas.

```
26 import br.edu.aeroporto.servico.ReservaServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Expira pendências e coordena reservas, cancelamentos e remarcações completas em transações repetíveis.

```
27 import br.edu.aeroporto.servico.FinanceiroServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Orquestra pagamentos simulados idempotentes e reembolsos, verificando saldo, situação e condições vendidas.

```
28 import br.edu.aeroporto.servico.AtendimentoServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Conecta item confirmado, documento, bilhete, assento, check-in, bagagem e embarque; aplica janelas e cobertura financeira.

```
29 import br.edu.aeroporto.servico.OperacaoServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Administra recursos, interdições, habilitações, escalas, manutenção, serviços de solo e ocorrências.

```
30 import br.edu.aeroporto.servico.VooServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Executa transições do voo e seus efeitos em itens, bilhetes, agendas, equipe, serviços de solo e reembolsos.

```
31 import br.edu.aeroporto.servico.RelatoriosCompletosServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Consulta relatórios paginados, painéis, mapa, rastreo e estatísticas; exporta todos os registros do relatório por streaming.

```
32 import br.edu.aeroporto.servico.ArquivosServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Importa passageiros de TXT tabulado e realiza cópia verificada de relatórios. A cópia de arquivo não equivale a backup completo PostgreSQL.

```
33 import br.edu.aeroporto.servico.DadosFicticiosServico;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Cria um cenário fictício completo e retorna ids úteis para demonstração. Repetir a mesma chave devolve os ids existentes.

```
34 import br.edu.aeroporto.infraestrutura.jdbc.CadastroJdbc;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. CRUD genérico limitado por TipoCadastro. Monta nomes SQL a partir do catálogo fechado e vincula os valores como parâmetros.

```
35 import br.edu.aeroporto.infraestrutura.jdbc.PlanejamentoJdbc;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Persiste programação, inventário, tarifa e agendas. Verifica continuidade física da aeronave entre os aeroportos e a margem de 75 minutos.

```
36 import br.edu.aeroporto.infraestrutura.jdbc.ReservaJdbc;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Escolhe assentos e persiste itens/ocupações, preserva os preços vendidos, recalcula situação, invalida documentos e administra expiração e devoluções.

```
37 import br.edu.aeroporto.infraestrutura.jdbc.Sql;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Centraliza PreparedStatement, parâmetros, ResultSet, listas, Optional e extração de identificadores. Não abre uma segunda conexão nem decide sozinho as regras de negócio.

```
38 import java.io.IOException;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
39 import java.nio.file.Files;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
40 import java.nio.file.Path;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. caminho de arquivo, sem garantir por si a existência desse arquivo

```
41 import java.sql.SQLException;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
42 import java.time.Clock;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
43 import java.time.DateTimeException;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
44 import java.util.Arrays;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
45 import java.util.logging.FileHandler;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
46 import java.util.logging.Level;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
47 import java.util.logging.Logger;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
48 import java.util.logging.SimpleFormatter;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
49
```

```
50 public final class Main {
```

Declara Main como tipo com campos e métodos. final impede herança.

```
51     private static final Logger LOG = Logger.getLogger(Main.class.getName());
```

Campo(s) LOG: private restringe acesso à classe. Tipo Logger: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

```
52
```

## Main - L53 a L53

Construtor vazio. Se for private, impede criar instâncias desta classe utilitária a partir de outras classes; os métodos static são usados pelo nome da classe.

Entradas declaradas: ().

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
53     private Main() {}
```

Início da declaração de Main. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
54
```

## main - L55 a L141

O primeiro if atende --ajuda antes de exigir banco. Depois valida argumentos, cria Terminal/configuração/banco, oferece migração e diagnóstico, exige a migração aditiva, inicializa ou autentica, expira reservas e monta os menus. Cada catch transforma uma categoria de erro em mensagem e, na inicialização, saída de processo.

Entradas declaradas: (String[] args).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Arrays.asList(args).contains, Arrays.asList, System.out.println, java.util.Set.of("--inicializar","--migrar","--diagnostico").contains, java.util.Set.of, System.err.println, System.exit, configurarLog, new Terminal, Configuracao.carregar, new BancoDados, new AuditoriaJdbc, new AutorizacaoJdbc, Clock.system, configuracao.fuso, args[0].equals, Files.readString, Path.of, banco.transacao, Sql.unico(c,"SELECT COUNT(\*) FROM pessoa",r->r.getLong(1)).orElse, Sql.unico, r.getLong, Sql.unico(c,"SELECT COUNT(\*) FROM reserva",r->r.getLong(1)).orElse, c.createStatement, comando.execute, banco.consultar, Sql.unico(c,"SELECT COUNT(\*) FROM information\_schema.tables WHERE table\_schema='aeroporto' AND table\_type='BASE TABLE'",r->r.getLong(1)).orElse, Sql.unico(c,"SELECT COUNT(\*) FROM usuario\_sistema",r->r.getLong(1)).orElse, Sql.unico(c,"SELECT EXISTS(SELECT 1 FROM information\_schema.columns WHERE table\_schema='aeroporto' AND table\_name='passageiro' AND column\_name='ativo'",r->r.getBoolean(1)).orElse, r.getBoolean, new AutenticacaoServico, new UsuarioJdbc, new RegraNegocioException, autenticacao.existeUsuario, inicializar, new InicializacaoServico, new InicializacaoJdbc, entrar, sessao.funcionario().nome, sessao.funcionario, new PassageiroServico, new PassageiroJdbc, new ConsultaServico, new ConsultasJdbc, new ReservaServico, new ReservaJdbc, reservas.expirarPendencias, new MenuOperacional, new CadastroServico, new CadastroJdbc, new AdministracaoServico, new PlanejamentoServico, new PlanejamentoJdbc, new FinanceiroServico, new AtendimentoServico, new OperacaoServico, new VooServico, new RelatoriosCompletoServico, new ArquivosServico, new DadosFicticiosServico, new MenuPrincipal, new MenuPrincipal, sessao, passageiros, consultas, new RelatorioServico(), configuracao.fuso(),operacional).executar, new MenuPrincipal, new RelatorioServico, erro.getMessage, erro.getCause, LOG.log, sql.getSQLState, erro.getClass().getSimpleName, erro.getClass.

```
55     public static void main(String[] args) {
```

Início da declaração de main. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
56         if (Arrays.asList(args).contains("--ajuda")) {
```

Decisão específica: (Arrays.asList(args).contains("--ajuda")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
57             System.out.println("""
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
58                 Sistema de aeroporto – Java 21 / PostgreSQL
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```
59                 .\\executar.ps1 -Inicializar Cria o primeiro administrador e seu aeroporto-base no Brasil.
```

Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o argumento/resultado daquela instrução.

```
60                 .\\executar.ps1 Abre o login e os menus.
```

Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o argumento/resultado daquela instrução.

```
61                 Configuração: config/application.properties (copie o .example e preencha).
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```
62                 O banco deve ter a estrutura criada por sql/criar_banco.sql.
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.



|    |                                                                                                                                                                                                                     |                                                   |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| 63 | <code>.\executar.ps1 -Migrar</code>                                                                                                                                                                                 | Aplica a migracao aditiva sem recriar tabelas.    |
|    | Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o argumento/resultado daquela instrução.                                                                    |                                                   |
| 64 | <code>.\executar.ps1 -Diagnostics</code>                                                                                                                                                                            | Confere conexao, schema e existencia de usuarios. |
|    | Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o argumento/resultado daquela instrução.                                                                    |                                                   |
| 65 | <code>Cadastros, voos, reservas, pagamentos simulados, check-in, bagagens, embarque e operacao.</code>                                                                                                              |                                                   |
|    | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                              |                                                   |
| 66 | <code>""");</code>                                                                                                                                                                                                  |                                                   |
|    | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                                      |                                                   |
| 67 | <code>return;</code>                                                                                                                                                                                                |                                                   |
|    | return encerra o caminho atual sem entregar valor (void).                                                                                                                                                           |                                                   |
| 68 | <code>}</code>                                                                                                                                                                                                      |                                                   |
| 69 | <code>if (args.length &gt; 1    args.length == 1 &amp;&amp; !java.util.Set.of("--inicializar", "--migrar", "--diagnostico").contains(args[0])) {</code>                                                             |                                                   |
|    | Decisão específica: (args.length > 1    args.length == 1 && !java.util.Set.of("--inicializar", "--migrar", "--diagnostico").contains(args[0])). Se for true, executa o ramo seguinte; se for false, pula esse ramo. |                                                   |
| 70 | <code>System.err.println("Argumento desconhecido. Use --ajuda.");</code>                                                                                                                                            |                                                   |
|    | Chama System.err.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.                                                                                                          |                                                   |
| 71 | <code>System.exit(1);</code>                                                                                                                                                                                        |                                                   |
|    | Chama System.exit: Encerra o processo com o código fornecido; não apenas o método atual.                                                                                                                            |                                                   |
| 72 | <code>}</code>                                                                                                                                                                                                      |                                                   |
| 73 | <code>configurarLog();</code>                                                                                                                                                                                       |                                                   |
|    | Chama Main.configurarLog: Cria logs, limita cada arquivo a 1.000.000 bytes e mantém até três arquivos com append. Usa UTF-8/SimpleFormatter. Se o arquivo estiver indisponível, avisa e continua.                   |                                                   |
| 74 | <code>try (Terminal terminal = new Terminal()) {</code>                                                                                                                                                             |                                                   |
|    | Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.                                                                                      |                                                   |
|    | new constrói Terminal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                       |                                                   |
| 75 | <code>Configuracao configuracao = Configuracao.carregar();</code>                                                                                                                                                   |                                                   |
|    | Declara variável local configuracao do tipo Configuracao. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                      |                                                   |
| 76 | <code>BancoDados banco = new BancoDados(configuracao);</code>                                                                                                                                                       |                                                   |
|    | Declara variável local banco do tipo BancoDados. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                               |                                                   |
|    | new constrói BancoDados: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                     |                                                   |
| 77 | <code>AuditoriaJdbc auditoria = new AuditoriaJdbc();</code>                                                                                                                                                         |                                                   |
|    | Declara variável local auditoria do tipo AuditoriaJdbc. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                        |                                                   |
|    | new constrói AuditoriaJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                  |                                                   |
| 78 | <code>AutorizacaoJdbc autorizacao = new AutorizacaoJdbc();</code>                                                                                                                                                   |                                                   |
|    | Declara variável local autorizacao do tipo AutorizacaoJdbc. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                    |                                                   |
|    | new constrói AutorizacaoJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                |                                                   |
| 79 | <code>Clock relógio = Clock.system(configuracao.fuso());</code>                                                                                                                                                     |                                                   |
|    | Declara variável local relógio do tipo Clock. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                  |                                                   |
| 80 | <code>if(args.length==1 &amp;&amp; args[0].equals("--migrar")) {</code>                                                                                                                                             |                                                   |
|    | Decisão específica: (args.length==1 && args[0].equals("--migrar")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.                                                                             |                                                   |



|    |                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 81 | <code>String migracao=Files.readString(Path.of("sql/migracoes/001_cadastros_ativos.sql"));</code>                                                                                                                                                                 | Declara variável local migracao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                                                                                                                        |
| 82 | <code>banco.transacao(c-&gt;{</code>                                                                                                                                                                                                                              | Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 83 | <code>long pessoas=Sql.unico(c,"SELECT COUNT(*) FROM pessoa",r-&gt;r.getLong(1)).orElse(0L);</code>                                                                                                                                                               | Declara variável local pessoas do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: pessoa. COUNT calcula uma contagem no banco, não carrega cada objeto para contar em Java.                                                                                                                                                                          |
| 84 | <code>long reservasExistentes=Sql.unico(c,"SELECT COUNT(*) FROM reserva",r-&gt;r.getLong(1)).orElse(0L);</code>                                                                                                                                                   | Declara variável local reservasExistentes do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: reserva. COUNT calcula uma contagem no banco, não carrega cada objeto para contar em Java.                                                                                                                                                              |
| 85 | <code>try(var comando=c.createStatement()) { comando.execute(migracao); }</code>                                                                                                                                                                                  | Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.                                                                                                                                                                                                                                                                                                                                                                                                              |
| 86 | <code>System.out.println("Migracao aditiva aplicada. Pessoas preservadas: "+pessoas+"; reservas: "+reservasExistentes);</code>                                                                                                                                    | Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 87 | <code>return null;</code>                                                                                                                                                                                                                                         | return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.                                                                                                                                                                                                                                                                                                                                   |
| 88 | <code>});return;</code>                                                                                                                                                                                                                                           | return encerra o caminho atual sem entregar valor (void).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 89 | <code>}</code>                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 90 | <code>if(args.length==1 &amp;&amp; args[0].equals("--diagnostico")) {</code>                                                                                                                                                                                      | Decisão específica: (args.length==1 && args[0].equals("--diagnostico")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.                                                                                                                                                                                                                                                                                                                                                                                                |
| 91 | <code>banco.consultar(c-&gt;{</code>                                                                                                                                                                                                                              | Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.                                                                                                                                                                                                                                                                                                                                                                                                           |
| 92 | <code>System.out.println("Conexao PostgreSQL estabelecida.");</code>                                                                                                                                                                                              | Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 93 | <code>System.out.println("Tabelas: "+Sql.unico(c,"SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='aeroporto' AND table_type='BASE TABLE'",r-&gt;r.getLong(1)).orElse(0L));</code>                                                              | Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.<br><br>Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: information_schema. COUNT calcula uma contagem no banco, não carrega cada objeto para contar em Java.                                                                                                                                                                                     |
| 94 | <code>System.out.println("Usuarios existentes: "+Sql.unico(c,"SELECT COUNT(*) FROM usuario_sistema",r-&gt;r.getLong(1)).orElse(0L));</code>                                                                                                                       | Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.<br><br>Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: usuario_sistema. COUNT calcula uma contagem no banco, não carrega cada objeto para contar em Java.                                                                                                                                                                                        |
| 95 | <code>System.out.println("Migracao aplicada: "+Sql.unico(c,"SELECT EXISTS(SELECT 1 FROM information_schema.columns WHERE table_schema='aeroporto' AND table_name='passageiro' AND column_name='ativo')",r-&gt;r.getBoolean(1)).orElse(false));return null;</code> | Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.<br><br>Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: information_schema. EXISTS verifica se alguma linha satisfaz os filtros.<br><br>return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo. |
| 96 | <code>});return;</code>                                                                                                                                                                                                                                           | return encerra o caminho atual sem entregar valor (void).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 97 | <code>}</code>                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

```
98         var autenticacao = new AutenticacaoServico(banco, new UsuarioJdbc(), auditoria);
```

Declara variável local autenticacao com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói AutenticacaoServico, UsuarioJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
99         boolean migracaoAplicada=banco.consultar(c->Sql.unico(c,"SELECT EXISTS(SELECT 1 FROM
information_schema.columns WHERE table_schema='aeroporto' AND table_name='passageiro' AND
column_name='ativo')",r->r.getBoolean(1)).orElse(false));
```

Declara variável local migracaoAplicada do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: information\_schema. EXISTS verifica se alguma linha satisfaz os filtros.

```
100         if(!migracaoAplicada) throw new RegraNegocioException("Aplique a migracao aditiva com
.\executar.ps1 -Migrar antes de entrar.");
```

Decisão específica: (!migracaoAplicada). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException("Aplique a migracao aditiva com .\executar.ps1 -Migrar antes de entrar."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
101         if (args.length == 1) {
```

Decisão específica: (args.length == 1). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
102         if (autenticacao.existeUsuario()) throw new RegraNegocioException("O primeiro acesso já
existe. Execute sem -Inicializar.");
```

Decisão específica: (autenticacao.existeUsuario()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException("O primeiro acesso já existe. Execute sem -Inicializar."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
103         inicializar(terminal, new InicializacaoServico(banco, new InicializacaoJdbc(), auditoria,
relogio));
```

Chama Main.inicializar: Lê os dados do primeiro administrador e do aeroporto-base, cria PrimeiroAcesso e chama InicializacaoServico.criar. O método não retorna id; o sucesso é exibido no terminal.

new constrói InicializacaoServico, InicializacaoJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
104         return;
```

return encerra o caminho atual sem entregar valor (void).

```
105     }
106     if (!autenticacao.existeUsuario()) {
```

Decisão específica: (!autenticacao.existeUsuario()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
107         System.out.println("Nenhum operador cadastrado. Execute .\executar.ps1 -Inicializar para
criar o primeiro acesso.");
```

Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```
108         return;
```

return encerra o caminho atual sem entregar valor (void).

```
109     }
110     Sessao sessao = entrar(terminal, autenticacao);
```

Declara variável local sessao do tipo Sessao. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
111     if (sessao == null) return;
```

Decisão específica: (sessao == null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

return encerra o caminho atual sem entregar valor (void).

```
112     System.out.println("Operador: " + sessao.funcionario().nome());
```

Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

|     |                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 113 | <pre>var passageiros = new PassageiroServico(banco, new PassageiroJdbc(), auditoria, autorizacao, relógio);</pre>                           | Declara variável local passageiros com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói PassageiroServico, PassageiroJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                |
| 114 | <pre>var consultas = new ConsultaServico(banco, new ConsultasJdbc(), autorizacao);</pre>                                                    | Declara variável local consultas com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói ConsultaServico, ConsultasJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                     |
| 115 | <pre>var reservas=new ReservaServico(banco,new ReservaJdbc(),autorizacao,auditoria,relógio);</pre>                                          | Declara variável local reservas com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói ReservaServico, ReservaJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                         |
| 116 | <pre>reservas.expirarPendencias();</pre>                                                                                                    | Chama ReservaServico.expirarPendencias: Rotina interna chamada ao iniciar e antes de vender; usa relógio e transação repetível, retornando quantidade expirada.                                                                                                                                                                                                                                                                            |
| 117 | <pre>var operacional=new MenuOperacional(terminal,sessao,passageiros,</pre>                                                                 | Declara variável local operacional com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói MenuOperacional: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                  |
| 118 | <pre>new CadastroServico(banco,new CadastroJdbc(),autorizacao,auditoria,relógio),</pre>                                                     | new constrói CadastroServico, CadastroJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                         |
| 119 | <pre>new AdministracaoServico(banco,autorizacao,auditoria),</pre>                                                                           | new constrói AdministracaoServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                                  |
| 120 | <pre>new PlanejamentoServico(banco,new PlanejamentoJdbc(),autorizacao,auditoria,relógio),reservas,</pre>                                    | new constrói PlanejamentoServico, PlanejamentoJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                 |
| 121 | <pre>new FinanceiroServico(banco,autorizacao,auditoria,relógio),</pre>                                                                      | new constrói FinanceiroServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                                     |
| 122 | <pre>new AtendimentoServico(banco,autorizacao,auditoria,relógio),</pre>                                                                     | new constrói AtendimentoServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                                    |
| 123 | <pre>new OperacaoServico(banco,autorizacao,auditoria,relógio),new VooServico(banco,autorizacao,auditoria,relógio),</pre>                    | new constrói OperacaoServico, VooServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                           |
| 124 | <pre>new RelatoriosCompletosServico(banco,autorizacao,auditoria),new ArquivosServico(banco,autorizacao,auditoria,relógio),</pre>            | new constrói RelatoriosCompletosServico, ArquivosServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                           |
| 125 | <pre>new DadosFicticiosServico(banco,autorizacao,auditoria,relógio));</pre>                                                                 | new constrói DadosFicticiosServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                                 |
| 126 | <pre>new MenuPrincipal(terminal, sessao, passageiros, consultas, new RelatorioServico(), configuracao.fuso(),operacional).executar();</pre> | Invoca new MenuPrincipal(terminal, sessao, passageiros, consultas, new RelatorioServico(), configuracao.fuso(),operacional).executar com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.<br><br>new constrói MenuPrincipal, RelatorioServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só. |

```
127         } catch (Terminal.FimDaEntrada fim) {
```

Trata a(s) exceção(ões) declarada(s) em catch (Terminal.FimDaEntrada fim). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
128         System.out.println("Entrada encerrada.");
```

Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```
129         } catch (RegraNegocioException | IllegalArgumentException | DateTimeException erro) {
```

Trata a(s) exceção(ões) declarada(s) em catch (RegraNegocioException | IllegalArgumentException | DateTimeException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
130         System.err.println(erro.getMessage());
```

Chama System.err.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```
131         System.exit(1);
```

Instrução idêntica à de Main:L71. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
132         } catch (PersistenciaException erro) {
```

Trata a(s) exceção(ões) declarada(s) em catch (PersistenciaException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
133         System.err.println(erro.getMessage());
```

Instrução idêntica à de Main:L130. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
134         if (erro.getCause() instanceof SQLException sql) LOG.log(Level.SEVERE, "Falha na
inicialização. SQLSTATE={0}", sql.getSQLState());
```

Decisão específica: (erro.getCause() instanceof SQLException sql). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
135         System.exit(1);
```

Instrução idêntica à de Main:L71. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
136         } catch (IOException erro) {
```

Trata a(s) exceção(ões) declarada(s) em catch (IOException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
137         System.err.println("Não foi possível ler a configuração. Confira o arquivo e as permissões.");
```

Chama System.err.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```
138         LOG.log(Level.SEVERE, "Falha de configuração: {0}", erro.getClass().getSimpleName());
```

Invoca LOG.log com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
139         System.exit(1);
```

Instrução idêntica à de Main:L71. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
140     }
```

```
141 }
```

```
142
```

## entrar - L143 a L154

Lê login e senha até três vezes. Devolve a Sessao no primeiro sucesso; após três recusas devolve null e Main não abre os menus.

Entradas declaradas: (Terminal terminal, AutenticacaoServico autenticacao).

Saída declarada: Sessao - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: Main.main (L55).

Chamadas observadas no corpo: terminal.ler, autenticacao.entrar, terminal.senha, System.out.println, erro.getMessage.

```
143     private static Sessao entrar(Terminal terminal, AutenticacaoServico autenticacao) {
```

Início da declaração de entrar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
144         for (int tentativa = 0; tentativa < 3; tentativa++) {
```

Laço controlado por inicialização, condição e atualização: for (int tentativa = 0; tentativa < 3; tentativa++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local tentativa do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```

145         String login = terminal.ler("Login");
Declara variável local login do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

146         try {
Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

147             return autenticacao.entrar(login, terminal.senha("Senha"));
Retorno: entrega autenticacao.entrar(login, terminal.senha("Senha")) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama AutenticacaoServico.entrar: Normaliza login e consulta Optional<Credenciais>. Ausência ou hash incorreto dão a mesma mensagem. Rejeita ausência de perfis, revalida acesso e registra ENTRAR; devolve Sessao e limpa senha mesmo em erro.

148         } catch (RegraNegocioException erro) {
Trata a(s) exceção(ões) declarada(s) em catch (RegraNegocioException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

149             System.out.println(erro.getMessage());
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

150         }
151     }
152     System.out.println("Limite de tentativas desta execução atingido.");
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

153     return null;
return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

154 }
155

```

## inicializar - L156 a L163

Lê os dados do primeiro administrador e do aeroporto-base, cria PrimeiroAcesso e chama InicializacaoServico.criar. O método não retorna id; o sucesso é exibido no terminal.

Entradas declaradas: (Terminal terminal, InicializacaoServico inicializacao).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: Main.main (L55).

Chamadas observadas no corpo: System.out.println, new PrimeiroAcesso, terminal.ler, terminal.data, inicializacao.criar, terminal.senha.

```

156     private static void inicializar(Terminal terminal, InicializacaoServico inicializacao) {
Início da declaração de inicializar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

157         System.out.println("PRIMEIRO ACESSO – criação do administrador e de seu aeroporto-base no
Brasil.");
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

158         var dados = new PrimeiroAcesso(terminal.ler("Nome do administrador"), terminal.data("Nascimento"),
Declara variável local dados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói PrimeiroAcesso: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

159         terminal.ler("Login"), terminal.ler("Cidade do aeroporto"), terminal.ler("Estado/região"),
Nesta parte da expressão, chama terminal.ler. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

160         terminal.ler("Nome do aeroporto"), terminal.ler("Código ICAO do aeroporto (4 letras)"));
Nesta parte da expressão, chama terminal.ler. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

161         inicializacao.criar(dados, terminal.senha("Senha (mínimo 12 caracteres)"));
Chama InicializacaoServico.criar: Senha entre 12 e 128; nascimento validado pelo Clock. Gera hash antes da transação, cria administrador e registra auditoria; finally zera char[].

162         System.out.println("Primeiro acesso criado. Execute novamente sem -Inicializar para entrar.");
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

163     }
164

```

## configurarLog - L165 a L175

Cria logs, limita cada arquivo a 1.000.000 bytes e mantém até três arquivos com append. Usa UTF-8/SimpleFormatter. Se o arquivo estiver indisponível, avisa e continua.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: Main.main (L55).

Chamadas observadas no corpo: Files.createDirectories, Path.of, new FileHandler, arquivo.setEncoding, arquivo.setFormatter, new SimpleFormatter, Logger.getLogger("br.edu.aeroporto").addHandler, Logger.getLogger, System.err.println.

```

165     private static void configurarLog() {
        Início da declaração de configurarLog. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
        é invocado.

166         try {
            Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

167             Files.createDirectories(Path.of("logs"));
            Invoca Files.createDirectories com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas
            do método para localizar o corpo no projeto.

168             var arquivo = new FileHandler("logs/sistema-%g.log", 1_000_000, 3, true);
            Declara variável local arquivo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
            escopo é o bloco atual.

            new constrói FileHandler: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si
            só.

169             arquivo.setEncoding("UTF-8");
            Invoca arquivo.setEncoding com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas
            do método para localizar o corpo no projeto.

170             arquivo.setFormatter(new SimpleFormatter());
            Invoca arquivo.setFormatter com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas
            do método para localizar o corpo no projeto.

            new constrói SimpleFormatter: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
            si só.

171             Logger.getLogger("br.edu.aeroporto").addHandler(arquivo);
            Invoca Logger.getLogger("br.edu.aeroporto").addHandler com os argumentos mostrados. O receptor/contrato determina a
            implementação; veja as chamadas do método para localizar o corpo no projeto.

172         } catch (IOException | SecurityException erro) {
            Trata a(s) exceção(ões) declarada(s) em catch (IOException | SecurityException erro). O tratamento mostrado abaixo decide se
            avisa, repete ou relança.

173             System.err.println("Log em arquivo indisponível; mensagens técnicas serão exibidas no
            terminal.");
            Chama System.err.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

174         }
175     }
176 }
```

## Configuracao

src/main/java/br/edu/aeroporto/config/Configuracao.java

Record com URL JDBC, usuário, senha e fuso. Carrega o arquivo local e aplica substituições por variáveis de ambiente; evita mostrar a senha em toString.

42 linhas | 3 métodos/construtores declarados | SHA-256: 96003a4b26ca7dde...

```

1 package br.edu.aeroporto.config;
  Declara o pacote deste arquivo: br.edu.aeroporto.config. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.io.IOException;
4 import java.io.Reader;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

5 import java.nio.charset.StandardCharsets;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

6 import java.nio.file.Files;
7 import java.nio.file.Path;
8 import java.time.ZoneId;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

9 import java.util.Map;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

10 import java.util.Properties;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

11
12 public record Configuracao(String url, String usuario, String senha, ZoneId fuso) {
  Declara Configuracao como portador de dados com membros gerados.
```

### carregar - L13 a L31

Lê Properties com UTF-8 se o arquivo existir; variáveis AEROPORTO\_DB\_\* têm prioridade. Exige URL PostgreSQL, usuário e senha não vazios; cria ZoneId e devolve Configuracao.

Entradas declaradas: ().

Saída declarada: Configuracao - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: Main.main (L55).

Chamadas observadas no corpo: new Properties, Path.of, Files.exists, Files.isRegularFile, new IOException, Files.newBufferedReader, propriedades.load, System.getenv, valor, ZoneId.of, url.startsWith, usuario.isBlank, senha.isBlank, new IllegalArgumentException, new Configuracao.

```

13 public static Configuracao carregar() throws IOException {
  Início da declaração de carregar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

14     Properties propriedades = new Properties();
  Declara variável local propriedades do tipo Properties. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

  new constrói Properties: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

15     Path arquivo = Path.of("config", "application.properties");
  Declara variável local arquivo do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

16     if (Files.exists(arquivo)) {
  Decisão específica: (Files.exists(arquivo)). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

17         if (!Files.isRegularFile(arquivo)) throw new IOException("A configuração precisa ser um
arquivo.");
  Decisão específica: (!Files.isRegularFile(arquivo)). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

  Lança new IOException("A configuração precisa ser um arquivo."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.
```



new constrói IOException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

18         try (Reader reader = Files.newBufferedReader(arquivo, StandardCharsets.UTF_8)) {
Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.
19             propriedades.load(reader);
Invoca propriedades.load com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
20         }
21     }
22     Map<String, String> ambiente = System.getenv();
Declara variável local ambiente do tipo Map<String, String>. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
23     String url = valor(propriedades, ambiente, "db.url", "AEROPORTO_DB_URL",
"jdbc:postgresql://localhost:5432/sistema_aeroporto");
Declara variável local url do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
24     String usuario = valor(propriedades, ambiente, "db.usuario", "AEROPORTO_DB_USUARIO", "postgres");
Declara variável local usuario do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
25     String senha = valor(propriedades, ambiente, "db.senha", "AEROPORTO_DB_SENHA", "");
Declara variável local senha do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
26     ZoneId fuso = ZoneId.of(valor(propriedades, ambiente, "app.fuso", "AEROPORTO_FUSO",
"America/Sao_Paulo"));
Declara variável local fuso do tipo ZoneId. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
27     if (!url.startsWith("jdbc:postgresql:") || usuario.isBlank() || senha.isBlank()) {
Decisão específica: (!url.startsWith("jdbc:postgresql:") || usuario.isBlank() || senha.isBlank()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.
28         throw new IllegalArgumentException("Configure db.url, db.usuario e db.senha em
config/application.properties ou nas variáveis de ambiente.");
Lança new IllegalArgumentException("Configure db.url, db.usuario e db.senha em config/application.properties ou nas variáveis de ambiente."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.
29     }
30     return new Configuracao(url, usuario, senha, fuso);
Retorno: entrega new Configuracao(url, usuario, senha, fuso) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói Configuracao: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
31     }
32

```

## valor - L33 a L36

getOrDefault procura a variável de ambiente; ausente, usa propriedade do arquivo; ausente também, usa o padrão.

Entradas declaradas: (Properties propriedades, Map<String, String> ambiente, String chave, String variavel, String padrao).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: Configuracao.carregar (L13).

Chamadas observadas no corpo: ambiente.getOrDefault, propriedades.getProperty.

```

33     private static String valor(Properties propriedades, Map<String, String> ambiente,
Início da declaração de valor. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
34         String chave, String variavel, String padrao) {
Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.
35         return ambiente.getOrDefault(variavel, propriedades.getProperty(chave, padrao));
Retorno: entrega ambiente.getOrDefault(variavel, propriedades.getProperty(chave, padrao)) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca ambiente.getOrDefault com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
36     }
37

```



## toString - L38 a L41

Devolve apenas o fuso. Não expõe URL, usuário ou senha ao imprimir a configuração.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
38     @Override
```

Anotação de contrato: `@Override` confere sobrescrita; `@FunctionalInterface` confere um único método abstrato, conforme a anotação mostrada.

```
39     public String toString() {
```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
40         return "Configuracao[fuso=" + fuso + "];
```

Retorno: entrega "Configuracao[fuso=" + fuso + "]" ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
41     }
```

```
42 }
```

# Java: domínio e validações

## CadastroPassageiro

src/main/java/br/edu/aeroporto dominio/CadastroPassageiro.java

Dados de entrada do cadastro. Normaliza nome, CPF, e-mail e telefone antes de chegar ao serviço; a data de nascimento é validada pelo serviço contra o relógio.

20 linhas | 1 métodos/construtores declarados | SHA-256: c25f936692b5c65c...

```
1 package br.edu.aeroporto.dominio;
```

Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

```
2
```

```
3 import br.edu.aeroporto.excecao.RegraNegocioException;
```

```
4 import java.time.LocalDate;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. data de calendário, sem hora ou fuso

```
5 import java.util.Locale;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
6
```

```
7 public record CadastroPassageiro(String nome, LocalDate nascimento, String cpf, String email, String
  telefone) {
```

Declara CadastroPassageiro como portador de dados com membros gerados.

### CadastroPassageiro - L8 a L19

Construtor compacto: normaliza nome e CPF; contatos null viram texto vazio; e-mail é minúsculo e telefone perde separadores. E-mail e telefone preenchidos precisam passar os formatos mostrados. Nascimento ainda será validado pelo serviço contra o Clock.

Construtor sem lista explícita de parâmetros. Em record compacto, recebe os componentes declarados no cabeçalho; nas demais classes, segue a declaração apresentada.

Possíveis chamadores identificados: ArquivosServico.importarPassageiros (L24); DadosFicticiosServico.criar (L23); MenuPrincipal.cadastrarPassageiro (L100).

Chamadas observadas no corpo: Validacao.texto, Validacao.cpf, email.strip().toLowerCase, email.strip, telefone.strip().replaceAll, telefone.strip, email.isEmpty, email.length, email.matches, new RegraNegocioException, telefone.isEmpty, telefone.matches.

```
8 public CadastroPassageiro {
```

Início da declaração de CadastroPassageiro. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
9     nome = Validacao.texto(nome, "Nome", 160);
```

Atualiza nome usando = e o resultado de Validacao.texto(nome, "Nome", 160). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
10     cpf = Validacao.cpf(cpf);
```

Atualiza cpf usando = e o resultado de Validacao.cpf(cpf). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
11     email = email == null ? "" : email.strip().toLowerCase(Locale.ROOT);
```

Atualiza email usando = e o resultado de email == null ? "" : email.strip().toLowerCase(Locale.ROOT). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

Escolha de valor: email == null. true escolhe ""; false escolhe email.strip().toLowerCase(Locale.ROOT). Só a alternativa escolhida é avaliada.

```
12     telefone = telefone == null ? "" : telefone.strip().replaceAll("[()\\-\\s]", "");
```

Atualiza telefone usando = e o resultado de telefone == null ? "" : telefone.strip().replaceAll("[()\\-\\s]", ""). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

Escolha de valor: telefone == null. true escolhe ""; false escolhe telefone.strip().replaceAll("[()\\-\\s]", ""). Só a alternativa escolhida é avaliada.

```
13         if (!email.isEmpty() && (email.length() > 160 || !email.matches("[^\\s@]+@[^\\s@]+\\.([^\\s@]+)"))
14     {
        Decisão específica: (!email.isEmpty() && (email.length() > 160 || !email.matches("[^\\s@]+@[^\\s@]+\\.([^\\s@]+)")). Se for true,
        executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

14         throw new RegraNegocioException("E-mail inválido.");
        Lança new RegraNegocioException("E-mail inválido."). Interrompe o caminho normal; a exceção sobe até uma captura
        compatível e pode provocar rollback.

15     }
16     if (!telefone.isEmpty() && !telefone.matches("\\+?[0-9]{8,15}")) {
        Decisão específica: (!telefone.isEmpty() && !telefone.matches("\\+?[0-9]{8,15}")). Se for true, executa o ramo seguinte; se for
        false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

17         throw new RegraNegocioException("Telefone inválido: use de 8 a 15 dígitos, com + opcional.");
        Lança new RegraNegocioException("Telefone inválido: use de 8 a 15 dígitos, com + opcional."). Interrompe o caminho normal; a
        exceção sobe até uma captura compatível e pode provocar rollback.

18     }
19 }
20 }
```

## Dados

src/main/java/br/edu/aeroporto/ dominio/Dados.java

Funções utilitárias para extrair valores dos Map vindos do JDBC, validar dinheiro, converter instantes, exigir condições e gerar códigos.

51 linhas | 8 métodos/construtores declarados | SHA-256: 907d60090795b49c...

```

1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.excecao.RegraNegocioException;
4 import java.math.BigDecimal;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. valor decimal,
  utilizado nos cálculos financeiros

5 import java.sql.Timestamp;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem
  executa o tipo; veja package/import no capítulo de sintaxe.

6 import java.time.Instant;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. ponto na linha do
  tempo

7 import java.time.OffsetDateTime;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem
  executa o tipo; veja package/import no capítulo de sintaxe.

8 import java.util.Map;
9 import java.util.UUID;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem
  executa o tipo; veja package/import no capítulo de sintaxe.

10
11 public final class Dados {
  Declara Dados como tipo com campos e métodos. final impede herança.

```

### Dados - L12 a L12

Construtor vazio. Se for private, impede criar instâncias desta classe utilitária a partir de outras classes; os métodos static são usados pelo nome da classe.

Entradas declaradas: ().

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```

12 private Dados() {}
  Início da declaração de Dados. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
  invocado.

13

```

### id - L14 a L18

Lê a chave no Map. Só aceita Number com longValue positivo; lança regra se faltar ou tiver tipo/valor errado.

Entradas declaradas: (Map<String, Object> dados, String campo).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AtendimentoServico.embarcar (L132); AtendimentoServico.rastrear (L111); BloqueiosJdbc.item (L29); CadastroServico.validar (L53); DadosFicticiosServico.criar (L23); FinanceiroJdbc.confirmar (L25); FinanceiroServico.cancelarPagamento (L55); FinanceiroServico.pagar (L22); FinanceiroServico.processarReembolso (L89); FinanceiroServico.solicitarReembolso (L71); IntegracaoTeste.cadastros (L62); IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cenariosAdicionais (L203); OperacaoJdbc.validarEscala (L20); OperacaoJdbc.validarEscalasDoVoo (L14); OperacaoJdbc.validarRecurso (L67); OperacaoServico.liberarAlocacao (L43); OperacaoServico.liberarEscala (L81); OperacaoServico.occurencia (L155); OperacaoServico.servicoSolo (L130); OperacaoServico.situacaoManutencao (L107); OperacaoServico.situacaoSolo (L144); PlanejamentoJdbc.agendas (L76); PlanejamentoJdbc.validarFrota (L15); PlanejamentoServico.bloquearAssento (L40); PlanejamentoServico.horarios (L71); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.trocarAeronave (L93); ReservaJdbc.cancelarItem (L93); ReservaJdbc.expirar (L148); ReservaJdbc.invalidar (L110); ReservaJdbc.reservar (L15); ReservaJdbc.solicitarDevolucao (L120); ReservaServico.remarcar (L49); VooServico.transicao (L22).

Chamadas observadas no corpo: dados.get, numero.longValue, new RegraNegocioException.

```

14    public static long id(Map<String, Object> dados, String campo) {
      Início da declaração de id. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
15        Object valor = dados.get(campo);
      Declara variável local valor do tipo Object. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
16        if (!(valor instanceof Number numero) || numero.longValue() <= 0) throw new
      RegraNegocioException("Id invalido: " + campo);
      Decisão específica: !(valor instanceof Number numero) || numero.longValue() <= 0). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

      Lança new RegraNegocioException("Id invalido: " + campo). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

      new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
17        return numero.longValue();
      Retorno: entrega numero.longValue() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

      Invoca numero.longValue com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
18    }
19

```

## texto - L20 a L22

Converte o valor para String. Atenção: String.valueOf(null) produz o texto "null"; esta função não é validação de presença.

Entradas declaradas: (Map<String, Object> dados, String campo).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: AtendimentoServico.rastrear (L111); CadastroServico.validar (L53); OperacaoJdbc.validarEscala (L20); OperacaoServico.situacaoManutencao (L107); OperacaoServico.situacaoSolo (L144); PlanejamentoServico.horarios (L71); ReservaJdbc.cancelarItem (L93); VooServico.transicao (L22).

Chamadas observadas no corpo: String.valueOf, dados.get.

```

20    public static String texto(Map<String, Object> dados, String campo) {
      Início da declaração de texto. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
21        return String.valueOf(dados.get(campo));
      Retorno: entrega String.valueOf(dados.get(campo)) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

      Chama String.valueOf: Converte ao tipo receptor conforme seu contrato; para enum exige uma constante válida.
22    }
23

```

## dinheiro - L24 a L27

Valor ausente vira BigDecimal.ZERO; demais valores são convertidos por sua representação textual.

Entradas declaradas: (Map<String, Object> dados, String campo).

Saída declarada: BigDecimal - valor decimal, utilizado nos cálculos financeiros.

Possíveis chamadores identificados: AtendimentoServico.despachar (L88); AtendimentoServico.excesso (L106); AtendimentoServico.rastrear (L111); FinanceiroServico.pagar (L22); FinanceiroServico.processarReembolso (L89); FinanceiroServico.solicitarReembolso (L71); OperacaoJdbc.validarRecurso (L67); ReservaJdbc.cancelarItem (L93); ReservaJdbc.reservar (L15); ReservaJdbc.solicitarDevolucao (L120).

Chamadas observadas no corpo: dados.get, new BigDecimal, valor.toString.

```

24    public static BigDecimal dinheiro(Map<String, Object> dados, String campo) {
      Início da declaração de dinheiro. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
25        Object valor = dados.get(campo);
      Instrução idêntica à de Dados:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

```
26         return valor == null ? BigDecimal.ZERO : new BigDecimal(valor.toString());
```

Retorno: entrega `valor == null ? BigDecimal.ZERO : new BigDecimal(valor.toString())` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Escolha de valor: `valor == null`. `true` escolhe `BigDecimal.ZERO`; `false` escolhe `new BigDecimal(valor.toString())`. Só a alternativa escolhida é avaliada.

`new` constrói `BigDecimal`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
27     }
28
```

## valor - L29 a L34

Verifica presença, no máximo duas casas, precisão de até 14 dígitos e sinal. `positivo=true` exige `>0`; `false` admite 0. Ajusta a escala final para 2.

Entradas declaradas: (`BigDecimal valor`, `boolean positivo`).

Saída declarada: `BigDecimal` - valor decimal, utilizado nos cálculos financeiros.

Possíveis chamadores identificados: `AtendimentoServico.despachar` (L88); `FinanceiroServico.pagar` (L22); `FinanceiroServico.solicitarReembolso` (L71); `OperacaoServico.manutencao` (L94); `OperacaoServico.servicoSolo` (L130); `PedidoTrecho.PedidoTrecho` (L6); `Tarifa.Tarifa` (L8); `TipoCadastro.converter` (L46); `UnidadeTeste.main` (L10).

Chamadas observadas no corpo: `valor.scale`, `valor.precision`, `valor.signum`, `new RegraNegocioException`, `valor.setScale`.

```
29     public static BigDecimal valor(BigDecimal valor, boolean positivo) {
```

Início da declaração de valor. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
30         if (valor == null || valor.scale() > 2 || valor.precision() > 14 || valor.signum() < (positivo ? 1
: 0)) {
```

Decisão específica: `(valor == null || valor.scale() > 2 || valor.precision() > 14 || valor.signum() < (positivo ? 1 : 0))`. Se for `true`, executa o ramo seguinte; se for `false`, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Escolha de valor: `positivo`. `true` escolhe 1; `false` escolhe 0. Só a alternativa escolhida é avaliada.

```
31         throw new RegraNegocioException("Valor monetario invalido; use ate duas casas decimais.");
```

Lança `new RegraNegocioException("Valor monetario invalido; use ate duas casas decimais.")`. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar `rollback`.

```
32     }
33     return valor.setScale(2);
```

Retorno: entrega `valor.setScale(2)` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca `valor.setScale` com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
34     }
35
```

## instante - L36 a L42

Reconhece `OffsetDateTime`, `Timestamp` e `Instant`. Os dois primeiros viram `Instant`; o último é truncado a microssegundos. Tipo inesperado lança regra.

Entradas declaradas: (`Map<String, Object> dados`, `String campo`).

Saída declarada: `Instant` - ponto na linha do tempo.

Possíveis chamadores identificados: `AtendimentoServico.embarcar` (L132); `AtendimentoServico.rastrear` (L111); `FinanceiroJdbc.confirmar` (L25); `FinanceiroServico.validarReserva` (L41); `IntegracaoTeste.cancelamentoAposEmbarque` (L309); `IntegracaoTeste.cenariosAdicionais` (L203); `OperacaoJdbc.validarEscala` (L20); `OperacaoServico.situacaoManutencao` (L107); `PlanejamentoJdbc.agendas` (L76); `PlanejamentoJdbc.efetivo` (L93); `PlanejamentoJdbc.validarFrota` (L15); `ReservaJdbc.expirar` (L148); `ReservaJdbc.janela` (L76); `VooServico.transicao` (L22).

Chamadas observadas no corpo: `dados.get`, `data.toInstant`, `data.truncatedTo`, `new RegraNegocioException`.

```
36     public static Instant instante(Map<String, Object> dados, String campo) {
```

Início da declaração de instante. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
37         Object valor = dados.get(campo);
```

Instrução idêntica à de `Dados:L15`. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
38         if (valor instanceof OffsetDateTime data) return data.toInstant();
```

Decisão específica: (valor instanceof OffsetDateTime data). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega data.toInstant() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
39         if (valor instanceof Timestamp data) return data.toInstant();
```

Decisão específica: (valor instanceof Timestamp data). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega data.toInstant() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
40         if (valor instanceof Instant data) return data.truncatedTo(java.time.temporal.ChronoUnit.MICROS);
```

Decisão específica: (valor instanceof Instant data). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega data.truncatedTo(java.time.temporal.ChronoUnit.MICROS) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
41         throw new RegraNegocioException("Horario nao informado: " + campo);
```

Lança new RegraNegocioException("Horario nao informado: " + campo). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

```
42     }
```

```
43
```

## exigir - L44 a L46

Guarda de regra: condição false lança RegraNegocioException com a mensagem; true apenas permite continuar.

Entradas declaradas: (boolean condicao, String mensagem).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AdministracaoServico.acesso (L47); AdministracaoServico.alterarSenha (L67); AdministracaoServico.criarUsuario (L32); AdministracaoServico.validarPerfis (L82); AdministracaoServico.validarSenha (L81); ArquivosServico.copiarRelatorio (L60); ArquivosServico.importarPassageiros (L24); AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AtendimentoServico.embarcar (L132); AtendimentoServico.janela (L158); AtendimentoServico.rastrear (L111); CadastroServico.listar (L25); CadastroServico.salvar (L30); CadastroServico.validar (L53); DadosFicticiosServico.criar (L23); FinanceiroJdbc.confirmar (L25); FinanceiroServico.cancelarPagamento (L55); FinanceiroServico.pagar (L22); FinanceiroServico.processar (L47); FinanceiroServico.processarReembolso (L89); FinanceiroServico.solicitarReembolso (L71); FinanceiroServico.validarReserva (L41); MenuOperacional.rascunho (L104); OperacaoJdbc.validarEscala (L20); OperacaoJdbc.validarRecurso (L67); OperacaoJdbc.validarTripulacao (L57); OperacaoServico.encerrarOcorrencia (L166); OperacaoServico.escalar (L70); OperacaoServico.habilitar (L56); OperacaoServico.interditar (L33); OperacaoServico.liberarAlocacao (L43); OperacaoServico.liberarEscala (L81); OperacaoServico.manutencao (L94); OperacaoServico.ocorrencia (L155); OperacaoServico.servicoSolo (L130); OperacaoServico.situacaoManutencao (L107); OperacaoServico.situacaoSolo (L144); PassageiroServico.editar (L55); PassageiroServico.vincular (L44); PedidoTrecho.PedidoTrecho (L6); PlanejamentoJdbc.inventario (L54); PlanejamentoJdbc.tarifa (L62); PlanejamentoJdbc.validarFrota (L15); PlanejamentoServico.bloquearAssento (L40); PlanejamentoServico.horarios (L71); PlanejamentoServico.programar (L20); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.tarifa (L30); PlanejamentoServico.trocarAeronave (L93); ProgramacaoVoo.ProgramacaoVoo (L7); ProgramacaoVoo.validarHorarios (L21); RelatoriosCompletoServico.consultar (L29); ReservaJdbc.cancelarItem (L93); ReservaJdbc.reservar (L15); ReservaServico.cancelar (L36); ReservaServico.remarcar (L49); ReservaServico.reservar (L26); Tarifa.Tarifa (L8); TipoCadastro.converter (L46); VooServico.transicao (L22).

Chamadas observadas no corpo: new RegraNegocioException.

```
44     public static void exigir(boolean condicao, String mensagem) {
```

Início da declaração de exigir. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
45         if (!condicao) throw new RegraNegocioException(mensagem);
```

Decisão específica: (!condicao). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException(mensagem). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
46     }
```

47

## codigo - L48 a L50

Gera UUID, remove hífens, corta para completar o tamanho total com o prefixo e converte em maiúsculas. Não é id numérico da tabela.

Entradas declaradas: (String prefixo, int tamanho).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); FinanceiroJdbc.confirmar (L25); ReservaJdbc.reservar (L15).

Chamadas observadas no corpo: UUID.randomUUID().toString().replace("-", "").substring(0, tamanho-prefixo.length()).toUpperCase, UUID.randomUUID().toString().replace("-", "").substring, UUID.randomUUID().toString().replace, UUID.randomUUID().toString, UUID.randomUUID, prefixo.length.

```
48     public static String codigo(String prefixo, int tamanho) {
```

Início da declaração de codigo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
49         return prefixo + UUID.randomUUID().toString().replace("-", "").substring(0,
tamanho-prefixo.length()).toUpperCase(java.util.Locale.ROOT);
```

Retorno: entrega prefixo + UUID.randomUUID().toString().replace("-", "").substring(0, tamanho-prefixo.length()).toUpperCase(java.util.Locale.ROOT) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
50     }
```

```
51 }
```



## Funcionario

src/main/java/br/edu/aeroporto dominio/Funcionario.java

Papel de uma Pessoa contratada. Acrescenta a matrícula; habilitações, escalas e cargos ficam em outras tabelas/serviços.

15 linhas | 2 métodos/construtores declarados | SHA-256: 04e4d90702469663...

```

1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.time.LocalDate;
4
5 public final class Funcionario extends Pessoa {
  Declara Funcionario como tipo com campos e métodos. final impede herança. extends define a superclasse.

6     private final String matricula;
  Campo(s) matricula: private restringe acesso à classe. Tipo String: texto imutável. final impede reatribuir esse campo depois da
  inicialização.

7

```

### Funcionario - L8 a L11

Chama super para inicializar Pessoa e valida/guarda a matrícula específica de funcionário.

Entradas declaradas: (long id, String nome, LocalDate nascimento, boolean ativo, String matricula).

Possíveis chamadores identificados: UsuarioJdbc.buscarAtivo (L13).

Chamadas observadas no corpo: super, Validacao.texto.

```

8     public Funcionario(long id, String nome, LocalDate nascimento, boolean ativo, String matricula) {
  Início da declaração de Funcionario. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
  invocado.

9         super(id, nome, nascimento, ativo);
  Inicializa a superclasse com os argumentos mostrados; o objeto mantém os campos herdados e os específicos desta classe.

10        this.matricula = Validacao.texto(matricula, "Matrícula", 30);
  Guarda no campo matricula da instância atual o valor de Validacao.texto(matricula, "Matrícula", 30). O this diferencia o campo de
  parâmetros com o mesmo nome.

11    }
12

```

### identificacao - L13 a L14

Implementa o contrato de Pessoa e devolve a matrícula; por isso Pessoa.toString usa a identificação do papel concreto.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```

13     @Override
  Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a
  anotação mostrada.

14     public String identificacao() { return matricula; }
  Retorno: entrega matricula ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
  caminho.

15 }

```

## MapaAssentos

src/main/java/br/edu/aeroporto/ dominio/ MapaAssentos.java

Transforma registros em vetores de filas/colunas e matriz de células. Mantém posições inexistentes como -- e devolve cópias defensivas.

37 linhas | 5 métodos/construtores declarados | SHA-256: 37a7ac035c020694...

```

1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.util.Arrays;
4 import java.util.List;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

5 import java.util.Map;
6
7 public final class MapaAssentos {
  Declara MapaAssentos como tipo com campos e métodos. final impede herança.

8     private final int[] filas;
  Campo(s) filas: private restringe acesso à classe. Tipo int[]: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

9     private final String[] colunas;
  Campo(s) colunas: private restringe acesso à classe. Tipo String[]: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

10    private final String[][] celulas;
  Campo(s) celulas: private restringe acesso à classe. Tipo String[][]: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

11

```

### MapaAssentos - L12 a L24

Extrai filas distintas/ordenadas e colunas distintas, ordena colunas, cria a matriz, preenche -- e localiza cada assento por busca binária. B significa bloqueado, O ocupado e L livre.

Entradas declaradas: (List<Map<String, Object>> assentos).

Possíveis chamadores identificados: RelatoriosCompletoServico.mapa (L44); UnidadeTeste.main (L10).

Chamadas observadas no corpo: assentos.stream().mapToInt(a->((Number)a.get("fila")).intValue()).distinct().sorted().toArray, assentos.stream().mapToInt(a->((Number)a.get("fila")).intValue()).distinct().sorted, assentos.stream().mapToInt(a->((Number)a.get("fila")).intValue()).distinct, assentos.stream().mapToInt, assentos.stream, ((Number)a.get("fila")).intValue, a.get, assentos.stream().map(a->a.get("coluna").toString()).distinct().toArray, assentos.stream().map(a->a.get("coluna").toString()).distinct, assentos.stream().map, a.get("coluna").toString, String[]::new, Arrays.sort, Arrays.fill, Arrays.binarySearch, ((Number)assento.get("fila")).intValue, assento.get, assento.get("coluna").toString, Boolean.TRUE.equals.

```

12 public MapaAssentos(List<Map<String, Object>> assentos) {
  Início da declaração de MapaAssentos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

13     filas=assentos.stream().mapToInt(a->((Number)a.get("fila")).intValue()).distinct().sorted().toArray();
  Atualiza filas usando = e o resultado de assentos.stream().mapToInt(a->((Number)a.get("fila")).intValue()).distinct().sorted().toArray(). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

14     colunas=assentos.stream().map(a->a.get("coluna").toString()).distinct().toArray(String[]::new);
  Atualiza colunas usando = e o resultado de assentos.stream().map(a->a.get("coluna").toString()).distinct().toArray(String[]::new). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

15     Arrays.sort(colunas);
  Invoca Arrays.sort com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

16     celulas=new String[filas.length][colunas.length];
  Atualiza celulas usando = e o resultado de new String[filas.length][colunas.length]. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

  new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

```
17         for(String[] fila:celulas) Arrays.fill(fila,"-");
```

Percorre cada elemento da coleção/vetor: for(String[] fila:celulas). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
18         for(var assento:assentos) {
```

Percorre cada elemento da coleção/vetor: for(var assento:assentos). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
19             int fila=Arrays.binarySearch(filas,((Number)assento.get("fila")).intValue());
```

Declara variável local fila do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
20             int coluna=Arrays.binarySearch(colunas,assento.get("coluna").toString());
```

Declara variável local coluna do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
21             String estado=Boolean.TRUE.equals(assento.get("bloqueado"))?"B":Boolean.TRUE.equals(assento.ge
t("ocupado"))?"O":"L";
```

Declara variável local estado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Escolha de valor: Boolean.TRUE.equals(assento.get("bloqueado")). true escolhe "B"; false escolhe Boolean.TRUE.equals(assento.get("ocupado"))?"O":"L". Só a alternativa escolhida é avaliada.

Escolha de valor: Boolean.TRUE.equals(assento.get("ocupado")). true escolhe "O"; false escolhe "L". Só a alternativa escolhida é avaliada.

```
22         celulas[fila][coluna]=assento.get("codigo")+":"+estado;
```

Atualiza celulas[fila][coluna] usando = e o resultado de assento.get("codigo")+":"+estado. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
23     }
```

```
24 }
```

```
25
```

## filas - L26 a L26

Devolve cópia do vetor numérico; alterar a cópia não muda o mapa.

Entradas declaradas: ().

Saída declarada: int[] - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Arrays.copyOf.

```
26     public int[] filas() { return Arrays.copyOf(filas,filas.length); }
```

Início da declaração de filas. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega Arrays.copyOf(filas,filas.length) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## colunas - L27 a L27

Devolve cópia do vetor de colunas.

Entradas declaradas: ().

Saída declarada: String[] - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Arrays.copyOf.

```
27     public String[] colunas() { return Arrays.copyOf(colunas,colunas.length); }
```

Início da declaração de colunas. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega Arrays.copyOf(colunas,colunas.length) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## celulas - L28 a L32

Cria outro vetor externo e copia cada linha com Arrays.setAll/copyOf, evitando compartilhar as linhas mutáveis.

Entradas declaradas: ().

Saída declarada: String[][] - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: UnidadeTeste.main (L10).

Chamadas observadas no corpo: Arrays.setAll, Arrays.copyOf.

```

28     public String[][] celulas() {
        Início da declaração de celulas. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

29         String[][] copia=new String[celulas.length][];
        Declara variável local copia do tipo String[[]]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
        new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

30         Arrays.setAll(copia,i->Arrays.copyOf(celulas[i],celulas[i].length));
        Invoca Arrays.setAll com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
        método para localizar o corpo no projeto.

31         return copia;
        Retorno: entrega copia ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
        caminho.

32     }
33 
```

## mesmoMapa - L34 a L36

Rejeita outro null e compara vetores com Arrays.equals e conteúdo das linhas com deepEquals.

Entradas declaradas: (MapaAssentos outro).

Saída declarada: boolean - condição verdadeira ou falsa.

Possíveis chamadores identificados: UnidadeTeste.main (L10).

Chamadas observadas no corpo: Arrays.equals, Arrays.deepEquals.

```

34     public boolean mesmoMapa(MapaAssentos outro) {
        Início da declaração de mesmoMapa. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

35         return outro!=null && Arrays.equals(filas,outro.filas) && Arrays.equals(colunas,outro.colunas) &&
        Arrays.deepEquals(celulas,outro.celulas);
        Retorno: entrega outro!=null && Arrays.equals(filas,outro.filas) && Arrays.equals(colunas,outro.colunas) &&
        Arrays.deepEquals(celulas,outro.celulas) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo
        executa por esse caminho.

36     }
37 
```

## Passageiro

src/main/java/br/edu/aeroporto dominio/Passageiro.java

Papel de uma Pessoa que pode viajar. Acrescenta o código de cliente e implementa identificacao; não representa sozinho reservas ou bilhetes.

17 linhas | 3 métodos/construtores declarados | SHA-256: c9da87d583289d7b...

```
1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.time.LocalDate;
4
5 public final class Passageiro extends Pessoa {
  Declara Passageiro como tipo com campos e métodos. final impede herança. extends define a superclasse.

6     private final String codigoCliente;
  Campo(s) codigoCliente: private restringe acesso à classe. Tipo String: texto imutável. final impede reatribuir esse campo depois da inicialização.

7
```

### Passageiro - L8 a L11

Chama super para inicializar a identidade Pessoa e valida/guarda o código de cliente específico do papel Passageiro.

Entradas declaradas: (long id, String nome, LocalDate nascimento, boolean ativo, String codigoCliente).

Possíveis chamadores identificados: PassageiroJdbc.cadastrar (L37); PassageiroJdbc.mapear (L70).

Chamadas observadas no corpo: super, Validacao.texto.

```
8     public Passageiro(long id, String nome, LocalDate nascimento, boolean ativo, String codigoCliente) {
  Início da declaração de Passageiro. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

9         super(id, nome, nascimento, ativo);
  Instrução idêntica à de Funcionario:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

10        this.codigoCliente = Validacao.texto(codigoCliente, "Código do cliente", 30);
  Guarda no campo codigoCliente da instância atual o valor de Validacao.texto(codigoCliente, "Código do cliente", 30). O this diferencia o campo de parâmetros com o mesmo nome.

11    }
12
```

## codigoCliente - L13 a L13

Devolve o código de cliente guardado nesta instância.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: MenuPrincipal.mostrar (L116).

```
13     public String codigoCliente() { return codigoCliente; }
```

Início da declaração de codigoCliente. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega codigoCliente ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
14
```

## identificacao - L15 a L16

Implementa o contrato abstrato de Pessoa e devolve o código de cliente usado na identificação textual.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
15     @Override
```

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

```
16     public String identificacao() { return codigoCliente; }
```

Retorno: entrega codigoCliente ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
17 }
```

## PedidoTrecho

src/main/java/br/edu/aeroporto dominio/PedidoTrecho.java

Um passageiro em um voo, usando uma tarifa e uma ordem de conexão. O assento null solicita escolha automática; desconto é BigDecimal.

11 linhas | 1 métodos/construtores declarados | SHA-256: e379e5763c52e87b...

```
1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.math.BigDecimal;
4
5 public record PedidoTrecho(long passageiro,long voo,long tarifa,int ordem,String assento,BigDecimal
  desconto) {
  Declara PedidoTrecho como portador de dados com membros gerados.
```

### PedidoTrecho - L6 a L10

Exige passageiro/voo/tarifa e ordem positivos, ordem até 32767. Assento ausente/branco vira null, indicando seleção automática; preenchido é limitado e convertido a maiúsculas. Desconto deve ser decimal não negativo; o teto de 10% é conferido no momento da reserva.

Construtor sem lista explícita de parâmetros. Em record compacto, recebe os componentes declarados no cabeçalho; nas demais classes, segue a declaração apresentada.

Possíveis chamadores identificados: IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.comercial (L109); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.trecho (L93).

Chamadas observadas no corpo: Dados.exigir, assento.isBlank, Validacao.texto(assento,"Assento",6).toUpperCase, Validacao.texto, Dados.valor.

```
6 public PedidoTrecho {
  Início da declaração de PedidoTrecho. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
  é invocado.

7     Dados.exigir(passageiro>0 && voo>0 && tarifa>0 && ordem>0 && ordem<=32767,"Trecho invalido.");
  Regra obrigatória: passageiro>0 && voo>0 && tarifa>0 && ordem>0 && ordem<=32767 deve resultar em true. Se false,
  Dados.exigir lança RegraNegocioException com "Trecho invalido.". Essa condição específica não é substituída por outra
  validação parecida.

8     assento=assento==null ||
  assento.isBlank()?null:Validacao.texto(assento,"Assento",6).toUpperCase(java.util.Locale.ROOT);
  Atualiza assento usando = e o resultado de assento==null ||
  assento.isBlank()?null:Validacao.texto(assento,"Assento",6).toUpperCase(java.util.Locale.ROOT). Atribuição modifica a
  variável/célula indicada; não é um teste de igualdade.

  Escolha de valor: assento==null || assento.isBlank(). true escolhe null; false escolhe
  Validacao.texto(assento,"Assento",6).toUpperCase(java.util.Locale.ROOT). Só a alternativa escolhida é avaliada.

9     desconto=Dados.valor(desconto,false);
  Atualiza desconto usando = e o resultado de Dados.valor(desconto,false). Atribuição modifica a variável/célula indicada; não é um
  teste de igualdade.

10 }
11 }
```

## Pessoa

src/main/java/br/edu/aeroporto dominio/Pessoa.java

Classe abstrata com a identidade comum de uma pessoa. Passageiro e Funcionario herdam os dados; equals/hashCode consideram o id da pessoa, mesmo quando os papéis são diferentes.

37 linhas | 9 métodos/construtores declarados | SHA-256: 745d08f1c5715a93...

```

1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.time.LocalDate;
4 import java.util.Objects;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

5
6 public abstract class Pessoa {
  Declara Pessoa como tipo com campos e métodos.

7     private final long id;
  Campo(s) id: private restringe acesso à classe. Tipo long: inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome. final impede reatribuir esse campo depois da inicialização.

8     private final String nome;
  Campo(s) nome: private restringe acesso à classe. Tipo String: texto imutável. final impede reatribuir esse campo depois da inicialização.

9     private final LocalDate nascimento;
  Campo(s) nascimento: private restringe acesso à classe. Tipo LocalDate: data de calendário, sem hora ou fuso. final impede reatribuir esse campo depois da inicialização.

10    private final boolean ativa;
  Campo(s) ativa: private restringe acesso à classe. Tipo boolean: condição verdadeira ou falsa. final impede reatribuir esse campo depois da inicialização.

11

```

### Pessoa - L12 a L18

Exige id persistido positivo, valida nome, rejeita nascimento null e guarda a atividade. Não cria linha no banco; monta um objeto já identificado.

Entradas declaradas: (long id, String nome, LocalDate nascimento, boolean ativa).

Possíveis chamadores identificados: Funcionario.Funcionario (L8); Passageiro.Passageiro (L8).

Chamadas observadas no corpo: new IllegalArgumentException, Validacao.texto, Objects.requireNonNull.

```

12     protected Pessoa(long id, String nome, LocalDate nascimento, boolean ativa) {
  Início da declaração de Pessoa. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

13         if (id <= 0) throw new IllegalArgumentException("Id persistido deve ser positivo.");
  Decisão específica: (id <= 0). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

  Lança new IllegalArgumentException("Id persistido deve ser positivo."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

  new constrói IllegalArgumentException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

14         this.id = id;
  Guarda no campo id da instância atual o valor de id. O this diferencia o campo de parâmetros com o mesmo nome.

15         this.nome = Validacao.texto(nome, "Nome", 160);
  Guarda no campo nome da instância atual o valor de Validacao.texto(nome, "Nome", 160). O this diferencia o campo de parâmetros com o mesmo nome.

16         this.nascimento = Objects.requireNonNull(nascimento);
  Guarda no campo nascimento da instância atual o valor de Objects.requireNonNull(nascimento). O this diferencia o campo de parâmetros com o mesmo nome.

```



```
17      this.ativa = ativa;
```

Guarda no campo `ativa` da instância atual o valor de `ativa`. O `this` diferencia o campo de parâmetros com o mesmo nome.

```
18    }
19
```

## id - L20 a L20

Acessor final: devolve o `id` já guardado na instância. `final` impede uma subclasse de alterar o comportamento deste acessor.

Entradas declaradas: ().

Saída declarada: `long` - inteiro primitivo de 64 bits; neste contexto representa `id`, contador ou valor inteiro identificado pelo nome.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
20    public final long id() { return id; }
```

Início da declaração de `id`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega `id` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## nome - L21 a L21

Acessor final do nome normalizado; não faz outra consulta ou validação.

Entradas declaradas: ().

Saída declarada: `String` - texto imutável.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
21    public final String nome() { return nome; }
```

Início da declaração de `nome`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega `nome` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## nascimento - L22 a L22

Acessor final da `LocalDate` guardada na pessoa.

Entradas declaradas: ().

Saída declarada: `LocalDate` - data de calendário, sem hora ou fuso.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
22    public final LocalDate nascimento() { return nascimento; }
```

Início da declaração de `nascimento`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega `nascimento` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## ativa - L23 a L23

Acessor final boolean da atividade guardada no objeto; ele não reconsulta automaticamente a situação atual no banco.

Entradas declaradas: ().

Saída declarada: `boolean` - condição verdadeira ou falsa.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
23    public final boolean ativa() { return ativa; }
```

Início da declaração de `ativa`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega `ativa` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## identificacao - L24 a L24

Contrato abstrato: Pessoa exige que a subclasse produza sua identificação textual. Não tem corpo executável nesta declaração; Passageiro e Funcionario implementam.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: Pessoa.toString (L35).

```
24    public abstract String identificacao();
```

Início da declaração de identificacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
25
```

## equals - L26 a L30

Igualdade é mesma referência OU outro objeto do tipo Pessoa com mesmo id. O && se aplica ao teste de tipo/id; duas subclasses podem ser iguais por identidade.

Entradas declaradas: (Object outro).

Saída declarada: boolean - condição verdadeira ou falsa.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
26    @Override
```

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

```
27    public final boolean equals(Object outro) {
```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
28        // A mesma pessoa pode ter os dois papéis; a identidade comum é pessoa.id.
```

Comentário/documentação do autor; descreve intenção ou convenção e não executa.

```
29        return this == outro || outro instanceof Pessoa pessoa && id == pessoa.id;
```

Retorno: entrega this == outro || outro instanceof Pessoa pessoa && id == pessoa.id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
30    }
```

```
31
```

## hashCode - L32 a L33

Devolve hash do id para manter o contrato de coleções: pessoas iguais têm o mesmo hash.

Entradas declaradas: ().

Saída declarada: int - inteiro primitivo de 32 bits, usado para índice, limite ou contagem.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Long.hashCode.

```
32    @Override
```

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

```
33     public final int hashCode() { return Long.hashCode(id); }
```

Retorno: entrega Long.hashCode(id) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
34
```

## toString - L35 a L36

Chama identificacao de forma polimórfica e combina com o nome; Funcionario usa matrícula, Passageiro usa código.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: identificacao.

```
35     @Override
```

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

```
36     public String toString() { return identificacao() + " - " + nome; }
```

Retorno: entrega identificacao() + " - " + nome ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
37 }
```

## PrimeiroAcesso

src/main/java/br/edu/aeroporto dominio/PrimeiroAcesso.java

Dados necessários para criar o administrador e o aeroporto-base. Valida nomes, login e o formato do ICAO.

18 linhas | 1 métodos/construtores declarados | SHA-256: c5d301f4fe85bd21...

```
1 package br.edu.aeroporto.dominio;
    Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.execcao.RegraNegocioException;
4 import java.time.LocalDate;
5 import java.util.Locale;
6
7 public record PrimeiroAcesso(String nome, LocalDate nascimento, String login,
    Declara PrimeiroAcesso como portador de dados com membros gerados.

8                               String cidade, String regioao, String aeroporto, String icao) {
    Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
    determina onde a instrução termina.
```

### PrimeiroAcesso - L9 a L17

Construtor compacto que normaliza os textos de pessoa, login, aeroporto e cidade e confere formato do ICAO. A criação efetiva de usuário/aeroporto está no serviço e no JDBC; montar este record não grava nada.

Construtor sem lista explícita de parâmetros. Em record compacto, recebe os componentes declarados no cabeçalho; nas demais classes, segue a declaração apresentada.

Possíveis chamadores identificados: IntegracaoTeste.inicializar (L50); Main.inicializar (L156).

Chamadas observadas no corpo: Validacao.texto, Validacao.login, Validacao.texto(icao, "ICAO", 4).toUpperCase, icao.matches, new RegraNegocioException.

```
9 public PrimeiroAcesso {
    Início da declaração de PrimeiroAcesso. Os parâmetros entram pela chamada; o corpo só executa quando esse
    método/construtor é invocado.

10     nome = Validacao.texto(nome, "Nome", 160);
    Instrução idêntica à de CadastroPassageiro:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
    desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

11     login = Validacao.login(login);
    Atualiza login usando = e o resultado de Validacao.login(login). Atribuição modifica a variável/célula indicada; não é um teste de
    igualdade.

12     cidade = Validacao.texto(cidade, "Cidade", 120);
    Atualiza cidade usando = e o resultado de Validacao.texto(cidade, "Cidade", 120). Atribuição modifica a variável/célula indicada;
    não é um teste de igualdade.

13     regioao = Validacao.texto(regiao, "Estado/região", 100);
    Atualiza regioao usando = e o resultado de Validacao.texto(regiao, "Estado/região", 100). Atribuição modifica a variável/célula
    indicada; não é um teste de igualdade.

14     aeroporto = Validacao.texto(aeroporto, "Aeroporto", 160);
    Atualiza aeroporto usando = e o resultado de Validacao.texto(aeroporto, "Aeroporto", 160). Atribuição modifica a variável/célula
    indicada; não é um teste de igualdade.

15     icao = Validacao.texto(icao, "ICAO", 4).toUpperCase(Locale.ROOT);
    Atualiza icao usando = e o resultado de Validacao.texto(icao, "ICAO", 4).toUpperCase(Locale.ROOT). Atribuição modifica a
    variável/célula indicada; não é um teste de igualdade.

16     if (!icao.matches("[A-Z]{4}")) throw new RegraNegocioException("ICAO deve conter quatro letras.");
    Decisão específica: (icao.matches("[A-Z]{4}")). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo
    interrompe a operação lançando a exceção.

    Lança new RegraNegocioException("ICAO deve conter quatro letras."). Interrompe o caminho normal; a exceção sobe até uma
    captura compatível e pode provocar rollback.

    new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
    banco por si só.

17 }
18 }
```

## ProgramacaoVoo

src/main/java/br/edu/aeroporto dominio/ProgramacaoVoo.java

Pacote de dados para programar/reprogramar um voo. Verifica ids, número, sequência dos horários e janelas; normaliza os instantes para microssegundos.

25 linhas | 2 métodos/construtores declarados | SHA-256: d6e191ae01673e11...

```
1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.time.Instant;
4
5 public record ProgramacaoVoo(long companhia,long rota,long aeronave,String numero,Instant partida,Instant
  chegada,
  Declara ProgramacaoVoo como portador de dados com membros gerados.

6                               Instant checkinAbre,Instant checkinFecha,Instant embarqueAbre,Instant
  embarqueFecha) {
  Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
  determina onde a instrução termina.
```

### ProgramacaoVoo - L7 a L19

Confere ids e número do voo; valida presença/ordem dos seis instantes, trunca todos a microssegundos e valida outra vez. A segunda validação preserva as desigualdades depois de dois horários muito próximos poderem se tornar iguais pelo truncamento.

Construtor sem lista explícita de parâmetros. Em record compacto, recebe os componentes declarados no cabeçalho; nas demais classes, segue a declaração apresentada.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L23); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.programacao (L502); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.programacao (L72).

Chamadas observadas no corpo: Dados.exigir, Validacao.texto, numero.matches, validarHorarios, partida.truncatedTo, chegada.truncatedTo, checkinAbre.truncatedTo, checkinFecha.truncatedTo, embarqueAbre.truncatedTo, embarqueFecha.truncatedTo.

```
7 public ProgramacaoVoo {
  Início da declaração de ProgramacaoVoo. Os parâmetros entram pela chamada; o corpo só executa quando esse
  método/construtor é invocado.

8     Dados.exigir(compania>0 && rota>0 && aeronave>0,"Cadastros do voo invalidos.");
  Regra obrigatória: companhia>0 && rota>0 && aeronave>0 deve resultar em true. Se false, Dados.exigir lança
  RegraNegocioException com "Cadastros do voo invalidos.". Essa condição específica não é substituída por outra validação
  parecida.

9     numero=Validacao.texto(numero,"Numero do voo",8);
  Atualiza numero usando = e o resultado de Validacao.texto(numero,"Numero do voo",8). Atribuição modifica a variável/célula
  indicada; não é um teste de igualdade.

10    Dados.exigir(numero.matches("[A-Z0-9]{1,8}"),"Numero de voo invalido.");
  Regra obrigatória: numero.matches("[A-Z0-9]{1,8}") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException
  com "Numero de voo invalido.". Essa condição específica não é substituída por outra validação parecida.

11    validarHorarios(partida,chegada,checkinAbre,checkinFecha,embarqueAbre,embarqueFecha);
  Chama ProgramacaoVoo.validarHorarios: Exige todos os instantes. Chegada deve ser posterior à partida; abertura vem antes do
  fechamento; check-in fecha no máximo ao fechar embarque; embarque fecha no máximo na partida.

12    partida=partida.truncatedTo(java.time.temporal.ChronoUnit.MICROS);
  Atualiza partida usando = e o resultado de partida.truncatedTo(java.time.temporal.ChronoUnit.MICROS). Atribuição modifica a
  variável/célula indicada; não é um teste de igualdade.

13    chegada=chegada.truncatedTo(java.time.temporal.ChronoUnit.MICROS);
  Atualiza chegada usando = e o resultado de chegada.truncatedTo(java.time.temporal.ChronoUnit.MICROS). Atribuição modifica a
  variável/célula indicada; não é um teste de igualdade.

14    checkinAbre=checkinAbre.truncatedTo(java.time.temporal.ChronoUnit.MICROS);
  Atualiza checkinAbre usando = e o resultado de checkinAbre.truncatedTo(java.time.temporal.ChronoUnit.MICROS). Atribuição
  modifica a variável/célula indicada; não é um teste de igualdade.
```

```

15      checkinFecha=checkinFecha.truncatedTo(java.time.temporal.ChronoUnit.MICROS);
      Atualiza checkinFecha usando = e o resultado de checkinFecha.truncatedTo(java.time.temporal.ChronoUnit.MICROS). Atribuição
      modifica a variável/célula indicada; não é um teste de igualdade.

16      embarqueAbre=embarqueAbre.truncatedTo(java.time.temporal.ChronoUnit.MICROS);
      Atualiza embarqueAbre usando = e o resultado de embarqueAbre.truncatedTo(java.time.temporal.ChronoUnit.MICROS).
      Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

17      embarqueFecha=embarqueFecha.truncatedTo(java.time.temporal.ChronoUnit.MICROS);
      Atualiza embarqueFecha usando = e o resultado de embarqueFecha.truncatedTo(java.time.temporal.ChronoUnit.MICROS).
      Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

18      validarHorarios(partida, chegada, checkinAbre, checkinFecha, embarqueAbre, embarqueFecha);
      Instrução idêntica à de ProgramacaoVoo:L11. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
      execução. A posição e o contexto do método acima mostram quando ela ocorre.

19  }
20

```

## validarHorarios - L21 a L24

Exige todos os instantes. Chegada deve ser posterior à partida; abertura vem antes do fechamento; check-in fecha no máximo ao fechar embarque; embarque fecha no máximo na partida.

Entradas declaradas: (Instant partida, Instant chegada, Instant ca, Instant cf, Instant ea, Instant ef).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: ProgramacaoVoo.ProgramacaoVoo (L7).

Chamadas observadas no corpo: Dados.exigir, chegada.isAfter, ca.isBefore, cf.isAfter, ea.isBefore, ef.isAfter.

```

21      public static void validarHorarios(Instant partida, Instant chegada, Instant ca, Instant cf, Instant
      ea, Instant ef) {
      Início da declaração de validarHorarios. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
      é invocado.

22      Dados.exigir(partida!=null && chegada!=null && ca!=null && cf!=null && ea!=null &&
      ef!=null, "Informe todos os horarios.");
      Regra obrigatória: partida!=null && chegada!=null && ca!=null && cf!=null && ea!=null && ef!=null deve resultar em true. Se false,
      Dados.exigir lança RegraNegocioException com "Informe todos os horarios.". Essa condição específica não é substituída por
      outra validação parecida.

23      Dados.exigir(chegada.isAfter(partida) && ca.isBefore(cf) && !cf.isAfter(ef) && ea.isBefore(ef) &&
      !ef.isAfter(partida), "Janelas e horarios do voo invalidos.");
      Regra obrigatória: chegada.isAfter(partida) && ca.isBefore(cf) && !cf.isAfter(ef) && ea.isBefore(ef) && !ef.isAfter(partida) deve
      resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Janelas e horarios do voo invalidos.". Essa condição
      específica não é substituída por outra validação parecida.

24      }
25  }

```

## Sessao

src/main/java/br/edu/aeroporto dominio/Sessao.java

Identifica o usuário autenticado, seu funcionário e os perfis recebidos no login. Copia os perfis; autorização é reconsultada no banco em cada operação.

10 linhas | 1 métodos/construtores declarados | SHA-256: 17fa279266d978df...

```
1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.
2
3 import java.util.Set;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.
4
5 public record Sessao(long usuarioId, Funcionario funcionario, Set<String> perfis) {
  Declara Sessao como portador de dados com membros gerados.
```

### Sessao - L6 a L8

Copia o conjunto de perfis com `Set.copyOf` para impedir que alterações da coleção original mudem esta sessão. A vigência dos acessos ainda é consultada no banco durante as operações.

Construtor sem lista explícita de parâmetros. Em record compacto, recebe os componentes declarados no cabeçalho; nas demais classes, segue a declaração apresentada.

Possíveis chamadores identificados: `UsuarioJdbc.buscarAtivo` (L13).

Chamadas observadas no corpo: `Set.copyOf`.

```
6     public Sessao {
  Início da declaração de Sessao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
7         perfis = Set.copyOf(perfis);
  Atualiza perfis usando = e o resultado de Set.copyOf(perfis). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.
8     }
9
10 }
```

## SituacaoReserva

src/main/java/br/edu/aeroporto/ dominio/SituacaoReserva.java

Enum dos estados do cabeçalho de reserva. A situação resulta dos itens, recalculada por ReservaJdbc; não se confunde com a situação de voo.

5 linhas | 0 métodos/construtores declarados | SHA-256: b759d75e0ad66f4b...

```
1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.
2
3 public enum SituacaoReserva {
  Declara SituacaoReserva como conjunto fechado de constantes.
4     PENDENTE, CONFIRMADA, PARCIAL_CANCELADA, CANCELADA, EXPIRADA, FINALIZADA
  Constante do enum PENDENTE. É uma opção do conjunto fechado de valores.
  Constante do enum CONFIRMADA. É uma opção do conjunto fechado de valores.
  Constante do enum PARCIAL_CANCELADA. É uma opção do conjunto fechado de valores.
  Constante do enum CANCELADA. É uma opção do conjunto fechado de valores.
  Constante do enum EXPIRADA. É uma opção do conjunto fechado de valores.
  Constante do enum FINALIZADA. É uma opção do conjunto fechado de valores.
5 }
```



## SituacaoVoo

src/main/java/br/edu/aeroporto dominio/SituacaoVoo.java

Enum dos estados Java e das transições possíveis. O enum estabelece a sequência; VooServico verifica janelas, equipe e recursos antes de efetivar a mudança.

17 linhas | 1 métodos/construtores declarados | SHA-256: 56bd5f33b3751d56...

```
1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.
2
3 public enum SituacaoVoo {
  Declara SituacaoVoo como conjunto fechado de constantes.
4     PROGRAMADO, CHECKIN_ABERTO, EMBARQUE, EM_VOO, CONCLUIDO, CANCELADO;
  Constante do enum PROGRAMADO. É uma opção do conjunto fechado de valores.
  Constante do enum CHECKIN_ABERTO. É uma opção do conjunto fechado de valores.
  Constante do enum EMBARQUE. É uma opção do conjunto fechado de valores.
  Constante do enum EM_VOO. É uma opção do conjunto fechado de valores.
  Constante do enum CONCLUIDO. É uma opção do conjunto fechado de valores.
  Constante do enum CANCELADO. É uma opção do conjunto fechado de valores.
5
```

### permiteTransicaoPara - L6 a L16

null ou mesmo estado não fazem transição. Cancelamento só parte de PROGRAMADO/CHECKIN\_ABERTO/EMBARQUE. A sequência normal avança um passo até CONCLUIDO, sem reabertura.

Entradas declaradas: (SituacaoVoo proxima).

Saída declarada: boolean - condição verdadeira ou falsa.

Possíveis chamadores identificados: VooServico.transicao (L22).

```
6     public boolean permiteTransicaoPara(SituacaoVoo proxima) {
  Início da declaração de permiteTransicaoPara. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
7         if (proxima == null || proxima == this) return false;
  Decisão específica: (proxima == null || proxima == this). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.
  Retorno: entrega false ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.
8         if (proxima == CANCELADO) return this == PROGRAMADO || this == CHECKIN_ABERTO || this == EMBARQUE;
  Decisão específica: (proxima == CANCELADO). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.
  Retorno: entrega this == PROGRAMADO || this == CHECKIN_ABERTO || this == EMBARQUE ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.
9         return switch (this) {
  Retorno: entrega switch (this) { case PROGRAMADO -> proxima == CHECKIN_ABERTO; case CHECKIN_ABERTO -> proxima == EMBARQUE; case EMBARQUE -> proxima == EM_VOO; case EM_VOO -> proxima == CONCLUIDO; case CONCLUIDO, CANCELADO -> false; } ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.
10             case PROGRAMADO -> proxima == CHECKIN_ABERTO;
  Ramo do switch: case PROGRAMADO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
11             case CHECKIN_ABERTO -> proxima == EMBARQUE;
  Ramo do switch: case CHECKIN_ABERTO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
12             case EMBARQUE -> proxima == EM_VOO;
  Ramo do switch: case EMBARQUE. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
13         case EM_VOO -> proxima == CONCLUIDO;
```

Ramo do switch: case EM\_VOO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
14         case CONCLUIDO, CANCELADO -> false;
```

Ramo do switch: case CONCLUIDO, CANCELADO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
15     };
```

```
16 }
```

```
17 }
```

## Tarifa

src/main/java/br/edu/aeroporto dominio/Tarifa.java

Contrato comercial: classe, preço, taxa, franquia, peças, cancelamento e multa. Sua cópia no item vendido preserva as condições anteriores.

15 linhas | 1 métodos/construtores declarados | SHA-256: 1a66897a938bcb4c...

```
1 package br.edu.aeroporto.dominio;
   Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.math.BigDecimal;
4 import java.util.Set;
5
6 public record Tarifa(String codigo,String classe,BigDecimal base,BigDecimal taxa,BigDecimal franquia,int
   pecas,
   Declara Tarifa como portador de dados com membros gerados.

7         boolean permiteCancelar,BigDecimal multa) {
   Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
   determina onde a instrução termina.
```

### Tarifa - L8 a L14

Normaliza código, aceita apenas ECONOMICA/EXECUTIVA/PRIMEIRA, valida decimais não negativos, limita peças de 0 a 10 e exige multa não maior que base mais taxa. Não consulta se o voo existe: isso pertence à gravação da tarifa.

Construtor sem lista explícita de parâmetros. Em record compacto, recebe os componentes declarados no cabeçalho; nas demais classes, segue a declaração apresentada.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L23); IntegracaoTeste.cadastros (L62); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: Validacao.texto, Dados.exigir, Set.of("ECONOMICA","EXECUTIVA","PRIMEIRA").contains, Set.of, Dados.valor, multa.compareTo, base.add.

```
8 public Tarifa {
   Início da declaração de Tarifa. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
   invocado.

9     codigo=Validacao.texto(codigo,"Codigo da tarifa",20);
   Atualiza codigo usando = e o resultado de Validacao.texto(codigo,"Codigo da tarifa",20). Atribuição modifica a variável/célula
   indicada; não é um teste de igualdade.

10    Dados.exigir(Set.of("ECONOMICA","EXECUTIVA","PRIMEIRA").contains(classe),"Classe invalida.");
   Regra obrigatória: Set.of("ECONOMICA","EXECUTIVA","PRIMEIRA").contains(classe) deve resultar em true. Se false,
   Dados.exigir lança RegraNegocioException com "Classe invalida.". Essa condição específica não é substituída por outra
   validação parecida.

11    base=Dados.valor(base,false); taxa=Dados.valor(taxa,false); franquia=Dados.valor(franquia,false);
   multa=Dados.valor(multa,false);
   Atualiza base usando = e o resultado de Dados.valor(base,false). Atribuição modifica a variável/célula indicada; não é um teste de
   igualdade.

   Atualiza taxa usando = e o resultado de Dados.valor(taxa,false). Atribuição modifica a variável/célula indicada; não é um teste de
   igualdade.

   Atualiza franquia usando = e o resultado de Dados.valor(franquia,false). Atribuição modifica a variável/célula indicada; não é um
   teste de igualdade.

   Atualiza multa usando = e o resultado de Dados.valor(multa,false). Atribuição modifica a variável/célula indicada; não é um teste
   de igualdade.

12    Dados.exigir(pecas>=0 && pecas<=10,"Limite de pecas invalido.");
   Regra obrigatória: pecas>=0 && pecas<=10 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com
   "Limite de pecas invalido.". Essa condição específica não é substituída por outra validação parecida.

13    Dados.exigir(multa.compareTo(base.add(taxa))<=0,"Multa supera a tarifa.");
   Regra obrigatória: multa.compareTo(base.add(taxa))<=0 deve resultar em true. Se false, Dados.exigir lança
   RegraNegocioException com "Multa supera a tarifa.". Essa condição específica não é substituída por outra validação parecida.

14 }
15 }
```

## TipoCadastro

src/main/java/br/edu/aeroporto/ dominio/TipoCadastro.java

Catálogo fechado das tabelas e campos editáveis pelo menu. A mini linguagem de tipos permite converter entradas sem aceitar nomes arbitrários de tabela/coluna.

67 linhas | 5 métodos/construtores declarados | SHA-256: 848a4ebf8c17a109...

```

1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.excecao.RegraNegocioException;
4 import java.math.BigDecimal;
5 import java.time.LocalDate;
6 import java.time.ZoneId;
7 import java.util.Arrays;
8 import java.util.List;
9
10 /** Lista fechada dos cadastros; nomes de tabelas/colunas nunca vem da entrada. */
  Comentário/documentação do autor; descreve intenção ou convenção e não executa.

11 public enum TipoCadastro {
  Declara TipoCadastro como conjunto fechado de constantes.

12     PAIS("pais", "id", "codigo:s2 nome:s100?"),
  Constante do enum PAIS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

13     CIDADE("cidade", "id", "pais_id:i nome:s120 regio:s100?"),
  Constante do enum CIDADE. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

14     AEROPORTO("aeroporto", "id", "cidade_id:i codigo_iata:s3? codigo_icao:s4 nome:s160 fuso_horario:s80
ativo:b"),
  Constante do enum AEROPORTO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

15     TERMINAL("terminal", "id", "aeroporto_id:i codigo:s15 nome:s100 ativo:b"),
  Constante do enum TERMINAL. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

16     RECURSO("recurso_aeroportuario", "id", "aeroporto_id:i terminal_id:i?
tipo:PORTAO,PISTA,POSICAO,ESTEIRA,BALCAO codigo:s20 situacao:DISPONIVEL,INTERDITADO,MANUTENCAO
comprimento_m:n? envergadura_max_m:n?"),
  Constante do enum RECURSO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

17     COMPANHIA("companhia_aerea", "id", "pais_id:i codigo_icao:s3 codigo_iata:s2? nome:s150 ativo:b"),
  Constante do enum COMPANHIA. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

18     PESSOA("pessoa", "id", "nacionalidade_id:i? nome:s160 nascimento:d ativo:b"),
  Constante do enum PESSOA. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

19     DOCUMENTO("documento_pessoa", "id", "pessoa_id:i pais_emissor_id:i tipo:CPF,RG,PASSAPORTE,OUTRO
numero:s40 validade:d?"),
  Constante do enum DOCUMENTO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

20     CONTATO("contato_pessoa", "id", "pessoa_id:i tipo:EMAIL,TELEFONE valor:s160 principal:b"),
  Constante do enum CONTATO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

21     CARGO("carga", "id", "nome:s80 area:ADMINISTRACAO,ATENDIMENTO,TRIPULACAO,MANUTENCAO,OPERACAO,SEGURANCA
ativo:b"),
  Constante do enum CARGO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

22     FUNCIONARIO("funcionario", "pessoa_id", "pessoa_id:i cargo_id:i aeroporto_base_id:i companhia_id:i?
matricula:s30 admissao:d desligamento:d?"),
  Constante do enum FUNCIONARIO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.

```

```

23     MODELO("modelo_aeronave", "id", "fabricante:s100 nome:s80 envergadura_m:n comprimento_pista_min_m:n
ativo:b"),
    Constante do enum MODELO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor
do enum.

24     AERONAVE("aeronave", "id", "modelo_id:i companhia_id:i matricula:s15 situacao:ATIVA,INATIVA,MANUTENCAO
fabricacao_ano:a"),
    Constante do enum AERONAVE. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo
construtor do enum.

25     ASSENTO("assento_aeronave", "id", "aeronave_id:i codigo:s6 fila:a coluna:s2
classe:ECONOMICA,EXECUTIVA,PRIMEIRA saida_emergencia:b ativo:b"),
    Constante do enum ASSENTO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo
construtor do enum.

26     ROTA("rota", "id", "origem_id:i destino_id:i distancia_km:n ativa:b");
    Constante do enum ROTA. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do
enum.

27
28     private final String tabela;
    Campo(s) tabela: private restringe acesso à classe. Tipo String: texto imutável. final impede reatribuir esse campo depois da
inicialização.

29     private final String chave;
    Campo(s) chave: private restringe acesso à classe. Tipo String: texto imutável. final impede reatribuir esse campo depois da
inicialização.

30     private final List<Campo> campos;
    Campo(s) campos: private restringe acesso à classe. Tipo List<Campo>: guarda referência/valor do tipo declarado, disponível aos
métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

31

```

## TipoCadastro - L32 a L39

Divide a definição por espaço e cada campo por dois-pontos. O ? indica opcionalidade, removido do código de tipo. Produz List<Campo>.

Entradas declaradas: (String tabela, String chave, String definicao).

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Arrays.stream(definicao.split(" ")).map(item -> { String[] partes = item.split(":", 2); return new Campo(partes[0], partes[1].replace("?", ""), partes[1].endsWith("?")); }).toList, Arrays.stream(definicao.split(" ")).map, Arrays.stream, definicao.split, item.split, new Campo, partes[1].replace, partes[1].endsWith.

```

32     TipoCadastro(String tabela, String chave, String definicao) {
    Início da declaração de TipoCadastro. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
é invocado.

33         this.tabela = tabela;
    Guarda no campo tabela da instância atual o valor de tabela. O this diferencia o campo de parâmetros com o mesmo nome.

34         this.chave = chave;
    Guarda no campo chave da instância atual o valor de chave. O this diferencia o campo de parâmetros com o mesmo nome.

35         campos = Arrays.stream(definicao.split(" ")).map(item -> {
    Atualiza campos usando = e o resultado de Arrays.stream(definicao.split(" ")).map(item -> { String[] partes = item.split(":", 2);
return new Campo(partes[0], partes[1].replace("?", "").... Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

36             String[] partes = item.split(":", 2);
    Declara variável local partes do tipo String[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

37             return new Campo(partes[0], partes[1].replace("?", ""), partes[1].endsWith("?"));
    Retorno: entrega new Campo(partes[0], partes[1].replace("?", ""), partes[1].endsWith("?")) ao chamador desta execução;
nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

    new constrói Campo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

38         }).toList();
    Fecha o bloco anterior e completa/continua a expressão ou abre o próximo bloco indicado nesta linha.

39     }
40

```

## tabela - L41 a L41

Devolve o nome de tabela definido pela constante, sem aceitar nome de tabela digitado livremente.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: CadastroJdbc.buscar (L32); CadastroJdbc.listar (L28); CadastroJdbc.salvar (L12); CadastroServico.salvar (L30).

```
41 public String tabela() { return tabela; }
```

Início da declaração de tabela. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega tabela ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## chave - L42 a L42

Devolve a coluna chave do catálogo; FUNCIONARIO usa pessoa\_id em vez de id.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: CadastroJdbc.buscar (L32); CadastroJdbc.listar (L28); CadastroJdbc.salvar (L12); MenuOperacional.cadastros (L50).

```
42 public String chave() { return chave; }
```

Início da declaração de chave. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega chave ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## campos - L43 a L43

Devolve os campos tipados definidos no catálogo, usados pelo menu e pelo JDBC para montar entradas seguras.

Entradas declaradas: ().

Saída declarada: List<Campo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: CadastroJdbc.salvar (L12); CadastroServico.salvar (L30); IntegracaoTeste.cad (L510); MenuOperacional.cadastros (L50).

```
43 public List<Campo> campos() { return campos; }
```

Início da declaração de campos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega campos ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
44
```

```
45 public record Campo(String nome, String tipo, boolean opcional) {
```

Declara Campo como portador de dados com membros gerados.

## converter - L46 a L65

Para campo opcional vazio retorna null. i é id positivo; a é inteiro 1..32767; n é decimal positivo; d é data; b aceita true/false; sN é texto com tamanho máximo N; outros tipos são opções separadas por vírgula. Erros de conversão são traduzidos.

Entradas declaradas: (String entrada).

Saída declarada: Object - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: entrada.isBlank, Validacao.texto, tipo.equals, Long.parseLong, Dados.exigir, Integer.parseInt, Dados.valor, new BigDecimal, LocalDate.parse, texto.equalsIgnoreCase, Boolean.parseBoolean, tipo.startsWith, tipo.substring, texto.toUpperCase, Arrays.asList(tipo.split(",")).contains, Arrays.asList, tipo.split, new RegraNegocioException.

```
46 public Object converter(String entrada) {
```

Início da declaração de converter. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
47         if ((entrada == null || entrada.isBlank()) && opcional) return null;
```

Decisão específica: ((entrada == null || entrada.isBlank()) && opcional). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```
48         String texto = Validacao.texto(entrada, nome, 200);
```

Declara variável local texto do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
49         try {
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

```
50             if (tipo.equals("i")) { long id = Long.parseLong(texto); Dados.exigir(id > 0, "Id deve ser positivo."); return id; }
```

Decisão específica: (tipo.equals("i")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Regra obrigatória: id > 0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Id deve ser positivo.". Essa condição específica não é substituída por outra validação parecida.

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
51             if (tipo.equals("a")) { int valor = Integer.parseInt(texto); Dados.exigir(valor > 0 && valor <= 32767, "Numero fora do intervalo."); return valor; }
```

Decisão específica: (tipo.equals("a")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Declara variável local valor do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Regra obrigatória: valor > 0 && valor <= 32767 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Numero fora do intervalo.". Essa condição específica não é substituída por outra validação parecida.

Retorno: entrega valor ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
52             if (tipo.equals("n")) return Dados.valor(new BigDecimal(texto), true);
```

Decisão específica: (tipo.equals("n")). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega Dados.valor(new BigDecimal(texto), true) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
53             if (tipo.equals("d")) return LocalDate.parse(texto);
```

Decisão específica: (tipo.equals("d")). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega LocalDate.parse(texto) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
54             if (tipo.equals("b")) {
```

Decisão específica: (tipo.equals("b")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
55                 Dados.exigir(texto.equalsIgnoreCase("true") || texto.equalsIgnoreCase("false"), "Use true ou false.");
```

Regra obrigatória: texto.equalsIgnoreCase("true") || texto.equalsIgnoreCase("false") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Use true ou false.". Essa condição específica não é substituída por outra validação parecida.

```
56                 return Boolean.parseBoolean(texto);
```

Retorno: entrega Boolean.parseBoolean(texto) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca Boolean.parseBoolean com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
57             }
```

```
58             if (tipo.startsWith("s")) return Validacao.texto(texto, nome, Integer.parseInt(tipo.substring(1)));
```

Decisão específica: (tipo.startsWith("s")). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega Validacao.texto(texto, nome, Integer.parseInt(tipo.substring(1))) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
59         texto = texto.toUpperCase(java.util.Locale.ROOT);  
        Atualiza texto usando = e o resultado de texto.toUpperCase(java.util.Locale.ROOT). Atribuição modifica a variável/célula indicada;  
        não é um teste de igualdade.  
  
60         Dados.exigir(Arrays.asList(tipo.split(",")).contains(texto), "Valor invalido para " + nome  
        + ": " + tipo);  
        Regra obrigatória: Arrays.asList(tipo.split(",")).contains(texto) deve resultar em true. Se false, Dados.exigir lança  
        RegraNegocioException com "Valor invalido para " + nome + ": " + tipo. Essa condição específica não é substituída por outra  
        validação parecida.  
  
61         return texto;  
        Retorno: entrega texto ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse  
        caminho.  
  
62     } catch (NumberFormatException | java.time.DateTimeException erro) {  
        Trata a(s) exceção(ões) declarada(s) em catch (NumberFormatException | java.time.DateTimeException erro). O tratamento  
        mostrado abaixo decide se avisa, repete ou relança.  
  
63         throw new RegraNegocioException("Formato invalido para " + nome + ".");  
        Lança new RegraNegocioException("Formato invalido para " + nome + "."). Interrompe o caminho normal; a exceção sobe até  
        uma captura compatível e pode provocar rollback.  
  
64     }  
65 }  
66 }  
67 }
```



## Validacao

src/main/java/br/edu/aeroporto dominio/Validacao.java

Funções estáticas de validação de texto, nascimento, login e CPF. Retornam valores normalizados ou interrompem a operação com RegraNegocioException.

44 linhas | 5 métodos/construtores declarados | SHA-256: 2a9e749cb9208f27...

```
1 package br.edu.aeroporto.dominio;
  Declara o pacote deste arquivo: br.edu.aeroporto.dominio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.excecao.RegraNegocioException;
4 import java.time.LocalDate;
5 import java.util.Locale;
6
7 public final class Validacao {
  Declara Validacao como tipo com campos e métodos. final impede herança.
```

### Validacao - L8 a L8

Construtor vazio. Se for private, impede criar instâncias desta classe utilitária a partir de outras classes; os métodos static são usados pelo nome da classe.

Entradas declaradas: ().

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
8 private Validacao() {}
  Início da declaração de Validacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

9
```

### texto - L10 a L18

Exige texto não vazio, usa strip, limita comprimento e rejeita caracteres de controle. Devolve o texto limpo.

Entradas declaradas: (String valor, String campo, int maximo).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: AtendimentoServico.despachar (L88); AtendimentoServico.rastrear (L111); CadastroPassageiro.CadastroPassageiro (L8); ConsultaServico.reserva (L45); DadosFicticiosServico.criar (L23); FinanceiroServico.pagar (L22); FinanceiroServico.solicitarReembolso (L71); Funcionario.Funcionario (L8); OperacaoServico.habilitar (L56); OperacaoServico.interditar (L33); OperacaoServico.manutencao (L94); OperacaoServico.occurencia (L155); OperacaoServico.servicoSolo (L130); Passageiro.Passageiro (L8); PassageiroServico.editar (L55); PassageiroServico.vincular (L44); PedidoTrecho.PedidoTrecho (L6); Pessoa.Pessoa (L12); PlanejamentoServico.horarios (L71); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.trocarAeronave (L93); PrimeiroAcesso.PrimeiroAcesso (L9); ProgramacaoVoo.ProgramacaoVoo (L7); RelatoriosCompletoServico.bagagem (L51); ReservaServico.cancelar (L36); Tarifa.Tarifa (L8); TipoCadastro.converter (L46); Validacao.login (L25); VooServico.transicao (L22).

Chamadas observadas no corpo: valor.isBlank(), new RegraNegocioException(), valor.strip(), resultado.length(), resultado.chars().anyMatch(), resultado.chars(), Character::isISOControl.

```
10 public static String texto(String valor, String campo, int maximo) {
  Início da declaração de texto. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

11     if (valor == null || valor.isBlank()) throw new RegraNegocioException(campo + " é obrigatório.");
  Decisão específica: (valor == null || valor.isBlank()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

  Lança new RegraNegocioException(campo + " é obrigatório."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

  new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

12     String resultado = valor.strip();
  Declara variável local resultado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
```

```
13         if (resultado.length() > maximo) throw new RegraNegocioException(campo + " excede " + maximo + "
    caracteres.");
```

Decisão específica: (resultado.length() > maximo). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException(campo + " excede " + maximo + " caracteres."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
14         if (resultado.chars().anyMatch(Character::isISOControl)) {
```

Decisão específica: (resultado.chars().anyMatch(Character::isISOControl)). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

```
15             throw new RegraNegocioException(campo + " contém caracteres de controle.");
```

Lança new RegraNegocioException(campo + " contém caracteres de controle."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

```
16         }
17         return resultado;
```

Retorno: entrega resultado ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
18     }
19 }
```

## nascimento - L20 a L23

Exige data não nula e não futura em relação ao hoje recebido. A função não decide sozinha qual relógio usar.

Entradas declaradas: (LocalDate data, LocalDate hoje).

Saída declarada: LocalDate - data de calendário, sem hora ou fuso.

Possíveis chamadores identificados: ArquivosServico.importarPassageiros (L24); CadastroServico.validar (L53); InicializacaoServico.criar (L27); PassageiroServico.cadastrar (L34); PassageiroServico.editar (L55); UnidadeTeste.main (L10).

Chamadas observadas no corpo: data.isAfter, new RegraNegocioException.

```
20     public static LocalDate nascimento(LocalDate data, LocalDate hoje) {
```

Início da declaração de nascimento. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
21         if (data == null || data.isAfter(hoje)) throw new RegraNegocioException("Informe uma data de
    nascimento válida, que não esteja no futuro.");
```

Decisão específica: (data == null || data.isAfter(hoje)). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException("Informe uma data de nascimento válida, que não esteja no futuro."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
22         return data;
```

Retorno: entrega data ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
23     }
24 }
```

## login - L25 a L27

Reutiliza texto com limite 60 e converte para minúsculas com Locale.ROOT.

Entradas declaradas: (String valor).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: AdministracaoServico.criarUsuario (L32); AutenticacaoServico.entrar (L27); PrimeiroAcesso.PrimeiroAcesso (L9).

Chamadas observadas no corpo: texto(valor, "Login", 60).toLowerCase, texto.

```
25     public static String login(String valor) {
```

Início da declaração de login. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
26         return texto(valor, "Login", 60).toLowerCase(Locale.ROOT);
```

Retorno: entrega texto(valor, "Login", 60).toLowerCase(Locale.ROOT) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca texto(valor, "Login", 60).toLowerCase com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

Chama Validacao.texto: Exige texto não vazio, usa strip, limita comprimento e rejeita caracteres de controle. Devolve o texto limpo.

```
27     }
```

```
28
```

## cpf - L29 a L43

Vazio é opcional e retorna "". Remove pontuação, exige 11 dígitos não todos iguais; calcula os dois dígitos verificadores com pesos decrescentes e módulo 11.

Entradas declaradas: (String valor).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: CadastroPassageiro.CadastroPassageiro (L8); CadastroServico.validar (L53); UnidadeTeste.main (L10).

Chamadas observadas no corpo: valor.isBlank(), valor.replaceAll(), numero.matches(), numero.chars().distinct().count(), numero.chars().distinct(), numero.chars(), new RegraNegocioException(), numero.charAt().

```
29     public static String cpf(String valor) {
```

Início da declaração de cpf. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
30         if (valor == null || valor.isBlank()) return "";
```

Decisão específica: (valor == null || valor.isBlank()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega "" ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
31         String numero = valor.replaceAll("[.\\-\\s]", "");
```

Declara variável local numero do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
32         if (!numero.matches("[0-9]{11}") || numero.chars().distinct().count() == 1) {
```

Decisão específica: (!numero.matches("[0-9]{11}") || numero.chars().distinct().count() == 1). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

```
33             throw new RegraNegocioException("CPF inválido.");
```

Lança new RegraNegocioException("CPF inválido."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

```
34         }
```

```
35         for (int tamanho = 9; tamanho <= 10; tamanho++) {
```

Laço controlado por inicialização, condição e atualização: for (int tamanho = 9; tamanho <= 10; tamanho++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local tamanho do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
36             int soma = 0;
```

Declara variável local soma do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
37             for (int i = 0; i < tamanho; i++) soma += (numero.charAt(i) - '0') * (tamanho + 1 - i);
```

Laço controlado por inicialização, condição e atualização: for (int i = 0; i < tamanho; i++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Atualiza soma usando += e o resultado de (numero.charAt(i) - '0') \* (tamanho + 1 - i). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
38             int digito = 11 - soma % 11;
```

Declara variável local digito do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
39             if (digito >= 10) digito = 0;
```

Decisão específica: (digito >= 10). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Atualiza digito usando = e o resultado de 0. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
40         if (digito != numero.charAt(tamanho) - '0') throw new RegraNegocioException("CPF inválido.");
```

Decisão específica: (digito != numero.charAt(tamanho) - '0'). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException("CPF inválido."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
41     }
```

```
42     return numero;
```

Retorno: entrega numero ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
43 }
```

```
44 }
```

# Java: DTOs de consulta

## AssentoResumo

src/main/java/br/edu/aeroporto/dto/AssentoResumo.java

DTO de consulta com código, classe, bloqueio e ocupação. livre exige simultaneamente ausência de bloqueio e ausência de ocupação.

5 linhas | 1 métodos/construtores declarados | SHA-256: 9376e27ecb7b9635...

```
1 package br.edu.aeroporto.dto;
  Declara o pacote deste arquivo: br.edu.aeroporto.dto. O nome completo distingue a classe de outras com o mesmo nome.
2
3 public record AssentoResumo(String codigo, String classe, boolean bloqueado, boolean ocupado) {
  Declara AssentoResumo como portador de dados com membros gerados.
```

### livre - L4 a L4

Devolve true apenas quando bloqueado=false E ocupado=false.

Entradas declaradas: ().

Saída declarada: boolean - condição verdadeira ou falsa.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
4     public boolean livre() { return !bloqueado && !ocupado; }
  Início da declaração de livre. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
  invocado.

  Retorno: entrega !bloqueado && !ocupado ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo
  executa por esse caminho.
5 }
```

## ReservaResumo

src/main/java/br/edu/aeroporto/dto/ReservaResumo.java

DTO do cabeçalho e dos trechos consultados. Sua soma é o total original das passagens, inclusive itens cancelados; não é o saldo financeiro atual.

18 linhas | 2 métodos/construtores declarados | SHA-256: 1213a7b76236b1a6...

```
1 package br.edu.aeroporto.dto;
  Declara o pacote deste arquivo: br.edu.aeroporto.dto. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.SituacaoReserva;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Enum dos estados do cabeçalho de reserva. A situação resulta dos itens, recalculada por ReservaJdbc; não se confunde com a situação de voo.

4 import java.math.BigDecimal;
5 import java.time.OffsetDateTime;
6 import java.util.List;
7
8 public record ReservaResumo(long id, String localizador, SituacaoReserva situacao, OffsetDateTime
  expiraEm,
  Declara ReservaResumo como portador de dados com membros gerados.

9                               List<Trecho> trechos) {
  Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.
```

### ReservaResumo - L10 a L10

Copia a lista de trechos com List.copyOf. Mantém os registros para consulta sem permitir adicionar/remover diretamente nessa lista; não calcula saldo financeiro no construtor.

Construtor sem lista explícita de parâmetros. Em record compacto, recebe os componentes declarados no cabeçalho; nas demais classes, segue a declaração apresentada.

Possíveis chamadores identificados: ConsultasJdbc.reserva (L52).

Chamadas observadas no corpo: List.copyOf.

```
10 public ReservaResumo { trechos = List.copyOf(trechos); }
  Início da declaração de ReservaResumo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

  Atualiza trechos usando = e o resultado de List.copyOf(trechos). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

11
12 public record Trecho(long id, String passageiro, String voo, int ordem,
  Declara Trecho como portador de dados com membros gerados.

13                               String situacao, String assento, BigDecimal valor) {}
  Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.
```

14

## totalOriginalPassagens - L15 a L17

Converte cada trecho em seu valor e reduz por soma a partir de zero. Inclui todos os trechos da consulta, inclusive cancelados.

Entradas declaradas: ().

Saída declarada: BigDecimal - valor decimal, utilizado nos cálculos financeiros.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: trechos.stream().map(Trecho::valor).reduce, trechos.stream().map, trechos.stream, Trecho::valor, BigDecimal::add.

```
15    public BigDecimal totalOriginalPassagens() {
```

Início da declaração de totalOriginalPassagens. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
16        return trechos.stream().map(Trecho::valor).reduce(BigDecimal.ZERO, BigDecimal::add);
```

Retorno: entrega trechos.stream().map(Trecho::valor).reduce(BigDecimal.ZERO, BigDecimal::add) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca trechos.stream().map(Trecho::valor).reduce com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

Chama trechos.stream().map: Transforma elementos no stream ou valor no Optional pela função recebida.

Chama trechos.stream: Cria o fluxo de processamento dos elementos; operações terminais produzem o resultado.

```
17    }
```

```
18 }
```

## TipoRelatorio

src/main/java/br/edu/aeroporto/dto/TipoRelatorio.java

Enum com SQL fixo para 16 relatórios e um indicador de acesso restrito. Não é uma entidade do banco; funciona como catálogo de consultas.

52 linhas | 3 métodos/construtores declarados | SHA-256: f4528e8d3fc635e4...

```

1 package br.edu.aeroporto.dto;
  Declara o pacote deste arquivo: br.edu.aeroporto.dto. O nome completo distingue a classe de outras com o mesmo nome.

2

3 public enum TipoRelatorio {
  Declara TipoRelatorio como conjunto fechado de constantes.

4     OCUPACAO(""""
  Constante do enum OCUPACAO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo
  construtor do enum.

5         SELECT v.id,v.numero,v.situacao,COUNT(i.id) AS capacidade,
  Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
  chamado, não pelo compilador Java.

6         COUNT(i.id) FILTER (WHERE NOT i.bloqueado) AS vendaveis,
  Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
  determina onde a instrução termina.

7         COUNT(o.id) AS ocupados,ROUND(100.0*COUNT(o.id)/NULLIF(COUNT(i.id) FILTER (WHERE NOT
  i.bloqueado),0),2) AS ocupacao_percentual
  Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
  determina onde a instrução termina.

8         FROM voo v LEFT JOIN inventario_assento_voo i ON i.voo_id=v.id
  Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
  chamado, não pelo compilador Java.

9         LEFT JOIN ocupacao_assento o ON o.inventario_id=i.id AND o.liberada_em IS NULL GROUP BY v.id ORDER
  BY v.id
  Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
  chamado, não pelo compilador Java.

10        """,false),
  Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu
  uso.

11        ATRASOS(""""
  Constante do enum ATRASOS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo
  construtor do enum.

12        SELECT v.id,v.numero,v.situacao,o.codigo_icao AS origem,d.codigo_icao AS destino,
  Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
  chamado, não pelo compilador Java.

13        v.partida_prevista,v.chegada_prevista,v.partida_estimada,v.chegada_estimada,v.partida_real,v.chegada_real,
  Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
  determina onde a instrução termina.

14        EXTRACT(EPOCH FROM
  (COALESCE(v.partida_real,v.partida_estimada,v.partida_prevista)-v.partida_prevista))/60 AS
  atraso_partida_min,
  Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
  determina onde a instrução termina.

15        EXTRACT(EPOCH FROM
  (COALESCE(v.chegada_real,v.chegada_estimada,v.chegada_prevista)-v.chegada_prevista))/60 AS
  atraso_chegada_min
  Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
  determina onde a instrução termina.

16        FROM voo v JOIN rota r ON r.id=v.rota_id JOIN aeroporto o ON o.id=r.origem_id JOIN aeroporto d ON
  d.id=r.destino_id ORDER BY v.id
  Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
  chamado, não pelo compilador Java.

```



|    |                                                                                                                                        |                                                                                                                                                                        |
|----|----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 17 | """,false),                                                                                                                            | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                         |
| 18 | MANIFESTO(""                                                                                                                           | Constante do enum MANIFESTO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.                                   |
| 19 | SELECT i.id,i.reserva_id,i.voo_id,p.nome,i.ordem_trecho,i.situacao,a.codigo AS assento,b.numero AS bilhete,                            | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.              |
| 20 | ci.realizado_em AS checkin,e.realizado_em AS embarque                                                                                  | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina. |
| 21 | FROM item_reserva i JOIN pessoa p ON p.id=i.passageiro_id                                                                              | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.              |
| 22 | LEFT JOIN ocupacao_assento o ON o.item_id=i.id AND o.liberada_em IS NULL                                                               | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.              |
| 23 | LEFT JOIN inventario_assento_voo a ON a.id=o.inventario_id LEFT JOIN bilhete b ON b.item_id=i.id                                       | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.              |
| 24 | LEFT JOIN check_in ci ON ci.item_id=i.id AND ci.cancelado_em IS NULL LEFT JOIN embarque e ON e.checkin_id=ci.id ORDER BY i.voo_id,i.id | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.              |
| 25 | """,true),                                                                                                                             | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                         |
| 26 | VENDAS(""                                                                                                                              | Constante do enum VENDAS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.                                      |
| 27 | SELECT r.id,r.localizador,r.situacao,r.criada_em,r.expira_em,COUNT(i.id) AS itens,                                                     | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.              |
| 28 | COALESCE(SUM(i.valor_base+i.taxa_embarque-i.desconto),0) AS total_original,                                                            | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina. |
| 29 | COALESCE(SUM(i.valor_base+i.taxa_embarque-i.desconto) FILTER (WHERE i.situacao NOT IN ('CANCELADO','EXPIRADO'))),0) AS total_vigente,  | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina. |
| 30 | COALESCE((SELECT SUM(p.valor) FROM pagamento p WHERE p.reserva_id=r.id AND p.situacao='APROVADO'),0) AS pago,                          | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina. |
| 31 | COALESCE((SELECT SUM(e.valor) FROM reembolso e WHERE e.reserva_id=r.id AND e.situacao='PROCESSADO'),0) AS devolvido                    | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina. |
| 32 | FROM reserva r LEFT JOIN item_reserva i ON i.reserva_id=r.id GROUP BY r.id ORDER BY r.id                                               | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.              |
| 33 | """,true),                                                                                                                             | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                         |
| 34 | PAGAMENTOS("SELECT id,reserva_id,forma,valor,situacao,criado_em,processado_em FROM pagamento ORDER BY id",true),                       | Constante do enum PAGAMENTOS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.                                  |

- 35 REEMBOLSOS("SELECT id,pagamento\_id,reserva\_id,valor,situacao,solicitado\_em,processado\_em FROM reembolso ORDER BY id",true),  
Constante do enum REEMBOLSOS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 36 RECURSOS("SELECT a.id,r.aeroporto\_id,r.tipo,r.codigo,a.voo\_id,a.finalidade,a.inicio,a.fim,a.ativa FROM alocao\_recurso a JOIN recurso\_aeroporto r ON r.id=a.recurso\_id ORDER BY a.inicio,a.id",false),  
Constante do enum RECURSOS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 37 AGENDAS("SELECT id,aeronave\_id,voo\_id,manutencao\_id,inicio,fim,ativa FROM agenda\_aeronave ORDER BY inicio,id",false),  
Constante do enum AGENDAS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 38 ESCALAS("SELECT e.id,e.funcionario\_id,p.nome,e.aeroporto\_id,e.voo\_id,e.funcao,e.inicio,e.fim,e.ativa FROM escala\_funcionario e JOIN pessoa p ON p.id=e.funcionario\_id ORDER BY e.inicio,e.id",false),  
Constante do enum ESCALAS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 39 HISTORICO("SELECT id,voo\_id,usuario\_id,alterado\_em,situacao\_anterior,situacao\_nova,motivo FROM historico\_voo ORDER BY id",false),  
Constante do enum HISTORICO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 40 AUDITORIA("SELECT id,usuario\_id,ocorrido\_em,acao,entidade,registro\_id FROM evento\_auditoria ORDER BY id",true),  
Constante do enum AUDITORIA. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 41 BAGAGENS("SELECT b.id,b.etiqueta,ci.voo\_id,ci.item\_id,b.peso\_kg,b.categoria,b.taxa\_excesso,b.situacao FROM bagagem b JOIN check\_in ci ON ci.id=b.checkin\_id ORDER BY b.id",true),  
Constante do enum BAGAGENS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 42 MANUTENCOES("SELECT id,aeronave\_id,responsavel\_id,tipo,situacao,inicio\_real,fim\_real,custo FROM manutencao\_aeronave ORDER BY id",false),  
Constante do enum MANUTENCOES. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 43 SOLO("SELECT id,voo\_id,aeroporto\_id,responsavel\_id,tipo,situacao,inicio,fim,quantidade,unidade,custo FROM servico\_solo ORDER BY id",false),  
Constante do enum SOLO. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 44 OCORRENCIAS("SELECT id,aeroporto\_id,voo\_id,tipo,gravidade,aberta\_em,encerrada\_em FROM ocorrencia\_operacional ORDER BY id",false),  
Constante do enum OCORRENCIAS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 45 TARIFAS("SELECT \* FROM tarifa\_voo ORDER BY voo\_id,id",false);  
Constante do enum TARIFAS. Os argumentos fornecem os metadados deste cadastro/relatório/estado, recebidos pelo construtor do enum.
- 46  
47 private final String sql;  
Campo(s) sql: private restringe acesso à classe. Tipo String: texto imutável. final impede reatribuir esse campo depois da inicialização.

```
48     private final boolean restrito;
```

Campo(s) restrito: private restringe acesso à classe. Tipo boolean: condição verdadeira ou falsa. final impede reatribuir esse campo depois da inicialização.

## TipoRelatorio - L49 a L49

Guarda o SQL fixo e a indicação de relatório restrito recebidos pela constante do enum.

Entradas declaradas: (String sql,boolean restrito).

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
49     TipoRelatorio(String sql,boolean restrito) { this.sql=sql;this.restrito=restrito; }
```

Início da declaração de TipoRelatorio. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Guarda no campo sql da instância atual o valor de sql. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo restrito da instância atual o valor de restrito. O this diferencia o campo de parâmetros com o mesmo nome.

## sql - L50 a L50

Devolve a consulta fixa do relatório selecionado.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: RelatoriosCompletoServico.consultar (L29); RelatoriosCompletoServico.exportar (L68).

```
50     public String sql() { return sql; }
```

Início da declaração de sql. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega sql ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## restrito - L51 a L51

Devolve o indicador usado na autorização dos relatórios; AUDITORIA recebe tratamento específico de administrador no serviço.

Entradas declaradas: ().

Saída declarada: boolean - condição verdadeira ou falsa.

Possíveis chamadores identificados: RelatoriosCompletoServico.permitir (L23).

```
51     public boolean restrito() { return restrito; }
```

Início da declaração de restrito. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega restrito ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
52 }
```

## VooResumo

src/main/java/br/edu/aeroporto/dto/VooResumo.java

DTO de consulta: companhia, trecho, fusos e horários. atrasado compara o horário real ou estimado com o previsto.

15 linhas | 1 métodos/construtores declarados | SHA-256: 109bd7f06e4d4332...

```

1 package br.edu.aeroporto.dto;
    Declara o pacote deste arquivo: br.edu.aeroporto.dto. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.SituacaoVoo;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Enum dos estados Java e das transições possíveis. O enum estabelece a sequência; VooServico verifica janelas, equipe e recursos antes de efetivar a mudança.

4 import java.time.OffsetDateTime;
5 import java.time.ZoneId;
6
7 public record VooResumo(long id, String companhia, String numero, String origem, String destino,
    Declara VooResumo como portador de dados com membros gerados.

8         ZoneId fusoOrigem, ZoneId fusoDestino, OffsetDateTime partida,
    Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

9         OffsetDateTime chegada, OffsetDateTime partidaEstimada,
    Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

10        OffsetDateTime partidaReal, SituacaoVoo situacao) {
    Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.
```

### atrasado - L11 a L14

Prioriza partidaReal; se não existir usa partidaEstimada. Se ambos faltarem, não considera atraso. Compara com a prevista usando isAfter.

Entradas declaradas: ().

Saída declarada: boolean - condição verdadeira ou falsa.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: referencia.isAfter.

```

11 public boolean atrasado() {
    Início da declaração de atrasado. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

12     OffsetDateTime referencia = partidaReal != null ? partidaReal : partidaEstimada;
    Declara variável local referencia do tipo OffsetDateTime. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    Escolha de valor: partidaReal != null. true escolhe partidaReal; false escolhe partidaEstimada. Só a alternativa escolhida é avaliada.

13     return referencia != null && referencia.isAfter(partida);
    Retorno: entrega referencia != null && referencia.isAfter(partida) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

14 }
15 }
```

# Java: contratos de repositório

## ConsultasRepositorio

src/main/java/br/edu/aeroporto/repositorio/ConsultasRepositorio.java

Contrato para consultas de voos, assentos e reserva. Usa DTOs e Optional para ausência de uma reserva.

16 linhas | 3 métodos/construtores declarados | SHA-256: c0d1689b61bf55fa...

```
1 package br.edu.aeroporto.repositorio;
```

Declara o pacote deste arquivo: br.edu.aeroporto.repositorio. O nome completo distingue a classe de outras com o mesmo nome.

```
2
```

```
3 import br.edu.aeroporto.dto.AssentoResumo;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. DTO de consulta com código, classe, bloqueio e ocupação. livre exige simultaneamente ausência de bloqueio e ausência de ocupação.

```
4 import br.edu.aeroporto.dto.ReservaResumo;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. DTO do cabeçalho e dos trechos consultados. Sua soma é o total original das passagens, inclusive itens cancelados; não é o saldo financeiro atual.

```
5 import br.edu.aeroporto.dto.VooResumo;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. DTO de consulta: companhia, trecho, fusos e horários. atrasado compara o horário real ou estimado com o previsto.

```
6 import java.sql.Connection;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. conexão JDBC fornecida à operação de banco

```
7 import java.sql.SQLException;
```

```
8 import java.time.Instant;
```

```
9 import java.util.List;
```

```
10 import java.util.Optional;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
11
```

```
12 public interface ConsultasRepositorio {  
    Declara ConsultasRepositorio como contrato de operações.
```

### voos - L13 a L13

Contrato de consulta por intervalo de instantes e página. A implementação ConsultasJdbc mapeia linhas em VooResumo.

Entradas declaradas: (Connection conexao, Instant inicio, Instant fim, int pagina).

Saída declarada: List<VooResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: ConsultaServico.voos (L28).

```
13 List<VooResumo> voos(Connection conexao, Instant inicio, Instant fim, int pagina) throws SQLException;  
    Início da declaração de voos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é  
    invocado.
```

### assentos - L14 a L14

Contrato de consulta do inventário por id de voo, devolvendo DTOs de bloqueio/ocupação.

Entradas declaradas: (Connection conexao, long vooId).

Saída declarada: List<AssentoResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: ConsultaServico.assentos (L37).

```
14 List<AssentoResumo> assentos(Connection conexao, long vooId) throws SQLException;  
    Início da declaração de assentos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é  
    invocado.
```

### reserva - L15 a L15

Contrato de busca por localizador; ausência é Optional vazio, não um DTO de reserva fictícia.

Entradas declaradas: (Connection conexao, String localizador).

Saída declarada: Optional<ReservaResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: ConsultaServico.reserva (L45).

```
15 Optional<ReservaResumo> reserva(Connection conexao, String localizador) throws SQLException;  
    Início da declaração de reserva. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é  
    invocado.
```

```
16 }
```

## PassageiroRepositorio

src/main/java/br/edu/aeroporto/repositorio/PassageiroRepositorio.java

Contrato para cadastrar, vincular, editar e buscar passageiros. Seus dois buscar são sobrecargas com parâmetros e retornos diferentes.

17 linhas | 5 métodos/construtores declarados | SHA-256: 6456b29d1ec60d8e...

```

1 package br.edu.aeroporto.repositorio;
  Declara o pacote deste arquivo: br.edu.aeroporto.repositorio. O nome completo distingue a classe de outras com o mesmo nome.

2

3 import br.edu.aeroporto.dominio.CadastroPassageiro;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Dados de entrada do cadastro. Normaliza nome, CPF, e-mail e telefone antes de chegar ao serviço; a data de nascimento é validada pelo serviço contra o relógio.

4 import br.edu.aeroporto.dominio.Passageiro;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Papel de uma Pessoa que pode viajar. Acrescenta o código de cliente e implementa identificacao; não representa sozinho reservas ou bilhetes.

5 import java.sql.Connection;
6 import java.sql.SQLException;
7 import java.util.List;
8 import java.util.Optional;
9

10 public interface PassageiroRepositorio {
  Declara PassageiroRepositorio como contrato de operações.

```

### vincular - L11 a L11

Contrato que cria o papel de passageiro para uma pessoa identificada, usando a conexão recebida.

Entradas declaradas: (java.sql.Connection conexao, long pessoald, String assistencia).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: PassageiroServico.vincular (L44).

```

11 Passageiro vincular(java.sql.Connection conexao, long pessoaId, String assistencia) throws
   java.sql.SQLException;
  Início da declaração de vincular. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

### editar - L12 a L13

Contrato de atualização dos dados/papel do passageiro e retorno do objeto resultante.

Entradas declaradas: (java.sql.Connection conexao, long id, String nome, java.time.LocalDate nascimento, Long nacionalidade, String assistencia, boolean ativo).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: PassageiroServico.editar (L55).

```

12 Passageiro editar(java.sql.Connection conexao, long id, String nome, java.time.LocalDate nascimento,
   Long nacionalidade, String assistencia, boolean ativo) throws
   java.sql.SQLException;
  Início da declaração de editar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

```
13      Long nacionalidade, String assistencia, boolean ativo) throws java.sql.SQLException;
      Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.
```

### cadastrear - L14 a L14

Contrato de criação a partir de CadastroPassageiro, devolvendo a identidade persistida.

Entradas declaradas: (Connection conexao, CadastroPassageiro cadastro).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: PassageiroServico.cadastrar (L34).

```
14      Passageiro cadastrar(Connection conexao, CadastroPassageiro cadastro) throws SQLException;
      Início da declaração de cadastrar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
```

### buscar - L15 a L15

Há duas sobrecargas: id devolve Optional<Passageiro>; filtro de nome com limite/deslocamento devolve List<Passageiro>. O tipo dos argumentos escolhe qual é chamada.

Entradas declaradas: (Connection conexao, long id).

Saída declarada: Optional<Passageiro> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: PassageiroServico.buscar (L71); PassageiroServico.buscar (L80).

```
15      Optional<Passageiro> buscar(Connection conexao, long id) throws SQLException;
      Início da declaração de buscar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
```

### buscar - L16 a L16

Há duas sobrecargas: id devolve Optional<Passageiro>; filtro de nome com limite/deslocamento devolve List<Passageiro>. O tipo dos argumentos escolhe qual é chamada.

Entradas declaradas: (Connection conexao, String nome, int limite, int deslocamento).

Saída declarada: List<Passageiro> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: PassageiroServico.buscar (L71); PassageiroServico.buscar (L80).

```
16      List<Passageiro> buscar(Connection conexao, String nome, int limite, int deslocamento) throws
      SQLException;
      Início da declaração de buscar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
```

```
17 }
```



# UsuarioRepositorio

src/main/java/br/edu/aeroporto/repositorio/UsuarioRepositorio.java

Contrato para consultar credenciais ativas e detectar existência de usuários. Credenciais agrupa Sessao e hash; toString oculta esse hash.

16 linhas | 3 métodos/construtores declarados | SHA-256: 8071b9287da7e469...

```

1 package br.edu.aeroporto.repositorio;
  Declara o pacote deste arquivo: br.edu.aeroporto.repositorio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.Sessao;
4 import java.sql.Connection;
5 import java.sql.SQLException;
6 import java.util.Optional;
7
8 public interface UsuarioRepositorio {
  Declara UsuarioRepositorio como contrato de operações.

9     record Credenciais(Sessao sessao, String senhaHash) {
  Declara Credenciais como portador de dados com membros gerados.

```

## toString - L10 a L11

O record interno Credenciais mostra somente usuariold. Não inclui senhaHash na representação textual, evitando expor o hash em logs/impressões.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: sessao.usuariold.

```

10     @Override
  Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a
  anotação mostrada.

11     public String toString() { return "Credenciais[usuarioId=" + sessao.usuarioId() + "]; }
  Retorno: entrega "Credenciais[usuarioId=" + sessao.usuarioId() + "]" ao chamador desta execução; nenhuma instrução posterior
  desse mesmo corpo executa por esse caminho.

```

```
12     }
13
```

### buscarAtivo - L14 a L14

Contrato de busca por login ativo: devolve Optional<Credenciais>, unindo Sessao e hash para a autenticação. UsuarioJdbc fornece o SQL e o mapeamento.

Entradas declaradas: (Connection conexao, String login).

Saída declarada: Optional<Credenciais> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: AutenticacaoServico.entrar (L27).

```
14     Optional<Credenciais> buscarAtivo(Connection conexao, String login) throws SQLException;
    Início da declaração de buscarAtivo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.
```

### existeUsuario - L15 a L15

Contrato de consulta boolean que determina se já existe alguma conta; UsuarioJdbc implementa a consulta.

Entradas declaradas: (Connection conexao).

Saída declarada: boolean - condição verdadeira ou falsa.

Possíveis chamadores identificados: AutenticacaoServico.existeUsuario (L23).

```
15     boolean existeUsuario(Connection conexao) throws SQLException;
    Início da declaração de existeUsuario. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
    é invocado.
```

```
16 }
```

# Java: infraestrutura JDBC

## AuditoriaJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/AuditoriaJdbc.java

Inserir eventos e definir variáveis locais de transação com autor/motivo usados pelo histórico de voo no PostgreSQL.

19 linhas | 2 métodos/construtores declarados | SHA-256: 67f62f8933c5e5d5...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.sql.Connection;
4 import java.sql.SQLException;
5
6 public final class AuditoriaJdbc {
    Declara AuditoriaJdbc como tipo com campos e métodos. final impede herança.
```

### registrar - L7 a L13

Grava usuário, ação, entidade e id na conexão recebida. O evento é confirmado ou revertido junto da operação.

Entradas declaradas: (Connection conexao, long usuarioId, String acao, String entidade, long registroId).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AdministracaoServico.acesso (L47); AdministracaoServico.alterarSenha (L67); AdministracaoServico.criarUsuario (L32); ArquivosServico.copiarRelatorio (L60); ArquivosServico.importarPassageiros (L24); AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AtendimentoServico.embarcar (L132); AtendimentoServico.rastrear (L111); AutenticacaoServico.entrar (L27); CadastroServico.salvar (L30); FinanceiroServico.cancelarPagamento (L55); FinanceiroServico.confirmar (L63); FinanceiroServico.pagar (L22); FinanceiroServico.processar (L47); FinanceiroServico.processarReembolso (L89); FinanceiroServico.solicitarReembolso (L71); InicializacaoServico.criar (L27); OperacaoServico.alocar (L24); OperacaoServico.encerrarOcorrencia (L166); OperacaoServico.escalar (L70); OperacaoServico.habilitar (L56); OperacaoServico.interditar (L33); OperacaoServico.liberarAlocacao (L43); OperacaoServico.liberarEscala (L81); OperacaoServico.manutencao (L94); OperacaoServico.ocorrencia (L155); OperacaoServico.servicoSolo (L130); OperacaoServico.situacaoManutencao (L107); OperacaoServico.situacaoSolo (L144); PassageiroServico.cadastrar (L34); PassageiroServico.editar (L55); PassageiroServico.vincular (L44); PlanejamentoServico.bloquearAssento (L40); PlanejamentoServico.horarios (L71); PlanejamentoServico.programar (L20); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.tarifa (L30); PlanejamentoServico.trocarAeronave (L93); RelatoriosCompletoServico.exportar (L68); ReservaServico.cancelar (L36); ReservaServico.reservar (L26); VooServico.transicao (L22).

Chamadas observadas no corpo: Sql.executar.

```
7 public void registrar(Connection conexao, long usuarioId, String acao, String entidade, long
  registroId)
    Início da declaração de registrar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

8 throws SQLException {
    Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

9 Sql.executar(conexao, ""
    Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: evento_auditoria. Parâmetros ligados em ordem: usuarioId, acao, entidade, registroId.

10 INSERT INTO evento_auditoria(usuario_id, acao, entidade, registro_id)
    Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

11 VALUES (?, ?, ?, ?)
    Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

12 "", usuarioId, acao, entidade, registroId);
    Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

13 }
14
```

## definirAutorDoVoo - L15 a L18

set\_config(...,true) define usuário/motivo apenas para a transação corrente; o trigger do banco usa esses valores no histórico.

Entradas declaradas: (Connection conexao, long usuariold, String motivo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L23); PlanejamentoServico.horarios (L71); PlanejamentoServico.programar (L20); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.trocarAeronave (L93); VooServico.transicao (L22).

Chamadas observadas no corpo: Sql.unico, linha.getString, Long.toString.

```
15      public void definirAutorDoVoo(Connection conexao, long usuarioId, String motivo) throws SQLException {
      Início da declaração de definirAutorDoVoo. Os parâmetros entram pela chamada; o corpo só executa quando esse
      método/construtor é invocado.

16          Sql.unico(conexao, "SELECT set_config('aeroporto.usuario_id', ?, true)", linha ->
      linha.getString(1), Long.toString(usuarioId));
      Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Parâmetros ligados
      em ordem: Long.toString(usuarioId).

17          Sql.unico(conexao, "SELECT set_config('aeroporto.motivo', ?, true)", linha -> linha.getString(1),
      motivo);
      Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Parâmetros ligados
      em ordem: motivo.

18      }
19  }
```

## AutorizacaoJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/AutorizacaoJdbc.java

Confere novamente usuário, pessoa, contrato e perfis atuais no banco. Assim uma Sessao antiga não conserva acesso depois do bloqueio do usuário.

57 linhas | 6 métodos/construtores declarados | SHA-256: 84ad8999f6b072cb...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
  Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.Sessao;
4 import br.edu.aeroporto.excecao.RegraNegocioException;
5 import java.sql.Connection;
6 import java.sql.SQLException;
7
8 public final class AutorizacaoJdbc {
  Declara AutorizacaoJdbc como tipo com campos e métodos. final impede herança.
```

### exigirConsulta - L9 a L11

Permite contas vigentes com qualquer um dos quatro perfis conhecidos.

Entradas declaradas: (Connection conexao, Sessao sessao).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AdministracaoServico.alterarSenha (L67); ConsultaServico.assentos (L37); ConsultaServico.reserva (L45); ConsultaServico.voos (L28); PassageiroServico.buscar (L71); PassageiroServico.buscar (L80); RelatoriosCompletoServico.mapa (L44); RelatoriosCompletoServico.painel (L34); RelatoriosCompletoServico.permitir (L23); RelatoriosCompletoServico.resumoAtrasos (L56).

Chamadas observadas no corpo: exigir.

```
9 public void exigirConsulta(Connection conexao, Sessao sessao) throws SQLException {
  Início da declaração de exigirConsulta. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

10     exigir(conexao, sessao, true);
  Chama AutorizacaoJdbc.exigir: O booleano apenasConsulta abre o acesso de consulta a todos os perfis; false exige atendimento/admin. Falha na consulta de permissão vira regra.

11 }
12
```

### exigirAtendimento - L13 a L15

Restringe a ADMINISTRADOR/ATENDIMENTO.

Entradas declaradas: (Connection conexao, Sessao sessao).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: ArquivosServico.copiarRelatorio (L60); ArquivosServico.importarPassageiros (L24); ArquivosServico.listar (L54); AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.cartao (L76); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AtendimentoServico.embarcar (L132); CadastroServico.permitir (L48); FinanceiroServico.cancelarPagamento (L55); FinanceiroServico.confirmar (L63); FinanceiroServico.pagar (L22); FinanceiroServico.processarReembolso (L89); FinanceiroServico.saldo (L67); FinanceiroServico.solicitarReembolso (L71); PassageiroServico.cadastrar (L34); PassageiroServico.editar (L55); PassageiroServico.vincular (L44); RelatoriosCompletoServico.bagagem (L51); RelatoriosCompletoServico.permitir (L23); ReservaServico.cancelar (L36); ReservaServico.expirar (L24); ReservaServico.remarcar (L49); ReservaServico.reservar (L26).

Chamadas observadas no corpo: exigir.

```
13 public void exigirAtendimento(Connection conexao, Sessao sessao) throws SQLException {
  Início da declaração de exigirAtendimento. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

14     exigir(conexao, sessao, false);
  Chama AutorizacaoJdbc.exigir: O booleano apenasConsulta abre o acesso de consulta a todos os perfis; false exige atendimento/admin. Falha na consulta de permissão vira regra.

15 }
16
```

## exigirAdministrador - L17 a L19

Envia ADMINISTRADOR em ambas as opções do verificador de perfil.

Entradas declaradas: (Connection conexao, Sessao sessao).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AdministracaoServico.acesso (L47); AdministracaoServico.criarUsuario (L32); AdministracaoServico.usuarios (L21); CadastroServico.permitir (L48); DadosFicticiosServico.criar (L23); OperacaoServico.habilitar (L56); RelatoriosCompletoServico.permitir (L23).

Chamadas observadas no corpo: exigirPerfil.

```

17     public void exigirAdministrador(Connection conexao, Sessao sessao) throws SQLException {
        Início da declaração de exigirAdministrador. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.

18         exigirPerfil(conexao, sessao, "ADMINISTRADOR", "ADMINISTRADOR");
        Chama AutorizacaoJdbc.exigirPerfil: Sessao null é recusada; SELECT EXISTS revalida usuário/pessoa/contrato e um dos dois
        nomes de perfil recebidos.

19     }
20 
```

## exigirOperacao - L21 a L23

Permite ADMINISTRADOR ou OPERACAO.

Entradas declaradas: (Connection conexao, Sessao sessao).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AtendimentoServico.rastrear (L111); OperacaoServico.alocar (L24); OperacaoServico.encerrarOcorrencia (L166); OperacaoServico.escalar (L70); OperacaoServico.interditar (L33); OperacaoServico.liberarAlocacao (L43); OperacaoServico.liberarEscala (L81); OperacaoServico.manutencao (L94); OperacaoServico.ocorrencia (L155); OperacaoServico.servicoSolo (L130); OperacaoServico.situacaoManutencao (L107); OperacaoServico.situacaoSolo (L144); PlanejamentoServico.bloquearAssento (L40); PlanejamentoServico.horarios (L71); PlanejamentoServico.programar (L20); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.tarifa (L30); PlanejamentoServico.trocarAeronave (L93); VooServico.transicao (L22).

Chamadas observadas no corpo: exigirPerfil.

```

21     public void exigirOperacao(Connection conexao, Sessao sessao) throws SQLException {
        Início da declaração de exigirOperacao. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.

22         exigirPerfil(conexao, sessao, "ADMINISTRADOR", "OPERACAO");
        Chama AutorizacaoJdbc.exigirPerfil: Sessao null é recusada; SELECT EXISTS revalida usuário/pessoa/contrato e um dos dois
        nomes de perfil recebidos.

23     }
24 
```

## exigirPerfil - L25 a L37

Sessao null é recusada; SELECT EXISTS revalida usuário/pessoa/contrato e um dos dois nomes de perfil recebidos.

Entradas declaradas: (Connection conexao, Sessao sessao, String primeiro, String segundo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AutorizacaoJdbc.exigirAdministrador (L17); AutorizacaoJdbc.exigirOperacao (L21).

Chamadas observadas no corpo: new RegraNegocioException, Sql.unico(conexao, "" SELECT EXISTS ( SELECT 1 FROM usuario\_sistema u JOIN funcionario f ON f.pessoa\_id=u.funcionario\_id JOIN pessoa p ON p.id=f.pessoa\_id JOIN usuario\_perfil up ON up.usuario\_id=u.id JOIN perfil\_acesso pa ON pa.id=up.perfil\_id WHERE u.id=? AND u.ativo AND p.ativo AND f.admissao<=CURRENT\_DATE AND (f.desligamento IS NULL OR f.desligamento>CURRENT\_DATE) AND pa.nome IN (?,?) ) "" , linha -> linha.getBoolean(1), sessao.usuarioId(), primeiro, segundo).orElse, Sql.unico, linha.getBoolean, sessao.usuarioId.

```

25     private void exigirPerfil(Connection conexao, Sessao sessao, String primeiro, String segundo) throws
        SQLException {
        Início da declaração de exigirPerfil. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

26         if (sessao == null) throw new RegraNegocioException("Entre no sistema antes de continuar.");
        Decisão específica: (sessao == null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a
        operação lançando a exceção.

        Lança new RegraNegocioException("Entre no sistema antes de continuar."). Interrompe o caminho normal; a exceção sobe até
        uma captura compatível e pode provocar rollback.

```

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

27           boolean permitido = Sql.unico(conexao, ""

Declara variável local permitido do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: usuario\_sistema, funcionario, pessoa, usuario\_perfil, perfil\_acesso. Parâmetros ligados em ordem: sessao.usuarioId(), primeiro, segundo.

28           SELECT EXISTS (

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

29           SELECT 1 FROM usuario\_sistema u JOIN funcionario f ON f.pessoa\_id=u.funcionario\_id

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

30           JOIN pessoa p ON p.id=f.pessoa\_id JOIN usuario\_perfil up ON up.usuario\_id=u.id

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

31           JOIN perfil\_acesso pa ON pa.id=up.perfil\_id

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

32           WHERE u.id=? AND u.ativo AND p.ativo AND f.admissao<=CURRENT\_DATE

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

33           AND (f.desligamento IS NULL OR f.desligamento>CURRENT\_DATE) AND pa.nome IN (?,?)

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

34           )

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

35           "", linha -> linha.getBoolean(1), sessao.usuarioId(), primeiro, segundo).orElse(false);

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

36           if (!permitido) throw new RegraNegocioException("Usuario inativo ou sem permissao para esta operacao.");

Decisão específica: (!permitido). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException("Usuario inativo ou sem permissao para esta operacao."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

37           }

38

## exigir - L39 a L56

O booleano apenasConsulta abre o acesso de consulta a todos os perfis; false exige atendimento/admin. Falha na consulta de permissão vira regra.

Entradas declaradas: (Connection conexao, Sessao sessao, boolean apenasConsulta).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AutorizacaoJdbc.exigirAtendimento (L13); AutorizacaoJdbc.exigirConsulta (L9).

Chamadas observadas no corpo: new RegraNegocioException, Sql.unico(conexao, "" SELECT EXISTS ( SELECT 1 FROM usuario\_sistema u JOIN funcionario f ON f.pessoa\_id = u.funcionario\_id JOIN pessoa p ON p.id = f.pessoa\_id JOIN usuario\_perfil up ON up.usuario\_id = u.id JOIN perfil\_acesso pa ON pa.id = up.perfil\_id WHERE u.id = ? AND u.ativo AND p.ativo AND f.admissao <= CURRENT\_DATE AND (f.desligamento IS NULL OR f.desligamento > CURRENT\_DATE) AND pa.nome IN ('ADMINISTRADOR', 'ATENDIMENTO', 'OPERACAO', 'CONSULTA') AND (? OR pa.nome IN ('ADMINISTRADOR', 'ATENDIMENTO')) ) "", linha -> linha.getBoolean(1), sessao.usuarioId(), apenasConsulta).orElse, Sql.unico, linha.getBoolean, sessao.usuarioId.

|    |                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 39 | <code>private void exigir(Connection conexao, Sessao sessao, boolean apenasConsulta) throws SQLException {</code> | Início da declaração de exigir. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                                                                                                                                                                                                                                                                    |
| 40 | <code>if (sessao == null) throw new RegraNegocioException("Entre no sistema antes de continuar.");</code>         | Decisão específica: (sessao == null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.<br><br>Lança new RegraNegocioException("Entre no sistema antes de continuar."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.<br><br>new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só. |
| 41 | <code>boolean permitido = Sql.unico(conexao, ""</code>                                                            | Instrução idêntica à de AutorizacaoJdbc:L27. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                          |
| 42 | <code>SELECT EXISTS (</code>                                                                                      | Instrução idêntica à de AutorizacaoJdbc:L28. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                          |
| 43 | <code>SELECT 1 FROM usuario_sistema u</code>                                                                      | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 44 | <code>JOIN funcionario f ON f.pessoa_id = u.funcionario_id</code>                                                 | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 45 | <code>JOIN pessoa p ON p.id = f.pessoa_id</code>                                                                  | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 46 | <code>JOIN usuario_perfil up ON up.usuario_id = u.id</code>                                                       | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 47 | <code>JOIN perfil_acesso pa ON pa.id = up.perfil_id</code>                                                        | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 48 | <code>WHERE u.id = ? AND u.ativo AND p.ativo</code>                                                               | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 49 | <code>AND f.admissao &lt;= CURRENT_DATE</code>                                                                    | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 50 | <code>AND (f.desligamento IS NULL OR f.desligamento &gt; CURRENT_DATE)</code>                                     | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 51 | <code>AND pa.nome IN ('ADMINISTRADOR', 'ATENDIMENTO', 'OPERACAO', 'CONSULTA')</code>                              | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |
| 52 | <code>AND (? OR pa.nome IN ('ADMINISTRADOR', 'ATENDIMENTO'))</code>                                               | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                          |



53                    )

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

54                    "", linha -> linha.getBoolean(1), sessao.usuarioId(), apenasConsulta).orElse(false);

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

55                    if (!permitido) throw new RegraNegocioException("Usuário inativo ou sem permissão para esta operação.");

Decisão específica: (!permitido). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException("Usuário inativo ou sem permissão para esta operação."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

56                    }

57                    }

## BancoDados

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/BancoDados.java

Abre e fecha a conexão JDBC; executa consultas/transações com commit ou rollback; pode repetir a transação inteira em deadlock ou falha de serialização.

69 linhas | 6 métodos/construtores declarados | SHA-256: dcac25b2bb912b88...

Dependências guardadas em campos: Configuracao. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.config.Configuracao;
4 import br.edu.aeroporto.excecao.PersistenciaException;
5 import java.sql.Connection;
6 import java.sql.DriverManager;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

7 import java.sql.SQLException;
8 import java.util.Properties;
9
10 public final class BancoDados {
    Declara BancoDados como tipo com campos e métodos. final impede herança.

11     private final Configuracao configuracao;
    Campo(s) configuracao: private restringe acesso à classe. Tipo Configuracao: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

12

```

### BancoDados - L13 a L15

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (Configuracao configuracao).

Possíveis chamadores identificados: Main.main (L55).

```

13     public BancoDados(Configuracao configuracao) {
    Início da declaração de BancoDados. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

14         this.configuracao = configuracao;
    Guarda no campo configuracao da instância atual o valor de configuracao. O this diferencia o campo de parâmetros com o mesmo nome.

15     }
16

```

### consultar - L17 a L19

Executa a operação em conexão somente leitura; usar T permite devolver DTO, booleano, lista ou outro resultado.

Entradas declaradas: (OperacaoSql<T> operacao).

Saída declarada: T - tipo genérico escolhido pelo contexto da chamada.

Possíveis chamadores identificados: AdministracaoServico.usuarios (L21); ArquivosServico.copiarRelatorio (L60); ArquivosServico.importarPassageiros (L24); ArquivosServico.listar (L54); AtendimentoServico.cartao (L76); AutenticacaoServico.entrar (L27); AutenticacaoServico.existeUsuario (L23); CadastroServico.listar (L25); ConsultaServico.assentos (L37); ConsultaServico.reserva (L45); ConsultaServico.voos (L28); FinanceiroServico.saldo (L67); Main.main (L55); PassageiroServico.buscar (L71); PassageiroServico.buscar (L80); RelatoriosCompletoServico.bagagem (L51); RelatoriosCompletoServico.consultar (L29); RelatoriosCompletoServico.mapa (L44); RelatoriosCompletoServico.painel (L34); RelatoriosCompletoServico.resumoAtrasos (L56).

Chamadas observadas no corpo: executar.

```

17     public <T> T consultar(OperacaoSql<T> operacao) {
    Início da declaração de consultar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

```
18         return executar(operacao, true);
```

Retorno: entrega executar(operacao, true) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama BancoDados.executar: Abre Connection em try com recursos, configura leitura/isolation e desativa autoCommit. Executa callback, confirma e retorna T. Em SQLException/RuntimeException tenta rollback e relança; preserva falha adicional como suppressed.

```
19     }
```

```
20 }
```

## transacao - L21 a L23

Executa a operação de escrita; commit/rollback é responsabilidade de executar.

Entradas declaradas: (OperacaoSql<T> operacao).

Saída declarada: T - tipo genérico escolhido pelo contexto da chamada.

Possíveis chamadores identificados: AdministracaoServico.acesso (L47); AdministracaoServico.alterarSenha (L67); AdministracaoServico.criarUsuario (L32); ArquivosServico.copiarRelatorio (L60); ArquivosServico.importarPassageiros (L24); AutenticacaoServico.entrar (L27); BancoDados.transacaoRepetivel (L25); CadastroServico.salvar (L30); DadosFicticiosServico.criar (L23); FinanceiroServico.cancelarPagamento (L55); InicializacaoServico.criar (L27); Main.main (L55); OperacaoServico.alocar (L24); OperacaoServico.encerrarOcorrencia (L166); OperacaoServico.escalar (L70); OperacaoServico.habilitar (L56); OperacaoServico.interditar (L33); OperacaoServico.liberarAlocacao (L43); OperacaoServico.liberarEscala (L81); OperacaoServico.manutencao (L94); OperacaoServico.ocorrencia (L155); OperacaoServico.servicoSolo (L130); OperacaoServico.situacaoManutencao (L107); OperacaoServico.situacaoSolo (L144); PassageiroServico.cadastrar (L34); PassageiroServico.editar (L55); PassageiroServico.vincular (L44); PlanejamentoServico.bloquearAssento (L40); PlanejamentoServico.horarios (L71); PlanejamentoServico.programar (L20); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.tarifa (L30); PlanejamentoServico.trocarAeronave (L93); RelatoriosCompletoServico.exportar (L68).

Chamadas observadas no corpo: executar.

```
21     public <T> T transacao(OperacaoSql<T> operacao) {
```

Início da declaração de transacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
22         return executar(operacao, false);
```

Retorno: entrega executar(operacao, false) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama BancoDados.executar: Abre Connection em try com recursos, configura leitura/isolation e desativa autoCommit. Executa callback, confirma e retorna T. Em SQLException/RuntimeException tenta rollback e relança; preserva falha adicional como suppressed.

```
23     }
```

```
24 }
```

## transacaoRepetivel - L25 a L34

Até três tentativas no total. Só repete SQLSTATE 40P01 (deadlock) ou 40001 (serialização); repete a operação inteira e não apenas o último comando.

Entradas declaradas: (OperacaoSql<T> operacao).

Saída declarada: T - tipo genérico escolhido pelo contexto da chamada.

Possíveis chamadores identificados: AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AtendimentoServico.embarcar (L132); AtendimentoServico.rastrear (L111); FinanceiroServico.confirmar (L63); FinanceiroServico.pagar (L22); FinanceiroServico.processarReembolso (L89); FinanceiroServico.solicitarReembolso (L71); ReservaServico.cancelar (L36); ReservaServico.expirar (L24); ReservaServico.expirarPendencias (L22); ReservaServico.remarcar (L49); ReservaServico.reservar (L26); VooServico.transicao (L22).

Chamadas observadas no corpo: transacao, ((SQLException) erro.getCause()).getSQLState, erro.getCause, "40P01".equals, "40001".equals.

```
25     public <T> T transacaoRepetivel(OperacaoSql<T> operacao) {
```

Início da declaração de transacaoRepetivel. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
26         for (int tentativa = 0; ; tentativa++) {
```

Laço controlado por inicialização, condição e atualização: for (int tentativa = 0; ; tentativa++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local tentativa do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
27         try {
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

```

28         return transacao(operacao);
Retorno: entrega transacao(operacao) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.
Chama BancoDados.transacao: Executa a operação de escrita; commit/rollback é responsabilidade de executar.
29     } catch (PersistenciaException erro) {
Trata a(s) exceção(ões) declarada(s) em catch (PersistenciaException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.
30         String estado = ((SQLException) erro.getCause()).getSQLState();
Declara variável local estado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
31         if (tentativa >= 2 || !("40P01".equals(estado) || "40001".equals(estado))) throw erro;
Decisão específica: (tentativa >= 2 || !("40P01".equals(estado) || "40001".equals(estado))). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.
Lança erro. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.
32     }
33 }
34 }
35

```

## executar - L36 a L56

Abre Connection em try com recursos, configura leitura/isolation e desativa autoCommit. Executa callback, confirma e retorna T. Em SQLException/RuntimeException tenta rollback e relança; preserva falha adicional como suppressed.

Entradas declaradas: (OperacaoSql<T> operacao, boolean somenteLeitura).

Saída declarada: T - tipo genérico escolhido pelo contexto da chamada.

Possíveis chamadores identificados: BancoDados.consultar (L17); BancoDados.transacao (L21).

Chamadas observadas no corpo: abrir, conexao.setReadOnly, conexao.setTransactionIsolation, conexao.setAutoCommit, operacao.executar, conexao.commit, conexao.rollback, erro.addSuppressed, new PersistenciaException.

```

36     private <T> T executar(OperacaoSql<T> operacao, boolean somenteLeitura) {
Início da declaração de executar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
37         try (Connection conexao = abrir()) {
Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.
38             conexao.setReadOnly(somenteLeitura);
Invoca conexao.setReadOnly com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
39             if (somenteLeitura) conexao.setTransactionIsolation(Connection.TRANSACTION_REPEATABLE_READ);
Decisão específica: (somenteLeitura). Se for true, executa o ramo seguinte; se for false, pula esse ramo.
40             conexao.setAutoCommit(false);
Invoca conexao.setAutoCommit com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
41             try {
Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.
42                 T resultado = operacao.executar(conexao);
Declara variável local resultado do tipo T. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
43                 conexao.commit();
Chama conexao.commit: Confirma as alterações da transação antes de entregar o resultado.
44                 return resultado;
Instrução idêntica à de Validacao:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.
45             } catch (SQLException | RuntimeException erro) {
Trata a(s) exceção(ões) declarada(s) em catch (SQLException | RuntimeException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.
46                 try {
Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.
47                     conexao.rollback();
Chama conexao.rollback: Desfaz alterações da transação que ainda não foram confirmadas.

```

```

48         } catch (SQLException falhaRollback) {
Trata a(s) exceção(ões) declarada(s) em catch (SQLException falhaRollback). O tratamento mostrado abaixo decide se avisa,
repete ou relança.

49         erro.addSuppressed(falhaRollback);
Chama erro.addSuppressed: Anexa uma falha secundária à exceção principal, preservando ambas.

50     }
51     throw erro;
Lança erro. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

52     }
53     } catch (SQLException erro) {
Trata a(s) exceção(ões) declarada(s) em catch (SQLException erro). O tratamento mostrado abaixo decide se avisa, repete ou
relança.

54         throw new PersistenciaException(erro);
Lança new PersistenciaException(erro). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode
provocar rollback.

55     }
56 }
57

```

## abrir - L58 a L68

Cria propriedades do driver com credenciais, schema aeroporto/public, nome da aplicação, timeouts e fuso.

DriverManager.getConnection devolve a conexão; não instala o schema.

Entradas declaradas: ().

Saída declarada: Connection - conexão JDBC fornecida à operação de banco.

Possíveis chamadores identificados: BancoDados.executar (L36).

Chamadas observadas no corpo: new Properties, propriedades.setProperty, configuracao.usuario, configuracao.senha, configuracao.fuso().getId, configuracao.fuso, DriverManager.getConnection, configuracao.url.

```

58     private Connection abrir() throws SQLException {
Início da declaração de abrir. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
invocado.

59         Properties propriedades = new Properties();
Instrução idêntica à de Configuracao:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
execução. A posição e o contexto do método acima mostram quando ela ocorre.

60         propriedades.setProperty("user", configuracao.usuario());
Invoca propriedades.setProperty com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
chamadas do método para localizar o corpo no projeto.

61         propriedades.setProperty("password", configuracao.senha());
Invoca propriedades.setProperty com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
chamadas do método para localizar o corpo no projeto.

62         propriedades.setProperty("currentSchema", "aeroporto,public");
Invoca propriedades.setProperty com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
chamadas do método para localizar o corpo no projeto.

63         propriedades.setProperty("ApplicationName", "sistema-aeroporto");
Invoca propriedades.setProperty com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
chamadas do método para localizar o corpo no projeto.

64         propriedades.setProperty("connectTimeout", "10");
Invoca propriedades.setProperty com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
chamadas do método para localizar o corpo no projeto.

```

```
65      propriedades.setProperty("socketTimeout", "60");
```

Invoca `propriedades.setProperty` com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
66      propriedades.setProperty("options", "-c timezone=" + configuracao.fuso().getId());
```

Invoca `propriedades.setProperty` com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
67      return DriverManager.getConnection(configuracao.url(), propriedades);
```

Retorno: entrega `DriverManager.getConnection(configuracao.url(), propriedades)` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama `DriverManager.getConnection`: Abre conexão JDBC com a URL e propriedades fornecidas.

```
68    }
```

```
69 }
```

## BloqueiosJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/BloqueiosJdbc.java

Concentra os bloqueios de linha e a ordem dos ids. Uma reserva bloqueia seus voos, cabeçalho e itens para reduzir disputas e deadlocks.

34 linhas | 5 métodos/construtores declarados | SHA-256: f95714cf776b6a55...

```

1 package br.edu.aeroporto.infraestrutura.jdbc;
  Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.Dados;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Funções utilitárias para extrair valores dos Map vindos do JDBC, validar dinheiro, converter instantes, exigir condições e gerar códigos.

4 import java.sql.Connection;
5 import java.sql.SQLException;
6 import java.util.Collection;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

7 import java.util.List;
8 import java.util.Map;
9
10 public final class BloqueiosJdbc {
  Declara BloqueiosJdbc como tipo com campos e métodos. final impede herança.
```

### BloqueiosJdbc - L11 a L11

Construtor vazio. Se for private, impede criar instâncias desta classe utilitária a partir de outras classes; os métodos static são usados pelo nome da classe.

Entradas declaradas: ().

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```

11 private BloqueiosJdbc() {}
  Início da declaração de BloqueiosJdbc. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

12
```

### voo - L13 a L15

SELECT FOR UPDATE devolve Map e mantém lock de linha até fim da transação.

Entradas declaradas: (Connection c, long id).

Saída declarada: Map<String, Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: BloqueiosJdbc.voos (L17); OperacaoServico.alocar (L24); OperacaoServico.escalar (L70); OperacaoServico.liberarAlocacao (L43); OperacaoServico.liberarEscala (L81); OperacaoServico.ocorrencia (L155); OperacaoServico.servicoSolo (L130); OperacaoServico.situacaoSolo (L144); PlanejamentoServico.bloquearAssento (L40); PlanejamentoServico.horarios (L71); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.tarifa (L30); PlanejamentoServico.trocarAeronave (L93); VooServico.transicao (L22).

Chamadas observadas no corpo: Sql.registro.

```

13 public static Map<String, Object> voo(Connection c, long id) throws SQLException {
  Início da declaração de voo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

14     return Sql.registro(c, "SELECT * FROM voo WHERE id=? FOR UPDATE", id);
  Retorno: entrega Sql.registro(c, "SELECT * FROM voo WHERE id=? FOR UPDATE", id) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

  Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

15 }
16
```

## voos - L17 a L19

Elimina ids repetidos, ordena e bloqueia cada voo. A ordem consistente reduz deadlocks.

Entradas declaradas: (Connection c, Collection<Long> ids).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: BloqueiosJdbc.reserva (L21); ReservaJdbc.expirar (L148); ReservaJdbc.reservar (L15); ReservaServico.remarcar (L49); VooServico.transicao (L22).

Chamadas observadas no corpo: ids.stream().distinct().sorted().toList, ids.stream().distinct().sorted, ids.stream().distinct, ids.stream, voo.

```

17     public static void voos(Connection c, Collection<Long> ids) throws SQLException {
        Início da declaração de voos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

18         for (long id : ids.stream().distinct().sorted().toList()) voo(c, id);
        Percorre cada elemento da coleção/vetor: for (long id : ids.stream().distinct().sorted().toList()). return, break ou exceção podem
        encerrar esse caminho; continue passa à próxima volta.

19     }
20 
```

## reserva - L21 a L27

Descobre e bloqueia os voos da reserva, depois cabeçalho e itens ordenados; devolve o cabeçalho.

Entradas declaradas: (Connection c, long id).

Saída declarada: Map<String, Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: BloqueiosJdbc.item (L29); FinanceiroServico.cancelarPagamento (L55); FinanceiroServico.confirmar (L63); FinanceiroServico.pagar (L22); FinanceiroServico.processarReembolso (L89); FinanceiroServico.solicitarReembolso (L71); ReservaJdbc.expirar (L148); ReservaServico.cancelar (L36); ReservaServico.remarcar (L49); VooServico.transicao (L22).

Chamadas observadas no corpo: Sql.listar, r.getLong, voos, Sql.registro.

```

21     public static Map<String, Object> reserva(Connection c, long id) throws SQLException {
        Início da declaração de reserva. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

22         List<Long> voos = Sql.listar(c, "SELECT DISTINCT voo_id FROM item_reserva WHERE reserva_id=? ORDER
        BY voo_id", r -> r.getLong(1), id);
        Declara variável local voos do tipo List<Long>. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

        Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. Parâmetros
        ligados em ordem: id.

23         voos(c, voos);
        Chama BloqueiosJdbc.voos: Elimina ids repetidos, ordena e bloqueia cada voo. A ordem consistente reduz deadlocks.

24         var reserva = Sql.registro(c, "SELECT * FROM reserva WHERE id=? FOR UPDATE", id);
        Declara variável local reserva com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
        reserva. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

25         Sql.listar(c, "SELECT id FROM item_reserva WHERE reserva_id=? ORDER BY id FOR UPDATE", r ->
        r.getLong(1), id);
        Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. O FOR UPDATE
        mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

26         return reserva;
        Retorno: entrega reserva ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
        caminho.

27     }
28 
```

## item - L29 a L33

Descobre a reserva, aplica todos os locks dela e finalmente devolve o item bloqueado.

Entradas declaradas: (Connection c, long id).

Saída declarada: Map<String, Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.



Possíveis chamadores identificados: AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AtendimentoServico.embarcar (L132); AtendimentoServico.rastrear (L111).

Chamadas observadas no corpo: Sql.registro, reserva, Dados.id.

```
29     public static Map<String, Object> item(Connection c, long id) throws SQLException {
    Início da declaração de item. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

30         var inicial = Sql.registro(c, "SELECT reserva_id FROM item_reserva WHERE id=?", id);
    Declara variável local inicial com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.

    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    item_reserva. Parâmetros ligados em ordem: id.

31         reserva(c, Dados.id(inicial, "reserva_id"));
    Chama BloqueiosJdbc.reserva: Descobre e bloqueia os voos da reserva, depois cabeçalho e itens ordenados; devolve o
    cabeçalho.

32         return Sql.registro(c, "SELECT * FROM item_reserva WHERE id=? FOR UPDATE", id);
    Retorno: entrega Sql.registro(c, "SELECT * FROM item_reserva WHERE id=? FOR UPDATE", id) ao chamador desta execução;
    nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    item_reserva. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

33     }
34 }
```

## CadastroJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/CadastroJdbc.java

CRUD genérico limitado por TipoCadastro. Monta nomes SQL a partir do catálogo fechado e vincula os valores como parâmetros.

35 linhas | 3 métodos/construtores declarados | SHA-256: 70bfc2be68578eb6...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.TipoCadastro;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Catálogo fechado das tabelas e campos editáveis pelo menu. A mini linguagem de tipos permite converter entradas sem aceitar nomes arbitrários de tabela/coluna.

4 import java.sql.Connection;
5 import java.sql.SQLException;
6 import java.util.ArrayList;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

7 import java.util.List;
8 import java.util.Map;
9 import java.util.stream.Collectors;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

10
11 public final class CadastroJdbc {
    Declara CadastroJdbc como tipo com campos e métodos. final impede herança.
```

### salvar - L12 a L26

Extrai nomes do catálogo. id null monta INSERT RETURNING; id presente monta UPDATE sem mudar chave. Os valores são parâmetros; nomes não vêm de texto arbitrário do usuário.

Entradas declaradas: (Connection c, TipoCadastro tipo, Long id, Map<String, Object> valores).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: CadastroServico.salvar (L30).

Chamadas observadas no corpo: tipo.campos().stream().map(TipoCadastro.Campo::nome).filter(nome -> id == null || !nome.equals(tipo.chave())).toList, tipo.campos().stream().map(TipoCadastro.Campo::nome).filter, tipo.campos().stream().map, tipo.campos(), tipo.campos(), TipoCadastro.Campo::nome, nome.equals, tipo.chave, new ArrayList<Object>, nomes.forEach, parametros.add, valores.get, Sql.inserir, tipo.tabela, String.join, nomes.stream().map(n -> "?").collect, nomes.stream().map, nomes.stream, Collectors.joining, parametros.toArray, Sql.executar, nomes.stream().map(n -> n + "=?").collect, br.edu.aeroporto.dominio.Dados.exigir.

```
12 public long salvar(Connection c, TipoCadastro tipo, Long id, Map<String, Object> valores) throws
    SQLException {
    Início da declaração de salvar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

13     var nomes = tipo.campos().stream().map(TipoCadastro.Campo::nome)
    Declara variável local nomes com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

14     .filter(nome -> id == null || !nome.equals(tipo.chave())).toList();
    Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o argumento/resultado daquela instrução.

15     var parametros = new ArrayList<Object>();
    Declara variável local parametros com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

16     nomes.forEach(nome -> parametros.add(valores.get(nome)));
    Chama nomes.forEach: Executa a ação fornecida para cada elemento.
```

```

17         if (id == null) {
    Decisão específica: (id == null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a
    execução atual com return.

18         return Sql.inserir(c, "INSERT INTO " + tipo.tabela() + "(" + String.join(",", nomes) + ")
VALUES ("
    Retorno: entrega Sql.inserir(c, "INSERT INTO " + tipo.tabela() + "(" + String.join(",", nomes) + ") VALUES (" +
    nomes.stream().map(n -> "?").collect(Collectors.joining(",")) + ") RETURNING " + tipo.chave(), parametros.toArray()) ao chamador
    desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

    Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Parâmetros ligados em ordem:
    parametros.toArray().

19         + nomes.stream().map(n -> "?").collect(Collectors.joining(",")) + ") RETURNING " +
    tipo.chave(), parametros.toArray());
    Nesta parte da expressão, chama nomes.stream().map(n -> "?").collect, nomes.stream().map, nomes.stream, Collectors.joining,
    tipo.chave, parametros.toArray. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

20     }
21     parametros.add(id);
    Chama parametros.add: Insere na coleção. Em Set, o boolean de retorno indica se o elemento era novo.

22     int alterados = Sql.executar(c, "UPDATE " + tipo.tabela() + " SET "
    Declara variável local alterados do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Parâmetros ligados em ordem:
    parametros.toArray().

23     + nomes.stream().map(n -> n + "=?").collect(Collectors.joining(",")) + " WHERE " +
    tipo.chave() + "=?", parametros.toArray());
    Nesta parte da expressão, chama nomes.stream().map(n -> n + "=?").collect, nomes.stream().map, nomes.stream,
    Collectors.joining, tipo.chave, parametros.toArray. Os valores retornados alimentam a instrução que começou nesta linha ou nas
    linhas anteriores.

24     br.edu.aeroporto.dominio.Dados.exigir(alterados == 1, "Cadastro nao encontrado.");
    Invoca br.edu.aeroporto.dominio.Dados.exigir com os argumentos mostrados. O receptor/contrato determina a implementação;
    veja as chamadas do método para localizar o corpo no projeto.

25     return id;
    Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

26 }
27

```

## listar - L28 a L30

Lista 20 registros do tipo com ordem da chave e OFFSET da página.

Entradas declaradas: (Connection c, TipoCadastro tipo, int pagina).

Saída declarada: List<Map<String, Object>> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: CadastroServico.listar (L25).

Chamadas observadas no corpo: Sql.listar, tipo.tabela, tipo.chave, Sql::linha.

```

28     public List<Map<String, Object>> listar(Connection c, TipoCadastro tipo, int pagina) throws
SQLException {
    Início da declaração de listar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

29     return Sql.listar(c, "SELECT * FROM " + tipo.tabela() + " ORDER BY " + tipo.chave() + " LIMIT 20
OFFSET ?", Sql::linha, (pagina-1)*20);
    Retorno: entrega Sql.listar(c, "SELECT * FROM " + tipo.tabela() + " ORDER BY " + tipo.chave() + " LIMIT 20 OFFSET ?",
    Sql::linha, (pagina-1)*20) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
    caminho.

    Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Parâmetros ligados em ordem: (pagina-1)*20.

30 }

```

31

## buscar - L32 a L34

Busca um id do tipo com FOR UPDATE, retornando Map ou erro.

Entradas declaradas: (Connection c, TipoCadastro tipo, long id).

Saída declarada: Map<String, Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: CadastroServico.salvar (L30).

Chamadas observadas no corpo: Sql.registro, tipo.tabela, tipo.chave.

```
32     public Map<String, Object> buscar(Connection c, TipoCadastro tipo, long id) throws SQLException {  
        Início da declaração de buscar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é  
        invocado.
```

```
33         return Sql.registro(c, "SELECT * FROM " + tipo.tabela() + " WHERE " + tipo.chave() + "=? FOR  
UPDATE", id);
```

Retorno: entrega Sql.registro(c, "SELECT \* FROM " + tipo.tabela() + " WHERE " + tipo.chave() + "=? FOR UPDATE", id) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

```
34     }  
35 }
```

## ConsultasJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/ConsultasJdbc.java

Implementa ConsultasRepositorio. Converte linhas SQL nos DTOs tipados para que os menus não precisem entender ResultSet.

72 linhas | 3 métodos/construtores declarados | SHA-256: ae1aaef99d69185...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
   Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.
```

```
2
3 import br.edu.aeroporto.dominio.SituacaoReserva;
4 import br.edu.aeroporto.dominio.SituacaoVoo;
5 import br.edu.aeroporto.dto.AssentoResumo;
6 import br.edu.aeroporto.dto.ReservaResumo;
7 import br.edu.aeroporto.dto.VooResumo;
8 import br.edu.aeroporto.repositorio.ConsultasRepositorio;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Contrato para consultas de voos, assentos e reserva. Usa DTOs e Optional para ausência de uma reserva.

```
9 import java.sql.Connection;
10 import java.sql.SQLException;
11 import java.time.Instant;
12 import java.time.OffsetDateTime;
13 import java.time.ZoneId;
14 import java.util.List;
15 import java.util.Optional;
16
17 public final class ConsultasJdbc implements ConsultasRepositorio {
```

Declara ConsultasJdbc como tipo com campos e métodos. final impede herança. implements liga o tipo aos contratos citados.

### voos - L18 a L38

Consulta intervalo [inicio,fim), JOIN localidades/companhia e LIMIT 20/OFFSET; mapeia colunas e fusos em VooResumo.

Entradas declaradas: (Connection conexao, Instant inicio, Instant fim, int pagina).

Saída declarada: List<VooResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Sql.listar, new VooResumo, linha.getLong, linha.getString, ZoneId.of, linha.getObject, SituacaoVoo.valueOf.

```
18 @Override
   Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.
```

```
19 public List<VooResumo> voos(Connection conexao, Instant inicio, Instant fim, int pagina) throws
   SQLException {
   Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.
```

```
20 return Sql.listar(conexao, ""
   Retorno: entrega Sql.listar(conexao, "" SELECT v.id, c.codigo_icao AS companhia, v.numero, ao.codigo_icao AS origem,
   ad.codigo_icao AS destino, ao.fuso_horario AS fuso_origem, ad.fuso_horario AS fuso_destino, v.partida_prevista, v.ch... ao
   chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.
```

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: voo, companhia\_aerea, rota, aeroporto. Parâmetros ligados em ordem: inicio, fim, (pagina - 1) \* 20.

```
21 SELECT v.id, c.codigo_icao AS companhia, v.numero,
   Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
   chamado, não pelo compilador Java.
```

```
22 ao.codigo_icao AS origem, ad.codigo_icao AS destino,
   Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
   determina onde a instrução termina.
```

```
23 ao.fuso_horario AS fuso_origem, ad.fuso_horario AS fuso_destino,
   Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
   determina onde a instrução termina.
```

|    |                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 24 | <code>v.partida_prevista, v.chegada_prevista, v.partida_estimada, v.partida_real,</code><br><code>v.situacao</code>                                                                                                                                     | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                         |
| 25 | <code>FROM voo v JOIN companhia_aerea c ON c.id = v.companhia_id</code>                                                                                                                                                                                 | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                      |
| 26 | <code>JOIN rota r ON r.id = v.rota_id JOIN aeroporto ao ON ao.id = r.origem_id</code>                                                                                                                                                                   | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                      |
| 27 | <code>JOIN aeroporto ad ON ad.id = r.destino_id</code>                                                                                                                                                                                                  | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                      |
| 28 | <code>WHERE v.partida_prevista &gt;= ? AND v.partida_prevista &lt; ?</code>                                                                                                                                                                             | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                      |
| 29 | <code>ORDER BY v.partida_prevista, v.id LIMIT 20 OFFSET ?</code>                                                                                                                                                                                        | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                      |
| 30 | <code>""</code> , linha -> <code>new VooResumo(linha.getLong("id"), linha.getString("companhia"),</code><br><code>new constrói VooResumo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.</code> |                                                                                                                                                                                                                                |
| 31 | <code>linha.getString("numero"), linha.getString("origem"), linha.getString("destino"),</code>                                                                                                                                                          | Nesta parte da expressão, chama <code>linha.getString</code> . Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                                                                   |
| 32 | <code>ZoneId.of(linha.getString("fuso_origem")), ZoneId.of(linha.getString("fuso_destino")),</code>                                                                                                                                                     | Nesta parte da expressão, chama <code>ZoneId.of</code> , <code>linha.getString</code> . Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                                          |
| 33 | <code>linha.getObject("partida_prevista", OffsetDateTime.class),</code>                                                                                                                                                                                 | Nesta parte da expressão, chama <code>linha.getObject</code> . Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                                                                   |
| 34 | <code>linha.getObject("chegada_prevista", OffsetDateTime.class),</code>                                                                                                                                                                                 | Nesta parte da expressão, chama <code>linha.getObject</code> . Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                                                                   |
| 35 | <code>linha.getObject("partida_estimada", OffsetDateTime.class),</code>                                                                                                                                                                                 | Nesta parte da expressão, chama <code>linha.getObject</code> . Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                                                                   |
| 36 | <code>linha.getObject("partida_real", OffsetDateTime.class),</code><br><code>SituacaoVoo.valueOf(linha.getString("situacao"))</code> ,                                                                                                                  | Nesta parte da expressão, chama <code>linha.getObject</code> , <code>SituacaoVoo.valueOf</code> , <code>linha.getString</code> . Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores. |
| 37 | <code>inicio, fim, (pagina - 1) * 20);</code>                                                                                                                                                                                                           | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                         |
| 38 | <code>}</code>                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                |
| 39 |                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                |

## assentos - L40 a L50

EXISTS considera somente ocupação ainda sem liberada\_em. Mapeia código/classe/bloqueio/ocupação e ordena pelo assento físico.

Entradas declaradas: (Connection conexao, long voold).

Saída declarada: List<AssentoResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: `Sql.listar`, `new AssentoResumo`, `linha.getString`, `linha.getBoolean`.

40      `@Override`

Anotação de contrato: `@Override` confere sobrescrita; `@FunctionalInterface` confere um único método abstrato, conforme a anotação mostrada.

41      `public List<AssentoResumo> assentos(Connection conexao, long vooId) throws SQLException {`

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

42      `return Sql.listar(conexao, ""`

Instrução idêntica à de ConsultasJdbc:L20. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

43      `SELECT i.codigo, i.classe, i.bloqueado,`

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

44      `EXISTS (SELECT 1 FROM ocupacao_assento o`

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

45      `WHERE o.inventario_id = i.id AND o.liberada_em IS NULL) AS ocupado`

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

46      `FROM inventario_assento_voo i JOIN assento_aeronave a ON a.id = i.assento_id`

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

47      `WHERE i.voo_id = ? ORDER BY a.fila, a.coluna`

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

48      `""", linha -> new AssentoResumo(linha.getString("codigo"), linha.getString("classe"),`

new constrói AssentoResumo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

49      `linha.getBoolean("bloqueado"), linha.getBoolean("ocupado")), vooId);`

Nesta parte da expressão, chama `linha.getBoolean`. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

50      `}`

51

## reserva - L52 a L71

Busca cabeçalho por localizador e os trechos por id; LEFT JOIN permite item sem assento ativo. Monta ReservaResumo com enum e lista; ausência cabeçalho retorna Optional vazio.

Entradas declaradas: (Connection conexao, String localizador).

Saída declarada: Optional<ReservaResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: `Sql.unico`, `linha.getLong`, `Sql.listar`, `new ReservaResumo.Trecho`, `item.getLong`, `item.getString`, `item.getInt`, `item.getBigDecimal`, `new ReservaResumo`, `linha.getString`, `SituacaoReserva.valueOf`, `linha.getObject`.

52      `@Override`

Anotação de contrato: `@Override` confere sobrescrita; `@FunctionalInterface` confere um único método abstrato, conforme a anotação mostrada.

53      `public Optional<ReservaResumo> reserva(Connection conexao, String localizador) throws SQLException {`

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

54      `return Sql.unico(conexao, "SELECT id, localizador, situacao, expira_em FROM reserva WHERE  
localizador = ?", linha -> {`

Retorno: entrega `Sql.unico(conexao, "SELECT id, localizador, situacao, expira_em FROM reserva WHERE localizador = ?", linha -> { long reservId = linha.getLong("id"); var trechos = Sql.listar(conexao, "" SELECT i.id, p.nome, c.codigo... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.`

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: reserva. Parâmetros ligados em ordem: localizador.

55      `long reservId = linha.getLong("id");`

Declara variável local `reservId` do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
56         var trechos = Sql.listar(conexao, ""
```

Declara variável local trechos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item\_reserva, pessoa, voo, companhia\_aerea, ocupacao\_assento, inventario\_assento\_voo. Parâmetros ligados em ordem: reservald.

```
57         SELECT i.id, p.nome, c.codigo_icao || v.numero AS voo, i.ordem_trecho,
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
58         i.situacao, a.codigo AS assento, i.valor_base + i.taxa_embarque - i.desconto AS
```

valor  
Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```
59         FROM item_reserva i JOIN pessoa p ON p.id = i.passageiro_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
60         JOIN voo v ON v.id = i.voo_id JOIN companhia_aerea c ON c.id = v.companhia_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
61         LEFT JOIN ocupacao_assento o ON o.item_id = i.id AND o.liberada_em IS NULL
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
62         LEFT JOIN inventario_assento_voo a ON a.id = o.inventario_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
63         WHERE i.reserva_id = ? ORDER BY p.nome, i.passageiro_id, i.ordem_trecho
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
64         "", item -> new ReservaResumo.Trecho(item.getLong("id"), item.getString("nome"),
```

new constrói ReservaResumo.Trecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
65         item.getString("voo"), item.getInt("ordem_trecho"), item.getString("situacao"),
```

Nesta parte da expressão, chama item.getString, item.getInt. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

```
66         item.getString("assento"), item.getBigDecimal("valor")), reservald);
```

Nesta parte da expressão, chama item.getString, item.getBigDecimal. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

```
67         return new ReservaResumo(reservald, linha.getString("localizador"),
```

Retorno: entrega new ReservaResumo(reservald, linha.getString("localizador"), SituacaoReserva.valueOf(linha.getString("situacao")), linha.getObject("expira\_em", OffsetDateTime.class), trechos) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói ReservaResumo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
68         SituacaoReserva.valueOf(linha.getString("situacao")),
```

Nesta parte da expressão, chama SituacaoReserva.valueOf, linha.getString. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

```
69         linha.getObject("expira_em", OffsetDateTime.class), trechos);
```

Nesta parte da expressão, chama linha.getObject. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

```
70     }, localizador);
```

Fecha o bloco anterior e completa/continua a expressão ou abre o próximo bloco indicado nesta linha.

```
71     }
```

```
72 }
```



## FinanceiroJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/FinanceiroJdbc.java

Calcula o devido e o recebido líquido comprometido por reembolsos; confirma os itens e emite bilhetes quando há cobertura.

41 linhas | 3 métodos/construtores declarados | SHA-256: 1cb5ccf7b0eaa164...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.
```

```
2
3 import br.edu.aeroporto.dominio.Dados;
4 import java.math.BigDecimal;
5 import java.sql.Connection;
6 import java.sql.SQLException;
7 import java.time.Instant;
8
9 public final class FinanceiroJdbc {
```

Declara FinanceiroJdbc como tipo com campos e métodos. final impede herança.

### devido - L10 a L17

Soma passagens não canceladas/expiradas e excesso de bagagem não retirado de itens vigentes.

Entradas declaradas: (Connection c, long reserva).

Saída declarada: BigDecimal - valor decimal, utilizado nos cálculos financeiros.

Possíveis chamadores identificados: AtendimentoServico.checkin (L32); AtendimentoServico.embarcar (L132); AtendimentoServico.rastrear (L111); FinanceiroJdbc.confirmar (L25); FinanceiroServico.processar (L47); FinanceiroServico.saldo (L67); FinanceiroServico.solicitarReembolso (L71); ReservaJdbc.solicitarDevolucao (L120).

Chamadas observadas no corpo: Sql.unico(c,"SELECT COALESCE(SUM(valor\_base+taxa\_embarque-desconto),0) FROM item\_reserva WHERE reserva\_id=? AND situacao NOT IN ('CANCELADO','EXPIRADO')",r->r.getBigDecimal(1),reserva).orElse, Sql.unico, r.getBigDecimal, Sql.unico(c,""" SELECT COALESCE(SUM(b.taxa\_excesso),0) FROM bagagem b JOIN check\_in ci ON ci.id=b.checkin\_id JOIN item\_reserva i ON i.id=ci.item\_id WHERE i.reserva\_id=? AND b.situacao<>'RETIRADA' AND i.situacao NOT IN ('CANCELADO','EXPIRADO') """,r->r.getBigDecimal(1),reserva).orElse, passagens.add.

```
10 public BigDecimal devido(Connection c,long reserva) throws SQLException {
    Início da declaração de devido. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
```

```
11 BigDecimal passagens=Sql.unico(c,"SELECT COALESCE(SUM(valor_base+taxa_embarque-desconto),0) FROM
item_reserva WHERE reserva_id=? AND situacao NOT IN
('CANCELADO','EXPIRADO')",r->r.getBigDecimal(1),reserva).orElse(BigDecimal.ZERO);
```

Declara variável local passagens do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: item\_reserva. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: reserva.

```
12 BigDecimal bagagens=Sql.unico(c,""
```

Declara variável local bagagens do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: bagagem, check\_in, item\_reserva. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: reserva.

```
13 SELECT COALESCE(SUM(b.taxa_excesso),0) FROM bagagem b JOIN check_in ci ON
ci.id=b.checkin_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
14 JOIN item_reserva i ON i.id=ci.item_id WHERE i.reserva_id=? AND b.situacao<>'RETIRADA' AND
i.situacao NOT IN ('CANCELADO','EXPIRADO')
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
15 """,r->r.getBigDecimal(1),reserva).orElse(BigDecimal.ZERO);
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
16         return passagens.add(bagagens);
```

Retorno: entrega passagens.add(bagagens) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama passagens.add: Insere na coleção. Em Set, o boolean de retorno indica se o elemento era novo.

```
17     }
```

```
18
```

## recebido - L19 a L23

Pagamentos aprovados menos reembolsos SOLICITADO/PROCESSADO. Solicitação já compromete saldo disponível; não conta apenas dinheiro devolvido.

Entradas declaradas: (Connection c,long reserva).

Saída declarada: BigDecimal - valor decimal, utilizado nos cálculos financeiros.

Possíveis chamadores identificados: AtendimentoServico.checkin (L32); AtendimentoServico.embarcar (L132); AtendimentoServico.rastrear (L111); FinanceiroJdbc.confirmar (L25); FinanceiroServico.processar (L47); FinanceiroServico.saldo (L67); FinanceiroServico.solicitarReembolso (L71); ReservaJdbc.solicitarDevolucao (L120).

Chamadas observadas no corpo: Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM pagamento WHERE reserva\_id=? AND situacao='APROVADO'",r->r.getBigDecimal(1),reserva).orElse, Sql.unico, r.getBigDecimal, Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM reembolso WHERE reserva\_id=? AND situacao IN ('SOLICITADO','PROCESSADO')",r->r.getBigDecimal(1),reserva).orElse, pagos.subtract.

```
19     public BigDecimal recebido(Connection c,long reserva) throws SQLException {
```

Início da declaração de recebido. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
20         BigDecimal pagos=Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM pagamento WHERE reserva_id=? AND situacao='APROVADO'",r->r.getBigDecimal(1),reserva).orElse(BigDecimal.ZERO);
```

Declara variável local pagos do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: pagamento. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: reserva.

```
21         BigDecimal devolvidos=Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM reembolso WHERE reserva_id=? AND situacao IN ('SOLICITADO','PROCESSADO')",r->r.getBigDecimal(1),reserva).orElse(BigDecimal.ZERO);
```

Declara variável local devolvidos do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: reembolso. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: reserva.

```
22         return pagos.subtract(devolvidos);
```

Retorno: entrega pagos.subtract(devolvidos) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca pagos.subtract com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
23     }
```

```
24
```

## confirmar - L25 a L40

Exige pendências, prazo e cobertura; confere voo/janela/ocupação, confirma cada item, emite bilhete e recalcula a reserva.

Entradas declaradas: (Connection c,long reserva,Instant agora).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: FinanceiroServico.confirmar (L63); FinanceiroServico.processar (L47).

Chamadas observadas no corpo: Sql.registro, Sql.listar, Sql::linha, Dados.exigir, pendentes.isEmpty, Dados.instante(r,"expira\_em").isAfter, Dados.instante, recebido(c,reserva).compareTo, recebido, devido, Dados.id, java.util.Set.of("PROGRAMADO","CHECKIN\_ABERTO").contains, java.util.Set.of, v.get, agora.isBefore, ReservaJdbc.janela, Sql.executar, Dados.codigo, new ReservaJdbc().recalcular, new ReservaJdbc.

```
25     public void confirmar(Connection c,long reserva,Instant agora) throws SQLException {
```

Início da declaração de confirmar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
26         var r=Sql.registro(c,"SELECT * FROM reserva WHERE id=?",reserva);
```

Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: reserva. Parâmetros ligados em ordem: reserva.

```
27      var pendentes=Sql.listar(c,"SELECT * FROM item_reserva WHERE reserva_id=? AND situacao='PENDENTE'
ORDER BY id",Sql::linha,reserva);
```

Declara variável local pendentes com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: reserva.

```
28      Dados.exigir(!pendentes.isEmpty(),"Nao ha itens pendentes para confirmar.");
```

Regra obrigatória: !pendentes.isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Nao ha itens pendentes para confirmar.". Essa condição específica não é substituída por outra validação parecida.

```
29      Dados.exigir(Dados.instante(r,"expira_em").isAfter( agora),"Reserva vencida.");
```

Regra obrigatória: Dados.instante(r,"expira\_em").isAfter( agora) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Reserva vencida.". Essa condição específica não é substituída por outra validação parecida.

```
30      Dados.exigir(recebido(c,reserva).compareTo(devido(c,reserva))>=0,"Pagamento insuficiente para
confirmar todos os itens.");
```

Regra obrigatória: recebido(c,reserva).compareTo(devido(c,reserva))>=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Pagamento insuficiente para confirmar todos os itens.". Essa condição específica não é substituída por outra validação parecida.

```
31      for(var i:pendentes) {
```

Percorre cada elemento da coleção/vetor: for(var i:pendentes). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
32          long id=Dados.id(i,"id");
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
33          var v=Sql.registro(c,"SELECT * FROM voo WHERE id=?",Dados.id(i,"voo_id"));
```

Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. Parâmetros ligados em ordem: Dados.id(i,"voo\_id").

```
34      Dados.exigir(java.util.Set.of("PROGRAMADO","CHECKIN_ABERTO").contains(v.get("situacao")) &&
agora.isBefore(ReservaJdbc.janela(v,"checkin_fecha")), "Voo indisponivel para confirmacao.");
```

Regra obrigatória: java.util.Set.of("PROGRAMADO","CHECKIN\_ABERTO").contains(v.get("situacao")) && agora.isBefore(ReservaJdbc.janela(v,"checkin\_fecha")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Voo indisponivel para confirmacao.". Essa condição específica não é substituída por outra validação parecida.

```
35      Sql.registro(c,"SELECT id FROM ocupacao_assento WHERE item_id=? AND liberada_em IS NULL",id);
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: ocupacao\_assento. Parâmetros ligados em ordem: id.

```
36      Sql.executar(c,"UPDATE item_reserva SET situacao='CONFIRMADO' WHERE id=?",id);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: id.

```
37      Sql.executar(c,"INSERT INTO bilhete(item_id,numero,emitido_em) VALUES
(?,?,?)",id,Dados.codigo("B",20),agora);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: bilhete. Parâmetros ligados em ordem: id, Dados.codigo("B",20), agora.

```
38      }
```

```
39      new ReservaJdbc().recalcular(c,reserva);
```

Invoca new ReservaJdbc().recalcular com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

new constrói ReservaJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
40      }
```

```
41 }
```

## InicializacaoJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/InicializacaoJdbc.java

Cria Brasil/localidades/cargo/pessoa/funcionário/usuário/perfis em primeiro acesso e impede dois administradores iniciais concorrentes.

63 linhas | 1 métodos/construtores declarados | SHA-256: c0168dd7ee02b809...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.PrimeiroAcesso;
4 import br.edu.aeroporto.excecao.RegraNegocioException;
5 import java.sql.Connection;
6 import java.sql.SQLException;
7 import java.time.LocalDate;
8 import java.time.ZoneId;
9 import java.util.UUID;
10
11 public final class InicializacaoJdbc {
    Declara InicializacaoJdbc como tipo com campos e métodos. final impede herança.
```

### criarAdministrador - L12 a L62

Lock dedicado ao primeiro acesso protege o teste de existência. Reutiliza Brasil/cidade/aeroporto/cargo quando houver conflito previsto, cria pessoa/funcionário/conta e atribui perfil administrador; retorna id de usuário.

Entradas declaradas: (Connection conexao, PrimeiroAcesso dados, String senhaHash, LocalDate hoje, ZoneId fuso).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: InicializacaoServico.criar (L27).

Chamadas observadas no corpo: Sql.unico, Sql.unico(conexao, "SELECT EXISTS (SELECT 1 FROM usuario\_sistema)", linha -> linha.getBoolean(1)).orElse, linha.getBoolean, new RegraNegocioException, Sql.executar, Sql.unico(conexao, "SELECT id FROM pais WHERE codigo = 'BR'", linha -> linha.getLong(1)).orElseThrow, linha.getLong, dados.cidade, dados.regiao, Sql.unico(conexao, "SELECT id FROM cidade WHERE pais\_id = ? AND nome = ? AND regiao = ?", linha -> linha.getLong(1), pais, dados.cidade(), dados.regiao()).orElseThrow, dados.icao, dados.aeroporto, fuso.getId, Sql.unico(conexao, "SELECT id FROM aeroporto WHERE codigo\_icao = ? AND ativo", linha -> linha.getLong(1), dados.icao()).orElseThrow, Sql.unico(conexao, "SELECT id FROM cargo WHERE nome = 'Administrador do sistema'", linha -> linha.getLong(1)).orElseThrow, Sql.inserir, dados.nome, dados.nascimento, UUID.randomUUID().toString().replace("-", "").substring, UUID.randomUUID().toString().replace, UUID.randomUUID().toString, UUID.randomUUID, dados.login.

```
12 public long criarAdministrador(Connection conexao, PrimeiroAcesso dados, String senhaHash,
    Início da declaração de criarAdministrador. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

13                                     LocalDate hoje, ZoneId fuso) throws SQLException {
    Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

14                                     // Serializa o primeiro cadastro mesmo com dois terminais abertos.
    Comentário/documentação do autor; descreve intenção ou convenção e não executa.

15                                     Sql.unico(conexao, "SELECT pg_advisory_xact_lock(hashtext('aeroporto.primeiro_acesso'))", linha ->
    true);
    Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado.

16                                     boolean existe = Sql.unico(conexao, "SELECT EXISTS (SELECT 1 FROM usuario_sistema)", linha ->
    linha.getBoolean(1)).orElse(false);
    Declara variável local existe do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas
    mencionadas: usuario_sistema.

17                                     if (existe) throw new RegraNegocioException("O primeiro acesso já foi criado. Use o login
    existente.");
    Decisão específica: (existe). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a
    operação lançando a exceção.

    Lança new RegraNegocioException("O primeiro acesso já foi criado. Use o login existente."). Interrompe o caminho normal; a
    exceção sobe até uma captura compatível e pode provocar rollback.

    new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
    banco por si só.
```

18

```
19      Sql.executar(conexao, "INSERT INTO pais(codigo, nome) VALUES ('BR', 'Brasil') ON CONFLICT (codigo)
DO NOTHING");
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: pais. ON CONFLICT define o tratamento da colisão descrita no SQL.

20

```
long pais = Sql.unico(conexao, "SELECT id FROM pais WHERE codigo = 'BR'", linha ->
linha.getLong(1)).orElseThrow();
```

Declara variável local pais do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: pais.

21

```
Sql.executar(conexao, ""
```

Instrução idêntica à de AuditoriaJdbc:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

22

```
INSERT INTO cidade(pais_id, nome, regiao) VALUES (?, ?, ?)
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

23

```
ON CONFLICT (pais_id, nome, regiao) DO NOTHING
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

24

```
""", pais, dados.cidade(), dados.regiao());
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

25

```
long cidade = Sql.unico(conexao, "SELECT id FROM cidade WHERE pais_id = ? AND nome = ? AND regiao
= ?",
```

Declara variável local cidade do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: cidade. Parâmetros ligados em ordem: pais, dados.cidade(), dados.regiao().

26

```
linha -> linha.getLong(1), pais, dados.cidade(), dados.regiao()).orElseThrow();
```

Nesta parte da expressão, chama linha.getLong, dados.cidade, dados.regiao. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

27

```
Sql.executar(conexao, ""
```

Instrução idêntica à de AuditoriaJdbc:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

28

```
INSERT INTO aeroporto(cidade_id, codigo_icao, nome, fuso_horario) VALUES (?, ?, ?, ?)
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

29

```
ON CONFLICT (codigo_icao) DO NOTHING
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

30

```
""", cidade, dados.icao(), dados.aeroporto(), fuso.getId());
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

31

```
long aeroporto = Sql.unico(conexao, "SELECT id FROM aeroporto WHERE codigo_icao = ? AND ativo",
```

Declara variável local aeroporto do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: aeroporto. Parâmetros ligados em ordem: dados.icao().

32

```
linha -> linha.getLong(1), dados.icao()).orElseThrow(() -> new RegraNegocioException("O
aeroporto está inativo."));
```

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

33

```
Sql.executar(conexao, ""
```

Instrução idêntica à de AuditoriaJdbc:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

34

```
INSERT INTO cargo(nome, area) VALUES ('Administrador do sistema', 'ADMINISTRACAO')
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

|    |                                                                                                       |                                                                                                                                                                                                        |
|----|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 35 | ON CONFLICT (nome) DO NOTHING                                                                         | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                 |
| 36 | """);                                                                                                 | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                         |
| 37 | long cargo = Sql.unico(conexao, "SELECT id FROM cargo WHERE nome = 'Administrador do sistema'",       | Declara variável local cargo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                        |
|    |                                                                                                       | Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: cargo.                                                                 |
| 38 | linha -> linha.getLong(1)).orElseThrow();                                                             | Nesta parte da expressão, chama linha.getLong. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                                                           |
| 39 | long pessoa = Sql.inserir(conexao, "INSERT INTO pessoa(nome, nascimento) VALUES (?, ?) RETURNING id", | Declara variável local pessoa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                       |
|    |                                                                                                       | Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: pessoa. Parâmetros ligados em ordem: dados.nome(), dados.nascimento().           |
| 40 | dados.nome(), dados.nascimento());                                                                    | Nesta parte da expressão, chama dados.nome, dados.nascimento. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                                            |
| 41 | String matricula = "F" + UUID.randomUUID().toString().replace("-", "").substring(0, 20);              | Declara variável local matricula do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                  |
| 42 | Sql.executar(conexao, ""                                                                              | Instrução idêntica à de AuditoriaJdbc:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre. |
| 43 | INSERT INTO funcionario(pessoa_id, cargo_id, aeroporto_base_id, matricula, admissao)                  | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                              |
| 44 | VALUES (?, ?, ?, ?, ?)                                                                                | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                              |
| 45 | """, pessoa, cargo, aeroporto, matricula, hoje);                                                      | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                         |
| 46 | long usuario = Sql.inserir(conexao, ""                                                                | Declara variável local usuario do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                      |
|    |                                                                                                       | Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: usuario_sistema. Parâmetros ligados em ordem: pessoa, dados.login(), senhaHash.  |
| 47 | INSERT INTO usuario_sistema(funcionario_id, login, senha_hash) VALUES (?, ?, ?) RETURNING id          | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                              |
| 48 | """, pessoa, dados.login(), senhaHash);                                                               | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                         |
| 49 | Sql.executar(conexao, ""                                                                              | Instrução idêntica à de AuditoriaJdbc:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre. |
| 50 | INSERT INTO perfil_acesso(nome, descricao)                                                            | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                              |
| 51 | VALUES ('ADMINISTRADOR', 'Administração do sistema'),                                                 | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                              |

|    |                                                                           |                                                                                                                                                                                                             |
|----|---------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 52 | <code>('ATENDIMENTO', 'Cadastro e atendimento a passageiros'),</code>     | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                      |
| 53 | <code>('OPERACAO', 'Operação e consulta de voos'),</code>                 | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                      |
| 54 | <code>('CONSULTA', 'Consulta de informações')</code>                      | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                      |
| 55 | <code>ON CONFLICT (nome) DO NOTHING</code>                                | Instrução idêntica à de InicializacaoJdbc:L35. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre. |
| 56 | <code>"";</code>                                                          | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                              |
| 57 | <code>Sql.executar(conexao, ""</code>                                     | Instrução idêntica à de AuditoriaJdbc:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.      |
| 58 | <code>INSERT INTO usuario_perfil(usuario_id, perfil_id)</code>            | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                   |
| 59 | <code>SELECT ?, id FROM perfil_acesso WHERE nome = 'ADMINISTRADOR'</code> | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                   |
| 60 | <code>"" , usuario);</code>                                               | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                              |
| 61 | <code>return usuario;</code>                                              | Retorno: entrega usuario ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.                                                                                |
| 62 | <code>}</code>                                                            |                                                                                                                                                                                                             |
| 63 | <code>}</code>                                                            |                                                                                                                                                                                                             |



## OperacaoJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/OperacaoJdbc.java

Valida escalas, licenças, descanso, quantidade mínima da equipe e compatibilidade/localização dos recursos.

82 linhas | 4 métodos/construtores declarados | SHA-256: 0b23184733675c1e...

```

1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

4 import java.sql.Connection;
5 import java.sql.SQLException;
6 import java.time.Duration;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

7 import java.time.Instant;
8 import java.time.LocalDate;
9 import java.time.ZoneId;
10 import java.util.Map;
11 import java.util.Set;
12
13 public final class OperacaoJdbc {
    Declara OperacaoJdbc como tipo com campos e métodos. final impede herança.
```

### validarEscalasDoVoo - L14 a L18

Percorre escalas ativas, revalidando cada uma com seu próprio id em ignorar para não conflitar consigo mesma.

Entradas declaradas: (Connection c,long voo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: OperacaoJdbc.validarTripulacao (L57); OperacaoServico.habilitar (L56); PlanejamentoServico.horarios (L71); PlanejamentoServico.reprogramar (L52).

Chamadas observadas no corpo: Sql.listar, Sql::linha, validarEscala, Dados.id.

```

14 public static void validarEscalasDoVoo(Connection c,long voo) throws SQLException {
    Início da declaração de validarEscalasDoVoo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

15     for(var escala:Sql.listar(c,"SELECT * FROM escala_funcionario WHERE voo_id=? AND ativa ORDER BY
funcionario_id",Sql::linha,voo)) {
    Percorre cada elemento da coleção/vetor: for(var escala:Sql.listar(c,"SELECT * FROM escala_funcionario WHERE voo_id=? AND ativa ORDER BY funcionario_id",Sql::linha,voo)). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

    Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: escala_funcionario. Parâmetros ligados em ordem: voo.

16         validarEscala(c,escala,Dados.id(escala,"id"));
    Chama OperacaoJdbc.validarEscala: Jornada <=14 h, contrato vigente, aeroporto/voo compatíveis, cobertura de 45 min antes/30 depois, licença válida. Jornada anterior/proxima exigem 12 h e continuidade; primeira parte da base.

17     }
18 }
19
```

### validarEscala - L20 a L55

Jornada <=14 h, contrato vigente, aeroporto/voo compatíveis, cobertura de 45 min antes/30 depois, licença válida. Jornada anterior/proxima exigem 12 h e continuidade; primeira parte da base.

Entradas declaradas: (Connection c,Map<String,Object> escala,Long ignorar).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: OperacaoJdbc.validarEscalasDoVoo (L14); OperacaoServico.escalar (L70).



Chamadas observadas no corpo: Dados.id, Dados.instante, Dados.texto, Dados.exigir, fim.isAfter, Duration.between(inicio,fim).compareTo, Duration.between, Duration.ofHours, Sql.registro, inicio.atZone(ZoneId.of(Sql.registro(c,"SELECT fuso\_horario FROM aeroporto WHERE id=? AND ativo",aeroporto).get("fuso\_horario").toString()))).toLocalDate, inicio.atZone, ZoneId.of, Sql.registro(c,"SELECT fuso\_horario FROM aeroporto WHERE id=? AND ativo",aeroporto).get("fuso\_horario").toString, Sql.registro(c,"SELECT fuso\_horario FROM aeroporto WHERE id=? AND ativo",aeroporto).get, fim.atZone(ZoneId.of("UTC"))).toLocalDate, fim.atZone, ((java.sql.Date)f.get("admissao")).toLocalDate, f.get, admissao.isAfter, ((java.sql.Date)f.get("desligamento")).toLocalDate().isAfter, ((java.sql.Date)f.get("desligamento")).toLocalDate, escala.get, Set.of("CANCELADO","CONCLUIDO").contains, Set.of, voo.get, Set.of("COMANDANTE","COPILOTO","COMISSARIO").contains, inicio.isAfter, PlanejamentoJdbc.efetivo(voo,"partida").minus, PlanejamentoJdbc.efetivo, Duration.ofMinutes, fim.isBefore, PlanejamentoJdbc.efetivo(voo,"chegada").plus, Sql.listar(c,"SELECT id FROM habilitacao\_tripulante WHERE funcionario\_id=? AND modelo\_id=? AND funcao=? AND validade>=?"),r->r.getLong(1),funcionario,Dados.id(voo,"modelo\_id"),funcao,ultimo).isEmpty, Sql.listar, r.getLong, Sql.unico, Sql::linha, anterior.isPresent, Dados.instante(anterior.get(), "fim").plus(Duration.ofHours(12)).isAfter, Dados.instante(anterior.get(), "fim").plus, anterior.get, proxima.isPresent, fim.plus(Duration.ofHours(12)).isAfter, fim.plus, proxima.get.

```
20 public static void validarEscala(Connection c,Map<String,Object> escala,Long ignorar) throws
   SQLException {
```

Início da declaração de validarEscala. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
21 long funcionario=Dados.id(escala,"funcionario_id"),aeroporto=Dados.id(escala,"aeroporto_id");
```

Declara variável local funcionario, aeroporto do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
22 Instant inicio=Dados.instante(escala,"inicio"),fim=Dados.instante(escala,"fim");
```

Declara variável local inicio, fim do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
23 String funcao=Dados.texto(escala,"funcao");
```

Declara variável local funcao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
24 Dados.exigir(fim.isAfter(inicio) &&
   Duration.between(inicio,fim).compareTo(Duration.ofHours(14))<=0,"Jornada invalida ou superior a 14h.");
```

Regra obrigatória: fim.isAfter(inicio) && Duration.between(inicio,fim).compareTo(Duration.ofHours(14))<=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Jornada invalida ou superior a 14 h.". Essa condição específica não é substituída por outra validação parecida.

```
25 var f=Sql.registro(c,"SELECT f.* FROM funcionario f JOIN pessoa p ON p.id=f.pessoa_id WHERE
   f.pessoa_id=? AND p.ativo",funcionario);
```

Declara variável local f com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: funcionario, pessoa. Parâmetros ligados em ordem: funcionario.

```
26 LocalDate dia=inicio.atZone(ZoneId.of(Sql.registro(c,"SELECT fuso_horario FROM aeroporto WHERE
   id=? AND ativo",aeroporto).get("fuso_horario").toString()))).toLocalDate();
```

Declara variável local dia do tipo LocalDate. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeroporto. Parâmetros ligados em ordem: aeroporto.

```
27 LocalDate ultimo=fim.atZone(ZoneId.of("UTC")).toLocalDate();
```

Declara variável local ultimo do tipo LocalDate. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
28 LocalDate admissao=((java.sql.Date)f.get("admissao")).toLocalDate();
```

Declara variável local admissao do tipo LocalDate. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
29 Dados.exigir(!admissao.isAfter(dia) && (f.get("desligamento")==null ||
   ((java.sql.Date)f.get("desligamento")).toLocalDate().isAfter(ultimo)),"Funcionario nao esta contratado
   durante a escala.");
```

Regra obrigatória: !admissao.isAfter(dia) && (f.get("desligamento")==null || ((java.sql.Date)f.get("desligamento")).toLocalDate().isAfter(ultimo)) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Funcionario nao esta contratado durante a escala.". Essa condição específica não é substituída por outra validação parecida.

```
30 long destino=aeroporto;
```

Declara variável local destino do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
31 if(escala.get("voo_id")!=null) {
```

Decisão específica: (escala.get("voo\_id")!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo e executa o else.

- 32        `var voo=Sql.registro(c,"SELECT v.*,r.origem_id,r.destino_id,a.modelo_id FROM voo v JOIN rota r ON r.id=v.rota_id JOIN aeronave a ON a.id=v.aeronave_id WHERE v.id=?",Dados.id(escala,"voo_id"));`  
 Declara variável local voo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo, rota, aeronave. Parâmetros ligados em ordem: Dados.id(escala,"voo\_id").
- 33        `Dados.exigir(!Set.of("CANCELADO","CONCLUIDO").contains(voo.get("situacao")), "Voo encerrado.");`  
 Regra obrigatória: !Set.of("CANCELADO","CONCLUIDO").contains(voo.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Voo encerrado.". Essa condição específica não é substituída por outra validação parecida.
- 34        `Dados.exigir(aeroporto==Dados.id(voo,"origem_id"),"Escala deve iniciar na origem.");`  
 Regra obrigatória: aeroporto==Dados.id(voo,"origem\_id") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Escala deve iniciar na origem.". Essa condição específica não é substituída por outra validação parecida.
- 35        `if(Set.of("COMANDANTE","COPILOTO","COMISSARIO").contains(funcao)) {`  
 Decisão específica: (Set.of("COMANDANTE","COPILOTO","COMISSARIO").contains(funcao)). Se for true, executa o ramo seguinte; se for false, pula esse ramo.
- 36        `Dados.exigir(!inicio.isAfter(PlanejamentoJdbc.efetivo(voo,"partida").minus(Duration.ofMinutes(45))) && !fim.isBefore(PlanejamentoJdbc.efetivo(voo,"chegada").plus(Duration.ofMinutes(30))), "Escala nao cobre preparacao e chegada.");`  
 Regra obrigatória: !inicio.isAfter(PlanejamentoJdbc.efetivo(voo,"partida").minus(Duration.ofMinutes(45))) && !fim.isBefore(PlanejamentoJdbc.efetivo(voo,"chegada").plus(Duration.ofMinutes(30))) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Escala nao cobre preparacao e chegada.". Essa condição específica não é substituída por outra validação parecida.
- 37        `Dados.exigir(!Sql.listar(c,"SELECT id FROM habilitacao_tripulante WHERE funcionario_id=? AND modelo_id=? AND funcao=? AND validade>=?",r->r.getLong(1),funcionario,Dados.id(voo,"modelo_id"),funcao,ultimo).isEmpty(),"Habilitacao ausente ou vencida.");`  
 Regra obrigatória: !Sql.listar(c,"SELECT id FROM habilitacao\_tripulante WHERE funcionario\_id=? AND modelo\_id=? AND funcao=? AND validade>=?",r->r.getLong(1),funcionario,Dados.id(voo,"modelo\_id"),funcao,ultimo).isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Habilitacao ausente ou vencida.". Essa condição específica não é substituída por outra validação parecida.
- Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: habilitacao\_tripulante. Parâmetros ligados em ordem: funcionario, Dados.id(voo,"modelo\_id"), funcao, ultimo.
- 38        `destino=Dados.id(voo,"destino_id");`  
 Atualiza destino usando = e o resultado de Dados.id(voo,"destino\_id"). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.
- 39        `}`  
 40        `} else Dados.exigir(!Set.of("COMANDANTE","COPILOTO","COMISSARIO").contains(funcao), "Tripulante requer voo.");`  
 Regra obrigatória: !Set.of("COMANDANTE","COPILOTO","COMISSARIO").contains(funcao) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Tripulante requer voo.". Essa condição específica não é substituída por outra validação parecida.
- 41        `var anterior=Sql.unico(c,""`  
 Declara variável local anterior com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: escala\_funcionario, voo, rota. Parâmetros ligados em ordem: funcionario, ignorar, inicio.
- 42        `SELECT e.*,CASE WHEN e.funcao IN ('COMANDANTE','COPILOTO','COMISSARIO') THEN r.destino_id ELSE e.aeroporto_id END AS destino`  
 Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.
- 43        `FROM escala_funcionario e LEFT JOIN voo v ON v.id=e.voo_id LEFT JOIN rota r ON r.id=v.rota_id`  
 Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.
- 44        `WHERE e.funcionario_id=? AND (e.ativa OR v.situacao='CONCLUIDO') AND e.id<>COALESCE(?,0) AND e.inicio<? ORDER BY e.inicio DESC LIMIT 1`  
 Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
45      "",Sql::linha,funcionario,ignorar,inicio);
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
46      if(anterior.isPresent()) {
```

Decisão específica: (anterior.isPresent()). Se for true, executa o ramo seguinte; se for false, pula esse ramo e executa o else.

```
47      Dados.exigir(!Dados.instante(anterior.get(),"fim").plus(Duration.ofHours(12)).isAfter(inicio),"Descanso
minimo de 12h nao atendido.");
```

Regra obrigatória: !Dados.instante(anterior.get(),"fim").plus(Duration.ofHours(12)).isAfter(inicio) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Descanso minimo de 12 h nao atendido.". Essa condição específica não é substituída por outra validação parecida.

```
48      Dados.exigir(Dados.id(anterior.get(),"destino")==aeroporto,"Funcionario esta em outro
aeroporto.");
```

Regra obrigatória: Dados.id(anterior.get(),"destino")==aeroporto deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Funcionario esta em outro aeroporto.". Essa condição específica não é substituída por outra validação parecida.

```
49      } else Dados.exigir(Dados.id(f,"aeroporto_base_id")==aeroporto,"Primeira escala deve iniciar na
base do funcionario.");
```

Regra obrigatória: Dados.id(f,"aeroporto\_base\_id")==aeroporto deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Primeira escala deve iniciar na base do funcionario.". Essa condição específica não é substituída por outra validação parecida.

```
50      var proxima=Sql.unico(c,"SELECT * FROM escala_funcionario WHERE funcionario_id=? AND ativa AND
id<>COALESCE(?,0) AND inicio>=? ORDER BY inicio LIMIT 1",Sql::linha,funcionario,ignorar,inicio);
```

Declara variável local proxima com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: escala\_funcionario. Parâmetros ligados em ordem: funcionario, ignorar, inicio.

```
51      if(proxima.isPresent()) {
```

Decisão específica: (proxima.isPresent()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
52      Dados.exigir(!fim.plus(Duration.ofHours(12)).isAfter(Dados.instante(proxima.get(),"inicio")), "Descanso ate
proxima jornada insuficiente.");
```

Regra obrigatória: !fim.plus(Duration.ofHours(12)).isAfter(Dados.instante(proxima.get(),"inicio")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Descanso ate proxima jornada insuficiente.". Essa condição específica não é substituída por outra validação parecida.

```
53      Dados.exigir(destino==Dados.id(proxima.get(),"aeroporto_id"),"Proxima escala em outro
aeroporto.");
```

Regra obrigatória: destino==Dados.id(proxima.get(),"aeroporto\_id") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Proxima escala em outro aeroporto.". Essa condição específica não é substituída por outra validação parecida.

```
54      }
```

```
55      }
```

```
56
```

## validarTripulacao - L57 a L65

Revalida jornadas e conta assentos vendáveis. Exige 1 comandante, 1 copiloto e ceil(assentos/50) comissários, mínimo 1.

Entradas declaradas: (Connection c, long voo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: VooServico.transicao (L22).

Chamadas observadas no corpo: validarEscalasDoVoo, Sql.unico(c,"SELECT COUNT(\*) FROM inventario\_assento\_voo WHERE voo\_id=? AND NOT bloqueado",r->r.getLong(1),voo).orElse, Sql.unico, r.getLong, Sql.unico(c,"SELECT COUNT(\*) FROM escala\_funcionario WHERE voo\_id=? AND ativa AND funcao=?",r->r.getLong(1),voo,funcao).orElse, funcao.equals, Math.max, Dados.exigir.

```
57      public static void validarTripulacao(Connection c,long voo) throws SQLException {
```

Início da declaração de validarTripulacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
58      validarEscalasDoVoo(c,voo);
```

Chama OperacaoJdbc.validarEscalasDoVoo: Percorre escalas ativas, revalidando cada uma com seu próprio id em ignorar para não conflitar consigo mesma.

```

59      long assentos=Sql.unico(c,"SELECT COUNT(*) FROM inventario_assento_voo WHERE voo_id=? AND NOT
bloqueado",r->r.getLong(1),voo).orElse(0L);
    Declara variável local assentos do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas
    mencionadas: inventario_assento_voo. COUNT calcula uma contagem no banco, não carrega cada objeto para contar em Java.
    Parâmetros ligados em ordem: voo.

60      for(String funcao:new String[]{"COMANDANTE","COPILOTO","COMISSARIO"}) {
    Percorre cada elemento da coleção/vetor: for(String funcao:new String[]{"COMANDANTE","COPILOTO","COMISSARIO"}). return,
    break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

    new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

61      long quantidade=Sql.unico(c,"SELECT COUNT(*) FROM escala_funcionario WHERE voo_id=? AND ativa
AND funcao=?",r->r.getLong(1),voo,funcao).orElse(0L);
    Declara variável local quantidade do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
    atual.

    Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas
    mencionadas: escala_funcionario. COUNT calcula uma contagem no banco, não carrega cada objeto para contar em Java.
    Parâmetros ligados em ordem: voo, funcao.

62      long minimo=funcao.equals("COMISSARIO"?Math.max(1,(assentos+49)/50):1;
    Declara variável local minimo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    Escolha de valor: funcao.equals("COMISSARIO"). true escolhe Math.max(1,(assentos+49)/50); false escolhe 1. Só a alternativa
    escolhida é avaliada.

63      Dados.exigir(quantidade>=minimo,"Tripulacao insuficiente: "+funcao+" minimo "+minimo);
    Regra obrigatória: quantidade>=minimo deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com
    "Tripulacao insuficiente: "+funcao+" minimo "+minimo. Essa condição específica não é substituída por outra validação parecida.

64      }
65      }
66

```

## validarRecurso - L67 a L81

Valida finalidade/intervalo, voo/localidades ativos, recurso disponível, aeroporto esperado, terminal vigente, envergadura e comprimento mínimo de pista.

Entradas declaradas: (Connection c,long voo,long recurso,String finalidade,Instant inicio,Instant fim).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: OperacaoServico.alocar (L24).

Chamadas observadas no corpo: Dados.exigir, Set.of("PARTIDA","CHEGADA","APOIO").contains, Set.of, fim.isAfter, Sql.registro, Set.of("CANCELADO","CONCLUIDO").contains, v.get, finalidade.equals, Dados.id, r.get, Dados.dinheiro(r,"envergadura\_max\_m").compareTo, Dados.dinheiro, "PISTA".equals, Dados.dinheiro(r,"comprimento\_m").compareTo.

```

67      public static void validarRecurso(Connection c,long voo,long recurso,String finalidade,Instant
inicio,Instant fim) throws SQLException {
    Início da declaração de validarRecurso. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
    é invocado.

68      Dados.exigir(Set.of("PARTIDA","CHEGADA","APOIO").contains(finalidade) &&
fim.isAfter(inicio),"Alocacao invalida.");
    Regra obrigatória: Set.of("PARTIDA","CHEGADA","APOIO").contains(finalidade) && fim.isAfter(inicio) deve resultar em true. Se
    false, Dados.exigir lança RegraNegocioException com "Alocacao invalida.". Essa condição específica não é substituída por outra
    validação parecida.

69      var v=Sql.registro(c,"""
    Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é
    o bloco atual.

    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    voo, rota, aeronave, modelo_aeronave. Parâmetros ligados em ordem: voo.

70      SELECT v.*,r.origem_id,r.destino_id,m.envergadura_m,m.comprimento_pista_min_m
    Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
    chamado, não pelo compilador Java.

71      FROM voo v JOIN rota r ON r.id=v.rota_id JOIN aeronave a ON a.id=v.aeronave_id JOIN
modelo_aeronave m ON m.id=a.modelo_id WHERE v.id=?
    Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
    chamado, não pelo compilador Java.

```

```
72         """, voo);
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
73         var r=Sql.registro(c,"SELECT * FROM recurso_aeroporto WHERE id=? AND
situacao='DISPONIVEL'",recurso);
```

Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: recurso\_aeroporto. Parâmetros ligados em ordem: recurso.

```
74         Dados.exigir(!Set.of("CANCELADO","CONCLUIDO").contains(v.get("situacao")), "Voo encerrado.");
```

Regra obrigatória: !Set.of("CANCELADO","CONCLUIDO").contains(v.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Voo encerrado.". Essa condição específica não é substituída por outra validação parecida.

```
75         long aeroporto=finalidade.equals("CHEGADA")?Dados.id(v,"destino_id"):Dados.id(v,"origem_id");
```

Declara variável local aeroporto do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Escolha de valor: finalidade.equals("CHEGADA"). true escolhe Dados.id(v,"destino\_id"); false escolhe Dados.id(v,"origem\_id"). Só a alternativa escolhida é avaliada.

```
76         Dados.exigir(Dados.id(r,"aeroporto_id")==aeroporto,"Recurso em outro aeroporto.");
```

Regra obrigatória: Dados.id(r,"aeroporto\_id")==aeroporto deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Recurso em outro aeroporto.". Essa condição específica não é substituída por outra validação parecida.

```
77         Sql.registro(c,"SELECT id FROM aeroporto WHERE id=? AND ativo",aeroporto);
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeroporto. Parâmetros ligados em ordem: aeroporto.

```
78         if(r.get("terminal_id")!=null) Sql.registro(c,"SELECT id FROM terminal WHERE id=? AND
ativo",Dados.id(r,"terminal_id"));
```

Decisão específica: (r.get("terminal\_id")!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: terminal. Parâmetros ligados em ordem: Dados.id(r,"terminal\_id").

```
79         if(r.get("envergadura_max_m")!=null) Dados.exigir(Dados.dinheiro(r,"envergadura_max_m").compareTo(
Dados.dinheiro(v,"envergadura_m"))>=0,"Envergadura incompativel.");
```

Decisão específica: (r.get("envergadura\_max\_m")!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: Dados.dinheiro(r,"envergadura\_max\_m").compareTo(Dados.dinheiro(v,"envergadura\_m"))>=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Envergadura incompativel.". Essa condição específica não é substituída por outra validação parecida.

```
80         if("PISTA".equals(r.get("tipo"))) Dados.exigir(Dados.dinheiro(r,"comprimento_m").compareTo(Dados.d
inheiro(v,"comprimento_pista_min_m"))>=0,"Pista muito curta.");
```

Decisão específica: ("PISTA".equals(r.get("tipo))). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: Dados.dinheiro(r,"comprimento\_m").compareTo(Dados.dinheiro(v,"comprimento\_pista\_min\_m"))>=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Pista muito curta.". Essa condição específica não é substituída por outra validação parecida.

```
81     }
```

```
82 }
```

## OperacaoSql

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/OperacaoSql.java

Interface funcional genérica que permite passar c -> {...} ao BancoDados. O T é o tipo do resultado da operação.

9 linhas | 1 métodos/construtores declarados | SHA-256: 5a0cc21df3fa43f6...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
```

Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

```
2
3 import java.sql.Connection;
4 import java.sql.SQLException;
5
6 @FunctionalInterface
```

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

```
7 public interface OperacaoSql<T> {
```

Declara OperacaoSql como contrato de operações.

### executar - L8 a L8

Único método abstrato da interface funcional. Recebe Connection e devolve T; pode lançar SQLException. Uma lambda de consulta/transação fornece o corpo, e BancoDados decide quando executá-la e confirmar/desfazer.

Entradas declaradas: (Connection conexao).

Saída declarada: T - tipo genérico escolhido pelo contexto da chamada.

Possíveis chamadores identificados: BancoDados.executar (L36).

```
8     T executar(Connection conexao) throws SQLException;
```

Início da declaração de executar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
9 }
```

## PassageiroJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/PassageiroJdbc.java

Implementa PassageiroRepositorio. Faz as escritas em pessoa, passageiro, documentos e contatos usando a conexão fornecida pelo serviço.

81 linhas | 7 métodos/construtores declarados | SHA-256: 21da02d0438c93ed...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
   Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.
```

```
2
3 import br.edu.aeroporto.dominio.CadastroPassageiro;
4 import br.edu.aeroporto.dominio.Passageiro;
5 import br.edu.aeroporto.execcao.RegraNegocioException;
6 import br.edu.aeroporto.repositorio.PassageiroRepositorio;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Contrato para cadastrar, vincular, editar e buscar passageiros. Seus dois buscar são sobrecargas com parâmetros e retornos diferentes.

```
7 import java.sql.Connection;
8 import java.sql.ResultSet;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. cursor das linhas retornadas pelo SQL

```
9 import java.sql.SQLException;
10 import java.time.LocalDate;
11 import java.util.List;
12 import java.util.Optional;
13 import java.util.UUID;
```

```
14
15 public final class PassageiroJdbc implements PassageiroRepositorio {
   Declara PassageiroJdbc como tipo com campos e métodos. final impede herança. implements liga o tipo aos contratos citados.
```

```
16     private static final String SELECT = ""
   Campo(s) SELECT: private restringe acesso à classe. Tipo String: texto imutável. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.
```

```
17         SELECT p.id, p.nome, p.nascimento, p.ativo AND pa.ativo AS ativo, pa.codigo_cliente
   Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.
```

```
18         FROM pessoa p JOIN passageiro pa ON pa.pessoa_id = p.id
   Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.
```

```
19         """;
   Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.
```

```
20
```

### vincular - L21 a L27

Bloqueia pessoa ativa; insere papel com ON CONFLICT DO NOTHING e consulta resultado. Repetição não cria segunda pessoa/papel.

Entradas declaradas: (Connection c, long pessoald, String assistencia).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Sql.registro, UUID.randomUUID().toString().replace("-", "").substring(0,20).toUpperCase, UUID.randomUUID().toString().replace("-", "").substring, UUID.randomUUID().toString().replace, UUID.randomUUID().toString, UUID.randomUUID, Sql.executar, buscar(c,pessoald).orElseThrow, buscar.

```
21     @Override
   Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.
```

```
22     public Passageiro vincular(Connection c, long pessoaId, String assistencia) throws SQLException {
   Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.
```



```

23      Sql.registro(c, "SELECT id FROM pessoa WHERE id=? AND ativo FOR UPDATE", pessoaId);
      Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
      pessoa. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: pessoald.

24      String codigo="P"+UUID.randomUUID().toString().replace("-",
      "").substring(0,20).toUpperCase(java.util.Locale.ROOT);
      Declara variável local código do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

25      Sql.executar(c,"INSERT INTO passageiro(pessoa_id,codigo_cliente,observacoes_assistencia) VALUES
      (?,?,?) ON CONFLICT (pessoa_id) DO NOTHING",pessoaId,codigo,assistencia);
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: passageiro. ON
      CONFLICT define o tratamento da colisão descrita no SQL. Parâmetros ligados em ordem: pessoald, codigo, assistencia.

26      return buscar(c,pessoaId).orElseThrow();
      Retorno: entrega buscar(c,pessoald).orElseThrow() ao chamador desta execução; nenhuma instrução posterior desse mesmo
      corpo executa por esse caminho.

      Chama buscar(c,pessoald).orElseThrow: Extrai o Optional ou lança a exceção fornecida quando estiver vazio.

      Chama PassageiroJdbc.buscar: Sobrecarga por id retorna Optional; por nome retorna lista com limite/deslocamento. strpos trata
      %/_ literalmente, não como coringas de LIKE.

27      }
28

```

## editar - L29 a L35

Bloqueia papel, atualiza dados comuns da pessoa e dados do passageiro em comandos distintos; a transação do serviço garante ambos ou nenhum.

Entradas declaradas: (Connection c,long id,String nome,LocalDate nascimento,Long nacionalidade,String assistencia,boolean ativo).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Sql.registro, Sql.executar, buscar(c,id).orElseThrow, buscar.

```

29      @Override
      Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a
      anotação mostrada.

30      public Passageiro editar(Connection c,long id,String nome,LocalDate nascimento,Long
      nacionalidade,String assistencia,boolean ativo) throws SQLException {
      Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

31      Sql.registro(c,"SELECT pessoa_id FROM passageiro WHERE pessoa_id=? FOR UPDATE",id);
      Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
      passageiro. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

32      Sql.executar(c,"UPDATE pessoa SET nome=?,nascimento=?,nacionalidade_id=? WHERE
      id=?",nome,nascimento,nacionalidade,id);
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: pessoa. Parâmetros
      ligados em ordem: nome, nascimento, nacionalidade, id.

33      Sql.executar(c,"UPDATE passageiro SET observacoes_assistencia=?,ativo=? WHERE
      pessoa_id=?",assistencia,ativo,id);
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: passageiro. Parâmetros
      ligados em ordem: assistencia, ativo, id.

34      return buscar(c,id).orElseThrow();
      Retorno: entrega buscar(c,id).orElseThrow() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo
      executa por esse caminho.

      Chama buscar(c,id).orElseThrow: Extrai o Optional ou lança a exceção fornecida quando estiver vazio.

      Chama PassageiroJdbc.buscar: Sobrecarga por id retorna Optional; por nome retorna lista com limite/deslocamento. strpos trata
      %/_ literalmente, não como coringas de LIKE.

35      }
36

```

## cadastrar - L37 a L54

INSERT pessoa retorna id; papel usa esse id, CPF opcional referencia BR e contatos opcionais são inseridos. Devolve objeto Passageiro.



Entradas declaradas: (Connection conexao, CadastroPassageiro cadastro).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: ArquivosServico.importarPassageiros (L24).

Chamadas observadas no corpo: Sql.inserir, cadastro.nome, cadastro.nascimento, UUID.randomUUID().toString().replace("-", "").substring(0, 20).toUpperCase, UUID.randomUUID().toString().replace("-", "").substring, UUID.randomUUID().toString().replace, UUID.randomUUID().toString, UUID.randomUUID, Sql.executar, cadastro.cpf().isEmpty, cadastro.cpf, Sql.unico(conexao, "SELECT id FROM pais WHERE codigo = 'BR'", linha -> linha.getLong(1)).orElseThrow, Sql.unico, linha.getLong, new RegraNegocioException, salvarContato, cadastro.email, cadastro.telefone, new Passageiro.

37       @Override

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

38       public Passageiro cadastrar(Connection conexao, CadastroPassageiro cadastro) throws SQLException {

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

39           long id = Sql.inserir(conexao, "INSERT INTO pessoa(nome, nascimento) VALUES (?, ?) RETURNING id",

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: pessoa. Parâmetros ligados em ordem: cadastro.nome(), cadastro.nascimento().

40           cadastro.nome(), cadastro.nascimento());

Nesta parte da expressão, chama cadastro.nome, cadastro.nascimento. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

41           String codigo = "P" + UUID.randomUUID().toString().replace("-", "").substring(0, 20).toUpperCase(java.util.Locale.ROOT);

Declara variável local codigo do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

42           Sql.executar(conexao, "INSERT INTO passageiro(pessoa\_id, codigo\_cliente) VALUES (?, ?)", id, codigo);

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: passageiro. Parâmetros ligados em ordem: id, codigo.

43           if (!cadastro.cpf().isEmpty()) {

Decisão específica: (!cadastro.cpf().isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

44           long brasil = Sql.unico(conexao, "SELECT id FROM pais WHERE codigo = 'BR'", linha -> linha.getLong(1))

Declara variável local brasil do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: pais.

45           .orElseThrow(() -> new RegraNegocioException("Cadastre o país BR antes de registrar CPF."));

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

46           Sql.executar(conexao, ""

Instrução idêntica à de AuditoriaJdbc:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

47           INSERT INTO documento\_pessoa(pessoa\_id, pais\_emissor\_id, tipo, numero)

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

48           VALUES (?, ?, 'CPF', ?)

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

49           "", id, brasil, cadastro.cpf());

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

50           }

51           salvarContato(conexao, id, "EMAIL", cadastro.email());

Chama PassageiroJdbc.salvarContato: Texto vazio retorna sem INSERT; presente grava contato principal do tipo correspondente.

52           salvarContato(conexao, id, "TELEFONE", cadastro.telefone());

Chama PassageiroJdbc.salvarContato: Texto vazio retorna sem INSERT; presente grava contato principal do tipo correspondente.

```
53         return new Passageiro(id, cadastro.nome(), cadastro.nascimento(), true, codigo);
```

Retorno: entrega new Passageiro(id, cadastro.nome(), cadastro.nascimento(), true, codigo) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói Passageiro: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
54     }
```

```
55
```

## buscar - L56 a L59

Sobrecarga por id retorna Optional; por nome retorna lista com limite/deslocamento. strpos trata %/\_ literalmente, não como coringas de LIKE.

Entradas declaradas: (Connection conexao, long id).

Saída declarada: Optional<Passageiro> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: PassageiroJdbc.editar (L29); PassageiroJdbc.vincular (L21).

Chamadas observadas no corpo: Sql.unico, PassageiroJdbc::mapear.

```
56     @Override
```

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

```
57     public Optional<Passageiro> buscar(Connection conexao, long id) throws SQLException {
```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
58         return Sql.unico(conexao, SELECT + " WHERE p.id = ?", PassageiroJdbc::mapear, id);
```

Retorno: entrega Sql.unico(conexao, SELECT + " WHERE p.id = ?", PassageiroJdbc::mapear, id) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Parâmetros ligados em ordem: id.

```
59     }
```

```
60
```

## buscar - L61 a L68

Sobrecarga por id retorna Optional; por nome retorna lista com limite/deslocamento. strpos trata %/\_ literalmente, não como coringas de LIKE.

Entradas declaradas: (Connection conexao, String nome, int limite, int deslocamento).

Saída declarada: List<Passageiro> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: PassageiroJdbc.editar (L29); PassageiroJdbc.vincular (L21).

Chamadas observadas no corpo: Sql.listar, PassageiroJdbc::mapear.

```
61     @Override
```

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

```
62     public List<Passageiro> buscar(Connection conexao, String nome, int limite, int deslocamento) throws
    SQLException {
```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
63         // strpos trata o filtro como texto literal, inclusive caracteres % e _.
```

Comentário/documentação do autor; descreve intenção ou convenção e não executa.

```
64         return Sql.listar(conexao, SELECT + ""
```

Retorno: entrega Sql.listar(conexao, SELECT + "" WHERE strpos(lower(p.nome), lower(?)) > 0 ORDER BY p.nome, p.id LIMIT ? OFFSET ? "", PassageiroJdbc::mapear, nome, limite, deslocamento) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Parâmetros ligados em ordem: nome, limite, deslocamento.

```
65         WHERE strpos(lower(p.nome), lower(?)) > 0
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
66         ORDER BY p.nome, p.id LIMIT ? OFFSET ?
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```

67         """ , PassageiroJdbc::mapear, nome, limite, deslocamento);
    Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu
    uso.

68     }
69

```

## mapear - L70 a L73

Transforma cada coluna em atributo do construtor de Passageiro; não faz outra consulta.

Entradas declaradas: (ResultSet linha).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: PassageiroJdbc.buscar (L56); PassageiroJdbc.buscar (L61).

Chamadas observadas no corpo: new Passageiro, linha.getLong, linha.getString, linha.getObject, linha.getBoolean.

```

70     private static Passageiro mapear(ResultSet linha) throws SQLException {
    Início da declaração de mapear. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

71         return new Passageiro(linha.getLong("id"), linha.getString("nome"),
    Retorno: entrega new Passageiro(linha.getLong("id"), linha.getString("nome"), linha.getObject("nascimento", LocalDate.class),
    linha.getBoolean("ativo"), linha.getString("codigo_cliente")) ao chamador desta execução; nenhuma instrução posterior desse
    mesmo corpo executa por esse caminho.

    new constrói Passageiro: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si
    só.

72         linha.getObject("nascimento", LocalDate.class), linha.getBoolean("ativo"),
    linha.getString("codigo_cliente"));
    Nesta parte da expressão, chama linha.getObject, linha.getBoolean, linha.getString. Os valores retornados alimentam a instrução
    que começou nesta linha ou nas linhas anteriores.

73     }
74

```

## salvarContato - L75 a L80

Texto vazio retorna sem INSERT; presente grava contato principal do tipo correspondente.

Entradas declaradas: (Connection conexao, long id, String tipo, String valor).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: PassageiroJdbc.cadastrar (L37).

Chamadas observadas no corpo: valor.isEmpty, Sql.executar.

```

75     private static void salvarContato(Connection conexao, long id, String tipo, String valor) throws
    SQLException {
    Início da declaração de salvarContato. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
    é invocado.

76         if (valor.isEmpty()) return;
    Decisão específica: (valor.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a
    execução atual com return.

    return encerra o caminho atual sem entregar valor (void).

77         Sql.executar(conexao, """
    Instrução idêntica à de AuditoriaJdbc:L9. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
    execução. A posição e o contexto do método acima mostram quando ela ocorre.

78         INSERT INTO contato_pessoa(pessoa_id, tipo, valor, principal) VALUES (?, ?, ?, TRUE)
    Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
    chamado, não pelo compilador Java.

79         """ , id, tipo, valor);
    Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu
    uso.

80     }
81 }

```

## PlanejamentoJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/PlanejamentoJdbc.java

Persiste programação, inventário, tarifa e agendas. Verifica continuidade física da aeronave entre os aeroportos e a margem de 75 minutos.

97 linhas | 7 métodos/construtores declarados | SHA-256: 55fe312bf66c2505...

```
1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import java.sql.Connection;
5 import java.sql.SQLException;
6 import java.time.Duration;
7 import java.time.Instant;
8 import java.util.Map;
9
10 public final class PlanejamentoJdbc {
    Declara PlanejamentoJdbc como tipo com campos e métodos. final impede herança.
```

### serializar - L11 a L13

Obtém advisory lock transacional da operação; só colaboradores que usam a mesma chave participam dessa serialização.

Entradas declaradas: (Connection c).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L23); OperacaoServico.alocar (L24); OperacaoServico.escalar (L70); OperacaoServico.habilitar (L56); OperacaoServico.interditar (L33); OperacaoServico.liberarAlocacao (L43); OperacaoServico.liberarEscala (L81); OperacaoServico.manutencao (L94); OperacaoServico.servicoSolo (L130); OperacaoServico.situacaoManutencao (L107); OperacaoServico.situacaoSolo (L144); PlanejamentoServico.horarios (L71); PlanejamentoServico.programar (L20); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.trocarAeronave (L93); VooServico.transicao (L22).

Chamadas observadas no corpo: Sql.unico.

```
11 public void serializar(Connection c) throws SQLException {
    Início da declaração de serializar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

12     Sql.unico(c,"SELECT pg_advisory_xact_lock(hashtext('aeroporto.operacao'))",r->true);
    Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado.

13 }
14
```

### validarFrota - L15 a L41

Exige aeronave/companhia/modelo/rota/localidades ativos. Verifica voo anterior e próximo, continuidade de aeroporto e 75 min; ignorar exclui o próprio voo ao editar.

Entradas declaradas: (Connection c,long aeronave,long companhia,long rota,Instant partida,Instant chegada,Long ignorar).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: PlanejamentoJdbc.programar (L43); PlanejamentoServico.horarios (L71); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.trocarAeronave (L93).

Chamadas observadas no corpo: Sql.registro, Dados.exigir, Dados.id, Sql.unico, Sql::linha, anterior.isPresent, anterior.get, Dados.instante(anterior.get(),"chegada").plus(Duration.ofMinutes(75)).isAfter, Dados.instante(anterior.get(),"chegada").plus, Dados.instante, Duration.ofMinutes, proximo.isPresent, proximo.get, chegada.plus(Duration.ofMinutes(75)).isAfter, chegada.plus.

```
15 public void validarFrota(Connection c,long aeronave,long companhia,long rota,Instant partida,Instant
    chegada,Long ignorar) throws SQLException {
    Início da declaração de validarFrota. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

16     var a=Sql.registro(c,"SELECT * FROM aeronave WHERE id=? AND situacao='ATIVA' FOR
    UPDATE",aeronave);
    Declara variável local a com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeronave. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: aeronave.

```
17      Dados.exigir(Dados.id(a,"companhia_id")==companhia,"A aeronave deve pertencer a companhia do
voo.");
```

Regra obrigatória: Dados.id(a,"companhia\_id")==companhia deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "A aeronave deve pertencer a companhia do voo.". Essa condição específica não é substituída por outra validação parecida.

```
18      Sql.registro(c,"SELECT id FROM companhia_aerea WHERE id=? AND ativo",companhia);
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: companhia\_aerea. Parâmetros ligados em ordem: companhia.

```
19      Sql.registro(c,"SELECT id FROM modelo_aeronave WHERE id=? AND ativo",Dados.id(a,"modelo_id"));
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: modelo\_aeronave. Parâmetros ligados em ordem: Dados.id(a,"modelo\_id").

```
20      var r=Sql.registro(c,"SELECT r.* FROM rota r JOIN aeroporto o ON o.id=r.origem_id JOIN aeroporto d
ON d.id=r.destino_id WHERE r.id=? AND r.ativa AND o.ativo AND d.ativo",rota);
```

Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: rota, aeroporto. Parâmetros ligados em ordem: rota.

```
21      var anterior=Sql.unico(c,""
```

Instrução idêntica à de OperacaoJdbc:L41. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
22      SELECT r.destino_id,COALESCE(v.chegada_real,v.chegada_estimada,v.chegada_prevista) AS
chegada
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
23      FROM voo v JOIN rota r ON r.id=v.rota_id WHERE v.aeronave_id=? AND v.situacao<>'CANCELADO'
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
24      AND v.id<>COALESCE(?,0) AND
COALESCE(v.partida_real,v.partida_estimada,v.partida_prevista)<?
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
25      ORDER BY COALESCE(v.partida_real,v.partida_estimada,v.partida_prevista) DESC LIMIT 1
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
26      """,Sql::linha,aeronave,ignorar,partida);
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
27      if(anterior.isPresent()) {
```

Decisão específica: (anterior.isPresent()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
28      Dados.exigir(Dados.id(anterior.get(),"destino_id")==Dados.id(r,"origem_id"),"Aeronave nao
chega a origem deste voo.");
```

Regra obrigatória: Dados.id(anterior.get(),"destino\_id")==Dados.id(r,"origem\_id") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Aeronave nao chega a origem deste voo.". Essa condição específica não é substituída por outra validação parecida.

```
29      Dados.exigir(!Dados.instante(anterior.get(),"chegada").plus(Duration.ofMinutes(75)).isAfter(pa
rtida),"Margem entre voos insuficiente.");
```

Regra obrigatória: !Dados.instante(anterior.get(),"chegada").plus(Duration.ofMinutes(75)).isAfter(partida) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Margem entre voos insuficiente.". Essa condição específica não é substituída por outra validação parecida.

```
30      }
```

```
31      var proximo=Sql.unico(c,""
```

Declara variável local proximo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: voo, rota. Parâmetros ligados em ordem: aeronave, ignorar, partida.

```
32      SELECT r.origem_id, COALESCE(v.partida_real, v.partida_estimada, v.partida_prevista) AS
partida
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
33      FROM voo v JOIN rota r ON r.id=v.rota_id WHERE v.aeronave_id=? AND v.situacao<>'CANCELADO'
```

Instrução idêntica à de PlanejamentoJdbc:L23. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
34      AND v.id<>COALESCE(?,0) AND
COALESCE(v.partida_real, v.partida_estimada, v.partida_prevista)>=?
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
35      ORDER BY COALESCE(v.partida_real, v.partida_estimada, v.partida_prevista) LIMIT 1
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
36      """, Sql::linha, aeronave, ignorar, partida);
```

Instrução idêntica à de PlanejamentoJdbc:L26. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
37      if(proximo.isPresent()) {
```

Decisão específica: (proximo.isPresent()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
38      Dados.exigir(Dados.id(proximo.get(), "origem_id")==Dados.id(r, "destino_id"), "Proximo voo inicia
em outro aeroporto.");
```

Regra obrigatória: Dados.id(proximo.get(), "origem\_id")==Dados.id(r, "destino\_id") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Proximo voo inicia em outro aeroporto.". Essa condição específica não é substituída por outra validação parecida.

```
39      Dados.exigir(!chegada.plus(Duration.ofMinutes(75)).isAfter(Dados.instante(proximo.get(), "parti
da")), "Margem antes do proximo voo insuficiente.");
```

Regra obrigatória: !chegada.plus(Duration.ofMinutes(75)).isAfter(Dados.instante(proximo.get(), "partida")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Margem antes do proximo voo insuficiente.". Essa condição específica não é substituída por outra validação parecida.

```
40      }
```

```
41      }
```

```
42
```

## programar - L43 a L52

Valida frota, insere voo RETURNING id, agenda de 45 min antes/30 depois e inventário. Retorna id só após todas as chamadas terminarem.

Entradas declaradas: (Connection c, ProgramacaoVoo v).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L23); PlanejamentoServico.programar (L20).

Chamadas observadas no corpo: validarFrota, v.aeronave, v.companhia, v.rota, v.partida, v.chegada, Sql.inserir, v.numero, v.checkinAbre, v.checkinFecha, v.embarqueAbre, v.embarqueFecha, Sql.executar, v.partida().minus, Duration.ofMinutes, v.chegada().plus, inventario.

```
43      public long programar(Connection c, ProgramacaoVoo v) throws SQLException {
```

Início da declaração de programar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
44      validarFrota(c, v.aeronave(), v.companhia(), v.rota(), v.partida(), v.chegada(), null);
```

Chama PlanejamentoJdbc.validarFrota: Exige aeronave/companhia/modelo/rota/localidades ativos. Verifica voo anterior e próximo, continuidade de aeroporto e 75 min; ignorar exclui o próprio voo ao editar.

```
45      long id=Sql.inserir(c, ""
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: voo. Parâmetros ligados em ordem: v.companhia(), v.rota(), v.aeronave(), v.numero(), v.partida(), v.chegada(), v.checkinAbre(), v.checkinFecha(), v.embarqueAbre(), v.embarqueFecha().

```
46      INSERT INTO voo(companhia_id, rota_id, aeronave_id, numero, partida_prevista, chegada_prevista,
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```

47      checkin_abre,checkin_fecha,embarque_abre,embarque_fecha) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
RETURNING id

```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```

48      """ ,v.companhia(),v.rota(),v.aeronave(),v.numero(),v.partida(),v.chegada(),v.checkinAbre()
,v.checkinFecha(),v.embarqueAbre(),v.embarqueFecha());

```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```

49      Sql.executar(c,"INSERT INTO agenda_aeronave(aeronave_id,voo_id,inicio,fim) VALUES (?, ?, ?, ?)",v.aer
onave(),id,v.partida().minus(Duration.ofMinutes(45)),v.chegada().plus(Duration.ofMinutes(30)));

```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: agenda\_aeronave. Parâmetros ligados em ordem: v.aeronave(), id, v.partida().minus(Duration.ofMinutes(45)), v.chegada().plus(Duration.ofMinutes(30)).

```

50      inventario(c,id,v.aeronave());

```

Chama PlanejamentoJdbc.inventario: INSERT SELECT copia assentos físicos ativos para o voo; exige ao menos uma linha afetada.

```

51      return id;

```

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```

52      }
53

```

## inventario - L54 a L60

INSERT SELECT copia assentos físicos ativos para o voo; exige ao menos uma linha afetada.

Entradas declaradas: (Connection c,long voo,long aeronave).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: PlanejamentoJdbc.programar (L43); PlanejamentoServico.trocarAeronave (L93).

Chamadas observadas no corpo: Sql.executar, Dados.exigir.

```

54      public void inventario(Connection c,long voo,long aeronave) throws SQLException {

```

Início da declaração de inventario. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

55      int quantidade=Sql.executar(c,""

```

Declara variável local quantidade do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: inventario\_assento\_voo, assento\_aeronave. Parâmetros ligados em ordem: voo, aeronave.

```

56      INSERT INTO inventario_assento_voo(voo_id,aeronave_id,assento_id,codigo,classe)

```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```

57      SELECT ?,aeronave_id,id,codigo,classe FROM assento_aeronave WHERE aeronave_id=? AND ativo
ORDER BY fila,coluna

```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```

58      """ ,voo,aeronave);

```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```

59      Dados.exigir(quantidade>0,"Configure pelo menos um assento fisico antes de programar.");

```

Regra obrigatória: quantidade>0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Configure pelo menos um assento fisico antes de programar.". Essa condição específica não é substituída por outra validação parecida.

```

60      }
61

```

## tarifa - L62 a L74

Se já vendida, preserva classe. id null cria; id presente atualiza tarifa do voo e exige uma linha. Itens vendidos têm cópia dos valores.

Entradas declaradas: (Connection c,long voo,Long id,Tarifa t,boolean ativa).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L23); PlanejamentoServico.tarifa (L30).



Chamadas observadas no corpo: `Sql.listar(c,"SELECT id FROM item_reserva WHERE tarifa_id=?",r->r.getLong(1),id).isEmpty`, `Sql.listar`, `r.getLong`, `Dados.exigir`, `t.classe().equals`, `t.classe`, `Sql.registro(c,"SELECT classe FROM tarifa_voo WHERE id=? AND voo_id=?",id,voo).get`, `Sql.registro`, `Sql.inserir`, `t.codigo`, `t.base`, `t.taxa`, `t.franquia`, `t.pecas`, `t.permiteCancelar`, `t.multa`, `Sql.executar`.

62     `public long tarifa(Connection c,long voo,Long id,Tarifa t,boolean ativa) throws SQLException {`  
     Início da declaração de tarifa. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

63     `if(id!=null && !Sql.listar(c,"SELECT id FROM item_reserva WHERE`  
     `tarifa_id=?",r->r.getLong(1),id).isEmpty()) {`  
     Decisão específica: `(id!=null && !Sql.listar(c,"SELECT id FROM item_reserva WHERE tarifa_id=?",r->r.getLong(1),id).isEmpty())`. Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve `List<T>`. Tabelas mencionadas: `item_reserva`. Parâmetros ligados em ordem: `id`.

64     `Dados.exigir(t.classe().equals(Sql.registro(c,"SELECT classe FROM tarifa_voo WHERE id=? AND`  
     `voo_id=?",id,voo).get("classe")), "Classe vendida nao pode ser alterada.");`  
     Regra obrigatória: `t.classe().equals(Sql.registro(c,"SELECT classe FROM tarifa_voo WHERE id=? AND voo_id=?",id,voo).get("classe"))` deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Classe vendida nao pode ser alterada.". Essa condição específica não é substituída por outra validação parecida.

Consulta uma linha como `Map<String,Object>`; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: `tarifa_voo`. Parâmetros ligados em ordem: `id`, `voo`.

65     `}`  
     `if(id==null) return Sql.inserir(c,""`  
     Decisão específica: `(id==null)`. Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com `return`.

Retorno: entrega `Sql.inserir(c,"" INSERT INTO tarifa_voo(voo_id,codigo,classe,valor_base,taxa_embarque,franquia_bagagem_kg,limite_pecas,permite_cancelar,multa_cancelamento,ativa) VALUES (?,?,?,?,?,?,?,?,?) RETURNING id ""`, `voo`, `t.co...` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa inserção com retorno de identificador; o `long` recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: `tarifa_voo`. Parâmetros ligados em ordem: `voo`, `t.codigo()`, `t.classe()`, `t.base()`, `t.taxa()`, `t.franquia()`, `t.pecas()`, `t.permiteCancelar()`, `t.multa()`, `ativa`.

67     `INSERT INTO tarifa_voo(voo_id,codigo,classe,valor_base,taxa_embarque,franquia_bagagem_kg,1`  
     `imite_pecas,permite_cancelar,multa_cancelamento,ativa)`  
     Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

68     `VALUES (?,?,?,?,?,?,?,?,?) RETURNING id`  
     Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

69     `""`, `voo`, `t.codigo()`, `t.classe()`, `t.base()`, `t.taxa()`, `t.franquia()`, `t.pecas()`, `t.permiteCancelar()`  
     `,t.multa(),ativa);`  
     Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

70     `Dados.exigir(Sql.executar(c,""`  
     Regra obrigatória: `Sql.executar(c,"" UPDATE tarifa_voo SET codigo=?,classe=?,valor_base=?,taxa_embarque=?,franquia_baga gem_kg=?,limite_pecas=?,permite_cancelar=?,multa_cancelamento=?,ativa=? WHERE id=? AND voo_id=? ""`, `t.codigo()`, `t.classe()`, `t.base()`, `t.taxa()`, `t.franquia()`, `t.pecas()`, `t.permiteCancelar()`, `t.multa()`, `ativa`, `id`, `voo`)==1 deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Tarifa nao encontrada.". Essa condição específica não é substituída por outra validação parecida.

Executa comando de alteração e devolve `int` com a quantidade de linhas afetadas. Tabelas mencionadas: `tarifa_voo`. Parâmetros ligados em ordem: `t.codigo()`, `t.classe()`, `t.base()`, `t.taxa()`, `t.franquia()`, `t.pecas()`, `t.permiteCancelar()`, `t.multa()`, `ativa`, `id`, `voo`.

71     `UPDATE tarifa_voo SET codigo=?,classe=?,valor_base=?,taxa_embarque=?,franquia_bagagem_kg=?`  
     `,limite_pecas=?,permite_cancelar=?,multa_cancelamento=?,ativa=? WHERE id=? AND voo_id=?`  
     Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

72     `""`, `t.codigo()`, `t.classe()`, `t.base()`, `t.taxa()`, `t.franquia()`, `t.pecas()`, `t.permiteCancelar()`, `t.m`  
     `ulta(),ativa,id,voo)==1,"Tarifa nao encontrada.");`  
     Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

73     `return id;`  
     Retorno: entrega `id` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

74     `}`  
     75



## agendas - L76 a L91

Calcula deltas de partida/chegada e desloca agenda de aeronave, recursos, escalas e serviços pendentes; CHEGADA usa delta de chegada.

Entradas declaradas: (Connection c, Map<String, Object> anterior, Instant partida, Instant chegada).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: PlanejamentoServico.horarios (L71); PlanejamentoServico.reprogramar (L52); VooServico.transicao (L22).

Chamadas observadas no corpo: Dados.id, efetivo, Duration.between, Sql.executar, partida.minus, Duration.ofMinutes, chegada.plus, Sql.listar, Sql::linha, "CHEGADA".equals, alocao.get, Dados.instante(alocacao,"inicio").plus, Dados.instante, Dados.instante(alocacao,"fim").plus, Dados.instante(escala,"inicio").plus, Dados.instante(escala,"fim").plus, Dados.instante(solo,"inicio").plus, solo.get, Dados.instante(solo,"fim").plus.

```

76     public void agendas(Connection c, Map<String, Object> anterior, Instant partida, Instant chegada) throws
       SQLException {
           Início da declaração de agendas. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
           invocado.

77         long voo=Dados.id(anterior,"id");
           Declara variável local voo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

78         Instant velhaPartida=efetivo(anterior,"partida"),velhaChegada=efetivo(anterior,"chegada");
           Declara variável local velhaPartida, velhaChegada do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o
           escopo é o bloco atual.

79         Duration
           deltaPartida=Duration.between(velhaPartida,partida),deltaChegada=Duration.between(velhaChegada,chegada);
           Declara variável local deltaPartida, deltaChegada do tipo Duration. O valor é obtido pela expressão de inicialização mostrada; o
           escopo é o bloco atual.

80         Sql.executar(c,"UPDATE agenda_aeronave SET inicio=?,fim=? WHERE voo_id=? AND
           ativa",partida.minus(Duration.ofMinutes(45)),chegada.plus(Duration.ofMinutes(30)),voo);
           Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: agenda_aeronave.
           Parâmetros ligados em ordem: partida.minus(Duration.ofMinutes(45)), chegada.plus(Duration.ofMinutes(30)), voo.

81         for(var alocao:Sql.listar(c,"SELECT * FROM alocao_recurso WHERE voo_id=? AND ativa ORDER BY id
           FOR UPDATE",Sql::linha,voo)) {
           Percorre cada elemento da coleção/vetor: for(var alocao:Sql.listar(c,"SELECT * FROM alocao_recurso WHERE voo_id=?
           AND ativa ORDER BY id FOR UPDATE",Sql::linha,voo)). return, break ou exceção podem encerrar esse caminho; continue passa
           à próxima volta.

           Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: alocao_recurso. O FOR
           UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: voo.

82             Duration delta="CHEGADA".equals(alocacao.get("finalidade"))?deltaChegada:deltaPartida;
           Declara variável local delta do tipo Duration. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

           Escolha de valor: "CHEGADA".equals(alocacao.get("finalidade")). true escolhe deltaChegada; false escolhe deltaPartida. Só a
           alternativa escolhida é avaliada.

83             Sql.executar(c,"UPDATE alocao_recurso SET inicio=?,fim=? WHERE id=?",Dados.instante(alocacao
           ,"inicio").plus(delta),Dados.instante(alocacao,"fim").plus(delta),Dados.id(alocacao,"id"));
           Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: alocao_recurso.
           Parâmetros ligados em ordem: Dados.instante(alocacao,"inicio").plus(delta), Dados.instante(alocacao,"fim").plus(delta),
           Dados.id(alocacao,"id").

84         }
85         for(var escala:Sql.listar(c,"SELECT * FROM escala_funcionario WHERE voo_id=? AND ativa ORDER BY id
           FOR UPDATE",Sql::linha,voo)) {
           Percorre cada elemento da coleção/vetor: for(var escala:Sql.listar(c,"SELECT * FROM escala_funcionario WHERE voo_id=? AND
           ativa ORDER BY id FOR UPDATE",Sql::linha,voo)). return, break ou exceção podem encerrar esse caminho; continue passa à
           próxima volta.

           Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: escala_funcionario. O FOR
           UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: voo.

86             Sql.executar(c,"UPDATE escala_funcionario SET inicio=?,fim=? WHERE id=?",Dados.instante(escala
           ,"inicio").plus(deltaPartida),Dados.instante(escala,"fim").plus(deltaChegada),Dados.id(escala,"id"));
           Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: escala_funcionario.
           Parâmetros ligados em ordem: Dados.instante(escala,"inicio").plus(deltaPartida),
           Dados.instante(escala,"fim").plus(deltaChegada), Dados.id(escala,"id").

87         }

```

```
88         for(var solo:Sql.listar(c,"SELECT * FROM servico_solo WHERE voo_id=? AND situacao='PENDENTE' AND
            inicio IS NOT NULL ORDER BY id",Sql::linha,voo)) {
            Percorre cada elemento da coleção/vetor: for(var solo:Sql.listar(c,"SELECT * FROM servico_solo WHERE voo_id=? AND
            situacao='PENDENTE' AND inicio IS NOT NULL ORDER BY id",Sql::linha,voo)). return, break ou exceção podem encerrar esse
            caminho; continue passa à próxima volta.
```

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: servico\_solo. Parâmetros ligados em ordem: voo.

```
89         Sql.executar(c,"UPDATE servico_solo SET inicio=?,fim=? WHERE id=?",Dados.instante(solo,"inicio
            ").plus(deltaPartida),solo.get("fim")==null?null:Dados.instante(solo,"fim").plus(deltaPartida),Dados.id(solo,"id"));
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: servico\_solo. Parâmetros ligados em ordem: Dados.instante(solo,"inicio").plus(deltaPartida), solo.get("fim")==null?null:Dados.instante(solo,"fim").plus(deltaPartida), Dados.id(solo,"id").

Escolha de valor: solo.get("fim")==null. true escolhe null; false escolhe Dados.instante(solo,"fim").plus(deltaPartida). Só a alternativa escolhida é avaliada.

```
90     }
91 }
92
```

## efetivo - L93 a L96

Procura campos \_real, \_estimada, \_prevista nessa ordem; devolve o primeiro horário disponível, ou lança erro.

Entradas declaradas: (Map<String,Object> voo,String prefixo).

Saída declarada: Instant - ponto na linha do tempo.

Possíveis chamadores identificados: AtendimentoServico.checkin (L32); OperacaoJdbc.validarEscala (L20); PlanejamentoJdbc.agendas (L76); PlanejamentoServico.horarios (L71); PlanejamentoServico.trocarAeronave (L93); ReservaJdbc.janela (L76); ReservaJdbc.reservar (L15); VooServico.transicao (L22).

Chamadas observadas no corpo: voo.get, Dados.instante, new IllegalArgumentException.

```
93     public static Instant efetivo(Map<String,Object> voo,String prefixo) {
            Início da declaração de efetivo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
            invocado.
```

```
94         for(String sufixo:new String[]{"_real","_estimada","_prevista"}) if(voo.get(prefixo+sufixo)!=null)
            return Dados.instante(voo,prefixo+sufixo);
```

Percorre cada elemento da coleção/vetor: for(String sufixo:new String[]{"\_real","\_estimada","\_prevista"}). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Decisão específica: (voo.get(prefixo+sufixo)!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega Dados.instante(voo,prefixo+sufixo) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
95         throw new IllegalArgumentException("Horario inexistente.");
            Lança new IllegalArgumentException("Horario inexistente."). Interrompe o caminho normal; a exceção sobe até uma captura
            compatível e pode provocar rollback.
```

```
96     }
97 }
```

## ReservaJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/ReservaJdbc.java

Escolhe assentos e persiste itens/ocupações, preserva os preços vendidos, recalcula situação, invalida documentos e administra expiração e devoluções.

171 linhas | 9 métodos/construtores declarados | SHA-256: a90fd363edc6a589...

```

1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import java.math.BigDecimal;
5 import java.sql.Connection;
6 import java.sql.SQLException;
7 import java.time.Duration;
8 import java.time.Instant;
9 import java.time.LocalDate;
10 import java.time.Period;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

11 import java.time.ZoneOffset;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

12 import java.util.*;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

13
14 public final class ReservaJdbc {
    Declara ReservaJdbc como tipo com campos e métodos. final impede herança.

```

### reservar - L15 a L67

Ordena pedidos por passageiro/ordem e controla ordens/assentos/percurso. Limita prazo ao check-in, verifica conexão >=60 min, tarifa/desconto, classe e saída de emergência. Depois bloqueia inventários em ordem, insere cabeçalho/itens/ocupações e retorna id.

Entradas declaradas: (Connection c,long usuario,long comprador,List<PedidoTrecho> trechos,Instant agora,int minutos).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: ReservaServico.remarcar (L49); ReservaServico.reservar (L26).

Chamadas observadas no corpo: BloqueiosJdbc.voos, trechos.stream().map(PedidoTrecho::voo).toList, trechos.stream().map, trechos.stream, PedidoTrecho::voo, Sql.registro, new ArrayList<Map<String,Object>>, new ArrayList<Long>, new HashSet<Long>, new HashMap<Long,Map<String,Object>>, new HashSet<String>, agora.plus, Duration.ofMinutes, trechos.stream().sorted(Comparator.comparingLong(PedidoTrecho::passageiro).thenComparingInt(PedidoTrecho::ordem)).toList, trechos.stream().sorted, Comparator.comparingLong(PedidoTrecho::passageiro).thenComparingInt, Comparator.comparingLong, PedidoTrecho::passageiro, PedidoTrecho::ordem, Dados.exigir, ordens.add, pedido.passageiro, pedido.ordem, pedido.voo, Set.of("PROGRAMADO","CHECKIN\_ABERTO").contains, Set.of, voo.get, janela, agora.isBefore, limite.isBefore, percurso.put, Dados.id, PlanejamentoJdbc.efetivo(anterior,"chegada").plus(Duration.ofMinutes(60)).isAfter, PlanejamentoJdbc.efetivo(anterior,"chegada").plus, PlanejamentoJdbc.efetivo, pedido.tarifa, pedido.desconto().compareTo, pedido.desconto, Dados.dinheiro(tarifa,"valor\_base").multiply, Dados.dinheiro, new BigDecimal, Sql.listar, Sql::linha, tarifa.get, pedido.assento, selecionados.contains, Boolean.TRUE.equals, assento.get, Period.between(((java.sql.Date)pessoa.get("nascimento")).toLocalDate(),PlanejamentoJdbc.efetivo(voo,"partida").atZone(ZoneOffset.UTC).toLocalDate()).getYears, Period.between, ((java.sql.Date)pessoa.get("nascimento")).toLocalDate, pessoa.get, PlanejamentoJdbc.efetivo(voo,"partida").atZone(ZoneOffset.UTC).toLocalDate, PlanejamentoJdbc.efetivo(voo,"partida").atZone, pessoa.get("observacoes\_assistencia").toString().isBlank, pessoa.get("observacoes\_assistencia").toString, selecionados.add, assentos.add, tarifas.add, assentos.stream().sorted().toList, assentos.stream().sorted, assentos.stream, Sql.inserir, Dados.codigo, ordenados.size, ordenados.get, tarifas.get, novoterm, Sql.executar, assentos.get.

```

15 public long reservar(Connection c,long usuario,long comprador,List<PedidoTrecho> trechos,Instant
    agora,int minutos) throws SQLException {
    Início da declaração de reservar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

16 BloqueiosJdbc.voos(c,trechos.stream().map(PedidoTrecho::voo).toList());
    Chama BloqueiosJdbc.voos: Elimina ids repetidos, ordena e bloqueia cada voo. A ordem consistente reduz deadlocks.

```

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 17 | <code>Sql.registro(c,"SELECT id FROM pessoa WHERE id=? AND ativo FOR SHARE",comprador);</code><br>Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pessoa. O FOR SHARE mantém lock compartilhado para proteger a validação. Parâmetros ligados em ordem: comprador.                                                                                                                                                   |
| 18 | <code>var tarifas=new ArrayList&lt;Map&lt;String,Object&gt;&gt;();</code><br>Declara variável local tarifas com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                |
| 19 | <code>var assentos=new ArrayList&lt;Long&gt;();</code><br>Declara variável local assentos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                  |
| 20 | <code>var selecionados=new HashSet&lt;Long&gt;();</code><br>Declara variável local selecionados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói HashSet: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                              |
| 21 | <code>var percurso=new HashMap&lt;Long,Map&lt;String,Object&gt;&gt;();</code><br>Declara variável local percurso com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói HashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                             |
| 22 | <code>var ordens=new HashSet&lt;String&gt;();</code><br>Declara variável local ordens com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói HashSet: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                        |
| 23 | <code>Instant prazo=agora.plus(Duration.ofMinutes(minutos));</code><br>Declara variável local prazo do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                   |
| 24 | <code>var ordenados=trecos.stream().sorted(Comparator.comparingLong(PedidoTrecos::passageiro).thenComparingInt(PedidoTrecos::ordem)).toList();</code><br>Declara variável local ordenados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                        |
| 25 | <code>for(var pedido:ordenados) {</code><br>Percorre cada elemento da coleção/vetor: for(var pedido:ordenados). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.                                                                                                                                                                                                                                                                                       |
| 26 | <code>Dados.exigir(ordens.add(pedido.passageiro()+"."+pedido.ordem()),"Ordem de trecho repetida.");</code><br>Regra obrigatória: ordens.add(pedido.passageiro()+"."+pedido.ordem()) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Ordem de trecho repetida.". Essa condição específica não é substituída por outra validação parecida.                                                                                                                     |
| 27 | <code>var voo=Sql.registro(c,"SELECT v.*,r.origem_id,r.destino_id FROM voo v JOIN rota r ON r.id=v.rota_id WHERE v.id=?",pedido.voo());</code><br>Declara variável local voo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo, rota. Parâmetros ligados em ordem: pedido.voo(). |
| 28 | <code>Dados.exigir(Set.of("PROGRAMADO","CHECKIN_ABERTO").contains(voo.get("situacao")), "Voo nao esta disponivel para venda.");</code><br>Regra obrigatória: Set.of("PROGRAMADO","CHECKIN_ABERTO").contains(voo.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Voo nao esta disponivel para venda.". Essa condição específica não é substituída por outra validação parecida.                                                              |
| 29 | <code>Instant limite=janela(voo,"checkin_fecha");</code><br>Declara variável local limite do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                             |
| 30 | <code>Dados.exigir(agora.isBefore(limite),"Venda encerrada para este voo.");</code><br>Regra obrigatória: agora.isBefore(limite) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Venda encerrada para este voo.". Essa condição específica não é substituída por outra validação parecida.                                                                                                                                                                   |
| 31 | <code>if(limite.isBefore(prazo)) prazo=limite;</code><br>Decisão específica: (limite.isBefore(prazo)). Se for true, executa o ramo seguinte; se for false, pula esse ramo.<br><br>Atualiza prazo usando = e o resultado de limite. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.                                                                                                                                                                             |

```
32         var anterior=percurso.put(pedido.passageiro(),voo);
```

Declara variável local anterior com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
33         if(anterior!=null) {
```

Decisão específica: (anterior!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
34         Dados.exigir(Dados.id(anterior,"destino_id")==Dados.id(voo,"origem_id"),"Conexao sem
continuidade.");
```

Regra obrigatória: Dados.id(anterior,"destino\_id")==Dados.id(voo,"origem\_id") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Conexao sem continuidade.". Essa condição específica não é substituída por outra validação parecida.

```
35         Dados.exigir(!PlanejamentoJdbc.efetivo(anterior,"chegada").plus(Duration.ofMinutes(60)).is
After(PlanejamentoJdbc.efetivo(voo,"partida")), "Conexao exige ao menos 60 minutos.");
```

Regra obrigatória:

!PlanejamentoJdbc.efetivo(anterior,"chegada").plus(Duration.ofMinutes(60)).isAfter(PlanejamentoJdbc.efetivo(voo,"partida")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Conexao exige ao menos 60 minutos.". Essa condição específica não é substituída por outra validação parecida.

```
36     }
```

```
37     var pessoa=Sql.registro(c,"SELECT p.*,pa.observacoes_assistencia FROM pessoa p JOIN passageiro
pa ON pa.pessoa_id=p.id WHERE p.id=? AND p.ativo AND pa.ativo FOR SHARE OF p,pa",pedido.passageiro());
```

Declara variável local pessoa com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pessoa, passageiro. O FOR SHARE mantém lock compartilhado para proteger a validação. Parâmetros ligados em ordem: pedido.passageiro().

```
38     var tarifa=Sql.registro(c,"SELECT * FROM tarifa_voo WHERE id=? AND voo_id=? AND
ativa",pedido.tarifa(),pedido.voo());
```

Declara variável local tarifa com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: tarifa\_voo. Parâmetros ligados em ordem: pedido.tarifa(), pedido.voo().

```
39     Dados.exigir(pedido.desconto().compareTo(Dados.dinheiro(tarifa,"valor_base").multiply(new
BigDecimal("0.10")))<=0,"Desconto maximo da simulacao: 10% do valor base.");
```

Regra obrigatória: pedido.desconto().compareTo(Dados.dinheiro(tarifa,"valor\_base").multiply(new BigDecimal("0.10")))<=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Desconto maximo da simulacao: 10% do valor base.". Essa condição específica não é substituída por outra validação parecida.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
40     var livres=Sql.listar(c,""
```

Declara variável local livres com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: inventario\_assento\_voo, assento\_aeronave, ocupacao\_assento. Parâmetros ligados em ordem: pedido.voo(), tarifa.get("classe"), pedido.assento(), pedido.assento().

```
41     SELECT i.*,a.saida_emergencia FROM inventario_assento_voo i JOIN assento_aeronave a ON
a.id=i.assento_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
42     WHERE i.voo_id=? AND i.classe=? AND NOT i.bloqueado AND (?:varchar IS NULL OR
i.codigo=?)
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
43     AND NOT EXISTS (SELECT 1 FROM ocupacao_assento o WHERE o.inventario_id=i.id AND
o.liberada_em IS NULL)
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
44     ORDER BY a.filas,a.coluna
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

|    |                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                 |
|----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 45 | """,Sql::linha,pedido.voo(),tarifa.get("classe"),pedido.asento(),pedido.asento());                                                                                                                 | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                                                                                                                  |
| 46 | Long escolhido=null;                                                                                                                                                                               | Declara variável local escolhido do tipo Long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                             |
| 47 | for(var asento:livres) {                                                                                                                                                                           | Percorre cada elemento da coleção/vetor: for(var asento:livres). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.                                                                                                                                          |
| 48 | long id=Dados.id(asento,"id");                                                                                                                                                                     | Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                    |
| 49 | if(selecionados.contains(id)) continue;                                                                                                                                                            | Decisão específica: (selecionados.contains(id)). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo ignora este candidato e segue para a próxima volta.                                                                                                            |
| 50 | if(Boolean.TRUE.equals(asento.get("saida_emergencia"))) {                                                                                                                                          | Decisão específica: (Boolean.TRUE.equals(asento.get("saida_emergencia"))). Se for true, executa o ramo seguinte; se for false, pula esse ramo.                                                                                                                                                  |
| 51 | int idade=Period.between(((java.sql.Date)pessoa.get("nascimento")).toLocalDate(),PlanejamentoJdbc.efetivo(voo,"partida").atZone(ZoneOffset.UTC).toLocalDate()).getYears();                         | Declara variável local idade do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                  |
| 52 | if(idade<16    idade>65    (pessoa.get("observacoes_assistencia")!=null && !pessoa.get("observacoes_assistencia").toString().isBlank())) continue;                                                 | Decisão específica: (idade<16    idade>65    (pessoa.get("observacoes_assistencia")!=null && !pessoa.get("observacoes_assistencia").toString().isBlank())). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo ignora este candidato e segue para a próxima volta. |
| 53 | }                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                 |
| 54 | escolhido=id;break;                                                                                                                                                                                | Atualiza escolhido usando = e o resultado de id. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.                                                                                                                                                                   |
| 55 | }                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                 |
| 56 | Dados.exigir(escolhido!=null,"Assento indisponível, bloqueado ou incompatível.");                                                                                                                  | Regra obrigatória: escolhido!=null deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Assento indisponível, bloqueado ou incompatível.". Essa condição específica não é substituída por outra validação parecida.                                                   |
| 57 | selecionados.add(escolhido);assentos.add(escolhido);tarifas.add(tarifa);                                                                                                                           | Chama selecionados.add: Insere na coleção. Em Set, o boolean de retorno indica se o elemento era novo.                                                                                                                                                                                          |
| 58 | }                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                 |
| 59 | for(long asento:assentos.stream().sorted().toList()) Sql.registro(c,"SELECT id FROM inventario_asento_voo WHERE id=? FOR UPDATE",asento);                                                          | Percorre cada elemento da coleção/vetor: for(long asento:assentos.stream().sorted().toList()). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.                                                                                                            |
|    |                                                                                                                                                                                                    | Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: inventario_asento_voo. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: asento.                                                     |
| 60 | long reserva=Sql.inserir(c,"INSERT INTO reserva(localizador,comprador_id,usuario_id,criada_em,expira_em) VALUES (?, ?, ?, ?, ?) RETURNING id",Dados.codigo("R",12),comprador,usuario,agora,prazo); | Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                               |
|    |                                                                                                                                                                                                    | Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: reserva. Parâmetros ligados em ordem: Dados.codigo("R",12), comprador, usuario, agora, prazo.                                                                             |
| 61 | for(int indice=0;indice<ordenados.size();indice++) {                                                                                                                                               | Laço controlado por inicialização, condição e atualização: for(int indice=0;indice<ordenados.size();indice++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.                                                                                            |
|    |                                                                                                                                                                                                    | Declara variável local indice do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                 |
| 62 | PedidoTrecho pedido=ordenados.get(indice);var tarifa=tarifas.get(indice);                                                                                                                          | Declara variável local pedido do tipo PedidoTrecho. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                        |
|    |                                                                                                                                                                                                    | Declara variável local tarifa com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                        |
| 63 | long item=novoItem(c,reserva,pedido,tarifa);                                                                                                                                                       | Declara variável local item do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                  |



```

64      Sql.executar(c,"INSERT INTO ocupacao_assento(item_id,voo_id,inventario_id,criada_em) VALUES
(?,?,?,?)",item,pedido.voo(),assentos.get(indice),agora);
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: ocupacao_assento.
      Parâmetros ligados em ordem: item, pedido.voo(), assentos.get(indice), agora.

65      }
66      return reserva;

```

Instrução idêntica à de BloqueiosJdbc:L26. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

67      }
68

```

## novoltem - L69 a L74

Copia base/taxa/franquia/peças/cancelamento/multa da tarifa para item\_reserva; usa desconto solicitado. RETURNING id dá o id do item.

Entradas declaradas: (Connection c,long reserva,PedidoTrecho p,Map<String,Object> tarifa).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: ReservaJdbc.reservar (L15).

Chamadas observadas no corpo: Sql.inserir, p.passageiro, p.voo, p.tarifa, p.ordem, tarifa.get, p.desconto.

```

69      public long novoItem(Connection c,long reserva,PedidoTrecho p,Map<String,Object> tarifa) throws
SQLException {
      Início da declaração de novoltem. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
      invocado.

70      return Sql.inserir(c,""
      Retorno: entrega Sql.inserir(c,"" INSERT INTO item_reserva(reserva_id,passageiro_id,voo_id,tarifa_id,ordem_trecho,valor_base,
      taxa_embarque,desconto,franquia_bagagem_kg,limite_pecas,permite_cancelar,multa_cancelamento) VALUES (?,?,?,?... ao
      chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

      Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas:
      item_reserva. Parâmetros ligados em ordem: reserva, p.passageiro(), p.voo(), p.tarifa(), p.ordem(), tarifa.get("valor_base"),
      tarifa.get("taxa_embarque"), p.desconto(), tarifa.get("franquia_bagagem_kg"), tarifa.get("limite_pecas"),
      tarifa.get("permite_cancelar"), tarifa.get("multa_cancelamento").

71      INSERT INTO item_reserva(reserva_id,passageiro_id,voo_id,tarifa_id,ordem_trecho,valor_base
,taxa_embarque,desconto,franquia_bagagem_kg,limite_pecas,permite_cancelar,multa_cancelamento)
      Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
      chamado, não pelo compilador Java.

72      VALUES (?,?,?, ?, ?, ?, ?, ?, ?, ?) RETURNING id
      Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
      chamado, não pelo compilador Java.

73      """,reserva,p.passageiro(),p.voo(),p.tarifa(),p.ordem(),tarifa.get("valor_base"),tarifa.ge
t("taxa_embarque"),p.desconto(),tarifa.get("franquia_bagagem_kg"),tarifa.get("limite_pecas"),tarifa.get("p
ermite_cancelar"),tarifa.get("multa_cancelamento"));
      Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu
      uso.

74      }
75

```

## janela - L76 a L79

Calcula atraso entre partida prevista e efetiva; soma esse atraso ao limite cadastrado. Não altera permanentemente o horário da janela no registro.

Entradas declaradas: (Map<String,Object> voo,String campo).

Saída declarada: Instant - ponto na linha do tempo.

Possíveis chamadores identificados: AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.janela (L158); FinanceiroJdbc.confirmar (L25); ReservaJdbc.reservar (L15); VooServico.transicao (L22).

Chamadas observadas no corpo: Duration.between, Dados.instante, PlanejamentoJdbc.efetivo, Dados.instante(voo,campo).plus.

```

76      public static Instant janela(Map<String,Object> voo,String campo) {
      Início da declaração de janela. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
      invocado.

```

```

77      Duration
      atraso=Duration.between(Dados.instante(voo,"partida_prevista"),PlanejamentoJdbc.efetivo(voo,"partida"));
      Declara variável local atraso do tipo Duration. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

78      return Dados.instante(voo,campo).plus(atraso);
      Retorno: entrega Dados.instante(voo,campo).plus(atraso) ao chamador desta execução; nenhuma instrução posterior desse
      mesmo corpo executa por esse caminho.

      Chama Dados.instante(voo,campo).plus: Devolve o resultado da adição sem mudar o objeto temporal/decimal original.

      Chama Dados.instante: Reconhece OffsetDateTime, Timestamp e Instant. Os dois primeiros viram Instant; o último é truncado a
      microssegundos. Tipo inesperado lança regra.

79      }
80

```

## recalcular - L81 a L91

Ordem das condições importa: todos expirados, todos cancelados/expirados, alguns cancelados/expirados, todos usados/ausentes, algum pendente, caso contrário confirmada.

Entradas declaradas: (Connection c,long reserva).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AtendimentoServico.embarcar (L132); ReservaJdbc.expirar (L148); ReservaServico.cancelar (L36); ReservaServico.remarcar (L49); VooServico.transicao (L22).

Chamadas observadas no corpo: Sql.listar, r.getString, estados.stream().allMatch, estados.stream(), s.equals, estados.stream().anyMatch, Sql.executar.

```

81      public void recalcular(Connection c,long reserva) throws SQLException {
      Início da declaração de recalcular. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
      invocado.

82      var estados=Sql.listar(c,"SELECT situacao FROM item_reserva WHERE
      reserva_id=?",r->r.getString(1),reserva);
      Declara variável local estados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
      escopo é o bloco atual.

      Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. Parâmetros
      ligados em ordem: reserva.

83      String estado;
      Declara variável local estado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

84      if(estados.stream().allMatch(s->s.equals("EXPIRADO"))) estado="EXPIRADA";
      Decisão específica: (estados.stream().allMatch(s->s.equals("EXPIRADO"))). Se for true, executa o ramo seguinte; se for false,
      pula esse ramo e executa o else.

      Atualiza estado usando = e o resultado de "EXPIRADA". Atribuição modifica a variável/célula indicada; não é um teste de
      igualdade.

85      else if(estados.stream().allMatch(s->s.equals("CANCELADO") || s.equals("EXPIRADO")))
      estado="CANCELADA";
      Decisão específica: (estados.stream().allMatch(s->s.equals("CANCELADO") || s.equals("EXPIRADO"))). Se for true, executa o
      ramo seguinte; se for false, pula esse ramo e executa o else.

      Atualiza estado usando = e o resultado de "CANCELADA". Atribuição modifica a variável/célula indicada; não é um teste de
      igualdade.

86      else if(estados.stream().anyMatch(s->s.equals("CANCELADO") || s.equals("EXPIRADO")))
      estado="PARCIAL_CANCELADA";
      Decisão específica: (estados.stream().anyMatch(s->s.equals("CANCELADO") || s.equals("EXPIRADO"))). Se for true, executa o
      ramo seguinte; se for false, pula esse ramo e executa o else.

      Atualiza estado usando = e o resultado de "PARCIAL_CANCELADA". Atribuição modifica a variável/célula indicada; não é um
      teste de igualdade.

87      else if(estados.stream().allMatch(s->s.equals("UTILIZADO") || s.equals("NAO_COMPARECEU")))
      estado="FINALIZADA";
      Decisão específica: (estados.stream().allMatch(s->s.equals("UTILIZADO") || s.equals("NAO_COMPARECEU"))). Se for true,
      executa o ramo seguinte; se for false, pula esse ramo e executa o else.

      Atualiza estado usando = e o resultado de "FINALIZADA". Atribuição modifica a variável/célula indicada; não é um teste de
      igualdade.

88      else if(estados.stream().anyMatch(s->s.equals("PENDENTE"))) estado="PENDENTE";
      Decisão específica: (estados.stream().anyMatch(s->s.equals("PENDENTE"))). Se for true, executa o ramo seguinte; se for false,
      pula esse ramo e executa o else.

```



Atualiza estado usando = e o resultado de "PENDENTE". Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
89         else estado="CONFIRMADA";
```

Atualiza estado usando = e o resultado de "CONFIRMADA". Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
90         Sql.executar(c,"UPDATE reserva SET situacao=? WHERE id=?",estado,reserva);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: reserva. Parâmetros ligados em ordem: estado, reserva.

```
91     }
```

```
92
```

## cancelarItem - L93 a L108

Já cancelado/expirado não cria novo direito. Distingue cancelamento integral da companhia de tarifa do passageiro; pode invalidar embarcado antes de decolar. Aplica multa quando cabível, soma taxas de bagagem e preserva registros.

Entradas declaradas: (Connection c, Map<String, Object> item, Instant agora, String motivo, boolean integral, long usuario).

Saída declarada: BigDecimal - valor decimal, utilizado nos cálculos financeiros.

Possíveis chamadores identificados: ReservaServico.cancelar (L36); ReservaServico.remarcar (L49); VooServico.transicao (L22).

Chamadas observadas no corpo: Dados.texto, Set.of("CANCELADO", "EXPIRADO").contains, Set.of, estado.equals, Set.of("PROGRAMADO", "CHECKIN\_ABERTO", "EMBARQUE").contains, Sql.registro(c, "SELECT situacao FROM voo WHERE id=?", Dados.id(item, "voo\_id")).get, Sql.registro, Dados.id, Dados.exigir, Set.of("PENDENTE", "CONFIRMADO").contains, Boolean.TRUE.equals, item.get, Dados.dinheiro(item, "valor\_base").add(Dados.dinheiro(item, "taxa\_embarque")).subtract, Dados.dinheiro(item, "valor\_base").add, Dados.dinheiro, total.subtract(Dados.dinheiro(item, "multa\_cancelamento")).max, total.subtract, Sql.unico(c, "SELECT COALESCE(SUM(b.taxa\_excesso),0) FROM bagagem b JOIN check\_in ci ON ci.id=b.checkin\_id WHERE ci.item\_id=? AND b.situacao<>'RETIRADA'", r->r.getBigDecimal(1), id).orElse, Sql.unico, r.getBigDecimal, Sql.executar, invalidar, devolucao.add.

```
93     public BigDecimal cancelarItem(Connection c, Map<String, Object> item, Instant agora, String
motivo, boolean integral, long usuario) throws SQLException {
```

Início da declaração de cancelarItem. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
94         String estado=Dados.texto(item, "situacao");
```

Declara variável local estado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
95         if(Set.of("CANCELADO", "EXPIRADO").contains(estado)) return BigDecimal.ZERO;
```

Decisão específica: (Set.of("CANCELADO", "EXPIRADO").contains(estado)). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega BigDecimal.ZERO ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
96         boolean embarcadoAntesDaPartida=integral && estado.equals("UTILIZADO")
```

Declara variável local embarcadoAntesDaPartida do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
97         && Set.of("PROGRAMADO", "CHECKIN_ABERTO", "EMBARQUE").contains(
```

Nesta parte da expressão, chama Set.of("PROGRAMADO", "CHECKIN\_ABERTO", "EMBARQUE").contains, Set.of. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

```
98         Sql.registro(c, "SELECT situacao FROM voo WHERE
id=?", Dados.id(item, "voo_id")).get("situacao");
```

Consulta uma linha como Map<String, Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. Parâmetros ligados em ordem: Dados.id(item, "voo\_id").

```
99         Dados.exigir(Set.of("PENDENTE", "CONFIRMADO").contains(estado) || embarcadoAntesDaPartida, "Item
utilizado ou ausente nao pode ser cancelado.");
```

Regra obrigatória: Set.of("PENDENTE", "CONFIRMADO").contains(estado) || embarcadoAntesDaPartida deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Item utilizado ou ausente nao pode ser cancelado.". Essa condição específica não é substituída por outra validação parecida.

```
100        if(!integral && estado.equals("CONFIRMADO"))
```

```
Dados.exigir(Boolean.TRUE.equals(item.get("permite_cancelar")), "Tarifa nao permite cancelamento.");
```

Decisão específica: (!integral && estado.equals("CONFIRMADO")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: Boolean.TRUE.equals(item.get("permite\_cancelar")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Tarifa nao permite cancelamento.". Essa condição específica não é substituída por outra validação parecida.

```
101         long id=Dados.id(item,"id");
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
102         BigDecimal total=Dados.dinheiro(item,"valor_base").add(Dados.dinheiro(item,"taxa_embarque")).subtr
act(Dados.dinheiro(item,"desconto"));
```

Declara variável local total do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
103         BigDecimal devolucao=integral || estado.equals("PENDENTE"?total:total.subtract(Dados.dinheiro(ite
m,"multa_cancelamento")).max(BigDecimal.ZERO);
```

Declara variável local devolucao do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Escolha de valor: integral || estado.equals("PENDENTE"). true escolhe total; false escolhe total.subtract(Dados.dinheiro(item,"multa\_cancelamento")).max(BigDecimal.ZERO). Só a alternativa escolhida é avaliada.

```
104         BigDecimal bagagens=Sql.unico(c,"SELECT COALESCE(SUM(b.taxa_excesso),0) FROM bagagem b JOIN
check_in ci ON ci.id=b.checkin_id WHERE ci.item_id=? AND
b.situacao<>'RETIRADA'",r->r.getBigDecimal(1),id).orElse(BigDecimal.ZERO);
```

Declara variável local bagagens do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: bagagem, check\_in. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: id.

```
105         Sql.executar(c,"UPDATE item_reserva SET situacao='CANCELADO',cancelado_em=?,motivo_cancelamento=?
WHERE id=?",agora,motivo,id);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: agora, motivo, id.

```
106         invalidar(c,id,agora,usuario);
```

Chama ReservaJdbc.invalidar: Libera ocupação, cancela bilhete/check-in e marca bagagens retiradas; grava eventos quando usuário estiver informado. Não apaga itens antigos.

```
107         return devolucao.add(bagagens);
```

Retorno: entrega devolucao.add(bagagens) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama devolucao.add: Insere na coleção. Em Set, o boolean de retorno indica se o elemento era novo.

```
108     }
```

```
109
```

## invalidar - L110 a L118

Libera ocupação, cancela bilhete/check-in e marca bagagens retiradas; grava eventos quando usuário estiver informado. Não apaga itens antigos.

Entradas declaradas: (Connection c,long item,Instant agora,Long usuario).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: ReservaJdbc.cancelarItem (L93); ReservaJdbc.expirar (L148); VooServico.transicao (L22).

Chamadas observadas no corpo: Sql.executar, Sql.listar, Sql::linha, Dados.id.

```
110     public void invalidar(Connection c,long item,Instant agora,Long usuario) throws SQLException {
```

Início da declaração de invalidar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
111         Sql.executar(c,"UPDATE ocupacao_assento SET liberada_em=? WHERE item_id=? AND liberada_em IS
NULL",agora,item);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: ocupacao\_assento. Parâmetros ligados em ordem: agora, item.

```
112         Sql.executar(c,"UPDATE bilhete SET situacao='CANCELADO' WHERE item_id=? AND situacao IN
('EMITIDO','UTILIZADO')",item);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: bilhete. Parâmetros ligados em ordem: item.

```
113         Sql.executar(c,"UPDATE check_in SET cancelado_em=? WHERE item_id=? AND cancelado_em IS
NULL",agora,item);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: check\_in. Parâmetros ligados em ordem: agora, item.

```

114         for(var b:Sql.listar(c,"SELECT b.id,r.origem_id FROM bagagem b JOIN check_in ci ON
ci.id=b.checkin_id JOIN voo v ON v.id=ci.voo_id JOIN rota r ON r.id=v.rota_id WHERE ci.item_id=? AND
b.situacao NOT IN ('RETIRADA','ENTREGUE') ORDER BY b.id FOR UPDATE OF b",Sql::linha,item)) {
    Percorre cada elemento da coleção/vetor: for(var b:Sql.listar(c,"SELECT b.id,r.origem_id FROM bagagem b JOIN check_in ci ON
ci.id=b.checkin_id JOIN voo v ON v.id=ci.voo_id JOIN rota r ON r.id=v.rota_id WHERE ci.item_id=? AND b.situacao NOT IN
('RETIRADA','ENTREGUE') ORDER BY b.id FOR UPD.... return, break ou exceção podem encerrar esse caminho; continue
passa à próxima volta.

    Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: bagagem, check_in, voo, rota,
    OF. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: item.

115         Sql.executar(c,"UPDATE bagagem SET situacao='RETIRADA' WHERE id=?",Dados.id(b,"id"));
    Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: bagagem. Parâmetros
    ligados em ordem: Dados.id(b,"id").

116         if(usuario!=null) Sql.executar(c,"INSERT INTO
evento_bagagem(bagagem_id,aeroporto_id,usuario_id,situacao,ocorrido_em,observacao) VALUES
(?,?,?, 'RETIRADA',?, 'Cancelamento')",Dados.id(b,"id"),Dados.id(b,"origem_id"),usuario,agora);
    Decisão específica: (usuario!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

    Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: evento_bagagem.
    Parâmetros ligados em ordem: Dados.id(b,"id"), Dados.id(b,"origem_id"), usuario, agora.

117     }
118 }
119

```

## solicitarDevolucao - L120 a L133

Limita o solicitado ao recebido livre. Divide entre pagamentos aprovados, desconta reembolsos comprometidos e usa chave por pagamento para evitar repetição.

Entradas declaradas: (Connection c,long reserva,BigDecimal valor,String chave,String motivo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.rastrear (L111); ReservaJdbc.expirar (L148); ReservaServico.cancelar (L36); ReservaServico.remarcar (L49); VooServico.transicao (L22).

Chamadas observadas no corpo: new FinanceiroJdbc, financeiro.recebido(c,reserva).subtract(financeiro.devido(c,reserva)).max, financeiro.recebido(c,reserva).subtract, financeiro.recebido, financeiro.devido, valor.min, Sql.listar, Sql::linha, restante.min, Dados.dinheiro, parte.signum, Sql.executar, Dados.id, restante.subtract, restante.signum.

```

120     public void solicitarDevolucao(Connection c,long reserva,BigDecimal valor,String chave,String motivo)
throws SQLException {
    Início da declaração de solicitarDevolucao. Os parâmetros entram pela chamada; o corpo só executa quando esse
    método/construtor é invocado.

121         var financeiro=new FinanceiroJdbc();
    Declara variável local financeiro com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.

    new constrói FinanceiroJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
    si só.

122         BigDecimal
livre=financeiro.recebido(c,reserva).subtract(financeiro.devido(c,reserva)).max(BigDecimal.ZERO);
    Declara variável local livre do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
    atual.

123         BigDecimal restante=valor.min(livre);
    Declara variável local restante do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
    atual.

124         for(var p:Sql.listar(c,"""
    Percorre cada elemento da coleção/vetor: for(var p:Sql.listar(c,""" SELECT p.*,p.valor-COALESCE((SELECT SUM(r.valor) FROM
reembolso r WHERE r.pagamento_id=p.id AND r.situacao IN ('SOLICITADO','PROCESSADO')),0) AS disponivel FROM
pagamento p WHERE p.reserva_id=? AND p.situacao='APROVADO' .... return, break ou exceção podem encerrar esse caminho;
continue passa à próxima volta.

    Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: reembolso, pagamento. O FOR
    UPDATE mantém lock das linhas até commit/rollback. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define
    o caso sem valores. Parâmetros ligados em ordem: reserva.

```

|     |                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                 |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 125 | SELECT p.*,p.valor-COALESCE((SELECT SUM(r.valor) FROM reembolso r WHERE r.pagamento_id=p.id AND r.situacao IN ('SOLICITADO','PROCESSADO')),0) AS disponivel                                                                       | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                       |
| 126 | FROM pagamento p WHERE p.reserva_id=? AND p.situacao='APROVADO' ORDER BY p.id FOR UPDATE                                                                                                                                          | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                       |
| 127 | """,Sql::linha,reserva)) {                                                                                                                                                                                                        | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                                                                                                  |
| 128 | BigDecimal parte=restante.min(Dados.dinheiro(p,"disponivel"));                                                                                                                                                                    | Declara variável local parte do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                           |
| 129 | if(parte.signum()<=0) continue;                                                                                                                                                                                                   | Decisão específica: (parte.signum()<=0). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo ignora este candidato e segue para a próxima volta.                                                                                                    |
| 130 | Sql.executar(c,"INSERT INTO reembolso(pagamento_id,reserva_id,valor,motivo,chave_idempotencia) VALUES (?,?,,?,?) ON CONFLICT (chave_idempotencia) DO NOTHING",Dados.id(p,"id"),reserva,parte,motivo,chave+": "+Dados.id(p,"id")); | Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: reembolso. ON CONFLICT define o tratamento da colisão descrita no SQL. Parâmetros ligados em ordem: Dados.id(p,"id"), reserva, parte, motivo, chave+": "+Dados.id(p,"id"). |
| 131 | restante=restante.subtract(parte);if(restante.signum()==0) break;                                                                                                                                                                 | Atualiza restante usando = e o resultado de restante.subtract(parte). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.<br><br>Decisão específica: (restante.signum()==0). Se for true, executa o ramo seguinte; se for false, pula esse ramo.       |
| 132 | }                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                 |
| 133 | }                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                 |
| 134 |                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                 |

## limiteDevolucao - L135 a L146

Soma direitos de itens cancelados/expirados e taxas retiradas, subtrai reembolsos já comprometidos e limita ao mínimo zero.

Entradas declaradas: (Connection c,long reserva).

Saída declarada: BigDecimal - valor decimal, utilizado nos cálculos financeiros.

Possíveis chamadores identificados: FinanceiroServico.solicitarReembolso (L71).

Chamadas observadas no corpo: Sql.unico(c,""" SELECT COALESCE(SUM(CASE WHEN i.situacao='EXPIRADO' THEN i.valor\_base+i.taxa\_embarque-i.desconto WHEN i.situacao='CANCELADO' THEN GREATEST(0,i.valor\_base+i.taxa\_embarque-i.desconto- CASE WHEN v.situacao='CANCELADO' OR b.id IS NULL THEN 0 ELSE i.multa\_cancelamento END) ELSE 0 END),0) FROM item\_reserva i JOIN voo v ON v.id=i.voo\_id LEFT JOIN bilhete b ON b.item\_id=i.id WHERE i.reserva\_id=? """,r->r.getBigDecimal(1),reserva).orElse, Sql.unico, r.getBigDecimal, Sql.unico(c,"SELECT COALESCE(SUM(b.taxa\_excesso),0) FROM bagagem b JOIN check\_in ci ON ci.id=b.checkin\_id JOIN item\_reserva i ON i.id=ci.item\_id WHERE i.reserva\_id=? AND b.situacao='RETIRADA'",r->r.getBigDecimal(1),reserva).orElse, Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM reembolso WHERE reserva\_id=? AND situacao IN ('SOLICITADO','PROCESSADO')",r->r.getBigDecimal(1),reserva).orElse, direito.add(taxas).subtract(comprometido).max, direito.add(taxas).subtract, direito.add.

|     |                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 135 | public BigDecimal limiteDevolucao(Connection c,long reserva) throws SQLException {               | Início da declaração de limiteDevolucao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                                                                                                                                                                                                 |
| 136 | BigDecimal direito=Sql.unico(c,"""                                                               | Declara variável local direito do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: item_reserva, voo, bilhete. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: reserva. |
| 137 | SELECT COALESCE(SUM(CASE WHEN i.situacao='EXPIRADO' THEN i.valor_base+i.taxa_embarque-i.desconto | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                |

|     |                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 138 | WHEN i.situacao='CANCELADO' THEN GREATEST(0,i.valor_base+i.taxa_embarque-i.desconto-                                                                                                                                                                                            | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 139 | CASE WHEN v.situacao='CANCELADO' OR b.id IS NULL THEN 0 ELSE i.multa_cancelamento<br>END)                                                                                                                                                                                       | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 140 | ELSE 0 END),0)                                                                                                                                                                                                                                                                  | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| 141 | FROM item_reserva i JOIN voo v ON v.id=i.voo_id LEFT JOIN bilhete b ON b.item_id=i.id<br>WHERE i.reserva_id=?                                                                                                                                                                   | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 142 | """,r->r.getBigDecimal(1),reserva).orElse(BigDecimal.ZERO);                                                                                                                                                                                                                     | Instrução idêntica à de FinanceiroJdbc:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 143 | BigDecimal taxas=Sql.unico(c,"SELECT COALESCE(SUM(b.taxa_excesso),0) FROM bagagem b JOIN check_in<br>ci ON ci.id=b.checkin_id JOIN item_reserva i ON i.id=ci.item_id WHERE i.reserva_id=? AND<br>b.situacao='RETIRADA'",r->r.getBigDecimal(1),reserva).orElse(BigDecimal.ZERO); | Declara variável local taxas do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: bagagem, check_in, item_reserva. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: reserva.                                                                                                                                                                                                                                            |
| 144 | BigDecimal comprometido=Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM reembolso WHERE<br>reserva_id=? AND situacao IN<br>( 'SOLICITADO', 'PROCESSADO' )",r->r.getBigDecimal(1),reserva).orElse(BigDecimal.ZERO);                                                              | Declara variável local comprometido do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: reembolso. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: reserva.                                                                                                                                                                                                                                                           |
| 145 | return direito.add(taxas).subtract(comprometido).max(BigDecimal.ZERO);                                                                                                                                                                                                          | Retorno: entrega direito.add(taxas).subtract(comprometido).max(BigDecimal.ZERO) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.<br><br>Invoca direito.add(taxas).subtract(comprometido).max com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.<br><br>Invoca direito.add(taxas).subtract com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.<br><br>Chama direito.add: Insere na coleção. Em Set, o boolean de retorno indica se o elemento era novo. |
| 146 | }                                                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 147 |                                                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |

## expirar - L148 a L170

Encontra cabeçalhos vencidos com itens pendentes, bloqueia voos/reservas em ordem e revalida prazo. Marca itens EXPIRADO, invalida, solicita devolução, recalcula e conta reservas tratadas.

Entradas declaradas: (Connection c,Instant agora).

Saída declarada: int - inteiro primitivo de 32 bits, usado para índice, limite ou contagem.

Possíveis chamadores identificados: ReservaServico.expirar (L24); ReservaServico.expirarPendencias (L22); VooServico.transicao (L22).

Chamadas observadas no corpo: Sql.listar, r.getLong, new ArrayList<Long>, voos.addAll, BloqueiosJdbc.voos, BloqueiosJdbc.reserva, Dados.instante(reserva,"expira\_em").isAfter, Dados.instante, Sql::linha, itens.isEmpty, Sql.executar, Dados.id, invalidar, Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM pagamento WHERE reserva\_id=? AND situacao='APROVADO'",r->r.getBigDecimal(1),id).orElse, Sql.unico, r.getBigDecimal, solicitarDevolucao, recalculat.

|     |                                                                      |                                                                                                                                  |
|-----|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| 148 | public int expirar(Connection c,Instant agora) throws SQLException { | Início da declaração de expirar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado. |
|-----|----------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|

```
149      var ids=Sql.listar(c,"SELECT id FROM reserva WHERE expira_em=? AND EXISTS (SELECT 1 FROM
item_reserva i WHERE i.reserva_id=reserva.id AND i.situacao='PENDENTE') ORDER BY
id",r->r.getLong(1),agora);
```

Declara variável local ids com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: reserva, item\_reserva. Parâmetros ligados em ordem: agora.

```
150      var voos=new ArrayList<Long>();
```

Declara variável local voos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
151      for(long id:ids) voos.addAll(Sql.listar(c,"SELECT voo_id FROM item_reserva WHERE
reserva_id=?",r->r.getLong(1),id));
```

Percorre cada elemento da coleção/vetor: for(long id:ids). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: id.

```
152      BloqueiosJdbc.voos(c,voos);
```

Chama BloqueiosJdbc.voos: Elimina ids repetidos, ordena e bloqueia cada voo. A ordem consistente reduz deadlocks.

```
153      int quantidade=0;
```

Declara variável local quantidade do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
154      for(long id:ids) {
```

Percorre cada elemento da coleção/vetor: for(long id:ids). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
155      var reserva=BloqueiosJdbc.reserva(c,id);
```

Declara variável local reserva com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
156      if(Dados.instante(reserva,"expira_em").isAfter(agora)) continue;
```

Decisão específica: (Dados.instante(reserva,"expira\_em").isAfter(agora)). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo ignora este candidato e segue para a próxima volta.

```
157      var itens=Sql.listar(c,"SELECT * FROM item_reserva WHERE reserva_id=? AND situacao='PENDENTE'
ORDER BY id",Sql::linha,id);
```

Declara variável local itens com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: id.

```
158      if(itens.isEmpty()) continue;
```

Decisão específica: (itens.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo ignora este candidato e segue para a próxima volta.

```
159      for(var item:itens) {
```

Percorre cada elemento da coleção/vetor: for(var item:itens). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
160      Sql.executar(c,"UPDATE item_reserva SET situacao='EXPIRADO' WHERE
id=?",Dados.id(item,"id"));
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: Dados.id(item,"id").

```
161      invalidar(c,Dados.id(item,"id"),agora,null);
```

Chama ReservaJdbc.invalidar: Libera ocupação, cancela bilhete/check-in e marca bagagens retiradas; grava eventos quando usuário estiver informado. Não apaga itens antigos.

```
162      }
```

```
163      BigDecimal aprovado=Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM pagamento WHERE
reserva_id=? AND situacao='APROVADO'",r->r.getBigDecimal(1),id).orElse(BigDecimal.ZERO);
```

Declara variável local aprovado do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: pagamento. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: id.

```
164         solicitarDevolucao(c,id,aprovado,"exp:"+id,"Expiracao de reserva pendente");
```

Chama ReservaJdbc.solicitarDevolucao: Limita o solicitado ao recebido livre. Divide entre pagamentos aprovados, desconta reembolsos comprometidos e usa chave por pagamento para evitar repetição.

```
165         recalcular(c,id);
```

Chama ReservaJdbc.recalcular: Ordem das condições importa: todos expirados, todos cancelados/expirados, alguns cancelados/expirados, todos usados/ausentes, algum pendente, caso contrário confirmada.

```
166         Sql.executar(c,"INSERT INTO evento_auditoria(acao,entidade,registro_id) VALUES  
( 'EXPIRAR_RESERVA', 'reserva',?)" ,id);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: evento\_auditoria. Parâmetros ligados em ordem: id.

```
167         quantidade++;
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```
168     }
```

```
169     return quantidade;
```

Retorno: entrega quantidade ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
170 }
```

```
171 }
```



## Sql

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/Sql.java

Centraliza PreparedStatement, parâmetros, ResultSet, listas, Optional e extração de identificadores. Não abre uma segunda conexão nem decide sozinho as regras de negócio.

88 linhas | 9 métodos/construtores declarados | SHA-256: 6c3b68520c5ef24a...

```

1 package br.edu.aeroporto.infraestrutura.jdbc;
    Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.sql.Connection;
4 import java.sql.PreparedStatement;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

5 import java.sql.ResultSet;
6 import java.sql.SQLException;
7 import java.time.Instant;
8 import java.time.ZoneOffset;
9 import java.util.ArrayList;
10 import java.util.List;
11 import java.util.Optional;
12 import java.util.LinkedHashMap;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

13 import java.util.Map;
14 import java.util.Collections;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

15
16 /** Centraliza recursos JDBC; as consultas e regras continuam nos seus módulos. */
    Comentário/documentação do autor; descreve intenção ou convenção e não executa.

17 public final class Sql {
    Declara Sql como tipo com campos e métodos. final impede herança.
```

### Sql - L18 a L18

Construtor vazio. Se for private, impede criar instâncias desta classe utilitária a partir de outras classes; os métodos static são usados pelo nome da classe.

Entradas declaradas: ().

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```

18     private Sql() {}
    Início da declaração de Sql. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

19
```

### linha - L20 a L27

Percorre metadados de ResultSet de 1 até a quantidade de colunas; usa labels/aliases como chaves e devolve Map não modificável.

Entradas declaradas: (ResultSet resultado).

Saída declarada: Map<String, Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: AdministracaoServico.usuarios (L21); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); CadastroJdbc.listar (L28); DadosFicticiosServico.criar (L23); FinanceiroJdbc.confirmar (L25); FinanceiroServico.pagar (L22); FinanceiroServico.solicitarReembolso (L71); OperacaoJdbc.validarEscala (L20); OperacaoJdbc.validarEscalasDoVoo (L14); PlanejamentoJdbc.agendas (L76); PlanejamentoJdbc.validarFrota (L15); RelatoriosCompletoServico.bagagem (L51); RelatoriosCompletoServico.consultar (L29); RelatoriosCompletoServico.mapa (L44); RelatoriosCompletoServico.painel (L34); ReservaJdbc.expirar (L148); ReservaJdbc.invalidar (L110); ReservaJdbc.reservar (L15); ReservaJdbc.solicitarDevolucao (L120); ReservaServico.cancelar (L36); Sql.registro (L29); VooServico.transicao (L22).



Chamadas observadas no corpo: `new LinkedHashMap<String, Object>`, `resultado.getMetaData`, `metadados.getColumnCount`, `valores.put`, `metadados.getColumnLabel`, `resultado.getObject`, `Collections.unmodifiableMap`.

```

20     public static Map<String, Object> linha(ResultSet resultado) throws SQLException {
        Início da declaração de linha. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

21         var valores = new LinkedHashMap<String, Object>();
        Declara variável local valores com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        new constrói LinkedHashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
        si só.

22         var metadados = resultado.getMetaData();
        Declara variável local metadados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

23         for (int coluna = 1; coluna <= metadados.getColumnCount(); coluna++) {
        Laço controlado por inicialização, condição e atualização: for (int coluna = 1; coluna <= metadados.getColumnCount(); coluna++).
        return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

        Declara variável local coluna do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

24         valores.put(metadados.getColumnLabel(coluna), resultado.getObject(coluna));
        Chama valores.put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.

25     }
26     return Collections.unmodifiableMap(valores);
        Retorno: entrega Collections.unmodifiableMap(valores) ao chamador desta execução; nenhuma instrução posterior desse mesmo
        corpo executa por esse caminho.

        Invoca Collections.unmodifiableMap com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
        chamadas do método para localizar o corpo no projeto.

27 }
28

```

## registro - L29 a L32

Chama unico com `Sql::linha`; ausência vira `RegraNegocioException` em vez de `Optional` vazio.

Entradas declaradas: (Connection conexao, String consulta, Object... parametros).

Saída declarada: `Map<String, Object>` - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: `AdministracaoServico.acesso` (L47); `AdministracaoServico.alterarSenha` (L67); `AdministracaoServico.criarUsuario` (L32); `AtendimentoServico.cancelarCheckin` (L60); `AtendimentoServico.cartao` (L76); `AtendimentoServico.checkin` (L32); `AtendimentoServico.despachar` (L88); `AtendimentoServico.embarcar` (L132); `AtendimentoServico.rastrear` (L111); `BloqueiosJdbc.item` (L29); `BloqueiosJdbc.reserva` (L21); `BloqueiosJdbc.voo` (L13); `CadastroJdbc.buscar` (L32); `DadosFicticiosServico.criar` (L23); `FinanceiroJdbc.confirmar` (L25); `FinanceiroServico.cancelarPagamento` (L55); `FinanceiroServico.processarReembolso` (L89); `FinanceiroServico.solicitarReembolso` (L71); `IntegracaoTeste.administracaoElImportacao` (L431); `IntegracaoTeste.cadastros` (L62); `IntegracaoTeste.cadastrosRestantes` (L460); `IntegracaoTeste.cancelamentoAposEmbarque` (L309); `IntegracaoTeste.cenariosAdicionais` (L203); `IntegracaoTeste.falhasEDeadlock` (L334); `IntegracaoTeste.regrasComplementares` (L369); `OperacaoJdbc.validarEscala` (L20); `OperacaoJdbc.validarRecurso` (L67); `OperacaoServico.funcionarioAtivo` (L170); `OperacaoServico.interditar` (L33); `OperacaoServico.liberarAlocacao` (L43); `OperacaoServico.liberarEscala` (L81); `OperacaoServico.manutencao` (L94); `OperacaoServico.ocorrencia` (L155); `OperacaoServico.servicoSolo` (L130); `OperacaoServico.situacaoManutencao` (L107); `OperacaoServico.situacaoSolo` (L144); `PassageiroJdbc.editar` (L29); `PassageiroJdbc.vincular` (L21); `PassageiroServico.editar` (L55); `PlanejamentoJdbc.tarifa` (L62); `PlanejamentoJdbc.validarFrota` (L15); `PlanejamentoServico.bloquearAssento` (L40); `RelatoriosCompletoServico.painel` (L34); `ReservaJdbc.cancelarItem` (L93); `ReservaJdbc.reservar` (L15); `ReservaServico.remarcar` (L49); `VooServico.transicao` (L22).

Chamadas observadas no corpo: `unico(conexao, consulta, Sql::linha, parametros).orElseThrow`, `unico`, `Sql::linha`, `new br.edu.aeroporto.excecao.RegraNegocioException`.

```

29     public static Map<String, Object> registro(Connection conexao, String consulta, Object... parametros)
        throws SQLException {
        Início da declaração de registro. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

30     return unico(conexao, consulta, Sql::linha, parametros)
        Retorno: entrega unico(conexao, consulta, Sql::linha, parametros) .orElseThrow(() -> new
        br.edu.aeroporto.excecao.RegraNegocioException("Registro nao encontrado.")) ao chamador desta execução; nenhuma
        instrução posterior desse mesmo corpo executa por esse caminho.

        Chama Sql.unico: Zero linhas: Optional.empty; uma: Optional.of(resultado); mais de uma: SQLException, protegendo a
        expectativa da consulta.

```

```
31 .orElseThrow(() -> new br.edu.aeroporto.execcao.RegraNegocioException("Registro nao
    encontrado."));
    new constrói br.edu.aeroporto.execcao.RegraNegocioException: aloca a instância e executa seu construtor com os argumentos
    mostrados; não insere no banco por si só.
```

```
32 }
```

```
33
```

```
34 @FunctionalInterface
```

Anotação de contrato: `@Override` confere sobrescrita; `@FunctionalInterface` confere um único método abstrato, conforme a anotação mostrada.

```
35 public interface Mapeador<T> {
```

Declara Mapeador como contrato de operações.

## mapear - L36 a L36

Assinatura abstrata de `Mapeador<T>`. Uma lambda ou referência como `Sql::linha` implementa esta função; recebe `ResultSet` posicionado e devolve `T`.

Entradas declaradas: (`ResultSet linha`).

Saída declarada: `T` - tipo genérico escolhido pelo contexto da chamada.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
36 T mapear(ResultSet linha) throws SQLException;
```

Início da declaração de mapear. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
37 }
```

```
38
```

## listar - L39 a L47

Prepara a consulta, abre `ResultSet`, chama o mapeador para cada next e devolve cópia da lista. Fecha statement e resultset automaticamente.

Entradas declaradas: (`Connection conexao`, `String sql`, `Mapeador<T> mapeador`, `Object... parametros`).

Saída declarada: `List<T>` - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: `AdministracaoServico.usuarios` (L21); `AtendimentoServico.cancelarCheckin` (L60); `AtendimentoServico.embarcar` (L132); `BloqueiosJdbc.reserva` (L21); `CadastroJdbc.listar` (L28); `CadastroServico.existe` (L132); `ConsultasJdbc.assentos` (L40); `ConsultasJdbc.reserva` (L52); `ConsultasJdbc.voos` (L18); `DadosFicticiosServico.codigoLivre` (L72); `FinanceiroJdbc.confirmar` (L25); `FinanceiroServico.processar` (L47); `FinanceiroServico.validarReserva` (L41); `IntegracaoTeste.icao` (L503); `OperacaoJdbc.validarEscala` (L20); `OperacaoJdbc.validarEscalasDoVoo` (L14); `OperacaoServico.habilitar` (L56); `PassageiroJdbc.buscar` (L61); `PassageiroServico.editar` (L55); `PlanejamentoJdbc.agendas` (L76); `PlanejamentoJdbc.tarifa` (L62); `PlanejamentoServico.bloquearAssento` (L40); `PlanejamentoServico.trocarAeronave` (L93); `RelatoriosCompletoServico.bagagem` (L51); `RelatoriosCompletoServico.consultar` (L29); `RelatoriosCompletoServico.mapa` (L44); `RelatoriosCompletoServico.painel` (L34); `RelatoriosCompletoServico.resumoAtrasos` (L56); `ReservaJdbc.expirar` (L148); `ReservaJdbc.invalidar` (L110); `ReservaJdbc.recalcular` (L81); `ReservaJdbc.reservar` (L15); `ReservaJdbc.solicitarDevolucao` (L120); `ReservaServico.cancelar` (L36); `ReservaServico.remarcar` (L49); `UsuarioJdbc.buscarAtivo` (L13); `VooServico.transicao` (L22).

Chamadas observadas no corpo: `preparar`, `comando.executeQuery`, `new ArrayList<>`, `linhas.next`, `resultado.add`, `mapeador.mapear`, `List.copyOf`.

```
39 public static <T> List<T> listar(Connection conexao, String sql, Mapeador<T> mapeador,
```

Início da declaração de listar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
40 Object... parametros) throws SQLException {
```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
41 try (PreparedStatement comando = preparar(conexao, sql, parametros);
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

```
42 ResultSet linhas = comando.executeQuery()) {
```

Nesta parte da expressão, chama `comando.executeQuery`. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

```
43 List<T> resultado = new ArrayList<>();
```

Declara variável local `resultado` do tipo `List<T>`. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

`new` constrói `ArrayList`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
44         while (linhas.next()) resultado.add(mapeador.mapear(linhas));
```

Repete enquanto a condição permanecer verdadeira: while (linhas.next()). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
45         return List.copyOf(resultado);
```

Retorno: entrega List.copyOf(resultado) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama List.copyOf: Cria uma cópia pelo contrato do tipo receptor; consulte o capítulo de coleções/cópias.

```
46     }
```

```
47 }
```

```
48
```

## unico - L49 a L58

Zero linhas: Optional.empty(); uma: Optional.of(resultado); mais de uma: SQLException, protegendo a expectativa da consulta.

Entradas declaradas: (Connection conexao, String sql, Mapeador<T> mapeador, Object... parametros).

Saída declarada: Optional<T> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: AdministracaoServico.acesso (L47); AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AuditoriaJdbc.definirAutorDoVoo (L15); AutorizacaoJdbc.exigir (L39); AutorizacaoJdbc.exigirPerfil (L25); CadastroServico.salvar (L30); ConsultasJdbc.reserva (L52); DadosFicticiosServico.criar (L23); FinanceiroJdbc.devido (L10); FinanceiroJdbc.recebido (L19); FinanceiroServico.pagar (L22); FinanceiroServico.processarReembolso (L89); FinanceiroServico.solicitarReembolso (L71); InicializacaoJdbc.criarAdministrador (L12); IntegracaoTeste.numero (L515); IntegracaoTeste.texto (L500); Main.main (L55); OperacaoJdbc.validarEscala (L20); OperacaoJdbc.validarTripulacao (L57); PassageiroJdbc.buscar (L56); PassageiroJdbc.cadastrar (L37); PlanejamentoJdbc.serialize (L11); PlanejamentoJdbc.validarFrota (L15); ReservaJdbc.cancelarItem (L93); ReservaJdbc.expirar (L148); ReservaJdbc.limiteDevolucao (L135); Sql.inserir (L60); Sql.registro (L29); UsuarioJdbc.buscarAtivo (L13); UsuarioJdbc.existeUsuario (L34).

Chamadas observadas no corpo: preparar, comando.executeQuery, linhas.next, Optional.empty, mapeador.mapear, new SQLException, Optional.of.

```
49     public static <T> Optional<T> unico(Connection conexao, String sql, Mapeador<T> mapeador,
```

Início da declaração de unico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
50         Object... parametros) throws SQLException {
```

Instrução idêntica à de Sql:L40. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
51         try (PreparedStatement comando = preparar(conexao, sql, parametros);
```

Instrução idêntica à de Sql:L41. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
52         ResultSet linhas = comando.executeQuery()) {
```

Instrução idêntica à de Sql:L42. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
53         if (!linhas.next()) return Optional.empty();
```

Decisão específica: (!linhas.next()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega Optional.empty() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
54         T resultado = mapeador.mapear(linhas);
```

Declara variável local resultado do tipo T. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
55         if (linhas.next()) throw new SQLException("A consulta esperava no máximo uma linha.");
```

Decisão específica: (linhas.next()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new SQLException("A consulta esperava no máximo uma linha."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói SQLException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
56         return Optional.of(resultado);
```

Retorno: entrega Optional.of(resultado) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama Optional.of: Fábrica estática: cria o valor do tipo receptor a partir dos argumentos.

```
57     }
```

```
58 }
```

## inserir - L60 a L63

Executa SQL com RETURNING, lê a primeira coluna como long e exige que o identificador tenha sido devolvido.

Entradas declaradas: (Connection conexao, String sql, Object... parametros).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: AdministracaoServico.criarUsuario (L32); AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AtendimentoServico.embarcar (L132); CadastroJdbc.salvar (L12); DadosFicticiosServico.criar (L23); FinanceiroServico.pagar (L22); FinanceiroServico.solicitarReembolso (L71); InicializacaoJdbc.criarAdministrador (L12); OperacaoServico.alocar (L24); OperacaoServico.escalar (L70); OperacaoServico.habilitar (L56); OperacaoServico.interditar (L33); OperacaoServico.manutencao (L94); OperacaoServico.occurencia (L155); OperacaoServico.servicoSolo (L130); PassageiroJdbc.cadastrar (L37); PlanejamentoJdbc.programar (L43); PlanejamentoJdbc.tarifa (L62); ReservaJdbc.novoItem (L69); ReservaJdbc.reservar (L15).

Chamadas observadas no corpo: unico(conexao, sql, linha -> linha.getLong(1), parametros).orElseThrow, unico, linha.getLong, new SQLException.

```

60     public static long inserir(Connection conexao, String sql, Object... parametros) throws SQLException {
        Início da declaração de inserir. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

61         return unico(conexao, sql, linha -> linha.getLong(1), parametros)
        Retorno: entrega unico(conexao, sql, linha -> linha.getLong(1), parametros) .orElseThrow(() -> new SQLException("A inserção
        não retornou o identificador.")) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
        caminho.

        Chama Sql.unico: Zero linhas: Optional.empty; uma: Optional.of(resultado); mais de uma: SQLException, protegendo a
        expectativa da consulta.

62         .orElseThrow(() -> new SQLException("A inserção não retornou o identificador."));
        new constrói SQLException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si
        só.

63     }
64 
```

## executar - L65 a L69

executeUpdate retorna quantidade de linhas afetadas; o nome igual ao de BancoDados.executar não significa a mesma tarefa.

Entradas declaradas: (Connection conexao, String sql, Object... parametros).

Saída declarada: int - inteiro primitivo de 32 bits, usado para índice, limite ou contagem.

Possíveis chamadores identificados: AdministracaoServico.acesso (L47); AdministracaoServico.alterarSenha (L67); AdministracaoServico.criarUsuario (L32); AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.embarcar (L132); AtendimentoServico.evento (L153); AuditoriaJdbc.registrar (L7); CadastroJdbc.salvar (L12); CadastroServico.validar (L53); DadosFicticiosServico.criar (L23); FinanceiroJdbc.confirma (L25); FinanceiroServico.cancelarPagamento (L55); FinanceiroServico.processar (L47); FinanceiroServico.processarReembolso (L89); InicializacaoJdbc.criarAdministrador (L12); IntegracaoTeste.cadastrados (L62); IntegracaoTeste.falhasEDeadlock (L334); OperacaoServico.encerrarOccurencia (L166); OperacaoServico.liberarAlocacao (L43); OperacaoServico.liberarEscala (L81); OperacaoServico.manutencao (L94); OperacaoServico.situacaoManutencao (L107); OperacaoServico.situacaoSolo (L144); PassageiroJdbc.cadastrar (L37); PassageiroJdbc.editar (L29); PassageiroJdbc.salvarContato (L75); PassageiroJdbc.vincular (L21); PlanejamentoJdbc.agendas (L76); PlanejamentoJdbc.inventario (L54); PlanejamentoJdbc.programar (L43); PlanejamentoJdbc.tarifa (L62); PlanejamentoServico.bloquearAssento (L40); PlanejamentoServico.horarios (L71); PlanejamentoServico.reprogramar (L52); PlanejamentoServico.trocarAeronave (L93); ReservaJdbc.cancelarItem (L93); ReservaJdbc.expirar (L148); ReservaJdbc.invalidar (L110); ReservaJdbc.recalcular (L81); ReservaJdbc.reservar (L15); ReservaJdbc.solicitarDevolucao (L120); ReservaServico.remarcar (L49); VooServico.transicao (L22).

Chamadas observadas no corpo: preparar, comando.executeUpdate.

```

65     public static int executar(Connection conexao, String sql, Object... parametros) throws SQLException {
        Início da declaração de executar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

66         try (PreparedStatement comando = preparar(conexao, sql, parametros)) {
        Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à
        criação.

67         return comando.executeUpdate();
        Retorno: entrega comando.executeUpdate() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo
        executa por esse caminho.

        Chama comando.executeUpdate: Executa a alteração SQL e devolve o número de linhas afetadas.

68     }
69 
```

## preparar - L71 a L87

Cria PreparedStatement, timeout de 30 s, converte Enum em name e Instant para UTC/microssegundos. Os parâmetros JDBC começam em 1, por isso i+1. Em falha fecha o statement antes de relançar.

Entradas declaradas: (Connection conexao, String sql, Object... parametros).

Saída declarada: PreparedStatement - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: Sql.executar (L65); Sql.listar (L39); Sql.unico (L49).

Chamadas observadas no corpo: conexao.prepareStatement, comando.setQueryTimeout, estado.name, instante.truncatedTo(java.time.temporal.ChronoUnit.MICROS).atOffset, instante.truncatedTo, comando.setObject, comando.close, erro.addSuppressed.

```

71     private static PreparedStatement preparar(Connection conexao, String sql, Object... parametros)
        Início da declaração de preparar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

72         throws SQLException {
        Instrução idêntica à de AuditoriaJdbc:L8. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

73         PreparedStatement comando = conexao.prepareStatement(sql);
        Declara variável local comando do tipo PreparedStatement. O valor é obtido pela expressão de inicialização mostrada; o escopo é
        o bloco atual.

74         try {
        Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

75             comando.setQueryTimeout(30);
        Invoca comando.setQueryTimeout com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
        chamadas do método para localizar o corpo no projeto.

76             for (int i = 0; i < parametros.length; i++) {
        Laço controlado por inicialização, condição e atualização: for (int i = 0; i < parametros.length; i++). return, break ou exceção
        podem encerrar esse caminho; continue passa à próxima volta.

        Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

77                 Object valor = parametros[i];
        Declara variável local valor do tipo Object. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

78                 if (valor instanceof Enum<?> estado) valor = estado.name();
        Decisão específica: (valor instanceof Enum<?> estado). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

        Atualiza valor usando = e o resultado de estado.name(). Atribuição modifica a variável/célula indicada; não é um teste de
        igualdade.

79                 if (valor instanceof Instant instante) valor =
        instante.truncatedTo(java.time.temporal.ChronoUnit.MICROS).atOffset(ZoneOffset.UTC);
        Decisão específica: (valor instanceof Instant instante). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

        Atualiza valor usando = e o resultado de instante.truncatedTo(java.time.temporal.ChronoUnit.MICROS).atOffset(ZoneOffset.UTC).
        Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

80                 comando.setObject(i + 1, valor);
        Chama comando.setObject: Liga um argumento a um marcador ? do PreparedStatement, com índice começando por 1.

81             }
82             return comando;
        Retorno: entrega comando ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
        caminho.

83         } catch (SQLException erro) {
        Trata a(s) exceção(ões) declarada(s) em catch (SQLException erro). O tratamento mostrado abaixo decide se avisa, repete ou
        relança.

```

```
84         try { comando.close(); } catch (SQLException fechamento) { erro.addSuppressed(fechamento); }
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Trata a(s) exceção(ões) declarada(s) em catch (SQLException fechamento). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
85         throw erro;
```

Lança erro. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

```
86     }
```

```
87 }
```

```
88 }
```

# UsuarioJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/UsuarioJdbc.java

Implementa UsuarioRepositorio usando PostgreSQL. Filtra usuário/pessoa ativos e vigência do contrato; reúne funcionário, perfis e hash.

38 linhas | 2 métodos/construtores declarados | SHA-256: bf7d6eebf13d12b3...

```

1 package br.edu.aeroporto.infraestrutura.jdbc;
  Declara o pacote deste arquivo: br.edu.aeroporto.infraestrutura.jdbc. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.Funcionario;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Papel de uma Pessoa contratada. Acrescenta a matrícula; habilitações, escalas e cargos ficam em outras tabelas/serviços.

4 import br.edu.aeroporto.dominio.Sessao;
5 import br.edu.aeroporto.repositorio.UsuarioRepositorio;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Contrato para consultar credenciais ativas e detectar existência de usuários. Credenciais agrupa Sessao e hash; toString oculta esse hash.

6 import java.sql.Connection;
7 import java.sql.SQLException;
8 import java.time.LocalDate;
9 import java.util.HashSet;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

10 import java.util.Optional;
11
12 public final class UsuarioJdbc implements UsuarioRepositorio {
  Declara UsuarioJdbc como tipo com campos e métodos. final impede herança. implements liga o tipo aos contratos citados.

```

## buscarAtivo - L13 a L32

Faz JOIN dos dados da conta/pessoa/funcionário; o mapeador constrói Funcionario, busca perfis e retorna Credenciais(Sessao,hash) dentro de Optional.

Entradas declaradas: (Connection conexao, String login).

Saída declarada: Optional<Credenciais> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: Sql.unico, linha.getLong, new Funcionario, linha.getString, linha.getObject, linha.getBoolean, new HashSet<>, Sql.listar, perfil.getString, new Credenciais, new Sessao.

```

13 @Override
  Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

14 public Optional<Credenciais> buscarAtivo(Connection conexao, String login) throws SQLException {
  Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

15     return Sql.unico(conexao, ""
  Retorno: entrega Sql.unico(conexao, "" SELECT u.id, u.senha_hash, f.pessoa_id, f.matricula, p.nome, p.nascimento, p.ativo FROM usuario_sistema u JOIN funcionario f ON f.pessoa_id = u.funcionario_id JOIN pessoa p ON p.id = f.pessoa_id... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

  Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: usuario_sistema, funcionario, pessoa. Parâmetros ligados em ordem: login.

16         SELECT u.id, u.senha_hash, f.pessoa_id, f.matricula, p.nome, p.nascimento, p.ativo
  Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

17     FROM usuario_sistema u JOIN funcionario f ON f.pessoa_id = u.funcionario_id
  Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```



|    |                                                                                                              |                                                                                                                                                                                                                 |
|----|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 18 | JOIN pessoa p ON p.id = f.pessoa_id                                                                          | Instrução idêntica à de AutorizacaoJdbc:L45. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.       |
| 19 | WHERE u.login = ? AND u.ativo AND p.ativo                                                                    | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                       |
| 20 | AND f.admissao <= CURRENT_DATE                                                                               | Instrução idêntica à de AutorizacaoJdbc:L49. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.       |
| 21 | AND (f.desligamento IS NULL OR f.desligamento > CURRENT_DATE)                                                | Instrução idêntica à de AutorizacaoJdbc:L50. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.       |
| 22 | """, linha -> {                                                                                              | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                                  |
| 23 | long usuarioId = linha.getLong("id");                                                                        | Declara variável local usuariold do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                             |
| 24 | var funcionario = new Funcionario(linha.getLong("pessoa_id"), linha.getString("nome"),                       | Declara variável local funcionario com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                   |
|    |                                                                                                              | new constrói Funcionario: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                |
| 25 | linha.getObject("nascimento", LocalDate.class), linha.getBoolean("ativo"),<br>linha.getString("matricula")); | Nesta parte da expressão, chama linha.getObject, linha.getBoolean, linha.getString. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                               |
| 26 | var perfis = new HashSet<>(Sql.listar(conexao, ""                                                            | Declara variável local perfis com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                        |
|    |                                                                                                              | Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: usuario_perfil, perfil_acesso. Parâmetros ligados em ordem: usuariold.                                           |
|    |                                                                                                              | new constrói HashSet: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                    |
| 27 | SELECT p.nome FROM usuario_perfil up JOIN perfil_acesso p ON p.id = up.perfil_id                             | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                       |
| 28 | WHERE up.usuario_id = ?                                                                                      | Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.                                                       |
| 29 | """, perfil -> perfil.getString(1), usuarioId));                                                             | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                                  |
| 30 | return new Credenciais(new Sessao(usuarioId, funcionario, perfis),<br>linha.getString("senha_hash"));        | Retorno: entrega new Credenciais(new Sessao(usuariold, funcionario, perfis), linha.getString("senha_hash")) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho. |
|    |                                                                                                              | new constrói Credenciais, Sessao: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                        |
| 31 | }, login);                                                                                                   | Fecha o bloco anterior e completa/continua a expressão ou abre o próximo bloco indicado nesta linha.                                                                                                            |
| 32 | }                                                                                                            |                                                                                                                                                                                                                 |
| 33 |                                                                                                              |                                                                                                                                                                                                                 |



## existeUsuario - L34 a L37

SELECT EXISTS devolve booleano; não filtra apenas contas ativas, porque o primeiro acesso já existiu mesmo se depois bloqueado.

Entradas declaradas: (Connection conexao).

Saída declarada: boolean - condição verdadeira ou falsa.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: `Sql.unico(conexao, "SELECT EXISTS (SELECT 1 FROM usuario_sistema)", linha -> linha.getBoolean(1)).orElse, Sql.unico, linha.getBoolean`.

34     @Override

Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

35     public boolean existeUsuario(Connection conexao) throws SQLException {

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

36         return Sql.unico(conexao, "SELECT EXISTS (SELECT 1 FROM usuario\_sistema)", linha ->  
linha.getBoolean(1)).orElse(false);

Retorno: entrega `Sql.unico(conexao, "SELECT EXISTS (SELECT 1 FROM usuario_sistema)", linha -> linha.getBoolean(1)).orElse(false)` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama `Sql.unico(conexao, "SELECT EXISTS (SELECT 1 FROM usuario_sistema)", linha -> linha.getBoolean(1)).orElse`: Extrai o Optional, usando o argumento de padrão se estiver vazio.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: `usuario_sistema`.

37     }

38 }

# Java: serviços e regras

## AdministracaoServico

src/main/java/br/edu/aeroporto/servico/AdministracaoServico.java

Administra contas, perfis, bloqueios e alteração de senha; preserva pelo menos um administrador ativo.

83 linhas | 7 métodos/construtores declarados | SHA-256: 2fb9f3b89785f5c7...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
  Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.infraestrutura.jdbc.*;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

5 import br.edu.aeroporto.seguranca.Senhas;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Implementa geração e conferência de hashes PBKDF2 com sal aleatório. Não descriptografa senhas; calcula novamente e compara os resultados.

6 import java.util.Arrays;
7 import java.util.List;
8 import java.util.Map;
9 import java.util.Set;
10
11 public final class AdministracaoServico {
  Declara AdministracaoServico como tipo com campos e métodos. final impede herança.

12     private static final Set<String> PERFIS=Set.of("ADMINISTRADOR", "ATENDIMENTO", "OPERACAO", "CONSULTA");
  Campo(s) PERFIS: private restringe acesso à classe. Tipo Set<String>: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

13     private final BancoDados banco;
  Campo(s) banco: private restringe acesso à classe. Tipo BancoDados: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

14     private final AutorizacaoJdbc autorizacao;
  Campo(s) autorizacao: private restringe acesso à classe. Tipo AutorizacaoJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

15     private final AuditoriaJdbc auditoria;
  Campo(s) auditoria: private restringe acesso à classe. Tipo AuditoriaJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

16

```

### AdministracaoServico - L17 a L19

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria).

Possíveis chamadores identificados: Main.main (L55).

```

17     public AdministracaoServico(BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria) {
  Início da declaração de AdministracaoServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

18         this.banco=banco; this.autorizacao=autorizacao; this.auditoria=auditoria;
  Guarda no campo banco da instância atual o valor de banco. O this diferencia o campo de parâmetros com o mesmo nome.

  Guarda no campo autorizacao da instância atual o valor de autorizacao. O this diferencia o campo de parâmetros com o mesmo nome.

  Guarda no campo auditoria da instância atual o valor de auditoria. O this diferencia o campo de parâmetros com o mesmo nome.

```

```
19     }
20 }
```

## usuarios - L21 a L30

Exige administrador e lista contas com array agregado de perfis, sem selecionar senha\_hash.

Entradas declaradas: (Sessao sessao).

Saída declarada: List<Map<String,Object>> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuOperacional.acesso (L63).

Chamadas observadas no corpo: banco.consultar, autorizacao.exigirAdministrador, Sql.listar, Sql::linha.

```
21     public List<Map<String,Object>> usuarios(Sessao sessao) {
        Início da declaração de usuarios. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

22         return banco.consultar(c->{
            Retorno: entrega banco.consultar(c->{ autorizacao.exigirAdministrador(c,sessao); return Sql.listar(c,"" SELECT
            u.id,u.funcionario_id,u.login,u.ativo,array_agg(p.nome ORDER BY p.nome) AS perfis FROM usuario_sistema u LEFT JOIN
            usuario... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

            Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da
            consulta.

23             autorizacao.exigirAdministrador(c,sessao);
            Confere no banco o acesso vigente do usuário para esta operação: exigirAdministrador(c, sessao).

24             return Sql.listar(c,"" SELECT u.id,u.funcionario_id,u.login,u.ativo,array_agg(p.nome ORDER BY p.nome) AS perfis
            FROM usuario_sistema u LEFT JOIN usuario_perfil up ON up.usuario_id=u.id LEFT JOIN perfil_acesso p ON p.id=up.p... ao
            chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

            Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: usuario_sistema, usuario_perfil,
            perfil_acesso.

25             SELECT u.id,u.funcionario_id,u.login,u.ativo,array_agg(p.nome ORDER BY p.nome) AS
            perfis
            Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
            chamado, não pelo compilador Java.

26             FROM usuario_sistema u LEFT JOIN usuario_perfil up ON up.usuario_id=u.id
            Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
            chamado, não pelo compilador Java.

27             LEFT JOIN perfil_acesso p ON p.id=up.perfil_id GROUP BY u.id ORDER BY u.id
            Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
            chamado, não pelo compilador Java.

28             "",Sql::linha);
            Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu
            uso.

29         });
30     }
31 }
```

## criarUsuario - L32 a L45

Valida senha/perfis/login e funcionário vigente, cria usuário com hash e cada perfil; tudo audita/retorna id na mesma transação. Limpa senha em finally.

Entradas declaradas: (Sessao sessao,long funcionario,String login,char[] senha,Set<String> perfis).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); MenuOperacional.acesso (L63).

Chamadas observadas no corpo: validarSenha, validarPerfis, Validacao.login, banco.transacao, autorizacao.exigirAdministrador, Sql.registro, Sql.inserir, Senhas.gerar, Dados.exigir, Sql.executar, auditoria.registrar, sessao.usuarioId, Arrays.fill.

```
32     public long criarUsuario(Sessao sessao,long funcionario,String login,char[] senha,Set<String> perfis)
    {
        Início da declaração de criarUsuario. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.
```

|    |                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 33 | try {                                                                                                                                                                                                                                           | Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 34 | validarSenha(senha); validarPerfis(perfis);                                                                                                                                                                                                     | Chama AdministracaoServico.validarSenha: Exige char[] presente com 12..128 caracteres.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
| 35 | String normalizado=Validacao.login(login);                                                                                                                                                                                                      | Declara variável local normalizado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 36 | return banco.transacao(c->{                                                                                                                                                                                                                     | Retorno: entrega banco.transacao(c->{ autorizacao.exigirAdministrador(c,sessao); Sql.registro(c,"SELECT f.pessoa_id FROM funcionario f JOIN pessoa p ON p.id=f.pessoa_id WHERE f.pessoa_id=? AND p.ativo AND f.admissao<=CURRENT_DATE AND ... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.<br><br>Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.                                                                                                                                                                                                                         |
| 37 | autorizacao.exigirAdministrador(c,sessao);                                                                                                                                                                                                      | Instrução idêntica à de AdministracaoServico:L23. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 38 | Sql.registro(c,"SELECT f.pessoa_id FROM funcionario f JOIN pessoa p ON p.id=f.pessoa_id WHERE f.pessoa_id=? AND p.ativo AND f.admissao<=CURRENT_DATE AND (f.desligamento IS NULL OR f.desligamento>CURRENT_DATE) FOR UPDATE OF f",funcionario); | Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: funcionario, pessoa, OF. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: funcionario.                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 39 | long id=Sql.inserir(c,"INSERT INTO usuario_sistema(funcionario_id,login,senha_hash) VALUES (?,?,:) RETURNING id",funcionario,normalizado,Senhas.gerar(senha));                                                                                  | Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: usuario_sistema. Parâmetros ligados em ordem: funcionario, normalizado, Senhas.gerar(senha).                                                                                                                                                                                                                                                                                                                                |
| 40 | for(String perfil:perfis) Dados.exigir(Sql.executar(c,"INSERT INTO usuario_perfil(usuario_id,perfil_id) SELECT ?,id FROM perfil_acesso WHERE nome=?",id,perfil))==1,"Perfil nao cadastrado.");                                                  | Percorre cada elemento da coleção/vetor: for(String perfil:perfis). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.<br><br>Regra obrigatória: Sql.executar(c,"INSERT INTO usuario_perfil(usuario_id,perfil_id) SELECT ?,id FROM perfil_acesso WHERE nome=?",id,perfil))==1 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Perfil nao cadastrado.". Essa condição específica não é substituída por outra validação parecida.<br><br>Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: usuario_perfil, perfil_acesso. Parâmetros ligados em ordem: id, perfil. |
| 41 | auditoria.registrar(c,sessao.usuarioId(),"CRIAR_USUARIO","usuario_sistema",id);                                                                                                                                                                 | Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "CRIAR_USUARIO", "usuario_sistema", id).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 42 | return id;                                                                                                                                                                                                                                      | Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 43 | });                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 44 | } finally { if(senha!=null) Arrays.fill(senha,'\0'); }                                                                                                                                                                                          | Bloco de finalização, executado ao sair do try/catch; neste corpo, observe quais recursos ou dados são limpos.<br><br>Decisão específica: (senha!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 45 | }                                                                                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| 46 |                                                                                                                                                                                                                                                 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

## acesso - L47 a L65

Usa lock de acesso, atualiza atividade e substitui vínculos de perfis. Consulta se sobrou administrador ativo; se não, toda a mudança sofre rollback.

Entradas declaradas: (Sessao sessao,long usuario,boolean ativo,Set<String> perfis).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElmportacao (L431); IntegracaoTeste.cadastros (L62); MenuOperacional.acesso (L63).

Chamadas observadas no corpo: validarPerfis, banco.transacao, Sql.unico, autorizacao.exigirAdministrador, Sql.registro, Sql.executar, Dados.exigir, Sql.unico(c,""" SELECT EXISTS (SELECT 1 FROM usuario\_sistema u JOIN usuario\_perfil up ON up.usuario\_id=u.id JOIN perfil\_acesso pa ON pa.id=up.perfil\_id JOIN funcionario f ON f.pessoa\_id=u.funcionario\_id JOIN pessoa p ON p.id=f.pessoa\_id WHERE u.ativo AND p.ativo AND pa.nome='ADMINISTRADOR' AND f.admissao<=CURRENT\_DATE AND (f.desligamento IS NULL OR f.desligamento>CURRENT\_DATE))

```
""",r->r.getBoolean(1)).orElse, r.getBoolean, auditoria.registrar, sessao.usuarioId.
```

```
47 public void acesso(Sessao sessao,long usuario,boolean ativo,Set<String> perfis) {
    Início da declaração de acesso. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

48     validarPerfis(perfis);
    Chama AdministracaoServico.validarPerfis: Exige conjunto presente/não vazio contendo apenas as quatro constantes permitidas.

49     banco.transacao(c->{
    Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A
    posição e o contexto do método acima mostram quando ela ocorre.

50         Sql.unico(c,"SELECT pg_advisory_xact_lock(hashtext('aeroporto.acesso'))",r->true);
    Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado.

51         autorizacao.exigirAdministrador(c,sessao);
    Instrução idêntica à de AdministracaoServico:L23. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
    desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

52         Sql.registro(c,"SELECT id FROM usuario_sistema WHERE id=? FOR UPDATE",usuario);
    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    usuario_sistema. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: usuario.

53         Sql.executar(c,"UPDATE usuario_sistema SET ativo=? WHERE id=?",ativo,usuario);
    Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: usuario_sistema.
    Parâmetros ligados em ordem: ativo, usuario.

54         Sql.executar(c,"DELETE FROM usuario_perfil WHERE usuario_id=?",usuario);
    Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: usuario_perfil.
    Parâmetros ligados em ordem: usuario.

55         for(String perfil:perfis) Dados.exigir(Sql.executar(c,"INSERT INTO
        usuario_perfil(usuario_id,perfil_id) SELECT ?,id FROM perfil_acesso WHERE
        nome=?",usuario,perfil)==1,"Perfil nao cadastrado.");
        Percorre cada elemento da coleção/vetor: for(String perfil:perfis). return, break ou exceção podem encerrar esse caminho;
        continue passa à próxima volta.

        Regra obrigatória: Sql.executar(c,"INSERT INTO usuario_perfil(usuario_id,perfil_id) SELECT ?,id FROM perfil_acesso WHERE
        nome=?",usuario,perfil)==1 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Perfil nao
        cadastrado.". Essa condição específica não é substituída por outra validação parecida.

        Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: usuario_perfil,
        perfil_acesso. Parâmetros ligados em ordem: usuario, perfil.

56         Dados.exigir(Sql.unico(c,"""
        Regra obrigatória: Sql.unico(c,""" SELECT EXISTS (SELECT 1 FROM usuario_sistema u JOIN usuario_perfil up ON
        up.usuario_id=u.id JOIN perfil_acesso pa ON pa.id=up.perfil_id JOIN funcionario f ON f.pessoa_id=u.funcionario_id JOIN pessoa
        p ON p.id=f.pessoa_id WHERE u.ativo AND p.ativo AND pa.nome='ADMINISTRADOR' AND f.admissao<=CURRENT_DATE
        AND (f.desligamento IS NULL OR f.desligamento>CURRENT_DATE)) """,r->r.getBoolean(1)).orElse(false) deve resultar em true.
        Se false, Dados.exigir lança RegraNegocioException com "Preserve pelo menos um administrador ativo.". Essa condição
        específica não é substituída por outra validação parecida.

        Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas
        mencionadas: usuario_sistema, usuario_perfil, perfil_acesso, funcionario, pessoa.

57         SELECT EXISTS (SELECT 1 FROM usuario_sistema u JOIN usuario_perfil up ON
        up.usuario_id=u.id
        Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
        chamado, não pelo compilador Java.

58         JOIN perfil_acesso pa ON pa.id=up.perfil_id JOIN funcionario f ON
        f.pessoa_id=u.funcionario_id
        Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
        chamado, não pelo compilador Java.

59         JOIN pessoa p ON p.id=f.pessoa_id WHERE u.ativo AND p.ativo AND
        pa.nome='ADMINISTRADOR'
        Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
        chamado, não pelo compilador Java.

60         AND f.admissao<=CURRENT_DATE AND (f.desligamento IS NULL OR
        f.desligamento>CURRENT_DATE))
        Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
        chamado, não pelo compilador Java.
```

```

61         """,r->r.getBoolean(1)).orElse(false),"Preserve pelo menos um administrador ativo.");
        Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

62         auditoria.registrar(c,sessao.usuarioId(),ativo?"ALTERAR_PERFIS":"BLOQUEAR_USUARIO","usuario_sistema",usuario);
        Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), ativo?"ALTERAR_PERFIS":"BLOQUEAR_USUARIO", "usuario_sistema", usuario).

        Escolha de valor: ativo. true escolhe "ALTERAR_PERFIS"; false escolhe "BLOQUEAR_USUARIO". Só a alternativa escolhida é avaliada.

63         return null;
        return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

64     });
65 }
66

```

## alterarSenha - L67 a L79

Valida nova senha, bloqueia o próprio usuário, confere senha atual, grava hash novo e audita. Zera atual e nova no finally.

Entradas declaradas: (Sessao sessao,char[] atual,char[] nova).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); MenuOperacional.acesso (L63).

Chamadas observadas no corpo: validarSenha, banco.transacao, autorizacao.exigirConsulta, Sql.registro(c,"SELECT senha\_hash FROM usuario\_sistema WHERE id=? FOR UPDATE",sessao.usuarioId()).get("senha\_hash").toString, Sql.registro(c,"SELECT senha\_hash FROM usuario\_sistema WHERE id=? FOR UPDATE",sessao.usuarioId()).get, Sql.registro, sessao.usuarioId, Dados.exigir, Senhas.conferir, Sql.executar, Senhas.gerar, auditoria.registrar, Arrays.fill.

```

67     public void alterarSenha(Sessao sessao,char[] atual,char[] nova) {
        Início da declaração de alterarSenha. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

68         try {
        Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

69             validarSenha(nova);
        Chama AdministracaoServico.validarSenha: Exige char[] presente com 12..128 caracteres.

70             banco.transacao(c->{
        Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

71                 autorizacao.exigirConsulta(c,sessao);
        Confere no banco o acesso vigente do usuário para esta operação: exigirConsulta(c, sessao).

72                 String hash=Sql.registro(c,"SELECT senha_hash FROM usuario_sistema WHERE id=? FOR
        UPDATE",sessao.usuarioId()).get("senha_hash").toString();
        Declara variável local hash do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: usuario_sistema. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: sessao.usuarioId().

73                 Dados.exigir(Senhas.conferir(atual,hash),"Senha atual incorreta.");
        Regra obrigatória: Senhas.conferir(atual,hash) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Senha atual incorreta.". Essa condição específica não é substituída por outra validação parecida.

74                 Sql.executar(c,"UPDATE usuario_sistema SET senha_hash=? WHERE
        id=?",Senhas.gerar(nova),sessao.usuarioId());
        Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: usuario_sistema. Parâmetros ligados em ordem: Senhas.gerar(nova), sessao.usuarioId().

75                 auditoria.registrar(c,sessao.usuarioId(),"ALTERAR_SENHA","usuario_sistema",sessao.usuarioId());
        Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "ALTERAR_SENHA", "usuario_sistema", sessao.usuarioId()).

76                 return null;
        return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

77             });

```

```
78         } finally { if(atual!=null) Arrays.fill(atual,'\0'); if(nova!=null) Arrays.fill(nova,'\0'); }
```

Bloco de finalização, executado ao sair do try/catch; neste corpo, observe quais recursos ou dados são limpos.

Decisão específica: (atual!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Decisão específica: (nova!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
79     }
80 }
```

## validarSenha - L81 a L81

Exige char[] presente com 12..128 caracteres.

Entradas declaradas: (char[] senha).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AdministracaoServico.alterarSenha (L67); AdministracaoServico.criarUsuario (L32).

Chamadas observadas no corpo: Dados.exigir.

```
81     private static void validarSenha(char[] senha) { Dados.exigir(senha!=null && senha.length>=12 &&
    senha.length<=128,"Senha deve ter de 12 a 128 caracteres."); }
```

Início da declaração de validarSenha. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Regra obrigatória: senha!=null && senha.length>=12 && senha.length<=128 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Senha deve ter de 12 a 128 caracteres.". Essa condição específica não é substituída por outra validação parecida.

## validarPerfis - L82 a L82

Exige conjunto presente/não vazio contendo apenas as quatro constantes permitidas.

Entradas declaradas: (Set<String> perfis).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AdministracaoServico.acesso (L47); AdministracaoServico.criarUsuario (L32).

Chamadas observadas no corpo: Dados.exigir, perfis.isEmpty, PERFIS.containsAll.

```
82     private static void validarPerfis(Set<String> perfis) { Dados.exigir(perfis!=null && !perfis.isEmpty()
    && PERFIS.containsAll(perfis),"Selecione perfis conhecidos: "+PERFIS); }
```

Início da declaração de validarPerfis. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Regra obrigatória: perfis!=null && !perfis.isEmpty() && PERFIS.containsAll(perfis) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Selecione perfis conhecidos: "+PERFIS. Essa condição específica não é substituída por outra validação parecida.

```
83 }
```



## ArquivosServico

src/main/java/br/edu/aeroporto/servico/ArquivosServico.java

Importa passageiros de TXT tabulado e realiza cópia verificada de relatórios. A cópia de arquivo não equivale a backup completo PostgreSQL.

77 linhas | 5 métodos/construtores declarados | SHA-256: cad6fe0b9ff60819...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, PassageiroJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```
1 package br.edu.aeroporto.servico;
```

Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

```
2
```

```
3 import br.edu.aeroporto.dominio.*;
```

```
4 import br.edu.aeroporto.infraestrutura.jdbc.*;
```

```
5 import java.io.*;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
6 import java.nio.charset.StandardCharsets;
```

```
7 import java.nio.file.*;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
8 import java.security.*;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
9 import java.time.Clock;
```

```
10 import java.time.LocalDate;
```

```
11 import java.util.*;
```

```
12
```

```
13 public final class ArquivosServico {
```

Declara ArquivosServico como tipo com campos e métodos. final impede herança.

```
14     private final BancoDados banco;
```

Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
15     private final AutorizacaoJdbc autorizacao;
```

Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
16     private final AuditoriaJdbc auditoria;
```

Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
17     private final Clock relógio;
```

Campo(s) relógio: private restringe acesso à classe. Tipo Clock: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```
18     private final PassageiroJdbc passageiros=new PassageiroJdbc();
```

Campo(s) passageiros: private restringe acesso à classe. Tipo PassageiroJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

new constrói PassageiroJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
19
```

### ArquivosServico - L20 a L22

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock relógio).

Possíveis chamadores identificados: IntegracaoTeste.administracaoElImportacao (L431); IntegracaoTeste.arquivos (L153); Main.main (L55).



```
20 public ArquivosServico(BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock
    relógio) {
```

Início da declaração de ArquivosServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
21 this.banco=banco;this.autorizacao=autorizacao;this.auditoria=auditoria;this.relogio=relogio;
```

Guarda no campo banco da instância atual o valor de banco. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo autorizacao da instância atual o valor de autorizacao. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo auditoria da instância atual o valor de auditoria. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo relógio da instância atual o valor de relógio. O this diferencia o campo de parâmetros com o mesmo nome.

```
22 }
```

```
23
```

## importarPassageiros - L24 a L52

Verifica permissão antes de ler. Exige arquivo legível <=5MB, cabeçalho exato, 5 campos TAB, CPF obrigatório/único e <=1000 passageiros. Valida tudo em memória; grava todos em uma transação e retorna quantidade.

Entradas declaradas: (Sessao s,Path arquivo).

Saída declarada: int - inteiro primitivo de 32 bits, usado para índice, limite ou contagem.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); IntegracaoTeste.arquivos (L153); MenuOperacional.arquivos (L177).

Chamadas observadas no corpo: banco.consultar, autorizacao.exigirAtendimento, arquivo.toFile, Dados.exigir, fonte.exists, fonte.isFile, fonte.canRead, fonte.length, new ArrayList<CadastroPassageiro>, new HashSet<String>, new BufferedReader, new FileReader, reader.readLine, texto.equals, texto.isBlank, texto.split, new CadastroPassageiro, LocalDate.parse, Validacao.nascimento, cadastro.nascimento, LocalDate.now, cadastro.cpf().isEmpty, cadastro.cpf, cpfs.add, cadastros.add, cadastros.size, new br.edu.aeroporto.excecao.RegraNegocioException, cadastros.isEmpty, banco.transacao, passageiros.cadastrar(c,cadastro).id, passageiros.cadastrar, auditoria.registrar, s.usuarioId.

```
24 public int importarPassageiros(Sessao s,Path arquivo) throws IOException {
```

Início da declaração de importarPassageiros. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
25 banco.consultar(c->{autorizacao.exigirAtendimento(c,s);return null;});
```

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```
26 File fonte=arquivo.toFile();
```

Declara variável local fonte do tipo File. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
27 Dados.exigir(fonte.exists() && fonte.isFile() && fonte.canRead() &&
    fonte.length()<=5_000_000,"Importacao requer arquivo legivel de ate 5 MB.");
```

Regra obrigatória: fonte.exists() && fonte.isFile() && fonte.canRead() && fonte.length()<=5\_000\_000 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Importacao requer arquivo legivel de ate 5 MB.". Essa condição específica não é substituída por outra validação parecida.

```
28 var cadastros=new ArrayList<CadastroPassageiro>();var cpfs=new HashSet<String>();int linha=0;
```

Declara variável local cadastros com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local cpfs com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local linha do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói ArrayList, HashSet: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
29 try(BufferedReader reader=new BufferedReader(new FileReader(fonte,StandardCharsets.UTF_8))) {
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

new constrói BufferedReader, FileReader: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
30 String texto;
```

Declara variável local texto do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
31 while((texto=reader.readLine())!=null) {
```

Repete enquanto a condição permanecer verdadeira: while((texto=reader.readLine())!=null). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Atualiza texto usando = e o resultado de reader.readLine(). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
32 linha++;if(linha==1) {
Dados.exigir(texto.equals("nome\tnascimento\tcpf\temail\ttelefone"),"Cabecalho de importacao
invalido.");continue; }
```

Decisão específica: (linha==1). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: texto.equals("nome\tnascimento\tcpf\temail\ttelefone") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Cabecalho de importacao invalido.". Essa condição específica não é substituída por outra validação parecida.

```
33 if(texto.isBlank()) continue;
```

Decisão específica: (texto.isBlank()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo ignora este candidato e segue para a próxima volta.

```
34 String[] campos=texto.split("\t",-1);
```

Declara variável local campos do tipo String[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
35 Dados.exigir(campos.length==5,"Numero de campos invalido na linha "+linha);
```

Regra obrigatória: campos.length==5 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Numero de campos invalido na linha "+linha. Essa condição específica não é substituída por outra validação parecida.

```
36 try {
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

```
37 var cadastro=new
CadastroPassageiro(campos[0],LocalDate.parse(campos[1]),campos[2],campos[3],campos[4]);
```

Declara variável local cadastro com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói CadastroPassageiro: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
38 Validacao.nascimento(cadastro.nascimento(),LocalDate.now(relogio));
```

Chama Validacao.nascimento: Exige data não nula e não futura em relação ao hoje recebido. A função não decide sozinha qual relógio usar.

```
39 Dados.exigir(!cadastro.cpf().isEmpty() && cpfs.add(cadastro.cpf()),"CPF obrigatorio ou
repetido na importacao.");cadastros.add(cadastro);
```

Regra obrigatória: !cadastro.cpf().isEmpty() && cpfs.add(cadastro.cpf()) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "CPF obrigatorio ou repetido na importacao.". Essa condição específica não é substituída por outra validação parecida.

```
40 Dados.exigir(cadastros.size()<=1000,"Limite de 1000 passageiros por arquivo.");
```

Regra obrigatória: cadastros.size()<=1000 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Limite de 1000 passageiros por arquivo.". Essa condição específica não é substituída por outra validação parecida.

```
41 } catch(RuntimeException erro) { throw new
br.edu.aeroporto.excecao.RegraNegocioException("Dados invalidos na linha "+linha+"; importacao nao
gravada."); }
```

Trata a(s) exceção(ões) declarada(s) em catch(RuntimeException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

Lança new br.edu.aeroporto.excecao.RegraNegocioException("Dados invalidos na linha "+linha+"; importacao nao gravada."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói br.edu.aeroporto.excecao.RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
42 }
```

```
43 }
```

```
44 Dados.exigir(linha>0 && !cadastros.isEmpty(),"Arquivo sem passageiros.");
```

Regra obrigatória: linha>0 && !cadastros.isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Arquivo sem passageiros.". Essa condição específica não é substituída por outra validação parecida.

```
45 return banco.transacao(c->{
```

Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
46 autorizacao.exigirAtendimento(c,s);
```

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

```

47         for(var cadastro:cadastros) {
            Percorre cada elemento da coleção/vetor: for(var cadastro:cadastros). return, break ou exceção podem encerrar esse caminho;
            continue passa à próxima volta.

48             long id=passageiros.cadastrar(c,cadastro).id();auditoria.registrar(c,s.usuarioId(),"IMPORT
            AR_PASSAGEIRO","passageiro",id);
            Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
            Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "IMPORTAR_PASSAGEIRO", "passageiro", id).

49         }
50         return cadastros.size();
    Retorno: entrega cadastros.size() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por
    esse caminho.

    Invoca cadastros.size com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
    método para localizar o corpo no projeto.

51     });
52 }
53

```

## listar - L54 a L58

Lista somente arquivos regulares .csv/.txt da pasta relatorios, em ordem; fecha o stream de diretório.

Entradas declaradas: (Sessao s).

Saída declarada: List<Path> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.arquivos (L153); MenuOperacional.arquivos (L177).

Chamadas observadas no corpo: banco.consultar, autorizacao.exigirAtendimento, Path.of, Files.createDirectories, Files.list, arquivos.filter(Files::isRegularFile).filter(p->p.toString().endsWith(".csv") || p.toString().endsWith(".txt")).sorted().toList, arquivos.filter(Files::isRegularFile).filter(p->p.toString().endsWith(".csv") || p.toString().endsWith(".txt")).sorted, arquivos.filter(Files::isRegularFile).filter, arquivos.filter, Files::isRegularFile, p.toString().endsWith, p.toString.

```

54     public List<Path> listar(Sessao s) throws IOException {
        Início da declaração de listar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

55         banco.consultar(c->{autorizacao.exigirAtendimento(c,s);return null;});
        Instrução idêntica à de ArquivosServico:L25. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

56         Path pasta=Path.of("relatorios");Files.createDirectories(pasta);
        Declara variável local pasta do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

57         try(var arquivos=Files.list(pasta)) { return
        arquivos.filter(Files::isRegularFile).filter(p->p.toString().endsWith(".csv") ||
        p.toString().endsWith(".txt")).sorted().toList(); }
        Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à
        criação.

        Retorno: entrega arquivos.filter(Files::isRegularFile).filter(p->p.toString().endsWith(".csv") ||
        p.toString().endsWith(".txt")).sorted().toList() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo
        executa por esse caminho.

58     }
59

```

## copiarRelatorio - L60 a L71

Resolve caminhos reais e limita origem a relatorios fora de backup. Copia bytes para temporário, compara SHA-256, audita e move para destino único; retorna Path.

Entradas declaradas: (Sessao s,Path arquivo).

Saída declarada: Path - caminho de arquivo, sem garantir por si a existência desse arquivo.

Possíveis chamadores identificados: IntegracaoTeste.arquivos (L153); MenuOperacional.arquivos (L177).

Chamadas observadas no corpo: banco.consultar, autorizacao.exigirAtendimento, Path.of("relatorios").toAbsolutePath().normalize().toRealPath, Path.of("relatorios").toAbsolutePath().normalize, Path.of("relatorios").toAbsolutePath, Path.of, arquivo.toAbsolutePath().normalize().toRealPath, arquivo.toAbsolutePath().normalize, arquivo.toAbsolutePath, Dados.exigir, origem.startsWith, Files.isRegularFile, raiz.resolve, Files.createDirectories, Files.createTempFile, pasta.resolve, UUID.randomUUID, origem.getFileName, new BufferedInputStream, Files.newInputStream, new BufferedOutputStream, Files.newOutputStream, in.transferTo, MessageDigest.isEqual, hash, banco.transacao, auditoria.registrar, s.usuarioId, Files.move, Files.deleteIfExists, erro.addSuppressed.

|    |                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 60 | <code>public Path copiarRelatorio(Sessao s, Path arquivo) throws IOException {</code>                                                                                                                             | Início da declaração de <code>copiarRelatorio</code> . Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| 61 | <code>banco.consultar(c-&gt;{autorizacao.exigirAtendimento(c,s);return null;});</code>                                                                                                                            | Instrução idêntica à de <code>ArquivosServico:L25</code> . Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                                                                                                             |
| 62 | <code>Path raiz=Path.of("relatorios").toAbsolutePath().normalize().toRealPath(), origem=arquivo.toAbsolutePath().normalize().toRealPath();</code>                                                                 | Declara variável local <code>raiz</code> , origem do tipo <code>Path</code> . O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 63 | <code>Dados.exigir(origem.startsWith(raiz) &amp;&amp; Files.isRegularFile(origem) &amp;&amp; !origem.startsWith(raiz.resolve("backup")), "Escolha um relatorio original da pasta relatorios.");</code>            | Regra obrigatória: <code>origem.startsWith(raiz) &amp;&amp; Files.isRegularFile(origem) &amp;&amp; !origem.startsWith(raiz.resolve("backup"))</code> deve resultar em <code>true</code> . Se <code>false</code> , <code>Dados.exigir</code> lança <code>RegraNegocioException</code> com "Escolha um relatorio original da pasta relatorios.". Essa condição específica não é substituída por outra validação parecida.                                                                                                                                                                                                             |
| 64 | <code>Path pasta=raiz.resolve("backup"); Files.createDirectories(pasta); Path temporario=Files.createTempFile(pasta, "copia-", ".tmp"), destino=pasta.resolve(UUID.randomUUID()+"-"+origem.getFileName());</code> | Declara variável local <code>pasta</code> do tipo <code>Path</code> . O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Declara variável local <code>temporario</code> , destino do tipo <code>Path</code> . O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                         |
| 65 | <code>try {</code>                                                                                                                                                                                                | Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 66 | <code>try(InputStream in=new BufferedInputStream(Files.newInputStream(origem)); OutputStream out=new BufferedOutputStream(Files.newOutputStream(temporario))) { in.transferTo(out); }</code>                      | Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.<br><br><code>new</code> constrói <code>BufferedInputStream</code> , <code>BufferedOutputStream</code> : aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                       |
| 67 | <code>Dados.exigir(MessageDigest.isEqual(hash(origem), hash(temporario)), "Copia falhou na verificacao SHA-256.");</code>                                                                                         | Regra obrigatória: <code>MessageDigest.isEqual(hash(origem), hash(temporario))</code> deve resultar em <code>true</code> . Se <code>false</code> , <code>Dados.exigir</code> lança <code>RegraNegocioException</code> com "Copia falhou na verificacao SHA-256.". Essa condição específica não é substituída por outra validação parecida.                                                                                                                                                                                                                                                                                          |
| 68 | <code>banco.transacao(c-&gt;{autorizacao.exigirAtendimento(c,s); auditoria.registrar(c,s.usuarioId(), "COPIAR_RELATORIO", "arquivo", 0); return null;});</code>                                                   | Executa o callback como unidade de gravação. O resultado só sai após <code>commit</code> ; falha provoca <code>rollback</code> .<br><br>Confere no banco o acesso vigente do usuário para esta operação: <code>exigirAtendimento(c, s)</code> .<br><br>Registra o evento de auditoria na conexão fornecida: <code>registrar(c, s.usuarioId(), "COPIAR_RELATORIO", "arquivo", 0)</code> .<br><br><code>return null</code> encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à <code>lambda</code> ou ao método externo. |
| 69 | <code>return Files.move(temporario, destino);</code>                                                                                                                                                              | Retorno: entrega <code>Files.move(temporario, destino)</code> ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.<br><br>Invoca <code>Files.move</code> com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.                                                                                                                                                                                                                                                                                   |
| 70 | <code>} catch(RuntimeException   IOException erro) { try { Files.deleteIfExists(temporario); } catch(IOException limpeza) { erro.addSuppressed(limpeza); } throw erro; }</code>                                   | Trata a(s) exceção(ões) declarada(s) em <code>catch(RuntimeException   IOException erro)</code> . O tratamento mostrado abaixo decide se avisa, repete ou relança.<br><br>Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.<br><br>Trata a(s) exceção(ões) declarada(s) em <code>catch(IOException limpeza)</code> . O tratamento mostrado abaixo decide se avisa, repete ou relança.<br><br>Lança <code>erro</code> . Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar <code>rollback</code> .                                                              |
| 71 | <code>}</code>                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 72 |                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |

## hash - L73 a L76

Lê bytes por `DigestInputStream` para SHA-256. `transferTo(nullOutputStream)` consome o conteúdo sem guardar outra cópia em memória.

Entradas declaradas: (Path arquivo).

Saída declarada: `byte[]` - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: `ArquivosServico.copiarRelatorio` (L60).

Chamadas observadas no corpo: `MessageDigest.getInstance`, `new DigestInputStream`, `Files.newInputStream`, `in.transferTo`, `OutputStream.nullOutputStream`, `digest.digest`, `new IllegalStateException`.

```
73     private byte[] hash(Path arquivo) throws IOException {
```

Início da declaração de hash. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
74         try { MessageDigest digest=MessageDigest.getInstance("SHA-256");try(InputStream in=new
DigestInputStream(Files.newInputStream(arquivo),digest)) { in.transferTo(OutputStream.nullOutputStream());
} return digest.digest(); }
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Declara variável local `digest` do tipo `MessageDigest`. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

Retorno: entrega `digest.digest()` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

`new` constrói `DigestInputStream`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
75         catch(NoSuchAlgorithmException erro) { throw new IllegalStateException("SHA-256
indisponivel.",erro); }
```

Trata a(s) exceção(ões) declarada(s) em `catch(NoSuchAlgorithmException erro)`. O tratamento mostrado abaixo decide se avisa, repete ou relança.

Lança `new IllegalStateException("SHA-256 indisponivel.",erro)`. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

`new` constrói `IllegalStateException`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
76     }
77 }
```

## AtendimentoServico

src/main/java/br/edu/aeroporto/servico/AtendimentoServico.java

Conecta item confirmado, documento, bilhete, assento, check-in, bagagem e embarque; aplica janelas e cobertura financeira.

161 linhas | 10 métodos/construtores declarados | SHA-256: fc956da919e23ff8...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, FinanceiroJdbc, ReservaJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.infraestrutura.jdbc.*;
5 import java.math.BigDecimal;
6 import java.sql.Connection;
7 import java.sql.SQLException;
8 import java.time.Clock;
9 import java.time.Instant;
10 import java.time.LocalDate;
11 import java.util.Map;
12 import java.util.Set;
13
14 public final class AtendimentoServico {
    Declara AtendimentoServico como tipo com campos e métodos. final impede herança.

15     private static final Map<String,Set<String>> BAGAGEM=Map.of(
        Campo(s) BAGAGEM: private restringe acesso à classe. Tipo Map<String,Set<String>>: guarda referência/valor do tipo
        declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo
        depois da inicialização.

16         "RECEBIDA",Set.of("INSPECIONADA","RETIRADA","EXTRAVIADA"),
        Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o
        argumento/resultado daquela instrução.

17         "INSPECIONADA",Set.of("CARREGADA","RETIRADA","EXTRAVIADA"),
        Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o
        argumento/resultado daquela instrução.

18         "CARREGADA",Set.of("DESCARREGADA","RETIRADA","EXTRAVIADA"),
        Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o
        argumento/resultado daquela instrução.

19         "DESCARREGADA",Set.of("ENTREGUE","EXTRAVIADA"),
        Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o
        argumento/resultado daquela instrução.

20         "EXTRAVIADA",Set.of("RECEBIDA","ENTREGUE","RETIRADA"));
        Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o
        argumento/resultado daquela instrução.

21     private final BancoDados banco;
        Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
        desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

22     private final AutorizacaoJdbc autorizacao;
        Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
        desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

23     private final AuditoriaJdbc auditoria;
        Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
        desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

24     private final Clock relógio;
        Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

25     private final ReservaJdbc reservas=new ReservaJdbc();
        Campo(s) reservas: private restringe acesso à classe. Tipo ReservaJdbc: guarda referência/valor do tipo declarado, disponível
        aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```

new constrói ReservaJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
26 private final FinanceiroJdbc financeiro=new FinanceiroJdbc();
```

Campo(s) financeiro: private restringe acesso à classe. Tipo FinanceiroJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

new constrói FinanceiroJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
27
```

## AtendimentoServico - L28 a L30

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock relógio).

Possíveis chamadores identificados: Main.main (L55).

```
28 public AtendimentoServico(BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock
    relógio) {
```

Início da declaração de AtendimentoServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
29 this.banco=banco;this.autorizacao=autorizacao;this.auditoria=auditoria;this.relogio=relógio;
```

Instrução idêntica à de ArquivosServico:L21. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
30 }
```

```
31
```

## checkin - L32 a L58

Exige item confirmado e voo CHECKIN\_ABERTO na janela, pessoa ativa, documento apropriado/válido, bilhete emitido, assento não bloqueado e cobertura. Check-in ativo retorna id existente; cancelado é renovado por upsert com novo código.

Entradas declaradas: (Sessao s,long item,long documento).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.bagagensEmConexao (L295);

IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.atendimento (L138).

Chamadas observadas no corpo: banco.transacaoRepetivel, autorizacao.exigirAtendimento, BloqueiosJdbc.item, Sql.registro, Dados.id, Dados.exigir, "CONFIRMADO".equals, i.get, "CHECKIN\_ABERTO".equals, v.get, janela, PlanejamentoJdbc.efetivo(v,"partida").atZone(java.time.ZoneOffset.UTC).toLocalDate, PlanejamentoJdbc.efetivo(v,"partida").atZone, PlanejamentoJdbc.efetivo, d.get, ((java.sql.Date)d.get("validade")).toLocalDate().isBefore, ((java.sql.Date)d.get("validade")).toLocalDate, Set.of("CPF","RG","PASSAPORTE").contains, Set.of, "PASSAPORTE".equals, financeiro.recebido(c,Dados.id(i,"reserva\_id")).compareTo, financeiro.recebido, financeiro.devido, Sql.unico, Sql::linha, anterior.isPresent, anterior.get().get, anterior.get, Sql.inserir, s.usuarioId, Dados.codigo, relógio.instant, auditoria.registrar.

```
32 public long checkin(Sessao s,long item,long documento) {
```

Início da declaração de checkin. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
33 return banco.transacaoRepetivel(c->{
```

Retorno: entrega banco.transacaoRepetivel(c->{ autorizacao.exigirAtendimento(c,s);var i=BloqueiosJdbc.item(c,item);var v=Sql.registro(c,"SELECT v.\*,r.origem\_id,r.destino\_id FROM voo v JOIN rota r ON r.id=v.rota\_id WHERE v.id=?",Dados.... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa uma transação completa com até três tentativas totais somente para deadlock/serialização (40P01/40001). O callback pode executar novamente.

```
34 autorizacao.exigirAtendimento(c,s);var i=BloqueiosJdbc.item(c,item);var
v=Sql.registro(c,"SELECT v.*,r.origem_id,r.destino_id FROM voo v JOIN rota r ON r.id=v.rota_id WHERE
v.id=?",Dados.id(i,"voo_id"));
```

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

Declara variável local i com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.



Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo, rota. Parâmetros ligados em ordem: Dados.id(i,"voo\_id").

```
35      Dados.exigir("CONFIRMADO".equals(i.get("situacao")), "Item nao confirmado.");
```

Regra obrigatória: "CONFIRMADO".equals(i.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Item nao confirmado.". Essa condição específica não é substituída por outra validação parecida.

```
36      Dados.exigir("CHECKIN_ABERTO".equals(v.get("situacao")), "Check-in nao aberto para o voo.");
```

Regra obrigatória: "CHECKIN\_ABERTO".equals(v.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Check-in nao aberto para o voo.". Essa condição específica não é substituída por outra validação parecida.

```
37      janela(v, "checkin_abre", "checkin_fecha");
```

Chama AtendimentoServico.janela: Usa agora do Clock e limites efetivos ajustados pelo atraso; início é inclusivo e fim é exclusivo.

```
38      Sql.registro(c, "SELECT id FROM pessoa WHERE id=? AND ativo", Dados.id(i, "passageiro_id"));
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pessoa. Parâmetros ligados em ordem: Dados.id(i,"passageiro\_id").

```
39      var d=Sql.registro(c, "SELECT * FROM documento_pessoa WHERE id=? AND  
pessoa_id=?", documento, Dados.id(i, "passageiro_id"));
```

Declara variável local d com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: documento\_pessoa. Parâmetros ligados em ordem: documento, Dados.id(i,"passageiro\_id").

```
40      LocalDate  
dia=PlanejamentoJdbc.efetivo(v, "partida").atZone(java.time.ZoneOffset.UTC).toLocalDate();
```

Declara variável local dia do tipo LocalDate. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
41      Dados.exigir(d.get("validade")==null ||  
!((java.sql.Date)d.get("validade")).toLocalDate().isBefore(dia), "Documento vencido.");
```

Regra obrigatória: d.get("validade")==null || ((java.sql.Date)d.get("validade")).toLocalDate().isBefore(dia) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Documento vencido.". Essa condição específica não é substituída por outra validação parecida.

```
42      long paisOrigem=Dados.id(Sql.registro(c, "SELECT ci.pais_id FROM aeroporto a JOIN cidade ci ON  
ci.id=a.cidade_id WHERE a.id=?", Dados.id(v, "origem_id")), "pais_id");
```

Declara variável local paisOrigem do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeroporto, cidade. Parâmetros ligados em ordem: Dados.id(v,"origem\_id").

```
43      long paisDestino=Dados.id(Sql.registro(c, "SELECT ci.pais_id FROM aeroporto a JOIN cidade ci ON  
ci.id=a.cidade_id WHERE a.id=?", Dados.id(v, "destino_id")), "pais_id");
```

Declara variável local paisDestino do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeroporto, cidade. Parâmetros ligados em ordem: Dados.id(v,"destino\_id").

```
44      Dados.exigir(paisOrigem==paisDestino?Set.of("CPF","RG","PASSAPORTE").contains(d.get("tipo")):"  
PASSAPORTE".equals(d.get("tipo")) && d.get("validade")!=null, "Documento inadequado ao trecho.");
```

Regra obrigatória:

paisOrigem==paisDestino?Set.of("CPF","RG","PASSAPORTE").contains(d.get("tipo")):"PASSAPORTE".equals(d.get("tipo")) && d.get("validade")!=null deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Documento inadequado ao trecho.". Essa condição específica não é substituída por outra validação parecida.

Escolha de valor: paisOrigem==paisDestino. true escolhe Set.of("CPF","RG","PASSAPORTE").contains(d.get("tipo")); false escolhe "PASSAPORTE".equals(d.get("tipo")) && d.get("validade")!=null. Só a alternativa escolhida é avaliada.

```
45      long bilhete=Dados.id(Sql.registro(c, "SELECT id FROM bilhete WHERE item_id=? AND  
situacao='EMITIDO'", item), "id");
```

Declara variável local bilhete do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: bilhete. Parâmetros ligados em ordem: item.

```
46      var o=Sql.registro(c, "SELECT * FROM ocupacao_assento WHERE item_id=? AND liberada_em IS  
NULL", item);
```

Declara variável local o com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.



Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: ocupacao\_assento. Parâmetros ligados em ordem: item.

```
47      Sql.registro(c,"SELECT id FROM inventario_assento_voo WHERE id=? AND NOT bloqueado FOR
UPDATE",Dados.id(o,"inventario_id"));
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: inventario\_assento\_voo. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: Dados.id(o,"inventario\_id").

```
48      Dados.exigir(financieiro.recebido(c,Dados.id(i,"reserva_id")).compareTo(financieiro.devido(c,Dad
os.id(i,"reserva_id")))>=0,"Reserva sem cobertura financeira.");
```

Regra obrigatória: financeiro.recebido(c,Dados.id(i,"reserva\_id")).compareTo(financieiro.devido(c,Dados.id(i,"reserva\_id")))>=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Reserva sem cobertura financeira.". Essa condição específica não é substituída por outra validação parecida.

```
49      var anterior=Sql.unico(c,"SELECT * FROM check_in WHERE item_id=?",Sql::linha,item);
```

Declara variável local anterior com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: check\_in. Parâmetros ligados em ordem: item.

```
50      if(anterior.isPresent() && anterior.get().get("cancelado_em")==null) return
Dados.id(anterior.get(),"id");
```

Decisão específica: (anterior.isPresent() && anterior.get().get("cancelado\_em")==null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega Dados.id(anterior.get(),"id") ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
51      long id=Sql.inserir(c,""
```

Instrução idêntica à de PlanejamentoJdbc:L45. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
52      INSERT INTO
check_in(item_id,voo_id,bilhete_id,ocupacao_id,usuario_id,codigo_cartao,realizado_em)
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
53      VALUES (?, ?, ?, ?, ?, ?) ON CONFLICT (item_id) DO UPDATE SET
ocupacao_id=EXCLUDED.ocupacao_id,usuario_id=EXCLUDED.usuario_id,
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
54      codigo_cartao=EXCLUDED.codigo_cartao,realizado_em=EXCLUDED.realizado_em,cancelado_em=NULL RETURNING id
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```
55      "",item,Dados.id(i,"voo_id"),bilhete,Dados.id(o,"id"),s.usuarioId(),Dados.codigo("C",
32),religio.instant());
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
56      auditoria.registrar(c,s.usuarioId(),"REALIZAR_CHECKIN","check_in",id);return id;
```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "REALIZAR\_CHECKIN", "check\_in", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
57      });
```

```
58      }
```

```
59
```

## cancelarCheckin - L60 a L74

Só antes de fechar embarque, item confirmado e passageiro não embarcado. Marca bagagens retiradas, cancela cartão/check-in e solicita devolução das taxas elegíveis.

Entradas declaradas: (Sessao s,long item).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.atendimento (L138).

Chamadas observadas no corpo: banco.transacaoRepetivel, autorizacao.exigirAtendimento, BloqueiosJdbc.item, Sql.registro, Dados.id, Dados.exigir, Set.of("CHECKIN\_ABERTO","EMBARQUE").contains, Set.of, v.get, religio.instant().isBefore, religio.instant, ReservaJdbc.janela, "CONFIRMADO".equals, i.get, Sql.listar(c,"SELECT id FROM embarque WHERE checkin\_id=?",r->r.getLong(1),Dados.id(ci,"id")).isEmpty, Sql.listar,

```
r.getLong, Sql.unico(c,"SELECT COALESCE(SUM(taxa_excesso),0) FROM bagagem WHERE checkin_id=? AND
situacao<>'RETIRADA'",r->r.getLong(1),Dados.id(ci,"id")).orElse, Sql.unico, r.getLong(1), evento, Sql.executar, reservas.solicitarDevolucao,
Dados.codigo, auditoria.registrar, s.usuarioId.
```

```
60 public void cancelarCheckin(Sessao s, long item) {
```

Início da declaração de cancelarCheckin. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
61 banco.transacaoRepetivel(c->{
```

Executa uma transação completa com até três tentativas totais somente para deadlock/serialização (40P01/40001). O callback pode executar novamente.

```
62 autorizacao.exigirAtendimento(c,s);var i=BloqueiosJdbc.item(c,item);var
v=Sql.registro(c,"SELECT * FROM voo WHERE id=?",Dados.id(i,"voo_id"));
```

Confere no banco o acesso vigente do usuário para esta operação: `exigirAtendimento(c, s)`.

Declara variável local `i` com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local `v` com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como `Map<String,Object>`; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: `voo`. Parâmetros ligados em ordem: `Dados.id(i,"voo_id")`.

```
63 Dados.exigir(Set.of("CHECKIN_ABERTO","EMBARQUE").contains(v.get("situacao")) &&
relogio.instant().isBefore(ReservaJdbc.janela(v,"embarque_fecha"),"Prazo de cancelamento do check-in
encerrado."));
```

Regra obrigatória: `Set.of("CHECKIN_ABERTO","EMBARQUE").contains(v.get("situacao")) && relogio.instant().isBefore(ReservaJdbc.janela(v,"embarque_fecha"))` deve resultar em `true`. Se `false`, `Dados.exigir` lança `RegraNegocioException` com "Prazo de cancelamento do check-in encerrado.". Essa condição específica não é substituída por outra validação parecida.

```
64 Dados.exigir("CONFIRMADO".equals(i.get("situacao")), "Item nao admite cancelamento de
check-in.");
```

Regra obrigatória: `"CONFIRMADO".equals(i.get("situacao"))` deve resultar em `true`. Se `false`, `Dados.exigir` lança `RegraNegocioException` com "Item nao admite cancelamento de check-in.". Essa condição específica não é substituída por outra validação parecida.

```
65 var ci=Sql.registro(c,"SELECT * FROM check_in WHERE item_id=? AND cancelado_em IS NULL FOR
UPDATE",item);
```

Declara variável local `ci` com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como `Map<String,Object>`; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: `check_in`. O `FOR UPDATE` mantém lock das linhas até `commit/rollback`. Parâmetros ligados em ordem: `item`.

```
66 Dados.exigir(Sql.listar(c,"SELECT id FROM embarque WHERE
checkin_id=?",r->r.getLong(1),Dados.id(ci,"id")).isEmpty(),"Passageiro ja embarcou.");
```

Regra obrigatória: `Sql.listar(c,"SELECT id FROM embarque WHERE checkin_id=?",r->r.getLong(1),Dados.id(ci,"id")).isEmpty()` deve resultar em `true`. Se `false`, `Dados.exigir` lança `RegraNegocioException` com "Passageiro ja embarcou.". Essa condição específica não é substituída por outra validação parecida.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve `List<T>`. Tabelas mencionadas: `embarque`. Parâmetros ligados em ordem: `Dados.id(ci,"id")`.

```
67 BigDecimal taxas=Sql.unico(c,"SELECT COALESCE(SUM(taxa_excesso),0) FROM bagagem WHERE
checkin_id=? AND situacao<>'RETIRADA'",r->r.getLong(1),Dados.id(ci,"id")).orElse(BigDecimal.ZERO);
```

Declara variável local `taxas` do tipo `BigDecimal`. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega `Optional` e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: `bagagem`. `SUM` agrega valores no PostgreSQL; `COALESCE`, quando presente, define o caso sem valores. Parâmetros ligados em ordem: `Dados.id(ci,"id")`.

```
68 var rota=Sql.registro(c,"SELECT origem_id FROM rota WHERE id=?",Dados.id(v,"rota_id"));
```

Declara variável local `rota` com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como `Map<String,Object>`; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: `rota`. Parâmetros ligados em ordem: `Dados.id(v,"rota_id")`.

```

69         for(long id:Sql.listar(c,"SELECT id FROM bagagem WHERE checkin_id=? AND situacao NOT IN
('RETIRADA','ENTREGUE') ORDER BY id FOR UPDATE",r->r.getLong(1),Dados.id(ci,"id")))
evento(c,s,id,Dados.id(rota,"origem_id"),"RETIRADA","Cancelamento de check-in");

```

Percorre cada elemento da coleção/vetor: for(long id:Sql.listar(c,"SELECT id FROM bagagem WHERE checkin\_id=? AND situacao NOT IN ('RETIRADA','ENTREGUE') ORDER BY id FOR UPDATE",r->r.getLong(1),Dados.id(ci,"id"))) return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: bagagem. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: Dados.id(ci,"id").

```

70         Sql.executar(c,"UPDATE check_in SET cancelado_em=? WHERE
id=?",religio.instant(),Dados.id(ci,"id"));

```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: check\_in. Parâmetros ligados em ordem: religio.instant(), Dados.id(ci,"id").

```

71         reservas.solicitarDevolucao(c,Dados.id(i,"reserva_id"),taxas,"ci:"+Dados.id(ci,"id")+":"+Dados
.codigo("",12),"Cancelamento de despacho");

```

Chama ReservaJdbc.solicitarDevolucao: Limita o solicitado ao recebido livre. Divide entre pagamentos aprovados, desconta reembolsos comprometidos e usa chave por pagamento para evitar repetição.

```

72         auditoria.registrar(c,s.usuarioId(),"CANCELAR_CHECKIN","check_in",Dados.id(ci,"id"));return
null;

```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "CANCELAR\_CHECKIN", "check\_in", Dados.id(ci,"id")).

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```

73     });
74 }
75

```

## cartao - L76 a L86

Consulta dados do cartão para check-in ativo, bilhete emitido e ocupação vigente. Retorna Map, sem gerar novo embarque.

Entradas declaradas: (Sessao s,long item).

Saída declarada: Map<String,Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.cancelamentoAposEmbarque (L309); MenuOperacional.atendimento (L138).

Chamadas observadas no corpo: banco.consultar, autorizacao.exigirAtendimento, Sql.registro.

```

76     public Map<String,Object> cartao(Sessao s,long item) {

```

Início da declaração de cartao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

77         return banco.consultar(c->{autorizacao.exigirAtendimento(c,s);return Sql.registro(c,""

```

Retorno: entrega banco.consultar(c->{autorizacao.exigirAtendimento(c,s);return Sql.registro(c,"" SELECT ci.id,ci.codigo\_cartao,p.nome,b.numero AS bilhete,v.numero AS voo,a.codigo,ro.codigo\_icao AS origem,rd.codigo\_icao AS destino, v.... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

Retorno: entrega Sql.registro(c,"" SELECT ci.id,ci.codigo\_cartao,p.nome,b.numero AS bilhete,v.numero AS voo,a.codigo,ro.codigo\_icao AS origem,rd.codigo\_icao AS destino, v.partida\_prevista,v.partida\_estimada,v.embarque\_abre,v.embarque... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: check\_in, item\_reserva, pessoa, bilhete, voo, rota, aeroporto, ocupacao\_assento, inventario\_assento\_voo. Parâmetros ligados em ordem: item.

```

78         SELECT ci.id,ci.codigo_cartao,p.nome,b.numero AS bilhete,v.numero AS
voo,a.codigo,ro.codigo_icao AS origem,rd.codigo_icao AS destino,

```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```

79         v.partida_prevista,v.partida_estimada,v.embarque_abre,v.embarque_fecha

```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```
80      FROM check_in ci JOIN item_reserva i ON i.id=ci.item_id JOIN pessoa p ON
      p.id=i.passageiro_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
81      JOIN bilhete b ON b.id=ci.bilhete_id JOIN voo v ON v.id=ci.voo_id JOIN rota r ON
      r.id=v.rota_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
82      JOIN aeroporto ro ON ro.id=r.origem_id JOIN aeroporto rd ON rd.id=r.destino_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
83      JOIN ocupacao_assento o ON o.id=ci.ocupacao_id JOIN inventario_assento_voo a ON
      a.id=o.inventario_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
84      WHERE ci.item_id=? AND ci.cancelado_em IS NULL AND b.situacao='EMITIDO' AND o.liberada_em
      IS NULL
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
85      """, item));
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
86    }
87
```

## despachar - L88 a L104

Valida peso/categoria e etiqueta; requer janela e check-in aberto. Repetir etiqueta exige mesmos dados. Calcula somente incremento de excesso, cria bagagem/evento e retorna id.

Entradas declaradas: (Sessao s,long item,String etiqueta,BigDecimal peso,String categoria).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.bagagensEmConexao (L295); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.atendimento (L138).

Chamadas observadas no corpo: Dados.valor, Dados.exigir, Set.of("DESPACHADA", "ESPECIAL").contains, Set.of, kg.compareTo, new BigDecimal, categoria.equals, etiqueta.isBlank, Dados.codigo, Validacao.texto, banco.transacaoRepetivel, autorizacao.exigirAtendimento, BloqueiosJdbc.item, Sql.registro, Dados.id, "CONFIRMADO".equals, i.get, "CHECKIN\_ABERTO".equals, v.get, janela, Sql.unico, Sql::linha, antiga.isPresent, antiga.get, Dados.dinheiro, antiga.get().get, ((Number)resumo.get("pecas")).intValue, resumo.get, excesso, Dados.dinheiro(resumo, "kg").add, Sql.inserir, depois.subtract, evento, auditoria.registrar, s.usuarioId.

```
88    public long despachar(Sessao s,long item,String etiqueta,BigDecimal peso,String categoria) {
      Início da declaração de despachar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
      invocado.
```

```
89    BigDecimal
      kg=Dados.valor(peso,true);Dados.exigir(Set.of("DESPACHADA", "ESPECIAL").contains(categoria) &&
      kg.compareTo(new BigDecimal(categoria.equals("ESPECIAL")?"60":"32"))<=0,"Categoria ou peso invalido.");
      Declara variável local kg do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Regra obrigatória: Set.of("DESPACHADA", "ESPECIAL").contains(categoria) && kg.compareTo(new
      BigDecimal(categoria.equals("ESPECIAL")?"60":"32"))<=0 deve resultar em true. Se false, Dados.exigir lança
      RegraNegocioException com "Categoria ou peso invalido.". Essa condição específica não é substituída por outra validação
      parecida.

      Escolha de valor: categoria.equals("ESPECIAL"). true escolhe "60"; false escolhe "32". Só a alternativa escolhida é avaliada.

      new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si
      só.
```

```
90    String tag=etiqueta==null ||
      etiqueta.isBlank()?Dados.codigo("M",24):Validacao.texto(etiqueta,"Etiqueta",30);
      Declara variável local tag do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Escolha de valor: etiqueta==null || etiqueta.isBlank(). true escolhe Dados.codigo("M",24); false escolhe
      Validacao.texto(etiqueta,"Etiqueta",30). Só a alternativa escolhida é avaliada.
```

```
91    return banco.transacaoRepetivel(c->{
      Instrução idêntica à de AtendimentoServico:L33. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
      desta execução. A posição e o contexto do método acima mostram quando ela ocorre.
```

```
92      autorizacao.exigirAtendimento(c,s);var i=BloqueiosJdbc.item(c,item);var
v=Sql.registro(c,"SELECT * FROM voo WHERE id=?",Dados.id(i,"voo_id"));
```

Instrução idêntica à de AtendimentoServico:L62. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
93      Dados.exigir("CONFIRMADO".equals(i.get("situacao")) &&
"CHECKIN_ABERTO".equals(v.get("situacao")), "Despacho requer item confirmado e check-in
aberto.");janela(v,"checkin_abre","checkin_fecha");
```

Regra obrigatória: "CONFIRMADO".equals(i.get("situacao")) && "CHECKIN\_ABERTO".equals(v.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Despacho requer item confirmado e check-in aberto.". Essa condição específica não é substituída por outra validação parecida.

```
94      var ci=Sql.registro(c,"SELECT * FROM check_in WHERE item_id=? AND cancelado_em IS NULL",item);
```

Declara variável local ci com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: check\_in. Parâmetros ligados em ordem: item.

```
95      var antiga=Sql.unico(c,"SELECT * FROM bagagem WHERE etiqueta=?",Sql::linha,tag);
```

Declara variável local antiga com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: bagagem. Parâmetros ligados em ordem: tag.

```
96      if(antiga.isPresent()) { Dados.exigir(Dados.id(antiga.get(),"checkin_id")==Dados.id(ci,"id")
&& kg.compareTo(Dados.dinheiro(antiga.get(),"peso_kg"))==0 &&
categoria.equals(antiga.get().get("categoria")), "Etiqueta ja usada para outra bagagem.");return
Dados.id(antiga.get(),"id"); }
```

Decisão específica: (antiga.isPresent()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: Dados.id(antiga.get(),"checkin\_id")==Dados.id(ci,"id") && kg.compareTo(Dados.dinheiro(antiga.get(),"peso\_kg"))==0 && categoria.equals(antiga.get().get("categoria")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Etiqueta ja usada para outra bagagem.". Essa condição específica não é substituída por outra validação parecida.

Retorno: entrega Dados.id(antiga.get(),"id") ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
97      var resumo=Sql.registro(c,"SELECT COUNT(*) AS pecas,COALESCE(SUM(peso_kg),0) AS kg FROM
bagagem WHERE checkin_id=? AND situacao<>'RETIRADA'",Dados.id(ci,"id"));
```

Declara variável local resumo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: bagagem. COUNT calcula uma contagem no banco, não carrega cada objeto para contar em Java. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: Dados.id(ci,"id").

```
98      int quantidade=((Number)resumo.get("pecas")).intValue();Dados.exigir(quantidade<10,"Limite
operacional de 10 pecas.");
```

Declara variável local quantidade do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Regra obrigatória: quantidade<10 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Limite operacional de 10 pecas.". Essa condição específica não é substituída por outra validação parecida.

```
99      BigDecimal antes=excesso(Dados.dinheiro(resumo,"kg"),quantidade,i),depois=excesso(Dados.dinhei
ro(resumo,"kg").add(kg),quantidade+1,i);
```

Declara variável local antes, depois do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
100     long id=Sql.inserir(c,"INSERT INTO bagagem(checkin_id,etiqueta,peso_kg,categoria,taxa_excesso)
VALUES (?, ?, ?, ?, ?) RETURNING id",Dados.id(ci,"id"),tag,kg,categoria,depois.subtract(antes));
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: bagagem. Parâmetros ligados em ordem: Dados.id(ci,"id"), tag, kg, categoria, depois.subtract(antes).

```
101     long origem=Dados.id(Sql.registro(c,"SELECT origem_id FROM rota WHERE
id=?",Dados.id(v,"rota_id")), "origem_id");
```

Declara variável local origem do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: rota. Parâmetros ligados em ordem: Dados.id(v,"rota\_id").

```

102         evento(c,s,id,origem,"RECEBIDA","Despacho");auditoria.registrar(c,s.usuarioId(),"DESPACHAR_BAG
    AGEN","bagagem",id);return id;

```

Chama AtendimentoServico.evento: Atualiza situação atual da bagagem e insere outro registro em evento\_bagagem. Estado atual e histórico têm funções diferentes.

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "DESPACHAR\_BAGAGEM", "bagagem", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```

103     });
104 }
105

```

## excesso - L106 a L109

Taxa total = max(0, peso-franquia)\*30 + max(0, peças-limite)\*100. Exemplo: franquia de 23 kg, peso de 25 kg, uma peça inclusa => R\$60.

Entradas declaradas: (BigDecimal kg,int pecas,Map<String,Object> item).

Saída declarada: BigDecimal - valor decimal, utilizado nos cálculos financeiros.

Possíveis chamadores identificados: AtendimentoServico.despachar (L88).

Chamadas observadas no corpo: kg.subtract(Dados.dinheiro(item,"franquia\_bagagem\_kg")).max(BigDecimal.ZERO).multiply(new BigDecimal("30")).add, kg.subtract(Dados.dinheiro(item,"franquia\_bagagem\_kg")).max(BigDecimal.ZERO).multiply, kg.subtract(Dados.dinheiro(item,"franquia\_bagagem\_kg")).max, kg.subtract, Dados.dinheiro, new BigDecimal, new BigDecimal(Math.max(0,pecas-((Number)item.get("limite\_pecas")).intValue())).multiply, Math.max, ((Number)item.get("limite\_pecas")).intValue, item.get.

```

106     private BigDecimal excesso(BigDecimal kg,int pecas,Map<String,Object> item) {

```

Início da declaração de excesso. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

107         return kg.subtract(Dados.dinheiro(item,"franquia_bagagem_kg")).max(BigDecimal.ZERO).multiply(new
    BigDecimal("30"))

```

Retorno: entrega kg.subtract(Dados.dinheiro(item,"franquia\_bagagem\_kg")).max(BigDecimal.ZERO).multiply(new BigDecimal("30")).add(new BigDecimal(Math.max(0,pecas-((Number)item.get("limite\_pecas")).intValue())).multiply(new BigDecimal(... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca kg.subtract(Dados.dinheiro(item,"franquia\_bagagem\_kg")).max(BigDecimal.ZERO).multiply com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

Invoca kg.subtract(Dados.dinheiro(item,"franquia\_bagagem\_kg")).max com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

Invoca kg.subtract com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

108         .add(new
    BigDecimal(Math.max(0,pecas-((Number)item.get("limite_pecas")).intValue())).multiply(new
    BigDecimal("100")));

```

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

109     }
110

```

## rastrear - L111 a L130

Confere transição no mapa BAGAGEM, local do evento, chegada para descarregar/entregar e cobertura para carregar. RETIRADA solicita devolução da taxa; atualiza e registra histórico.

Entradas declaradas: (Sessao s,String etiqueta,long aeroporto,String estado,String observacao).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.bagagensEmConexao (L295);

MenuOperacional.atendimento (L138).

Chamadas observadas no corpo: Validacao.texto, banco.transacaoRepetivel, autorizacao.exigirOperacao, Sql.registro, BloqueiosJdbc.item, Dados.id, Dados.exigir, BAGAGEM.getOrDefault(Dados.texto(b,"situacao"),Set.of()).contains, BAGAGEM.getOrDefault, Dados.texto, Set.of, Set.of("DESCARREGADA","ENTREGUE").contains, estado.equals, v.get, Dados.instante(v,"chegada\_real").isAfter, Dados.instante, relógio.instant, financeiro.recebido(c,Dados.id(i,"reserva\_id")).compareTo, financeiro.recebido, financeiro.devido, evento, reservas.solicitarDevolucao, Dados.dinheiro, auditoria.registrar, s.usuarioId.



```

111 public void rastrear(Sessao s,String etiqueta,long aeroporto,String estado,String observacao) {
    Início da declaração de rastrear. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

112 String nota=Validacao.texto(observacao,"Observacao",500);
    Declara variável local nota do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

113 banco.transacaoRepetivel(c->{
    Instrução idêntica à de AtendimentoServico:L61. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
    desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

114 autorizacao.exigirOperacao(c,s);var inicial=Sql.registro(c,"SELECT b.id,ci.item_id FROM
    bagagem b JOIN check_in ci ON ci.id=b.checkin_id WHERE b.etiqueta=?",etiqueta);
    Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, s).

    Declara variável local inicial com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.

    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    bagagem, check_in. Parâmetros ligados em ordem: etiqueta.

115 var i=BloqueiosJdbc.item(c,Dados.id(inicial,"item_id"));var b=Sql.registro(c,"SELECT * FROM
    bagagem WHERE id=? FOR UPDATE",Dados.id(inicial,"id"));
    Declara variável local i com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
    bloco atual.

    Declara variável local b com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é
    o bloco atual.

    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    bagagem. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: Dados.id(inicial,"id").

116 Dados.exigir(BAGAGEM.getDefault(Dados.texto(b,"situacao"),Set.of()).contains(estado),"Transicao de
    bagagem invalida.");
    Regra obrigatória: BAGAGEM.getDefault(Dados.texto(b,"situacao"),Set.of()).contains(estado) deve resultar em true. Se false,
    Dados.exigir lança RegraNegocioException com "Transicao de bagagem invalida.". Essa condição específica não é substituída
    por outra validação parecida.

117 var r=Sql.registro(c,"SELECT r.* FROM voo v JOIN rota r ON r.id=v.rota_id WHERE
    v.id=?",Dados.id(i,"voo_id"));
    Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
    bloco atual.

    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    voo, rota. Parâmetros ligados em ordem: Dados.id(i,"voo_id").

118 long esperado=Set.of("DESCARREGADA","ENTREGUE").contains(estado)?Dados.id(r,"destino_id"):Dados
    s.id(r,"origem_id");
    Declara variável local esperado do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    Escolha de valor: Set.of("DESCARREGADA","ENTREGUE").contains(estado). true escolhe Dados.id(r,"destino_id"); false escolhe
    Dados.id(r,"origem_id"). Só a alternativa escolhida é avaliada.

119 if(estado.equals("EXTRAVIADA")) Dados.exigir(aeroporto==Dados.id(r,"origem_id") ||
    aeroporto==Dados.id(r,"destino_id"),"Aeroporto fora do trecho.");
    Decisão específica: (estado.equals("EXTRAVIADA")). Se for true, executa o ramo seguinte; se for false, pula esse ramo e executa
    o else.

    Regra obrigatória: aeroporto==Dados.id(r,"origem_id") || aeroporto==Dados.id(r,"destino_id") deve resultar em true. Se false,
    Dados.exigir lança RegraNegocioException com "Aeroporto fora do trecho.". Essa condição específica não é substituída por outra
    validação parecida.

120 else Dados.exigir(aeroporto==esperado,"Evento em aeroporto incorreto.");
    Regra obrigatória: aeroporto==esperado deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Evento
    em aeroporto incorreto.". Essa condição específica não é substituída por outra validação parecida.

121 if(Set.of("DESCARREGADA","ENTREGUE").contains(estado)) {
    Decisão específica: (Set.of("DESCARREGADA","ENTREGUE").contains(estado)). Se for true, executa o ramo seguinte; se for
    false, pula esse ramo.

122 var v=Sql.registro(c,"SELECT * FROM voo WHERE id=?",Dados.id(i,"voo_id"));
    Instrução idêntica à de FinanceiroJdbc:L33. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
    execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

```

123         Dados.exigir(v.get("chegada_real")!=null &&
!Dados.instante(v,"chegada_real").isAfter(relogio.instant()),"Aeronave ainda nao chegou.");
    Regra obrigatória: v.get("chegada_real")!=null && !Dados.instante(v,"chegada_real").isAfter(relogio.instant()) deve resultar em
    true. Se false, Dados.exigir lança RegraNegocioException com "Aeronave ainda nao chegou.". Essa condição específica não é
    substituída por outra validação parecida.

124     }
125     if(estado.equals("CARREGADA")) Dados.exigir(financieiro.recebido(c,Dados.id(i,"reserva_id")).co
mpareTo(financieiro.devido(c,Dados.id(i,"reserva_id"))>=0,"Excesso de bagagem nao pago.");
    Decisão específica: (estado.equals("CARREGADA")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

    Regra obrigatória: financeiro.recebido(c,Dados.id(i,"reserva_id")).compareTo(financieiro.devido(c,Dados.id(i,"reserva_id"))>=0
    deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Excesso de bagagem nao pago.". Essa
    condição específica não é substituída por outra validação parecida.

126     evento(c,s,Dados.id(b,"id"),aeroporto,estado,nota);
    Chama AtendimentoServico.evento: Atualiza situação atual da bagagem e insere outro registro em evento_bagagem. Estado
    atual e histórico têm funções diferentes.

127     if(estado.equals("RETIRADA")) reservas.solicitarDevolucao(c,Dados.id(i,"reserva_id"),Dados.din
heiro(b,"taxa_excesso"),"retirada:"+Dados.id(b,"id"),"Retirada de bagagem");
    Decisão específica: (estado.equals("RETIRADA")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

128     auditoria.registrar(c,s.usuarioId(),"RASTREAR_BAGAGEM","bagagem",Dados.id(b,"id"));return
null;
    Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "RASTREAR_BAGAGEM", "bagagem",
    Dados.id(b,"id")).

    return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o
    contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

129     });
130 }
131

```

## embarcar - L132 a L151

Procura item pelo código do cartão, bloqueia e exige janela, situação EMBARQUE, bilhete e ocupação ativos, portão correto/disponível, alocação vigente, bagagens aptas e saldo coberto. Grava embarque e marca item/bilhete UTILIZADO.

Entradas declaradas: (Sessao s,String cartao,long alocao).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.cancelamentoAposEmbarque (L309); MenuOperacional.atendimento (L138).

Chamadas observadas no corpo: banco.transacaoRepetivel, autorizacao.exigirAtendimento, Sql.registro, BloqueiosJdbc.item, Dados.id, Dados.exigir, "EMBARQUE".equals, v.get, janela, "CONFIRMADO".equals, i.get, relogio.instant().isBefore, relogio.instant, Dados.instante, Sql.listar(c,"SELECT id FROM bagagem WHERE checkin\_id=? AND situacao NOT IN ('CARREGADA','RETIRADA')",r->r.getLong(1),Dados.id(ci,"id")).isEmpty, Sql.listar, r.getLong, financeiro.recebido(c,Dados.id(i,"reserva\_id")).compareTo, financeiro.recebido, financeiro.devido, Sql.inserir, s.usuarioId, Sql.executar, reservas.recalcular, auditoria.registrar.

```

132     public long embarcar(Sessao s,String cartao,long alocao) {
    Início da declaração de embarcar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

133     return banco.transacaoRepetivel(c->{
    Instrução idêntica à de AtendimentoServico:L33. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
    desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

134         autorizacao.exigirAtendimento(c,s);var inicial=Sql.registro(c,"SELECT item_id FROM check_in
WHERE codigo_cartao=?",cartao);var i=BloqueiosJdbc.item(c,Dados.id(inicial,"item_id"));
    Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

    Declara variável local inicial com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.

    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    check_in. Parâmetros ligados em ordem: cartao.

    Declara variável local i com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
    bloco atual.

135         var ci=Sql.registro(c,"SELECT * FROM check_in WHERE codigo_cartao=? AND cancelado_em IS
NULL",cartao);var v=Sql.registro(c,"SELECT * FROM voo WHERE id=?",Dados.id(i,"voo_id"));
    Declara variável local ci com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é
    o bloco atual.

```



Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: check\_in. Parâmetros ligados em ordem: cartao.

Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. Parâmetros ligados em ordem: Dados.id(i,"voo\_id").

```
136      Dados.exigir("EMBARQUE".equals(v.get("situacao")), "Voo nao esta em
embarque.");janela(v,"embarque_abre","embarque_fecha");
```

Regra obrigatória: "EMBARQUE".equals(v.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Voo nao esta em embarque.". Essa condição específica não é substituída por outra validação parecida.

```
137      Dados.exigir("CONFIRMADO".equals(i.get("situacao")), "Passagem ja utilizada ou invalida.");
```

Regra obrigatória: "CONFIRMADO".equals(i.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Passagem ja utilizada ou invalida.". Essa condição específica não é substituída por outra validação parecida.

```
138      Sql.registro(c,"SELECT id FROM bilhete WHERE id=? AND
situacao='EMITIDO'",Dados.id(ci,"bilhete_id"));
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: bilhete. Parâmetros ligados em ordem: Dados.id(ci,"bilhete\_id").

```
139      Sql.registro(c,"SELECT id FROM ocupacao_assento WHERE id=? AND liberada_em IS
NULL",Dados.id(ci,"ocupacao_id"));
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: ocupacao\_assento. Parâmetros ligados em ordem: Dados.id(ci,"ocupacao\_id").

```
140      var a=Sql.registro(c,""
```

Declara variável local a com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: alocao\_recurso, recurso\_aeroportuario, voo, rota. Parâmetros ligados em ordem: alocao, Dados.id(i,"voo\_id").

```
141      SELECT a.* FROM alocao_recurso a JOIN recurso_aeroportuario r ON r.id=a.recurso_id
JOIN voo v ON v.id=a.voo_id JOIN rota rt ON rt.id=v.rota_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
142      WHERE a.id=? AND a.voo_id=? AND a.ativa AND a.finalidade='PARTIDA' AND r.tipo='PORTAO'
AND r.situacao='DISPONIVEL' AND r.aeroporto_id=rt.origem_id
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
143      """,alocacao,Dados.id(i,"voo_id"));
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
144      Dados.exigir(!relogio.instant().isBefore(Dados.instante(a,"inicio")) &&
relogio.instant().isBefore(Dados.instante(a,"fim")), "Alocacao fora de horario.");
```

Regra obrigatória: !relogio.instant().isBefore(Dados.instante(a,"inicio")) && relogio.instant().isBefore(Dados.instante(a,"fim")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Alocacao fora de horario.". Essa condição específica não é substituída por outra validação parecida.

```
145      Dados.exigir(Sql.listar(c,"SELECT id FROM bagagem WHERE checkin_id=? AND situacao NOT IN
('CARREGADA','RETIRADA')",r->r.getLong(1),Dados.id(ci,"id")).isEmpty(),"Bagagem ainda nao carregada ou
extraviada.");
```

Regra obrigatória: Sql.listar(c,"SELECT id FROM bagagem WHERE checkin\_id=? AND situacao NOT IN ('CARREGADA','RETIRADA')",r->r.getLong(1),Dados.id(ci,"id")).isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Bagagem ainda nao carregada ou extraviada.". Essa condição específica não é substituída por outra validação parecida.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: bagagem. Parâmetros ligados em ordem: Dados.id(ci,"id").

```
146      Dados.exigir(financieiro.recebido(c,Dados.id(i,"reserva_id")).compareTo(financieiro.devido(c,Dad
os.id(i,"reserva_id")))>=0,"Reserva com saldo pendente.");
```

Regra obrigatória: financeiro.recebido(c,Dados.id(i,"reserva\_id")).compareTo(financieiro.devido(c,Dados.id(i,"reserva\_id")))>=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Reserva com saldo pendente.". Essa condição específica não é substituída por outra validação parecida.

```

147         long id=Sql.inserir(c,"INSERT INTO
embarque(checkin_id,voo_id,alocacao_id,usuario_id,realizado_em) VALUES (?, ?, ?, ?, ?) RETURNING
id",Dados.id(ci,"id"),Dados.id(i,"voo_id"),alocacao,s.usuarioId(),relogio.instant());

```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: embarque. Parâmetros ligados em ordem: Dados.id(ci,"id"), Dados.id(i,"voo\_id"), alocao, s.usuarioId(), relogio.instant().

```

148         Sql.executar(c,"UPDATE item_reserva SET situacao='UTILIZADO' WHERE
id=?",Dados.id(i,"id"));Sql.executar(c,"UPDATE bilhete SET situacao='UTILIZADO' WHERE
id=?",Dados.id(ci,"bilhete_id"));

```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: item\_reserva.

Parâmetros ligados em ordem: Dados.id(i,"id").

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: bilhete. Parâmetros ligados em ordem: Dados.id(ci,"bilhete\_id").

```

149         reservas.recalcular(c,Dados.id(i,"reserva_id"));auditoria.registrar(c,s.usuarioId(),"EMBARCAR"
,"embarque",id);return id;

```

Chama ReservaJdbc.recalcular: Ordem das condições importa: todos expirados, todos cancelados/expirados, alguns cancelados/expirados, todos usados/ausentes, algum pendente, caso contrário confirmada.

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "EMBARCAR", "embarque", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```

150     });
151 }
152

```

## evento - L153 a L156

Atualiza situação atual da bagagem e insere outro registro em evento\_bagagem. Estado atual e histórico têm funções diferentes.

Entradas declaradas: (Connection c,Sessao s,long bagagem,long aeroporto,String estado,String nota).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AtendimentoServico.cancelarCheckin (L60); AtendimentoServico.despachar (L88); AtendimentoServico.rastrear (L111).

Chamadas observadas no corpo: Sql.executar, s.usuarioId, relogio.instant.

```

153     private void evento(Connection c,Sessao s,long bagagem,long aeroporto,String estado,String nota)
throws SQLException {

```

Início da declaração de evento. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

154         Sql.executar(c,"UPDATE bagagem SET situacao=? WHERE id=?",estado,bagagem);

```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: bagagem. Parâmetros ligados em ordem: estado, bagagem.

```

155         Sql.executar(c,"INSERT INTO
evento_bagagem(bagagem_id,aeroporto_id,usuario_id,situacao,ocorrido_em,observacao) VALUES
(?, ?, ?, ?, ?, ?)",bagagem,aeroporto,s.usuarioId(),estado,relogio.instant(),nota);

```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: evento\_bagagem.

Parâmetros ligados em ordem: bagagem, aeroporto, s.usuarioId(), estado, relogio.instant(), nota.

```

156     }

```

157

## janela - L158 a L160

Usa agora do Clock e limites efetivos ajustados pelo atraso; início é inclusivo e fim é exclusivo.

Entradas declaradas: (Map<String, Object> voo, String inicio, String fim).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: AtendimentoServico.checkin (L32); AtendimentoServico.despachar (L88); AtendimentoServico.embarcar (L132).

Chamadas observadas no corpo: relógio.instant, Dados.exigir, agora.isBefore, ReservaJdbc.janela.

```
158     private void janela(Map<String, Object> voo, String inicio, String fim) {  
        Início da declaração de janela. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é  
        invocado.
```

```
159         Instant agora=relógio.instant();Dados.exigir(!agora.isBefore(ReservaJdbc.janela(voo, inicio)) &&  
        agora.isBefore(ReservaJdbc.janela(voo, fim)), "Fora da janela de atendimento.");  
        Declara variável local agora do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
        Regra obrigatória: !agora.isBefore(ReservaJdbc.janela(voo, inicio)) && agora.isBefore(ReservaJdbc.janela(voo, fim)) deve resultar  
        em true. Se false, Dados.exigir lança RegraNegocioException com "Fora da janela de atendimento.". Essa condição específica  
        não é substituída por outra validação parecida.
```

```
160     }  
161 }
```

## AutenticacaoServico

src/main/java/br/edu/aeroporto/servico/AutenticacaoServico.java

Valida login/senha, obtém a Sessao e registra acesso; limpa o char[] da senha no finally.

46 linhas | 3 métodos/construtores declarados | SHA-256: 6bbda729c6efc2a5...

Dependências guardadas em campos: AuditoriaJdbc, BancoDados, UsuarioRepositorio. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.Sessao;
4 import br.edu.aeroporto.dominio.Validacao;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Funções estáticas de validação de texto, nascimento, login e CPF. Retornam valores normalizados ou interrompem a operação com RegraNegocioException.

5 import br.edu.aeroporto.excecao.RegraNegocioException;
6 import br.edu.aeroporto.infraestrutura.jdbc.AuditoriaJdbc;
7 import br.edu.aeroporto.infraestrutura.jdbc.BancoDados;
8 import br.edu.aeroporto.repositorio.UsuarioRepositorio;
9 import br.edu.aeroporto.seguranca.Senhas;
10 import java.util.Arrays;
11
12 public final class AutenticacaoServico {
    Declara AutenticacaoServico como tipo com campos e métodos. final impede herança.

13     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

14     private final UsuarioRepositorio usuarios;
    Campo(s) usuarios: private restringe acesso à classe. Tipo UsuarioRepositorio: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

15     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

16

```

### AutenticacaoServico - L17 a L21

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco, UsuarioRepositorio usuarios, AuditoriaJdbc auditoria).

Possíveis chamadores identificados: IntegracaoTeste.administracaoElImportacao (L431); IntegracaoTeste.cadastros (L62); IntegracaoTeste.inicializar (L50); Main.main (L55).

```

17     public AutenticacaoServico(BancoDados banco, UsuarioRepositorio usuarios, AuditoriaJdbc auditoria) {
    Início da declaração de AutenticacaoServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

18         this.banco = banco;
    Guarda no campo banco da instância atual o valor de banco. O this diferencia o campo de parâmetros com o mesmo nome.

19         this.usuarios = usuarios;
    Guarda no campo usuarios da instância atual o valor de usuarios. O this diferencia o campo de parâmetros com o mesmo nome.

20         this.auditoria = auditoria;
    Guarda no campo auditoria da instância atual o valor de auditoria. O this diferencia o campo de parâmetros com o mesmo nome.

21     }
22

```

### existeUsuario - L23 a L25

Passa a referência usuarios::existeUsuario para banco.consultar e devolve booleano.

Entradas declaradas: ().

Saída declarada: boolean - condição verdadeira ou falsa.

Possíveis chamadores identificados: IntegracaoTeste.inicializar (L50); Main.main (L55).

Chamadas observadas no corpo: banco.consultar, usuarios::existeUsuario.

```

23     public boolean existeUsuario() {
        Início da declaração de existeUsuario. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

24         return banco.consultar(usuarios::existeUsuario);
        Retorno: entrega banco.consultar(usuarios::existeUsuario) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

        Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

25     }
26 
```

## entrar - L27 a L45

Normaliza login e consulta Optional<Credenciais>. Ausência ou hash incorreto dão a mesma mensagem. Rejeita ausência de perfis, revalida acesso e registra ENTRAR; devolve Sessao e limpa senha mesmo em erro.

Entradas declaradas: (String login, char[] senha).

Saída declarada: Sessao - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); IntegracaoTeste.inicializar (L50); Main.entrar (L143).

Chamadas observadas no corpo: Validacao.login, banco.consultar, usuarios.buscarAtivo, credenciais.isEmpty, Senhas.conferir, credenciais.get().senhaHash, credenciais.get, new RegraNegocioException, credenciais.get().sessao, sessao.perfis().isEmpty, sessao.perfis, banco.transacao, new br.edu.aeroporto.infraestrutura.jdbc.AutorizacaoJdbc().exigirConsulta, new br.edu.aeroporto.infraestrutura.jdbc.AutorizacaoJdbc, auditoria.registrar, sessao.usuariold, Arrays.fill.

```

27     public Sessao entrar(String login, char[] senha) {
        Início da declaração de entrar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

28         try {
            Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

29             String normalizado = Validacao.login(login);
            Declara variável local normalizado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

30             var credenciais = banco.consultar(conexao -> usuarios.buscarAtivo(conexao, normalizado));
            Declara variável local credenciais com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

            Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

31             if (credenciais.isEmpty() || !Senhas.conferir(senha, credenciais.get().senhaHash())) {
                Decisão específica: (credenciais.isEmpty() || !Senhas.conferir(senha, credenciais.get().senhaHash())). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

32                 throw new RegraNegocioException("Login ou senha inválidos.");
                Lança new RegraNegocioException("Login ou senha inválidos."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

33             }
34             Sessao sessao = credenciais.get().sessao();
            Declara variável local sessao do tipo Sessao. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

35             if (sessao.perfis().isEmpty()) throw new RegraNegocioException("O usuário não possui perfil de
                acesso.");
            Decisão específica: (sessao.perfis().isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

            Lança new RegraNegocioException("O usuário não possui perfil de acesso."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

            new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

```
36         banco.transacao(conexao -> {  
    Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.  
37             new br.edu.aeroporto.infraestrutura.jdbc.AutorizacaoJdbc().exigirConsulta(conexao, sessao);  
    Invoca new br.edu.aeroporto.infraestrutura.jdbc.AutorizacaoJdbc().exigirConsulta com os argumentos mostrados. O  
    receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.  
  
    new constrói br.edu.aeroporto.infraestrutura.jdbc.AutorizacaoJdbc: aloca a instância e executa seu construtor com os argumentos  
    mostrados; não insere no banco por si só.  
38         auditoria.registrar(conexao, sessao.usuarioId(), "ENTRAR", "usuario_sistema",  
    sessao.usuarioId());  
    Registra o evento de auditoria na conexão fornecida: registrar(conexao, sessao.usuarioId(), "ENTRAR", "usuario_sistema",  
    sessao.usuarioId()).  
39         return null;  
    return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o  
    contrato genérico; confira se o retorno pertence à lambda ou ao método externo.  
40     });  
41     return sessao;  
    Retorno: entrega sessao ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse  
    caminho.  
42     } finally {  
    Bloco de finalização, executado ao sair do try/catch; neste corpo, observe quais recursos ou dados são limpos.  
43         Arrays.fill(senha, '\0');  
    Chama Arrays.fill: Substitui todas as posições do vetor pelo valor fornecido; usado para limpar char[] de senha.  
44     }  
45 }  
46 }
```

## CadastroServico

src/main/java/br/edu/aeroporto/servico/CadastroServico.java

Valida o cadastro conforme seu tipo e protege operações/históricos durante edição ou inativação.

135 linhas | 6 métodos/construtores declarados | SHA-256: 932fea6819fad4f3...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, CadastroJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1  package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3  import br.edu.aeroporto.dominio.*;
4  import br.edu.aeroporto.infraestrutura.jdbc.*;
5  import java.sql.Connection;
6  import java.sql.SQLException;
7  import java.time.Clock;
8  import java.time.LocalDate;
9  import java.time.ZoneId;
10 import java.util.LinkedHashMap;
11 import java.util.List;
12 import java.util.Map;
13
14 public final class CadastroServico {
    Declara CadastroServico como tipo com campos e métodos. final impede herança.

15     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

16     private final CadastroJdbc repositorio;
    Campo(s) repositorio: private restringe acesso à classe. Tipo CadastroJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

17     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

18     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

19     private final Clock relógio;
    Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

20

```

### CadastroServico - L21 a L23

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco, CadastroJdbc repositorio, AutorizacaoJdbc autorizacao, AuditoriaJdbc auditoria, Clock relógio).

Possíveis chamadores identificados: Main.main (L55).

```

21     public CadastroServico(BancoDados banco, CadastroJdbc repositorio, AutorizacaoJdbc autorizacao,
    AuditoriaJdbc auditoria, Clock relógio) {
    Início da declaração de CadastroServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

22         this.banco=banco; this.repositorio=repositorio; this.autorizacao=autorizacao;
        this.auditoria=auditoria; this.relógio=relógio;
        Guarda no campo banco da instância atual o valor de banco. O this diferencia o campo de parâmetros com o mesmo nome.

        Guarda no campo repositorio da instância atual o valor de repositorio. O this diferencia o campo de parâmetros com o mesmo nome.

        Guarda no campo autorizacao da instância atual o valor de autorizacao. O this diferencia o campo de parâmetros com o mesmo nome.

```

Guarda no campo auditoria da instância atual o valor de auditoria. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo relógio da instância atual o valor de relógio. O this diferencia o campo de parâmetros com o mesmo nome.

```
23     }
24
```

## listar - L25 a L28

Valida página, aplica permissão segundo o tipo e devolve registros Map do repositório.

Entradas declaradas: (Sessao sessao, TipoCadastro tipo, int pagina).

Saída declarada: List<Map<String, Object>> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuOperacional.cadastros (L50).

Chamadas observadas no corpo: Dados.exigir, banco.consultar, permitir, repositorio.listar.

```
25     public List<Map<String, Object>> listar(Sessao sessao, TipoCadastro tipo, int pagina) {
        Início da declaração de listar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

26         Dados.exigir(pagina>0 && pagina<=100000, "Pagina invalida.");
        Regra obrigatória: pagina>0 && pagina<=100000 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com
        "Pagina invalida.". Essa condição específica não é substituída por outra validação parecida.

27         return banco.consultar(c -> { permitir(c,sessao,tipo); return repositorio.listar(c,tipo,pagina);
    });
        Retorno: entrega banco.consultar(c -> { permitir(c,sessao,tipo); return repositorio.listar(c,tipo,pagina); }) ao chamador desta
        execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

        Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da
        consulta.

        Retorno: entrega repositorio.listar(c,tipo,pagina) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo
        executa por esse caminho.

28     }
29
```

## salvar - L30 a L46

Rejeita chaves desconhecidas, converte todos os campos pelo catálogo, serializa operação, obtém dados anteriores quando id existe, valida regras do tipo, salva e audita; retorna id.

Entradas declaradas: (Sessao sessao, TipoCadastro tipo, Long id, Map<String, Object> entrada).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElmportacao (L431); IntegracaoTeste.cad (L510); IntegracaoTeste.cadastros (L62); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.cadastros (L50).

Chamadas observadas no corpo: new LinkedHashMap<String,Object>, Dados.exigir, entrada.keySet().stream().allMatch, entrada.keySet().stream, entrada.keySet, tipo.campos().stream().anyMatch, tipo.campos().stream, tipo.campos, f.nome().equals, f.nome, entrada.get, campo.nome, valores.put, campo.converter, valor.toString, banco.transacao, permitir, Sql.unico, Map.of, repositorio.buscar, validar, repositorio.salvar, auditoria.registrar, sessao.usuariold, tipo.tabela.

```
30     public long salvar(Sessao sessao, TipoCadastro tipo, Long id, Map<String, Object> entrada) {
        Início da declaração de salvar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

31         var valores = new LinkedHashMap<String,Object>();
        Declara variável local valores com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        new constrói LinkedHashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
        si só.

32         Dados.exigir(entrada.keySet().stream().allMatch(k -> tipo.campos().stream().anyMatch(f ->
        f.nome().equals(k))), "Campo desconhecido.");
        Regra obrigatória: entrada.keySet().stream().allMatch(k -> tipo.campos().stream().anyMatch(f -> f.nome().equals(k))) deve resultar
        em true. Se false, Dados.exigir lança RegraNegocioException com "Campo desconhecido.". Essa condição específica não é
        substituída por outra validação parecida.

33         for (var campo:tipo.campos()) {
        Percorre cada elemento da coleção/vetor: for (var campo:tipo.campos()). return, break ou exceção podem encerrar esse caminho;
        continue passa à próxima volta.
```



```
34      Object valor = entrada.get(campo.nome());
```

Declara variável local valor do tipo Object. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
35      valores.put(campo.nome(), campo.converter(valor==null ? null : valor.toString()));
```

Chama valores.put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.

Escolha de valor: valor==null. true escolhe null; false escolhe valor.toString(). Só a alternativa escolhida é avaliada.

```
36      }
```

```
37      return banco.transacao(c -> {
```

Retorno: entrega banco.transacao(c -> { permitir(c,sessao,tipo); Sql.unico(c,"SELECT pg\_advisory\_xact\_lock(hashtext('aeroporto.operacao'))",r->true); Map<String,Object> anterior=id==null?Map.of():repositorio.buscar(c,tipo,id); validar... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.

```
38      permitir(c,sessao,tipo);
```

Chama CadastroServico.permitir: Pessoa/documento/contato exigem atendimento. Os demais tipos exigem administrador; é uma condição específica, não mero código repetido.

```
39      Sql.unico(c,"SELECT pg_advisory_xact_lock(hashtext('aeroporto.operacao'))",r->true);
```

Instrução idêntica à de PlanejamentoJdbc:L12. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
40      Map<String,Object> anterior=id==null?Map.of():repositorio.buscar(c,tipo,id);
```

Declara variável local anterior do tipo Map<String,Object>. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Escolha de valor: id==null. true escolhe Map.of(); false escolhe repositorio.buscar(c,tipo,id). Só a alternativa escolhida é avaliada.

```
41      validar(c,tipo,id,valores,anterior);
```

Chama CadastroServico.validar: Cada case possui regra própria: código de país/aeroporto, nascimento, vínculo de documento/contato, CPF BR, contratação, preservação de agenda/mapa/rota e proibição de inativar cadastros com operações ativas.

```
42      long salvo=repositorio.salvar(c,tipo,id,valores);
```

Declara variável local salvo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
43      auditoria.registrar(c,sessao.usuarioId(),id==null?"CADASTRAR":"EDITAR",tipo.tabela(),salvo);
```

Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), id==null?"CADASTRAR":"EDITAR", tipo.tabela(), salvo).

Escolha de valor: id==null. true escolhe "CADASTRAR"; false escolhe "EDITAR". Só a alternativa escolhida é avaliada.

```
44      return salvo;
```

Retorno: entrega salvo ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
45      });
```

```
46      }
```

```
47
```

## permitir - L48 a L51

Pessoa/documento/contato exigem atendimento. Os demais tipos exigem administrador; é uma condição específica, não mero código repetido.

Entradas declaradas: (Connection c,Sessao s,TipoCadastro t).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: CadastroServico.listar (L25); CadastroServico.salvar (L30).

Chamadas observadas no corpo: autorizacao.exigirAtendimento, autorizacao.exigirAdministrador.

```
48      private void permitir(Connection c,Sessao s,TipoCadastro t) throws SQLException {
```

Início da declaração de permitir. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
49      if (t==TipoCadastro.PESSOA || t==TipoCadastro.DOCUMENTO || t==TipoCadastro.CONTATO)
        autorizacao.exigirAtendimento(c,s);
```

Decisão específica: (t==TipoCadastro.PESSOA || t==TipoCadastro.DOCUMENTO || t==TipoCadastro.CONTATO). Se for true, executa o ramo seguinte; se for false, pula esse ramo e executa o else.

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

```
50      else autorizacao.exigirAdministrador(c,s);
```

Confere no banco o acesso vigente do usuário para esta operação: exigirAdministrador(c, s).

```
51     }
52
```

## validar - L53 a L130

Cada case possui regra própria: código de país/aeroporto, nascimento, vínculo de documento/contato, CPF BR, contratação, preservação de agenda/mapa/rota e proibição de inativar cadastros com operações ativas.

Entradas declaradas: (Connection c,TipoCadastro tipo,Long id,Map<String,Object> v,Map<String,Object> antes).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: CadastroServico.salvar (L30).

Chamadas observadas no corpo: v.get, v.put, Dados.exigir, Dados.texto(v,"codigo").matches, Dados.texto, ZoneId.of, Dados.texto(v,"codigo\_icao").matches, Dados.texto(v,"codigo\_iata").matches, Validacao.nascimento, LocalDate.now, Boolean.FALSE.equals, existe, Dados.id, "CPF".equals, Validacao.cpf, ((LocalDate)v.get("validade")).isBefore, "EMAIL".equals, valor.matches, Boolean.TRUE.equals, Sql.executar, v.get("carga\_id").equals, antes.get, v.get("aeroporto\_base\_id").equals, ((LocalDate)v.get("desligamento")).isAfter, "ATIVA".equals, v.get("modelo\_id").equals, v.get("companhia\_id").equals, Dados.texto(v,"coluna").matches, Dados.texto(v,"codigo").equals, v.get("origem\_id").equals, v.get("destino\_id").equals, v.get("envergadura\_m").equals, v.get("comprimento\_pista\_min\_m").equals.

```
53     private void validar(Connection c,TipoCadastro tipo,Long id,Map<String,Object> v,Map<String,Object>
antes) throws SQLException {
    Início da declaração de validar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.
```

```
54     switch(tipo) {
    Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.
```

```
55         case CIDADE -> { if(v.get("regiao")==null) v.put("regiao",""); }
    Ramo do switch: case CIDADE. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
    Decisão específica: (v.get("regiao")==null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.
```

```
56         case PAIS -> Dados.exigir(Dados.texto(v,"codigo").matches("[A-Z]{2}"),"Codigo de pais: duas
letras maiusculas.");
    Ramo do switch: case PAIS. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
    Regra obrigatória: Dados.texto(v,"codigo").matches("[A-Z]{2}") deve resultar em true. Se false, Dados.exigir lança
    RegraNegocioException com "Codigo de pais: duas letras maiusculas.". Essa condição específica não é substituída por outra
    validação parecida.
```

```
57         case AEROPORTO -> {
    Ramo do switch: case AEROPORTO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
58             ZoneId.of(Dados.texto(v,"fuso_horario"));
    Chama ZoneId.of: Fábrica estática: cria o valor do tipo receptor a partir dos argumentos.
```

```
59             Dados.exigir(Dados.texto(v,"codigo_icao").matches("[A-Z]{4}"),"ICAO invalido.");
    Regra obrigatória: Dados.texto(v,"codigo_icao").matches("[A-Z]{4}") deve resultar em true. Se false, Dados.exigir lança
    RegraNegocioException com "ICAO invalido.". Essa condição específica não é substituída por outra validação parecida.
```

```
60             if(v.get("codigo_iata")!=null)
Dados.exigir(Dados.texto(v,"codigo_iata").matches("[A-Z]{3}"),"IATA invalido.");
    Decisão específica: (v.get("codigo_iata")!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.
    Regra obrigatória: Dados.texto(v,"codigo_iata").matches("[A-Z]{3}") deve resultar em true. Se false, Dados.exigir lança
    RegraNegocioException com "IATA invalido.". Essa condição específica não é substituída por outra validação parecida.
```

```
61         }
62         case PESSOA -> {
    Ramo do switch: case PESSOA. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
63             Validacao.nascimento(((LocalDate)v.get("nascimento"),LocalDate.now(relogio)));
    Chama Validacao.nascimento: Exige data não nula e não futura em relação ao hoje recebido. A função não decide sozinha qual
    relógio usar.
```

```
64             if(id!=null && Boolean.FALSE.equals(v.get("ativo"))) {
    Decisão específica: (id!=null && Boolean.FALSE.equals(v.get("ativo"))). Se for true, executa o ramo seguinte; se for false, pula
    esse ramo.
```

```
65             Dados.exigir(!existe(c,"SELECT 1 FROM item_reserva WHERE passageiro_id=? AND situacao
IN ('PENDENTE','CONFIRMADO')",id),"Pessoa tem passagens ativas.");
    Regra obrigatória: !existe(c,"SELECT 1 FROM item_reserva WHERE passageiro_id=? AND situacao IN
('PENDENTE','CONFIRMADO')",id) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Pessoa tem
passagens ativas.". Essa condição específica não é substituída por outra validação parecida.
```

```

66      Dados.exigir(!existe(c,"SELECT 1 FROM usuario_sistema u JOIN usuario_perfil up ON
up.usuario_id=u.id JOIN perfil_acesso p ON p.id=up.perfil_id WHERE u.funcionario_id=? AND u.ativo AND
p.nome='ADMINISTRADOR'",id),"Bloqueie o acesso administrativo antes de inativar a pessoa.");
Regra obrigatória: !existe(c,"SELECT 1 FROM usuario_sistema u JOIN usuario_perfil up ON up.usuario_id=u.id JOIN
perfil_acesso p ON p.id=up.perfil_id WHERE u.funcionario_id=? AND u.ativo AND p.nome='ADMINISTRADOR'",id) deve resultar
em true. Se false, Dados.exigir lança RegraNegocioException com "Bloqueie o acesso administrativo antes de inativar a pessoa.".
Essa condição específica não é substituída por outra validação parecida.

67      }
68      }
69      case DOCUMENTO -> {
Ramo do switch: case DOCUMENTO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

70      Dados.exigir(id==null || Dados.id(antes,"pessoa_id")==Dados.id(v,"pessoa_id"),"Nao
transfira documento entre pessoas.");
Regra obrigatória: id==null || Dados.id(antes,"pessoa_id")==Dados.id(v,"pessoa_id") deve resultar em true. Se false, Dados.exigir
lança RegraNegocioException com "Nao transfira documento entre pessoas.". Essa condição específica não é substituída por
outra validação parecida.

71      if("CPF".equals(v.get("tipo"))) {
Decisão específica: ("CPF".equals(v.get("tipo"))). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

72      v.put("numero",Validacao.cpf(Dados.texto(v,"numero")));
Chama v.put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.

73      Dados.exigir(existe(c,"SELECT 1 FROM pais WHERE id=? AND
codigo='BR'",Dados.id(v,"pais_emissor_id"),"CPF requer emissor BR.");
Regra obrigatória: existe(c,"SELECT 1 FROM pais WHERE id=? AND codigo='BR'",Dados.id(v,"pais_emissor_id")) deve resultar
em true. Se false, Dados.exigir lança RegraNegocioException com "CPF requer emissor BR.". Essa condição específica não é
substituída por outra validação parecida.

74      }
75      if(v.get("validade")!=null)
Dados.exigir(!((LocalDate)v.get("validade")).isBefore(LocalDate.now(relogio)),"Documento ja vencido.");
Decisão específica: (v.get("validade")!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: !((LocalDate)v.get("validade")).isBefore(LocalDate.now(relogio)) deve resultar em true. Se false, Dados.exigir
lança RegraNegocioException com "Documento ja vencido.". Essa condição específica não é substituída por outra validação
parecida.

76      }
77      case CONTATO -> {
Ramo do switch: case CONTATO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

78      Dados.exigir(id==null || Dados.id(antes,"pessoa_id")==Dados.id(v,"pessoa_id"),"Nao
transfira contato.");
Regra obrigatória: id==null || Dados.id(antes,"pessoa_id")==Dados.id(v,"pessoa_id") deve resultar em true. Se false, Dados.exigir
lança RegraNegocioException com "Nao transfira contato.". Essa condição específica não é substituída por outra validação
parecida.

79      String valor=Dados.texto(v,"valor");
Declara variável local valor do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

80      Dados.exigir("EMAIL".equals(v.get("tipo"))?valor.matches("[^\\s@]+@[^\\s@]+\\.([\\s@]+)" :
valor.matches("\\+[0-9]{8,15}"),"Contato invalido.");
Regra obrigatória: "EMAIL".equals(v.get("tipo"))?valor.matches("[^\\s@]+@[^\\s@]+\\.([\\s@]+)" : valor.matches("\\+[0-9]{8,15}")
deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Contato invalido.". Essa condição específica
não é substituída por outra validação parecida.

Escolha de valor: "EMAIL".equals(v.get("tipo")). true escolhe valor.matches("[^\\s@]+@[^\\s@]+\\.([\\s@]+)"; false escolhe
valor.matches("\\+[0-9]{8,15}"). Só a alternativa escolhida é avaliada.

81      if(Boolean.TRUE.equals(v.get("principal"))) Sql.executar(c,"UPDATE contato_pessoa SET
principal=false WHERE pessoa_id=? AND tipo=?",Dados.id(v,"pessoa_id"),v.get("tipo"));
Decisão específica: (Boolean.TRUE.equals(v.get("principal"))). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: contato_pessoa.
Parâmetros ligados em ordem: Dados.id(v,"pessoa_id"), v.get("tipo").

82      }
83      case FUNCIONARIO -> {
Ramo do switch: case FUNCIONARIO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

```

84      Dados.exigir(id==null || id==Dados.id(v,"pessoa_id"),"Nao altere a identidade do
funcionario.");
    Regra obrigatória: id==null || id==Dados.id(v,"pessoa_id") deve resultar em true. Se false, Dados.exigir lança
    RegraNegocioException com "Nao altere a identidade do funcionario.". Essa condição específica não é substituída por outra
    validação parecida.

85      Dados.exigir(existe(c,"SELECT 1 FROM pessoa WHERE id=? AND
ativo",Dados.id(v,"pessoa_id")), "Pessoa inativa.");
    Regra obrigatória: existe(c,"SELECT 1 FROM pessoa WHERE id=? AND ativo",Dados.id(v,"pessoa_id")) deve resultar em true.
    Se false, Dados.exigir lança RegraNegocioException com "Pessoa inativa.". Essa condição específica não é substituída por outra
    validação parecida.

86      Dados.exigir(existe(c,"SELECT 1 FROM cargo WHERE id=? AND
ativo",Dados.id(v,"cargo_id")), "Cargo inativo.");
    Regra obrigatória: existe(c,"SELECT 1 FROM cargo WHERE id=? AND ativo",Dados.id(v,"cargo_id")) deve resultar em true. Se
    false, Dados.exigir lança RegraNegocioException com "Cargo inativo.". Essa condição específica não é substituída por outra
    validação parecida.

87      if(id!=null && (!v.get("cargo_id").equals(antes.get("cargo_id"))) ||
!v.get("aeroporto_base_id").equals(antes.get("aeroporto_base_id"))) {
    Decisão específica: (id!=null && (!v.get("cargo_id").equals(antes.get("cargo_id"))) ||
!v.get("aeroporto_base_id").equals(antes.get("aeroporto_base_id")))). Se for true, executa o ramo seguinte; se for false, pula esse
    ramo.

88      Dados.exigir(!existe(c,"SELECT 1 FROM escala_funcionario WHERE funcionario_id=? AND
ativa AND fim>CURRENT_TIMESTAMP",id),"Libere as escalas futuras antes de mudar cargo ou base.");
    Regra obrigatória: !existe(c,"SELECT 1 FROM escala_funcionario WHERE funcionario_id=? AND ativa AND
    fim>CURRENT_TIMESTAMP",id) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Libere as
    escalas futuras antes de mudar cargo ou base.". Essa condição específica não é substituída por outra validação parecida.

89      }
90      if(v.get("desligamento")!=null &&
!((LocalDate)v.get("desligamento")).isAfter(LocalDate.now(relogio))) {
    Decisão específica: (v.get("desligamento")!=null && !((LocalDate)v.get("desligamento")).isAfter(LocalDate.now(relogio))). Se for
    true, executa o ramo seguinte; se for false, pula esse ramo.

91      Dados.exigir(!existe(c,"SELECT 1 FROM escala_funcionario WHERE funcionario_id=? AND
ativa AND fim>CURRENT_TIMESTAMP",Dados.id(v,"pessoa_id")), "Funcionario tem escalas futuras.");
    Regra obrigatória: !existe(c,"SELECT 1 FROM escala_funcionario WHERE funcionario_id=? AND ativa AND
    fim>CURRENT_TIMESTAMP",Dados.id(v,"pessoa_id")) deve resultar em true. Se false, Dados.exigir lança
    RegraNegocioException com "Funcionario tem escalas futuras.". Essa condição específica não é substituída por outra validação
    parecida.

92      Dados.exigir(!existe(c,"SELECT 1 FROM usuario_sistema u JOIN usuario_perfil up ON
up.usuario_id=u.id JOIN perfil_acesso p ON p.id=up.perfil_id WHERE u.funcionario_id=? AND u.ativo AND
p.nome='ADMINISTRADOR'",Dados.id(v,"pessoa_id")), "Bloqueie o acesso administrativo antes do
desligamento.");
    Regra obrigatória: !existe(c,"SELECT 1 FROM usuario_sistema u JOIN usuario_perfil up ON up.usuario_id=u.id JOIN
    perfil_acesso p ON p.id=up.perfil_id WHERE u.funcionario_id=? AND u.ativo AND
    p.nome='ADMINISTRADOR'",Dados.id(v,"pessoa_id")) deve resultar em true. Se false, Dados.exigir lança
    RegraNegocioException com "Bloqueie o acesso administrativo antes do desligamento.". Essa condição específica não é
    substituída por outra validação parecida.

93      }
94      }
95      case AERONAVE -> {
    Ramo do switch: case AERONAVE. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

96      if(id!=null && (!"ATIVA".equals(v.get("situacao"))) ||
!v.get("modelo_id").equals(antes.get("modelo_id"))) ||
!v.get("companhia_id").equals(antes.get("companhia_id"))) {
    Decisão específica: (id!=null && (!"ATIVA".equals(v.get("situacao"))) || !v.get("modelo_id").equals(antes.get("modelo_id"))) ||
!v.get("companhia_id").equals(antes.get("companhia_id")))). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

97      Dados.exigir(!existe(c,"SELECT 1 FROM agenda_aeronave WHERE aeronave_id=? AND ativa
AND fim>CURRENT_TIMESTAMP",id),"Libere a agenda da aeronave antes da alteracao.");
    Regra obrigatória: !existe(c,"SELECT 1 FROM agenda_aeronave WHERE aeronave_id=? AND ativa AND
    fim>CURRENT_TIMESTAMP",id) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Libere a
    agenda da aeronave antes da alteracao.". Essa condição específica não é substituída por outra validação parecida.

98      }
99      }
100     case ASSENTO -> {
    Ramo do switch: case ASSENTO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

```
101      Dados.exigir(Dados.texto(v,"coluna").matches("[A-Z]{1,2}") &&
      Dados.texto(v,"codigo").equals(v.get("fila")+Dados.texto(v,"coluna")), "Codigo de assento deve combinar
      fila e coluna.");
```

Regra obrigatória: `Dados.texto(v,"coluna").matches("[A-Z]{1,2}") && Dados.texto(v,"codigo").equals(v.get("fila")+Dados.texto(v,"coluna"))` deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Codigo de assento deve combinar fila e coluna.". Essa condição específica não é substituída por outra validação parecida.

```
102      if(id!=null) Dados.exigir(!existe(c,"SELECT 1 FROM inventario_assento_voo WHERE
      assento_id=?",id),"Assento ja copiado para voo; preserve o mapa historico.");
```

Decisão específica: `(id!=null)`. Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: `!existe(c,"SELECT 1 FROM inventario_assento_voo WHERE assento_id=?",id)` deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Assento ja copiado para voo; preserve o mapa historico.". Essa condição específica não é substituída por outra validação parecida.

```
103      Dados.exigir(!existe(c,"SELECT 1 FROM voo WHERE aeronave_id=? AND situacao NOT IN
      ('CONCLUIDO','CANCELADO')"),Dados.id(v,"aeronave_id")), "Configure assentos antes de programar voos.");
```

Regra obrigatória: `!existe(c,"SELECT 1 FROM voo WHERE aeronave_id=? AND situacao NOT IN ('CONCLUIDO','CANCELADO')",Dados.id(v,"aeronave_id"))` deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Configure assentos antes de programar voos.". Essa condição específica não é substituída por outra validação parecida.

```
104      }
105      case ROTA -> {
```

Ramo do switch: case ROTA. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
106      if(id!=null && (!v.get("origem_id").equals(antes.get("origem_id"))) ||
      !v.get("destino_id").equals(antes.get("destino_id")))) Dados.exigir(!existe(c,"SELECT 1 FROM voo WHERE
      rota_id=?",id),"Rota ja utilizada; cadastre outro trecho para preservar historico.");
```

Decisão específica: `(id!=null && (!v.get("origem_id").equals(antes.get("origem_id"))) || !v.get("destino_id").equals(antes.get("destino_id")))`. Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: `!existe(c,"SELECT 1 FROM voo WHERE rota_id=?",id)` deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Rota ja utilizada; cadastre outro trecho para preservar historico.". Essa condição específica não é substituída por outra validação parecida.

```
107      if(id!=null && Boolean.FALSE.equals(v.get("ativa"))) Dados.exigir(!existe(c,"SELECT 1 FROM
      voo WHERE rota_id=? AND situacao NOT IN ('CONCLUIDO','CANCELADO')",id),"Rota tem voos ativos.");
```

Decisão específica: `(id!=null && Boolean.FALSE.equals(v.get("ativa")))`. Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: `!existe(c,"SELECT 1 FROM voo WHERE rota_id=? AND situacao NOT IN ('CONCLUIDO','CANCELADO')",id)` deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Rota tem voos ativos.". Essa condição específica não é substituída por outra validação parecida.

```
108      }
109      case RECURSO -> {
```

Ramo do switch: case RECURSO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
110      if(id!=null) Dados.exigir(!existe(c,"SELECT 1 FROM alocao_recurso WHERE recurso_id=? AND
      ativa AND fim>CURRENT_TIMESTAMP",id),"Recurso tem alocoes futuras; use interdicao na agenda.");
```

Decisão específica: `(id!=null)`. Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: `!existe(c,"SELECT 1 FROM alocao_recurso WHERE recurso_id=? AND ativa AND fim>CURRENT_TIMESTAMP",id)` deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Recurso tem alocoes futuras; use interdicao na agenda.". Essa condição específica não é substituída por outra validação parecida.

```
111      }
112      case MODELO -> {
```

Ramo do switch: case MODELO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
113      if(id!=null && (!v.get("envergadura_m").equals(antes.get("envergadura_m"))) ||
      !v.get("comprimento_pista_min_m").equals(antes.get("comprimento_pista_min_m")))) {
```

Decisão específica: `(id!=null && (!v.get("envergadura_m").equals(antes.get("envergadura_m"))) || !v.get("comprimento_pista_min_m").equals(antes.get("comprimento_pista_min_m")))`. Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
114      Dados.exigir(!existe(c,"SELECT 1 FROM agenda_aeronave ag JOIN aeronave a ON
      a.id=ag.aeronave_id WHERE a.modelo_id=? AND ag.ativa AND ag.fim>CURRENT_TIMESTAMP",id),"Modelo tem
      aeronaves em agenda ativa.");
```

Regra obrigatória: `!existe(c,"SELECT 1 FROM agenda_aeronave ag JOIN aeronave a ON a.id=ag.aeronave_id WHERE a.modelo_id=? AND ag.ativa AND ag.fim>CURRENT_TIMESTAMP",id)` deve resultar em true. Se false, `Dados.exigir` lança `RegraNegocioException` com "Modelo tem aeronaves em agenda ativa.". Essa condição específica não é substituída por outra validação parecida.

```
115      }
```

```

116         }
117         default -> { }

```

Ramo do switch: default. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

118     }
119     if(id!=null && Boolean.FALSE.equals(v.get("ativo"))) {

```

Decisão específica: (id!=null && Boolean.FALSE.equals(v.get("ativo"))). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```

120         String consulta=switch(tipo) {

```

Declara variável local consulta do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```

121             case COMPANHIA -> "SELECT 1 FROM voo WHERE companhia_id=? AND situacao NOT IN
('CONCLUIDO','CANCELADO')";

```

Ramo do switch: case COMPANHIA. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

122             case AEROPORTO -> "SELECT 1 FROM voo v JOIN rota r ON r.id=v.rota_id WHERE ? IN
(r.origem_id,r.destino_id) AND v.situacao NOT IN ('CONCLUIDO','CANCELADO')";

```

Ramo do switch: case AEROPORTO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

123             case TERMINAL -> "SELECT 1 FROM alocao_recurso a JOIN recurso_aeroportuario r ON
r.id=a.recurso_id WHERE r.terminal_id=? AND a.ativa AND a.fim>CURRENT_TIMESTAMP";

```

Ramo do switch: case TERMINAL. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

124             case CARGO -> "SELECT 1 FROM funcionario WHERE cargo_id=? AND (desligamento IS NULL OR
desligamento>CURRENT_DATE)";

```

Ramo do switch: case CARGO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

125             case MODELO -> "SELECT 1 FROM aeronave WHERE modelo_id=? AND situacao<>'INATIVA'";

```

Ramo do switch: case MODELO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

126         default -> null;

```

Ramo do switch: default. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

127     };
128     if(consulta!=null) Dados.exigir(!existe(c,consulta,id),"Cadastro tem operacoes ativas.");

```

Decisão específica: (consulta!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: !existe(c,consulta,id) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Cadastro tem operacoes ativas.". Essa condição específica não é substituída por outra validação parecida.

```

129     }
130 }
131

```

## existe - L132 a L134

Retorna true quando a consulta com id produzir ao menos uma linha. Usa mapeador r->true porque interessa existência, não o conteúdo.

Entradas declaradas: (Connection c,String sql,long id).

Saída declarada: boolean - condição verdadeira ou falsa.

Possíveis chamadores identificados: CadastroServico.validar (L53).

Chamadas observadas no corpo: Sql.listar(c,sql,r->true,id).isEmpty, Sql.listar.

```

132     private boolean existe(Connection c,String sql,long id) throws SQLException {

```

Início da declaração de existe. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

133         return !Sql.listar(c,sql,r->true,id).isEmpty();

```

Retorno: entrega !Sql.listar(c,sql,r->true,id).isEmpty() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>.

```

134     }
135 }

```



## ConsultaServico

src/main/java/br/edu/aeroporto/servico/ConsultaServico.java

Valida filtros, limites e permissões das consultas básicas e converte um dia/fuso em intervalo de instantes.

52 linhas | 4 métodos/construtores declarados | SHA-256: 35f0a56cb8374143...

Dependências guardadas em campos: AutorizacaoJdbc, BancoDados, ConsultasRepositorio. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.Sessao;
4 import br.edu.aeroporto.dominio.Validacao;
5 import br.edu.aeroporto.dto.AssentoResumo;
6 import br.edu.aeroporto.dto.ReservaResumo;
7 import br.edu.aeroporto.dto.VooResumo;
8 import br.edu.aeroporto.excecao.RegraNegocioException;
9 import br.edu.aeroporto.infraestrutura.jdbc.AutorizacaoJdbc;
10 import br.edu.aeroporto.infraestrutura.jdbc.BancoDados;
11 import br.edu.aeroporto.repositorio.ConsultasRepositorio;
12 import java.time.LocalDate;
13 import java.time.ZoneId;
14 import java.util.List;
15 import java.util.Locale;
16
17 public final class ConsultaServico {
    Declara ConsultaServico como tipo com campos e métodos. final impede herança.

18     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

19     private final ConsultasRepositorio consultas;
    Campo(s) consultas: private restringe acesso à classe. Tipo ConsultasRepositorio: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

20     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

21

```

### ConsultaServico - L22 a L26

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco, ConsultasRepositorio consultas, AutorizacaoJdbc autorizacao).

Possíveis chamadores identificados: Main.main (L55).

```

22     public ConsultaServico(BancoDados banco, ConsultasRepositorio consultas, AutorizacaoJdbc autorizacao)
    {
        Início da declaração de ConsultaServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

23         this.banco = banco;
        Instrução idêntica à de AutenticacaoServico:L18. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

24         this.consultas = consultas;
        Guarda no campo consultas da instância atual o valor de consultas. O this diferencia o campo de parâmetros com o mesmo nome.

25         this.autorizacao = autorizacao;
        Guarda no campo autorizacao da instância atual o valor de autorizacao. O this diferencia o campo de parâmetros com o mesmo nome.

26     }
27

```

## voos - L28 a L35

Recusa data/fuso ausentes e página fora de 1 a 100.000. Verifica consulta e converte o dia local em intervalo de instantes do início daquele dia ao início do próximo; o repositório retorna a página de VooResumo.

Entradas declaradas: (Sessao sessao, LocalDate dia, ZoneId fuso, int pagina).

Saída declarada: List<VooResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuPrincipal.consultarVoos (L121).

Chamadas observadas no corpo: new RegraNegocioException, banco.consultar, autorizacao.exigirConsulta, consultas.voos, dia.atStartOfDay(fuso).toInstant, dia.atStartOfDay, dia.plusDays(1).atStartOfDay(fuso).toInstant, dia.plusDays(1).atStartOfDay, dia.plusDays.

```

28     public List<VooResumo> voos(Sessao sessao, LocalDate dia, ZoneId fuso, int pagina) {
        Início da declaração de voos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

29         if (dia == null || fuso == null || pagina < 1 || pagina > 100_000) throw new
            RegraNegocioException("Data, fuso ou página inválidos.");
        Decisão específica: (dia == null || fuso == null || pagina < 1 || pagina > 100_000). Se for true, executa o ramo seguinte; se for false,
        pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

        Lança new RegraNegocioException("Data, fuso ou página inválidos."). Interrompe o caminho normal; a exceção sobe até uma
        captura compatível e pode provocar rollback.

        new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
        banco por si só.

30         return banco.consultar(conexao -> {
            Retorno: entrega banco.consultar(conexao -> { autorizacao.exigirConsulta(conexao, sessao); return consultas.voos(conexao,
            dia.atStartOfDay(fuso).toInstant(), dia.plusDays(1).atStartOfDay(fuso).toInstant(), pagina); }) ao chamador desta execução;
            nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

            Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da
            consulta.

31             autorizacao.exigirConsulta(conexao, sessao);
            Confere no banco o acesso vigente do usuário para esta operação: exigirConsulta(conexao, sessao).

32             return consultas.voos(conexao, dia.atStartOfDay(fuso).toInstant(),
                Retorno: entrega consultas.voos(conexao, dia.atStartOfDay(fuso).toInstant(), dia.plusDays(1).atStartOfDay(fuso).toInstant(),
                pagina) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

33                 dia.plusDays(1).atStartOfDay(fuso).toInstant(), pagina);
            Nesta parte da expressão, chama dia.plusDays(1).atStartOfDay(fuso).toInstant, dia.plusDays(1).atStartOfDay, dia.plusDays. Os
            valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

34         });
35     }
36 
```

## assentos - L37 a L43

Exige id de voo positivo e acesso de consulta. Devolve List<AssentoResumo> obtida pelo repositório, dentro da conexão somente leitura.

Entradas declaradas: (Sessao sessao, long vooId).

Saída declarada: List<AssentoResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuPrincipal.mostrarAssentos (L138).

Chamadas observadas no corpo: new RegraNegocioException, banco.consultar, autorizacao.exigirConsulta, consultas.assentos.

```

37     public List<AssentoResumo> assentos(Sessao sessao, long vooId) {
        Início da declaração de assentos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

38         if (vooId <= 0) throw new RegraNegocioException("O id do voo deve ser positivo.");
        Decisão específica: (vooId <= 0). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a
        operação lançando a exceção.

        Lança new RegraNegocioException("O id do voo deve ser positivo."). Interrompe o caminho normal; a exceção sobe até uma
        captura compatível e pode provocar rollback.

        new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
        banco por si só.

```



```

39         return banco.consultar(conexao -> {
Instrução idêntica à de ConsultaServico:L30. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

40         autorizacao.exigirConsulta(conexao, sessao);
Instrução idêntica à de ConsultaServico:L31. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

41         return consultas.assentos(conexao, vooId);
Retorno: entrega consultas.assentos(conexao, vooId) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama ConsultasRepositorio.assentos: Contrato de consulta do inventário por id de voo, devolvendo DTOs de bloqueio/ocupação.

42     });
43 }
44

```

## reserva - L45 a L51

Valida localizador com limite 12 e converte a maiúsculas. Exige consulta e busca Optional<ReservaResumo>; ausência vira RegraNegocioException "Reserva não encontrada".

Entradas declaradas: (Sessao sessao, String localizador).

Saída declarada: ReservaResumo - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuPrincipal.mostrarReserva (L147).

Chamadas observadas no corpo: Validacao.texto(localizador, "Localizador", 12).toUpperCase, Validacao.texto, banco.consultar(conexao -> { autorizacao.exigirConsulta(conexao, sessao); return consultas.reserva(conexao, codigo); }).orElseThrow, banco.consultar, autorizacao.exigirConsulta, consultas.reserva, new RegraNegocioException.

```

45     public ReservaResumo reserva(Sessao sessao, String localizador) {
Início da declaração de reserva. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

46         String codigo = Validacao.texto(localizador, "Localizador", 12).toUpperCase(Locale.ROOT);
Declara variável local codigo do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

47         return banco.consultar(conexao -> {
Instrução idêntica à de ConsultaServico:L30. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

48         autorizacao.exigirConsulta(conexao, sessao);
Instrução idêntica à de ConsultaServico:L31. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

49         return consultas.reserva(conexao, codigo);
Retorno: entrega consultas.reserva(conexao, codigo) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama ConsultasRepositorio.reserva: Contrato de busca por localizador; ausência é Optional vazio, não um DTO de reserva fictícia.

50     }).orElseThrow(() -> new RegraNegocioException("Reserva não encontrada."));
new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

51 }
52 }

```

## DadosFicticiosServico

src/main/java/br/edu/aeroporto/servico/DadosFicticiosServico.java

Cria um cenário fictício completo e retorna ids úteis para demonstração. Repetir a mesma chave devolve os ids existentes.

80 linhas | 5 métodos/construtores declarados | SHA-256: 2f437a8cf3c652ad...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```
1 package br.edu.aeroporto.servico;
```

Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

```
2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.infraestrutura.jdbc.*;
5 import java.math.BigDecimal;
6 import java.time.*;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
7 import java.util.*;
```

```
8
9 public final class DadosFicticiosServico {
```

Declara DadosFicticiosServico como tipo com campos e métodos. final impede herança.

```
10     private final BancoDados banco;
```

Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
11     private final AutorizacaoJdbc autorizacao;
```

Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
12     private final AuditoriaJdbc auditoria;
```

Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
13     private final Clock relógio;
```

Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
14
```

### DadosFicticiosServico - L15 a L17

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock relógio).

Possíveis chamadores identificados: IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cenariosAdicionais (L203); Main.main (L55).

```
15     public DadosFicticiosServico(BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc
auditoria,Clock relógio) {
```

Início da declaração de DadosFicticiosServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
16         this.banco=banco;this.autorizacao=autorizacao;this.auditoria=auditoria;this.relógio=relógio;
```

Instrução idêntica à de ArquivosServico:L21. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
17     }
18
```

### criar - L19 a L21

A sobrecarga sem chave usa PRINCIPAL. A versão com chave valida A-Z/0-9 e até 8, serializa e procura evento CRIAR\_DEMO anterior. Se novo, cria localidades/frota/mapa/voo/tarifa/passageiro/equipe/portão e devolve Map com ids.

Entradas declaradas: (Sessao s).

Saída declarada: Map<String,Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L19); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.executar (L35).

Chamadas observadas no corpo: criar.

```

19      public Map<String,Object> criar(Sessao s) {
        Início da declaração de criar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

20          return criar(s,"PRINCIPAL");
        Retorno: entrega criar(s,"PRINCIPAL") ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa
        por esse caminho.

        Chama DadosFicticiosServico.criar: A sobrecarga sem chave usa PRINCIPAL. A versão com chave valida A-Z/0-9 e até 8,
        serializa e procura evento CRIAR_DEMO anterior. Se novo, cria localidades/frota/mapa/voo/tarifa/passageiro/equipe/portão e
        devolve Map com ids.

21      }
22  
```

## criar - L23 a L68

A sobrecarga sem chave usa PRINCIPAL. A versão com chave valida A-Z/0-9 e até 8, serializa e procura evento CRIAR\_DEMO anterior. Se novo, cria localidades/frota/mapa/voo/tarifa/passageiro/equipe/portão e devolve Map com ids.

Entradas declaradas: (Sessao s,String cenario).

Saída declarada: Map<String,Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L19); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.executar (L35).

Chamadas observadas no corpo: Validacao.texto(cenario,"Cenario",8).toUpperCase, Validacao.texto, Dados.exigir, chave.matches, chave.equals, banco.transacao, autorizacao.exigirAdministrador, Sql.unico, Sql::linha, existente.isPresent, existente.get, new PlanejamentoJdbc, planejamento.serializar, Sql.executar, Dados.id, Sql.registro, Sql.inserir, codigoLivre, new ArrayList<>, List.of, Collections.shuffle, new Random, Integer.parseInt, codigo.substring, relógio.instant().plus, relógio.instant, Duration.ofHours, partida.plus, auditoria.definirAutorDoVoo, s.usuarioId, planejamento.programar, new ProgramacaoVoo, partida.minus, Duration.ofMinutes, planejamento.tarifa, new Tarifa, new BigDecimal, new PassageiroJdbc().cadastrar(c,new CadastroPassageiro("Passageiro FICTICIO DEMO",LocalDate.of(1990,1,1),"","demo@example.invalid","").id, new PassageiroJdbc().cadastrar, new PassageiroJdbc, new CadastroPassageiro, LocalDate.of, LocalDate.now(relógio).plusYears, LocalDate.now, LocalDate.now(relógio).minusDays, chegada.plus, Map.of.

```

23      public Map<String,Object> criar(Sessao s,String cenario) {
        Início da declaração de criar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

24          String chave=Validacao.texto(cenario,"Cenario",8).toUpperCase(Locale.ROOT);
        Declara variável local chave do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

25          Dados.exigir(chave.matches("[A-Z0-9]{1,8}"),"Identificador de cenario invalido.");
        Regra obrigatória: chave.matches("[A-Z0-9]{1,8}") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException
        com "Identificador de cenario invalido.". Essa condição específica não é substituída por outra validação parecida.

26          boolean principal=chave.equals("PRINCIPAL");
        Declara variável local principal do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

27          return banco.transacao(c->{
        Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
        desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

28              autorizacao.exigirAdministrador(c,s);
        Confere no banco o acesso vigente do usuário para esta operação: exigirAdministrador(c, s).

29              Sql.unico(c,"SELECT pg_advisory_xact_lock(hashtext('aeroporto.demo'))",r->true);
        Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado.

30              var existente=Sql.unico(c,"""
        Declara variável local existente com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas
        mencionadas: evento_auditoria. Parâmetros ligados em ordem: chave.

31              SELECT registro_id AS voo,(detalhes->>'passageiro')::bigint AS
        passageiro,(detalhes->>'documento')::bigint AS documento,
        Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
        chamado, não pelo compilador Java.

```

```
32      (detalhes->>'tarifa')::bigint AS tarifa,(detalhes->>'alocacao')::bigint AS
alocacao
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```
33      FROM evento_auditoria WHERE acao='CRIAR_DEMO' AND
COALESCE(detalhes->>'cenario','PRINCIPAL')=? ORDER BY id LIMIT 1
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
34      """,Sql::linha,chave);
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
35      if(existente.isPresent()) return existente.get();
```

Decisão específica: (existente.isPresent()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega existente.get() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
36      var planejamento=new PlanejamentoJdbc();planejamento.serializar(c);
```

Declara variável local planejamento com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói PlanejamentoJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
37      Sql.executar(c,"INSERT INTO pais(codigo,nome) VALUES ('ZZ','Pais FICTICIO de demonstracao') ON
CONFLICT DO NOTHING");
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: pais. ON CONFLICT define o tratamento da colisão descrita no SQL.

```
38      long pais=Dados.id(Sql.registro(c,"SELECT id FROM pais WHERE codigo='ZZ'"),"id");
```

Declara variável local pais do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pais.

```
39      long cidade=Sql.inserir(c,"INSERT INTO cidade(pais_id,nome) VALUES (?,?) RETURNING
id",pais,"Cidade FICTICIA DEMO "+chave);
```

Declara variável local cidade do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: cidade. Parâmetros ligados em ordem: pais, "Cidade FICTICIA DEMO "+chave.

```
40      long origem=Sql.inserir(c,"INSERT INTO aeroporto(cidade_id,codigo_icao,nome,fuso_horario)
VALUES (?,?,'Origem FICTICIA DEMO','America/Sao_Paulo') RETURNING
id",cidade,principal?"XDEA":codigoLivre(c,true));
```

Declara variável local origem do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: aeroporto. Parâmetros ligados em ordem: cidade, principal?"XDEA":codigoLivre(c,true).

Escolha de valor: principal. true escolhe "XDEA"; false escolhe codigoLivre(c,true). Só a alternativa escolhida é avaliada.

```
41      long destino=Sql.inserir(c,"INSERT INTO aeroporto(cidade_id,codigo_icao,nome,fuso_horario)
VALUES (?,?,'Destino FICTICIO DEMO','America/Sao_Paulo') RETURNING
id",cidade,principal?"XDEB":codigoLivre(c,true));
```

Declara variável local destino do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: aeroporto. Parâmetros ligados em ordem: cidade, principal?"XDEB":codigoLivre(c,true).

Escolha de valor: principal. true escolhe "XDEB"; false escolhe codigoLivre(c,true). Só a alternativa escolhida é avaliada.

```
42      long terminal=Sql.inserir(c,"INSERT INTO terminal(aeroporto_id,codigo,nome) VALUES
(?, 'DEMO', 'Terminal FICTICIO') RETURNING id",origem);
```

Declara variável local terminal do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: terminal. Parâmetros ligados em ordem: origem.

```
43      long portao=Sql.inserir(c,"INSERT INTO
recurso_aeroportuario(aeroporto_id,terminal_id,tipo,codigo,envergadura_max_m) VALUES
(?,?, 'PORTAO', 'DEMO',40) RETURNING id",origem,terminal);
```

Declara variável local portao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: recurso\_aeroportuario. Parâmetros ligados em ordem: origem, terminal.

```
44      long companhia=Sql.inserir(c,"INSERT INTO companhia_aerea(pais_id,codigo_icao,nome) VALUES
(?,?, 'Companhia FICTICIA DEMO') RETURNING id",pais,principal?"FIC":codigoLivre(c,false));
```

Declara variável local companhia do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: companhia\_aerea. Parâmetros ligados em ordem: pais, principal?"FIC":codigoLivre(c,false).

Escolha de valor: principal. true escolhe "FIC"; false escolhe codigoLivre(c,false). Só a alternativa escolhida é avaliada.

```
45      long modelo=Sql.inserir(c,"INSERT INTO
modelo_aeronave(fabricante,nome,envergadura_m,comprimento_pista_min_m) VALUES ( 'FICTICIO',?,30,1500)
RETURNING id", "DEMO-"+chave);
```

Declara variável local modelo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: modelo\_aeronave. Parâmetros ligados em ordem: "DEMO-"+chave.

```
46      long aeronave=Sql.inserir(c,"INSERT INTO
aeronave(modelo_id,companhia_id,matricula,fabricacao_ano) VALUES (?,?,?,2025) RETURNING
id",modelo,companhia, "DEMO-"+chave);
```

Declara variável local aeronave do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: aeronave. Parâmetros ligados em ordem: modelo, companhia, "DEMO-"+chave.

```
47      var codigos=new ArrayList<>(List.of("1A","1B","2A","2B","3A"));Collections.shuffle(codigos,new
Random(42));
```

Declara variável local codigos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói ArrayList, Random: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
48      for(String codigo:codigos) Sql.executar(c,"INSERT INTO
assento_aeronave(aeronave_id,codigo,fila,coluna,classe) VALUES
(?,?,?, 'ECONOMICA')",aeronave,codigo,Integer.parseInt(codigo.substring(0,1)),codigo.substring(1));
```

Percorre cada elemento da coleção/vetor: for(String codigo:codigos). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: assento\_aeronave. Parâmetros ligados em ordem: aeronave, codigo, Integer.parseInt(codigo.substring(0,1)), codigo.substring(1).

```
49      long rota=Sql.inserir(c,"INSERT INTO rota(origem_id,destino_id,distancia_km) VALUES (?,?,1000)
RETURNING id",origem,destino);
```

Declara variável local rota do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: rota. Parâmetros ligados em ordem: origem, destino.

```
50      Instant
partida=relógio.instant().plus(Duration.ofHours(2)),chegada=partida.plus(Duration.ofHours(2));
```

Declara variável local partida, chegada do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
51      auditoria.definirAutorDoVoo(c,s.usuarioId(),"Demonstracao FICTICIA");
```

Registra o evento de auditoria na conexão fornecida: definirAutorDoVoo(c, s.usuarioId(), "Demonstracao FICTICIA").

```
52      long voo=planejamento.programar(c,new ProgramacaoVoo(companhia,rota,aeronave,"DEMO01",partida,
chegada,partida.minus(Duration.ofHours(3)),partida.minus(Duration.ofMinutes(45)),partida.minus(Duration.of
Minutes(40)),partida.minus(Duration.ofMinutes(10))));
```

Declara variável local voo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói ProgramacaoVoo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
53      long tarifa=planejamento.tarifa(c,voo,null,new Tarifa("DEMO","ECONOMICA",new
BigDecimal("100"),new BigDecimal("20"),new BigDecimal("23"),1,true,new BigDecimal("10")),true);
```

Declara variável local tarifa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
54      long pessoa=new PassageiroJdbc().cadastrar(c,new CadastroPassageiro("Passageiro FICTICIO
DEMO",LocalDate.of(1990,1,1),"","demo@example.invalid","").id());
```

Declara variável local pessoa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói PassageiroJdbc, CadastroPassageiro: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

55      Sql.executar(c,"UPDATE pessoa SET nacionalidade_id=? WHERE id=?",pais,pessoa);
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: pessoa. Parâmetros ligados em ordem: pais, pessoa.

56      long documento=Sql.inserir(c,"INSERT INTO
documento_pessoa(pessoa_id,pais_emissor_id,numero,validade) VALUES (?,?,'PASSAPORTE',?,?) RETURNING
id",pessoa,pais,"FICTICIO-DEMOPAX-"+chave,LocalDate.now(relogio).plusYears(2));
      Declara variável local documento do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: documento_pessoa. Parâmetros ligados em ordem: pessoa, pais, "FICTICIO-DEMOPAX-"+chave,
      LocalDate.now(relogio).plusYears(2).

57      long cargo=Sql.inserir(c,"INSERT INTO cargo(nome,area) VALUES (?, 'TRIPULACAO') RETURNING
id","Tripulacao FICTICIA DEMO "+chave);
      Declara variável local cargo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: cargo. Parâmetros ligados em ordem: "Tripulacao FICTICIA DEMO "+chave.

58      for(String funcao:new String[]{"COMANDANTE","COPILOTO","COMISSARIO"}) {
      Percorre cada elemento da coleção/vetor: for(String funcao:new String[]{"COMANDANTE","COPILOTO","COMISSARIO"}). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

      new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

59      long tripulante=Sql.inserir(c,"INSERT INTO pessoa(nome,nascimento) VALUES (?,?) RETURNING
id",funcao+" FICTICIO DEMO",LocalDate.of(1985,1,1));
      Declara variável local tripulante do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: pessoa. Parâmetros ligados em ordem: funcao+" FICTICIO DEMO", LocalDate.of(1985,1,1).

60      Sql.executar(c,"INSERT INTO
funcionario(pessoa_id,cargo_id,aeroporto_base_id,companhia_id,matricula,admissao) VALUES (?,?,,?,?,?)",tr
ipulante,cargo,origem,companhia,"DEMO-"+funcao+"-"+chave,LocalDate.now(relogio).minusDays(1));
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: funcionario. Parâmetros ligados em ordem: tripulante, cargo, origem, companhia, "DEMO-"+funcao+"-"+chave, LocalDate.now(relogio).minusDays(1).

61      Sql.executar(c,"INSERT INTO
habilitacao_tripulante(funcionario_id,modelo_id,funcao,numero_licenca,validade) VALUES
(?,?,?,?,?)",tripulante,modelo,funcao,"FICTICIA-"+funcao,LocalDate.now(relogio).plusYears(2));
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: habilitacao_tripulante. Parâmetros ligados em ordem: tripulante, modelo, funcao, "FICTICIA-"+funcao, LocalDate.now(relogio).plusYears(2).

62      Sql.executar(c,"INSERT INTO
escala_funcionario(funcionario_id,aeroporto_id,voo_id,funcao,inicio,fim) VALUES (?,?,,?,?,?)",tripulante,
origem,voo,funcao,partida.minus(Duration.ofMinutes(45)),chegada.plus(Duration.ofMinutes(30)));
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: escala_funcionario. Parâmetros ligados em ordem: tripulante, origem, voo, funcao, partida.minus(Duration.ofMinutes(45)), chegada.plus(Duration.ofMinutes(30)).

63      }
64      long alocao=Sql.inserir(c,"INSERT INTO
alocacao_recurso(recurso_id,voo_id,finalidade,inicio,fim) VALUES (?,?,'PARTIDA',?,?) RETURNING
id",portao,voo,partida.minus(Duration.ofMinutes(45)),partida);
      Declara variável local alocao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: alocacao_recurso. Parâmetros ligados em ordem: portao, voo, partida.minus(Duration.ofMinutes(45)), partida.

65      Sql.executar(c,"INSERT INTO evento_auditoria(usuario_id,acao,entidade,registro_id,detalhes)
VALUES (?, 'CRIAR_DEMO', 'voo', ?,jsonb_build_object('passageiro',?::bigint,'documento',?::bigint,'tarifa',?::
:bigint,'alocacao',?::bigint,'cenario',?:text))",s.usuarioId(),voo,pessoa,documento,tarifa,alocacao,chave
);
      Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: evento_auditoria. Parâmetros ligados em ordem: s.usuarioId(), voo, pessoa, documento, tarifa, alocacao, chave.

66      return
Map.of("voo",voo,"passageiro",pessoa,"documento",documento,"tarifa",tarifa,"alocacao",alocacao);
      Retorno: entrega Map.of("voo",voo,"passageiro",pessoa,"documento",documento,"tarifa",tarifa,"alocacao",alocacao) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

      Chama Map.of: Fábrica estática: cria o valor do tipo receptor a partir dos argumentos.

67      });
68      }
69

```



## icao - L70 a L70

Gera código fictício X seguido por três letras aleatórias.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: DadosFicticiosServico.codigoLivre (L72).

Chamadas observadas no corpo: new StringBuilder, new java.security.SecureRandom().ints(3,0,26).forEach, new java.security.SecureRandom().ints, new java.security.SecureRandom, codigo.append, codigo.toString.

```
70 private String icao() { var codigo=new StringBuilder("X");new
  java.security.SecureRandom().ints(3,0,26).forEach(n->codigo.append((char)('A'+n)));return
  codigo.toString(); }
```

Início da declaração de icao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Declara variável local codigo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Retorno: entrega codigo.toString() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói StringBuilder, java.security.SecureRandom: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

71

## codigoLivre - L72 a L79

Tenta até 1000 códigos e consulta a tabela correta antes de usá-los. Retorna o primeiro livre; esgotamento lança regra.

Entradas declaradas: (java.sql.Connection c,boolean aeroporto).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: DadosFicticiosServico.criar (L23).

Chamadas observadas no corpo: icao, icao().substring, Sql.listar(c,"SELECT id FROM "+tabela+" WHERE codigo\_icao=?",r->r.getLong(1),codigo).isEmpty, Sql.listar, r.getLong, new br.edu.aeroporto.excecao.RegraNegocioException.

```
72 private String codigoLivre(java.sql.Connection c,boolean aeroporto) throws java.sql.SQLException {
  Início da declaração de codigoLivre. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
  invocado.
```

```
73 for(int tentativa=0;tentativa<1000;tentativa++) {
  Laço controlado por inicialização, condição e atualização: for(int tentativa=0;tentativa<1000;tentativa++). return, break ou exceção
  podem encerrar esse caminho; continue passa à próxima volta.

  Declara variável local tentativa do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
```

```
74 String codigo=aeroporto?icao():icao().substring(1);
  Declara variável local codigo do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

  Escolha de valor: aeroporto. true escolhe icao(); false escolhe icao().substring(1). Só a alternativa escolhida é avaliada.
```

```
75 String tabela=aeroporto?"aeroporto":"companhia_aerea";
  Declara variável local tabela do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

  Escolha de valor: aeroporto. true escolhe "aeroporto"; false escolhe "companhia_aerea". Só a alternativa escolhida é avaliada.
```

```
76 if(Sql.listar(c,"SELECT id FROM "+tabela+" WHERE
  codigo_icao=?",r->r.getLong(1),codigo).isEmpty()) return codigo;
  Decisão específica: (Sql.listar(c,"SELECT id FROM "+tabela+" WHERE codigo_icao=?",r->r.getLong(1),codigo).isEmpty()). Se for
  true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

  Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Parâmetros ligados em ordem: codigo.

  Retorno: entrega codigo ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
  caminho.
```

```
77 }
78 throw new br.edu.aeroporto.excecao.RegraNegocioException("Sem codigo ficticio disponivel.");
  Lança new br.edu.aeroporto.excecao.RegraNegocioException("Sem codigo ficticio disponivel."). Interrompe o caminho normal; a
  exceção sobe até uma captura compatível e pode provocar rollback.
```

```
79 }
80 }
```

## FinanceiroServico

src/main/java/br/edu/aeroporto/servico/FinanceiroServico.java

Orquestra pagamentos simulados idempotentes e reembolsos, verificando saldo, situação e condições vendidas.

101 linhas | 9 métodos/construtores declarados | SHA-256: 5f514d0bf90e498b...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, FinanceiroJdbc, ReservaJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.infraestrutura.jdbc.*;
5 import java.math.BigDecimal;
6 import java.time.Clock;
7 import java.util.Map;
8 import java.util.Set;
9
10 public final class FinanceiroServico {
    Declara FinanceiroServico como tipo com campos e métodos. final impede herança.

11     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

12     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

13     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

14     private final Clock relógio;
    Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

15     private final FinanceiroJdbc repositorio=new FinanceiroJdbc();
    Campo(s) repositorio: private restringe acesso à classe. Tipo FinanceiroJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

    new constrói FinanceiroJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

16     private final ReservaJdbc reservas=new ReservaJdbc();
    Instrução idêntica à de AtendimentoServico:L25. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

17

```

### FinanceiroServico - L18 a L20

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock relógio).

Possíveis chamadores identificados: Main.main (L55).

```

18     public FinanceiroServico(BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock
    relógio) {
    Início da declaração de FinanceiroServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

19         this.banco=banco;this.autorizacao=autorizacao;this.auditoria=auditoria;this.relógio=relógio;
    Instrução idêntica à de ArquivosServico:L21. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

20     }
21

```



## pagar - L22 a L39

Valida chave/forma/valor/resultado. Uma chave existente precisa ter os mesmos dados; pendente pode ser processado posteriormente; já processado retorna mesmo id somente com resultado igual. Nova chave cria pagamento; aprovação pode confirmar reserva.

Entradas declaradas: (Sessao s,long reserva,String chave,String forma,BigDecimal valor,String resultado).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.comercial (L109); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.financeiro (L127).

Chamadas observadas no corpo: Validacao.texto, Dados.valor, Dados.exigir, Set.of("PIX","CARTAO","DINHEIRO").contains, Set.of, Set.of("PENDENTE","APROVADO","RECUSADO").contains, banco.transacaoRepetivel, autorizacao.exigirAtendimento, BloqueiosJdbc.reserva, Sql.unico, Sql::linha, anterior.isPresent, anterior.get, Dados.id, forma.equals, p.get, quantia.compareTo, Dados.dinheiro, "PENDENTE".equals, resultado.equals, processar, validarReserva, Sql.inserir, relógio.instant, auditoria.registrar, s.usuarioId.

22        public long pagar(Sessao s,long reserva,String chave,String forma,BigDecimal valor,String resultado) {  
Início da declaração de pagar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

23        String token=Validacao.texto(chave,"Chave de idempotencia",80);BigDecimal  
quantia=Dados.valor(valor,true);  
Declara variável local token do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
Declara variável local quantia do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

24        Dados.exigir(Set.of("PIX","CARTAO","DINHEIRO").contains(forma) &&  
Set.of("PENDENTE","APROVADO","RECUSADO").contains(resultado),"Pagamento simulado invalido.");  
Regra obrigatória: Set.of("PIX","CARTAO","DINHEIRO").contains(forma) &&  
Set.of("PENDENTE","APROVADO","RECUSADO").contains(resultado) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Pagamento simulado invalido.". Essa condição específica não é substituída por outra validação parecida.

25        return banco.transacaoRepetivel(c->{  
Instrução idêntica à de AtendimentoServico:L33. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

26        autorizacao.exigirAtendimento(c,s);var r=BloqueiosJdbc.reserva(c,reserva);  
Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).  
Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

27        var anterior=Sql.unico(c,"SELECT \* FROM pagamento WHERE chave\_idempotencia=? FOR  
UPDATE",Sql::linha,token);  
Declara variável local anterior com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: pagamento. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: token.

28        if(anterior.isPresent()) {  
Decisão específica: (anterior.isPresent()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

29        var p=anterior.get();Dados.exigir(Dados.id(p,"reserva\_id")==reserva &&  
forma.equals(p.get("forma")) && quantia.compareTo(Dados.dinheiro(p,"valor"))==0,"Chave já usada para outro  
pagamento.");  
Declara variável local p com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
Regra obrigatória: Dados.id(p,"reserva\_id")==reserva && forma.equals(p.get("forma")) &&  
quantia.compareTo(Dados.dinheiro(p,"valor"))==0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Chave já usada para outro pagamento.". Essa condição específica não é substituída por outra validação parecida.

30        if(!"PENDENTE".equals(p.get("situacao"))) {  
Dados.exigir(resultado.equals(p.get("situacao")),"Resultado difere do pagamento já processado.");return  
Dados.id(p,"id"); }  
Decisão específica: (!"PENDENTE".equals(p.get("situacao"))). Se for true, executa o ramo seguinte; se for false, pula esse ramo.  
Regra obrigatória: resultado.equals(p.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Resultado difere do pagamento já processado.". Essa condição específica não é substituída por outra validação parecida.  
Retorno: entrega Dados.id(p,"id") ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
31         if(resultado.equals("PENDENTE")) return Dados.id(p,"id");
```

Decisão específica: (resultado.equals("PENDENTE")). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega Dados.id(p,"id") ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
32         processar(c,s,reserva,Dados.id(p,"id"),quantia,resultado,r);return Dados.id(p,"id");
```

Chama FinanceiroServico.processar: Confere prazo e evita aprovação acima do saldo. Atualiza resultado/processado\_em; com cobertura integral e itens pendentes, emite bilhetes pelo JDBC.

Retorno: entrega Dados.id(p,"id") ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
33     }
34     validarReserva(c,reserva,r);
```

Chama FinanceiroServico.validarReserva: Rejeita reserva encerrada. Se existir item pendente, exige prazo de reserva ainda futuro.

```
35     long id=Sql.inserir(c,"INSERT INTO
pagamento(reserva_id,chave_idempotencia,forma,valor,criado_em) VALUES (?,?=?,?,?) RETURNING
id",reserva,token,forma,quantia,religio.instant());
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: pagamento. Parâmetros ligados em ordem: reserva, token, forma, quantia, religio.instant().

```
36         if(!resultado.equals("PENDENTE")) processar(c,s,reserva,id,quantia,resultado,r);
```

Decisão específica: (!resultado.equals("PENDENTE")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
37         auditoria.registrar(c,s.usuarioId(),"REGISTRAR_PAGAMENTO","pagamento",id);return id;
```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "REGISTRAR\_PAGAMENTO", "pagamento", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
38     });
39 }
40
```

## validarReserva - L41 a L45

Rejeita reserva encerrada. Se existir item pendente, exige prazo de reserva ainda futuro.

Entradas declaradas: (java.sql.Connection c,long reserva,Map<String,Object> r).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: FinanceiroServico.pagar (L22); FinanceiroServico.processar (L47).

Chamadas observadas no corpo: Dados.exigir, Set.of("CANCELADA","EXPIRADA","FINALIZADA").contains, Set.of, r.get, Sql.listar(c,"SELECT id FROM item\_reserva WHERE reserva\_id=? AND situacao='PENDENTE'",l->l.getLong(1),reserva).isEmpty, Sql.listar, l.getLong, Dados.instante(r,"expira\_em").isAfter, Dados.instante, religio.instant.

```
41     private void validarReserva(java.sql.Connection c,long reserva,Map<String,Object> r) throws
java.sql.SQLException {
```

Início da declaração de validarReserva. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
42         Dados.exigir(!Set.of("CANCELADA","EXPIRADA","FINALIZADA").contains(r.get("situacao")), "Reserva
encerrada.");
```

Regra obrigatória: !Set.of("CANCELADA","EXPIRADA","FINALIZADA").contains(r.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Reserva encerrada.". Essa condição específica não é substituída por outra validação parecida.

```
43         boolean pendente=!Sql.listar(c,"SELECT id FROM item_reserva WHERE reserva_id=? AND
situacao='PENDENTE'",l->l.getLong(1),reserva).isEmpty();
```

Declara variável local pendente do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: reserva.

```
44         if(pendente) Dados.exigir(Dados.instante(r,"expira_em").isAfter(religio.instant()), "Reserva
vencida; execute expiracao.");
```

Decisão específica: (pendente). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: Dados.instante(r,"expira\_em").isAfter(religio.instant()) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Reserva vencida; execute expiracao.". Essa condição específica não é substituída por outra validação parecida.

```

45     }
46

```

## processar - L47 a L53

Confere prazo e evita aprovação acima do saldo. Atualiza resultado/processado\_em; com cobertura integral e itens pendentes, emite bilhetes pelo JDBC.

Entradas declaradas: (java.sql.Connection c,Sessao s,long reserva,long id,BigDecimal valor,String resultado,Map<String,Object> r).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: FinanceiroServico.pagar (L22).

Chamadas observadas no corpo: validarReserva, resultado.equals, Dados.exigir, valor.compareTo, repositorio.devido(c,reserva).subtract, repositorio.devido, repositorio.recebido, Sql.executar, relógio.instant, repositorio.recebido(c,reserva).compareTo, Sql.listar(c,"SELECT id FROM item\_reserva WHERE reserva\_id=? AND situacao='PENDENTE'",l->l.getLong(1),reserva).isEmpty, Sql.listar, l.getLong, repositorio.confirmar, auditoria.registrar, s.usuarioId.

```

47     private void processar(java.sql.Connection c,Sessao s,long reserva,long id,BigDecimal valor,String
        resultado,Map<String,Object> r) throws java.sql.SQLException {
        Início da declaração de processar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

48         validarReserva(c,reserva,r);
        Instrução idêntica à de FinanceiroServico:L34. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

49         if(resultado.equals("APROVADO")) Dados.exigir(valor.compareTo(repositorio.devido(c,reserva).subtra
            ct(repositorio.recebido(c,reserva)))<=0,"Pagamento supera o saldo devido.");
        Decisão específica: (resultado.equals("APROVADO")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

        Regra obrigatória: valor.compareTo(repositorio.devido(c,reserva).subtract(repositorio.recebido(c,reserva)))<=0 deve resultar em
        true. Se false, Dados.exigir lança RegraNegocioException com "Pagamento supera o saldo devido.". Essa condição específica
        não é substituída por outra validação parecida.

50         Sql.executar(c,"UPDATE pagamento SET situacao=?,processado_em=? WHERE
            id=?",resultado,relógio.instant(),id);
        Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: pagamento.
        Parâmetros ligados em ordem: resultado, relógio.instant(), id.

51         if(resultado.equals("APROVADO") &&
            repositorio.recebido(c,reserva).compareTo(repositorio.devido(c,reserva))>=0 && !Sql.listar(c,"SELECT id
            FROM item_reserva WHERE reserva_id=? AND situacao='PENDENTE'",l->l.getLong(1),reserva).isEmpty())
            repositorio.confirmar(c,reserva,relógio.instant());
        Decisão específica: (resultado.equals("APROVADO")) &&
            repositorio.recebido(c,reserva).compareTo(repositorio.devido(c,reserva))>=0 && !Sql.listar(c,"SELECT id FROM item_reserva
            WHERE reserva_id=? AND situacao='PENDENTE'",l->l.getLong(1),reserva).isEmpty()). Se for true, executa o ramo seguinte; se
            for false, pula esse ramo.

        Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. Parâmetros
        ligados em ordem: reserva.

52         auditoria.registrar(c,s.usuarioId(),"PROCESSAR_PAGAMENTO","pagamento",id);
        Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "PROCESSAR_PAGAMENTO", "pagamento", id).

53     }
54

```

## cancelarPagamento - L55 a L61

Só UPDATE de pagamento PENDENTE pode afetar uma linha. Mantém o registro com CANCELADO e audita.

Entradas declaradas: (Sessao s,long id).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.regrasComplementares (L369); MenuOperacional.financeiro (L127).

Chamadas observadas no corpo: banco.transacao, autorizacao.exigirAtendimento, Sql.registro, BloqueiosJdbc.reserva, Dados.id, Dados.exigir, Sql.executar, relógio.instant, auditoria.registrar, s.usuarioId.

```

55     public void cancelarPagamento(Sessao s,long id) {
        Início da declaração de cancelarPagamento. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.

56         banco.transacao(c->{
        Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A
        posição e o contexto do método acima mostram quando ela ocorre.

```

```
57      autorizacao.exigirAtendimento(c,s);var p=Sql.registro(c,"SELECT reserva_id FROM pagamento
WHERE id=?",id);BloqueiosJdbc.reserva(c,Dados.id(p,"reserva_id"));
```

Confere no banco o acesso vigente do usuário para esta operação: `exigirAtendimento(c, s)`.

Declara variável local `p` com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como `Map<String,Object>`; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: `pagamento`. Parâmetros ligados em ordem: `id`.

```
58      Dados.exigir(Sql.executar(c,"UPDATE pagamento SET situacao='CANCELADO',processado_em=? WHERE
id=? AND situacao='PENDENTE'",relogio.instant(),id)==1,"Apenas pagamento pendente pode ser cancelado.");
```

Regra obrigatória: `Sql.executar(c,"UPDATE pagamento SET situacao='CANCELADO',processado_em=? WHERE id=? AND situacao='PENDENTE'",relogio.instant(),id)==1` deve resultar em `true`. Se `false`, `Dados.exigir` lança `RegraNegocioException` com "Apenas pagamento pendente pode ser cancelado.". Essa condição específica não é substituída por outra validação parecida.

Executa comando de alteração e devolve `int` com a quantidade de linhas afetadas. Tabelas mencionadas: `pagamento`.

Parâmetros ligados em ordem: `relogio.instant()`, `id`.

```
59      auditoria.registrar(c,s.usuarioId(),"CANCELAR_PAGAMENTO","pagamento",id);return null;
```

Registra o evento de auditoria na conexão fornecida: `registrar(c, s.usuarioId(), "CANCELAR_PAGAMENTO", "pagamento", id)`.

`return null` encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à `lambda` ou ao método externo.

```
60    });
61    }
62
```

## confirmar - L63 a L65

Permissão, lock de reserva e confirmação dos itens; a insuficiência de pagamento é recusada pelo JDBC.

Entradas declaradas: (Sessao `s`,long `reserva`).

Saída declarada: `void` - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: `MenuOperacional.financeiro` (L127).

Chamadas observadas no corpo: `banco.transacaoRepetivel`, `autorizacao.exigirAtendimento`, `BloqueiosJdbc.reserva`, `repositorio.confirmar`, `relogio.instant`, `auditoria.registrar`, `s.usuarioId`.

```
63    public void confirmar(Sessao s,long reserva) {
```

Início da declaração de `confirmar`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
64      banco.transacaoRepetivel(c->{autorizacao.exigirAtendimento(c,s);BloqueiosJdbc.reserva(c,reserva);r
epositorio.confirmar(c,reserva,relogio.instant());auditoria.registrar(c,s.usuarioId(),"CONFIRMAR_RESERVA",
"reserva",reserva);return null;});
```

Executa uma transação completa com até três tentativas totais somente para deadlock/serialização (40P01/40001). O callback pode executar novamente.

Confere no banco o acesso vigente do usuário para esta operação: `exigirAtendimento(c, s)`.

Registra o evento de auditoria na conexão fornecida: `registrar(c, s.usuarioId(), "CONFIRMAR_RESERVA", "reserva", reserva)`.

`return null` encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à `lambda` ou ao método externo.

```
65    }
66
```

## saldo - L67 a L69

Devolve devido menos recebido; não altera reserva ou pagamentos.

Entradas declaradas: (Sessao `s`,long `reserva`).

Saída declarada: `BigDecimal` - valor decimal, utilizado nos cálculos financeiros.

Possíveis chamadores identificados: `MenuOperacional.financeiro` (L127).

Chamadas observadas no corpo: `banco.consultar`, `autorizacao.exigirAtendimento`, `repositorio.devido(c,reserva).subtract`, `repositorio.devido`, `repositorio.recebido`.

```
67    public BigDecimal saldo(Sessao s,long reserva) {
```

Início da declaração de `saldo`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
68         return banco.consultar(c->{autorizacao.exigirAtendimento(c,s);return
repositorio.devido(c,reserva).subtract(repositorio.recebido(c,reserva));});
```

Retorno: entrega banco.consultar(c->{autorizacao.exigirAtendimento(c,s);return repositorio.devido(c,reserva).subtract(repositorio.recebido(c,reserva));}) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

Retorno: entrega repositorio.devido(c,reserva).subtract(repositorio.recebido(c,reserva)) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
69     }
70 }
```

## solicitarReembolso - L71 a L87

A chave é idempotente e valida pagamento/quantia/motivo. Exige pagamento aprovado, disponibilidade, valores já liberados e direito da tarifa; retorna id da solicitação.

Entradas declaradas: (Sessao s,long pagamento,BigDecimal valor,String chave,String motivo).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.concorrencia (L164); MenuOperacional.financeiro (L127).

Chamadas observadas no corpo: Dados.valor, Validacao.texto, banco.transacaoRepetivel, autorizacao.exigirAtendimento, Sql.registro, Dados.id, BloqueiosJdbc.reserva, Sql.unico, Sql::linha, anterior.isPresent, Dados.exigir, anterior.get, quantia.compareTo, Dados.dinheiro, razao.equals, anterior.get().get, "APROVADO".equals, p.get, Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM reembolso WHERE pagamento\_id=? AND situacao IN ('SOLICITADO','PROCESSADO')"),r->r.getBigDecimal(1,pagamento).orElse, r.getBigDecimal, Dados.dinheiro(p,"valor").subtract, repositorio.recebido(c,reserva).subtract, repositorio.recebido, repositorio.devido, reservas.limiteDevolucao, Sql.inserir, relógio.instant, auditoria.registrar, s.usuarioId.

```
71     public long solicitarReembolso(Sessao s,long pagamento,BigDecimal valor,String chave,String motivo) {
Início da declaração de solicitarReembolso. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
```

```
72         BigDecimal quantia=Dados.valor(valor,true);String token=Validacao.texto(chave,"Chave",80);String
razao=Validacao.texto(motivo,"Motivo",500);
```

Declara variável local quantia do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local token do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local razao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
73         return banco.transacaoRepetivel(c->{
```

Instrução idêntica à de AtendimentoServico:L33. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
74         autorizacao.exigirAtendimento(c,s);var inicial=Sql.registro(c,"SELECT reserva_id FROM
pagamento WHERE id=?",pagamento);long
reserva=Dados.id(inicial,"reserva_id");BloqueiosJdbc.reserva(c,reserva);
```

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

Declara variável local inicial com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pagamento. Parâmetros ligados em ordem: pagamento.

Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
75         var p=Sql.registro(c,"SELECT * FROM pagamento WHERE id=? FOR UPDATE",pagamento);
```

Declara variável local p com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pagamento. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: pagamento.

```
76         var anterior=Sql.unico(c,"SELECT * FROM reembolso WHERE
chave_idempotencia=?",Sql::linha,token);
```

Declara variável local anterior com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: reembolso. Parâmetros ligados em ordem: token.

```
77      if(anterior.isPresent()) { Dados.exigir(Dados.id(anterior.get(),"pagamento_id")==pagamento &&
quantia.compareTo(Dados.dinheiro(anterior.get(),"valor"))==0 &&
razao.equals(anterior.get().get("motivo")),"Chave de reembolso ja utilizada com outros dados.");return
Dados.id(anterior.get(),"id"); }
```

Decisão específica: (anterior.isPresent()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Regra obrigatória: Dados.id(anterior.get(),"pagamento\_id")==pagamento && quantia.compareTo(Dados.dinheiro(anterior.get(),"valor"))==0 && razao.equals(anterior.get().get("motivo")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Chave de reembolso ja utilizada com outros dados.". Essa condição específica não é substituída por outra validação parecida.

Retorno: entrega Dados.id(anterior.get(),"id") ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
78      Dados.exigir("APROVADO".equals(p.get("situacao")), "Pagamento nao aprovado.");
```

Regra obrigatória: "APROVADO".equals(p.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Pagamento nao aprovado.". Essa condição específica não é substituída por outra validação parecida.

```
79      BigDecimal comprometido=Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM reembolso WHERE
pagamento_id=? AND situacao IN
('SOLICITADO','PROCESSADO')",r->r.getBigDecimal(1,pagamento).orElse(BigDecimal.ZERO));
```

Declara variável local comprometido do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: reembolso. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: pagamento.

```
80      Dados.exigir(quantia.compareTo(Dados.dinheiro(p,"valor").subtract(comprometido))<=0,"Devolucao
supera pagamento disponivel.");
```

Regra obrigatória: quantia.compareTo(Dados.dinheiro(p,"valor").subtract(comprometido))<=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Devolucao supera pagamento disponivel.". Essa condição específica não é substituída por outra validação parecida.

```
81      BigDecimal liberado=repositorio.recebido(c,reserva).subtract(repositorio.devido(c,reserva));
```

Declara variável local liberado do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
82      Dados.exigir(quantia.compareTo(liberado)<=0,"Cancele os itens correspondentes antes de
devolver valores devidos.");
```

Regra obrigatória: quantia.compareTo(liberado)<=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Cancele os itens correspondentes antes de devolver valores devidos.". Essa condição específica não é substituída por outra validação parecida.

```
83      Dados.exigir(quantia.compareTo(reservas.limiteDevolucao(c,reserva))<=0,"Devolucao supera as
condicoes de cancelamento vendidas.");
```

Regra obrigatória: quantia.compareTo(reservas.limiteDevolucao(c,reserva))<=0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Devolucao supera as condicoes de cancelamento vendidas.". Essa condição específica não é substituída por outra validação parecida.

```
84      long id=Sql.inserir(c,"INSERT INTO
reembolso(pagamento_id,reserva_id,valor,motivo,chave_idempotencia,solicitado_em) VALUES (?,?,,?,?,?)
RETURNING id",pagamento,reserva,quantia,razao,token,religio.instant());
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: reembolso. Parâmetros ligados em ordem: pagamento, reserva, quantia, razao, token, religio.instant().

```
85      auditoria.registrar(c,s.usuarioId(),"SOLICITAR_REEMBOLSO","reembolso",id);return id;
```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "SOLICITAR\_REEMBOLSO", "reembolso", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
86      });
87      }
88  }
```

## processarReembolso - L89 a L100

Repetir o destino já alcançado não muda nada. Só SOLICITADO pode ir a PROCESSADO/RECUSADO; confere compromissos contra pagamento e audita.

Entradas declaradas: (Sessao s,long id,boolean aprovar).



Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.concorrencia (L164); MenuOperacional.financeiro (L127).

Chamadas observadas no corpo: banco.transacaoRepetivel, autorizacao.exigirAtendimento, Sql.registro, BloqueiosJdbc.reserva, Dados.id, destino.equals, r.get, Dados.exigir, "SOLICITADO".equals, Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM reembolso WHERE pagamento\_id=? AND situacao IN ('SOLICITADO','PROCESSADO')",l->l.getBigDecimal(1),Dados.id(p,"id")).orElse, Sql.unico, l.getBigDecimal, comprometido.compareTo, Dados.dinheiro, Sql.executar, relógio.instant, auditoria.registrar, s.usuarioId.

```
89 public void processarReembolso(Sessao s,long id,boolean aprovar) {
```

Início da declaração de processarReembolso. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
90 banco.transacaoRepetivel(c->{
```

Instrução idêntica à de AtendimentoServico:L61. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
91 autorizacao.exigirAtendimento(c,s);var inicial=Sql.registro(c,"SELECT * FROM reembolso WHERE id=?",id);BloqueiosJdbc.reserva(c,Dados.id(inicial,"reserva_id"));
```

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

Declara variável local inicial com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: reembolso. Parâmetros ligados em ordem: id.

```
92 var p=Sql.registro(c,"SELECT * FROM pagamento WHERE id=? FOR UPDATE",Dados.id(inicial,"pagamento_id"));var r=Sql.registro(c,"SELECT * FROM reembolso WHERE id=? FOR UPDATE",id);
```

Declara variável local p com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pagamento. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: Dados.id(inicial,"pagamento\_id").

Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: reembolso. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

```
93 String destino=aprovar?"PROCESSADO":"RECUSADO";
```

Declara variável local destino do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Escolha de valor: aprovar. true escolhe "PROCESSADO"; false escolhe "RECUSADO". Só a alternativa escolhida é avaliada.

```
94 if(destino.equals(r.get("situacao"))) return null;
```

Decisão específica: (destino.equals(r.get("situacao"))). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```
95 Dados.exigir("SOLICITADO".equals(r.get("situacao")), "Reembolso ja encerrado.");
```

Regra obrigatória: "SOLICITADO".equals(r.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Reembolso ja encerrado.". Essa condição específica não é substituída por outra validação parecida.

```
96 BigDecimal comprometido=Sql.unico(c,"SELECT COALESCE(SUM(valor),0) FROM reembolso WHERE pagamento_id=? AND situacao IN ('SOLICITADO','PROCESSADO')",l->l.getBigDecimal(1),Dados.id(p,"id")).orElse(BigDecimal.ZERO);
```

Declara variável local comprometido do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado. Tabelas mencionadas: reembolso. SUM agrega valores no PostgreSQL; COALESCE, quando presente, define o caso sem valores. Parâmetros ligados em ordem: Dados.id(p,"id").

```
97         Dados.exigir(comprometido.compareTo(Dados.dinheiro(p,"valor"))<=0,"Reembolsos superam  
pagamento.");
```

Regra obrigatória: `comprometido.compareTo(Dados.dinheiro(p,"valor"))<=0` deve resultar em `true`. Se `false`, `Dados.exigir` lança `RegraNegocioException` com "Reembolsos superam pagamento.". Essa condição específica não é substituída por outra validação parecida.

```
98         Sql.executar(c,"UPDATE reembolso SET situacao=?,processado_em=? WHERE id=?",destino,religio.in  
stant(),id);auditoria.registrar(c,s.usuarioId(),"PROCESSAR_REEMBOLSO","reembolso",id);return null;
```

Executa comando de alteração e devolve `int` com a quantidade de linhas afetadas. Tabelas mencionadas: `reembolso`. Parâmetros ligados em ordem: `destino`, `religio.instant()`, `id`.

Registra o evento de auditoria na conexão fornecida: `registrar(c, s.usuarioId(), "PROCESSAR_REEMBOLSO", "reembolso", id)`.

`return null` encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à `lambda` ou ao método externo.

```
99         });  
100     }  
101 }
```



## InicializacaoServico

src/main/java/br/edu/aeroporto/servico/InicializacaoServico.java

Valida senha e nascimento, gera hash e chama a criação transacional do primeiro acesso com auditoria.

42 linhas | 2 métodos/construtores declarados | SHA-256: 154aa473ffafd7...

Dependências guardadas em campos: AuditoriaJdbc, BancoDados, InicializacaoJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.PrimeiroAcesso;
4 import br.edu.aeroporto.dominio.Validacao;
5 import br.edu.aeroporto.excecao.RegraNegocioException;
6 import br.edu.aeroporto.infraestrutura.jdbc.AuditoriaJdbc;
7 import br.edu.aeroporto.infraestrutura.jdbc.BancoDados;
8 import br.edu.aeroporto.infraestrutura.jdbc.InicializacaoJdbc;
9 import br.edu.aeroporto.seguranca.Senhas;
10 import java.time.Clock;
11 import java.time.LocalDate;
12 import java.util.Arrays;
13
14 public final class InicializacaoServico {
    Declara InicializacaoServico como tipo com campos e métodos. final impede herança.

15     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

16     private final InicializacaoJdbc inicializacao;
    Campo(s) inicializacao: private restringe acesso à classe. Tipo InicializacaoJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

17     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

18     private final Clock relógio;
    Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

19

```

### InicializacaoServico - L20 a L25

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco, InicializacaoJdbc inicializacao, AuditoriaJdbc auditoria, Clock relógio).

Possíveis chamadores identificados: IntegracaoTeste.inicializar (L50); Main.main (L55).

```

20     public InicializacaoServico(BancoDados banco, InicializacaoJdbc inicializacao, AuditoriaJdbc
auditoria, Clock relógio) {
    Início da declaração de InicializacaoServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

21         this.banco = banco;
    Instrução idêntica à de AutenticacaoServico:L18. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

22         this.inicializacao = inicializacao;
    Guarda no campo inicializacao da instância atual o valor de inicializacao. O this diferencia o campo de parâmetros com o mesmo nome.

23         this.auditoria = auditoria;
    Instrução idêntica à de AutenticacaoServico:L20. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

24         this.relógio = relógio;
    Guarda no campo relógio da instância atual o valor de relógio. O this diferencia o campo de parâmetros com o mesmo nome.

```

```

25     }
26

```

## criar - L27 a L41

Senha entre 12 e 128; nascimento validado pelo Clock. Gera hash antes da transação, cria administrador e registra auditoria; finally zera char[].

Entradas declaradas: (PrimeiroAcesso dados, char[] senha).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.inicializar (L50); Main.inicializar (L156).

Chamadas observadas no corpo: new RegraNegocioException, LocalDate.now, Validacao.nascimento, dados.nascimento, Senhas.gerar, banco.transacao, inicializacao.criarAdministrador, relógio.getZone, auditoria.registrar, Arrays.fill.

```

27     public void criar(PrimeiroAcesso dados, char[] senha) {
        Início da declaração de criar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

28         try {
            Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

29             if (senha.length < 12 || senha.length > 128) throw new RegraNegocioException("Use uma senha de
            12 a 128 caracteres.");
            Decisão específica: (senha.length < 12 || senha.length > 128). Se for true, executa o ramo seguinte; se for false, pula esse ramo.
            Aqui o ramo interrompe a operação lançando a exceção.

            Lança new RegraNegocioException("Use uma senha de 12 a 128 caracteres."). Interrompe o caminho normal; a exceção sobe
            até uma captura compatível e pode provocar rollback.

            new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
            banco por si só.

30             LocalDate hoje = LocalDate.now(relógio);
            Declara variável local hoje do tipo LocalDate. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

31             Validacao.nascimento(dados.nascimento(), hoje);
            Chama Validacao.nascimento: Exige data não nula e não futura em relação ao hoje recebido. A função não decide sozinha qual
            relógio usar.

32             String hash = Senhas.gerar(senha);
            Declara variável local hash do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

33             banco.transacao(conexao -> {
                Instrução idêntica à de AutenticacaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
                desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

34                 long usuario = inicializacao.criarAdministrador(conexao, dados, hash, hoje,
                relógio.getZone());
                Declara variável local usuario do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

35                 auditoria.registrar(conexao, usuario, "CRIAR_PRIMEIRO_ACESSO", "usuario_sistema",
                usuario);
                Registra o evento de auditoria na conexão fornecida: registrar(conexao, usuario, "CRIAR_PRIMEIRO_ACESSO",
                "usuario_sistema", usuario).

36                 return null;
                return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o
                contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

37             });
38         } finally {
            Bloco de finalização, executado ao sair do try/catch; neste corpo, observe quais recursos ou dados são limpos.

39             Arrays.fill(senha, '\0');
            Instrução idêntica à de AutenticacaoServico:L43. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
            desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

40         }
41     }
42 }

```

## OperacaoServico

src/main/java/br/edu/aeroporto/servico/OperacaoServico.java

Administra recursos, interdições, habilitações, escalas, manutenção, serviços de solo e ocorrências.

173 linhas | 14 métodos/construtores declarados | SHA-256: 58aba7c7caa438e5...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, PlanejamentoJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.infraestrutura.jdbc.*;
5 import java.math.BigDecimal;
6 import java.time.Clock;
7 import java.time.Instant;
8 import java.time.LocalDate;
9 import java.util.List;
10 import java.util.Map;
11 import java.util.Set;
12
13 public final class OperacaoServico {
    Declara OperacaoServico como tipo com campos e métodos. final impede herança.

14     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

15     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

16     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

17     private final Clock relógio;
    Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

18     private final PlanejamentoJdbc planejamento=new PlanejamentoJdbc();
    Campo(s) planejamento: private restringe acesso à classe. Tipo PlanejamentoJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

    new constrói PlanejamentoJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

19

```

### OperacaoServico - L20 a L22

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock relógio).

Possíveis chamadores identificados: Main.main (L55).

```

20     public OperacaoServico(BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock
        relógio) {
    Início da declaração de OperacaoServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

21         this.banco=banco;this.autorizacao=autorizacao;this.auditoria=auditoria;this.relógio=relógio;
    Instrução idêntica à de ArquivosServico:L21. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

22     }
23

```

## alocar - L24 a L31

Valida recurso por voo/local/compatibilidade e grava alocação depois de lock/serialização.

Entradas declaradas: (Sessao s,long voo,long recurso,String finalidade,Instant inicio,Instant fim).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: banco.transacao, autorizacao.exigirOperacao, planejamento.serializar, BloqueiosJdbc.voo, OperacaoJdbc.validarRecurso, Sql.inserir, auditoria.registrar, s.usuarioId.

```

24     public long alocar(Sessao s,long voo,long recurso,String finalidade,Instant inicio,Instant fim) {
        Início da declaração de alocar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

25         return banco.transacao(c->{
            Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
            desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

26             autorizacao.exigirOperacao(c,s);planejamento.serializar(c);BloqueiosJdbc.voo(c,voo);
            Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, s).

27             OperacaoJdbc.validarRecurso(c,voo,recurso,finalidade,inicio,fim);
            Chama OperacaoJdbc.validarRecurso: Valida finalidade/intervalo, voo/localidades ativos, recurso disponível, aeroporto esperado,
            terminal vigente, envergadura e comprimento mínimo de pista.

28             long id=Sql.inserir(c,"INSERT INTO alocao_recurso(recurso_id,voo_id,finalidade,inicio,fim)
VALUES (?,?,?,?) RETURNING id",recurso,voo,finalidade,inicio,fim);
            Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

            Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas:
            alocao_recurso. Parâmetros ligados em ordem: recurso, voo, finalidade, inicio, fim.

29             auditoria.registrar(c,s.usuarioId(),"ALOCAR_RECURSO","alocacao_recurso",id);return id;
            Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "ALOCAR_RECURSO", "alocacao_recurso", id).

            Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

30         });
31     }
32 
```

## interditar - L33 a L41

Cria alocação INTERDICAÇÃO para um intervalo sem voo; exclusões do banco rejeitam sobreposição incompatível.

Entradas declaradas: (Sessao s,long recurso,Instant inicio,Instant fim,String motivo).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: Dados.exigir, fim.isAfter, Validacao.texto, banco.transacao, autorizacao.exigirOperacao, planejamento.serializar, Sql.registro, Sql.inserir, auditoria.registrar, s.usuarioId.

```

33     public long interditar(Sessao s,long recurso,Instant inicio,Instant fim,String motivo) {
        Início da declaração de interditar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

34         Dados.exigir(fim.isAfter(inicio),"Intervalo invalido.");String
        razao=Validacao.texto(motivo,"Motivo",1000);
        Regra obrigatória: fim.isAfter(inicio) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Intervalo
        invalido.". Essa condição específica não é substituída por outra validação parecida.

        Declara variável local razao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

35         return banco.transacao(c->{
            Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
            desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

36             autorizacao.exigirOperacao(c,s);planejamento.serializar(c);
            Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, s).

37             Sql.registro(c,"SELECT id FROM recurso_aeroportuario WHERE id=? FOR UPDATE",recurso);
            Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
            recurso_aeroportuario. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: recurso.

```

```

38      long id=Sql.inserir(c,"INSERT INTO
alocacao_recurso(recurso_id,finalidade,inicio,fim,observacao) VALUES (?, 'INTERDICAÇÃO',?,?,?) RETURNING
id",recurso,inicio,fim,razao);

```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: alocação\_recurso. Parâmetros ligados em ordem: recurso, inicio, fim, razao.

```

39      auditoria.registrar(c,s.usuarioId(),"INTERDITAR_RECURSO","alocacao_recurso",id);return id;

```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "INTERDITAR\_RECURSO", "alocacao\_recurso", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```

40    });
41  }
42

```

## liberarAlocacao - L43 a L54

Desativa alocação preservando registro; se ligada a voo, impede liberação durante EMBARQUE.

Entradas declaradas: (Sessao s,long id).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: banco.transacao, autorizacao.exigirOperacao, planejamento.serializar, Sql.registro, a.get, BloqueiosJdbc.voo, Dados.id, Dados.exigir, "EMBARQUE".equals, v.get, Sql.executar, auditoria.registrar, s.usuarioId.

```

43  public void liberarAlocacao(Sessao s,long id) {

```

Início da declaração de liberarAlocacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

44      banco.transacao(c->{

```

Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

45      autorizacao.exigirOperacao(c,s);planejamento.serializar(c);

```

Instrução idêntica à de OperacaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

46      var a=Sql.registro(c,"SELECT * FROM alocação_recurso WHERE id=?",id);

```

Declara variável local a com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: alocação\_recurso. Parâmetros ligados em ordem: id.

```

47      if(a.get("voo_id")!=null) {

```

Decisão específica: (a.get("voo\_id")!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```

48      var v=BloqueiosJdbc.voo(c,Dados.id(a,"voo_id"));

```

Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```

49      Dados.exigir(!"EMBARQUE".equals(v.get("situacao")),"Nao libere o recurso durante
embarque.");

```

Regra obrigatória: !"EMBARQUE".equals(v.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Nao libere o recurso durante embarque.". Essa condição específica não é substituída por outra validação parecida.

```

50    }
51    Sql.executar(c,"UPDATE alocação_recurso SET ativa=false WHERE id=?",id);

```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: alocação\_recurso. Parâmetros ligados em ordem: id.

```

52      auditoria.registrar(c,s.usuarioId(),"LIBERAR_RECURSO","alocacao_recurso",id);return null;

```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "LIBERAR\_RECURSO", "alocacao\_recurso", id).

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```

53    });
54  }
55

```

## habilitar - L56 a L68

Administra licença por funcionário/modelo/função com upsert; validade deve estar vigente e as escalas futuras são revalidadas.

Entradas declaradas: (Sessao s,long funcionario,long modelo,String funcao,String licenca,LocalDate validade).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: Dados.exigir, Set.of("COMANDANTE","COPILOTO","COMISSARIO").contains, Set.of, validade.isBefore, LocalDate.now, Validacao.texto, banco.transacao, autorizacao.exigirAdministrador, planejamento.serialize, Sql.inserir, Sql.listar, r.getLong, OperacaoJdbc.validarEscalasDoVoo, auditoria.registrar, s.usuarioId.

```

56     public long habilitar(Sessao s,long funcionario,long modelo,String funcao,String licenca,LocalDate
      validade) {
      Início da declaração de habilitar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
      invocado.

57         Dados.exigir(Set.of("COMANDANTE","COPILOTO","COMISSARIO").contains(funcao) && validade!=null &&
      !validade.isBefore(LocalDate.now(relogio)),"Habilitacao invalida.");
      Regra obrigatória: Set.of("COMANDANTE","COPILOTO","COMISSARIO").contains(funcao) && validade!=null &&
      !validade.isBefore(LocalDate.now(relogio)) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com
      "Habilitacao invalida.". Essa condição específica não é substituída por outra validação parecida.

58         String numero=Validacao.texto(licenca,"Licenca",40);
      Declara variável local numero do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

59         return banco.transacao(c->{
      Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
      desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

60             autorizacao.exigirAdministrador(c,s);planejamento.serialize(c);
      Confere no banco o acesso vigente do usuário para esta operação: exigirAdministrador(c, s).

61             long id=Sql.inserir(c,""
      Instrução idêntica à de PlanejamentoJdbc:L45. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
      execução. A posição e o contexto do método acima mostram quando ela ocorre.

62             INSERT INTO
      habilitacao_tripulante(funcionario_id,modelo_id,funcao,numero_licenca,validade) VALUES (?, ?, ?, ?, ?)
      Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
      chamado, não pelo compilador Java.

63             ON CONFLICT (funcionario_id,modelo_id,funcao) DO UPDATE SET
      numero_licenca=EXCLUDED.numero_licenca,validade=EXCLUDED.validade RETURNING id
      Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
      determina onde a instrução termina.

64             """,funcionario,modelo,funcao,numero,validade);
      Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu
      uso.

65             for(long voo:Sql.listar(c,"SELECT DISTINCT voo_id FROM escala_funcionario WHERE
      funcionario_id=? AND ativa AND voo_id IS NOT NULL AND fim>CURRENT_TIMESTAMP",r->r.getLong(1),funcionario))
      OperacaoJdbc.validarEscalasDoVoo(c,voo);
      Percorre cada elemento da coleção/vetor: for(long voo:Sql.listar(c,"SELECT DISTINCT voo_id FROM escala_funcionario WHERE
      funcionario_id=? AND ativa AND voo_id IS NOT NULL AND fim>CURRENT_TIMESTAMP",r->r.getLong(1),funcionario)). return,
      break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

      Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: escala_funcionario. Parâmetros
      ligados em ordem: funcionario.

66             auditoria.registrar(c,s.usuarioId(),"HABILITAR_TRIPULANTE","habilitacao_tripulante",id);return
      id;
      Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "HABILITAR_TRIPULANTE",
      "habilitacao_tripulante", id).

      Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

67         });
68     }
69 }

```

## escalar - L70 a L79

Aceita funções do catálogo, monta Map com dados da jornada e valida antes de INSERT. Voo pode ser null para funções de solo.

Entradas declaradas: (Sessao s,long funcionario,long aeroporto,Long voo,String funcao,Instant inicio,Instant fim).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: Dados.exigir, Set.of("COMANDANTE","COPILOTO","COMISSARIO","ATENDIMENTO","SOLO","MANUTENCAO","SEGURANCA").contains, Set.of, banco.transacao, autorizacao.exigirOperacao, planejamento.serializar, BloqueiosJdbc.voo, new java.util.HashMap<String,Object>, dados.put, OperacaoJdbc.validarEscala, Sql.inserir, auditoria.registrar, s.usuarioId.

```
70 public long escalar(Sessao s,long funcionario,long aeroporto,Long voo,String funcao,Instant
    inicio,Instant fim) {
```

Início da declaração de escalar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
71 Dados.exigir(Set.of("COMANDANTE","COPILOTO","COMISSARIO","ATENDIMENTO","SOLO","MANUTENCAO","SEGURA
    NCA").contains(funcao),"Funcao invalida.");
```

Regra obrigatória: Set.of("COMANDANTE","COPILOTO","COMISSARIO","ATENDIMENTO","SOLO","MANUTENCAO","SEGURANCA").contains(funcao) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Funcao invalida.". Essa condição específica não é substituída por outra validação parecida.

```
72 return banco.transacao(c->{
```

Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
73 autorizacao.exigirOperacao(c,s);planejamento.serializar(c);if(voo!=null)
    BloqueiosJdbc.voo(c,voo);
```

Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, s).

Decisão específica: (voo!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
74 var dados=new java.util.HashMap<String,Object>();dados.put("funcionario_id",funcionario);dados
    .put("aeroporto_id",aeroporto);dados.put("voo_id",voo);dados.put("funcao",funcao);dados.put("inicio",inici
    o);dados.put("fim",fim);
```

Declara variável local dados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói java.util.HashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
75 OperacaoJdbc.validarEscala(c,dados,null);
```

Chama OperacaoJdbc.validarEscala: Jornada <=14 h, contrato vigente, aeroporto/voo compatíveis, cobertura de 45 min antes/30 depois, licença válida. Jornada anterior/proxima exigem 12 h e continuidade; primeira parte da base.

```
76 long id=Sql.inserir(c,"INSERT INTO
    escala_funcionario(funcionario_id,aeroporto_id,voo_id,funcao,inicio,fim) VALUES (?, ?, ?, ?, ?, ?) RETURNING
    id",funcionario,aeroporto,voo,funcao,inicio,fim);
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: escala\_funcionario. Parâmetros ligados em ordem: funcionario, aeroporto, voo, funcao, inicio, fim.

```
77 auditoria.registrar(c,s.usuarioId(),"ESCALAR_FUNCIONARIO","escala_funcionario",id);return id;
```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "ESCALAR\_FUNCIONARIO", "escala\_funcionario", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
78     });
79 }
80
```

## liberarEscala - L81 a L92

Desativa jornada; quando vinculada a voo, recusa retirada da equipe durante EMBARQUE/EM\_VOO.

Entradas declaradas: (Sessao s,long id).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.operacao (L150).



Chamadas observadas no corpo: banco.transacao, autorizacao.exigirOperacao, planejamento.serialize, Sql.registro, e.get, BloqueiosJdbc.voo, Dados.id, Dados.exigir, Set.of("PROGRAMADO","CHECKIN\_ABERTO","CANCELADO","CONCLUIDO").contains, Set.of, v.get, Sql.executar, auditoria.registrar, s.usuarioId.

```

81     public void liberarEscala(Sessao s,long id) {
        Início da declaração de liberarEscala. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

82         banco.transacao(c->{
            Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A
            posição e o contexto do método acima mostram quando ela ocorre.

83             autorizacao.exigirOperacao(c,s);planejamento.serialize(c);
            Instrução idêntica à de OperacaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
            execução. A posição e o contexto do método acima mostram quando ela ocorre.

84             var e=Sql.registro(c,"SELECT * FROM escala_funcionario WHERE id=?",id);
            Declara variável local e com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é
            o bloco atual.

            Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
            escala_funcionario. Parâmetros ligados em ordem: id.

85             if(e.get("voo_id")!=null) {
                Decisão específica: (e.get("voo_id")!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

86                 var v=BloqueiosJdbc.voo(c,Dados.id(e,"voo_id"));
                Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é
                o bloco atual.

87                 Dados.exigir(Set.of("PROGRAMADO","CHECKIN_ABERTO","CANCELADO","CONCLUIDO").contains(v.get(
                    "situacao")), "Equipe em operacao.");
                Regra obrigatória: Set.of("PROGRAMADO","CHECKIN_ABERTO","CANCELADO","CONCLUIDO").contains(v.get("situacao"))
                deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Equipe em operacao.". Essa condição
                específica não é substituída por outra validação parecida.

88             }
89             Sql.executar(c,"UPDATE escala_funcionario SET ativa=false WHERE id=?",id);
            Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: escala_funcionario.
            Parâmetros ligados em ordem: id.

90             auditoria.registrar(c,s.usuarioId(),"LIBERAR_ESCALA","escala_funcionario",id);return null;
            Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "LIBERAR_ESCALA", "escala_funcionario", id).

            return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o
            contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

91         });
92     }
93 
```

## manutencao - L94 a L105

Valida tipo/intervalo/custo, aeronave não inativa e responsável ativo; insere manutenção e agenda associada na mesma transação.

Entradas declaradas: (Sessao s,long aeronave,long responsavel,String tipo,String descricao,Instant inicio,Instant fim,BigDecimal custo).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: Dados.exigir, Set.of("PREVENTIVA","CORRETIVA","INSPECAO").contains, Set.of, fim.isAfter, Validacao.texto, Dados.valor, banco.transacao, autorizacao.exigirOperacao, planejamento.serialize, Sql.registro, funcionarioAtivo, Sql.inserir, Sql.executar, auditoria.registrar, s.usuarioId.

```

94     public long manutencao(Sessao s,long aeronave,long responsavel,String tipo,String descricao,Instant
        inicio,Instant fim,BigDecimal custo) {
        Início da declaração de manutencao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

95         Dados.exigir(Set.of("PREVENTIVA","CORRETIVA","INSPECAO").contains(tipo) &&
            fim.isAfter(inicio),"Manutencao invalida.");
            Regra obrigatória: Set.of("PREVENTIVA","CORRETIVA","INSPECAO").contains(tipo) && fim.isAfter(inicio) deve resultar em true.
            Se false, Dados.exigir lança RegraNegocioException com "Manutencao invalida.". Essa condição específica não é substituída por
            outra validação parecida.

```



```
96      String texto=Validacao.texto(descricao,"Descricao",2000);BigDecimal
      valor=Dados.valor(custo,false);
```

Declara variável local texto do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local valor do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
97      return banco.transacao(c->{
```

Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
98      autorizacao.exigirOperacao(c,s);planejamento.serializar(c);
```

Instrução idêntica à de OperacaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
99      Sql.registro(c,"SELECT id FROM aeronave WHERE id=? AND situacao<>'INATIVA' FOR
      UPDATE",aeronave);
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeronave. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: aeronave.

```
100     funcionarioAtivo(c,responsavel);
```

Chama OperacaoServico.funcionarioAtivo: Consulta pessoa ativa e vigência de admissão/desligamento. Ausência vira erro por Sql.registro.

```
101     long id=Sql.inserir(c,"INSERT INTO
      manutencao_aeronave(aeronave_id,responsavel_id,tipo,descricao,custo) VALUES (?, ?, ?, ?, ?) RETURNING
      id",aeronave,responsavel,tipo,texto,valor);
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: manutencao\_aeronave. Parâmetros ligados em ordem: aeronave, responsavel, tipo, texto, valor.

```
102     Sql.executar(c,"INSERT INTO agenda_aeronave(aeronave_id,manutencao_id,inicio,fim) VALUES
      (?, ?, ?, ?)",aeronave,id,inicio,fim);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: agenda\_aeronave. Parâmetros ligados em ordem: aeronave, id, inicio, fim.

```
103     auditoria.registrar(c,s.usuarioId(),"AGENDAR_MANUTENCAO","manutencao_aeronave",id);return id;
```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "AGENDAR\_MANUTENCAO", "manutencao\_aeronave", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
104     });
```

```
105     }
```

```
106
```

## situacaoManutencao - L107 a L128

AGENDADA vai a EM\_EXECUCAO/CANCELADA; EM\_EXECUCAO vai a CONCLUIDA. Ao iniciar exige janela e marca aeronave em manutenção. Ao concluir estende agenda até fim real e encerra sem esconder conflito.

Entradas declaradas: (Sessao s,long id,String estado).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: banco.transacao, autorizacao.exigirOperacao, planejamento.serializar, Sql.registro, Dados.texto, Dados.exigir, antes.equals, Set.of("EM\_EXECUCAO","CANCELADA").contains, Set.of, estado.equals, relógio.instant().isBefore, relógio.instant, Dados.instante, Sql.executar, Dados.id, auditoria.registrar, s.usuarioId.

```
107     public void situacaoManutencao(Sessao s,long id,String estado) {
```

Início da declaração de situacaoManutencao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
108     banco.transacao(c->{
```

Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
109     autorizacao.exigirOperacao(c,s);planejamento.serializar(c);
```

Instrução idêntica à de OperacaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
110      var m=Sql.registro(c,"SELECT * FROM manutencao_aeronave WHERE id=? FOR UPDATE",id);String
      antes=Dados.texto(m,"situacao");
```

Declara variável local m com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: manutencao\_aeronave. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

Declara variável local antes do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
111      Dados.exigir((antes.equals("AGENDADA") && Set.of("EM_EXECUCAO","CANCELADA").contains(estado))
      || (antes.equals("EM_EXECUCAO") && estado.equals("CONCLUIDA")),"Transicao de manutencao invalida.");
      Regra obrigatória: (antes.equals("AGENDADA") && Set.of("EM_EXECUCAO","CANCELADA").contains(estado)) ||
      (antes.equals("EM_EXECUCAO") && estado.equals("CONCLUIDA")) deve resultar em true. Se false, Dados.exigir lança
      RegraNegocioException com "Transicao de manutencao invalida.". Essa condição específica não é substituída por outra
      validação parecida.
```

```
112      var a=Sql.registro(c,"SELECT * FROM agenda_aeronave WHERE manutencao_id=?",id);
```

Declara variável local a com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: agenda\_aeronave. Parâmetros ligados em ordem: id.

```
113      if(estado.equals("EM_EXECUCAO")) {
```

Decisão específica: (estado.equals("EM\_EXECUCAO")). Se for true, executa o ramo seguinte; se for false, pula esse ramo e executa o else.

```
114      Dados.exigir(!religio.instant().isBefore(Dados.instante(a,"inicio")) &&
      religio.instant().isBefore(Dados.instante(a,"fim")),"Fora da janela de manutencao.");
```

Regra obrigatória: !religio.instant().isBefore(Dados.instante(a,"inicio")) && religio.instant().isBefore(Dados.instante(a,"fim")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Fora da janela de manutencao.". Essa condição específica não é substituída por outra validação parecida.

```
115      Sql.executar(c,"UPDATE manutencao_aeronave SET inicio_real=?,situacao=? WHERE
      id=?",religio.instant(),estado,id);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: manutencao\_aeronave. Parâmetros ligados em ordem: religio.instant(), estado, id.

```
116      Sql.executar(c,"UPDATE aeronave SET situacao='MANUTENCAO' WHERE
      id=?",Dados.id(m,"aeronave_id"));
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: aeronave. Parâmetros ligados em ordem: Dados.id(m,"aeronave\_id").

```
117      } else {
```

Fecha o bloco anterior e completa/continua a expressão ou abre o próximo bloco indicado nesta linha.

```
118      if(estado.equals("CONCLUIDA")) {
```

Decisão específica: (estado.equals("CONCLUIDA")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
119      Dados.exigir(!religio.instant().isBefore(Dados.instante(m,"inicio_real")), "Fim real
      invalido.");
```

Regra obrigatória: !religio.instant().isBefore(Dados.instante(m,"inicio\_real")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Fim real invalido.". Essa condição específica não é substituída por outra validação parecida.

```
120      Sql.executar(c,"UPDATE agenda_aeronave SET fim=GREATEST(fim,?) WHERE manutencao_id=?
      AND ativa",religio.instant(),id);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: agenda\_aeronave. Parâmetros ligados em ordem: religio.instant(), id.

```
121      }
```

```
122      Sql.executar(c,"UPDATE manutencao_aeronave SET situacao=?,fim_real=? WHERE
      id=?",estado,estado.equals("CONCLUIDA"?religio.instant():null,id);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: manutencao\_aeronave. Parâmetros ligados em ordem: estado, estado.equals("CONCLUIDA"?religio.instant():null, id.

Escolha de valor: estado.equals("CONCLUIDA"). true escolhe religio.instant(); false escolhe null. Só a alternativa escolhida é avaliada.

```
123      Sql.executar(c,"UPDATE agenda_aeronave SET ativa=false WHERE manutencao_id=?",id);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: agenda\_aeronave. Parâmetros ligados em ordem: id.

```
124      Sql.executar(c,"UPDATE aeronave SET situacao='ATIVA' WHERE id=? AND
      situacao='MANUTENCAO'",Dados.id(m,"aeronave_id"));
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: aeronave. Parâmetros ligados em ordem: Dados.id(m,"aeronave\_id").

```

125         }
126         auditoria.registrar(c,s.usuarioId(),"SITUACAO_MANUTENCAO", "manutencao_aeronave", id);return
    null;

    Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "SITUACAO_MANUTENCAO",
    "manutencao_aeronave", id).

    return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o
    contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

127     });
128 }
129

```

## servicoSolo - L130 a L142

Aceita cinco tipos, exige voo ainda no solo e aeroporto do trecho/responsável ativo. Cria serviço com medidas/custo.

Entradas declaradas: (Sessao s,long voo,long aeroporto,long responsavel,String tipo,BigDecimal quantidade,String unidade,BigDecimal custo).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: Dados.exigir, Set.of("ABASTECIMENTO","LIMPEZA","CATERING","BAGAGEM","REBOQUE").contains, Set.of, Dados.valor, Validacao.texto, banco.transacao, autorizacao.exigirOperacao, planejamento.serializar, BloqueiosJdbc.voo, Set.of("EM\_VOO","CONCLUIDO","CANCELADO").contains, v.get, Sql.registro, Dados.id, funcionarioAtivo, Sql.inserir, auditoria.registrar, s.usuarioId.

```

130     public long servicoSolo(Sessao s,long voo,long aeroporto,long responsavel,String tipo,BigDecimal
    quantidade,String unidade,BigDecimal custo) {
        Início da declaração de servicoSolo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

131         Dados.exigir(Set.of("ABASTECIMENTO","LIMPEZA","CATERING","BAGAGEM","REBOQUE").contains(tipo),"Tipo
    de servico invalido.");
        Regra obrigatória: Set.of("ABASTECIMENTO","LIMPEZA","CATERING","BAGAGEM","REBOQUE").contains(tipo) deve resultar
        em true. Se false, Dados.exigir lança RegraNegocioException com "Tipo de servico invalido.". Essa condição específica não é
        substituída por outra validação parecida.

132         BigDecimal valor=Dados.valor(custo,false);BigDecimal medida=Dados.valor(quantidade,false);
        Declara variável local valor do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

        Declara variável local medida do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

133         String un=Validacao.texto(unidade,"Unidade",20);
        Declara variável local un do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

134         return banco.transacao(c->{
        Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
        desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

135         autorizacao.exigirOperacao(c,s);planejamento.serializar(c);var v=BloqueiosJdbc.voo(c,voo);
        Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, s).

        Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é
        o bloco atual.

136         Dados.exigir(!Set.of("EM_VOO","CONCLUIDO","CANCELADO").contains(v.get("situacao")),"Voo nao
    admite servico.");
        Regra obrigatória: !Set.of("EM_VOO","CONCLUIDO","CANCELADO").contains(v.get("situacao")) deve resultar em true. Se false,
        Dados.exigir lança RegraNegocioException com "Voo nao admite servico.". Essa condição específica não é substituída por outra
        validação parecida.

137         var rota=Sql.registro(c,"SELECT origem_id,destino_id FROM rota WHERE
    id=?",Dados.id(v,"rota_id"));
        Declara variável local rota com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo
        é o bloco atual.

        Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
        rota. Parâmetros ligados em ordem: Dados.id(v,"rota_id").

138         Dados.exigir(aeroporto==Dados.id(rota,"origem_id") ||
    aeroporto==Dados.id(rota,"destino_id"),"Servico fora do trecho.");funcionarioAtivo(c,responsavel);
        Regra obrigatória: aeroporto==Dados.id(rota,"origem_id") || aeroporto==Dados.id(rota,"destino_id") deve resultar em true. Se
        false, Dados.exigir lança RegraNegocioException com "Servico fora do trecho.". Essa condição específica não é substituída por
        outra validação parecida.
    
```

```

139      long id=Sql.inserir(c,"INSERT INTO
servico_solo(voo_id,aeroporto_id,responsavel_id,tipo,quantidade,unidade,custo) VALUES (?, ?, ?, ?, ?, ?, ?)
RETURNING id",voo,aeroporto,responsavel,tipo,medida,un,valor);

```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas: servico\_solo. Parâmetros ligados em ordem: voo, aeroporto, responsavel, tipo, medida, un, valor.

```

140      auditoria.registrar(c,s.usuarioId(),"CRIAR_SERVICO_SOLO","servico_solo",id);return id;

```

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "CRIAR\_SERVICO\_SOLO", "servico\_solo", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```

141      });

```

```

142      }

```

```

143

```

## situacaoSolo - L144 a L153

Só transições previstas. Grava início se ainda ausente e fim ao concluir/cancelar; não apaga serviço.

Entradas declaradas: (Sessao s,long id,String estado).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: banco.transacao, autorizacao.exigirOperacao, planejamento.serializar, Sql.registro, BloqueiosJdbc.voo, Dados.id, Dados.texto, Dados.exigir, antes.equals, Set.of("EM\_EXECUCAO","CANCELADO").contains, Set.of, Set.of("CONCLUIDO","CANCELADO").contains, Sql.executar, relgio.instant, auditoria.registrar, s.usuarioId.

```

144      public void situacaoSolo(Sessao s,long id,String estado) {

```

Início da declaração de situacaoSolo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

145      banco.transacao(c->{

```

Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

146      autorizacao.exigirOperacao(c,s);planejamento.serializar(c);

```

Instrução idêntica à de OperacaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

147      var inicial=Sql.registro(c,"SELECT voo_id FROM servico_solo WHERE
id=?",id);BloqueiosJdbc.voo(c,Dados.id(inicial,"voo_id"));

```

Declara variável local inicial com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: servico\_solo. Parâmetros ligados em ordem: id.

```

148      var solo=Sql.registro(c,"SELECT * FROM servico_solo WHERE id=? FOR UPDATE",id);String
antes=Dados.texto(solo,"situacao");

```

Declara variável local solo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: servico\_solo. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

Declara variável local antes do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```

149      Dados.exigir((antes.equals("PENDENTE") && Set.of("EM_EXECUCAO","CANCELADO").contains(estado))
|| (antes.equals("EM_EXECUCAO") && Set.of("CONCLUIDO","CANCELADO").contains(estado)),"Transicao de servico
invalida.");

```

Regra obrigatória: (antes.equals("PENDENTE") && Set.of("EM\_EXECUCAO","CANCELADO").contains(estado)) || (antes.equals("EM\_EXECUCAO") && Set.of("CONCLUIDO","CANCELADO").contains(estado)) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Transicao de servico invalida.". Essa condição específica não é substituída por outra validação parecida.

```

150      Sql.executar(c,"UPDATE servico_solo SET situacao=?,inicio=COALESCE(inicio,?),fim=? WHERE id=?"
,estado,relgio.instant(),Set.of("CONCLUIDO","CANCELADO").contains(estado)?relgio.instant():null,id);

```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: servico\_solo.

Parâmetros ligados em ordem: estado, relgio.instant(), Set.of("CONCLUIDO","CANCELADO").contains(estado)?relgio.instant():null, id.

Escolha de valor: Set.of("CONCLUIDO","CANCELADO").contains(estado). true escolhe relgio.instant(); false escolhe null. Só a alternativa escolhida é avaliada.

```

151      auditoria.registrar(c,s.usuarioId(),"SITUACAO_SERVICO_SOLO","servico_solo",id);return null;
      Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "SITUACAO_SERVICO_SOLO", "servico_solo", id).

      return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

152      });
153  }
154

```

## ocorrencia - L155 a L164

Valida tipo/gravidade, verifica se aeroporto pertence ao trecho quando há voo e retorna id de ocorrência.

Entradas declaradas: (Sessao s,long aeroporto,Long voo,String tipo,String gravidade,String descricao).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: Dados.exigir, Set.of("ATRASO","METEOROLOGIA","SEGURANCA","TECNICA","OUTRA").contains, Set.of, Set.of("BAIXA","MEDIA","ALTA").contains, Validacao.texto, banco.transacao, autorizacao.exigirOperacao, BloqueiosJdbc.voo, Sql.registro, Dados.id, Sql.inserir, s.usuarioId, auditoria.registrar.

```

155      public long ocorrencia(Sessao s,long aeroporto,Long voo,String tipo,String gravidade,String descricao)
      {
      Início da declaração de ocorrencia. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

156      Dados.exigir(Set.of("ATRASO","METEOROLOGIA","SEGURANCA","TECNICA","OUTRA").contains(tipo) &&
      Set.of("BAIXA","MEDIA","ALTA").contains(gravidade),"Ocorrencia invalida.");
      Regra obrigatória: Set.of("ATRASO","METEOROLOGIA","SEGURANCA","TECNICA","OUTRA").contains(tipo) &&
      Set.of("BAIXA","MEDIA","ALTA").contains(gravidade) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException
      com "Ocorrencia invalida.". Essa condição específica não é substituída por outra validação parecida.

157      String texto=Validacao.texto(descricao,"Descricao",2000);
      Declara variável local texto do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

158      return banco.transacao(c->{
      Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
      desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

159      autorizacao.exigirOperacao(c,s);
      Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, s).

160      if(voo!=null) { var v=BloqueiosJdbc.voo(c,voo);var r=Sql.registro(c,"SELECT
      origem_id,destino_id FROM rota WHERE
      id=?",Dados.id(v,"rota_id"));Dados.exigir(aeroporto==Dados.id(r,"origem_id") ||
      aeroporto==Dados.id(r,"destino_id"),"Ocorrencia fora do trecho."); }
      Decisão específica: (voo!=null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

      Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
      rota. Parâmetros ligados em ordem: Dados.id(v,"rota_id").

      Regra obrigatória: aeroporto==Dados.id(r,"origem_id") || aeroporto==Dados.id(r,"destino_id") deve resultar em true. Se false,
      Dados.exigir lança RegraNegocioException com "Ocorrencia fora do trecho.". Essa condição específica não é substituída por
      outra validação parecida.

161      long id=Sql.inserir(c,"INSERT INTO
      ocorrencia_operacional(aeroporto_id,voo_id,usuario_id,tipo,gravidade,descricao) VALUES (?,?,,?,?,,?)
      RETURNING id",aeroporto,voo,s.usuarioId(),tipo,gravidade,texto);
      Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

      Executa inserção com retorno de identificador; o long recebido vem da coluna devolvida pelo SQL. Tabelas mencionadas:
      ocorrencia_operacional. Parâmetros ligados em ordem: aeroporto, voo, s.usuarioId(), tipo, gravidade, texto.

162      auditoria.registrar(c,s.usuarioId(),"ABRIR_OCORRENCIA","ocorrencia_operacional",id);return id;
      Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "ABRIR_OCORRENCIA",
      "ocorrencia_operacional", id).

      Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

163      });

```

```

164     }
165 }

```

## encerrarOcorrencia - L166 a L168

Só encerra registro existente ainda aberto; quantidade afetada precisa ser 1.

Entradas declaradas: (Sessao s,long id).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.operacao (L150).

Chamadas observadas no corpo: banco.transacao, autorizacao.exigirOperacao, Dados.exigir, Sql.executar, relógio.instant, auditoria.registrar, s.usuarioId.

```

166     public void encerrarOcorrencia(Sessao s,long id) {
        Início da declaração de encerrarOcorrencia. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.

```

```

167         banco.transacao(c->{autorizacao.exigirOperacao(c,s);Dados.exigir(Sql.executar(c,"UPDATE
        ocorrencia_operacional SET encerrada_em=? WHERE id=? AND encerrada_em IS
        NULL",relógio.instant(),id)==1,"Ocorrencia ja encerrada ou inexistente.");auditoria.registrar(c,s.usuarioI
        d(),"ENCERRAR_OCORRENCIA","ocorrencia_operacional",id);return null;});

```

Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.

Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, s).

Regra obrigatória: Sql.executar(c,"UPDATE ocorrencia\_operacional SET encerrada\_em=? WHERE id=? AND encerrada\_em IS NULL",relógio.instant(),id)==1 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Ocorrencia ja encerrada ou inexistente.". Essa condição específica não é substituída por outra validação parecida.

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: ocorrencia\_operacional. Parâmetros ligados em ordem: relógio.instant(), id.

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "ENCERRAR\_OCORRENCIA", "ocorrencia\_operacional", id).

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```

168     }
169 }

```

## funcionarioAtivo - L170 a L172

Consulta pessoa ativa e vigência de admissão/desligamento. Ausência vira erro por Sql.registro.

Entradas declaradas: (java.sql.Connection c,long id).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: OperacaoServico.manutencao (L94); OperacaoServico.servicoSolo (L130).

Chamadas observadas no corpo: Sql.registro.

```

170     private void funcionarioAtivo(java.sql.Connection c,long id) throws java.sql.SQLException {
        Início da declaração de funcionarioAtivo. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.

```

```

171         Sql.registro(c,"SELECT f.pessoa_id FROM funcionario f JOIN pessoa p ON p.id=f.pessoa_id WHERE
        f.pessoa_id=? AND p.ativo AND f.admissao<=CURRENT_DATE AND (f.desligamento IS NULL OR
        f.desligamento>CURRENT_DATE)",id);

```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: funcionario, pessoa. Parâmetros ligados em ordem: id.

```

172     }
173 }

```



## PassageiroService

src/main/java/br/edu/aeroporto/servico/PassageiroService.java

Aplica validações, permissões e auditoria antes de usar o contrato de persistência dos passageiros.

88 linhas | 6 métodos/construtores declarados | SHA-256: 12d86eb08ea3b440...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, PassageiroRepositorio. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.CadastroPassageiro;
4 import br.edu.aeroporto.dominio.Passageiro;
5 import br.edu.aeroporto.dominio.Sessao;
6 import br.edu.aeroporto.dominio.Validacao;
7 import br.edu.aeroporto.excecao.RegraNegocioException;
8 import br.edu.aeroporto.infraestrutura.jdbc.AuditoriaJdbc;
9 import br.edu.aeroporto.infraestrutura.jdbc.AutorizacaoJdbc;
10 import br.edu.aeroporto.infraestrutura.jdbc.BancoDados;
11 import br.edu.aeroporto.repositorio.PassageiroRepositorio;
12 import java.time.Clock;
13 import java.time.LocalDate;
14 import java.util.List;
15 import br.edu.aeroporto.infraestrutura.jdbc.Sql;
16 import br.edu.aeroporto.dominio.Dados;
17
18 public final class PassageiroService {
    Declara PassageiroService como tipo com campos e métodos. final impede herança.

19     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

20     private final PassageiroRepositorio repositorio;
    Campo(s) repositorio: private restringe acesso à classe. Tipo PassageiroRepositorio: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

21     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

22     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

23     private final Clock relógio;
    Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

24

```

### PassageiroService - L25 a L32

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco, PassageiroRepositorio repositorio, AuditoriaJdbc auditoria, AutorizacaoJdbc autorizacao, Clock relógio).

Possíveis chamadores identificados: Main.main (L55).

```

25     public PassageiroService(BancoDados banco, PassageiroRepositorio repositorio,
    Início da declaração de PassageiroService. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

26                             AuditoriaJdbc auditoria, AutorizacaoJdbc autorizacao, Clock relógio) {
    Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

27         this.banco = banco;
    Instrução idêntica à de AutenticacaoServico:L18. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

```
28      this.repositorio = repositorio;
```

Guarda no campo repositorio da instância atual o valor de repositorio. O this diferencia o campo de parâmetros com o mesmo nome.

```
29      this.auditoria = auditoria;
```

Instrução idêntica à de AutenticacaoServico:L20. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
30      this.autorizacao = autorizacao;
```

Instrução idêntica à de ConsultaServico:L25. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
31      this.relogio = relogio;
```

Instrução idêntica à de InicializacaoServico:L24. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
32    }
```

```
33
```

## cadastrear - L34 a L42

Valida nascimento e, na mesma transação, exige atendimento, chama repositorio.cadastrar e registra auditoria; retorna Passageiro.

Entradas declaradas: (Sessao sessao, CadastroPassageiro cadastro).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuPrincipal.cadastrarPassageiro (L100).

Chamadas observadas no corpo: Validacao.nascimento, cadastro.nascimento, LocalDate.now, banco.transacao, autorizacao.exigirAtendimento, repositorio.cadastrar, auditoria.registrar, sessao.usuarioId, passageiro.id.

```
34    public Passageiro cadastrar(Sessao sessao, CadastroPassageiro cadastro) {
```

Início da declaração de cadastrar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
35        Validacao.nascimento(cadastro.nascimento(), LocalDate.now(relogio));
```

Chama Validacao.nascimento: Exige data não nula e não futura em relação ao hoje recebido. A função não decide sozinha qual relógio usar.

```
36        return banco.transacao(conexao -> {
```

Retorno: entrega banco.transacao(conexao -> { autorizacao.exigirAtendimento(conexao, sessao); Passageiro passageiro = repositorio.cadastrar(conexao, cadastro); auditoria.registrar(conexao, sessao.usuarioId(), "CADASTRAR\_PASSAGEIRO", "... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.

```
37            autorizacao.exigirAtendimento(conexao, sessao);
```

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(conexao, sessao).

```
38            Passageiro passageiro = repositorio.cadastrar(conexao, cadastro);
```

Declara variável local passageiro do tipo Passageiro. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
39            auditoria.registrar(conexao, sessao.usuarioId(), "CADASTRAR_PASSAGEIRO", "passageiro",
passageiro.id());
```

Registra o evento de auditoria na conexão fornecida: registrar(conexao, sessao.usuarioId(), "CADASTRAR\_PASSAGEIRO", "passageiro", passageiro.id()).

```
40            return passageiro;
```

Retorno: entrega passageiro ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
41        });
```

```
42    }
```

```
43
```

## vincular - L44 a L53

Exige id positivo e normaliza assistência opcional. Vincula o papel à pessoa existente e retorna Passageiro, sem duplicar pessoa.

Entradas declaradas: (Sessao sessao, long pessoald, String assistencia).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); MenuOperacional.passageiros (L44).



Chamadas observadas no corpo: Dados.exigir, assistencia.isBlank, Validacao.texto, banco.transacao, autorizacao.exigirAtendimento, repositorio.vincular, auditoria.registrar, sessao.usuarioId.

```

44     public Passageiro vincular(Sessao sessao,long pessoaId,String assistencia) {
        Início da declaração de vincular. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

45         Dados.exigir(pessoaId>0,"Id invalido.");
        Regra obrigatória: pessoaId>0 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Id invalido.".
        Essa condição específica não é substituída por outra validação parecida.

46         String observacao=assistencia==null ||
        assistencia.isBlank()?null:Validacao.texto(assistencia,"Assistencia",2000);
        Declara variável local observacao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

        Escolha de valor: assistencia==null || assistencia.isBlank(). true escolhe null; false escolhe
        Validacao.texto(assistencia,"Assistencia",2000). Só a alternativa escolhida é avaliada.

47         return banco.transacao(c -> {
        Instrução idêntica à de CadastroServico:L37. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

48             autorizacao.exigirAtendimento(c,sessao);
        Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, sessao).

49             Passageiro passageiro=repositorio.vincular(c,pessoaId,observacao);
        Declara variável local passageiro do tipo Passageiro. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
        bloco atual.

50             auditoria.registrar(c,sessao.usuarioId(),"VINCULAR_PASSAGEIRO","pessoa",pessoaId);
        Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "VINCULAR_PASSAGEIRO", "pessoa",
        pessoaId).

51             return passageiro;
        Instrução idêntica à de PassageiroServico:L40. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
        desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

52         });
53     }
54

```

## editar - L55 a L69

Valida nome/nascimento/assistência, bloqueia pessoa e, ao inativar, impede passagens pendentes/confirmadas. Edita e audita o papel.

Entradas declaradas: (Sessao sessao,long id,String nome,LocalDate nascimento,Long nacionalidade,String assistencia,boolean ativo).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); MenuOperacional.passageiros (L44).

Chamadas observadas no corpo: Validacao.texto, Validacao.nascimento, LocalDate.now, assistencia.isBlank, banco.transacao, autorizacao.exigirAtendimento, Sql.registro, Dados.exigir, Sql.listar(c,"SELECT id FROM item\_reserva WHERE passageiro\_id=? AND situacao IN ('PENDENTE','CONFIRMADO')",r->r.getLong(1),id).isEmpty, Sql.listar, r.getLong, repositorio.editar, auditoria.registrar, sessao.usuarioId.

```

55     public Passageiro editar(Sessao sessao,long id,String nome,LocalDate nascimento,Long
        nacionalidade,String assistencia,boolean ativo) {
        Início da declaração de editar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

56         String nomeValidado=Validacao.texto(nome,"Nome",160);
        Declara variável local nomeValidado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

57         Validacao.nascimento(nascimento,LocalDate.now(relogio));
        Chama Validacao.nascimento: Exige data não nula e não futura em relação ao hoje recebido. A função não decide sozinha qual
        relógio usar.

58         String observacao=assistencia==null ||
        assistencia.isBlank()?null:Validacao.texto(assistencia,"Assistencia",2000);
        Declara variável local observacao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

        Escolha de valor: assistencia==null || assistencia.isBlank(). true escolhe null; false escolhe
        Validacao.texto(assistencia,"Assistencia",2000). Só a alternativa escolhida é avaliada.

```

```

59         return banco.transacao(c -> {
Instrução idêntica à de CadastroServico:L37. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

60             autorizacao.exigirAtendimento(c,sessao);
Instrução idêntica à de PassageiroServico:L48. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

61             Sql.registro(c,"SELECT id FROM pessoa WHERE id=? FOR UPDATE",id);
Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pessoa. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: id.

62             if(!ativo) {
Decisão específica: (!ativo). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

63                 Dados.exigir(Sql.listar(c,"SELECT id FROM item_reserva WHERE passageiro_id=? AND situacao
IN ( 'PENDENTE', 'CONFIRMADO')",r->r.getLong(1),id).isEmpty(),"Passageiro tem passagens ativas.");
Regra obrigatória: Sql.listar(c,"SELECT id FROM item_reserva WHERE passageiro_id=? AND situacao IN
('PENDENTE','CONFIRMADO')",r->r.getLong(1),id).isEmpty() deve resultar em true. Se false, Dados.exigir lança
RegraNegocioException com "Passageiro tem passagens ativas.". Essa condição específica não é substituída por outra validação
parecida.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. Parâmetros
ligados em ordem: id.

64             }
65             Passageiro
passageiro=repositorio.editar(c,id,nomeValidado,nascimento,nacionalidade,observacao,ativo);
Declara variável local passageiro do tipo Passageiro. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
bloco atual.

66             auditoria.registrar(c,sessao.usuarioId(),ativo?"EDITAR_PASSAGEIRO":"INATIVAR_PASSAGEIRO","passageiro",id);
Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(),
ativo?"EDITAR_PASSAGEIRO":"INATIVAR_PASSAGEIRO", "passageiro", id).

Escolha de valor: ativo. true escolhe "EDITAR_PASSAGEIRO"; false escolhe "INATIVAR_PASSAGEIRO". Só a alternativa
escolhida é avaliada.

67             return passageiro;
Instrução idêntica à de PassageiroServico:L40. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

68         });
69     }
70 }

```

## buscar - L71 a L78

Há duas sobrecargas: por long exige id positivo e transforma Optional vazio em erro; por String/página valida filtro e paginação, usa 20 registros e devolve lista.

Entradas declaradas: (Sessao sessao, long id).

Saída declarada: Passageiro - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); MenuPrincipal.buscarPassageiros (L108); MenuPrincipal.executar (L45).

Chamadas observadas no corpo: new RegraNegocioException, banco.consultar(conexao -> { autorizacao.exigirConsulta(conexao, sessao); return repositorio.buscar(conexao, id); }).orElseThrow, banco.consultar, autorizacao.exigirConsulta, repositorio.buscar.

```

71     public Passageiro buscar(Sessao sessao, long id) {
Início da declaração de buscar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
invocado.

72         if (id <= 0) throw new RegraNegocioException("O id deve ser positivo.");
Decisão específica: (id <= 0). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a
operação lançando a exceção.

Lança new RegraNegocioException("O id deve ser positivo."). Interrompe o caminho normal; a exceção sobe até uma captura
compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
banco por si só.

```

```

73         return banco.consultar(conexao -> {
Instrução idêntica à de ConsultaServico:L30. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

74             autorizacao.exigirConsulta(conexao, sessao);
Instrução idêntica à de ConsultaServico:L31. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

75             return repositorio.buscar(conexao, id);
Retorno: entrega repositorio.buscar(conexao, id) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama PassageiroRepositorio.buscar: Há duas sobrecargas: id devolve Optional<Passageiro>; filtro de nome com limite/deslocamento devolve List<Passageiro>. O tipo dos argumentos escolhe qual é chamada.

76         })
Fecha o bloco anterior e completa/continua a expressão ou abre o próximo bloco indicado nesta linha.

77         .orElseThrow(() -> new RegraNegocioException("Passageiro não encontrado."));
new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

78     }
79

```

## buscar - L80 a L87

Há duas sobrecargas: por long exige id positivo e transforma Optional vazio em erro; por String/página valida filtro e paginação, usa 20 registros e devolve lista.

Entradas declaradas: (Sessao sessao, String nome, int pagina).

Saída declarada: List<Passageiro> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); MenuPrincipal.buscarPassageiros (L108); MenuPrincipal.executar (L45).

Chamadas observadas no corpo: new RegraNegocioException, nome.length, banco.consultar, autorizacao.exigirConsulta, repositorio.buscar, nome.strip.

```

80     public List<Passageiro> buscar(Sessao sessao, String nome, int pagina) {
Início da declaração de buscar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

81         if (pagina < 1 || pagina > 100_000) throw new RegraNegocioException("Página fora do intervalo permitido.");
Decisão específica: (pagina < 1 || pagina > 100_000). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException("Página fora do intervalo permitido."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

82         if (nome == null || nome.length() > 160) throw new RegraNegocioException("Filtro de nome inválido.");
Decisão específica: (nome == null || nome.length() > 160). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new RegraNegocioException("Filtro de nome inválido."). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

83         return banco.consultar(conexao -> {
Instrução idêntica à de ConsultaServico:L30. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

```
84         autorizacao.exigirConsulta(conexao, sessao);
```

Instrução idêntica à de ConsultaServico:L31. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
85         return repositorio.buscar(conexao, nome.strip(), 20, (pagina - 1) * 20);
```

Retorno: entrega repositorio.buscar(conexao, nome.strip(), 20, (pagina - 1) \* 20) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama PassageiroRepositorio.buscar: Há duas sobrecargas: id devolve Optional<Passageiro>; filtro de nome com limite/deslocamento devolve List<Passageiro>. O tipo dos argumentos escolhe qual é chamada.

```
86     });
```

```
87 }
```

```
88 }
```

## PlanejamentoServico

src/main/java/br/edu/aeroporto/servico/PlanejamentoServico.java

Coordena programação, tarifas, bloqueios, reprogramação, horários e troca de aeronave com auditoria e atualização das agendas.

110 linhas | 7 métodos/construtores declarados | SHA-256: 70e7bab8623fd085...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, PlanejamentoJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.infraestrutura.jdbc.*;
5 import java.time.Clock;
6 import java.time.Duration;
7 import java.time.Instant;
8
9 public final class PlanejamentoServico {
    Declara PlanejamentoServico como tipo com campos e métodos. final impede herança.

10     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

11     private final PlanejamentoJdbc repositorio;
    Campo(s) repositorio: private restringe acesso à classe. Tipo PlanejamentoJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

12     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

13     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

14     private final Clock relógio;
    Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

15

```

### PlanejamentoServico - L16 a L18

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco, PlanejamentoJdbc repositorio, AutorizacaoJdbc autorizacao, AuditoriaJdbc auditoria, Clock relógio).

Possíveis chamadores identificados: Main.main (L55).

```

16     public PlanejamentoServico(BancoDados banco, PlanejamentoJdbc repositorio, AutorizacaoJdbc
    autorizacao, AuditoriaJdbc auditoria, Clock relógio) {
    Início da declaração de PlanejamentoServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

17         this.banco=banco; this.repositorio=repositorio; this.autorizacao=autorizacao;
        this.auditoria=auditoria; this.relógio=relógio;
    Instrução idêntica à de CadastroServico:L22. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

18     }
19

```

### programar - L20 a L28

Exige partida futura, permissão de operação e serialização; define autor de histórico, chama JDBC para voo/agenda/inventário e retorna id.

Entradas declaradas: (Sessao sessao, ProgramacaoVoo voo).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: Dados.exigir, voo.partida().isAfter, voo.partida, relógio.instant, banco.transacao, autorizacao.exigirOperacao, repositorio.serializar, auditoria.definirAutorDoVoo, sessao.usuarioId, repositorio.programar, auditoria.registrar.

```

20    public long programar(Sessao sessao, ProgramacaoVoo voo) {
        Início da declaração de programar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

21        Dados.exigir(voo.partida().isAfter(relógio.instant()), "Programe uma partida futura.");
        Regra obrigatória: voo.partida().isAfter(relógio.instant()) deve resultar em true. Se false, Dados.exigir lança
        RegraNegocioException com "Programe uma partida futura.". Essa condição específica não é substituída por outra validação
        parecida.

22        return banco.transacao(c->{
            Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
            desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

23            autorizacao.exigirOperacao(c, sessao); repositorio.serializar(c);
            Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, sessao).

24            auditoria.definirAutorDoVoo(c, sessao.usuarioId(), "Programacao");
            Registra o evento de auditoria na conexão fornecida: definirAutorDoVoo(c, sessao.usuarioId(), "Programacao").

25            long id=repositorio.programar(c, voo);
            Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

26            auditoria.registrar(c, sessao.usuarioId(), "PROGRAMAR_VOO", "voo", id); return id;
            Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "PROGRAMAR_VOO", "voo", id).

            Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

27        });
28    }
29
```

## tarifa - L30 a L38

Bloqueia voo e só permite mudança comercial em PROGRAMADO/CHECKIN\_ABERTO; cria/edita contrato, audita e retorna id.

Entradas declaradas: (Sessao sessao, long voo, Long id, Tarifa tarifa, boolean ativa).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: banco.transacao, autorizacao.exigirOperacao, BloqueiosJdbc.voo, Dados.exigir, "PROGRAMADO".equals, v.get, "CHECKIN\_ABERTO".equals, repositorio.tarifa, auditoria.registrar, sessao.usuarioId.

```

30    public long tarifa(Sessao sessao, long voo, Long id, Tarifa tarifa, boolean ativa) {
        Início da declaração de tarifa. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

31        return banco.transacao(c->{
            Instrução idêntica à de AdministracaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
            desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

32            autorizacao.exigirOperacao(c, sessao);
            Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, sessao).

33            var v=BloqueiosJdbc.voo(c, voo);
            Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é
            o bloco atual.

34            Dados.exigir("PROGRAMADO".equals(v.get("situacao")) ||
            "CHECKIN_ABERTO".equals(v.get("situacao")), "Voo nao admite alteracao comercial.");
            Regra obrigatória: "PROGRAMADO".equals(v.get("situacao")) || "CHECKIN_ABERTO".equals(v.get("situacao")) deve resultar em
            true. Se false, Dados.exigir lança RegraNegocioException com "Voo nao admite alteracao comercial.". Essa condição específica
            não é substituída por outra validação parecida.

35            long salvo=repositorio.tarifa(c, voo, id, tarifa, ativa);
            Declara variável local salvo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
```

```

36      auditoria.registrar(c,sessao.usuarioId(),"SALVAR_TARIFA","tarifa_voo",salvo); return salvo;
Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "SALVAR_TARIFA", "tarifa_voo", salvo).

Retorno: entrega salvo ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
caminho.

37      });
38  }
39

```

## bloquearAssento - L40 a L50

Obtém o voo antes de bloquear o inventário e rejeita assento com ocupação ativa. Atualiza flag de bloqueio e audita.

Entradas declaradas: (Sessao sessao, long inventario, boolean bloqueado).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: banco.transacao, autorizacao.exigirOperacao, Sql.registro, BloqueiosJdbc.voo, Dados.id, Dados.exigir, Sql.listar(c, "SELECT id FROM ocupacao\_assento WHERE inventario\_id=? AND liberada\_em IS NULL", r->r.getLong(1), inventario).isEmpty, Sql.listar, r.getLong, Sql.executar, auditoria.registrar, sessao.usuarioId.

```

40  public void bloquearAssento(Sessao sessao, long inventario, boolean bloqueado) {
Início da declaração de bloquearAssento. Os parâmetros entram pela chamada; o corpo só executa quando esse
método/construtor é invocado.

41      banco.transacao(c->{
Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A
posição e o contexto do método acima mostram quando ela ocorre.

42      autorizacao.exigirOperacao(c, sessao);
Instrução idêntica à de PlanejamentoServico:L32. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

43      var inicial=Sql.registro(c, "SELECT voo_id FROM inventario_assento_voo WHERE id=?", inventario);
Declara variável local inicial com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
escopo é o bloco atual.

Consulta uma linha como Map<String, Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
inventario_assento_voo. Parâmetros ligados em ordem: inventario.

44      BloqueiosJdbc.voo(c, Dados.id(inicial, "voo_id"));
Chama BloqueiosJdbc.voo: SELECT FOR UPDATE devolve Map e mantém lock de linha até fim da transação.

45      Sql.registro(c, "SELECT id FROM inventario_assento_voo WHERE id=? FOR UPDATE", inventario);
Consulta uma linha como Map<String, Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
inventario_assento_voo. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: inventario.

46      Dados.exigir(Sql.listar(c, "SELECT id FROM ocupacao_assento WHERE inventario_id=? AND
liberada_em IS NULL", r->r.getLong(1), inventario).isEmpty(), "Assento ocupado.");
Regra obrigatória: Sql.listar(c, "SELECT id FROM ocupacao_assento WHERE inventario_id=? AND liberada_em IS
NULL", r->r.getLong(1), inventario).isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com
"Assento ocupado.". Essa condição específica não é substituída por outra validação parecida.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: ocupacao_assento. Parâmetros
ligados em ordem: inventario.

47      Sql.executar(c, "UPDATE inventario_assento_voo SET bloqueado=? WHERE
id=?", bloqueado, inventario);
Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas:
inventario_assento_voo. Parâmetros ligados em ordem: bloqueado, inventario.

48      auditoria.registrar(c, sessao.usuarioId(), "BLOQUEAR_ASSENTO", "inventario_assento_voo", inventario); return
null;
Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "BLOQUEAR_ASSENTO",
"inventario_assento_voo", inventario).

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o
contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

49      });
50  }
51

```



## reprogramar - L52 a L69

Só voo PROGRAMADO; preserva companhia/rota/aeronave/número e muda horários. Valida frota, apaga previsões estimadas, desloca agendas e revalida escalas.

Entradas declaradas: (Sessao sessao,long id,ProgramacaoVoo nova,String motivo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: Validacao.texto, banco.transacao, autorizacao.exigirOperacao, repositorio.serializar, BloqueiosJdbc.voo, Dados.exigir, "PROGRAMADO".equals, anterior.get, Dados.id, nova.companhia, nova.rota, nova.aeronave, anterior.get("numero").equals, nova.numero, nova.partida().isAfter, nova.partida, relógio.instant, repositorio.validarFrota, nova.chegada, auditoria.definirAutorDoVoo, sessao.usuarioId, Sql.executar, nova.checkinAbre, nova.checkinFecha, nova.embarqueAbre, nova.embarqueFecha, repositorio.agendas, OperacaoJdbc.validarEscalasDoVoo, auditoria.registrar.

```
52 public void reprogramar(Sessao sessao,long id,ProgramacaoVoo nova,String motivo) {
    Início da declaração de reprogramar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

53     String razao=Validacao.texto(motivo,"Motivo",500);
    Declara variável local razao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

54     banco.transacao(c->{
    Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A
    posição e o contexto do método acima mostram quando ela ocorre.

55         autorizacao.exigirOperacao(c,sessao);repositorio.serializar(c);
    Confere no banco o acesso vigente do usuário para esta operação: exigirOperacao(c, sessao).

56         var anterior=BloqueiosJdbc.voo(c,id);
    Declara variável local anterior com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.

57         Dados.exigir("PROGRAMADO".equals(anterior.get("situacao")), "Reprogramacao somente antes de
    abrir check-in.");
    Regra obrigatória: "PROGRAMADO".equals(anterior.get("situacao")) deve resultar em true. Se false, Dados.exigir lança
    RegraNegocioException com "Reprogramacao somente antes de abrir check-in.". Essa condição específica não é substituída por
    outra validação parecida.

58         Dados.exigir(Dados.id(anterior,"companhia_id")==nova.companhia() &&
    Dados.id(anterior,"rota_id")==nova.rota() && Dados.id(anterior,"aeronave_id")==nova.aeronave() &&
    anterior.get("numero").equals(nova.numero()), "Reprogramacao altera horarios; use a troca de aeronave
    separadamente.");
    Regra obrigatória: Dados.id(anterior,"companhia_id")==nova.companhia() && Dados.id(anterior,"rota_id")==nova.rota() &&
    Dados.id(anterior,"aeronave_id")==nova.aeronave() && anterior.get("numero").equals(nova.numero()) deve resultar em true. Se
    false, Dados.exigir lança RegraNegocioException com "Reprogramacao altera horarios; use a troca de aeronave
    separadamente.". Essa condição específica não é substituída por outra validação parecida.

59         Dados.exigir(nova.partida().isAfter(relógio.instant()), "Partida deve ser futura.");
    Regra obrigatória: nova.partida().isAfter(relógio.instant()) deve resultar em true. Se false, Dados.exigir lança
    RegraNegocioException com "Partida deve ser futura.". Essa condição específica não é substituída por outra validação parecida.

60     repositorio.validarFrota(c,nova.aeronave(),nova.companhia(),nova.rota(),nova.partida(),nova.chegada(),id);
    Chama PlanejamentoJdbc.validarFrota: Exige aeronave/companhia/modelo/rota/localidades ativos. Verifica voo anterior e
    próximo, continuidade de aeroporto e 75 min; ignorar exclui o próprio voo ao editar.

61     auditoria.definirAutorDoVoo(c,sessao.usuarioId(),razao);
    Registra o evento de auditoria na conexão fornecida: definirAutorDoVoo(c, sessao.usuarioId(), razao).

62     Sql.executar(c, ""
    Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: voo. Parâmetros
    ligados em ordem: nova.partida(), nova.chegada(), nova.checkinAbre(), nova.checkinFecha(), nova.embarqueAbre(),
    nova.embarqueFecha(), id.

63     UPDATE voo SET partida_prevista=?, chegada_prevista=?, checkin_abre=?, checkin_fecha=?, em
    barque_abre=?, embarque_fecha=?, partida_estimada=NULL, chegada_estimada=NULL WHERE id=?
    Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é
    chamado, não pelo compilador Java.

64     """,nova.partida(),nova.chegada(),nova.checkinAbre(),nova.checkinFecha(),nova.embarque
    Abre(),nova.embarqueFecha(),id);
    Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu
    uso.
```



```

65      repositorio.agendas(c,anterior,nova.partida(),nova.chegada());
Chama PlanejamentoJdbc.agendas: Calcula deltas de partida/chegada e desloca agenda de aeronave, recursos, escalas e
serviços pendentes; CHEGADA usa delta de chegada.

66      OperacaoJdbc.validarEscalasDoVoo(c,id);
Chama OperacaoJdbc.validarEscalasDoVoo: Percorre escalas ativas, revalidando cada uma com seu próprio id em ignorar para
não conflitar consigo mesma.

67      auditoria.registrar(c,sessao.usuarioId(),"REPROGRAMAR_VOO","voo",id);return null;
Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "REPROGRAMAR_VOO", "voo", id).

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o
contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

68      });
69      }
70

```

## horarios - L71 a L91

Distingue horário real (EM\_VOO e instantes já ocorridos) de estimado (antes de EM\_VOO). Verifica frota, grava horários, atualiza agendas/escalas e audita.

Entradas declaradas: (Sessao sessao,long id,Instant partida,Instant chegada,boolean reais,String motivo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: Dados.exigir, chegada.isAfter, Validacao.texto, banco.transacao, autorizacao.exigirOperacao, repositorio.serializar, BloqueiosJdbc.voo, Dados.texto, estado.equals, partida.isAfter, relógio.instant, repositorio.validarFrota, Dados.id, auditoria.definirAutorDoVoo, sessao.usuarioId, Sql.executar, Duration.between, PlanejamentoJdbc.efetivo, repositorio.agendas, OperacaoJdbc.validarEscalasDoVoo, auditoria.registrar.

```

71      public void horarios(Sessao sessao,long id,Instant partida,Instant chegada,boolean reais,String
motivo) {
Início da declaração de horarios. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
invocado.

72      Dados.exigir(partida!=null && chegada!=null && chegada.isAfter(partida),"Horarios invalidos.");
Regra obrigatória: partida!=null && chegada!=null && chegada.isAfter(partida) deve resultar em true. Se false, Dados.exigir lança
RegraNegocioException com "Horarios invalidos.". Essa condição específica não é substituída por outra validação parecida.

73      String razao=Validacao.texto(motivo,"Motivo",500);
Instrução idêntica à de PlanejamentoServico:L53. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

74      banco.transacao(c->{
Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A
posição e o contexto do método acima mostram quando ela ocorre.

75      autorizacao.exigirOperacao(c,sessao);repositorio.serializar(c);
Instrução idêntica à de PlanejamentoServico:L55. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

76      var v=BloqueiosJdbc.voo(c,id);String estado=Dados.texto(v,"situacao");
Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é
o bloco atual.

Declara variável local estado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

77      Dados.exigir(!estado.equals("CONCLUIDO") && !estado.equals("CANCELADO"),"Voo encerrado.");
Regra obrigatória: !estado.equals("CONCLUIDO") && !estado.equals("CANCELADO") deve resultar em true. Se false,
Dados.exigir lança RegraNegocioException com "Voo encerrado.". Essa condição específica não é substituída por outra validação
parecida.

78      if(reais) Dados.exigir(estado.equals("EM_VOO") && !partida.isAfter(relógio.instant()) &&
!chegada.isAfter(relógio.instant()),"Horarios reais requerem voo em voo e instantes já ocorridos.");
Decisão específica: (reais). Se for true, executa o ramo seguinte; se for false, pula esse ramo e executa o else.

Regra obrigatória: estado.equals("EM_VOO") && !partida.isAfter(relógio.instant()) && !chegada.isAfter(relógio.instant()) deve
resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Horarios reais requerem voo em voo e instantes já
ocorridos.". Essa condição específica não é substituída por outra validação parecida.

```

```

79         else Dados.exigir(!estado.equals("EM_VOO"), "Voo em voo requer registro real.");
Regra obrigatória: !estado.equals("EM_VOO") deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com
"Voo em voo requer registro real.". Essa condição específica não é substituída por outra validação parecida.

80         repositorio.validarFrota(c,Dados.id(v,"aeronave_id"),Dados.id(v,"companhia_id"),Dados.id(v,"ro
ta_id"),partida,chegada,id);
Chama PlanejamentoJdbc.validarFrota: Exige aeronave/companhia/modelo/rota/localidades ativos. Verifica voo anterior e
próximo, continuidade de aeroporto e 75 min; ignorar exclui o próprio voo ao editar.

81         auditoria.definirAutorDoVoo(c,sessao.usuarioId(),razao);
Instrução idêntica à de PlanejamentoServico:L61. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

82         if(reais) Sql.executar(c,"UPDATE voo SET partida_real=?,chegada_real=?,motivo_atraso=? WHERE
id=?",partida,chegada,razao,id);
Decisão específica: (reais). Se for true, executa o ramo seguinte; se for false, pula esse ramo e executa o else.

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: voo. Parâmetros
ligados em ordem: partida, chegada, razao, id.

83         else {
Continua a decisão anterior pelo caminho em que a condição não foi atendida, ou abre um novo teste do else if.

84         Duration delta=Duration.between(PlanejamentoJdbc.efetivo(v,"partida"),partida);
Declara variável local delta do tipo Duration. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

85         Sql.executar(c,"UPDATE voo SET partida_estimada=?,chegada_estimada=?,motivo_atraso=? WHERE
id=?",partida,chegada,razao,id);
Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: voo. Parâmetros
ligados em ordem: partida, chegada, razao, id.

86         // O DDL ancora as janelas na partida prevista; em atraso a janela efetiva e calculada
pelo atendimento.
Comentário/documentação do autor; descreve intenção ou convenção e não executa.

87         }
88         repositorio.agendas(c,v,partida,chegada);OperacaoJdbc.validarEscalasDoVoo(c,id);
Chama PlanejamentoJdbc.agendas: Calcula deltas de partida/chegada e desloca agenda de aeronave, recursos, escalas e
serviços pendentes; CHEGADA usa delta de chegada.

89         auditoria.registrar(c,sessao.usuarioId(),reais?"HORARIOS_REAIS":"HORARIOS_ESTIMADOS","voo",id);return
null;
Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(),
reais?"HORARIOS_REAIS":"HORARIOS_ESTIMADOS", "voo", id).

Escolha de valor: reais. true escolhe "HORARIOS_REAIS"; false escolhe "HORARIOS_ESTIMADOS". Só a alternativa escolhida
é avaliada.

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o
contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

90     });
91     }
92

```

## trocarAeronave - L93 a L109

Só PROGRAMADO, sem qualquer item histórico e sem recursos/equipe ativos. Valida nova frota; remove inventário/agenda não históricos e recria para a nova aeronave.

Entradas declaradas: (Sessao sessao,long voo,long aeronave,String motivo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.comercial (L109); MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: Validacao.texto, banco.transacao, autorizacao.exigirOperacao, repositorio.serializar, BloqueiosJdbc.voo, Dados.exigir, "PROGRAMADO".equals, v.get, Sql.listar(c,"SELECT id FROM item\_reserva WHERE voo\_id=?",r->r.getLong(1),voo).isEmpty, Sql.listar, r.getLong, Sql.listar(c,"SELECT id FROM alocao\_recurso WHERE voo\_id=? AND ativa UNION ALL SELECT id FROM escala\_funcionario WHERE voo\_id=? AND ativa",r->r.getLong(1),voo,voo).isEmpty, repositorio.validarFrota, Dados.id, PlanejamentoJdbc.efetivo, auditoria.definirAutorDoVoo, sessao.usuarioId, Sql.executar, PlanejamentoJdbc.efetivo(v,"partida").minus, Duration.ofMinutes, PlanejamentoJdbc.efetivo(v,"chegada").plus, repositorio.inventario, auditoria.registrar.

```

93     public void trocarAeronave(Sessao sessao,long voo,long aeronave,String motivo) {
Início da declaração de trocarAeronave. Os parâmetros entram pela chamada; o corpo só executa quando esse
método/construtor é invocado.

```

|     |                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 94  | <code>String razao=Validacao.texto(motivo,"Motivo",500);</code>                                                                                                                                                                                                      | Instrução idêntica à de PlanejamentoServico:L53. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                                                                                    |
| 95  | <code>banco.transacao(c-&gt;{</code>                                                                                                                                                                                                                                 | Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                                                                                                   |
| 96  | <code>autorizacao.exigirOperacao(c,sessao);repositorio.serializar(c);</code>                                                                                                                                                                                         | Instrução idêntica à de PlanejamentoServico:L55. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                                                                                    |
| 97  | <code>var v=BloqueiosJdbc.voo(c,voo);</code>                                                                                                                                                                                                                         | Instrução idêntica à de PlanejamentoServico:L33. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                                                                                    |
| 98  | <code>Dados.exigir("PROGRAMADO".equals(v.get("situacao")), "Troca somente em voo programado.");</code>                                                                                                                                                               | Regra obrigatória: "PROGRAMADO".equals(v.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Troca somente em voo programado.". Essa condição específica não é substituída por outra validação parecida.                                                                                                                                                                                                                                                                                                                                             |
| 99  | <code>Dados.exigir(Sql.listar(c,"SELECT id FROM item_reserva WHERE voo_id=?",r-&gt;r.getLong(1),voo).isEmpty(), "Troca recusada: existem itens historicos.");</code>                                                                                                 | Regra obrigatória: Sql.listar(c,"SELECT id FROM item_reserva WHERE voo_id=?",r->r.getLong(1),voo).isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Troca recusada: existem itens historicos.". Essa condição específica não é substituída por outra validação parecida.<br><br>Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. Parâmetros ligados em ordem: voo.                                                                                                                            |
| 100 | <code>Dados.exigir(Sql.listar(c,"SELECT id FROM alocao_recurso WHERE voo_id=? AND ativa UNION ALL SELECT id FROM escala_funcionario WHERE voo_id=? AND ativa",r-&gt;r.getLong(1),voo,voo).isEmpty(), "Libere recursos e equipe antes de trocar a aeronave.");</code> | Regra obrigatória: Sql.listar(c,"SELECT id FROM alocao_recurso WHERE voo_id=? AND ativa UNION ALL SELECT id FROM escala_funcionario WHERE voo_id=? AND ativa",r->r.getLong(1),voo,voo).isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Libere recursos e equipe antes de trocar a aeronave.". Essa condição específica não é substituída por outra validação parecida.<br><br>Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: alocao_recurso, escala_funcionario. Parâmetros ligados em ordem: voo, voo. |
| 101 | <code>repositorio.validarFrota(c,aeronave,Dados.id(v,"companhia_id"),Dados.id(v,"rota_id"),PlanejamentoJdbc.efetivo(v,"partida"),PlanejamentoJdbc.efetivo(v,"chegada"),voo);</code>                                                                                  | Chama PlanejamentoJdbc.validarFrota: Exige aeronave/companhia/modelo/rota/localidades ativos. Verifica voo anterior e próximo, continuidade de aeroporto e 75 min; ignorar exclui o próprio voo ao editar.                                                                                                                                                                                                                                                                                                                                                                                       |
| 102 | <code>auditoria.definirAutorDoVoo(c,sessao.usuarioId(),razao);</code>                                                                                                                                                                                                | Instrução idêntica à de PlanejamentoServico:L61. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                                                                                    |
| 103 | <code>Sql.executar(c,"DELETE FROM inventario_assento_voo WHERE voo_id=?",voo);</code>                                                                                                                                                                                | Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: inventario_assento_voo. Parâmetros ligados em ordem: voo.                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| 104 | <code>Sql.executar(c,"DELETE FROM agenda_aeronave WHERE voo_id=?",voo);</code>                                                                                                                                                                                       | Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: agenda_aeronave. Parâmetros ligados em ordem: voo.                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 105 | <code>Sql.executar(c,"UPDATE voo SET aeronave_id=? WHERE id=?",aeronave,voo);</code>                                                                                                                                                                                 | Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: voo. Parâmetros ligados em ordem: aeronave, voo.                                                                                                                                                                                                                                                                                                                                                                                                                                            |

```
106      Sql.executar(c,"INSERT INTO agenda_aeronave(aeronave_id,voo_id,inicio,fim) VALUES (?, ?, ?, ?)",a
    aeronave,voo,PlanejamentoJdbc.efetivo(v,"partida").minus(Duration.ofMinutes(45)),PlanejamentoJdbc.efetivo(v
    ,"chegada").plus(Duration.ofMinutes(30)));
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: agenda\_aeronave. Parâmetros ligados em ordem: aeronave, voo, PlanejamentoJdbc.efetivo(v,"partida").minus(Duration.ofMinutes(45)), PlanejamentoJdbc.efetivo(v,"chegada").plus(Duration.ofMinutes(30)).

```
107      repositorio.inventario(c,voo,aeronave);auditoria.registrar(c,sessao.usuarioId(),"TROCAR_AERONA
    VE","voo",voo);return null;
```

Chama PlanejamentoJdbc.inventario: INSERT SELECT copia assentos físicos ativos para o voo; exige ao menos uma linha afetada.

Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "TROCAR\_AERONAVE", "voo", voo).

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```
108      });
109    }
110  }
```

## RelatoriosCompletosServico

src/main/java/br/edu/aeroporto/servico/RelatoriosCompletosServico.java

Consulta relatórios paginados, painéis, mapa, rastreo e estatísticas; exporta todos os registros do relatório por streaming.

104 linhas | 9 métodos/construtores declarados | SHA-256: 3f3026aa89c28df3...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.dto.TipoRelatorio;
    Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Enum com SQL fixo para 16 relatórios e um indicador de acesso restrito. Não é uma entidade do banco; funciona como catálogo de consultas.

5 import br.edu.aeroporto.infraestrutura.jdbc.*;
6 import br.edu.aeroporto.execcao.PersistenciaException;
7 import java.io.*;
8 import java.nio.charset.StandardCharsets;
9 import java.nio.file.*;
10 import java.sql.*;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

11 import java.time.*;
12 import java.util.*;
13
14 public final class RelatoriosCompletosServico {
    Declara RelatoriosCompletosServico como tipo com campos e métodos. final impede herança.

15     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

16     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

17     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

18

```

### RelatoriosCompletosServico - L19 a L21

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria).

Possíveis chamadores identificados: Main.main (L55).

```

19     public RelatoriosCompletosServico(BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc
    auditoria) {
    Início da declaração de RelatoriosCompletosServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

20         this.banco=banco;this.autorizacao=autorizacao;this.auditoria=auditoria;
    Guarda no campo banco da instância atual o valor de banco. O this diferencia o campo de parâmetros com o mesmo nome.

    Guarda no campo autorizacao da instância atual o valor de autorizacao. O this diferencia o campo de parâmetros com o mesmo nome.

    Guarda no campo auditoria da instância atual o valor de auditoria. O this diferencia o campo de parâmetros com o mesmo nome.

21     }
22

```

## permitir - L23 a L27

AUDITORIA exige administrador; outros restritos exigem atendimento; demais exigem consulta.

Entradas declaradas: (Connection c,Sessao s,TipoRelatorio tipo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: RelatoriosCompletosServico.consultar (L29); RelatoriosCompletosServico.exportar (L68).

Chamadas observadas no corpo: autorizacao.exigirAdministrador, tipo.restrito, autorizacao.exigirAtendimento, autorizacao.exigirConsulta.

```

23     private void permitir(Connection c,Sessao s,TipoRelatorio tipo) throws SQLException {
        Início da declaração de permitir. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

24         if(tipo==TipoRelatorio.AUDITORIA) autorizacao.exigirAdministrador(c,s);
        Decisão específica: (tipo==TipoRelatorio.AUDITORIA). Se for true, executa o ramo seguinte; se for false, pula esse ramo e
        executa o else.

        Confere no banco o acesso vigente do usuário para esta operação: exigirAdministrador(c, s).

25         else if(tipo.restrito()) autorizacao.exigirAtendimento(c,s);
        Decisão específica: (tipo.restrito()). Se for true, executa o ramo seguinte; se for false, pula esse ramo e executa o else.

        Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, s).

26         else autorizacao.exigirConsulta(c,s);
        Confere no banco o acesso vigente do usuário para esta operação: exigirConsulta(c, s).

27     }
28 
```

## consultar - L29 a L32

Envolve SQL fixo do enum em subconsulta e aplica LIMIT 20/OFFSET da página, preservando permissão.

Entradas declaradas: (Sessao s,TipoRelatorio tipo,int pagina).

Saída declarada: List<Map<String,Object>> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); IntegracaoTeste.cadastros (L62); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.icao (L503); IntegracaoTeste.numero (L515); IntegracaoTeste.regrasComplementares (L369); IntegracaoTeste.texto (L500); MenuOperacional.relatorios (L167).

Chamadas observadas no corpo: Dados.exigir, banco.consultar, permitir, Sql.listar, tipo.sql().strip, tipo.sql, Sql::linha.

```

29     public List<Map<String,Object>> consultar(Sessao s,TipoRelatorio tipo,int pagina) {
        Início da declaração de consultar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

30         Dados.exigir(pagina>=1 && pagina<=100000,"Pagina invalida.");
        Regra obrigatória: pagina>=1 && pagina<=100000 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException
        com "Pagina invalida.". Essa condição específica não é substituída por outra validação parecida.

31         return banco.consultar(c->{permitir(c,s,tipo);return Sql.listar(c,"SELECT * FROM
        ("+tipo.sql().strip()+") relatorio LIMIT 20 OFFSET ?",Sql::linha,(pagina-1)*20);});
        Retorno: entrega banco.consultar(c->{permitir(c,s,tipo);return Sql.listar(c,"SELECT * FROM ("+tipo.sql().strip()+") relatorio LIMIT
        20 OFFSET ?",Sql::linha,(pagina-1)*20);}) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo
        executa por esse caminho.

        Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da
        consulta.

        Retorno: entrega Sql.listar(c,"SELECT * FROM ("+tipo.sql().strip()+") relatorio LIMIT 20 OFFSET ?",Sql::linha,(pagina-1)*20) ao
        chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

        Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Parâmetros ligados em ordem: (pagina-1)*20.

32     }
33 
```

## painel - L34 a L42

Obtém fuso do aeroporto e calcula intervalo local [início do dia, início do próximo). Escolhe campo/lado conforme partidas ou chegadas.

Entradas declaradas: (Sessao s,long aeroporto,LocalDate data,boolean partidas).

Saída declarada: List<Map<String,Object>> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.relatorios (L167).

Chamadas observadas no corpo: banco.consultar, autorizacao.exigirConsulta, Sql.registro, ZoneId.of, a.get("fuso\_horario").toString, a.get, data.atStartOfDay(fuso).toInstant, data.atStartOfDay, data.plusDays(1).atStartOfDay(fuso).toInstant, data.plusDays(1).atStartOfDay, data.plusDays, Sql.listar, Sql::linha.

|    |                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 34 | <code>public List&lt;Map&lt;String,Object&gt;&gt; painel(Sessao s,long aeroporto,LocalDate data,boolean partidas) {</code>                                                                                                                                                                                                                                | Início da declaração de painel. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                                                                                                                                                                                                                                                                                                    |
| 35 | <code>return banco.consultar(c-&gt;{</code>                                                                                                                                                                                                                                                                                                               | Instrução idêntica à de AdministracaoServico:L22. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                                                                                                                                                                                                                                     |
| 36 | <code>autorizacao.exigirConsulta(c,s);</code>                                                                                                                                                                                                                                                                                                             | Confere no banco o acesso vigente do usuário para esta operação: exigirConsulta(c, s).                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 37 | <code>var a=Sql.registro(c,"SELECT fuso_horario FROM aeroporto WHERE id=?",aeroporto);ZoneId<br/>fuso=ZoneId.of(a.get("fuso_horario").toString());</code>                                                                                                                                                                                                 | Declara variável local a com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeroporto. Parâmetros ligados em ordem: aeroporto.<br><br>Declara variável local fuso do tipo ZoneId. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                       |
| 38 | <code>Instant<br/>inicio=data.atStartOfDay(fuso).toInstant(),fim=data.plusDays(1).atStartOfDay(fuso).toInstant();</code>                                                                                                                                                                                                                                  | Declara variável local inicio, fim do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                                                                                                           |
| 39 | <code>String<br/>campo=partidas?"partida_prevista":"chegada_prevista",lado=partidas?"origem_id":"destino_id";</code>                                                                                                                                                                                                                                      | Declara variável local campo, lado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Escolha de valor: partidas. true escolhe "partida_prevista"; false escolhe "chegada_prevista". Só a alternativa escolhida é avaliada.<br><br>Escolha de valor: partidas. true escolhe "origem_id"; false escolhe "destino_id". Só a alternativa escolhida é avaliada.                                                                                                               |
| 40 | <code>return Sql.listar(c,"SELECT v.*,o.codigo_icao AS origem,d.codigo_icao AS destino FROM voo v<br/>JOIN rota r ON r.id=v.rota_id JOIN aeroporto o ON o.id=r.origem_id JOIN aeroporto d ON d.id=r.destino_id<br/>WHERE r."+lado+"=? AND v."+campo+"&gt;=? AND v."+campo+"&lt;=? ORDER BY<br/>v."+campo+",v.id",Sql::linha,aeroporto,inicio,fim);</code> | Retorno: entrega Sql.listar(c,"SELECT v.*,o.codigo_icao AS origem,d.codigo_icao AS destino FROM voo v JOIN rota r ON r.id=v.rota_id JOIN aeroporto o ON o.id=r.origem_id JOIN aeroporto d ON d.id=r.destino_id WHERE r."+lado+"=? AND v."+... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.<br><br>Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: voo, rota, aeroporto. Parâmetros ligados em ordem: aeroporto, inicio, fim. |
| 41 | <code>});</code>                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 42 | <code>}</code>                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| 43 |                                                                                                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

## mapa - L44 a L49

Consulta filas/colunas e ocupação ativa do inventário, depois constrói MapaAssentos.

Entradas declaradas: (Sessao s,long voo).

Saída declarada: MapaAssentos - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.cadastros (L62); MenuOperacional.relatorios (L167).

Chamadas observadas no corpo: banco.consultar, autorizacao.exigirConsulta, new MapaAssentos, Sql.listar, Sql::linha.

|    |                                                                                                                         |                                                                                                                                                                                                                                               |
|----|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 44 | <code>public MapaAssentos mapa(Sessao s,long voo) {</code>                                                              | Início da declaração de mapa. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                 |
| 45 | <code>return banco.consultar(c-&gt;{autorizacao.exigirConsulta(c,s);return new<br/>MapaAssentos(Sql.listar(c,"""</code> | Retorno: entrega banco.consultar(c->{autorizacao.exigirConsulta(c,s);return new MapaAssentos(Sql.listar(c,""" SELECT i.codigo,a.fila,a.coluna,i.bloqueado,EXISTS (SELECT 1 FROM ocupacao_assento o WHERE o.inventario_id=i.id AND o.libera... |



Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Confere no banco o acesso vigente do usuário para esta operação: `exigirConsulta(c, s)`.

Retorno: entrega `new MapaAssentos(Sql.listar(c, "" SELECT i.codigo,a.fila,a.coluna,i.bloqueado,EXISTS (SELECT 1 FROM ocupacao_assento o WHERE o.inventario_id=i.id AND o.liberada_em IS NULL) AS ocupado FROM inventario_assento_voo i JOIN...` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve `List<T>`. Tabelas mencionadas: `ocupacao_assento`, `inventario_assento_voo`, `assento_aeronave`. Parâmetros ligados em ordem: `voo`.

`new` constrói `MapaAssentos`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
46         SELECT i.codigo,a.fila,a.coluna,i.bloqueado,EXISTS (SELECT 1 FROM ocupacao_assento o WHERE
o.inventario_id=i.id AND o.liberada_em IS NULL) AS ocupado
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
47         FROM inventario_assento_voo i JOIN assento_aeronave a ON a.id=i.assento_id WHERE
i.voo_id=? ORDER BY a.fila,a.coluna
```

Continuação do texto SQL. Os filtros/joins/ordenação escritos aqui são executados pelo PostgreSQL quando o statement é chamado, não pelo compilador Java.

```
48         """,Sql::linha,voo));});
```

Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.

```
49     }
```

```
50
```

## bagagem - L51 a L54

Valida etiqueta e retorna situação atual junto à sequência de eventos históricos.

Entradas declaradas: (`Sessao s,String etiqueta`).

Saída declarada: `List<Map<String,Object>>` - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: `IntegracaoTeste.atendimento (L127)`; `IntegracaoTeste.bagagensEmConexao (L295)`; `MenuOperacional.atendimento (L138)`.

Chamadas observadas no corpo: `Validacao.texto`, `banco.consultar`, `autorizacao.exigirAtendimento`, `Sql.listar`, `Sql::linha`.

```
51     public List<Map<String,Object>> bagagem(Sessao s,String etiqueta) {
```

Início da declaração de `bagagem`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
52         String tag=Validacao.texto(etiqueta,"Etiqueta",30);
```

Declara variável local `tag` do tipo `String`. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
53         return banco.consultar(c->{autorizacao.exigirAtendimento(c,s);return Sql.listar(c,"SELECT
b.etiqueta,b.situacao AS atual,e.aeroporto_id,e.situacao,e.ocorrido_em,e.observacao FROM bagagem b JOIN
evento_bagagem e ON e.bagagem_id=b.id WHERE b.etiqueta=? ORDER BY e.ocorrido_em,e.id",Sql::linha,tag));});
```

Retorno: entrega `banco.consultar(c->{autorizacao.exigirAtendimento(c,s);return Sql.listar(c,"SELECT b.etiqueta,b.situacao AS atual,e.aeroporto_id,e.situacao,e.ocorrido_em,e.observacao FROM bagagem b JOIN evento_bagagem e ON e.bagagem_...` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Confere no banco o acesso vigente do usuário para esta operação: `exigirAtendimento(c, s)`.

Retorno: entrega `Sql.listar(c,"SELECT b.etiqueta,b.situacao AS atual,e.aeroporto_id,e.situacao,e.ocorrido_em,e.observacao FROM bagagem b JOIN evento_bagagem e ON e.bagagem_id=b.id WHERE b.etiqueta=? ORDER BY e.ocorrido_em,e.id",Sql::l...` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve `List<T>`. Tabelas mencionadas: `bagagem`, `evento_bagagem`. Parâmetros ligados em ordem: `tag`.

```
54     }
```

```
55
```

## resumoAtrasos - L56 a L66

Calcula minutos não negativos de atraso, ordena cópia e devolve mínimo/máximo/mediana. Lista vazia devolve `Map` vazio; tamanho par usa média dos centrais.



Entradas declaradas: (Sessao s).

Saída declarada: Map<String,java.math.BigDecimal> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.relatorios (L167).

Chamadas observadas no corpo: banco.consultar, autorizacao.exigirConsulta, new ArrayList<>, Sql.listar, r.getBigDecimal, valores.isEmpty, Map.of, Collections.sort, Collections.max, Collections.min, valores.size, valores.get, valores.get(valores.size()/2-1).add(valores.get(valores.size()/2)).divide, valores.get(valores.size()/2-1).add, new java.math.BigDecimal.

```

56     public Map<String,java.math.BigDecimal> resumoAtrasos(Sessao s) {
        Início da declaração de resumoAtrasos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
        é invocado.

57         return banco.consultar(c->{
            Instrução idêntica à de AdministracaoServico:L22. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
            desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

58             autorizacao.exigirConsulta(c,s);
            Instrução idêntica à de RelatoriosCompletosServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis
            locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

59             var valores=new ArrayList<>(Sql.listar(c,"SELECT GREATEST(0,EXTRACT(EPOCH
FROM(COALESCE(partida_real,partida_estimada,partida_prevista)-partida_prevista))/60) FROM voo WHERE
situacao<>'CANCELADO'",r->r.getBigDecimal(1)));
            Declara variável local valores com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
            escopo é o bloco atual.

            Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: voo.

            new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

60             if(valores.isEmpty()) return Map.of();
            Decisão específica: (valores.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a
            execução atual com return.

            Retorno: entrega Map.of() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
            caminho.

61             Collections.sort(valores);
            Invoca Collections.sort com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
            método para localizar o corpo no projeto.

62             var maximo=Collections.max(valores);var minimo=Collections.min(valores);
            Declara variável local maximo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
            escopo é o bloco atual.

            Declara variável local minimo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
            escopo é o bloco atual.

63             var mediana=valores.size()%2==1?valores.get(valores.size()/2):valores.get(valores.size()/2-1).
add(valores.get(valores.size()/2)).divide(new java.math.BigDecimal("2"));
            Declara variável local mediana com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
            escopo é o bloco atual.

            Escolha de valor: valores.size()%2==1. true escolhe valores.get(valores.size()/2); false escolhe
            valores.get(valores.size()/2-1).add(valores.get(valores.size()/2)).divide(new java.math.BigDecimal("2")). Só a alternativa escolhida
            é avaliada.

            new constrói java.math.BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
            banco por si só.

64             return Map.of("minimo",minimo,"maximo",maximo,"mediana",mediana);
            Retorno: entrega Map.of("minimo",minimo,"maximo",maximo,"mediana",mediana) ao chamador desta execução; nenhuma
            instrução posterior desse mesmo corpo executa por esse caminho.

            Chama Map.of: Fábrica estática: cria o valor do tipo receptor a partir dos argumentos.

65         });
66     }
67

```

## exportar - L68 a L93

Sem LIMIT de página: streaming com fetchSize de 500, metadados para cabeçalho e todas as linhas. Grava em temporário, audita e depois move; falha remove temporário e relança causa de IO.

Entradas declaradas: (Sessao s,TipoRelatorio tipo,boolean csv).

Saída declarada: Path - caminho de arquivo, sem garantir por si a existência desse arquivo.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); IntegracaoTeste.arquivos (L153); IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.relatorios (L167).

Chamadas observadas no corpo: Path.of("relatorios").toAbsolutePath().normalize, Path.of("relatorios").toAbsolutePath, Path.of, Files.createDirectories, Files.createTempFile, pasta.resolve, tipo.name().toLowerCase, tipo.name, UUID.randomUUID, banco.transacao, permitir, new BufferedWriter, new FileWriter, temporario.toFile, c.prepareStatement, tipo.sql, statement.setFetchSize, statement.setQueryTimeout, statement.executeQuery, r.getMetaData, meta.getColumnCount, meta.getColumnLabel, escrever, r.next, r.getObject, r.getObject(i+1).toString, new UncheckedIOException, auditoria.registrar, s.usuarioId, Files.move, Files.deleteIfExists, erro.addSuppressed, io.getCause.

```
68     public Path exportar(Sessao s,TipoRelatorio tipo,boolean csv) throws IOException {
    Início da declaração de exportar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

69         Path pasta=Path.of("relatorios").toAbsolutePath().normalize();Files.createDirectories(pasta);
    Declara variável local pasta do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

70         Path temporario=Files.createTempFile(pasta,"relatorio-",".tmp"),destino=pasta.resolve(tipo.name().
    toLowerCase(Locale.ROOT)+"-"+UUID.randomUUID().toString().replace("-",".")+".csv");
    Declara variável local temporario, destino do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
    bloco atual.

    Escolha de valor: csv. true escolhe ".csv"; false escolhe ".txt". Só a alternativa escolhida é avaliada.

71         try {
    Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

72             banco.transacao(c->{
    Instrução idêntica à de Main:L82. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A
    posição e o contexto do método acima mostram quando ela ocorre.

73                 permitir(c,s,tipo);
    Chama RelatoriosCompletosServico.permitir: AUDITORIA exige administrador; outros restritos exigem atendimento; demais
    exigem consulta.

74                 try(BufferedWriter writer=new BufferedWriter(new
    FileWriter(temporario.toFile(),StandardCharsets.UTF_8));PreparedStatement
    statement=c.prepareStatement(tipo.sql())) {
    Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à
    criação.

    new constrói BufferedWriter, FileWriter: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
    banco por si só.

75                     statement.setFetchSize(500);statement.setQueryTimeout(30);
    Invoca statement.setFetchSize com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
    chamadas do método para localizar o corpo no projeto.

76                     try(ResultSet r=statement.executeQuery()) {
    Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à
    criação.

77                         var meta=r.getMetaData();String[] campos=new String[meta.getColumnCount()];
    Declara variável local meta com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.

    Declara variável local campos do tipo String[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

78                         for(int i=0;i<campos.length;i++) campos[i]=meta.getColumnLabel(i+1);
    Laço controlado por inicialização, condição e atualização: for(int i=0;i<campos.length;i++). return, break ou exceção podem
    encerrar esse caminho; continue passa à próxima volta.

    Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

    Atualiza campos[i] usando = e o resultado de meta.getColumnLabel(i+1). Atribuição modifica a variável/célula indicada; não é um
    teste de igualdade.

79                         escrever(writer,campos,csv);
    Chama RelatoriosCompletosServico.escrever: Limpa CR/LF dos valores. CSV usa ;, aspas duplicadas e prefixo de neutralização
    para fórmulas; TXT usa separador |.

80                         while(r.next()) {
    Repete enquanto a condição permanecer verdadeira: while(r.next()). return, break ou exceção podem encerrar esse caminho;
    continue passa à próxima volta.
```

```

81         for(int i=0;i<campos.length;i++)
campos[i]=r.getObject(i+1)==null?"":r.getObject(i+1).toString();

```

Laço controlado por inicialização, condição e atualização: `for(int i=0;i<campos.length;i++)`. `return`, `break` ou exceção podem encerrar esse caminho; `continue` passa à próxima volta.

Declara variável local `i` do tipo `int`. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Atualiza `campos[i]` usando `=` e o resultado de `r.getObject(i+1)==null?"":r.getObject(i+1).toString()`. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

Escolha de valor: `r.getObject(i+1)==null`. `true` escolhe `""`; `false` escolhe `r.getObject(i+1).toString()`. Só a alternativa escolhida é avaliada.

```

82         escrever(writer, campos, csv);

```

Instrução idêntica à de `RelatoriosCompletosServico:L79`. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

83         }
84     }
85     } catch(IOException erro) { throw new UncheckedIOException(erro); }

```

Trata a(s) exceção(ões) declarada(s) em `catch(IOException erro)`. O tratamento mostrado abaixo decide se avisa, repete ou relança.

Lança `new UncheckedIOException(erro)`. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar `rollback`.

`new` constrói `UncheckedIOException`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

86         auditoria.registrar(c,s.usuarioId(),"EXPORTAR_RELATORIO",tipo.name(),0);return null;

```

Registra o evento de auditoria na conexão fornecida: `registrar(c, s.usuarioId(), "EXPORTAR_RELATORIO", tipo.name(), 0)`.

`return null` encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```

87     });
88     return Files.move(temporario,destino);

```

Instrução idêntica à de `ArquivosServico:L69`. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

89     } catch(RuntimeException | IOException erro) {

```

Trata a(s) exceção(ões) declarada(s) em `catch(RuntimeException | IOException erro)`. O tratamento mostrado abaixo decide se avisa, repete ou relança.

```

90         try { Files.deleteIfExists(temporario); } catch(IOException limpeza) {
erro.addSuppressed(limpeza); }

```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Trata a(s) exceção(ões) declarada(s) em `catch(IOException limpeza)`. O tratamento mostrado abaixo decide se avisa, repete ou relança.

```

91         if(erro instanceof UncheckedIOException io) throw io.getCause();throw erro;

```

Decisão específica: `(erro instanceof UncheckedIOException io)`. Se `for true`, executa o ramo seguinte; se `for false`, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança `io.getCause()`. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar `rollback`.

Lança `erro`. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar `rollback`.

```

92     }
93 }
94

```

## escrever - L95 a L103

Limpa CR/LF dos valores. CSV usa `;`, aspas duplicadas e prefixo de neutralização para fórmulas; TXT usa separador `|`.

Entradas declaradas: `(BufferedWriter writer,String[] campos,boolean csv)`.

Saída declarada: `void` - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: `RelatoriosCompletosServico.exportar (L68)`.

Chamadas observadas no corpo: `new StringJoiner`, `original.replace("\r",'')`.`replace`, `original.replace`, `valor.stripLeading().matches`, `valor.stripLeading`, `valor.replace`, `linha.add`, `writer.write`, `linha.toString`, `writer.newLine`.

```

95     private void escrever(BufferedWriter writer,String[] campos,boolean csv) throws IOException {

```

Início da declaração de `escrever`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
96      StringJoiner linha=new StringJoiner(csv?";":" | ");
```

Declara variável local linha do tipo StringJoiner. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Escolha de valor: csv. true escolhe ";", false escolhe " | ". Só a alternativa escolhida é avaliada.

new constrói StringJoiner: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
97      for(String original:campos) {
```

Percorre cada elemento da coleção/vetor: for(String original:campos). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
98          String valor=original.replace('\r',' ').replace('\n',' ');
```

Declara variável local valor do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
99          if(csv) { if(valor.stripLeading().matches("[^=+@-].*"))
valor=" "+valor;valor="\""+valor.replace("\"","\"\\\"")+"\""; }
```

Decisão específica: (csv). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Decisão específica: (valor.stripLeading().matches("[^=+@-].\*")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Atualiza valor usando = e o resultado de ""+valor. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

Atualiza valor usando = e o resultado de "\"" + valor.replace("\"", "\\\"") + "\"". Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
100          linha.add(valor);
```

Chama linha.add: Insere na coleção. Em Set, o boolean de retorno indica se o elemento era novo.

```
101      }
```

```
102      writer.write(linha.toString());writer.newLine();
```

Chama writer.write: Grava texto/bytes no destino fornecido; é efeito em arquivo/stream.

```
103    }
```

```
104 }
```

## RelatorioServico

src/main/java/br/edu/aeroporto/servico/RelatorioServico.java

Exporta apenas a lista de voos fornecida pelo menu, usando uma implementação de ExportadorVoos e arquivo temporário.

29 linhas | 1 métodos/construtores declarados | SHA-256: c85fd3ec3dbca1a0...

```
1 package br.edu.aeroporto.servico;
```

Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

```
2
```

```
3 import br.edu.aeroporto.dto.VooResumo;
```

```
4 import br.edu.aeroporto.relatorio.ExportadorVoos;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Interface do formato de saída. RelatorioServico pode chamar escrever/extensao sem depender de TXT ou CSV específicos.

```
5 import java.io.BufferedWriter;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
6 import java.io.IOException;
```

```
7 import java.nio.charset.StandardCharsets;
```

```
8 import java.nio.file.Files;
```

```
9 import java.nio.file.Path;
```

```
10 import java.util.List;
```

```
11 import java.util.UUID;
```

```
12
```

```
13 public final class RelatorioServico {
```

Declara RelatorioServico como tipo com campos e métodos. final impede herança.

### exportar - L14 a L28

Cria pasta, temporário e nome único. Chama escrever da estratégia recebida, fecha writer e move para destino. Falha tenta remover temporário; retorna Path.

Entradas declaradas: (List<VooResumo> voos, ExportadorVoos exportador).

Saída declarada: Path - caminho de arquivo, sem garantir por si a existência desse arquivo.

Possíveis chamadores identificados: MenuPrincipal.exportarVoos (L158).

Chamadas observadas no corpo: Path.of("relatorios").toAbsolutePath().normalize, Path.of("relatorios").toAbsolutePath, Path.of, Files.createDirectories, Files.createTempFile, pasta.resolve, UUID.randomUUID, exportador.extensao, Files.newBufferedWriter, exportador.escrever, Files.move, Files.deleteIfExists, erro.addSuppressed.

```
14 public Path exportar(List<VooResumo> voos, ExportadorVoos exportador) throws IOException {
```

Início da declaração de exportar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
15 Path pasta = Path.of("relatorios").toAbsolutePath().normalize();
```

Declara variável local pasta do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
16 Files.createDirectories(pasta);
```

Invoca Files.createDirectories com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
17 Path temporario = Files.createTempFile(pasta, "voos-", ".tmp");
```

Declara variável local temporario do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
18 Path destino = pasta.resolve("voos-" + UUID.randomUUID() + "." + exportador.extensao());
```

Declara variável local destino do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
19 try {
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

```
20 try (BufferedWriter arquivo = Files.newBufferedWriter(temporario, StandardCharsets.UTF_8)) {
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

```
21 exportador.escrever(arquivo, voos);
```

Chama ExportadorVoos.escrever: Contrato de escrita no BufferedWriter dos VooResumo recebidos. A implementação escolhida decide formatação e escape; IOException pode escapar ao serviço.

```
22 }
```

```
23         return Files.move(temporario, destino);
```

Retorno: entrega Files.move(temporario, destino) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca Files.move com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
24     } catch (IOException | RuntimeException erro) {
```

Trata a(s) exceção(ões) declarada(s) em catch (IOException | RuntimeException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
25         try { Files.deleteIfExists(temporario); } catch (IOException limpeza) {  
erro.addSuppressed(limpeza); }
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Trata a(s) exceção(ões) declarada(s) em catch (IOException limpeza). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
26         throw erro;
```

Lança erro. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

```
27     }
```

```
28 }
```

```
29 }
```

## ReservaServico

src/main/java/br/edu/aeroporto/servico/ReservaServico.java

Expira pendências e coordena reservas, cancelamentos e remarcações completas em transações repetíveis.

66 linhas | 6 métodos/construtores declarados | SHA-256: a87255bb63188175...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, ReservaJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.infraestrutura.jdbc.*;
5 import java.math.BigDecimal;
6 import java.time.Clock;
7 import java.util.ArrayList;
8 import java.util.List;
9 import java.util.Set;
10
11 public final class ReservaServico {
    Declara ReservaServico como tipo com campos e métodos. final impede herança.

12     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

13     private final ReservaJdbc repositorio;
    Campo(s) repositorio: private restringe acesso à classe. Tipo ReservaJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

14     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

15     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

16     private final Clock relógio;
    Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

17

```

### ReservaServico - L18 a L20

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco, ReservaJdbc repositorio, AutorizacaoJdbc autorizacao, AuditoriaJdbc auditoria, Clock relógio).

Possíveis chamadores identificados: Main.main (L55).

```

18     public ReservaServico(BancoDados banco, ReservaJdbc repositorio, AutorizacaoJdbc
    autorizacao, AuditoriaJdbc auditoria, Clock relógio) {
    Início da declaração de ReservaServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

19         this.banco=banco;this.repositorio=repositorio;this.autorizacao=autorizacao;this.auditoria=auditori
a;this.relógio=relógio;
    Guarda no campo banco da instância atual o valor de banco. O this diferencia o campo de parâmetros com o mesmo nome.

    Guarda no campo repositorio da instância atual o valor de repositorio. O this diferencia o campo de parâmetros com o mesmo nome.

    Guarda no campo autorizacao da instância atual o valor de autorizacao. O this diferencia o campo de parâmetros com o mesmo nome.

    Guarda no campo auditoria da instância atual o valor de auditoria. O this diferencia o campo de parâmetros com o mesmo nome.

    Guarda no campo relógio da instância atual o valor de relógio. O this diferencia o campo de parâmetros com o mesmo nome.

20     }

```

21

## expirarPendencias - L22 a L22

Rotina interna chamada ao iniciar e antes de vender; usa relógio e transação repetível, retornando quantidade expirada.

Entradas declaradas: ().

Saída declarada: int - inteiro primitivo de 32 bits, usado para índice, limite ou contagem.

Possíveis chamadores identificados: IntegracaoTeste.comercial (L109); Main.main (L55); ReservaServico.remarcar (L49); ReservaServico.reservar (L26).

Chamadas observadas no corpo: banco.transacaoRepetivel, repositorio.expirar, relógio.instant.

```
22 public int expirarPendencias() { return
    banco.transacaoRepetivel(c->repositorio.expirar(c,relógio.instant())); }
```

Início da declaração de expirarPendencias. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega banco.transacaoRepetivel(c->repositorio.expirar(c,relógio.instant())) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa uma transação completa com até três tentativas totais somente para deadlock/serialização (40P01/40001). O callback pode executar novamente.

23

## expirar - L24 a L24

Versão solicitada pelo usuário: exige atendimento antes de executar a rotina.

Entradas declaradas: (Sessao sessao).

Saída declarada: int - inteiro primitivo de 32 bits, usado para índice, limite ou contagem.

Possíveis chamadores identificados: MenuOperacional.reservas (L95).

Chamadas observadas no corpo: banco.transacaoRepetivel, autorizacao.exigirAtendimento, repositorio.expirar, relógio.instant.

```
24 public int expirar(Sessao sessao) { return
    banco.transacaoRepetivel(c->{autorizacao.exigirAtendimento(c,sessao);return
    repositorio.expirar(c,relógio.instant());}); }
```

Início da declaração de expirar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega banco.transacaoRepetivel(c->{autorizacao.exigirAtendimento(c,sessao);return repositorio.expirar(c,relógio.instant());}) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa uma transação completa com até três tentativas totais somente para deadlock/serialização (40P01/40001). O callback pode executar novamente.

Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, sessao).

Retorno: entrega repositorio.expirar(c,relógio.instant()) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

25

## reservar - L26 a L34

Exige 1..100 itens e prazo 1..120 minutos, copia a lista e expira pendências; na transação exige atendimento, grava reserva inteira e audita, devolvendo long.

Entradas declaradas: (Sessao sessao,long comprador,List<PedidoTrecho> trechos,int prazoMinutos).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.comercial (L109); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.rascunho (L104).

Chamadas observadas no corpo: Dados.exigir, trechos.isEmpty, trechos.size, List.copyOf, expirarPendencias, banco.transacaoRepetivel, autorizacao.exigirAtendimento, repositorio.reservar, sessao.usuarioId, relógio.instant, auditoria.registrar.

```
26 public long reservar(Sessao sessao,long comprador,List<PedidoTrecho> trechos,int prazoMinutos) {
```

Início da declaração de reservar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.



```

27      Dados.exigir(trechos!=null && !trechos.isEmpty() && trechos.size()<=100 && prazoMinutos>=1 &&
prazoMinutos<=120,"Reserva deve ter de 1 a 100 itens e prazo de 1 a 120 minutos.");
Regra obrigatória: trechos!=null && !trechos.isEmpty() && trechos.size()<=100 && prazoMinutos>=1 && prazoMinutos<=120 deve
resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Reserva deve ter de 1 a 100 itens e prazo de 1 a 120
minutos.". Essa condição específica não é substituída por outra validação parecida.

28      List<PedidoTrecho> copia=List.copyOf(trechos);expirarPendencias();
Declara variável local copia do tipo List<PedidoTrecho>. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
bloco atual.

29      return banco.transacaoRepetivel(c->{
Instrução idêntica à de AtendimentoServico:L33. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

30      autorizacao.exigirAtendimento(c,sessao);
Instrução idêntica à de PassageiroServico:L48. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

31      long
id=repositorio.reservar(c,sessao.usuarioId(),comprador,copia,religio.instant(),prazoMinutos);
Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

32      auditoria.registrar(c,sessao.usuarioId(),"RESERVAR","reserva",id);return id;
Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "RESERVAR", "reserva", id).

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

33      });
34      }
35

```

## cancelar - L36 a L47

Item null representa toda a reserva; um id limita a seleção. Soma os direitos de devolução dos itens, solicita reembolso e recalcula cabeçalho, preservando histórico.

Entradas declaradas: (Sessao sessao,long reserva,Long item,String motivo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.concorrencia (L164); MenuOperacional.reservas (L95).

Chamadas observadas no corpo: Validacao.texto, banco.transacaoRepetivel, autorizacao.exigirAtendimento, BloqueiosJdbc.reserva, Sql.listar, Sql::linha, Dados.exigir, itens.isEmpty, devolucao.add, repositorio.cancelarItem, religio.instant, sessao.usuarioId, repositorio.solicitarDevolucao, repositorio.recalcular, auditoria.registrar.

```

36      public void cancelar(Sessao sessao,long reserva,Long item,String motivo) {
Início da declaração de cancelar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
invocado.

37      String razao=Validacao.texto(motivo,"Motivo",500);
Instrução idêntica à de PlanejamentoServico:L53. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

38      banco.transacaoRepetivel(c->{
Instrução idêntica à de AtendimentoServico:L61. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

39      autorizacao.exigirAtendimento(c,sessao);BloqueiosJdbc.reserva(c,reserva);
Confere no banco o acesso vigente do usuário para esta operação: exigirAtendimento(c, sessao).

40      var itens=Sql.listar(c,"SELECT * FROM item_reserva WHERE reserva_id=? AND (:?:bigint IS NULL
OR id=?) ORDER BY id",Sql::linha,reserva,item,item);
Declara variável local itens com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
escopo é o bloco atual.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. Parâmetros
ligados em ordem: reserva, item, item.

41      Dados.exigir(!itens.isEmpty(),"Item nao encontrado nesta reserva.");
Regra obrigatória: !itens.isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Item nao
encontrado nesta reserva.". Essa condição específica não é substituída por outra validação parecida.

42      BigDecimal devolucao=BigDecimal.ZERO;
Declara variável local devolucao do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
bloco atual.

```

```

43         for(var i:itens)
            devolucao=devolucao.add(repositorio.cancelarItem(c,i,religio.instant(),razao,false,sessao.usuarioId()));
    Percorre cada elemento da coleção/vetor: for(var i:itens). return, break ou exceção podem encerrar esse caminho; continue passa
    à próxima volta.

    Atualiza devolucao usando = e o resultado de
    devolucao.add(repositorio.cancelarItem(c,i,religio.instant(),razao,false,sessao.usuarioId())). Atribuição modifica a variável/célula
    indicada; não é um teste de igualdade.

44     repositorio.solicitarDevolucao(c,reserva,devolucao,"cancel:"+reserva+"-"+(item==null?"todos":item),razao);
    Chama ReservaJdbc.solicitarDevolucao: Limita o solicitado ao recebido livre. Divide entre pagamentos aprovados, desconta
    reembolsos comprometidos e usa chave por pagamento para evitar repetição.

    Escolha de valor: item==null. true escolhe "todos"; false escolhe item. Só a alternativa escolhida é avaliada.

45     repositorio.recalcular(c,reserva);auditoria.registrar(c,sessao.usuarioId(),"CANCELAR_RESERVA",
    "reserva",reserva);return null;
    Chama ReservaJdbc.recalcular: Ordem das condições importa: todos expirados, todos cancelados/expirados, alguns
    cancelados/expirados, todos usados/ausentes, algum pendente, caso contrário confirmada.

    Registra o evento de auditoria na conexão fornecida: registrar(c, sessao.usuarioId(), "CANCELAR_RESERVA", "reserva",
    reserva).

    return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o
    contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

46     });
47 }
48

```

## remarcar - L49 a L65

Bloqueia voos antigos e novo em ordem; preserva passageiro e comprador, cancela item anterior segundo tarifa, cria outra reserva e liga ambos na auditoria.

Entradas declaradas: (Sessao sessao,long item,PedidoTrecho novo,int prazo).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203); MenuOperacional.reservas (L95).

Chamadas observadas no corpo: Dados.exigir, expirarPendencias, banco.transacaoRepetivel, autorizacao.exigirAtendimento, Sql.registro, Dados.id, new ArrayList<>, Sql.listar, r.getLong, voos.add, novo.voo, BloqueiosJdbc.voos, BloqueiosJdbc.reserva, novo.passageiro, Set.of("PENDENTE","CONFIRMADO").contains, Set.of, antigo.get, repositorio.cancelarItem, religio.instant, sessao.usuarioId, repositorio.solicitarDevolucao, repositorio.recalcular, repositorio.reservar, List.of, Sql.executar.

```

49     public long remarcar(Sessao sessao,long item,PedidoTrecho novo,int prazo) {
    Início da declaração de remarcar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

50         Dados.exigir(prazo>=1 && prazo<=120,"Prazo invalido.");expirarPendencias();
    Regra obrigatória: prazo>=1 && prazo<=120 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com
    "Prazo invalido". Essa condição específica não é substituída por outra validação parecida.

51         return banco.transacaoRepetivel(c->{
    Instrução idêntica à de AtendimentoServico:L33. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
    desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

52             autorizacao.exigirAtendimento(c,sessao);
    Instrução idêntica à de PassageiroServico:L48. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais
    desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

53             var inicial=Sql.registro(c,"SELECT reserva_id FROM item_reserva WHERE id=?",item);
    Declara variável local inicial com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.

    Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas:
    item_reserva. Parâmetros ligados em ordem: item.

54             long reserva=Dados.id(inicial,"reserva_id");
    Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

55             var voos=new ArrayList<>(Sql.listar(c,"SELECT voo_id FROM item_reserva WHERE
    reserva_id=?",r->r.getLong(1),reserva));voos.add(novo.voo());BloqueiosJdbc.voos(c,voos);
    Declara variável local voos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.

```

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: reserva.

new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
56      var r=BloqueiosJdbc.reserva(c,reserva);var antigo=Sql.registro(c,"SELECT * FROM item_reserva
WHERE id=? FOR UPDATE",item);
```

Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local antigo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: item\_reserva. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: item.

```
57      Dados.exigir(Dados.id(antigo,"passageiro_id")==novo.passageiro(),"Remarcacao preserva
passageiro.");
```

Regra obrigatória: Dados.id(antigo,"passageiro\_id")==novo.passageiro() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Remarcacao preserva passageiro.". Essa condição específica não é substituída por outra validação parecida.

```
58      Dados.exigir(Set.of("PENDENTE","CONFIRMADO").contains(antigo.get("situacao")), "Item nao admite
remarcacao.");
```

Regra obrigatória: Set.of("PENDENTE","CONFIRMADO").contains(antigo.get("situacao")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Item nao admite remarcacao.". Essa condição específica não é substituída por outra validação parecida.

```
59      BigDecimal
devolucao=repositorio.cancelarItem(c,antigo,religio.instant(),"Remarcacao",false,sessao.usuarioId());
```

Declara variável local devolucao do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
60      repositorio.solicitarDevolucao(c,reserva,devolucao,"remarcacao:"+item,"Remarcacao");repositorio.
recalcular(c,reserva);
```

Chama ReservaJdbc.solicitarDevolucao: Limita o solicitado ao recebido livre. Divide entre pagamentos aprovados, desconta reembolsos comprometidos e usa chave por pagamento para evitar repetição.

```
61      long nova=repositorio.reservar(c,sessao.usuarioId(),Dados.id(r,"comprador_id"),List.of(novo),r
elogio.instant(),prazo);
```

Declara variável local nova do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
62      Sql.executar(c,"INSERT INTO evento_auditoria(usuario_id,acao,entidade,registro_id,detalhes)
VALUES (?,?, 'REMARCAR', 'item_reserva', ?,jsonb_build_object('reserva_anterior',?:::bigint,'nova_reserva',?:::bi
gint))",sessao.usuarioId(),item,reserva,nova);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: evento\_auditoria. Parâmetros ligados em ordem: sessao.usuarioId(), item, reserva, nova.

```
63      return nova;
```

Retorno: entrega nova ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
64      });
65      }
66      }
```

## VooServico

src/main/java/br/edu/aeroporto/servico/VooServico.java

Executa transições do voo e seus efeitos em itens, bilhetes, agendas, equipe, serviços de solo e reembolsos.

73 linhas | 2 métodos/construtores declarados | SHA-256: 2e4086f381b32d9c...

Dependências guardadas em campos: AuditoriaJdbc, AutorizacaoJdbc, BancoDados, PlanejamentoJdbc, ReservaJdbc. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1  package br.edu.aeroporto.servico;
    Declara o pacote deste arquivo: br.edu.aeroporto.servico. O nome completo distingue a classe de outras com o mesmo nome.

2
3  import br.edu.aeroporto.dominio.*;
4  import br.edu.aeroporto.infraestrutura.jdbc.*;
5  import java.math.BigDecimal;
6  import java.time.Clock;
7  import java.util.ArrayList;
8  import java.util.Set;
9
10 public final class VooServico {
    Declara VooServico como tipo com campos e métodos. final impede herança.

11     private final BancoDados banco;
    Instrução idêntica à de AdministracaoServico:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

12     private final AutorizacaoJdbc autorizacao;
    Instrução idêntica à de AdministracaoServico:L14. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

13     private final AuditoriaJdbc auditoria;
    Instrução idêntica à de AdministracaoServico:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

14     private final Clock relógio;
    Instrução idêntica à de ArquivosServico:L17. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

15     private final ReservaJdbc reservas=new ReservaJdbc();
    Instrução idêntica à de AtendimentoServico:L25. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

16     private final PlanejamentoJdbc planejamento=new PlanejamentoJdbc();
    Instrução idêntica à de OperacaoServico:L18. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

17

```

### VooServico - L18 a L20

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock relógio).

Possíveis chamadores identificados: Main.main (L55).

```

18     public VooServico(BancoDados banco,AutorizacaoJdbc autorizacao,AuditoriaJdbc auditoria,Clock relógio)
    {
        Início da declaração de VooServico. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

19         this.banco=banco;this.autorizacao=autorizacao;this.auditoria=auditoria;this.relogio=relógio;
        Instrução idêntica à de ArquivosServico:L21. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

20     }
21

```

## transicao - L22 a L72

Expira pendências, reúne reservas/voos relacionados e bloqueia em ordem. Valida transição enum. Cada próximo estado tem efeitos próprios: janela/equipe/portão, ausência na partida, liberação na conclusão ou cancelamento com devolução e ocorrência. Atualiza o voo e audita.

Entradas declaradas: (Sessao s,long voo,SituacaoVoo proxima,String motivo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.atendimento (L127); IntegracaoTeste.bagagensEmConexao (L295); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.regrasComplementares (L369); MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: Validacao.texto, banco.transacaoRepetivel, reservas.expirar, relógio.instant, autorizacao.exigirOperacao, planejamento.serializar, Sql.listar, r.getLong, new ArrayList<Long>, todos.add, todos.addAll, BloqueiosJdbc.voos, BloqueiosJdbc.reserva, BloqueiosJdbc.voo, SituacaoVoo.valueOf, Dados.texto, Dados.exigir, atual.permiteTransicaoPara, auditoria.definirAutorDoVoo, s.usuarioId, relógio.instant().isBefore, ReservaJdbc.janela, OperacaoJdbc.validarTripulacao, Sql.listar(c,"SELECT a.id FROM alocao\_recurso a JOIN recurso\_aeroporto r ON r.id=a.recurso\_id WHERE a.voo\_id=? AND a.ativa AND a.finalidade='PARTIDA' AND r.tipo='PORTAO' AND a.inicio<=? AND a.fim>=?"),r->r.getLong(1),voo,ReservaJdbc.janela(v,"embarque\_abre"),ReservaJdbc.janela(v,"embarque\_fecha"),isEmpty, Sql.registro, Dados.id, Sql.listar(c,"SELECT id FROM servico\_solo WHERE voo\_id=? AND situacao IN ('PENDENTE','EM\_EXECUCAO')"),r->r.getLong(1),voo.isEmpty, Sql:linha, "CONFIRMADO".equals, item.get, Sql.executar, reservas.invalidate, reservas.recalcular, planejamento.agendas, PlanejamentoJdbc.efetivo, v.get, Dados.instante(v,"chegada\_real").isAfter, Dados.instante, valor.add, reservas.cancelarItem, reservas.solicitarDevolucao, auditoria.registrar.

|    |                                                                                                                                              |                                                                                                                                                                                                                                                                                                             |
|----|----------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 22 | <code>public void transicao(Sessao s,long voo,SituacaoVoo proxima,String motivo) {</code>                                                    | Início da declaração de transicao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                                                                          |
| 23 | <code>String razao=Validacao.texto(motivo,"Motivo",500);</code>                                                                              | Instrução idêntica à de PlanejamentoServico:L53. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                               |
| 24 | <code>banco.transacaoRepetivel(c-&gt;reservas.expirar(c,relógio.instant()));</code>                                                          | Executa uma transação completa com até três tentativas totais somente para deadlock/serialização (40P01/40001). O callback pode executar novamente.                                                                                                                                                         |
| 25 | <code>banco.transacaoRepetivel(c-&gt;{</code>                                                                                                | Instrução idêntica à de AtendimentoServico:L61. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                |
| 26 | <code>autorizacao.exigirOperacao(c,s);planejamento.serializar(c);</code>                                                                     | Instrução idêntica à de OperacaoServico:L36. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                                                                   |
| 27 | <code>var ids=Sql.listar(c,"SELECT DISTINCT reserva_id FROM item_reserva WHERE voo_id=? ORDER BY reserva_id",r-&gt;r.getLong(1),voo);</code> | Declara variável local ids com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. Parâmetros ligados em ordem: voo. |
| 28 | <code>var todos=new ArrayList&lt;Long&gt;();todos.add(voo);</code>                                                                           | Declara variável local todos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.               |
| 29 | <code>for(long id:ids) todos.addAll(Sql.listar(c,"SELECT voo_id FROM item_reserva WHERE reserva_id=?",r-&gt;r.getLong(1),id));</code>        | Percorre cada elemento da coleção/vetor: for(long id:ids). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.<br><br>Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item_reserva. Parâmetros ligados em ordem: id.       |
| 30 | <code>BloqueiosJdbc.voos(c,todos);for(long id:ids) BloqueiosJdbc.reserva(c,id);</code>                                                       | Chama BloqueiosJdbc.voos: Elimina ids repetidos, ordena e bloqueia cada voo. A ordem consistente reduz deadlocks.<br><br>Percorre cada elemento da coleção/vetor: for(long id:ids). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.                                   |
| 31 | <code>var v=BloqueiosJdbc.voo(c,voo);SituacaoVoo atual=SituacaoVoo.valueOf(Dados.texto(v,"situacao"));</code>                                | Declara variável local v com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                         |

Declara variável local atual do tipo SituacaoVoo. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
32      Dados.exigir(atual.permiteTransicaoPara(proxima),"Transicao de voo invalida.");
```

Regra obrigatória: atual.permiteTransicaoPara(proxima) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Transicao de voo invalida.". Essa condição específica não é substituída por outra validação parecida.

```
33      auditoria.definirAutorDoVoo(c,s.usuarioId(),razao);
```

Registra o evento de auditoria na conexão fornecida: definirAutorDoVoo(c, s.usuarioId(), razao).

```
34      switch(proxima) {
```

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

```
35          case CHECKIN_ABERTO ->
```

```
Dados.exigir(!relogio.instant().isBefore(ReservaJdbc.janela(v,"checkin_abre")) &&
relogio.instant().isBefore(ReservaJdbc.janela(v,"checkin_fecha")), "Fora da janela de check-in.");
```

Ramo do switch: case CHECKIN\_ABERTO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

Regra obrigatória: !relogio.instant().isBefore(ReservaJdbc.janela(v,"checkin\_abre")) && relogio.instant().isBefore(ReservaJdbc.janela(v,"checkin\_fecha")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Fora da janela de check-in.". Essa condição específica não é substituída por outra validação parecida.

```
36          case EMBARQUE -> {
```

Ramo do switch: case EMBARQUE. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
37      Dados.exigir(!relogio.instant().isBefore(ReservaJdbc.janela(v,"embarque_abre")) &&
relogio.instant().isBefore(ReservaJdbc.janela(v,"embarque_fecha")), "Fora da janela de embarque.");
```

Regra obrigatória: !relogio.instant().isBefore(ReservaJdbc.janela(v,"embarque\_abre")) && relogio.instant().isBefore(ReservaJdbc.janela(v,"embarque\_fecha")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Fora da janela de embarque.". Essa condição específica não é substituída por outra validação parecida.

```
38      OperacaoJdbc.validarTripulacao(c,voo);
```

Chama OperacaoJdbc.validarTripulacao: Revalida jornadas e conta assentos vendáveis. Exige 1 comandante, 1 copiloto e ceil(assentos/50) comissários, mínimo 1.

```
39      Dados.exigir(!Sql.listar(c,"SELECT a.id FROM alocao_recurso a JOIN
recurso_aeroporto r ON r.id=a.recurso_id WHERE a.voo_id=? AND a.ativa AND a.finalidade='PARTIDA' AND
r.tipo='PORTAO' AND a.inicio<=? AND a.fim>=?",r->r.getLong(1),voo,ReservaJdbc.janela(v,"embarque_abre"),Re
servaJdbc.janela(v,"embarque_fecha")).isEmpty(), "Portao nao cobre a janela de embarque.");
```

Regra obrigatória: !Sql.listar(c,"SELECT a.id FROM alocao\_recurso a JOIN recurso\_aeroporto r ON r.id=a.recurso\_id WHERE a.voo\_id=? AND a.ativa AND a.finalidade='PARTIDA' AND r.tipo='PORTAO' AND a.inicio<=? AND a.fim>=?",r->r.getLong(1),voo,ReservaJdbc.janela(v,"embarque\_abre"),ReservaJdbc.janela(v,"embarque\_fecha")).isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Portao nao cobre a janela de embarque.". Essa condição específica não é substituída por outra validação parecida.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: alocao\_recurso, recurso\_aeroporto. Parâmetros ligados em ordem: voo, ReservaJdbc.janela(v,"embarque\_abre"), ReservaJdbc.janela(v,"embarque\_fecha").

```
40      }
41      case EM_VOO -> {
```

Ramo do switch: case EM\_VOO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
42      Dados.exigir(!relogio.instant().isBefore(ReservaJdbc.janela(v,"embarque_fecha")), "Embarque ainda nao
encerrou.");OperacaoJdbc.validarTripulacao(c,voo);
```

Regra obrigatória: !relogio.instant().isBefore(ReservaJdbc.janela(v,"embarque\_fecha")) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Embarque ainda nao encerrou.". Essa condição específica não é substituída por outra validação parecida.

```
43      Sql.registro(c,"SELECT id FROM aeronave WHERE id=? AND situacao='ATIVA' FOR
SHARE",Dados.id(v,"aeronave_id"));
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeronave. O FOR SHARE mantém lock compartilhado para proteger a validação. Parâmetros ligados em ordem: Dados.id(v,"aeronave\_id").



```
44      Dados.exigir(Sql.listar(c,"SELECT id FROM servico_solo WHERE voo_id=? AND situacao IN
('PENDENTE','EM_EXECUCAO')",r->r.getLong(1),voo).isEmpty(),"Servicos de solo pendentes.");
Regra obrigatória: Sql.listar(c,"SELECT id FROM servico_solo WHERE voo_id=? AND situacao IN
('PENDENTE','EM_EXECUCAO')",r->r.getLong(1),voo).isEmpty() deve resultar em true. Se false, Dados.exigir lança
RegraNegocioException com "Servicos de solo pendentes.". Essa condição específica não é substituída por outra validação
parecida.
```

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: servico\_solo. Parâmetros ligados em ordem: voo.

```
45      for(var item:Sql.listar(c,"SELECT * FROM item_reserva WHERE voo_id=? AND situacao IN
('PENDENTE','CONFIRMADO') ORDER BY id",Sql::linha,voo)) {
Percorre cada elemento da coleção/vetor: for(var item:Sql.listar(c,"SELECT * FROM item_reserva WHERE voo_id=? AND
situacao IN ('PENDENTE','CONFIRMADO') ORDER BY id",Sql::linha,voo)). return, break ou exceção podem encerrar esse
caminho; continue passa à próxima volta.
```

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: voo.

```
46      String
estado="CONFIRMADO".equals(item.get("situacao"))?"NAO_COMPARECEU":"EXPIRADO";
Declara variável local estado do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
Escolha de valor: "CONFIRMADO".equals(item.get("situacao")). true escolhe "NAO_COMPARECEU"; false escolhe "EXPIRADO".
Só a alternativa escolhida é avaliada.
```

```
47      Sql.executar(c,"UPDATE item_reserva SET situacao=? WHERE id=?",estado,Dados.id(item,"id"));reservas.invalidate(c,Dados.id(item,"id"),relogio.instant(),s.usuarioId());
Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: item_reserva.
Parâmetros ligados em ordem: estado, Dados.id(item,"id").
```

```
48      reservas.recalcular(c,Dados.id(item,"reserva_id"));
Chama ReservaJdbc.recalcular: Ordem das condições importa: todos expirados, todos cancelados/expirados, alguns
cancelados/expirados, todos usados/ausentes, algum pendente, caso contrário confirmada.
```

```
49      }
50      Sql.executar(c,"UPDATE voo SET partida_real=? WHERE id=?",relogio.instant(),voo);
Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: voo. Parâmetros
ligados em ordem: relogio.instant(), voo.
```

```
51      planejamento.agendas(c,v,relogio.instant(),PlanejamentoJdbc.efetivo(v,"chegada"));
Chama PlanejamentoJdbc.agendas: Calcula deltas de partida/chegada e desloca agenda de aeronave, recursos, escalas e
serviços pendentes; CHEGADA usa delta de chegada.
```

```
52      }
53      case CONCLUIDO -> {
Ramo do switch: case CONCLUIDO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
54      Dados.exigir(v.get("chegada_real")!=null &&
!Dados.instante(v,"chegada_real").isAfter(relogio.instant()),"Registre a chegada real antes de
concluir.");
Regra obrigatória: v.get("chegada_real")!=null && !Dados.instante(v,"chegada_real").isAfter(relogio.instant()) deve resultar em
true. Se false, Dados.exigir lança RegraNegocioException com "Registre a chegada real antes de concluir.". Essa condição
específica não é substituída por outra validação parecida.
```

```
55      Sql.executar(c,"UPDATE agenda_aeronave SET ativa=false WHERE
voo_id=?",voo);Sql.executar(c,"UPDATE alocao_recurso SET ativa=false WHERE
voo_id=?",voo);Sql.executar(c,"UPDATE escala_funcionario SET ativa=false WHERE voo_id=?",voo);
Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: agenda_aeronave.
Parâmetros ligados em ordem: voo.
Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: alocao_recurso.
Parâmetros ligados em ordem: voo.
Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: escala_funcionario.
Parâmetros ligados em ordem: voo.
```

```
56      }
57      case CANCELADO -> {
Ramo do switch: case CANCELADO. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
58      for(long reserva:ids) {
Percorre cada elemento da coleção/vetor: for(long reserva:ids). return, break ou exceção podem encerrar esse caminho; continue
passa à próxima volta.
```

```
59         BigDecimal valor=BigDecimal.ZERO;
```

Declara variável local valor do tipo BigDecimal. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
60         for(var item:Sql.listar(c,"SELECT * FROM item_reserva WHERE voo_id=? AND
reserva_id=? ORDER BY id",Sql::linha,voo,reserva))
valor=valor.add(reservas.cancelarItem(c,item,religio.instant(),razao,true,s.usuarioId()));
```

Percorre cada elemento da coleção/vetor: for(var item:Sql.listar(c,"SELECT \* FROM item\_reserva WHERE voo\_id=? AND reserva\_id=? ORDER BY id",Sql::linha,voo,reserva)). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: item\_reserva. Parâmetros ligados em ordem: voo, reserva.

Atualiza valor usando = e o resultado de valor.add(reservas.cancelarItem(c,item,religio.instant(),razao,true,s.usuarioId())). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
61     reservas.solicitarDevolucao(c,reserva,valor,"voo:"+voo+": "+reserva,razao);reservas.recalcular(c,reserva);
```

Chama ReservaJdbc.solicitarDevolucao: Limita o solicitado ao recebido livre. Divide entre pagamentos aprovados, desconta reembolsos comprometidos e usa chave por pagamento para evitar repetição.

```
62     }
63     Sql.executar(c,"UPDATE agenda_aeronave SET ativa=false WHERE
voo_id=?",voo);Sql.executar(c,"UPDATE alocao_recurso SET ativa=false WHERE
voo_id=?",voo);Sql.executar(c,"UPDATE escala_funcionario SET ativa=false WHERE voo_id=?",voo);
```

Instrução idêntica à de VooService:L55. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
64     Sql.executar(c,"UPDATE servico_solo SET situacao='CANCELADO' WHERE voo_id=? AND
situacao IN ('PENDENTE','EM_EXECUCAO')",voo);
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: servico\_solo. Parâmetros ligados em ordem: voo.

```
65     var rota=Sql.registro(c,"SELECT origem_id FROM rota WHERE
id=?",Dados.id(v,"rota_id"));
```

Instrução idêntica à de AtendimentoServico:L68. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
66     Sql.executar(c,"INSERT INTO
ocorrencia_operacional(aeroporto_id,voo_id,usuario_id,tipo,gravidade,descricao,aberta_em,encerrada_em)
VALUES (?, ?, ?, 'OUTRA', 'MEDIA', ?, ?, ?)",Dados.id(rota,"origem_id"),voo,s.usuarioId(),"Cancelamento:
"+razao,religio.instant(),religio.instant());
```

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: ocorrencia\_operacional. Parâmetros ligados em ordem: Dados.id(rota,"origem\_id"), voo, s.usuarioId(), "Cancelamento: "+razao, religio.instant(), religio.instant().

```
67     }
68     default -> { }
```

Ramo do switch: default. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
69     }
70     auditoria.definirAutorDoVoo(c,s.usuarioId(),razao);Sql.executar(c,"UPDATE voo SET situacao=?
WHERE id=?",proxima,voo);auditoria.registrar(c,s.usuarioId(),"TRANSICAO_VOO","voo",voo);return null;
```

Registra o evento de auditoria na conexão fornecida: definirAutorDoVoo(c, s.usuarioId(), razao).

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: voo. Parâmetros ligados em ordem: proxima, voo.

Registra o evento de auditoria na conexão fornecida: registrar(c, s.usuarioId(), "TRANSICAO\_VOO", "voo", voo).

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```
71     });
72     }
73 }
```



# Java: terminal e menus

## Terminal

src/main/java/br/edu/aeroporto/cli/Terminal.java

Converte a entrada de texto em valores Java e repete a pergunta enquanto a entrada não for válida. Também detecta fim da entrada e fecha o Scanner.

107 linhas | 11 métodos/construtores declarados | SHA-256: cb829cf5ec6f967c...

```
1 package br.edu.aeroporto.cli;
    Declara o pacote deste arquivo: br.edu.aeroporto.cli. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import java.io.Console;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

4 import java.time.LocalDate;
5 import java.time.format.DateTimeParseException;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

6 import java.util.Scanner;
    Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

7
8 public final class Terminal implements AutoCloseable {
    Declara Terminal como tipo com campos e métodos. final impede herança. implements liga o tipo aos contratos citados.

9     private final Console console = System.console();
    Campo(s) console: private restringe acesso à classe. Tipo Console: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

10    private final Scanner entrada = new Scanner(System.in, java.nio.charset.StandardCharsets.UTF_8);
    Campo(s) entrada: private restringe acesso à classe. Tipo Scanner: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

    new constrói Scanner: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

11
```

### ler - L12 a L14

Chama lerBruto e remove espaços nas extremidades com strip; devolve String.

Entradas declaradas: (String rotulo).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: Main.entrar (L143); Main.inicializar (L156); MenuOperacional.acesso (L63); MenuOperacional.arquivos (L177); MenuOperacional.atendimento (L138); MenuOperacional.cadastros (L50); MenuOperacional.executar (L35); MenuOperacional.financeiro (L127); MenuOperacional.operacao (L150); MenuOperacional.passageiros (L44); MenuOperacional.perfis (L61); MenuOperacional.planejamento (L77); MenuOperacional.programacao (L72); MenuOperacional.reservas (L95); MenuOperacional.trecho (L93); MenuPrincipal.buscarPassageiros (L108); MenuPrincipal.cadastrarPassageiro (L100); MenuPrincipal.mostrarReserva (L147); Terminal.data (L61); Terminal.decimal (L78); Terminal.id (L49); Terminal.idOpcional (L86); Terminal.instante (L71); Terminal.inteiro (L37); Terminal.sim (L94).

Chamadas observadas no corpo: lerBruto(rotulo).strip, lerBruto.

```
12 public String ler(String rotulo) {
    Início da declaração de ler. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

13     return lerBruto(rotulo).strip();
    Retorno: entrega lerBruto(rotulo).strip() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

    Chama lerBruto(rotulo).strip: Remove espaços em branco das extremidades e devolve outra String.

    Chama Terminal.lerBruto: Usa Console quando disponível. Caso contrário imprime o rótulo e usa Scanner. Se a entrada terminar, lança FimDaEntrada; não devolve uma linha inventada.

14 }
15
```

## lerBruto - L16 a L25

Usa Console quando disponível. Caso contrário imprime o rótulo e usa Scanner. Se a entrada terminar, lança FimDaEntrada; não devolve uma linha inventada.

Entradas declaradas: (String rotulo).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: Terminal.ler (L12); Terminal.senha (L27).

Chamadas observadas no corpo: console.readLine, new FimDaEntrada, System.out.print, entrada.hasNextLine, entrada.nextLine.

```

16     private String lerBruto(String rotulo) {
        Início da declaração de lerBruto. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

17         if (console != null) {
            Decisão específica: (console != null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

18             String linha = console.readLine("%s: ", rotulo);
            Declara variável local linha do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

19             if (linha == null) throw new FimDaEntrada();
            Decisão específica: (linha == null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a
            operação lançando a exceção.

            Lança new FimDaEntrada(). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.
            new constrói FimDaEntrada: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
            si só.

20             return linha;
            Retorno: entrega linha ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
            caminho.

21         }
22         System.out.print(rotulo + ": ");
        Invoca System.out.print com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
        método para localizar o corpo no projeto.

23         if (!entrada.hasNextLine()) throw new FimDaEntrada();
        Decisão específica: (!entrada.hasNextLine()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo
        interrompe a operação lançando a exceção.

        Lança new FimDaEntrada(). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.
        new constrói FimDaEntrada: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
        si só.

24         return entrada.nextLine();
        Retorno: entrega entrada.nextLine() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por
        esse caminho.

        Invoca entrada.nextLine com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
        método para localizar o corpo no projeto.

25     }
26

```

## senha - L27 a L35

Console.readPassword oculta a digitação. Sem Console, usa entrada visível e converte para char[], avisando. Não aplica strip à senha.

Entradas declaradas: (String rotulo).

Saída declarada: char[] - vetor mutável de caracteres, usado para senha.

Possíveis chamadores identificados: Main.entrar (L143); Main.inicializar (L156); MenuOperacional.acesso (L63).

Chamadas observadas no corpo: console.readPassword, new FimDaEntrada, System.out.println, lerBruto(rotulo).toCharArray, lerBruto.

```

27     public char[] senha(String rotulo) {
        Início da declaração de senha. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

28         if (console != null) {
            Decisão específica: (console != null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```

```
29         char[] senha = console.readPassword("%s: ", rotulo);
```

Declara variável local senha do tipo char[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
30         if (senha == null) throw new FimDaEntrada();
```

Decisão específica: (senha == null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new FimDaEntrada(). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback. new constrói FimDaEntrada: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
31         return senha;
```

Retorno: entrega senha ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
32     }
33     System.out.println("Este terminal não oferece entrada oculta; a senha ficará visível ao
34     digitar.");
```

Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```
34     return lerBruto(rotulo).toCharArray();
```

Retorno: entrega lerBruto(rotulo).toCharArray() ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Invoca lerBruto(rotulo).toCharArray com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

Chama Terminal.lerBruto: Usa Console quando disponível. Caso contrário imprime o rótulo e usa Scanner. Se a entrada terminar, lança FimDaEntrada; não devolve uma linha inventada.

```
35     }
36 }
```

## inteiro - L37 a L47

Tenta parseInt dentro de while(true), aceita valor entre mínimo e máximo inclusive e pergunta novamente nos demais casos.

Entradas declaradas: (String rotulo, int minimo, int maximo).

Saída declarada: int - inteiro primitivo de 32 bits, usado para índice, limite ou contagem.

Possíveis chamadores identificados: MenuOperacional.acesso (L63); MenuOperacional.arquivos (L177); MenuOperacional.atendimento (L138); MenuOperacional.cadastros (L50); MenuOperacional.escolher (L185); MenuOperacional.financieiro (L127); MenuOperacional.operacao (L150); MenuOperacional.passageiros (L44); MenuOperacional.planejamento (L77); MenuOperacional.rascunho (L104); MenuOperacional.relatorios (L167); MenuOperacional.reservas (L95); MenuOperacional.trecho (L93); MenuPrincipal.buscarPassageiros (L108); MenuPrincipal.consultarVoos (L121); MenuPrincipal.executar (L45); MenuPrincipal.exportarVoos (L158).

Chamadas observadas no corpo: Integer.parseInt, ler, System.out.printf.

```
37     public int inteiro(String rotulo, int minimo, int maximo) {
```

Início da declaração de inteiro. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
38         while (true) {
```

Repete enquanto a condição permanecer verdadeira: while (true). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
39         try {
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

```
40             int valor = Integer.parseInt(ler(rotulo));
```

Declara variável local valor do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
41             if (valor >= minimo && valor <= maximo) return valor;
```

Decisão específica: (valor >= minimo && valor <= maximo). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega valor ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
42         } catch (NumberFormatException erro) {
```

Trata a(s) exceção(ões) declarada(s) em catch (NumberFormatException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
43             // Entrada incorreta mantém o usuário no campo atual.
```

Comentário/documentação do autor; descreve intenção ou convenção e não executa.

```
44         }
```

```

45      System.out.printf("Informe um número entre %d e %d.%n", minimo, maximo);
      Chama System.out.printf: Formata a saída usando os marcadores da string de formato e os argumentos seguintes.

46      }
47  }
48

```

## id - L49 a L59

Tenta `parseLong` e aceita somente valor positivo. IDs podem ultrapassar a faixa de `int`.

Entradas declaradas: (String rotulo).

Saída declarada: `long` - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: `MenuOperacional.acesso` (L63); `MenuOperacional.atendimento` (L138); `MenuOperacional.cadastros` (L50); `MenuOperacional.financeiro` (L127); `MenuOperacional.operacao` (L150); `MenuOperacional.passageiros` (L44); `MenuOperacional.planejamento` (L77); `MenuOperacional.programacao` (L72); `MenuOperacional.rascunho` (L104); `MenuOperacional.relatorios` (L167); `MenuOperacional.reservas` (L95); `MenuOperacional.trecho` (L93); `MenuPrincipal.executar` (L45); `MenuPrincipal.mostrarAssentos` (L138).

Chamadas observadas no corpo: `Long.parseLong`, `ler`, `System.out.println`.

```

49      public long id(String rotulo) {
      Início da declaração de id. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

50      while (true) {
      Repete enquanto a condição permanecer verdadeira: while (true). return, break ou exceção podem encerrar esse caminho;
      continue passa à próxima volta.

51      try {
      Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

52      long valor = Long.parseLong(ler(rotulo));
      Declara variável local valor do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

53      if (valor > 0) return valor;
      Decisão específica: (valor > 0). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a
      execução atual com return.

      Retorno: entrega valor ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
      caminho.

54      } catch (NumberFormatException erro) {
      Trata a(s) exceção(ões) declarada(s) em catch (NumberFormatException erro). O tratamento mostrado abaixo decide se avisa,
      repete ou relança.

55      // IDs podem exceder a capacidade de int.
      Comentário/documentação do autor; descreve intenção ou convenção e não executa.

56      }
57      System.out.println("Informe um id positivo.");
      Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

58      }
59  }
60

```

## data - L61 a L69

Tenta `LocalDate.parse` em formato ISO. A captura de `DateTimeParseException` mantém a pergunta ativa.

Entradas declaradas: (String rotulo).

Saída declarada: `LocalDate` - data de calendário, sem hora ou fuso.

Possíveis chamadores identificados: `Main.inicializar` (L156); `MenuOperacional.operacao` (L150); `MenuOperacional.passageiros` (L44); `MenuOperacional.relatorios` (L167); `MenuPrincipal.cadastrarPassageiro` (L100); `MenuPrincipal.consultarVoos` (L121).

Chamadas observadas no corpo: `LocalDate.parse`, `ler`, `System.out.println`.

```

61      public LocalDate data(String rotulo) {
      Início da declaração de data. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
      invocado.

62      while (true) {
      Repete enquanto a condição permanecer verdadeira: while (true). return, break ou exceção podem encerrar esse caminho;
      continue passa à próxima volta.

63      try {
      Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

```

```

64         return LocalDate.parse(ler(rotulo + " (AAAA-MM-DD)"));
Retorno: entrega LocalDate.parse(ler(rotulo + " (AAAA-MM-DD)")) ao chamador desta execução; nenhuma instrução posterior
desse mesmo corpo executa por esse caminho.

Invoca LocalDate.parse com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
método para localizar o corpo no projeto.

65     } catch (DateTimeParseException erro) {
Trata a(s) exceção(ões) declarada(s) em catch (DateTimeParseException erro). O tratamento mostrado abaixo decide se avisa,
repete ou relança.

66         System.out.println("Data inválida. Exemplo: 2000-06-15.");
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

67     }
68 }
69 }
70

```

## instante - L71 a L76

Lê `OffsetDateTime` com deslocamento explícito e converte para `Instant`; não aceita horário sem indicação de fuso.

Entradas declaradas: (String rotulo).

Saída declarada: `java.time.Instant` - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: `MenuOperacional.operacao` (L150); `MenuOperacional.planejamento` (L77); `MenuOperacional.programacao` (L72).

Chamadas observadas no corpo: `java.time.OffsetDateTime.parse(ler(rotulo+" (AAAA-MM-DDTHH:mm:ss-03:00))).toInstant`,  
`java.time.OffsetDateTime.parse, ler, System.out.println`.

```

71     public java.time.Instant instante(String rotulo) {
Início da declaração de instante. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
invocado.

72         while(true) {
Repete enquanto a condição permanecer verdadeira: while(true). return, break ou exceção podem encerrar esse caminho;
continue passa à próxima volta.

73         try { return java.time.OffsetDateTime.parse(ler(rotulo+"
(AAAA-MM-DDTHH:mm:ss-03:00))).toInstant(); }
Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Retorno: entrega java.time.OffsetDateTime.parse(ler(rotulo+" (AAAA-MM-DDTHH:mm:ss-03:00))).toInstant() ao chamador desta
execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

74         catch(java.time.DateTimeException erro) { System.out.println("Horario invalido; inclua o
fuso."); }
Trata a(s) exceção(ões) declarada(s) em catch(java.time.DateTimeException erro). O tratamento mostrado abaixo decide se
avisa, repete ou relança.

75     }
76 }
77

```

## decimal - L78 a L84

Troca vírgula por ponto, constrói `BigDecimal` e aceita valor não negativo com até duas casas; validações mais específicas ficam nos serviços.

Entradas declaradas: (String rotulo).

Saída declarada: `java.math.BigDecimal` - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: `MenuOperacional.atendimento` (L138); `MenuOperacional.financieiro` (L127); `MenuOperacional.operacao` (L150);  
`MenuOperacional.planejamento` (L77); `MenuOperacional.trecho` (L93).

Chamadas observadas no corpo: `new java.math.BigDecimal, ler(rotulo).replace, ler, valor.signum, valor.scale, System.out.println`.

```

78     public java.math.BigDecimal decimal(String rotulo) {
Início da declaração de decimal. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
invocado.

79         while(true) {
Repete enquanto a condição permanecer verdadeira: while(true). return, break ou exceção podem encerrar esse caminho;
continue passa à próxima volta.

```

```
80      try { var valor=new java.math.BigDecimal(ler(rotulo).replace(',','.'));if(valor.signum()>=0 &&
valor.scale()<=2) return valor; }
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Declara variável local valor com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Decisão específica: (valor.signum()>=0 && valor.scale()<=2). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega valor ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói java.math.BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
81      catch(NumberFormatException erro) { }
```

Trata a(s) exceção(ões) declarada(s) em catch(NumberFormatException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
82      System.out.println("Informe valor nao negativo com ate duas casas decimais.");
```

Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```
83      }
84  }
85
```

## idOpcional - L86 a L92

Entrada vazia devolve null; entrada numérica positiva devolve Long. A ausência só pode ser representada pelo tipo objeto Long.

Entradas declaradas: (String rotulo).

Saída declarada: Long - objeto inteiro de 64 bits; pode ser null quando a ausência é permitida.

Possíveis chamadores identificados: MenuOperacional.operacao (L150); MenuOperacional.passageiros (L44); MenuOperacional.planejamento (L77); MenuOperacional.reservas (L95).

Chamadas observadas no corpo: ler, texto.isBlank, Long.parseLong, System.out.println.

```
86      public Long idOpcional(String rotulo) {
```

Início da declaração de idOpcional. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
87      while(true) {
```

Repete enquanto a condição permanecer verdadeira: while(true). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
88      String texto=ler(rotulo+" (vazio se nao se aplica));if(texto.isBlank()) return null;
```

Declara variável local texto do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Decisão específica: (texto.isBlank()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

```
89      try { long id=Long.parseLong(texto);if(id>0) return id; } catch(NumberFormatException erro) {
```

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Decisão específica: (id>0). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Trata a(s) exceção(ões) declarada(s) em catch(NumberFormatException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.

```
90      System.out.println("Informe id positivo ou deixe vazio.");
```

Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```
91      }
92  }
93
```

## sim - L94 a L101

Aceita s/n sem distinguir maiúsculas; devolve true/false e repete a pergunta em qualquer outro texto.

Entradas declaradas: (String rotulo).

Saída declarada: boolean - condição verdadeira ou falsa.

Possíveis chamadores identificados: MenuOperacional.acesso (L63); MenuOperacional.financeiro (L127); MenuOperacional.passageiros (L44); MenuOperacional.planejamento (L77); MenuOperacional.relatorios (L167).

Chamadas observadas no corpo: ler, texto.equalsIgnoreCase, System.out.println.

```

94     public boolean sim(String rotulo) {
        Início da declaração de sim. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

95         while(true) {
            Repete enquanto a condição permanecer verdadeira: while(true). return, break ou exceção podem encerrar esse caminho;
            continue passa à próxima volta.

96             String texto=ler(rotulo+" (s/n)");
            Declara variável local texto do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

97             if(texto.equalsIgnoreCase("s")) return true;
            Decisão específica: (texto.equalsIgnoreCase("s")). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo
            encerra a execução atual com return.

            Retorno: entrega true ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

98             if(texto.equalsIgnoreCase("n")) return false;
            Decisão específica: (texto.equalsIgnoreCase("n")). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo
            encerra a execução atual com return.

            Retorno: entrega false ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse
            caminho.

99             System.out.println("Informe s ou n.");
            Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

100         }
101     }
102 
```

## close - L103 a L104

Fecha Scanner. O try com recursos no Main faz a chamada automaticamente ao sair do escopo.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: entrada.close.

```

103     @Override
        Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a
        anotação mostrada.

104     public void close() { entrada.close(); }
        Nesta parte da expressão, chama entrada.close. Os valores retornados alimentam a instrução que começou nesta linha ou nas
        linhas anteriores.

105
106     public static final class FimDaEntrada extends RuntimeException {}
        Declara FimDaEntrada como tipo com campos e métodos. final impede herança. extends define a superclasse.

107 }
```

## MenuPrincipal

src/main/java/br/edu/aeroporto/cli/MenuPrincipal.java

Apresenta as opções 0 a 18, coordena consultas básicas e encaminha os módulos completos ao MenuOperacional. Captura erros para manter a sessão no menu.

168 linhas | 10 métodos/construtores declarados | SHA-256: c21cb107a14d2811...

Dependências guardadas em campos: ConsultaServico, MenuOperacional, PassageiroServico, RelatorioServico, Sessao, Terminal. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```
1 package br.edu.aeroporto.cli;
```

Declara o pacote deste arquivo: br.edu.aeroporto.cli. O nome completo distingue a classe de outras com o mesmo nome.

```
2
3 import br.edu.aeroporto.dominio.CadastroPassageiro;
4 import br.edu.aeroporto.dominio.Passageiro;
5 import br.edu.aeroporto.dominio.Sessao;
6 import br.edu.aeroporto.dto.VooResumo;
7 import br.edu.aeroporto.excecao.PersistenciaException;
8 import br.edu.aeroporto.excecao.RegraNegocioException;
9 import br.edu.aeroporto.relatorio.ExportadorCsv;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Implementação CSV com ponto e vírgula, aspas escapadas e proteção contra fórmulas; converte os horários para o fuso de cada aeroporto.

```
10 import br.edu.aeroporto.relatorio.ExportadorTxt;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Implementação de relatório textual da página consultada, com contagem ao final.

```
11 import br.edu.aeroporto.relatorio.ExportadorVoos;
12 import br.edu.aeroporto.servico.ConsultaServico;
13 import br.edu.aeroporto.servico.PassageiroServico;
14 import br.edu.aeroporto.servico.RelatorioServico;
15 import java.io.IOException;
16 import java.sql.SQLException;
17 import java.time.ZoneId;
18 import java.time.format.DateTimeFormatter;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
19 import java.util.List;
20 import java.util.logging.Level;
21 import java.util.logging.Logger;
22
23 public final class MenuPrincipal {
```

Declara MenuPrincipal como tipo com campos e métodos. final impede herança.

```
24     private static final Logger LOG = Logger.getLogger(MenuPrincipal.class.getName());
```

Campo(s) LOG: private restringe acesso à classe. Tipo Logger: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

```
25     private static final DateTimeFormatter DATA = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm XXX");
```

Campo(s) DATA: private restringe acesso à classe. Tipo DateTimeFormatter: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

```
26     private final Terminal terminal;
```

Campo(s) terminal: private restringe acesso à classe. Tipo Terminal: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```
27     private final Sessao sessao;
```

Campo(s) sessao: private restringe acesso à classe. Tipo Sessao: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```
28     private final PassageiroServico passageiros;
```

Campo(s) passageiros: private restringe acesso à classe. Tipo PassageiroServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```
29     private final ConsultaServico consultas;
```

Campo(s) consultas: private restringe acesso à classe. Tipo ConsultaServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.



```
30     private final RelatorioServico relatorios;
```

Campo(s) relatorios: private restringe acesso à classe. Tipo RelatorioServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```
31     private final ZoneId fuso;
```

Campo(s) fuso: private restringe acesso à classe. Tipo ZoneId: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```
32     private final MenuOperacional operacional;
```

Campo(s) operacional: private restringe acesso à classe. Tipo MenuOperacional: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```
33
```

## MenuPrincipal - L34 a L43

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (Terminal terminal, Sessao sessao, PassageiroServico passageiros, ConsultaServico consultas, RelatorioServico relatorios, ZoneId fuso, MenuOperacional operacional).

Possíveis chamadores identificados: Main.main (L55).

```
34     public MenuPrincipal(Terminal terminal, Sessao sessao, PassageiroServico passageiros,
```

Início da declaração de MenuPrincipal. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
35         ConsultaServico consultas, RelatorioServico relatorios, ZoneId fuso,
MenuOperacional operacional) {
```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
36         this.terminal = terminal;
```

Guarda no campo terminal da instância atual o valor de terminal. O this diferencia o campo de parâmetros com o mesmo nome.

```
37         this.sessao = sessao;
```

Guarda no campo sessao da instância atual o valor de sessao. O this diferencia o campo de parâmetros com o mesmo nome.

```
38         this.passageiros = passageiros;
```

Guarda no campo passageiros da instância atual o valor de passageiros. O this diferencia o campo de parâmetros com o mesmo nome.

```
39         this.consultas = consultas;
```

Instrução idêntica à de ConsultaServico:L24. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
40         this.relatorios = relatorios;
```

Guarda no campo relatorios da instância atual o valor de relatorios. O this diferencia o campo de parâmetros com o mesmo nome.

```
41         this.fuso = fuso;
```

Guarda no campo fuso da instância atual o valor de fuso. O this diferencia o campo de parâmetros com o mesmo nome.

```
42         this.operacional=operacional;
```

Guarda no campo operacional da instância atual o valor de operacional. O this diferencia o campo de parâmetros com o mesmo nome.

```
43     }
```

```
44
```

## executar - L45 a L98

O do/while mostra o menu ao menos uma vez. A opção 0 desativa continuar; 1 a 7 chamam ações básicas; o default encaminha 8 a 18. Erros de regra, formato, JDBC e arquivo são tratados sem encerrar a sessão.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: System.out.println, terminal.inteiro, cadastrarPassageiro, buscarPassageiros, mostrar, passageiros.buscar, terminal.id, mostrarVoos, consultarVoos, mostrarAssentos, mostrarReserva, exportarVoos, operacional.executar, erro.getMessage, erro.getCause, LOG.log, sql.getSQLState, erro.getClass().getSimpleName, erro.getClass.

```
45     public void executar() {
```

Início da declaração de executar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

|    |                                                       |                                                                                                                                                                                                |
|----|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 46 | <code>boolean continuar = true;</code>                | Declara variável local continuar do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                         |
| 47 | <code>do {</code>                                     | Executa o bloco antes de verificar se repete: do. return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.                                                        |
| 48 | <code>System.out.println("""</code>                   | Instrução idêntica à de Main:L57. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre. |
| 49 |                                                       |                                                                                                                                                                                                |
| 50 | <code>SISTEMA DE AEROPORTO</code>                     | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 51 | <code>1 – Cadastrar passageiro</code>                 | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 52 | <code>2 – Buscar passageiros por nome</code>          | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 53 | <code>3 – Consultar passageiro por id</code>          | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 54 | <code>4 – Consultar voos por data</code>              | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 55 | <code>5 – Consultar mapa de assentos</code>           | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 56 | <code>6 – Consultar reserva por localizador</code>    | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 57 | <code>7 – Exportar página de voos (TXT/CSV)</code>    | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 58 | <code>8 – Editar passageiros e vincular papeis</code> | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 59 | <code>9 – Cadastros administrativos</code>            | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 60 | <code>10 – Usuarios, acesso e senha</code>            | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 61 | <code>11 – Planejamento e situacao de voos</code>     | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 62 | <code>12 – Reservas, conexoes e remarcacao</code>     | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 63 | <code>13 – Pagamentos e reembolsos simulados</code>   | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 64 | <code>14 – Check-in, bagagens e embarque</code>       | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |
| 65 | <code>15 – Equipe, recursos e operacao</code>         | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                         |

|    |                                                                                 |                                                                                                                                                                                                                                                  |
|----|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 66 | 16 – Paineis e relatorios completos                                             | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                                           |
| 67 | 17 – Importacao e copia de arquivos                                             | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                                           |
| 68 | 18 – Dados FICTICIOS de demonstracao                                            | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                                           |
| 69 | 0 – Sair                                                                        | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                                           |
| 70 | """);                                                                           | Abre/fecha um text block Java: texto de várias linhas usado na mensagem ou no SQL. A instrução que o recebe determina seu uso.                                                                                                                   |
| 71 | try {                                                                           | Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.                                                                                                                                                               |
| 72 | int opcaoAtual=terminal.inteiro("Opção",0,18);                                  | Declara variável local opcaoAtual do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                              |
| 73 | switch (opcaoAtual) {                                                           | Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.                                                                           |
| 74 | case 0 -> continuar = false;                                                    | Ramo do switch: case 0. A instrução associada atende este valor; -> não cai automaticamente no próximo case.<br>Atualiza continuar usando = e o resultado de false. Atribuição modifica a variável/célula indicada; não é um teste de igualdade. |
| 75 | case 1 -> cadastrarPassageiro();                                                | Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.                                                                                                                                     |
| 76 | case 2 -> buscarPassageiros();                                                  | Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.                                                                                                                                     |
| 77 | case 3 -> mostrar(passageiros.buscar(sessao, terminal.id("Id do passageiro"))); | Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.                                                                                                                                     |
| 78 | case 4 -> mostrarVoos(consultarVoos());                                         | Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.                                                                                                                                     |
| 79 | case 5 -> mostrarAssentos();                                                    | Ramo do switch: case 5. A instrução associada atende este valor; -> não cai automaticamente no próximo case.                                                                                                                                     |
| 80 | case 6 -> mostrarReserva();                                                     | Ramo do switch: case 6. A instrução associada atende este valor; -> não cai automaticamente no próximo case.                                                                                                                                     |
| 81 | case 7 -> exportarVoos();                                                       | Ramo do switch: case 7. A instrução associada atende este valor; -> não cai automaticamente no próximo case.                                                                                                                                     |
| 82 | default -> operacional.executar(opcaoAtual);                                    | Ramo do switch: default. A instrução associada atende este valor; -> não cai automaticamente no próximo case.                                                                                                                                    |
| 83 | }                                                                               |                                                                                                                                                                                                                                                  |
| 84 | } catch (RegraNegocioException erro) {                                          | Trata a(s) exceção(ões) declarada(s) em catch (RegraNegocioException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.                                                                                                     |
| 85 | System.out.println(erro.getMessage());                                          | Instrução idêntica à de Main:L149. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.                                                  |
| 86 | } catch (IllegalArgumentException   java.time.DateTimeException erro) {         | Trata a(s) exceção(ões) declarada(s) em catch (IllegalArgumentException   java.time.DateTimeException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.                                                                    |
| 87 | System.out.println("Formato de entrada invalido. Confira o campo informado.");  | Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.                                                                                                                                       |
| 88 | } catch (PersistenciaException erro) {                                          | Trata a(s) exceção(ões) declarada(s) em catch (PersistenciaException erro). O tratamento mostrado abaixo decide se avisa, repete ou relança.                                                                                                     |

```

89         System.out.println(erro.getMessage());
Instrução idêntica à de Main:L149. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução.
A posição e o contexto do método acima mostram quando ela ocorre.

90         if (erro.getCause() instanceof SQLException sql) {
Decisão específica: (erro.getCause() instanceof SQLException sql). Se for true, executa o ramo seguinte; se for false, pula esse
ramo.

91             LOG.log(Level.WARNING, "Falha JDBC no menu. SQLSTATE={0}", sql.getSQLState());
Invoca LOG.log com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método
para localizar o corpo no projeto.

92         }
93     } catch (IOException erro) {
Trata a(s) exceção(ões) declarada(s) em catch (IOException erro). O tratamento mostrado abaixo decide se avisa, repete ou
relança.

94         System.out.println("Não foi possível acessar o arquivo. Confira a pasta e as
permissões.");
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

95         LOG.log(Level.WARNING, "Falha de exportação: {0}", erro.getClass().getSimpleName());
Invoca LOG.log com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método
para localizar o corpo no projeto.

96     }
97 } while (continuar);
Fecha o bloco anterior e completa/continua a expressão ou abre o próximo bloco indicado nesta linha.

98 }
99

```

## cadastrarPassageiro - L100 a L106

Lê campos, constrói CadastroPassageiro, chama o serviço e mostra o objeto persistido. O id exibido vem do PostgreSQL.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuPrincipal.executar (L45).

Chamadas observadas no corpo: new CadastroPassageiro, terminal.ler, terminal.data, passageiros.cadastrar, System.out.println, salvo.id, mostrar.

```

100     private void cadastrarPassageiro() {
Início da declaração de cadastrarPassageiro. Os parâmetros entram pela chamada; o corpo só executa quando esse
método/construtor é invocado.

101         var cadastro = new CadastroPassageiro(terminal.ler("Nome"), terminal.data("Nascimento"),
Declara variável local cadastro com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
escopo é o bloco atual.

new constrói CadastroPassageiro: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco
por si só.

102         terminal.ler("CPF (opcional)"), terminal.ler("E-mail (opcional)"), terminal.ler("Telefone
(opcional)"));
Nesta parte da expressão, chama terminal.ler. Os valores retornados alimentam a instrução que começou nesta linha ou nas
linhas anteriores.

103         Passageiro salvo = passageiros.cadastrar(sessao, cadastro);
Declara variável local salvo do tipo Passageiro. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
atual.

104         System.out.println("Passageiro cadastrado com id " + salvo.id() + ".");
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

105         mostrar(salvo);
Chama MenuPrincipal.mostrar: Formata um passageiro já obtido. printf não consulta nem altera o banco.

106     }
107

```

## buscarPassageiros - L108 a L114

Lê filtro e página de 20 registros, recebe List<Passageiro> e mostra cada elemento; lista vazia tem mensagem própria.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuPrincipal.executar (L45).

Chamadas observadas no corpo: terminal.ler, terminal.inteiro, passageiros.buscar, resultado.isEmpty, System.out.println, resultado.forEach, MenuPrincipal::mostrar.

```

108     private void buscarPassageiros() {
        Início da declaração de buscarPassageiros. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.
109         String nome = terminal.ler("Parte do nome (vazio para listar)");
        Declara variável local nome do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
110         int pagina = terminal.inteiro("Página (20 registros por página)", 1, 100_000);
        Declara variável local pagina do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
111         List<Passageiro> resultado = passageiros.buscar(sessao, nome, pagina);
        Declara variável local resultado do tipo List<Passageiro>. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
        bloco atual.
112         if (resultado.isEmpty()) System.out.println("Nenhum passageiro nesta página.");
        Decisão específica: (resultado.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.
113         resultado.forEach(MenuPrincipal::mostrar);
        Chama resultado.forEach: Executa a ação fornecida para cada elemento.
114     }
115 
```

## mostrar - L116 a L119

Formata um passageiro já obtido. printf não consulta nem altera o banco.

Entradas declaradas: (Passageiro passageiro).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuPrincipal.buscarPassageiros (L108); MenuPrincipal.cadastrarPassageiro (L100); MenuPrincipal.executar (L45).

Chamadas observadas no corpo: System.out.printf, passageiro.id, passageiro.codigoCliente, passageiro.nome, passageiro.nascimento, passageiro.ativa.

```

116     private static void mostrar(Passageiro passageiro) {
        Início da declaração de mostrar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.
117         System.out.printf("%d | %s | %s | Nascimento: %s | %s\n", passageiro.id(),
        Nesta parte da expressão, chama System.out.printf, passageiro.id. Os valores retornados alimentam a instrução que começou
        nesta linha ou nas linhas anteriores.
118         passageiro.codigoCliente(), passageiro.nome(), passageiro.nascimento(), passageiro.ativa()
        ? "Ativo" : "Inativo");
        Escolha de valor: passageiro.ativa(). true escolhe "Ativo"; false escolhe "Inativo". Só a alternativa escolhida é avaliada.
119     }
120 
```

## consultarVoos - L121 a L125

Lê data e página; passa também o fuso configurado ao serviço e devolve List<VooResumo>.

Entradas declaradas: ().

Saída declarada: List<VooResumo> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuPrincipal.executar (L45); MenuPrincipal.exportarVoos (L158).

Chamadas observadas no corpo: System.out.println, consultas.voos, terminal.data, terminal.inteiro.

```

121     private List<VooResumo> consultarVoos() {
        Início da declaração de consultarVoos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
        é invocado.
122         System.out.println("O filtro usa o dia de partida no fuso " + fuso + ". Exibição usa o fuso de
        cada aeroporto.");
        Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```

```

123         return consultas.voos(sessao, terminal.data("Dia de partida"), fuso,
Retorno: entrega consultas.voos(sessao, terminal.data("Dia de partida"), fuso, terminal.inteiro("Página (20 registros por página)",
1, 100_000)) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

124         terminal.inteiro("Página (20 registros por página)", 1, 100_000));
Nesta parte da expressão, chama terminal.inteiro. Os valores retornados alimentam a instrução que começou nesta linha ou nas
linhas anteriores.

125     }
126

```

## mostrarVoos - L127 a L136

Converte os instantes para os fusos de origem e destino; apresenta situação e marca atrasos calculados pelo DTO.

Entradas declaradas: (List<VooResumo> voos).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuPrincipal.executar (L45).

Chamadas observadas no corpo: voos.isEmpty, System.out.println, System.out.printf, voo.id, voo.companhia, voo.numero, voo.origem, voo.destino, DATA.format, voo.partida().atZoneSameInstant, voo.partida, voo.fusoOrigem, voo.chegada().atZoneSameInstant, voo.chegada, voo.fusoDestino, voo.situacao, voo.atrasado.

```

127     private static void mostrarVoos(List<VooResumo> voos) {
Início da declaração de mostrarVoos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
invocado.

128         if (voos.isEmpty()) System.out.println("Nenhum voo nesta página.");
Decisão específica: (voos.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

129         for (VooResumo voo : voos) {
Percorre cada elemento da coleção/vetor: for (VooResumo voo : voos). return, break ou exceção podem encerrar esse caminho;
continue passa à próxima volta.

130             System.out.printf("%d | %s%s | %s → %s | Partida: %s | Chegada: %s | %s%s%n", voo.id(),
Nesta parte da expressão, chama System.out.printf, voo.id. Os valores retornados alimentam a instrução que começou nesta
linha ou nas linhas anteriores.

131             voo.companhia(), voo.numero(), voo.origem(), voo.destino(),
Nesta parte da expressão, chama voo.companhia, voo.numero, voo.origem, voo.destino. Os valores retornados alimentam a
instrução que começou nesta linha ou nas linhas anteriores.

132             DATA.format(voo.partida().atZoneSameInstant(voo.fusoOrigem())),
Nesta parte da expressão, chama DATA.format, voo.partida().atZoneSameInstant, voo.partida, voo.fusoOrigem. Os valores
retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

133             DATA.format(voo.chegada().atZoneSameInstant(voo.fusoDestino())),
Nesta parte da expressão, chama DATA.format, voo.chegada().atZoneSameInstant, voo.chegada, voo.fusoDestino. Os valores
retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

134             voo.situacao(), voo.atrasado() ? " - ATRASADO" : "";
Escolha de valor: voo.atrasado(). true escolhe " - ATRASADO"; false escolhe "". Só a alternativa escolhida é avaliada.

135         }
136     }
137

```

## mostrarAssentos - L138 a L145

Busca os DTOs e prioriza o texto Bloqueado, depois Ocupado, depois Livre. Inexistência ou falta de inventário produz lista vazia.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuPrincipal.executar (L45).

Chamadas observadas no corpo: consultas.assentos, terminal.id, assentos.isEmpty, System.out.println, assento.bloqueado, assento.ocupado, System.out.printf, assento.codigo, assento.classe.

```

138     private void mostrarAssentos() {
Início da declaração de mostrarAssentos. Os parâmetros entram pela chamada; o corpo só executa quando esse
método/construtor é invocado.

```

```

139         var assentos = consultas.assentos(sessao, terminal.id("Id do voo"));
Declara variável local assentos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
escopo é o bloco atual.

140         if (assentos.isEmpty()) System.out.println("Voo inexistente ou sem inventário de assentos.");
Decisão específica: (assentos.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

141         for (var assento : assentos) {
Percorre cada elemento da coleção/vetor: for (var assento : assentos). return, break ou exceção podem encerrar esse caminho;
continue passa à próxima volta.

142             String situacao = assento.bloqueado() ? "Bloqueado" : assento.ocupado() ? "Ocupado" : "Livre";
Declara variável local situacao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Escolha de valor: assento.bloqueado(). true escolhe "Bloqueado"; false escolhe assento.ocupado() ? "Ocupado" : "Livre". Só a
alternativa escolhida é avaliada.

Escolha de valor: assento.ocupado(). true escolhe "Ocupado"; false escolhe "Livre". Só a alternativa escolhida é avaliada.

143             System.out.printf("%s | %s | %s%n", assento.codigo(), assento.classe(), situacao);
Chama System.out.printf: Formata a saída usando os marcadores da string de formato e os argumentos seguintes.

144         }
145     }
146

```

## mostrarReserva - L147 a L156

Mostra cabeçalho e cada trecho; para assento ausente usa traço. A soma final é histórica, incluindo cancelados.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuPrincipal.executar (L45).

Chamadas observadas no corpo: consultas.reserva, terminal.ler, System.out.printf, reserva.localizador, reserva.situacao, DATA.format, reserva.expiraEm().atZoneSameInstant, reserva.expiraEm, reserva.trechos, trecho.passageiro, trecho.ordem, trecho.voo, trecho.situacao, trecho.assento, trecho.valor().toString, trecho.valor, System.out.println, reserva.totalOriginalPassagens().toString, reserva.totalOriginalPassagens.

```

147     private void mostrarReserva() {
Início da declaração de mostrarReserva. Os parâmetros entram pela chamada; o corpo só executa quando esse
método/construtor é invocado.

148         var reserva = consultas.reserva(sessao, terminal.ler("Localizador"));
Declara variável local reserva com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
escopo é o bloco atual.

149         System.out.printf("Reserva %s | %s | Expira: %s%n", reserva.localizador(), reserva.situacao(),
Nesta parte da expressão, chama System.out.printf, reserva.localizador, reserva.situacao. Os valores retornados alimentam a
instrução que começou nesta linha ou nas linhas anteriores.

150             DATA.format(reserva.expiraEm().atZoneSameInstant(fuso));
Nesta parte da expressão, chama DATA.format, reserva.expiraEm().atZoneSameInstant, reserva.expiraEm. Os valores
retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

151         for (var trecho : reserva.trechos()) {
Percorre cada elemento da coleção/vetor: for (var trecho : reserva.trechos()). return, break ou exceção podem encerrar esse
caminho; continue passa à próxima volta.

152             System.out.printf("%s | Trecho %d | %s | %s | Assento: %s | R$ %s%n", trecho.passageiro(),
Nesta parte da expressão, chama System.out.printf, trecho.passageiro. Os valores retornados alimentam a instrução que
começou nesta linha ou nas linhas anteriores.

153             trecho.ordem(), trecho.voo(), trecho.situacao(), trecho.assento() == null ? "-" :
trecho.assento(), trecho.valor().toString());
Escolha de valor: trecho.assento() == null. true escolhe "-"; false escolhe trecho.assento(). Só a alternativa escolhida é avaliada.

154         }
155         System.out.println("Total original das passagens, incluindo itens cancelados: R$ " +
reserva.totalOriginalPassagens().toString());
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

156     }
157

```



## exportarVoos - L158 a L167

Obtém a página de voos e escolhe ExportadorTxt ou ExportadorCsv por switch. Passa a implementação ao serviço e mostra o Path criado.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuPrincipal.executar (L45).

Chamadas observadas no corpo: consultarVoos, voos.isEmpty, new RegraNegocioException, terminal.inteiro, new ExportadorTxt, new ExportadorCsv, new IllegalStateException, System.out.println, relatorios.exportar.

```

158     private void exportarVoos() throws IOException {
Início da declaração de exportarVoos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
é invocado.

159         List<VooResumo> voos = consultarVoos();
Declara variável local voos do tipo List<VooResumo>. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
bloco atual.

160         if (voos.isEmpty()) throw new RegraNegocioException("Não há voos nesta página para exportar.");
Decisão específica: (voos.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a
operação lançando a exceção.

Lança new RegraNegocioException("Não há voos nesta página para exportar."). Interrompe o caminho normal; a exceção sobe
até uma captura compatível e pode provocar rollback.

new constrói RegraNegocioException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
banco por si só.

161         ExportadorVoos exportador = switch (terminal.inteiro("Formato: 1 TXT / 2 CSV", 1, 2)) {
Declara variável local exportador do tipo ExportadorVoos. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
bloco atual.

162             case 1 -> new ExportadorTxt();
Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

new constrói ExportadorTxt: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si
só.

163             case 2 -> new ExportadorCsv();
Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

new constrói ExportadorCsv: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
si só.

164             default -> throw new IllegalStateException("Formato inválido.");
Ramo do switch: default. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

Lança new IllegalStateException("Formato inválido."). Interrompe o caminho normal; a exceção sobe até uma captura compatível
e pode provocar rollback.

new constrói IllegalStateException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
banco por si só.

165         };
166         System.out.println("Relatório salvo em " + relatorios.exportar(voos, exportador));
Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

167     }
168 }
```



## MenuOperacional

src/main/java/br/edu/aeroporto/cli/MenuOperacional.java

Lê os parâmetros de cada operação e chama o serviço correspondente. Contém o rascunho de reservas em ArrayList e os menus financeiros, operacionais e de arquivos.

187 linhas | 18 métodos/construtores declarados | SHA-256: 5fd94e29e5390811...

Dependências guardadas em campos: AdministracaoServico, ArquivosServico, AtendimentoServico, CadastroServico, DadosFicticiosServico, FinanceiroServico, OperacaoServico, PassageiroServico, PlanejamentoServico, RelatoriosCompletoServico, ReservaServico, Sessao, Terminal, VooServico. Os construtores e atribuições a seguir mostram como essas referências são guardadas.

```

1 package br.edu.aeroporto.cli;
  Declara o pacote deste arquivo: br.edu.aeroporto.cli. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.dto.TipoRelatorio;
5 import br.edu.aeroporto.servico.*;
  Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

6 import java.io.IOException;
7 import java.math.BigDecimal;
8 import java.nio.file.Path;
9 import java.util.*;
10
11 public final class MenuOperacional {
  Declara MenuOperacional como tipo com campos e métodos. final impede herança.

12     private final Terminal t;
  Campo(s) t: private restringe acesso à classe. Tipo Terminal: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

13     private final Sessao s;
  Campo(s) s: private restringe acesso à classe. Tipo Sessao: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

14     private final PassageiroServico passageiros;
  Instrução idêntica à de MenuPrincipal:L28. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

15     private final CadastroServico cadastros;
  Campo(s) cadastros: private restringe acesso à classe. Tipo CadastroServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

16     private final AdministracaoServico acesso;
  Campo(s) acesso: private restringe acesso à classe. Tipo AdministracaoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

17     private final PlanejamentoServico planejamento;
  Campo(s) planejamento: private restringe acesso à classe. Tipo PlanejamentoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

18     private final ReservaServico reservas;
  Campo(s) reservas: private restringe acesso à classe. Tipo ReservaServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

19     private final FinanceiroServico financeiro;
  Campo(s) financeiro: private restringe acesso à classe. Tipo FinanceiroServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

20     private final AtendimentoServico atendimento;
  Campo(s) atendimento: private restringe acesso à classe. Tipo AtendimentoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

21     private final OperacaoServico operacao;
  Campo(s) operacao: private restringe acesso à classe. Tipo OperacaoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

```

22      `private final VooServico voos;`

Campo(s) voos: private restringe acesso à classe. Tipo VooServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

23      `private final RelatoriosCompletoServico relatorios;`

Campo(s) relatorios: private restringe acesso à classe. Tipo RelatoriosCompletoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

24      `private final ArquivosServico arquivos;`

Campo(s) arquivos: private restringe acesso à classe. Tipo ArquivosServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

25      `private final DadosFicticiosServico demo;`

Campo(s) demo: private restringe acesso à classe. Tipo DadosFicticiosServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. final impede reatribuir esse campo depois da inicialização.

26

## MenuOperacional - L27 a L33

Construtor: inicializa os campos da instância com os parâmetros e executa as validações/chamadas indicadas abaixo. Não devolve um valor por return; a expressão new produz a referência ao objeto construído.

Entradas declaradas: (Terminal t,Sessao s,PassageiroServico passageiros,CadastroServico cadastros,AdministracaoServico acesso,PlanejamentoServico planejamento,ReservaServico reservas,FinanceiroServico financeiro,AtendimentoServico atendimento, OperacaoServico operacao,VooServico voos,RelatoriosCompletoServico relatorios,ArquivosServico arquivos,DadosFicticiosServico demo).

Possíveis chamadores identificados: Main.main (L55).

27      `public MenuOperacional(Terminal t,Sessao s,PassageiroServico passageiros,CadastroServico cadastros,AdministracaoServico acesso,`

Início da declaração de MenuOperacional. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

28      `PlanejamentoServico planejamento,ReservaServico reservas,FinanceiroServico financeiro,AtendimentoServico atendimento,`

Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação determina onde a instrução termina.

29      `OperacaoServico operacao,VooServico voos,RelatoriosCompletoServico relatorios,ArquivosServico arquivos,DadosFicticiosServico demo) {`

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

30      `this.t=t;this.s=s;this.passageiros=passageiros;this.cadastros=cadastros;this.acesso=acesso;this.planejamento=planejamento;`

Guarda no campo t da instância atual o valor de t. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo s da instância atual o valor de s. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo passageiros da instância atual o valor de passageiros. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo cadastros da instância atual o valor de cadastros. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo acesso da instância atual o valor de acesso. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo planejamento da instância atual o valor de planejamento. O this diferencia o campo de parâmetros com o mesmo nome.

31      `this.reservas=reservas;this.financeiro=financeiro;this.atendimento=atendimento;this.operacao=operacao;this.voos=voos;`

Guarda no campo reservas da instância atual o valor de reservas. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo financeiro da instância atual o valor de financeiro. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo atendimento da instância atual o valor de atendimento. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo operacao da instância atual o valor de operacao. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo voos da instância atual o valor de voos. O this diferencia o campo de parâmetros com o mesmo nome.

32      `this.relatorios=relatorios;this.arquivos=arquivos;this.demo=demo;`

Guarda no campo relatorios da instância atual o valor de relatorios. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo arquivos da instância atual o valor de arquivos. O this diferencia o campo de parâmetros com o mesmo nome.

Guarda no campo demo da instância atual o valor de demo. O this diferencia o campo de parâmetros com o mesmo nome.

```

33     }
34

```

## executar - L35 a L42

Mapeia números do menu principal para submenus. Opção 18 cria cenário fictício, usando PRINCIPAL quando o nome for vazio.

Entradas declaradas: (int opcao).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuPrincipal.executar (L45).

Chamadas observadas no corpo: passageiros, cadastros, acesso, planejamento, reservas, financeiro, atendimento, operacao, relatorios, arquivos, t.ler, System.out.println, demo.criar, chave.isBlank, new IllegalArgumentException.

```

35     public void executar(int opcao) throws IOException {
        Início da declaração de executar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

```

```

36         switch(opcao) {
            Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```

```

37             case 8 -> passageiros();case 9 -> cadastros();case 10 -> acesso();case 11 -> planejamento();
            Ramo do switch: case 8. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Ramo do switch: case 9. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Ramo do switch: case 10. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Ramo do switch: case 11. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

```

38             case 12 -> reservas();case 13 -> financeiro();case 14 -> atendimento();case 15 -> operacao();
            Ramo do switch: case 12. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Ramo do switch: case 13. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Ramo do switch: case 14. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Ramo do switch: case 15. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

```

39             case 16 -> relatorios();case 17 -> arquivos();case 18 -> { String chave=t.ler("Cenario (vazio:
                PRINCIPAL)");System.out.println(demo.criar(s,chave.isBlank()?"PRINCIPAL":chave)); }
            Ramo do switch: case 16. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Ramo do switch: case 17. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Ramo do switch: case 18. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Declara variável local chave do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
            Escolha de valor: chave.isBlank(). true escolhe "PRINCIPAL"; false escolhe chave. Só a alternativa escolhida é avaliada.

```

```

40             default -> throw new IllegalArgumentException("Modulo desconhecido.");
            Ramo do switch: default. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
            Lança new IllegalArgumentException("Modulo desconhecido."). Interrompe o caminho normal; a exceção sobe até uma captura
            compatível e pode provocar rollback.
            new constrói IllegalArgumentException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no
            banco por si só.

```

```

41         }
42     }
43

```

## passageiros - L44 a L48

Escolhe entre vincular uma pessoa existente e editar/inativar o papel de passageiro. O id opcional de nacionalidade pode ser null.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: t.inteiro, System.out.println, passageiros.vincular, t.id, t.ler, passageiros.editar, t.data, t.idOpcional, t.sim.

```

44     private void passageiros() {
        Início da declaração de passageiros. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

45         int opcao=t.inteiro("1 Editar/inativar / 2 Vincular pessoa existente",1,2);
        Declara variável local opcao do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

46         if(opcao==2) {
            System.out.println(passageiros.vincular(s,t.id("Pessoa"),t.ler("Assistencia")));return; }
            Decisão específica: (opcao==2). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

            return encerra o caminho atual sem entregar valor (void).

47         System.out.println(passageiros.editar(s,t.id("Passageiro"),t.ler("Nome"),t.data("Nascimento"),t.id
            Opcional("Pais de nacionalidade"),t.ler("Assistencia"),t.sim("Papel de passageiro ativo")));
            Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

48     }
49

```

## cadastros - L50 a L59

Escolhe TipoCadastro; listar retorna cedo. Na criação/edição percorre campos do catálogo, converte cada entrada e monta LinkedHashMap. Na edição não muda a chave do funcionário.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: escolher, TipoCadastro.values, t.inteiro, mostrar, cadastros.listar, t.id, new LinkedHashMap<String,Object>, tipo.campos, campo.nome().equals, campo.nome, tipo.chave, valores.put, campo.converter, t.ler, campo.tipo, campo.opcional, System.out.println, cadastros.salvar.

```

50     private void cadastros() {
        Início da declaração de cadastros. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

51         TipoCadastro tipo=escolher(TipoCadastro.values());int opcao=t.inteiro("1 Listar / 2 Cadastrar / 3
        Editar/inativar",1,3);
        Declara variável local tipo do tipo TipoCadastro. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

        Declara variável local opcao do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

52         if(opcao==1) { mostrar(cadastros.listar(s,tipo,t.inteiro("Pagina",1,10000));return; }
        Decisão específica: (opcao==1). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

        return encerra o caminho atual sem entregar valor (void).

53         Long id=opcao==3?t.id("Id do cadastro"):null;var valores=new LinkedHashMap<String,Object>();
        Declara variável local id do tipo Long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        Escolha de valor: opcao==3. true escolhe t.id("Id do cadastro"); false escolhe null. Só a alternativa escolhida é avaliada.

        Declara variável local valores com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        new constrói LinkedHashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
        si só.

54         for(var campo:tipo.campos()) {
            Percorre cada elemento da coleção/vetor: for(var campo:tipo.campos()). return, break ou exceção podem encerrar esse caminho;
            continue passa à próxima volta.

55             if(campo.nome().equals(tipo.chave()) && id!=null) { valores.put(campo.nome(),id);continue; }
            Decisão específica: (campo.nome().equals(tipo.chave()) && id!=null). Se for true, executa o ramo seguinte; se for false, pula esse
            ramo.

56             valores.put(campo.nome(),campo.converter(t.ler(campo.nome())+"
            ["+campo.tipo()+"]"+"(campo.opcional()?" opcional":""));
            Chama valores.put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.

            Escolha de valor: campo.opcional(). true escolhe " opcional"; false escolhe "". Só a alternativa escolhida é avaliada.

57         }
58         System.out.println("Cadastro salvo: "+cadastros.salvar(s,tipo,id,valores));
        Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

59     }

```

60

## perfis - L61 a L61

Normaliza para maiúsculas, remove espaços e divide por vírgula. HashSet elimina perfis repetidos.

Entradas declaradas: ().

Saída declarada: Set<String> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuOperacional.acesso (L63).

Chamadas observadas no corpo: new HashSet<>, Arrays.asList, t.ler("Perfis separados por virgula:

ADMINISTRADOR,ATENDIMENTO,OPERACAO,CONSULTA").toUpperCase(Locale.ROOT).replace(" ", "").split, t.ler("Perfis separados por virgula:

ADMINISTRADOR,ATENDIMENTO,OPERACAO,CONSULTA").toUpperCase(Locale.ROOT).replace, t.ler("Perfis separados por virgula:

ADMINISTRADOR,ATENDIMENTO,OPERACAO,CONSULTA").toUpperCase, t.ler.

```
61 private Set<String> perfis() { return new HashSet<>(Arrays.asList(t.ler("Perfis separados por virgula:
ADMINISTRADOR,ATENDIMENTO,OPERACAO,CONSULTA").toUpperCase(Locale.ROOT).replace(" ", "").split(","))); }
```

Início da declaração de perfis. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega new HashSet<>(Arrays.asList(t.ler("Perfis separados por virgula:

ADMINISTRADOR,ATENDIMENTO,OPERACAO,CONSULTA").toUpperCase(Locale.ROOT).replace(" ", "").split(","))) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói HashSet: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

62

## acesso - L63 a L70

Lê os dados das quatro operações de usuário e repassa ao AdministracaoServico; senha é char[].

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: t.inteiro, mostrar, acesso.usuarios, t.id, t.ler, perfis, System.out.println, acesso.criarUsuario, t.senha, acesso.acesso, t.sim, acesso.alterarSenha.

```
63 private void acesso() {
Início da declaração de acesso. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
```

```
64 switch(t.inteiro("1 Usuarios / 2 Criar usuario / 3 Acesso e perfis / 4 Minha senha",1,4)) {
Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.
```

```
65 case 1 -> mostrar(acesso.usuarios(s));
Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
66 case 2 -> { long funcionario=t.id("Funcionario");String login=t.ler("Login");Set<String>
perfis=perfis();System.out.println("Usuario: "+acesso.criarUsuario(s,funcionario,login,t.senha("Senha
inicial"),perfis)); }
```

Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

Declara variável local funcionario do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local login do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local perfis do tipo Set<String>. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
67 case 3 -> acesso.acesso(s,t.id("Usuario"),t.sim("Acesso ativo"),perfis());
Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
68 case 4 -> acesso.alterarSenha(s,t.senha("Senha atual"),t.senha("Nova senha"));
Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
69 }
70 }
71 }
```

## programacao - L72 a L75

Agrupa ids, número e seis instantes em ProgramacaoVoo. Retorna esse record para programação/reprogramação.

Entradas declaradas: ().

Saída declarada: ProgramacaoVoo - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuOperacional.planejamento (L77).

Chamadas observadas no corpo: t.id, t.ler, new ProgramacaoVoo, t.instante.

```
72     private ProgramacaoVoo programacao() {
        Início da declaração de programacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
        invocado.

73         long companhia=t.id("Companhia"),rota=t.id("Rota"),aeronave=t.id("Aeronave");String
        numero=t.ler("Numero do voo");
        Declara variável local companhia, rota, aeronave do tipo long. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        Declara variável local numero do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

74         return new ProgramacaoVoo(companhia,rota,aeronave,numero,t.instante("Partida
        prevista"),t.instante("Chegada prevista"),t.instante("Check-in abre"),t.instante("Check-in
        fecha"),t.instante("Embarque abre"),t.instante("Embarque fecha"));
        Retorno: entrega new ProgramacaoVoo(companhia,rota,aeronave,numero,t.instante("Partida prevista"),t.instante("Chegada
        prevista"),t.instante("Check-in abre"),t.instante("Check-in fecha"),t.instante("Embarque abre"),t.instante("Embarque... ao
        chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

        new constrói ProgramacaoVoo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco
        por si só.

75     }
76 }
```

## planejamento - L77 a L91

Trata separadamente cada condição de operação: programar, tarifa, bloqueio, reprogramar, trocar aeronave, estimado/real e transição de estado.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: t.inteiro, System.out.println, planejamento.programar, programacao, t.id, t.idOpcional, new Tarifa, t.ler, t.ler("Classe: ECONOMICA,EXECUTIVA,PRIMEIRA").toUpperCase, t.decimal, t.sim, planejamento.tarifa, planejamento.bloquearAssento, planejamento.reprogramar, planejamento.trocarAeronave, planejamento.horarios, t.instante, voos.transicao, escolher, SituacaoVoo.values.

```
77     private void planejamento() {
        Início da declaração de planejamento. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor
        é invocado.

78         switch(t.inteiro("1 Programar / 2 Tarifa / 3 Bloquear assento / 4 Reprogramar / 5 Trocar aeronave
        / 6 Horarios / 7 Situacao",1,7)) {
        Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

79             case 1 -> System.out.println("Voo: "+planejamento.programar(s,programacao()));
        Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

80             case 2 -> {
        Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

81                 long voo=t.id("Voo");Long id=t.idOpcional("Tarifa existente");
        Declara variável local voo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        Declara variável local id do tipo Long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

82                 var tarifa=new Tarifa(t.ler("Codigo"),t.ler("Classe:
        ECONOMICA,EXECUTIVA,PRIMEIRA").toUpperCase(Locale.ROOT),t.decimal("Valor base"),t.decimal("Taxa de
        embarque"),t.decimal("Franquia kg"),t.inteiro("Pecas",0,10),t.sim("Permite cancelar"),t.decimal("Multa"));
        Declara variável local tarifa com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        new constrói Tarifa: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

83                 System.out.println("Tarifa: "+planejamento.tarifa(s,voo,id,tarifa,t.sim("Tarifa ativa")));
        Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

84             }
85             case 3 -> planejamento.bloquearAssento(s,t.id("Inventario do assento"),t.sim("Bloqueado"));
        Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```
86         case 4 -> { long
voo=t.id("Voo");planejamento.reprogramar(s,voo,programacao(),t.ler("Motivo")); }
```

Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

Declara variável local voo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
87         case 5 -> planejamento.trocarAeronave(s,t.id("Voo"),t.id("Nova aeronave"),t.ler("Motivo"));
```

Ramo do switch: case 5. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
88         case 6 ->
planejamento.horarios(s,t.id("Voo"),t.instante("Partida"),t.instante("Chegada"),t.sim("Horarios
reais"),t.ler("Motivo"));
```

Ramo do switch: case 6. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
89         case 7 -> voos.transicao(s,t.id("Voo"),escolher(SituacaoVoo.values()),t.ler("Motivo"));
```

Ramo do switch: case 7. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
90     }
91 }
92
```

## trecho - L93 a L93

Cria PedidoTrecho com passageiro/voo/tarifa/ordem/assento/desconto. Assento vazio é convertido em null pelo record.

Entradas declaradas: ().

Saída declarada: PedidoTrecho - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: MenuOperacional.rascunho (L104); MenuOperacional.reservas (L95).

Chamadas observadas no corpo: new PedidoTrecho, t.id, t.inteiro, t.ler, t.decimal.

```
93     private PedidoTrecho trecho() { return new
PedidoTrecho(t.id("Passageiro"),t.id("Voo"),t.id("Tarifa"),t.inteiro("Ordem do
trecho",1,32767),t.ler("Assento (vazio: automatico)"),t.decimal("Desconto")); }
```

Início da declaração de trecho. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega new PedidoTrecho(t.id("Passageiro"),t.id("Voo"),t.id("Tarifa"),t.inteiro("Ordem do trecho",1,32767),t.ler("Assento (vazio: automatico)"),t.decimal("Desconto")) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
94
```

## reservas - L95 a L102

Encaminha nova reserva ao rascunho; cancelar pode receber item null para toda a reserva; remarcar recebe novo trecho; expirar informa quantidade.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: t.inteiro, rascunho, reservas.cancelar, t.id, t.idOpcional, t.ler, System.out.println, reservas.remarcar, trecho, reservas.expirar.

```
95     private void reservas() {
```

Início da declaração de reservas. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
96         switch(t.inteiro("1 Nova reserva / 2 Cancelar / 3 Remarcar / 4 Expirar pendencias",1,4)) {
Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.
```

```
97         case 1 -> rascunho();
```

Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
98         case 2 -> reservas.cancelar(s,t.id("Reserva"),t.idOpcional("Item (vazio: toda
reserva)"),t.ler("Motivo"));
```

Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
99         case 3 -> { long item=t.id("Item anterior");System.out.println("Nova reserva:
"+reservas.remarcar(s,item,trecho(),t.inteiro("Prazo em minutos",1,120))); }
```

Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

Declara variável local item do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.



```

100         case 4 -> System.out.println("Reservas expiradas: "+reservas.expirar(s));
        Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

101     }
102 }
103

```

## rascunho - L104 a L125

Mantém os itens somente em memória até Gravar. add insere, contains/indexOf detectam repetição, set edita, remove apaga uma posição, subList pagina e clear esvazia. A posição digitada é base 1; a lista usa base 0. Gravar chama o serviço para toda a lista.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.reservas (L95).

Chamadas observadas no corpo: t.id, new ArrayList<PedidoTrecho>, t.inteiro, Dados.exigir, itens.size, trecho, itens.contains, itens.indexOf, itens.add, itens.isEmpty, System.out.println, itens.get, itens.set, itens.remove, itens.subList, Math.min, parte.size, parte.get, itens.clear, reservas.reservar.

```

104     private void rascunho() {
        Início da declaração de rascunho. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

105         long comprador=t.id("Pessoa compradora");var itens=new ArrayList<PedidoTrecho>();
        Declara variável local comprador do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        Declara variável local itens com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

106         while(true) {
        Repete enquanto a condição permanecer verdadeira: while(true). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

107             switch(t.inteiro("1 Adicionar / 2 Editar / 3 Remover / 4 Listar / 5 Limpar / 6 Gravar / 0
            Voltar",0,6)) {
        Nesta parte da expressão, chama t.inteiro. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

108                 case 0 -> { return; }
        Ramo do switch: case 0. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

        return encerra o caminho atual sem entregar valor (void).

109                 case 1 -> {
        Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

110                     Dados.exigir(itens.size()<100,"Limite de 100 itens.");var pedido=trecho();
        Regra obrigatória: itens.size()<100 deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Limite de 100 itens.". Essa condição específica não é substituída por outra validação parecida.

        Declara variável local pedido com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

111                     Dados.exigir(!itens.contains(pedido),"Trecho repetido na posicao
                    "+(itens.indexOf(pedido)+1)+".");itens.add(pedido);
        Regra obrigatória: !itens.contains(pedido) deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Trecho repetido na posicao "+(itens.indexOf(pedido)+1)+".". Essa condição específica não é substituída por outra validação parecida.

112                     }
113                     case 2 -> { Dados.exigir(!itens.isEmpty(),"Sem itens.");int indice=t.inteiro("Posicao",1,i
                    tens.size()-1;System.out.println(itens.get(indice));itens.set(indice,trecho()); }
        Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

        Regra obrigatória: !itens.isEmpty() deve resultar em true. Se false, Dados.exigir lança RegraNegocioException com "Sem itens.". Essa condição específica não é substituída por outra validação parecida.

        Declara variável local indice do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

114                     case 3 -> { Dados.exigir(!itens.isEmpty(),"Sem
                    itens.");itens.remove(t.inteiro("Posicao",1,itens.size()-1); }
        Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```



Regra obrigatória: `itens.isEmpty()` deve resultar em `true`. Se `false`, `Dados.exigir` lança `RegraNegocioException` com "Sem itens.". Essa condição específica não é substituída por outra validação parecida.

```

115         case 4 -> {
Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

116             if(itens.isEmpty()) { System.out.println("Sem itens.");break; }
Decisão específica: (itens.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

117             int pagina=t.inteiro("Pagina",1,(itens.size()+19)/20),inicio=(pagina-1)*20;
Declara variável local pagina, início do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

118             var parte=itens.subList(inicio,Math.min(inicio+20,itens.size()));
Declara variável local parte com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

119             for(int i=0;i<parte.size();i++) System.out.println((inicio+i+1)+" | "+parte.get(i));
Laço controlado por inicialização, condição e atualização: for(int i=0;i<parte.size();i++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

120         }
121         case 5 -> itens.clear();
Ramo do switch: case 5. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

122         case 6 -> { System.out.println("Reserva:
"+reservas.reservar(s,comprador,itens,t.inteiro("Prazo em minutos",1,120)));return; }
Ramo do switch: case 6. A instrução associada atende este valor; -> não cai automaticamente no próximo case.
return encerra o caminho atual sem entregar valor (void).

123     }
124 }
125 }
126

```

## financeiro - L127 a L136

Seleciona pagamento, confirmação, saldo, solicitação/processamento de reembolso ou cancelamento pendente; cada ramo pede parâmetros próprios.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: `MenuOperacional.executar` (L35).

Chamadas observadas no corpo: `t.inteiro`, `System.out.println`, `financeiro.pagar`, `t.id`, `t.ler`, `t.ler("Forma: PIX,CARTAO,DINHEIRO").toUpperCase`, `t.decimal`, `t.ler("Resultado: PENDENTE,APROVADO,RECUSADO").toUpperCase`, `financeiro.confirmar`, `financeiro.saldo`, `financeiro.solicitarReembolso`, `financeiro.processarReembolso`, `t.sim`, `financeiro.cancelarPagamento`.

```

127     private void financeiro() {
Início da declaração de financeiro. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

128         switch(t.inteiro("1 Pagamento simulado / 2 Confirmar / 3 Saldo / 4 Solicitar reembolso / 5
Processar reembolso / 6 Cancelar pagamento pendente",1,6)) {
Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

129             case 1 -> System.out.println("Pagamento: "+financeiro.pagar(s,t.id("Reserva"),t.ler("Chave de
idempotencia"),t.ler("Forma:
PIX, CARTAO, DINHEIRO").toUpperCase(Locale.ROOT),t.decimal("Valor"),t.ler("Resultado:
PENDENTE, APROVADO, RECUSADO").toUpperCase(Locale.ROOT)));
Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

130             case 2 -> financeiro.confirmar(s,t.id("Reserva"));
Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

131             case 3 -> System.out.println("Saldo: "+financeiro.saldo(s,t.id("Reserva")));
Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

132             case 4 -> System.out.println("Reembolso:
"+financeiro.solicitarReembolso(s,t.id("Pagamento"),t.decimal("Valor"),t.ler("Chave"),t.ler("Motivo")));
Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

133             case 5 -> financeiro.processarReembolso(s,t.id("Reembolso"),t.sim("Aprovar"));
Ramo do switch: case 5. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

```
134         case 6 -> financeiro.cancelarPagamento(s,t.id("Pagamento"));
```

Ramo do switch: case 6. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
135     }
```

```
136 }
```

```
137
```

## atendimento - L138 a L148

Seleciona check-in, cartão, cancelamento, despacho, rastreo ou embarque. Usa id de item para atendimento e código do cartão para embarque.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: t.inteiro, System.out.println, atendimento.checkin, t.id, atendimento.cartao, atendimento.cancelarCheckin, atendimento.despachar, t.ler, t.decimal, t.ler("Categoria: DESPACHADA,ESPECIAL").toUpperCase, atendimento.rastrear, t.ler("Situacao").toUpperCase, atendimento.embarcar, mostrar, relatorios.bagagem.

```
138     private void atendimento() {
```

Início da declaração de atendimento. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
139         switch(t.inteiro("1 Check-in / 2 Cartao / 3 Cancelar check-in / 4 Despachar / 5 Evento da bagagem
/ 6 Embarcar / 7 Rastreo",1,7)) {
```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
140         case 1 -> System.out.println("Check-in:
"+atendimento.checkin(s,t.id("Item"),t.id("Documento")));
```

Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
141         case 2 -> System.out.println(atendimento.cartao(s,t.id("Item"));
```

Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
142         case 3 -> atendimento.cancelarCheckin(s,t.id("Item"));
```

Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
143         case 4 -> System.out.println("Bagagem: "+atendimento.despachar(s,t.id("Item"),t.ler("Etiqueta
(vazio: gerar")),t.decimal("Peso kg"),t.ler("Categoria: DESPACHADA,ESPECIAL").toUpperCase(Locale.ROOT));
```

Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
144         case 5 -> atendimento.rastrear(s,t.ler("Etiqueta"),t.id("Aeroporto"),t.ler("Situacao").toUpperCase
Case(Locale.ROOT),t.ler("Observacao"));
```

Ramo do switch: case 5. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
145         case 6 -> System.out.println("Embarque: "+atendimento.embarcar(s,t.ler("Codigo do
cartao"),t.id("Alocacao do portao"));
```

Ramo do switch: case 6. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
146         case 7 -> mostrar(relatorios.bagagem(s,t.ler("Etiqueta"));
```

Ramo do switch: case 7. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
147     }
```

```
148 }
```

```
149
```

## operacao - L150 a L165

Distribui 12 operações de equipe/recursos/manutenção/solo/ocorrências. Não implementa as regras dessas operações: lê entradas e chama serviços.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: t.inteiro, System.out.println, operacao.alocar, t.id, t.ler("Finalidade: PARTIDA,CHEGADA,APOIO").toUpperCase, t.ler, t.instante, operacao.interditar, operacao.liberarAlocacao, operacao.habilitar, t.ler("Funcao").toUpperCase, t.data, operacao.escalar, t.idOpcional, operacao.liberarEscala, operacao.manutencao, t.ler("Tipo: PREVENTIVA,CORRETIVA,INSPECAO").toUpperCase, t.decimal, operacao.situacaoManutencao, t.ler("Situacao").toUpperCase, operacao.servicoSolo, t.ler("Tipo").toUpperCase, operacao.situacaoSolo, operacao.ocorrencia, t.ler("Gravidade").toUpperCase, operacao.encerrarOcorrencia.

```
150     private void operacao() {
```

Início da declaração de operacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

151      switch(t.inteiro("1 Alocar / 2 Interditar / 3 Liberar recurso / 4 Habilitacao / 5 Escalar / 6
Liberar escala / 7 Manutencao / 8 Situacao manutencao / 9 Solo / 10 Situacao solo / 11 Ocorrencia / 12
Encerrar ocorrencia",1,12)) {

```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```

152      case 1 -> System.out.println("Alocacao:
"+operacao.alocar(s,t.id("Voo"),t.id("Recurso"),t.ler("Finalidade:
PARTIDA,CHEGADA,APOIO").toUpperCase(Locale.ROOT),t.instante("Inicio"),t.instante("Fim"))));

```

Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

153      case 2 -> System.out.println("Interdicao:
"+operacao.interditar(s,t.id("Recurso"),t.instante("Inicio"),t.instante("Fim"),t.ler("Motivo"))));

```

Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

154      case 3 -> operacao.liberarAlocacao(s,t.id("Alocacao"));

```

Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

155      case 4 -> System.out.println("Habilitacao: "+operacao.habilitar(s,t.id("Funcionario"),t.id("Mo
delo"),t.ler("Funcao").toUpperCase(Locale.ROOT),t.ler("Licenca"),t.data("Validade"))));

```

Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

156      case 5 -> System.out.println("Escala: "+operacao.escalar(s,t.id("Funcionario"),t.id("Aeroporto
"),t.idOpcional("Voo"),t.ler("Funcao").toUpperCase(Locale.ROOT),t.instante("Inicio"),t.instante("Fim"))));

```

Ramo do switch: case 5. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

157      case 6 -> operacao.liberarEscala(s,t.id("Escala"));

```

Ramo do switch: case 6. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

158      case 7 -> System.out.println("Manutencao:
"+operacao.manutencao(s,t.id("Aeronave"),t.id("Responsavel"),t.ler("Tipo: PREVENTIVA,CORRETIVA,INSPECAO").
toUpperCase(Locale.ROOT),t.ler("Descricao"),t.instante("Inicio"),t.instante("Fim"),t.decimal("Custo"))));

```

Ramo do switch: case 7. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

159      case 8 ->
operacao.situacaoManutencao(s,t.id("Manutencao"),t.ler("Situacao").toUpperCase(Locale.ROOT));

```

Ramo do switch: case 8. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

160      case 9 -> System.out.println("Servico: "+operacao.servicoSolo(s,t.id("Voo"),t.id("Aeroporto"),
t.id("Responsavel"),t.ler("Tipo").toUpperCase(Locale.ROOT),t.decimal("Quantidade"),t.ler("Unidade"),t.deci
mal("Custo"))));

```

Ramo do switch: case 9. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

161      case 10 ->
operacao.situacaoSolo(s,t.id("Servico"),t.ler("Situacao").toUpperCase(Locale.ROOT));

```

Ramo do switch: case 10. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

162      case 11 -> System.out.println("Ocorrencia: "+operacao.ocorrencia(s,t.id("Aeroporto"),t.idOpcio
nal("Voo"),t.ler("Tipo").toUpperCase(Locale.ROOT),t.ler("Gravidade").toUpperCase(Locale.ROOT),t.ler("Descr
icao"))));

```

Ramo do switch: case 11. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

163      case 12 -> operacao.encerrarOcorrencia(s,t.id("Ocorrencia"));

```

Ramo do switch: case 12. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```

164      }
165      }
166

```

## relatorios - L167 a L175

Consulta página e pode inverter uma cópia da lista; exportação é completa. Pannel usa aeroporto/data; mapa imprime vetores e matriz; estatísticas mostram mínimo/máximo/mediana.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: t.inteiro, escolher, TipoRelatorio.values, new ArrayList<>, relatorios.consultar, t.sim, Collections.reverse, mostrar, System.out.println, relatorios.exportar, relatorios.painel, t.id, t.data, relatorios.mapa, Arrays.toString, mapa.colunas, mapa.filas, mapa.celulas, String.join, relatorios.resumoAtrasos.

```

167      private void relatorios() throws IOException {

```

Início da declaração de relatorios. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

168      switch(t.inteiro("1 Consultar / 2 Exportar completo / 3 Pannel / 4 Mapa / 5 Estatisticas
atrasos",1,5)) {

```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
169         case 1 -> { TipoRelatorio tipo=escolher(TipoRelatorio.values());var linhas=new
ArrayList<>(relatorios.consultar(s,tipo,t.inteiro("Pagina",1,100000)));if(t.sim("Ordem inversa"))
Collections.reverse(linhas);mostrar(linhas); }
```

Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

Declara variável local tipo do tipo TipoRelatorio. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local linhas com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Decisão específica: (t.sim("Ordem inversa")). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
170         case 2 ->
System.out.println(relatorios.exportar(s,escolher(TipoRelatorio.values()),t.sim("CSV")));
```

Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
171         case 3 -> mostrar(relatorios.painel(s,t.id("Aeroporto"),t.data("Dia"),t.sim("Partidas")));
```

Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
172         case 4 -> { var
mapa=relatorios.mapa(s,t.id("Voo"));System.out.println(Arrays.toString(mapa.colunas()));int[]
filas=mapa.filas();String[][] celulas=mapa.celulas();for(int i=0;i<filas.length;i++)
System.out.println(filas[i]+" | "+String.join(" | ",celulas[i])); }
```

Ramo do switch: case 4. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

Declara variável local mapa com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local filas do tipo int[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local celulas do tipo String[][]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Laço controlado por inicialização, condição e atualização: for(int i=0;i<filas.length;i++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
173         case 5 -> System.out.println(relatorios.resumoAtrasos(s));
```

Ramo do switch: case 5. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
174     }
175 }
176
```

## arquivos - L177 a L183

Converte caminhos digitados em Path e chama importar/listar/copiar. A importação informa quantidade; a cópia devolve destino.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.executar (L35).

Chamadas observadas no corpo: t.inteiro, System.out.println, arquivos.importarPassageiros, Path.of, t.ler, arquivos.listar(s).forEach, arquivos.listar, System.out::println, arquivos.copiarRelatorio.

```
177     private void arquivos() throws IOException {
```

Início da declaração de arquivos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
178         switch(t.inteiro("1 Importar passageiros TXT/TSV / 2 Listar relatorios / 3 Copiar relatorio",1,3))
{
```

Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

```
179         case 1 -> System.out.println("Importados:
"+arquivos.importarPassageiros(s,Path.of(t.ler("Caminho do arquivo"))));
```

Ramo do switch: case 1. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
180         case 2 -> arquivos.listar(s).forEach(System.out::println);
```

Ramo do switch: case 2. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
181         case 3 -> System.out.println(arquivos.copiarRelatorio(s,Path.of(t.ler("Caminho do
relatorio"))));
```

Ramo do switch: case 3. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

```
182     }
```

```

183     }
184

```

## escolher - L185 a L185

Método genérico restrito a Enum<T>. Mostra constantes, lê índice válido e devolve a constante selecionada, descontando 1 da entrada.

Entradas declaradas: (T[] opcoes).

Saída declarada: T - tipo genérico escolhido pelo contexto da chamada.

Possíveis chamadores identificados: MenuOperacional.cadastros (L50); MenuOperacional.planejamento (L77); MenuOperacional.relatorios (L167).

Chamadas observadas no corpo: System.out.println, opcoes[i].name, t.inteiro.

```

185     private <T extends Enum<T>> T escolher(T[] opcoes) { for(int i=0;i<opcoes.length;i++)
System.out.println((i+1)+" | "+opcoes[i].name());return opcoes[t.inteiro("Opcao",1,opcoes.length)-1]; }

```

Início da declaração de escolher. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Laço controlado por inicialização, condição e atualização: for(int i=0;i<opcoes.length;i++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Retorno: entrega opcoes[t.inteiro("Opcao",1,opcoes.length)-1] ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## mostrar - L186 a L186

Mostra registros Map de uma consulta e mensagem para lista vazia.

Entradas declaradas: (List<Map<String,Object>> linhas).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: MenuOperacional.acesso (L63); MenuOperacional.atendimento (L138); MenuOperacional.cadastros (L50); MenuOperacional.relatorios (L167).

Chamadas observadas no corpo: linhas.isEmpty, System.out.println, linhas.forEach, System.out::println.

```

186     private void mostrar(List<Map<String,Object>> linhas) { if(linhas.isEmpty())
System.out.println("Nenhum registro.");linhas.forEach(System.out::println); }

```

Início da declaração de mostrar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Decisão específica: (linhas.isEmpty()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```

187 }

```

# Java: senhas, exceções e exportadores

## PersistenciaException

src/main/java/br/edu/aeroporto/excecao/PersistenciaException.java

Traduz SQLSTATE do JDBC em mensagens de aplicação e preserva a SQLException como causa para diagnóstico e repetição controlada.

25 linhas | 2 métodos/construtores declarados | SHA-256: e97addea6511d99d...

```
1 package br.edu.aeroporto.excecao;
    Declara o pacote deste arquivo: br.edu.aeroporto.excecao. O nome completo distingue a classe de outras com o mesmo nome.
2
3 import java.sql.SQLException;
4
5 public final class PersistenciaException extends RuntimeException {
    Declara PersistenciaException como tipo com campos e métodos. final impede herança. extends define a superclasse.
```

### PersistenciaException - L6 a L8

Chama o construtor da exceção com a mensagem traduzida do SQLSTATE e preserva SQLException como causa para diagnóstico, sem mostrar ao operador o comando SQL bruto.

Entradas declaradas: (SQLException causa).

Possíveis chamadores identificados: BancoDados.executar (L36).

Chamadas observadas no corpo: super, mensagem, causa.getSQLState.

```
6 public PersistenciaException(SQLException causa) {
    Início da declaração de PersistenciaException. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
7     super(mensagem(causa.getSQLState()), causa);
    Inicializa a superclasse com os argumentos mostrados; o objeto mantém os campos herdados e os específicos desta classe.
8 }
9
```

### mensagem - L10 a L24

Switch por SQLSTATE diferencia duplicidade, FK, CHECK/NOT NULL, agenda, concorrência, schema, autenticação e conexão. Preserva mensagem amigável sem imprimir SQL bruto.

Entradas declaradas: (String estado).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: PersistenciaException.PersistenciaException (L6).

Chamadas observadas no corpo: estado.startsWith.

```
10 private static String mensagem(String estado) {
    Início da declaração de mensagem. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
11     if (estado == null) return "Não foi possível acessar o banco de dados.";
    Decisão específica: (estado == null). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.
    Retorno: entrega "Não foi possível acessar o banco de dados." ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.
12     return switch (estado) {
    Retorno: entrega switch (estado) { case "23505" -> "Já existe um registro com essa identificação."; case "23503" -> "Um cadastro referenciado não existe ou ainda está em uso."; case "23514", "23502" -> "Os dados não atendem às regras ... ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.
13         case "23505" -> "Já existe um registro com essa identificação.";
    Ramo do switch: case "23505". A instrução associada atende este valor; -> não cai automaticamente no próximo case.
14         case "23503" -> "Um cadastro referenciado não existe ou ainda está em uso.";
    Ramo do switch: case "23503". A instrução associada atende este valor; -> não cai automaticamente no próximo case.
```

```

15         case "23514", "23502" -> "Os dados não atendem às regras do banco.";
    Ramo do switch: case "23514", "23502". A instrução associada atende este valor; -> não cai automaticamente no próximo case.
16         case "23P01" -> "Há conflito de horário na agenda.";
    Ramo do switch: case "23P01". A instrução associada atende este valor; -> não cai automaticamente no próximo case.
17         case "40001", "40P01" -> "A operação concorrente foi interrompida. Tente novamente.";
    Ramo do switch: case "40001", "40P01". A instrução associada atende este valor; -> não cai automaticamente no próximo case.
18         case "42P01", "3F000" -> "A estrutura do banco não está pronta. Execute sql/criar_banco.sql.";
    Ramo do switch: case "42P01", "3F000". A instrução associada atende este valor; -> não cai automaticamente no próximo case.
19         case "28P01", "28000" -> "Confira o usuário e a senha da conexão PostgreSQL.";
    Ramo do switch: case "28P01", "28000". A instrução associada atende este valor; -> não cai automaticamente no próximo case.
20         default -> estado.startsWith("08")
    Ramo do switch: default. A instrução associada atende este valor; -> não cai automaticamente no próximo case.

    Escolha de valor: estado.startsWith("08"). true escolhe "Não foi possível conectar ao PostgreSQL. Confira o serviço e a
    configuração."; false escolhe "Não foi possível concluir a operação no banco. Consulte o log técnico.". Só a alternativa escolhida é
    avaliada.
21         ? "Não foi possível conectar ao PostgreSQL. Confira o serviço e a configuração."
    Continuação da expressão/texto da linha anterior. As partes concatenadas ou chamadas encadeadas compõem o
    argumento/resultado daquela instrução.
22         : "Não foi possível concluir a operação no banco. Consulte o log técnico.";
    Continua a declaração, expressão ou literal iniciada no bloco. Leia junto às linhas imediatamente anterior e seguinte; a pontuação
    determina onde a instrução termina.

23     };
24 }
25 }

```

## RegraNegocioException

src/main/java/br/edu/aeroporto/excecao/RegraNegocioException.java

Exceção não verificada para uma operação recusada pelas regras. A mensagem pode ser apresentada ao usuário.

7 linhas | 1 métodos/construtores declarados | SHA-256: d1649821d6e52a6f...

```
1 package br.edu.aeroporto.excecao;
    Declara o pacote deste arquivo: br.edu.aeroporto.excecao. O nome completo distingue a classe de outras com o mesmo nome.
2
3 public final class RegraNegocioException extends RuntimeException {
    Declara RegraNegocioException como tipo com campos e métodos. final impede herança. extends define a superclasse.
```

### RegraNegocioException - L4 a L6

Encapsula a mensagem de uma recusa de negócio como RuntimeException. A operação pode lançá-la sem throws obrigatório; os menus a capturam para apresentar a mensagem.

Entradas declaradas: (String mensagem).

Possíveis chamadores identificados: ArquivosServico.importarPassageiros (L24); AutenticacaoServico.entrar (L27); AutorizacaoJdbc.exigir (L39); AutorizacaoJdbc.exigirPerfil (L25); CadastroPassageiro.CadastroPassageiro (L8); ConsultaServico.assentos (L37); ConsultaServico.reserva (L45); ConsultaServico.voos (L28); Dados.exigir (L44); Dados.id (L14); Dados.instante (L36); Dados.valor (L29); DadosFicticiosServico.codigoLivre (L72); InicializacaoJdbc.criarAdministrador (L12); InicializacaoServico.criar (L27); Main.main (L55); MenuPrincipal.exportarVoos (L158); PassageiroJdbc.cadastrar (L37); PassageiroServico.buscar (L71); PassageiroServico.buscar (L80); PrimeiroAcesso.PrimeiroAcesso (L9); Sql.registro (L29); TipoCadastro.converter (L46); Validacao.cpf (L29); Validacao.nascimento (L20); Validacao.texto (L10).

Chamadas observadas no corpo: super.

```
4 public RegraNegocioException(String mensagem) {
    Início da declaração de RegraNegocioException. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.
5     super(mensagem);
    Inicializa a superclasse com os argumentos mostrados; o objeto mantém os campos herdados e os específicos desta classe.
6 }
7 }
```



## ExportadorCsv

src/main/java/br/edu/aeroporto/relatorio/ExportadorCsv.java

Implementação CSV com ponto e vírgula, aspas escapadas e proteção contra fórmulas; converte os horários para o fuso de cada aeroporto.

31 linhas | 3 métodos/construtores declarados | SHA-256: 4a4d8d30276e2d95...

```
1 package br.edu.aeroporto.relatorio;
  Declara o pacote deste arquivo: br.edu.aeroporto.relatorio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dto.VooResumo;
4 import java.io.BufferedWriter;
5 import java.io.IOException;
6 import java.util.List;
7
8 public final class ExportadorCsv implements ExportadorVoos {
  Declara ExportadorCsv como tipo com campos e métodos. final impede herança. implements liga o tipo aos contratos citados.
```

### extensao - L9 a L10

Retorna csv para compor nome do arquivo.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
9 @Override
  Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

10 public String extensao() { return "csv"; }
  Retorno: entrega "csv" ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

11
```

### escrever - L12 a L24

Grava cabeçalho e uma linha por VooResumo; usa campo para escapar valores e horários locais.

Entradas declaradas: (BufferedWriter arquivo, List<VooResumo> voos).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: arquivo.write, arquivo.newLine, String.join, Long.toString, voo.id, campo, voo.companhia, voo.numero, voo.origem, voo.destino, voo.partida().atZoneSameInstant(voo.fusoOrigem()).toOffsetDateTime().toString, voo.partida().atZoneSameInstant(voo.fusoOrigem()).toOffsetDateTime, voo.partida().atZoneSameInstant, voo.partida, voo.fusoOrigem, voo.chegada().atZoneSameInstant(voo.fusoDestino()).toOffsetDateTime().toString, voo.chegada().atZoneSameInstant(voo.fusoDestino()).toOffsetDateTime, voo.chegada().atZoneSameInstant, voo.chegada, voo.fusoDestino, voo.situacao().name, voo.situacao.

```
12 @Override
  Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

13 public void escrever(BufferedWriter arquivo, List<VooResumo> voos) throws IOException {
  Continuação da assinatura: completa os parâmetros e/ou declara as exceções que podem escapar da operação.

14     arquivo.write("id;voo;origem;destino;partida_local;chegada_local;situacao");
  Chama arquivo.write: Grava texto/bytes no destino fornecido; é efeito em arquivo/stream.

15     arquivo.newLine();
  Invoca arquivo.newLine com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

16     for (VooResumo voo : voos) {
  Percorre cada elemento da coleção/vetor: for (VooResumo voo : voos). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.
```

```

17      arquivo.write(String.join(";", Long.toString(voo.id()), campo(voo.companhia() + voo.numero()),
    Nesta parte da expressão, chama arquivo.write, String.join, Long.toString, voo.id, campo, voo.companhia, voo.numero. Os valores
    retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

18      campo(voo.origem()), campo(voo.destino()),
    Nesta parte da expressão, chama campo, voo.origem, voo.destino. Os valores retornados alimentam a instrução que começou
    nesta linha ou nas linhas anteriores.

19      campo(voo.partida().atZoneSameInstant(voo.fusoOrigem()).toOffsetDateTime().toString()),
    Nesta parte da expressão, chama campo, voo.partida().atZoneSameInstant(voo.fusoOrigem()).toOffsetDateTime().toString,
    voo.partida().atZoneSameInstant(voo.fusoOrigem()).toOffsetDateTime, voo.partida().atZoneSameInstant, voo.partida,
    voo.fusoOrigem. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

20      campo(voo.chegada().atZoneSameInstant(voo.fusoDestino()).toOffsetDateTime().toString()),
    Nesta parte da expressão, chama campo, voo.chegada().atZoneSameInstant(voo.fusoDestino()).toOffsetDateTime().toString,
    voo.chegada().atZoneSameInstant(voo.fusoDestino()).toOffsetDateTime, voo.chegada().atZoneSameInstant, voo.chegada,
    voo.fusoDestino. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

21      campo(voo.situacao().name()));
    Nesta parte da expressão, chama campo, voo.situacao().name, voo.situacao. Os valores retornados alimentam a instrução que
    começou nesta linha ou nas linhas anteriores.

22      arquivo.newLine();
    Instrução idêntica à de ExportadorCsv:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
    execução. A posição e o contexto do método acima mostram quando ela ocorre.

23    }
24  }
25

```

## campo - L26 a L30

Se começar com =,+,@ ou -, prefixa apóstrofo. Duplica aspas internas e envolve o resultado em aspas.

Entradas declaradas: (String texto).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: ExportadorCsv.escrever (L12).

Chamadas observadas no corpo: texto.matches, seguro.replace.

```

26      private static String campo(String texto) {
    Início da declaração de campo. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
    invocado.

27      // Neutraliza fórmulas quando o CSV é aberto em uma planilha.
    Comentário/documentação do autor; descreve intenção ou convenção e não executa.

28      String seguro = texto.matches("^=[+@-].*") ? "'" + texto : texto;
    Declara variável local seguro do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
    Escolha de valor: texto.matches("^=[+@-].*"). true escolhe "'" + texto; false escolhe texto. Só a alternativa escolhida é avaliada.

29      return "\"" + seguro.replace("\"", "\\\"") + "\"";
    Retorno: entrega "\"" + seguro.replace("\"", "\\\"") + "\"" ao chamador desta execução; nenhuma instrução posterior desse mesmo
    corpo executa por esse caminho.

30    }
31  }

```

## ExportadorTxt

src/main/java/br/edu/aeroporto/relatorio/ExportadorTxt.java

Implementação de relatório textual da página consultada, com contagem ao final.

26 linhas | 2 métodos/construtores declarados | SHA-256: 3bfbd3386d2eb947...

```

1 package br.edu.aeroporto.relatorio;
    Declara o pacote deste arquivo: br.edu.aeroporto.relatorio. O nome completo distingue a classe de outras com o mesmo nome.

2
3 import br.edu.aeroporto.dto.VooResumo;
4 import java.io.BufferedWriter;
5 import java.io.IOException;
6 import java.time.format.DateTimeFormatter;
7 import java.util.List;
8
9 public final class ExportadorTxt implements ExportadorVoos {
    Declara ExportadorTxt como tipo com campos e métodos. final impede herança. implements liga o tipo aos contratos citados.

10     private static final DateTimeFormatter DATA = DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm XXX");
    Instrução idêntica à de MenuPrincipal:L25. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

11

```

### extensao - L12 a L13

Retorna txt.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```

12     @Override
    Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

13     public String extensao() { return "txt"; }
    Retorno: entrega "txt" ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

14

```

### escrever - L15 a L25

Escreve título, cada voo formatado no fuso da origem e a contagem da lista fornecida.

Entradas declaradas: (BufferedWriter arquivo, List<VooResumo> voos).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: arquivo.write, arquivo.newLine, "%d | %s%s | %s → %s | %s | %s%n".formatted, voo.id, voo.companhia, voo.numero, voo.origem, voo.destino, DATA.format, voo.partida().atZoneSameInstant, voo.partida, voo.fusoOrigem, voo.situacao, voos.size.

```

15     @Override
    Anotação de contrato: @Override confere sobrescrita; @FunctionalInterface confere um único método abstrato, conforme a anotação mostrada.

16     public void escrever(BufferedWriter arquivo, List<VooResumo> voos) throws IOException {
    Instrução idêntica à de ExportadorCsv:L13. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

17         arquivo.write("RELATÓRIO DE VOOS – página consultada");
    Chama arquivo.write: Grava texto/bytes no destino fornecido; é efeito em arquivo/stream.

18         arquivo.newLine();
    Instrução idêntica à de ExportadorCsv:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

```
19         for (VooResumo voo : voos) {
```

Percorre cada elemento da coleção/vetor: for (VooResumo voo : voos). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
20             arquivo.write("%d | %s%s | %s → %s | %s | %s%n".formatted(voo.id(), voo.companhia(),  
voo.numero()),
```

Nesta parte da expressão, chama arquivo.write, "%d | %s%s | %s → %s | %s | %s%n".formatted, voo.id, voo.companhia, voo.numero. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

```
21                 voo.origem(), voo.destino(),  
DATA.format(voo.partida().atZoneSameInstant(voo.fusoOrigem()), voo.situacao()));
```

Nesta parte da expressão, chama voo.origem, voo.destino, DATA.format, voo.partida().atZoneSameInstant, voo.partida, voo.fusoOrigem, voo.situacao. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.

```
22         }
```

```
23         arquivo.write("Total de registros nesta página: " + voos.size());
```

Chama arquivo.write: Grava texto/bytes no destino fornecido; é efeito em arquivo/stream.

```
24         arquivo.newLine();
```

Instrução idêntica à de ExportadorCsv:L15. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
25     }
```

```
26 }
```

## ExportadorVoos

src/main/java/br/edu/aeroporto/relatorio/ExportadorVoos.java

Interface do formato de saída. RelatorioServico pode chamar escrever/extensao sem depender de TXT ou CSV específicos.

11 linhas | 2 métodos/construtores declarados | SHA-256: 52ae54dc9822e847...

```
1 package br.edu.aeroporto.relatorio;
```

Declara o pacote deste arquivo: br.edu.aeroporto.relatorio. O nome completo distingue a classe de outras com o mesmo nome.

```
2
3 import br.edu.aeroporto.dto.VooResumo;
4 import java.io.BufferedWriter;
5 import java.io.IOException;
6 import java.util.List;
```

```
7
8 public interface ExportadorVoos {
```

Declara ExportadorVoos como contrato de operações.

### extensao - L9 a L9

Contrato da extensão usada no nome do relatório. Cada estratégia fornece txt ou csv.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: RelatorioServico.exportar (L14).

```
9 String extensao();
```

Início da declaração de extensao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

### escrever - L10 a L10

Contrato de escrita no BufferedWriter dos VooResumo recebidos. A implementação escolhida decide formatação e escape; IOException pode escapar ao serviço.

Entradas declaradas: (BufferedWriter arquivo, List<VooResumo> voos).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: RelatorioServico.exportar (L14).

```
10 void escrever(BufferedWriter arquivo, List<VooResumo> voos) throws IOException;
```

Início da declaração de escrever. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
11 }
```

## Senhas

src/main/java/br/edu/aeroporto/seguranca/Senhas.java

Implementa geração e conferência de hashes PBKDF2 com sal aleatório. Não descriptografa senhas; calcula novamente e compara os resultados.

57 linhas | 4 métodos/construtores declarados | SHA-256: 1c1f0b71ce8b2a9a...

```

1 package br.edu.aeroporto.seguranca;
  Declara o pacote deste arquivo: br.edu.aeroporto.seguranca. O nome completo distingue a classe de outras com o mesmo nome.

2

3 import java.security.GeneralSecurityException;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

4 import java.security.MessageDigest;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

5 import java.security.SecureRandom;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

6 import java.util.Arrays;
7 import java.util.Base64;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

8 import javax.crypto.SecretKeyFactory;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

9 import javax.crypto.spec.PBEKeySpec;
  Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

10

11 public final class Senhas {
  Declara Senhas como tipo com campos e métodos. final impede herança.

12     private static final int ITERACOES = 600_000;
  Campo(s) ITERACOES: private restringe acesso à classe. Tipo int: inteiro primitivo de 32 bits, usado para índice, limite ou contagem. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

13     private static final SecureRandom ALEATORIO = new SecureRandom();
  Campo(s) ALEATORIO: private restringe acesso à classe. Tipo SecureRandom: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

  new constrói SecureRandom: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

14

```

### Senhas - L15 a L15

Construtor vazio. Se for private, impede criar instâncias desta classe utilitária a partir de outras classes; os métodos static são usados pelo nome da classe.

Entradas declaradas: ().

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```

15     private Senhas() {}
  Início da declaração de Senhas. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

16

```

### gerar - L17 a L27

Cria sal de 16 bytes com SecureRandom, deriva hash e monta algoritmo\$iterações\$sal\$hash em Base64. Limpa o vetor hash no finally; não apaga a String já retornada.

Entradas declaradas: (char[] senha).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: AdministracaoServico.alterarSenha (L67); AdministracaoServico.criarUsuario (L32); InicializacaoServico.criar (L27); UnidadeTeste.main (L10).

Chamadas observadas no corpo: ALEATORIO.nextBytes, derivar, Base64.getEncoder().encodeToString, Base64.getEncoder, Arrays.fill.

|    |                                                                                       |                                                                                                                                                                                                                                                              |
|----|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 17 | public static String gerar(char[] senha) {                                            | Início da declaração de gerar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                               |
| 18 | byte[] sal = new byte[16];                                                            | Declara variável local sal do tipo byte[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br>new constrói byte: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só. |
| 19 | ALEATORIO.nextBytes(sal);                                                             | Invoca ALEATORIO.nextBytes com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.                                                                                         |
| 20 | byte[] hash = derivar(senha, sal, ITERACOES);                                         | Declara variável local hash do tipo byte[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                             |
| 21 | try {                                                                                 | Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.                                                                                                                                                                           |
| 22 | return "pbkdf2-sha256\$" + ITERACOES + "\$" + Base64.getEncoder().encodeToString(sal) | Retorno: entrega "pbkdf2-sha256\$" + ITERACOES + "\$" + Base64.getEncoder().encodeToString(sal) + "\$" + Base64.getEncoder().encodeToString(hash) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.        |
| 23 | + "\$" + Base64.getEncoder().encodeToString(hash);                                    | Nesta parte da expressão, chama Base64.getEncoder().encodeToString, Base64.getEncoder. Os valores retornados alimentam a instrução que começou nesta linha ou nas linhas anteriores.                                                                         |
| 24 | } finally {                                                                           | Bloco de finalização, executado ao sair do try/catch; neste corpo, observe quais recursos ou dados são limpos.                                                                                                                                               |
| 25 | Arrays.fill(hash, (byte) 0);                                                          | Chama Arrays.fill: Substitui todas as posições do vetor pelo valor fornecido; usado para limpar char[] de senha.                                                                                                                                             |
| 26 | }                                                                                     |                                                                                                                                                                                                                                                              |
| 27 | }                                                                                     |                                                                                                                                                                                                                                                              |
| 28 |                                                                                       |                                                                                                                                                                                                                                                              |

## conferir - L29 a L45

Divide o formato armazenado, limita iterações/tamanhos, deriva novamente com o sal e usa MessageDigest.isEqual.

Formato inválido devolve false; limpa o cálculo no finally.

Entradas declaradas: (char[] senha, String armazenada).

Saída declarada: boolean - condição verdadeira ou falsa.

Possíveis chamadores identificados: AdministracaoServico.alterarSenha (L67); AutenticacaoServico.entrar (L27); UnidadeTeste.main (L10).

Chamadas observadas no corpo: armazenada.split, partes[0].equals, Integer.parseInt, Base64.getDecoder().decode, Base64.getDecoder, derivar, MessageDigest.isEqual, Arrays.fill.

|    |                                                                             |                                                                                                                                                                                                                                                                                                                                        |
|----|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 29 | public static boolean conferir(char[] senha, String armazenada) {           | Início da declaração de conferir. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                                                                                                      |
| 30 | String[] partes = armazenada.split("\\\$");                                 | Declara variável local partes do tipo String[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                   |
| 31 | if (partes.length != 4    !partes[0].equals("pbkdf2-sha256")) return false; | Decisão específica: (partes.length != 4    !partes[0].equals("pbkdf2-sha256")). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.<br><br>Retorno: entrega false ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho. |
| 32 | byte[] calculado = null;                                                    | Declara variável local calculado do tipo byte[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                  |
| 33 | try {                                                                       | Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.                                                                                                                                                                                                                                                     |

```

34         int iteracoes = Integer.parseInt(partes[1]);
        Declara variável local iteracoes do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

35         byte[] sal = Base64.getDecoder().decode(partes[2]);
        Declara variável local sal do tipo byte[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

36         byte[] esperado = Base64.getDecoder().decode(partes[3]);
        Declara variável local esperado do tipo byte[]. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

37         if (iteracoes < 100_000 || iteracoes > 2_000_000 || sal.length < 16 || esperado.length != 32)
return false;
        Decisão específica: (iteracoes < 100_000 || iteracoes > 2_000_000 || sal.length < 16 || esperado.length != 32). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

        Retorno: entrega false ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

38         calculado = derivar(senha, sal, iteracoes);
        Atualiza calculado usando = e o resultado de derivar(senha, sal, iteracoes). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

39         return MessageDigest.isEqual(esperado, calculado);
        Retorno: entrega MessageDigest.isEqual(esperado, calculado) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

        Invoca MessageDigest.isEqual com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

40     } catch (IllegalArgumentException formatoInvalido) {
        Trata a(s) exceção(ões) declarada(s) em catch (IllegalArgumentException formatoInvalido). O tratamento mostrado abaixo decide se avisa, repete ou relança.

41         return false;
        Retorno: entrega false ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

42     } finally {
        Bloco de finalização, executado ao sair do try/catch; neste corpo, observe quais recursos ou dados são limpos.

43         if (calculado != null) Arrays.fill(calculado, (byte) 0);
        Decisão específica: (calculado != null). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

44     }
45 }
46

```

## derivar - L47 a L56

PBEKeySpec usa senha/sal/iterações e saída de 256 bits. SecretKeyFactory executa PBKDF2WithHmacSHA256. Limpa a senha interna da chave ao final.

Entradas declaradas: (char[] senha, byte[] sal, int iteracoes).

Saída declarada: byte[] - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: Senhas.conferir (L29); Senhas.gerar (L17).

Chamadas observadas no corpo: new PBEKeySpec, SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(chave).getEncoded, SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret, SecretKeyFactory.getInstance, new IllegalStateException, chave.clearPassword.

```

47     private static byte[] derivar(char[] senha, byte[] sal, int iteracoes) {
        Início da declaração de derivar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

48         PBEKeySpec chave = new PBEKeySpec(senha, sal, iteracoes, 256);
        Declara variável local chave do tipo PBEKeySpec. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        new constrói PBEKeySpec: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

49         try {
        Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

```



```

50         return
        SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(chave).getEncoded();
        Retorno: entrega SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(chave).getEncoded() ao chamador
        desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

        Invoca SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret(chave).getEncoded com os argumentos
        mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

        Invoca SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256").generateSecret com os argumentos mostrados. O
        receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

        Invoca SecretKeyFactory.getInstance com os argumentos mostrados. O receptor/contrato determina a implementação; veja as
        chamadas do método para localizar o corpo no projeto.

51     } catch (GeneralSecurityException erro) {
        Trata a(s) exceção(ões) declarada(s) em catch (GeneralSecurityException erro). O tratamento mostrado abaixo decide se avisa,
        repete ou relança.

52         throw new IllegalStateException("O ambiente Java não suporta o algoritmo de senha.", erro);
        Lança new IllegalStateException("O ambiente Java não suporta o algoritmo de senha.", erro). Interrompe o caminho normal; a
        exceção sobe até uma captura compatível e pode provocar rollback.

53     } finally {
        Bloco de finalização, executado ao sair do try/catch; neste corpo, observe quais recursos ou dados são limpos.

54         chave.clearPassword();
        Invoca chave.clearPassword com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas
        do método para localizar o corpo no projeto.

55     }
56 }
57 }

```

# Java: testes e preparação

## UnidadeTeste

src/test/java/br/edu/aeroporto/UnidadeTeste.java

Executa 12 verificações de regras locais, mapa e hashes sem precisar de PostgreSQL.

32 linhas | 3 métodos/construtores declarados | SHA-256: 66732125555cabba...

```
1 package br.edu.aeroporto;
    Declara o pacote deste arquivo: br.edu.aeroporto. O nome completo distingue a classe de outras com o mesmo nome.
2
3 import br.edu.aeroporto.dominio.*;
4 import br.edu.aeroporto.seguranca.Senhas;
5 import java.math.BigDecimal;
6 import java.time.*;
7 import java.util.*;
8
9 public final class UnidadeTeste {
    Declara UnidadeTeste como tipo com campos e métodos. final impede herança.
```

### main - L10 a L28

Executa as 12 asserções mostradas sobre CPF, nascimento, dinheiro, transição, matriz/copias e hashes.

Entradas declaradas: (String[] args).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: assertiva, Validacao.cpf("529.982.247-25").equals, Validacao.cpf, falha, Validacao.nascimento, LocalDate.now().plusDays, LocalDate.now, Dados.valor, new BigDecimal, SituacaoVoo.CONCLUIDO.permiteTransicaoPara, List.of, Map.of, new MapaAssentos, Arrays.deepEquals, mapa.celulas, mapa.celulas()[0][0].equals, mapa.mesmoMapa, TipoCadastro.Campo.class.cast(new TipoCadastro.Campo("ativo","b",false)).converter, TipoCadastro.Campo.class.cast, new TipoCadastro.Campo, Senhas.gerar, "SenhaFicticial2026".toCharArray, Senhas.conferir, "incorreta".toCharArray, "senha".toCharArray, System.out.println.

```
10 public static void main(String[] args) {
    Instrução idêntica à de Main:L55. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A
    posição e o contexto do método acima mostram quando ela ocorre.
11     assertiva(Validacao.cpf("529.982.247-25").equals("52998224725"), "CPF valido normalizado");
    Verificação deste cenário: Validacao.cpf("529.982.247-25").equals("52998224725"), "CPF valido normalizado". A
    operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
12     falha(()->Validacao.cpf("1111111111"), "CPF repetido rejeitado");
    Verificação deste cenário: ()->Validacao.cpf("1111111111"), "CPF repetido rejeitado". A operação/condição deve terminar
    lançando a falha esperada; divergência faz o programa de teste falhar.
13     falha(()->Validacao.nascimento(LocalDate.now().plusDays(1),LocalDate.now()), "Nascimento futuro
    rejeitado");
    Verificação deste cenário: ()->Validacao.nascimento(LocalDate.now().plusDays(1),LocalDate.now()), "Nascimento futuro
    rejeitado". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.
14     falha(()->Dados.valor(new BigDecimal("1.001"),false), "Dinheiro com 3 casas rejeitado");
    Verificação deste cenário: ()->Dados.valor(new BigDecimal("1.001"),false), "Dinheiro com 3 casas rejeitado". A operação/condição
    deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

    new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si
    só.
15     assertiva(!SituacaoVoo.CONCLUIDO.permiteTransicaoPara(SituacaoVoo.PROGRAMADO), "Voo encerrado nao
    reabre");
    Verificação deste cenário: !SituacaoVoo.CONCLUIDO.permiteTransicaoPara(SituacaoVoo.PROGRAMADO), "Voo encerrado nao
    reabre". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
16     var linhas=List.of(Map.<String,Object>of("codigo","2C","fila",2,"coluna","C","ocupado",true,"bloqu
    eado",false),Map.<String,Object>of("codigo","1A","fila",1,"coluna","A","ocupado",false,"bloqueado",false))
    ;
    Declara variável local linhas com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
    escopo é o bloco atual.
```

|    |                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----|--------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 17 | <pre>var mapa=new MapaAssentos(linhas);</pre>                                                                                                          | Declara variável local mapa com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói MapaAssentos: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                       |
| 18 | <pre>assertiva(Arrays.deepEquals(mapa.celulas(),new String[][]{{"1A:L","--"},{"--","2C:O"}}),"Matriz irregular de assentos");</pre>                    | Verificação deste cenário: Arrays.deepEquals(mapa.celulas(),new String[][]{{"1A:L","--"},{"--","2C:O"}}), "Matriz irregular de assentos". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.<br><br>new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                             |
| 19 | <pre>var copia=mapa.celulas();copia[0][0]="alterado";</pre>                                                                                            | Declara variável local copia com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Atualiza copia[0][0] usando = e o resultado de "alterado". Atribuição modifica a variável/célula indicada; não é um teste de igualdade.                                                                                                                |
| 20 | <pre>assertiva(mapa.celulas()[0][0].equals("1A:L"),"Copia defensiva da matriz");</pre>                                                                 | Verificação deste cenário: mapa.celulas()[0][0].equals("1A:L"), "Copia defensiva da matriz". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.                                                                                                                                                                                                                             |
| 21 | <pre>assertiva(mapa.mesmoMapa(new MapaAssentos(linhas)),"Comparacao de mapas");</pre>                                                                  | Verificação deste cenário: mapa.mesmoMapa(new MapaAssentos(linhas)), "Comparacao de mapas". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.<br><br>new constrói MapaAssentos: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                     |
| 22 | <pre>falha(()-&gt;TipoCadastro.Campo.class.cast(new TipoCadastro.Campo("ativo","b",false)).converter("qualquer"),"Booleano invalido rejeitado");</pre> | Verificação deste cenário: ()->TipoCadastro.Campo.class.cast(new TipoCadastro.Campo("ativo","b",false)).converter("qualquer"), "Booleano invalido rejeitado". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.<br><br>new constrói TipoCadastro.Campo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só. |
| 23 | <pre>String hash=Senhas.gerar("SenhaFicticia!2026".toCharArray());</pre>                                                                               | Declara variável local hash do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                      |
| 24 | <pre>assertiva(Senhas.conferir("SenhaFicticia!2026".toCharArray(),hash),"Senha correta");</pre>                                                        | Verificação deste cenário: Senhas.conferir("SenhaFicticia!2026".toCharArray(),hash), "Senha correta". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.                                                                                                                                                                                                                    |
| 25 | <pre>assertiva(!Senhas.conferir("incorreta".toCharArray(),hash),"Senha incorreta rejeitada");</pre>                                                    | Verificação deste cenário: !Senhas.conferir("incorreta".toCharArray(),hash), "Senha incorreta rejeitada". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.                                                                                                                                                                                                                |
| 26 | <pre>assertiva(!Senhas.conferir("senha".toCharArray(),"hash-invalido"),"Hash invalido rejeitado");</pre>                                               | Verificação deste cenário: !Senhas.conferir("senha".toCharArray(),"hash-invalido"), "Hash invalido rejeitado". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.                                                                                                                                                                                                           |
| 27 | <pre>System.out.println("12 verificacoes unitarias Java aprovadas.");</pre>                                                                            | Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.                                                                                                                                                                                                                                                                                                            |

```
28     }
29
```

## assertiva - L30 a L30

Falha de teste é `AssertionError`, distinta de uma regra de negócio esperada.

Entradas declaradas: (boolean condicao,String nome).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: `UnidadeTeste.main` (L10).

Chamadas observadas no corpo: `new AssertionError`.

```
30     static void assertiva(boolean condicao,String nome) { if(!condicao) throw new AssertionError(nome); }
```

Início da declaração de assertiva. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Decisão específica: (!condicao). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança `new AssertionError(nome)`. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

`new` constrói `AssertionError`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

## falha - L31 a L31

Aceita apenas `RegraNegocioException` como falha esperada; operação que não falhar gera `AssertionError`.

Entradas declaradas: (Runnable operacao,String nome).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: `UnidadeTeste.main` (L10).

Chamadas observadas no corpo: `operacao.run`, `new AssertionError`.

```
31     static void falha(Runnable operacao,String nome) { try { operacao.run(); }
    catch(br.edu.aeroporto.excecao.RegraNegocioException esperado) { return; } throw new AssertionError(nome);
    }
```

Início da declaração de falha. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Trata a(s) exceção(ões) declarada(s) em `catch(br.edu.aeroporto.excecao.RegraNegocioException esperado)`. O tratamento mostrado abaixo decide se avisa, repete ou relança.

`return` encerra o caminho atual sem entregar valor (void).

Lança `new AssertionError(nome)`. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

`new` constrói `AssertionError`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
32 }
```

## IntegracaoTeste

src/test/java/br/edu/aeroporto/IntegracaoTeste.java

Exercita os fluxos contra um PostgreSQL isolado. Usa relógio controlável, asserções próprias, tarefas concorrentes e processos Java para testar o terminal.

518 linhas | 33 métodos/construtores declarados | SHA-256: 37f33cf89e2a1ec3...

Dependências guardadas em campos: AdministracaoServico, AtendimentoServico, AuditoriaJdbc, AutorizacaoJdbc, BancoDados, CadastroServico, FinanceiroServico, OperacaoServico, PassageiroServico, PlanejamentoServico, RelatoriosCompletoServico, ReservaServico, Sessao, VooServico. São referências da classe; as atribuições dos cenários mostram sua inicialização.

```
1 package br.edu.aeroporto;
    Declara o pacote deste arquivo: br.edu.aeroporto. O nome completo distingue a classe de outras com o mesmo nome.
```

```
2
3 import br.edu.aeroporto.config.Configuracao;
4 import br.edu.aeroporto.dominio.*;
5 import br.edu.aeroporto.dto.TipoRelatorio;
6 import br.edu.aeroporto.execcao.*;
```

Primeira ocorrência deste import: permite usar o nome de tipo do próprio projeto sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
7 import br.edu.aeroporto.infraestrutura.jdbc.*;
8 import br.edu.aeroporto.servico.*;
9 import java.math.BigDecimal;
10 import java.nio.file.*;
11 import java.time.*;
12 import java.util.*;
13 import java.util.concurrent.*;
```

Primeira ocorrência deste import: permite usar o nome de API/biblioteca Java sem escrevê-lo por completo. Não cria objetos nem executa o tipo; veja package/import no capítulo de sintaxe.

```
14
15 /** Usa exclusivamente o cluster isolado de artifacts na porta 55439; nunca a configuracao local do
    usuario. */
```

Comentário/documentação do autor; descreve intenção ou convenção e não executa.

```
16 public final class IntegracaoTeste {
    Declara IntegracaoTeste como tipo com campos e métodos. final impede herança.
```

```
17     private static final class Relogio extends Clock {
    Declara Relogio como tipo com campos e métodos. final impede herança. extends define a superclasse.
```

```
18         private Instant instante=Instant.now().truncatedTo(java.time.temporal.ChronoUnit.MICROS);
    Campo(s) instante: private restringe acesso à classe. Tipo Instant: ponto na linha do tempo.
```

### getZone - L19 a L19

Clock de teste declara UTC.

Entradas declaradas: ().

Saída declarada: ZoneId - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
19     public ZoneId getZone(){return ZoneOffset.UTC;}
    Início da declaração de getZone. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

    Retorno: entrega ZoneOffset.UTC ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.
```

### withZone - L20 a L20

Neste relógio simplificado devolve a mesma instância; não aplica o ZoneId recebido.

Entradas declaradas: (ZoneId zone).

Saída declarada: Clock - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
20      public Clock withZone(ZoneId zone){return this;}
```

Início da declaração de withZone. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega this ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## instant - L21 a L21

Retorna o instante controlado pelo cenário.

Entradas declaradas: ().

Saída declarada: Instant - ponto na linha do tempo.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
21      public Instant instant(){return instante;}
```

Início da declaração de instant. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega instante ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## definir - L22 a L22

Altera o relógio de teste, permitindo verificar expiração/janelas sem esperar tempo real.

Entradas declaradas: (Instant valor).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

```
22      void definir(Instant valor){instante=valor;}
```

Início da declaração de definir. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Atualiza instante usando = e o resultado de valor. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
23      }
```

```
24      private static final Relogio relógio=new Relogio();
```

Campo(s) relógio: private restringe acesso à classe. Tipo Relogio: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói Relogio: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
25      private static final BancoDados banco=new BancoDados(new Configuracao("jdbc:postgresql://127.0.0.1:554
39/sistema_aeroporto", "aeroporto_teste", "", ZoneId.of("America/Sao_Paulo")));
```

Campo(s) banco: private restringe acesso à classe. Tipo BancoDados: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói BancoDados, Configuracao: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
26      private static final AutorizacaoJdbc autorizacao=new AutorizacaoJdbc();
```

Campo(s) autorizacao: private restringe acesso à classe. Tipo AutorizacaoJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói AutorizacaoJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
27      private static final AuditoriaJdbc auditoria=new AuditoriaJdbc();
```

Campo(s) auditoria: private restringe acesso à classe. Tipo AuditoriaJdbc: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói AuditoriaJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
28      private static final CadastroServico cadastros=new CadastroServico(banco,new
CadastroJdbc(),autorizacao,auditoria,relógio);
```

Campo(s) cadastros: private restringe acesso à classe. Tipo CadastroServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói CadastroServico, CadastroJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
29 private static final PassageiroServico passageiros=new PassageiroServico(banco,new
PassageiroJdbc(),auditoria,autorizacao,religio);
```

Campo(s) passageiros: private restringe acesso à classe. Tipo PassageiroServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói PassageiroServico, PassageiroJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
30 private static final ReservaServico reservas=new ReservaServico(banco,new
ReservaJdbc(),autorizacao,auditoria,religio);
```

Campo(s) reservas: private restringe acesso à classe. Tipo ReservaServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói ReservaServico, ReservaJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
31 private static final PlanejamentoServico planejamento=new PlanejamentoServico(banco,new
PlanejamentoJdbc(),autorizacao,auditoria,religio);
```

Campo(s) planejamento: private restringe acesso à classe. Tipo PlanejamentoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói PlanejamentoServico, PlanejamentoJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
32 private static final FinanceiroServico financeiro=new
FinanceiroServico(banco,autorizacao,auditoria,religio);
```

Campo(s) financeiro: private restringe acesso à classe. Tipo FinanceiroServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói FinanceiroServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
33 private static final AtendimentoServico atendimento=new
AtendimentoServico(banco,autorizacao,auditoria,religio);
```

Campo(s) atendimento: private restringe acesso à classe. Tipo AtendimentoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói AtendimentoServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
34 private static final OperacaoServico operacao=new
OperacaoServico(banco,autorizacao,auditoria,religio);
```

Campo(s) operacao: private restringe acesso à classe. Tipo OperacaoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói OperacaoServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
35 private static final VooServico voos=new VooServico(banco,autorizacao,auditoria,religio);
```

Campo(s) voos: private restringe acesso à classe. Tipo VooServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói VooServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
36 private static final AdministracaoServico administracao=new
AdministracaoServico(banco,autorizacao,auditoria);
```

Campo(s) administracao: private restringe acesso à classe. Tipo AdministracaoServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói AdministracaoServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
37 private static final RelatoriosCompletosServico relatorios=new
RelatoriosCompletosServico(banco,autorizacao,auditoria);
```

Campo(s) relatorios: private restringe acesso à classe. Tipo RelatoriosCompletosServico: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

new constrói RelatoriosCompletoServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
38     private static Sessao admin;
```

Campo(s) admin: private restringe acesso à classe. Tipo Sessao: guarda referência/valor do tipo declarado, disponível aos métodos que têm acesso. static faz o campo pertencer à classe.

```
39     private static int verificacoes;
```

Campo(s) verificacoes: private restringe acesso à classe. Tipo int: inteiro primitivo de 32 bits, usado para índice, limite ou contagem. static faz o campo pertencer à classe.

```
40     private static final String sufixo=UUID.randomUUID().toString().substring(0,8);
```

Campo(s) sufixo: private restringe acesso à classe. Tipo String: texto imutável. static faz o campo pertencer à classe. final impede reatribuir esse campo depois da inicialização.

```
41     private static long
    pais,origem,destino,companhia,modelo,aeronave,rota,voo,tarifa,portao,documento,passageiro,outroPassageiro;
```

Campo(s) pais, origem, destino, companhia, modelo, aeronave, rota, voo, tarifa, portao, documento, passageiro, outroPassageiro: private restringe acesso à classe. Tipo long: inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome. static faz o campo pertencer à classe.

```
42     private static Instant partida,chegada;
```

Campo(s) partida, chegada: private restringe acesso à classe. Tipo Instant: ponto na linha do tempo. static faz o campo pertencer à classe.

```
43     private static int numeroVoo;
```

Campo(s) numeroVoo: private restringe acesso à classe. Tipo int: inteiro primitivo de 32 bits, usado para índice, limite ou contagem. static faz o campo pertencer à classe.

```
44
```

## main - L45 a L48

Chama os cenários sequencialmente e só informa a contagem final se nenhum deles lançar erro.

Entradas declaradas: (String[] args).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: inicializar, cadastros, comercial, atendimento, arquivos, concorrência, cancelamentos, cenariosAdicionais, regrasComplementares, administracaoEImportacao, cadastrosRestantes, System.out.println.

```
45     public static void main(String[] args) throws Exception {
```

Início da declaração de main. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
46         inicializar();cadastros();comercial();atendimento();arquivos();concorrência();cancelamentos();cena
        riosAdicionais();regrasComplementares();administracaoEImportacao();cadastrosRestantes();
```

Chama IntegracaoTeste.inicializar: Cria/autentica administrador fictício e verifica primeira inicialização única, recusa login errado e quantidade de tabelas.

```
47         System.out.println(verificacoes+" verificacoes PostgreSQL aprovadas; banco isolado preservado.");
```

Chama System.out.println: Escreve texto e quebra a linha na saída indicada; não grava o conteúdo no banco.

```
48     }
```

```
49
```

## inicializar - L50 a L60

Cria/autentica administrador fictício e verifica primeira inicialização única, recusa login errado e quantidade de tabelas.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: new AutenticacaoServico, new UsuarioJdbc, new InicializacaoServico, new InicializacaoJdbc, new PrimeiroAcesso, LocalDate.of, autenticacao.existeUsuario, inicializacao.criar, "SenhaFicticia!2026".toCharArray, autenticacao.entrar, verificar, admin.perfis().contains, admin.perfis, rejeitar, "incorreta".toCharArray, contar.

```
50     private static void inicializar() {
```

Início da declaração de inicializar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.



```

51      var autenticacao=new AutenticacaoServico(banco,new UsuarioJdbc(),auditoria);
Declara variável local autenticacao com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói AutenticacaoServico, UsuarioJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

52      var inicializacao=new InicializacaoServico(banco,new InicializacaoJdbc(),auditoria,relógio);
Declara variável local inicializacao com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói InicializacaoServico, InicializacaoJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

53      var dados=new PrimeiroAcesso("Administrador FICTICIO",LocalDate.of(1980,1,1),"teste_admin","Cidade FICTICIA","Teste","Aeroporto FICTICIO","XINI");
Declara variável local dados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói PrimeiroAcesso: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

54      if(!autenticacao.existeUsuario()) inicializacao.criar(dados,"SenhaFicticia!2026".toCharArray());
Decisão específica: (!autenticacao.existeUsuario()). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

55      admin=autenticacao.entrar("teste_admin","SenhaFicticia!2026".toCharArray());
Atualiza admin usando = e o resultado de autenticacao.entrar("teste_admin","SenhaFicticia!2026".toCharArray()). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

56      verificar(admin.perfis().contains("ADMINISTRADOR"),"Primeiro administrador e login");
Verificação deste cenário: admin.perfis().contains("ADMINISTRADOR"), "Primeiro administrador e login". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

57      rejeitar(()->inicializacao.criar(dados,"SenhaFicticia!2026".toCharArray()),"Inicializacao nao duplica admin");
Verificação deste cenário: ()->inicializacao.criar(dados,"SenhaFicticia!2026".toCharArray()), "Inicializacao nao duplica admin". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

58      rejeitar(()->autenticacao.entrar("teste_admin","incorreta".toCharArray()),"Login invalido");
Verificação deste cenário: ()->autenticacao.entrar("teste_admin","incorreta".toCharArray()), "Login invalido". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

59      verificar(contar("SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='aeroporto' AND table_type='BASE TABLE'")==41,"41 tabelas preservadas");
Verificação deste cenário: contar("SELECT COUNT(*) FROM information_schema.tables WHERE table_schema='aeroporto' AND table_type='BASE TABLE'")==41, "41 tabelas preservadas". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

60      }
61

```

## cadastros - L62 a L107

Cria localidades/frota/pessoas, testa dupla identidade, edição/inativação, autorização reconsultada e preservação de administrador.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: banco.transacao, Sql.executar, Dados.id, Sql.registro, cad, icao, icao().substring, passageiros.vincular, verificar, contar, passageiros.editar, LocalDate.of, passageiros.buscar(admin,passageiro).nome().contains, passageiros.buscar(admin,passageiro).nome, passageiros.buscar, LocalDate.now().minusDays, LocalDate.now, administracao.criarUsuario, "SenhaFicticia!2026".toCharArray, Set.of, new AutenticacaoServico(banco,new UsuarioJdbc(),auditoria).entrar, new AutenticacaoServico, new UsuarioJdbc, rejeitar, cadastros.salvar, mapa, administracao.acesso, relatorios.consultar, admin.usuarioId, relógio.instant().plus, relógio.instant, Duration.ofHours, partida.plus, planejamento.programar, programacao, planejamento.tarifa, new Tarifa, new BigDecimal, partida.plusSeconds, chegada.plusSeconds, operacao.habilitar, LocalDate.now().plusYears, operacao.escalar, partida.minus, Duration.ofMinutes, chegada.plus, relatorios.mapa(admin,voo).celulas, relatorios.mapa.

```

62      private static void cadastros() {
Início da declaração de cadastros. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

```
63      pais=banco.transacao(c->{Sql.executar(c,"INSERT INTO pais(codigo,nome) VALUES ('ZZ','Pais
FICTICIO') ON CONFLICT DO NOTHING");return Dados.id(Sql.registro(c,"SELECT id FROM pais WHERE
codigo='ZZ',"id"))});
```

Atualiza pais usando = e o resultado de banco.transacao(c->{Sql.executar(c,"INSERT INTO pais(codigo,nome) VALUES ('ZZ','Pais FICTICIO') ON CONFLICT DO NOTHING");return Dados.id(... Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: pais. ON CONFLICT define o tratamento da colisão descrita no SQL.

Retorno: entrega Dados.id(Sql.registro(c,"SELECT id FROM pais WHERE codigo='ZZ',"id")) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: pais.

```
64      long cidade=cad(TipoCadastro.CIDADE,"pais_id",pais,"nome","Cidade FICTICIA "+sufixo,"regiao","");
```

Declara variável local cidade do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
65      origem=cad(TipoCadastro.AEROPORTO,"cidade_id",cidade,"codigo_icao",icao(),"nome","Origem
FICTICIA","fuso_horario","UTC");
```

Atualiza origem usando = e o resultado de cad(TipoCadastro.AEROPORTO,"cidade\_id",cidade,"codigo\_icao",icao(),"nome","Origem FICTICIA","fuso\_horario","UTC"). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
66      destino=cad(TipoCadastro.AEROPORTO,"cidade_id",cidade,"codigo_icao",icao(),"nome","Destino
FICTICIO","fuso_horario","UTC");
```

Atualiza destino usando = e o resultado de cad(TipoCadastro.AEROPORTO,"cidade\_id",cidade,"codigo\_icao",icao(),"nome","Destino FICTICIO","fuso\_horario","UTC"). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
67      long terminal=cad(TipoCadastro.TERMINAL,"aeroporto_id",origem,"codigo","T1","nome","Terminal
FICTICIO");
```

Declara variável local terminal do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
68      portao=cad(TipoCadastro.RECURSO,"aeroporto_id",origem,"terminal_id",terminal,"tipo","PORTAO","codi
go","P1","situacao","DISPONIVEL","envergadura_max_m","40");
```

Atualiza portao usando = e o resultado de cad(TipoCadastro.RECURSO,"aeroporto\_id",origem,"terminal\_id",terminal,"tipo","PORTAO","codigo","P1","situacao","DISPONIVEL","envergadura\_.... Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
69      companhia=cad(TipoCadastro.COMPANHIA,"pais_id",pais,"codigo_icao",icao().substring(1),"nome","Companhia
FICTICIA "+sufixo);
```

Atualiza companhia usando = e o resultado de cad(TipoCadastro.COMPANHIA,"pais\_id",pais,"codigo\_icao",icao().substring(1),"nome","Companhia FICTICIA "+sufixo). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
70      modelo=cad(TipoCadastro.MODELO,"fabricante","FICTICIO","nome",sufixo,"envergadura_m","30","comprim
ento_pista_min_m","1500");
```

Atualiza modelo usando = e o resultado de cad(TipoCadastro.MODELO,"fabricante","FICTICIO","nome",sufixo,"envergadura\_m","30","comprimento\_pista\_min\_m","1500"). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
71      aeronave=cad(TipoCadastro.AERONAVE,"modelo_id",modelo,"companhia_id",companhia,"matricula","PT-"+s
ufixo,"situacao","ATIVA","fabricacao_ano",2025);
```

Atualiza aeronave usando = e o resultado de cad(TipoCadastro.AERONAVE,"modelo\_id",modelo,"companhia\_id",companhia,"matricula","PT-"+sufixo,"situacao","ATIVA","fabricacao\_ano",2025). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
72      for(int fila=1;fila<=3;fila++) for(String coluna:new String[]{"A","B"}) cad(TipoCadastro.ASSENTO,"
aeronave_id",aeronave,"codigo",fila+coluna,"fila",fila,"coluna",coluna,"classe","ECONOMICA","saida_emergen
cia",false);
```

Laço controlado por inicialização, condição e atualização: for(int fila=1;fila<=3;fila++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local fila do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Percorre cada elemento da coleção/vetor: for(String coluna:new String[]{"A","B"}). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
73      rota=cad(TipoCadastro.ROTA,"origem_id",origem,"destino_id",destino,"distancia_km","1000");
```

Atualiza rota usando = e o resultado de cad(TipoCadastro.ROTA,"origem\_id",origem,"destino\_id",destino,"distancia\_km","1000"). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

- 74 `passageiro=cad(TipoCadastro.PESSOA,"nome","Passageiro FICTICIO  
"+sufixo,"nascimento","1990-01-01","nacionalidade_id",pais);`  
Atualiza passageiro usando = e o resultado de `cad(TipoCadastro.PESSOA,"nome","Passageiro FICTICIO  
"+sufixo,"nascimento","1990-01-01","nacionalidade_id",pais)`. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.
- 75 `passageiros.vincular(admin,passageiro,"Assistencia  
ficticia");passageiros.vincular(admin,passageiro,"Assistencia ficticia");`  
Chama `PassageiroServico.vincular`: Exige id positivo e normaliza assistência opcional. Vincula o papel à pessoa existente e retorna `Passageiro`, sem duplicar pessoa.
- 76 `verificar(contar("SELECT COUNT(*) FROM passageiro WHERE pessoa_id="+passageiro)==1,"Vinculo nao  
duplica pessoa");`  
Verificação deste cenário: `contar("SELECT COUNT(*) FROM passageiro WHERE pessoa_id="+passageiro)==1`, "Vinculo nao duplica pessoa". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 77 `documento=cad(TipoCadastro.DOCUMENTO,"pessoa_id",passageiro,"pais_emissor_id",pais,"tipo","PASSAPO  
RTE","numero","FICTICIO-"+sufixo,"validade","2099-01-01");`  
Atualiza documento usando = e o resultado de `cad(TipoCadastro.DOCUMENTO,"pessoa_id",passageiro,"pais_emissor_id",pais,"tipo","PASSAPORTE","numero","FICTICIO-"+sufixo,"validade","2099....`. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.
- 78 `cad(TipoCadastro.CONTATO,"pessoa_id",passageiro,"tipo","EMAIL","valor",sufixo+"@example.invalid",  
principal",true);`  
Chama `IntegracaoTeste.cad`: Monta entrada do cadastro e chama serviço, devolvendo id.
- 79 `passageiros.editar(admin,passageiro,"Passageiro EDITADO FICTICIO  
"+sufixo,LocalDate.of(1990,1,1),pais,"",true);`  
Chama `PassageiroServico.editar`: Valida nome/nascimento/assistência, bloqueia pessoa e, ao inativar, impede passagens pendentes/confirmadas. Edita e audita o papel.
- 80 `verificar(passageiros.buscar(admin,passageiro).nome().contains("EDITADO"),"Edicao de passageiro");`  
Verificação deste cenário: `passageiros.buscar(admin,passageiro).nome().contains("EDITADO")`, "Edicao de passageiro". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 81 `outroPassageiro=cad(TipoCadastro.PESSOA,"nome","Outro FICTICIO  
"+sufixo,"nascimento","1991-01-01");passageiros.vincular(admin,outroPassageiro,"");`  
Atualiza `outroPassageiro` usando = e o resultado de `cad(TipoCadastro.PESSOA,"nome","Outro FICTICIO  
"+sufixo,"nascimento","1991-01-01")`. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.
- 82 `long cargo=cad(TipoCadastro.CARGO,"nome","Tripulacao FICTICIA "+sufixo,"area","TRIPULACAO");`  
Declara variável local `cargo` do tipo `long`. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 83 `cad(TipoCadastro.FUNCIONARIO,"pessoa_id",passageiro,"cargo_id",cargo,"aeroporto_base_id",origem,"m  
atricula","DUAL-"+sufixo,"admissao",LocalDate.now().minusDays(1));`  
Chama `IntegracaoTeste.cad`: Monta entrada do cadastro e chama serviço, devolvendo id.
- 84 `passageiros.editar(admin,passageiro,"Passageiro EDITADO FICTICIO  
"+sufixo,LocalDate.of(1990,1,1),pais,"",false);`  
Chama `PassageiroServico.editar`: Valida nome/nascimento/assistência, bloqueia pessoa e, ao inativar, impede passagens pendentes/confirmadas. Edita e audita o papel.
- 85 `verificar(contar("SELECT COUNT(*) FROM funcionario f JOIN pessoa p ON p.id=f.pessoa_id WHERE  
f.pessoa_id="+passageiro+" AND p.ativo")==1,"Inativacao de papel passageiro preserva funcionario");`  
Verificação deste cenário: `contar("SELECT COUNT(*) FROM funcionario f JOIN pessoa p ON p.id=f.pessoa_id WHERE  
f.pessoa_id="+passageiro+" AND p.ativo")==1`, "Inativacao de papel passageiro preserva funcionario". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 86 `passageiros.editar(admin,passageiro,"Passageiro EDITADO FICTICIO  
"+sufixo,LocalDate.of(1990,1,1),pais,"",true);`  
Instrução idêntica à de `IntegracaoTeste:L79`. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.
- 87 `long consulta=administracao.criarUsuario(admin,passageiro,"consulta_"+sufixo,"SenhaFicticia!2026".  
toCharArray(),Set.of("CONSULTA"));`  
Declara variável local `consulta` do tipo `long`. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 88 `var leitor=new AutenticacaoServico(banco,new  
UsuarioJdbc(),auditoria).entrar("consulta_"+sufixo,"SenhaFicticia!2026".toCharArray());`  
Declara variável local `leitor` com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- `new` constrói `AutenticacaoServico`, `UsuarioJdbc`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 89  | <pre>rejeitar()-&gt;cadastros.salvar(leitor,TipoCadastro.PAIS,null,mapa("codigo","XY","nome","Proibido")) ,"Permissoes revalidadas no servico");</pre> <p>Verificação deste cenário: ()-&gt;cadastros.salvar(leitor,TipoCadastro.PAIS,null,mapa("codigo","XY","nome","Proibido")), "Permissoes revalidadas no servico". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                       |
| 90  | <pre>administracao.acesso(admin,consulta,false,Set.of("CONSULTA"));rejeitar()-&gt;relatorios.consultar(le itor,TipoRelatorio.OCUPACAO,1),"Sessao bloqueada perde acesso");</pre> <p>Chama AdministracaoServico.acesso: Usa lock de acesso, atualiza atividade e substitui vínculos de perfis. Consulta se sobrou administrador ativo; se não, toda a mudança sofre rollback.</p> <p>Verificação deste cenário: ()-&gt;relatorios.consultar(leitor,TipoRelatorio.OCUPACAO,1), "Sessao bloqueada perde acesso". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.</p> |
| 91  | <pre>rejeitar()-&gt;administracao.acesso(admin,admin.usuarioId(),false,Set.of("ADMINISTRADOR")), "Ultimo administrador preservado");</pre> <p>Verificação deste cenário: ()-&gt;administracao.acesso(admin,admin.usuarioId(),false,Set.of("ADMINISTRADOR")), "Ultimo administrador preservado". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                                               |
| 92  | <pre>partida=religio.instant().plus(Duration.ofHours(3));chegada=partida.plus(Duration.ofHours(2));</pre> <p>Atualiza partida usando = e o resultado de religio.instant().plus(Duration.ofHours(3)). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.</p> <p>Atualiza chegada usando = e o resultado de partida.plus(Duration.ofHours(2)). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.</p>                                                                                                                                                                    |
| 93  | <pre>voo=planejamento.programar(admin,programacao(aeronave,partida,chegada));</pre> <p>Atualiza voo usando = e o resultado de planejamento.programar(admin,programacao(aeronave,partida,chegada)). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.</p>                                                                                                                                                                                                                                                                                                                                        |
| 94  | <pre>tarifa=planejamento.tarifa(admin,voo,null,new Tarifa("FICTICIA","ECONOMICA",new BigDecimal("100"),BigDecimal.ZERO,new BigDecimal("23"),1,true,new BigDecimal("10")),true);</pre> <p>Atualiza tarifa usando = e o resultado de planejamento.tarifa(admin,voo,null,new Tarifa("FICTICIA","ECONOMICA",new BigDecimal("100"),BigDecimal.ZERO,new BigDecimal("23"),1,true,ne.... Atribuição modifica a variável/célula indicada; não é um teste de igualdade.</p> <p>new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.</p>           |
| 95  | <pre>verificar(contar("SELECT COUNT(*) FROM inventario_assento_voo WHERE voo_id="+voo)==6,"Inventario gerado");</pre> <p>Verificação deste cenário: contar("SELECT COUNT(*) FROM inventario_assento_voo WHERE voo_id="+voo)==6, "Inventario gerado". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                                                                                                      |
| 96  | <pre>verificar(contar("SELECT COUNT(*) FROM agenda_aeronave WHERE voo_id="+voo)==1,"Agenda gerada");</pre> <p>Verificação deste cenário: contar("SELECT COUNT(*) FROM agenda_aeronave WHERE voo_id="+voo)==1, "Agenda gerada". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                                                                                                                            |
| 97  | <pre>rejeitar()-&gt;planejamento.programar(admin,programacao(aeronave,partida.plusSeconds(60),chegada.plu sSeconds(60))), "Conflito e continuidade da frota");</pre> <p>Verificação deste cenário: ()-&gt;planejamento.programar(admin,programacao(aeronave,partida.plusSeconds(60),chegada.plusSeconds(60))), "Conflito e continuidade da frota". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.</p>                                                                                                                                                            |
| 98  | <pre>verificar(contar("SELECT COUNT(*) FROM voo WHERE aeronave_id="+aeronave)==1,"Planejamento rejeitado nao grava parcialmente");</pre> <p>Verificação deste cenário: contar("SELECT COUNT(*) FROM voo WHERE aeronave_id="+aeronave)==1, "Planejamento rejeitado nao grava parcialmente". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                                                                |
| 99  | <pre>for(String funcao:new String[]{"COMANDANTE","COPILOTO","COMISSARIO"}) {</pre> <p>Percorre cada elemento da coleção/vetor: for(String funcao:new String[]{"COMANDANTE","COPILOTO","COMISSARIO"}). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.</p> <p>new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.</p>                                                                                                                                                                                         |
| 100 | <pre>long pessoa=cad(TipoCadastro.PESSOA,"nome",funcao+" FICTICIO "+sufixo,"nascimento","1985-01-01");</pre> <p>Declara variável local pessoa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p>                                                                                                                                                                                                                                                                                                                                                                       |
| 101 | <pre>cad(TipoCadastro.FUNCAONARIO,"pessoa_id",pessoa,"carga_id",carga,"aeroporto_base_id",origem,"m atricula",funcao+"-"+sufixo,"admissao",LocalDate.now().minusDays(1));</pre> <p>Chama IntegracaoTeste.cad: Monta entrada do cadastro e chama serviço, devolvendo id.</p>                                                                                                                                                                                                                                                                                                                                                |

102

```
operacao.habilitar(admin,pessoa,modelo,funcao,"FICTICIA-"+sufixo,LocalDate.now().plusYears(1));
```

Chama OperacaoServico.habilitar: Administra licença por funcionário/modelo/função com upsert; validade deve estar vigente e as escalas futuras são revalidadas.

103

```
operacao.escalar(admin,pessoa,origem,voo,funcao,partida.minus(Duration.ofMinutes(45)),chegada.  
plus(Duration.ofMinutes(30)));
```

Chama OperacaoServico.escalar: Aceita funções do catálogo, monta Map com dados da jornada e valida antes de INSERT. Voo pode ser null para funções de solo.

104

```
rejeitar(()->operacao.escalar(admin,pessoa,origem,voo,funcao,partida,chegada),"Escala  
conflitante ou incompleta rejeitada");
```

Verificação deste cenário: ()->operacao.escalar(admin,pessoa,origem,voo,funcao,partida,chegada), "Escala conflitante ou incompleta rejeitada". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

105

}

106

```
verificar(relatorios.mapa(admin,voo).celulas().length==3,"Mapa matriz integrado ao banco");
```

Verificação deste cenário: relatorios.mapa(admin,voo).celulas().length==3, "Mapa matriz integrado ao banco". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

107

}

108

## comercial - L109 a L125

Exercita inventário, agenda, preço vendido, pagamentos idempotentes, confirmação/bilhetes e expiração.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: reservas.reservar, List.of, new PedidoTrecho, numero, verificar, financeiro.pagar, new BigDecimal, rejeitar, contar, planejamento.trocarAeronave, relógio.instant, relógio.definir, anterior.plusSeconds, reservas.expirarPendencias.

109

```
private static void comercial() throws Exception {
```

Início da declaração de comercial. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

110

```
long reserva=reservas.reservar(admin,passageiro,List.of(new  
PedidoTrecho(passageiro,voo,tarifa,1,"1A",BigDecimal.ZERO)),15);
```

Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

111

```
long item=numero("SELECT id FROM item_reserva WHERE reserva_id="+reserva);
```

Declara variável local item do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

112

```
verificar(numero("SELECT valor_base FROM item_reserva WHERE id="+item)==100,"Preco copiado");
```

Verificação deste cenário: numero("SELECT valor\_base FROM item\_reserva WHERE id="+item)==100, "Preco copiado". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

113

```
long pagamento=financeiro.pagar(admin,reserva,"p-"+sufixo,"PIX",new BigDecimal("100"),"APROVADO");
```

Declara variável local pagamento do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

114

```
verificar(financeiro.pagar(admin,reserva,"p-"+sufixo,"PIX",new  
BigDecimal("100"),"APROVADO")==pagamento,"Pagamento idempotente");
```

Verificação deste cenário: financeiro.pagar(admin,reserva,"p-"+sufixo,"PIX",new BigDecimal("100"),"APROVADO")==pagamento, "Pagamento idempotente". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

115

```
rejeitar(()->financeiro.pagar(admin,reserva,"p-"+sufixo,"PIX",new  
BigDecimal("90"),"APROVADO"),"Idempotencia valida parametros");
```

Verificação deste cenário: ()->financeiro.pagar(admin,reserva,"p-"+sufixo,"PIX",new BigDecimal("90"),"APROVADO"), "Idempotencia valida parametros". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 116 | <pre> verificar(contar("SELECT COUNT(*) FROM bilhete WHERE item_id="+item)==1,"Bilhete emitido apos cobertura"); </pre> <p>Verificação deste cenário: contar("SELECT COUNT(*) FROM bilhete WHERE item_id="+item)==1, "Bilhete emitido apos cobertura". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                     |
| 117 | <pre> rejeitar(()-&gt;planejamento.trocarAeronave(admin,voo,aeronave,"Teste"),"Troca preserva itens historicos"); </pre> <p>Verificação deste cenário: ()-&gt;planejamento.trocarAeronave(admin,voo,aeronave,"Teste"), "Troca preserva itens historicos". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.</p>                                                                                                                                                                      |
| 118 | <pre> long outro=reservas.reservar(admin,outroPassageiro,List.of(new PedidoTrecho(outroPassageiro,voo,tarifa,1,"1B",BigDecimal.ZERO)),1); </pre> <p>Declara variável local outro do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p> <p>new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.</p>                                                                                                            |
| 119 | <pre> financeiro.pagar(admin,outro,"parcial-"+sufixo,"PIX",new BigDecimal("50"),"APROVADO"); </pre> <p>Chama FinanceiroServico.pagar: Valida chave/forma/valor/resultado. Uma chave existente precisa ter os mesmos dados; pendente pode ser processado posteriormente; já processado retorna mesmo id somente com resultado igual. Nova chave cria pagamento; aprovação pode confirmar reserva.</p> <p>new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.</p> |
| 120 | <pre> verificar(contar("SELECT COUNT(*) FROM bilhete b JOIN item_reserva i ON i.id=b.item_id WHERE i.reserva_id="+outro)==0,"Pagamento parcial nao emite bilhete"); </pre> <p>Verificação deste cenário: contar("SELECT COUNT(*) FROM bilhete b JOIN item_reserva i ON i.id=b.item_id WHERE i.reserva_id="+outro)==0, "Pagamento parcial nao emite bilhete". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                                                               |
| 121 | <pre> Instant anterior=religio.instant();religio.definir(anterior.plusSeconds(61));verificar(reservas.ex pirarPendencias())&gt;=1,"Expiracao executada"); </pre> <p>Declara variável local anterior do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p> <p>Verificação deste cenário: reservas.expirarPendencias()&gt;=1, "Expiracao executada". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                     |
| 122 | <pre> verificar(contar("SELECT COUNT(*) FROM ocupacao_assento o JOIN item_reserva i ON i.id=o.item_id WHERE i.reserva_id="+outro+" AND o.liberada_em IS NULL")==0,"Expiracao libera assento"); </pre> <p>Verificação deste cenário: contar("SELECT COUNT(*) FROM ocupacao_assento o JOIN item_reserva i ON i.id=o.item_id WHERE i.reserva_id="+outro+" AND o.liberada_em IS NULL")==0, "Expiracao libera assento". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                         |
| 123 | <pre> verificar(numero("SELECT SUM(valor) FROM reembolso WHERE reserva_id="+outro)==50,"Pagamento parcial expirado gera devolucao"); </pre> <p>Verificação deste cenário: numero("SELECT SUM(valor) FROM reembolso WHERE reserva_id="+outro)==50, "Pagamento parcial expirado gera devolucao". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                                                                                                                             |
| 124 | <pre> religio.definir(anterior); </pre> <p>Invoca religio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.</p>                                                                                                                                                                                                                                                                                                                             |
| 125 | }                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 126 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

## atendimento - L127 a L151

Simula janelas, check-in repetido, bagagem, excesso, recurso, embarque e rastreo usando Clock controlável.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: numero, rejeitar, atendimento.checkin, voos.transicao, atendimento.cancelarCheckin, verificar, atendimento.despachar, new BigDecimal, atendimento.rastrear, financeiro.pagar, operacao.alocar, partida.minus, Duration.ofMinutes, operacao.interditar, religio.definir, atendimento.cartao(admin,item).get("codigo\_cartao").toString, atendimento.cartao(admin,item).get, atendimento.cartao, atendimento.embarcar, contar, chegada.plusSeconds, planejamento.horarios, relatorios.bagagem(admin,"M-"+sufixo).size, relatorios.bagagem.

|     |                                                                                                                                                                                              |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 127 | <pre> private static void atendimento() { </pre> <p>Início da declaração de atendimento. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.</p> |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|



|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 128 | <pre>long item=numero("SELECT i.id FROM item_reserva i WHERE i.voo_id="+voo+" AND i.passageiro_id="+passageiro);</pre> <p>Declara variável local item do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p>                                                                                                                                                                                                                                                                               |
| 129 | <pre>rejeitar()-&gt;atendimento.checkin(admin,item,documento),"Estado de voo bloqueia check-in");</pre> <p>Verificação deste cenário: ()-&gt;atendimento.checkin(admin,item,documento), "Estado de voo bloqueia check-in". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                                    |
| 130 | <pre>voos.transicao(admin,voo,SituacaoVoo.CHECKIN_ABERTO,"Teste");</pre> <p>Chama VooServico.transicao: Expira pendências, reúne reservas/voos relacionados e bloqueia em ordem. Valida transição enum. Cada próximo estado tem efeitos próprios: janela/equipe/portão, ausência na partida, liberação na conclusão ou cancelamento com devolução e ocorrência. Atualiza o voo e audita.</p>                                                                                                                                               |
| 131 | <pre>long ci=atendimento.checkin(admin,item,documento);atendimento.cancelarCheckin(admin,item);</pre> <p>Declara variável local ci do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p>                                                                                                                                                                                                                                                                                                  |
| 132 | <pre>verificar(atendimento.checkin(admin,item,documento)==ci,"Refaz check-in no mesmo registro");</pre> <p>Verificação deste cenário: atendimento.checkin(admin,item,documento)==ci, "Refaz check-in no mesmo registro". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                                                  |
| 133 | <pre>long mala=atendimento.despachar(admin,item,"M-"+sufixo,new BigDecimal("25"),"DESPACHADA");</pre> <p>Declara variável local mala do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p> <p>new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.</p>                                                                                                                                                         |
| 134 | <pre>verificar(numero("SELECT taxa_excesso FROM bagagem WHERE id="+mala)==60,"Excesso calculado");</pre> <p>Verificação deste cenário: numero("SELECT taxa_excesso FROM bagagem WHERE id="+mala)==60, "Excesso calculado". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p>                                                                                                                                                                                                                |
| 135 | <pre>verificar(atendimento.despachar(admin,item,"M-"+sufixo,new BigDecimal("25"),"DESPACHADA")==mala,"Etiqueta nao duplica despacho");</pre> <p>Verificação deste cenário: atendimento.despachar(admin,item,"M-"+sufixo,new BigDecimal("25"),"DESPACHADA")==mala, "Etiqueta nao duplica despacho". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.</p> <p>new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.</p> |
| 136 | <pre>atendimento.rastrear(admin,"M-"+sufixo,origem,"INSPECIONADA","Teste");</pre> <p>Chama AtendimentoServico.rastrear: Confere transição no mapa BAGAGEM, local do evento, chegada para descarregar/entregar e cobertura para carregar. RETIRADA solicita devolução da taxa; atualiza e registra histórico.</p>                                                                                                                                                                                                                           |
| 137 | <pre>rejeitar()-&gt;atendimento.rastrear(admin,"M-"+sufixo,origem,"CARREGADA","Teste"),"Excesso nao pago bloqueia carga");</pre> <p>Verificação deste cenário: ()-&gt;atendimento.rastrear(admin,"M-"+sufixo,origem,"CARREGADA","Teste"), "Excesso nao pago bloqueia carga". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.</p>                                                                                                                                                  |
| 138 | <pre>long reserva=numero("SELECT reserva_id FROM item_reserva WHERE id="+item);financeiro.pagar(admin,reserva,"bag-"+sufixo,"PIX",new BigDecimal("60"),"APROVADO");</pre> <p>Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p> <p>new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.</p>                                                                                  |
| 139 | <pre>atendimento.rastrear(admin,"M-"+sufixo,origem,"CARREGADA","Teste");</pre> <p>Chama AtendimentoServico.rastrear: Confere transição no mapa BAGAGEM, local do evento, chegada para descarregar/entregar e cobertura para carregar. RETIRADA solicita devolução da taxa; atualiza e registra histórico.</p>                                                                                                                                                                                                                              |
| 140 | <pre>long<br/>alocacao=operacao.alocar(admin,voo,portao,"PARTIDA",partida.minus(Duration.ofMinutes(45)),partida);</pre> <p>Declara variável local alocacao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p>                                                                                                                                                                                                                                                                          |
| 141 | <pre>rejeitar()-&gt;operacao.interditar(admin,portao,partida.minus(Duration.ofMinutes(30)),partida,"Teste<br/>"),"Interdicao conflitante rejeitada");</pre> <p>Verificação deste cenário: ()-&gt;operacao.interditar(admin,portao,partida.minus(Duration.ofMinutes(30)),partida,"Teste"), "Interdicao conflitante rejeitada". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.</p>                                                                                                 |
| 142 | <pre>relógio.definir(partida.minus(Duration.ofMinutes(35)));voos.transicao(admin,voo,SituacaoVoo.EMBARQUE,"Teste");</pre> <p>Invoca relógio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.</p>                                                                                                                                                                                                                                          |
| 143 | <pre>String codigo=atendimento.cartao(admin,item).get("codigo_cartao").toString();atendimento.embarcar(admin,codigo,alocacao);</pre> <p>Declara variável local codigo do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.</p>                                                                                                                                                                                                                                                             |

```

144      verificar(contar("SELECT COUNT(*) FROM bilhete WHERE item_id="+item+" AND
situacao='UTILIZADO')")==1,"Embarque consome bilhete");
Verificação deste cenário: contar("SELECT COUNT(*) FROM bilhete WHERE item_id="+item+" AND situacao='UTILIZADO')")==1,
"Embarque consome bilhete". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

145      rejeitar(()->atendimento.embarcar(admin,codigo,alocacao),"Embarque nao se repete");
Verificação deste cenário: ()->atendimento.embarcar(admin,codigo,alocacao), "Embarque nao se repete". A operação/condição
deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

146      relógio.definir(partida);voos.transicao(admin,voo,SituacaoVoo.EM_VOO,"Teste");
Invoca relógio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
método para localizar o corpo no projeto.

147      relógio.definir(chegada.plusSeconds(1));planejamento.horarios(admin,voo,partida,chegada,true,"Chegada
ficticia");voos.transicao(admin,voo,SituacaoVoo.CONCLUIDO,"Teste");
Invoca relógio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
método para localizar o corpo no projeto.

148      atendimento.rastrear(admin,"M-"+sufixo,destino,"DESCARREGADA","Teste");atendimento.rastrear(admin,
"M-"+sufixo,destino,"ENTREGUE","Teste");
Chama AtendimentoServico.rastrear: Confere transição no mapa BAGAGEM, local do evento, chegada para descarregar/entregar
e cobertura para carregar. RETIRADA solicita devolução da taxa; atualiza e registra histórico.

149      verificar(relatorios.bagagem(admin,"M-"+sufixo).size()==5,"Historico de bagagem completo");
Verificação deste cenário: relatorios.bagagem(admin,"M-"+sufixo).size()==5, "Historico de bagagem completo". A
operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

150      verificar(contar("SELECT COUNT(*) FROM historico_voo WHERE voo_id="+voo+" AND usuario_id IS
NULL")==0,"Historico de voo tem autor");
Verificação deste cenário: contar("SELECT COUNT(*) FROM historico_voo WHERE voo_id="+voo+" AND usuario_id IS
NULL")==0, "Historico de voo tem autor". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste
falhar.

151      }
152

```

## arquivos - L153 a L162

Exporta/copia relatório e testa rejeição de importação sem gravação parcial.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: new ArquivosServico, relatorios.exportar, arquivos.copiarRelatorio, verificar, Files.mismatch, Path.of, Files.writeString, contar, rejeitar, arquivos.importarPassageiros, new RuntimeException, arquivos.listar(admin).isEmpty, arquivos.listar.

```

153      private static void arquivos() throws Exception {
Início da declaração de arquivos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é
invocado.

154      var arquivos=new ArquivosServico(banco,autorizacao,auditoria,relógio);
Declara variável local arquivos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
escopo é o bloco atual.

new constrói ArquivosServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por
si só.

155      Path csv=relatorios.exportar(admin,TipoRelatorio.MANIFESTO,true);
Declara variável local csv do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

156      Path copia=arquivos.copiarRelatorio(admin,csv);verificar(Files.mismatch(csv,copia)==-1,"Backup de
arquivo verificado");
Declara variável local copia do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Verificação deste cenário: Files.mismatch(csv,copia)==-1, "Backup de arquivo verificado". A operação/condição deve resultar em
verdadeiro; divergência faz o programa de teste falhar.

157      Path entrada=Path.of("artifacts","importacao-"+sufixo+".txt");
Declara variável local entrada do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

158      Files.writeString(entrada,"nome\t nascimento\t cpf\t email\t telefone\nFICTICIO\t1990-01-01\t529982247
25\tficticio@example.invalid\t\nINVALIDO\t2099-01-01\t123\t\t\n");
Invoca Files.writeString com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
método para localizar o corpo no projeto.

```



```
159      long antes=contar("SELECT COUNT(*) FROM
pessoa");rejeitar()->{try{arquivos.importarPassageiros(admin,entrada);}catch(java.io.IOException e){throw
new RuntimeException(e);}}, "Importacao invalida rejeitada");
```

Declara variável local antes do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Verificação deste cenário: ()->{try{arquivos.importarPassageiros(admin,entrada);}catch(java.io.IOException e){throw new RuntimeException(e);}}, "Importacao invalida rejeitada". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Trata a(s) exceção(ões) declarada(s) em catch(java.io.IOException e). O tratamento mostrado abaixo decide se avisa, repete ou relança.

Lança new RuntimeException(e). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RuntimeException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
160      verificar(contar("SELECT COUNT(*) FROM pessoa")==antes,"Importacao nao grava parcialmente");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM pessoa")==antes, "Importacao nao grava parcialmente". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
161      verificar(!arquivos.listar(admin).isEmpty(),"Arquivos disponiveis no menu");
```

Verificação deste cenário: !arquivos.listar(admin).isEmpty(), "Arquivos disponiveis no menu". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
162      }
```

```
163
```

## concorrencia - L164 a L188

ExecutorService e barreiras fazem chamadas simultâneas para mesmo assento/pagamento. Futures permitem observar os resultados de cada thread.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: cad, relógio.instant().plus, relógio.instant, Duration.ofHours, planejamento.programar, programacao, inicio.plus, planejamento.tarifa, new Tarifa, new BigDecimal, Executors.newFixedThreadPool, new CountDownLatch, new ArrayList<Future<Boolean>>, tarefas.add, executor.submit, inicioSimultaneo.await, reservas.reservar, List.of, new PedidoTrecho, inicioSimultaneo.countDown, tarefa.get, verificar, contar, numero, financeiro.pagar, a.get(40, TimeUnit.SECONDS).equals, a.get, b.get, reservas.cancelar, financeiro.processarReembolso, rejeitar, financeiro.solicitarReembolso.

```
164      private static void concorrancia() throws Exception {
```

Início da declaração de concorrancia. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
165          // Nova aeronave para evitar conflito com a localizacao da aeronave concluida.
```

Comentário/documentação do autor; descreve intenção ou convenção e não executa.

```
166          long aviao=cad(TipoCadastro.AERONAVE,"modelo_id",modelo,"companhia_id",companhia,"matricula","PC-"
+sufixo,"situacao","ATIVA","fabricacao_ano",2025);
```

Declara variável local aviao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
167          cad(TipoCadastro.ASENT0,"aeronave_id",aviao,"codigo","1A","fila",1,"coluna","A","classe","ECONOMI
CA","saida_emergencia",false);
```

Chama IntegracaoTeste.cad: Monta entrada do cadastro e chama serviço, devolvendo id.

```
168          Instant inicio=relógio.instant().plus(Duration.ofHours(3));long
flight=planejamento.programar(admin,programacao(aviao,inicio,inicio.plus(Duration.ofHours(2))));
```

Declara variável local inicio do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local flight do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
169          long fare=planejamento.tarifa(admin,flight,null,new Tarifa("TESTE","ECONOMICA",new
BigDecimal("100"),BigDecimal.ZERO,BigDecimal.ZERO,0,true,BigDecimal.ZERO),true);
```

Declara variável local fare do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
170          try(var executor=Executors.newFixedThreadPool(2)) {
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

```
171      var inicioSimultaneo=new CountDownLatch(1);var tarefas=new ArrayList<Future<Boolean>>();
```

Declara variável local inicioSimultaneo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local tarefas com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói CountDownLatch, ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
172      for(long pessoa:new long[]{passageiro,outroPassageiro})
tarefas.add(executor.submit()->{inicioSimultaneo.await();try{reservas.reservar(admin,pessoa,List.of(new
PedidoTrecho(pessoa,flight,fare,1,"1A",BigDecimal.ZERO)),15);return true;}catch(RegraNegocioException |
PersistenciaException esperado){return false;}}));
```

Percorre cada elemento da coleção/vetor: for(long pessoa:new long[]{passageiro,outroPassageiro}). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Retorno: entrega true ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Trata a(s) exceção(ões) declarada(s) em catch(RegraNegocioException | PersistenciaException esperado). O tratamento mostrado abaixo decide se avisa, repete ou relança.

Retorno: entrega false ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói long, PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
173      inicioSimultaneo.countDown();int sucessos=0;for(var tarefa:tarefas)
if(tarefa.get(40,TimeUnit.SECONDS)) sucessos++;
```

Invoca inicioSimultaneo.countDown com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

Declara variável local sucessos do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Percorre cada elemento da coleção/vetor: for(var tarefa:tarefas). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Decisão específica: (tarefa.get(40,TimeUnit.SECONDS)). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
174      verificar(sucessos==1,"Dois operadores nao vendem o mesmo assento");
```

Verificação deste cenário: sucessos==1, "Dois operadores nao vendem o mesmo assento". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
175      }
176      verificar(contar("SELECT COUNT(*) FROM item_reserva WHERE voo_id="+flight)==1,"Concorrenca nao
cria item orfao");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM item\_reserva WHERE voo\_id="+flight)==1, "Concorrenca nao cria item orfao". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
177      long reserva=numero("SELECT reserva_id FROM item_reserva WHERE voo_id="+flight);
```

Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
178      try(var executor=Executors.newFixedThreadPool(2)) {
```

Instrução idêntica à de IntegracaoTeste:L170. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
179      var a=executor.submit()->financeiro.pagar(admin,reserva,"conc-"+sufixo,"PIX",new
BigDecimal("100"),"APROVADO"));
```

Declara variável local a com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
180      var b=executor.submit()->financeiro.pagar(admin,reserva,"conc-"+sufixo,"PIX",new
BigDecimal("100"),"APROVADO"));
```

Declara variável local b com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
181      verificar(a.get(40,TimeUnit.SECONDS).equals(b.get(40,TimeUnit.SECONDS)),"Pagamento simultaneo
idempotente");
```

Verificação deste cenário: a.get(40,TimeUnit.SECONDS).equals(b.get(40,TimeUnit.SECONDS)), "Pagamento simultaneo idempotente". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```

182     }
183     reservas.cancelar(admin,reserva,null,"Cancelamento FICTICIO");long refund=numero("SELECT id FROM
reembolso WHERE reserva_id="+reserva);

```

Chama ReservaServico.cancelar: Item null representa toda a reserva; um id limita a seleção. Soma os direitos de devolução dos itens, solicita reembolso e recalcula cabeçalho, preservando histórico.

Declara variável local refund do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```

184     financeiro.processarReembolso(admin,refund,true);financeiro.processarReembolso(admin,refund,true);

```

Chama FinanceiroServico.processarReembolso: Repetir o destino já alcançado não muda nada. Só SOLICITADO pode ir a PROCESSADO/RECUSADO; confere compromissos contra pagamento e audita.

```

185     verificar(numero("SELECT SUM(valor) FROM reembolso WHERE reserva_id="+reserva+" AND
situacao='PROCESSADO'")==100,"Reembolso nao se repete");

```

Verificação deste cenário: numero("SELECT SUM(valor) FROM reembolso WHERE reserva\_id="+reserva+" AND situacao='PROCESSADO'")==100, "Reembolso nao se repete". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```

186     rejeitar()->financeiro.solicitarReembolso(admin,numero("SELECT id FROM pagamento WHERE
reserva_id="+reserva),new BigDecimal("1"),"excedente-"+sufixo,"Teste"),"Reembolso acima do pago
rejeitado");

```

Verificação deste cenário: ()->financeiro.solicitarReembolso(admin,numero("SELECT id FROM pagamento WHERE reserva\_id="+reserva),new BigDecimal("1"),"excedente-"+sufixo,"Teste"), "Reembolso acima do pago rejeitado". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

187     verificar(contar("SELECT COUNT(*) FROM ocupacao_assento o JOIN item_reserva i ON i.id=o.item_id
WHERE i.reserva_id="+reserva+" AND o.liberada_em IS NULL")==0,"Cancelamento libera assento");

```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM ocupacao\_assento o JOIN item\_reserva i ON i.id=o.item\_id WHERE i.reserva\_id="+reserva+" AND o.liberada\_em IS NULL")==0, "Cancelamento libera assento". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```

188     }
189

```

## cancelamentos - L190 a L201

Testa devolução/limites, liberação de assento, tarifa não cancelável e cancelamento integral de voo.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: cad, relógio.instant().plus, relógio.instant, Duration.ofHours, planejamento.programar, programacao, inicio.plus, planejamento.tarifa, new Tarifa, new BigDecimal, reservas.reservar, List.of, new PedidoTrecho, financeiro.pagar, rejeitar, reservas.cancelar, voos.transicao, verificar, numero, contar.

```

190     private static void cancelamentos() {

```

Início da declaração de cancelamentos. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

191         long aviao=cad(TipoCadastro.AERONAVE,"modelo_id",modelo,"companhia_id",companhia,"matricula","PV-"
+sufixo,"situacao","ATIVA","fabricacao_ano",2025);

```

Declara variável local aviao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```

192         cad(TipoCadastro.ASENT0,"aeronave_id",aviao,"codigo","1A","fila",1,"coluna","A","classe","ECONOMI
CA","saida_emergencia",false);

```

Instrução idêntica à de IntegracaoTeste:L167. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

193         Instant inicio=relógio.instant().plus(Duration.ofHours(3));long
flight=planejamento.programar(admin,programacao(aviao,inicio,inicio.plus(Duration.ofHours(2))));

```

Instrução idêntica à de IntegracaoTeste:L168. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

194         long fare=planejamento.tarifa(admin,flight,null,new Tarifa("TESTE","ECONOMICA",new
BigDecimal("100"),BigDecimal.ZERO,BigDecimal.ZERO,0,false,new BigDecimal("50")),true);

```

Declara variável local fare do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
195 long reserva=reservas.reservar(admin,outroPassageiro,List.of(new PedidoTrecho(outroPassageiro,flight,fare,1,null,BigDecimal.ZERO)),15);financeiro.pagar(admin,reserva,"cancelvoo-"+sufixo,"PIX",new
BigDecimal("100"),"APROVADO");
```

Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói PedidoTrecho, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
196 rejeitar()->reservas.cancelar(admin,reserva,null,"Teste"),"Tarifa nao cancelavel respeitada");
```

Verificação deste cenário: ()->reservas.cancelar(admin,reserva,null,"Teste"), "Tarifa nao cancelavel respeitada". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

```
197 voos.transicao(admin,flight,SituacaoVoo.CANCELADO,"Cancelamento pela companhia FICTICIA");
```

Chama VooServico.transicao: Expira pendências, reúne reservas/voos relacionados e bloqueia em ordem. Valida transição enum. Cada próximo estado tem efeitos próprios: janela/equipe/portão, ausência na partida, liberação na conclusão ou cancelamento com devolução e ocorrência. Atualiza o voo e audita.

```
198 verificar(numero("SELECT SUM(valor) FROM reembolso WHERE reserva_id="+reserva)==100,"Cancelamento
de voo restitui integralmente");
```

Verificação deste cenário: numero("SELECT SUM(valor) FROM reembolso WHERE reserva\_id="+reserva)==100, "Cancelamento de voo restitui integralmente". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
199 verificar(contar("SELECT COUNT(*) FROM agenda_aeronave WHERE voo_id="+flight+" AND
ativa")==0,"Cancelamento libera agendas");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM agenda\_aeronave WHERE voo\_id="+flight+" AND ativa")==0, "Cancelamento libera agendas". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
200 verificar(contar("SELECT COUNT(*) FROM bilhete b JOIN item_reserva i ON i.id=b.item_id WHERE
i.reserva_id="+reserva+" AND b.situacao='CANCELADO'")==1,"Cancelamento preserva bilhete anterior");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM bilhete b JOIN item\_reserva i ON i.id=b.item\_id WHERE i.reserva\_id="+reserva+" AND b.situacao='CANCELADO'")==1, "Cancelamento preserva bilhete anterior". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
201 }
```

```
202
```

## cenariosAdicionais - L203 a L293

Cria grupo com conexões/comprador diferente, remarcação, rollback no segundo item, tarifas, troca, agendas, recursos, manutenção, solo, relatórios e ausência.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: relógio.instant, novoAviao, base.plus, Duration.ofHours, inicio.plus, planejamento.programar, programacao, cad, fim.plus, inicioVolta.plus, new ProgramacaoVoo, p.numero, p.partida, p.chegada, p.checkinAbre, p.checkinFecha, p.embarqueAbre, p.embarqueFecha, planejamento.tarifa, new Tarifa, new BigDecimal, List.of, new PedidoTrecho, reservas.reservar, verificar, numero, contar, financeiro.pagar, reservas.cancelar, texto("SELECT situacao FROM reserva WHERE id="+reserva).equals, texto, financeiro.processarReembolso, rejeitar, financeiro.solicitarReembolso, reservas.remarcar, texto("SELECT situacao FROM item\_reserva WHERE id="+novoltem).equals, planejamento.bloquearAssento, planejamento.trocarAeronave, banco.consultar, Sql.registro, Dados.instante(atual,"partida\_prevista").plus, Dados.instante, Dados.instante(atual,"chegada\_prevista").plus, planejamento.reprogramar, atual.get("numero").toString, atual.get, Dados.instante(atual,"checkin\_abre").plus, Dados.instante(atual,"checkin\_fecha").plus, Dados.instante(atual,"embarque\_abre").plus, Dados.instante(atual,"embarque\_fecha").plus, operacao.alocar, novaPartida.minus, Duration.ofMinutes, planejamento.horarios, novaPartida.plus, novaChegada.plus, banco.consultar(c->Dados.instante(Sql.registro(c,"SELECT inicio FROM alocao\_recurso WHERE id=?",alocacao),"inicio")).equals, banco.consultar(c->Dados.instante(Sql.registro(c,"SELECT inicio FROM agenda\_aeronave WHERE voo\_id=?",semVendas),"inicio")).equals, relatorios.resumoAtrasos(admin).get("maximo").compareTo, relatorios.resumoAtrasos(admin).get, relatorios.resumoAtrasos, novaPartida.plusSeconds, operacao.interditar, base.plusSeconds, operacao.liberarAlocacao, operacao.manutencao, admin.funcionario().id, admin.funcionario, base.minusSeconds, operacao.situacaoManutencao, texto("SELECT situacao FROM manutencao\_aeronave WHERE id="+manutencao).equals, operacao.servicoSolo, operacao.situacaoSolo, operacao.ocorrencia, operacao.encerrarOcorrencia, TipoRelatorio.values, relatorios.consultar, relatorios.exportar, relatorios.painel(admin,origem,inicio.atZone(ZoneOffset.UTC).toLocalDate(),true).isEmpty, relatorios.painel, inicio.atZone(ZoneOffset.UTC).toLocalDate, inicio.atZone, relatorios.painel(admin,destino,fim.atZone(ZoneOffset.UTC).toLocalDate(),false).isEmpty, fim.atZone(ZoneOffset.UTC).toLocalDate, fim.atZone, bagagensEmConexao, new DadosFicticiosServico, demo.criar, sufixo.substring, dados.equals, Dados.id, Files.readAllLines(csv).size, Files.readAllLines, voos.transicao, atendimento.checkin, atendimento.despachar, atendimento.cancelarCheckin, relógio.definir, saida.minus, texto("SELECT situacao FROM item\_reserva WHERE id="+demoltem).equals, cancelamentoAposEmbarque, falhasEDeadlock, testarTerminal.

```
203 private static void cenariosAdicionais() throws Exception {
```

Início da declaração de cenariosAdicionais. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
204 Instant base=relógio.instant();
```

Declara variável local base do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

|     |                                                                                                                                                                                                                                                                                                                    |                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 205 | <code>long aviao=novoAviao("GX",6);</code>                                                                                                                                                                                                                                                                         | Declara variável local aviao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                  |
| 206 | <code>Instant inicio=base.plus(Duration.ofHours(3)),fim=inicio.plus(Duration.ofHours(2));</code>                                                                                                                                                                                                                   | Declara variável local inicio, fim do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                         |
| 207 | <code>long primeiro=planejamento.programar(admin,programacao(aviao,inicio,fim));</code>                                                                                                                                                                                                                            | Declara variável local primeiro do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                               |
| 208 | <code>long reversa=cad(TipoCadastro.ROTA,"origem_id",destino,"destino_id",origem,"distancia_km","1000");</code>                                                                                                                                                                                                    | Declara variável local reversa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                |
| 209 | <code>Instant inicioVolta=fim.plus(Duration.ofHours(2)),fimVolta=inicioVolta.plus(Duration.ofHours(2));</code>                                                                                                                                                                                                     | Declara variável local inicioVolta, fimVolta do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                               |
| 210 | <code>var p=programacao(aviao,inicioVolta,fimVolta);</code>                                                                                                                                                                                                                                                        | Declara variável local p com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                              |
| 211 | <code>long segundo=planejamento.programar(admin,new ProgramacaoVoo(companhia,reversa,aviao,p.numero(),p.partida(),p.chegada(),p.checkinAbre(),p.checkinFecha(),p.embarqueAbre(),p.embarqueFecha()));</code>                                                                                                        | Declara variável local segundo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói ProgramacaoVoo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                     |
| 212 | <code>long fare=planejamento.tarifa(admin,primeiro,null,new Tarifa("GRUPO","ECONOMICA",new BigDecimal("100"),BigDecimal.ZERO,new BigDecimal("23"),1,true,new BigDecimal("10")),true);</code>                                                                                                                       | Declara variável local fare do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                    |
| 213 | <code>long volta=planejamento.tarifa(admin,segundo,null,new Tarifa("GRUPO","ECONOMICA",new BigDecimal("100"),BigDecimal.ZERO,new BigDecimal("23"),1,true,new BigDecimal("10")),true);</code>                                                                                                                       | Declara variável local volta do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                   |
| 214 | <code>long comprador=cad(TipoCadastro.PESSOA,"nome","Comprador que nao viaja FICTICIO "+sufixo,"nascimento","1980-01-01");</code>                                                                                                                                                                                  | Declara variável local comprador do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                              |
| 215 | <code>var grupo=List.of(new PedidoTrecho(passageiro,primeiro,fare,1,null,BigDecimal.ZERO),new PedidoTrecho(passageiro,segundo,volta,2,null,BigDecimal.ZERO),new PedidoTrecho(outroPassageiro,primeiro,fare,1,null,BigDecimal.ZERO),new PedidoTrecho(outroPassageiro,segundo,volta,2,null,BigDecimal.ZERO));</code> | Declara variável local grupo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                 |
| 216 | <code>long reserva=reservas.reservar(admin,comprador,grupo,15);</code>                                                                                                                                                                                                                                             | Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                |
| 217 | <code>verificar(numero("SELECT comprador_id FROM reserva WHERE id="+reserva)==comprador &amp;&amp; contar("SELECT COUNT(*) FROM item_reserva WHERE reserva_id="+reserva)==4,"Grupo e conexoes preservam comprador e passageiros");</code>                                                                          | Verificação deste cenário: numero("SELECT comprador_id FROM reserva WHERE id="+reserva)==comprador && contar("SELECT COUNT(*) FROM item_reserva WHERE reserva_id="+reserva)==4, "Grupo e conexoes preservam comprador e passageiros". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.                                                                                               |
| 218 | <code>financeiro.pagar(admin,reserva,"grupo-"+sufixo,"PIX",new BigDecimal("400"),"APROVADO");</code>                                                                                                                                                                                                               | Chama FinanceiroServico.pagar: Valida chave/forma/valor/resultado. Uma chave existente precisa ter os mesmos dados; pendente pode ser processado posteriormente; já processado retorna mesmo id somente com resultado igual. Nova chave cria pagamento; aprovação pode confirmar reserva.<br><br>new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só. |
| 219 | <code>long item=numero("SELECT MIN(id) FROM item_reserva WHERE reserva_id="+reserva);</code>                                                                                                                                                                                                                       | Declara variável local item do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                   |



- 220 `reservas.cancelar(admin,reserva,item,"Parcial FICTICIO");`  
Chama ReservaServico.cancelar: Item null representa toda a reserva; um id limita a seleção. Soma os direitos de devolução dos itens, solicita reembolso e recalcula cabeçalho, preservando histórico.
- 221 `verificar(texto("SELECT situacao FROM reserva WHERE id="+reserva).equals("PARCIAL_CANCELADA"),"Cancelamento parcial recalcula cabecalho");`  
Verificação deste cenário: texto("SELECT situacao FROM reserva WHERE id="+reserva).equals("PARCIAL\_CANCELADA"), "Cancelamento parcial recalcula cabecalho". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 222 `verificar(numero("SELECT SUM(valor) FROM reembolso WHERE reserva_id="+reserva)==90,"Multa vendida aplicada");`  
Verificação deste cenário: numero("SELECT SUM(valor) FROM reembolso WHERE reserva\_id="+reserva)==90, "Multa vendida aplicada". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 223 `long reembolso=numero("SELECT id FROM reembolso WHERE reserva_id="+reserva);financeiro.processarReembolso(admin,reembolso,false);`  
Declara variável local reembolso do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 224 `rejeitar(()->financeiro.solicitarReembolso(admin,numero("SELECT id FROM pagamento WHERE reserva_id="+reserva),new BigDecimal("91"),"multa-"+sufixo,"Teste"),"Solicitacao manual nao ignora multa");`  
Verificação deste cenário: ()->financeiro.solicitarReembolso(admin,numero("SELECT id FROM pagamento WHERE reserva\_id="+reserva),new BigDecimal("91"),"multa-"+sufixo,"Teste"), "Solicitacao manual nao ignora multa". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.  
  
new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 225 `long pedido=financeiro.solicitarReembolso(admin,numero("SELECT id FROM pagamento WHERE reserva_id="+reserva),new BigDecimal("90"),"manual-"+sufixo,"Teste");financeiro.processarReembolso(admin,pedido,true);`  
Declara variável local pedido do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 226 `long pendente=reservas.reservar(admin,passageiro,List.of(new PedidoTrecho(passageiro,primeiro,fare,1,null,BigDecimal.ZERO)),15);`  
Declara variável local pendente do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 227 `long novoItem=numero("SELECT id FROM item_reserva WHERE reserva_id="+pendente);`  
Declara variável local novoltem do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 228 `long remarcada=reservas.remarcar(admin,novoItem,new PedidoTrecho(passageiro,primeiro,fare,1,null,BigDecimal.ZERO),15);`  
Declara variável local remarcada do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 229 `verificar(texto("SELECT situacao FROM item_reserva WHERE id="+novoItem).equals("CANCELADO") && remarcada!=pendente,"Remarcacao preserva item e reserva anteriores");`  
Verificação deste cenário: texto("SELECT situacao FROM item\_reserva WHERE id="+novoItem).equals("CANCELADO") && remarcada!=pendente, "Remarcacao preserva item e reserva anteriores". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 230 `reservas.cancelar(admin,remarcada,null,"Teste");`  
Chama ReservaServico.cancelar: Item null representa toda a reserva; um id limita a seleção. Soma os direitos de devolução dos itens, solicita reembolso e recalcula cabeçalho, preservando histórico.
- 231 `long quantidade=contar("SELECT COUNT(*) FROM reserva");`  
Declara variável local quantidade do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 232 `rejeitar(()->reservas.reservar(admin,passageiro,List.of(new PedidoTrecho(passageiro,primeiro,fare,1,"3A",BigDecimal.ZERO),new PedidoTrecho(passageiro,primeiro,fare,2,"3B",BigDecimal.ZERO)),15),"Duplicidade de passageiro falha durante gravacao");`  
Verificação deste cenário: ()->reservas.reservar(admin,passageiro,List.of(new PedidoTrecho(passageiro,primeiro,fare,1,"3A",BigDecimal.ZERO),new PedidoTrecho(passageiro,primeiro,fare,2,"3B",BigDecimal.ZERO)),15), "Duplicidade de passageiro falha durante gravacao". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
233      verificar(contar("SELECT COUNT(*) FROM reserva")==quantidade,"Falha no segundo item desfaz toda a transacao");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM reserva")==quantidade, "Falha no segundo item desfaz toda a transacao". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
234      rejeitar(()->reservas.reservar(admin,passageiro,List.of(new
PedidoTrecho(passageiro,primeiro,fare,1,null,new BigDecimal("11"))),15),"Desconto fora da politica rejeitado");
```

Verificação deste cenário: ()->reservas.reservar(admin,passageiro,List.of(new PedidoTrecho(passageiro,primeiro,fare,1,null,new BigDecimal("11"))),15), "Desconto fora da política rejeitado". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

new constrói PedidoTrecho, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
235      long inventario=numero("SELECT id FROM inventario_assento_voo WHERE voo_id="+primeiro+" AND
codigo='3A'");planejamento.bloquearAssento(admin,inventario,true);
```

Declara variável local inventario do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
236      rejeitar(()->reservas.reservar(admin,passageiro,List.of(new
PedidoTrecho(passageiro,primeiro,fare,1,"3A",BigDecimal.ZERO)),15),"Assento bloqueado nao vendido");planejamento.bloquearAssento(admin,inventario,false);
```

Verificação deste cenário: ()->reservas.reservar(admin,passageiro,List.of(new PedidoTrecho(passageiro,primeiro,fare,1,"3A",BigDecimal.ZERO)),15), "Assento bloqueado nao vendido". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
237      long executiva=planejamento.tarifa(admin,primeiro,null,new Tarifa("EXECUTIVA","EXECUTIVA",new
BigDecimal("200"),BigDecimal.ZERO,BigDecimal.ZERO,0,true,BigDecimal.ZERO),true);
```

Declara variável local executiva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
238      rejeitar(()->reservas.reservar(admin,passageiro,List.of(new
PedidoTrecho(passageiro,primeiro,executiva,1,"3A",BigDecimal.ZERO)),15),"Classe de assento incompativel rejeitada");
```

Verificação deste cenário: ()->reservas.reservar(admin,passageiro,List.of(new PedidoTrecho(passageiro,primeiro,executiva,1,"3A",BigDecimal.ZERO)),15), "Classe de assento incompativel rejeitada". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
239      planejamento.tarifa(admin,primeiro,fare,new Tarifa("GRUPO","ECONOMICA",new
BigDecimal("200"),BigDecimal.ZERO,new BigDecimal("23"),1,true,new BigDecimal("20")),true);
```

Chama PlanejamentoServico.tarifa: Bloqueia voo e só permite mudança comercial em PROGRAMADO/CHECKIN\_ABERTO; cria/edita contrato, audita e retorna id.

new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
240      verificar(numero("SELECT valor_base FROM item_reserva WHERE id="+item)==100,"Edicao de tarifa
preserva preco e condicoes vendidos");
```

Verificação deste cenário: numero("SELECT valor\_base FROM item\_reserva WHERE id="+item)==100, "Edicao de tarifa preserva preco e condicoes vendidos". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
241      long livre=novoAviao("TX",2);long semVendas=planejamento.programar(admin,programacao(livre,base.pl
us(Duration.ofHours(15)),base.plus(Duration.ofHours(17))));
```

Declara variável local livre do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local semVendas do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
242      long troca=novoAviao("TY",3);planejamento.trocarAeronave(admin,semVendas,troca,"Teste sem
vendas");
```

Declara variável local troca do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

- 243 `verificar(contar("SELECT COUNT(*) FROM inventario_assento_voo WHERE voo_id="+semVendas)==3,"Troca sem vendas recria inventario compativel");`  
Verificação deste cenário: contar("SELECT COUNT(\*) FROM inventario\_assento\_voo WHERE voo\_id="+semVendas)==3, "Troca sem vendas recria inventario compativel". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 244 `var atual=banco.consultar(c->Sql.registro(c,"SELECT * FROM voo WHERE id=?",semVendas));`  
Declara variável local atual com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.  
  
Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. Parâmetros ligados em ordem: semVendas.
- 245 `Instant novaPartida=Dados.instante(atual,"partida_prevista").plus(Duration.ofHours(1)),novaChegada=Dados.instante(atual,"chegada_prevista").plus(Duration.ofHours(1));`  
Declara variável local novaPartida, novaChegada do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 246 `planejamento.reprogramar(admin,semVendas,new ProgramacaoVoo(companhia,rota,troca,atual.get("numero").toString(),novaPartida,novaChegada,Dados.instante(atual,"checkin_abre").plus(Duration.ofHours(1)),Dados.instante(atual,"checkin_fecha").plus(Duration.ofHours(1)),Dados.instante(atual,"embarque_abre").plus(Duration.ofHours(1)),Dados.instante(atual,"embarque_fecha").plus(Duration.ofHours(1))), "Teste");`  
Chama PlanejamentoServico.reprogramar: Só voo PROGRAMADO; preserva companhia/rota/aeronave/número e muda horários. Valida frota, apaga previsões estimadas, desloca agendas e revalida escalas.  
  
new constrói ProgramacaoVoo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 247 `long alocao=operacao.alocar(admin,semVendas,portao,"PARTIDA",novaPartida.minus(Duration.ofMinutes(45)),novaPartida);`  
Declara variável local alocao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 248 `planejamento.horarios(admin,semVendas,novaPartida.plus(Duration.ofMinutes(20)),novaChegada.plus(Duration.ofMinutes(20)),false,"Atraso FICTICIO");`  
Chama PlanejamentoServico.horarios: Distingue horário real (EM\_VOO e instantes já ocorridos) de estimado (antes de EM\_VOO). Verifica frota, grava horários, atualiza agendas/escalas e audita.
- 249 `verificar(banco.consultar(c->Dados.instante(Sql.registro(c,"SELECT inicio FROM alocao_recurso WHERE id=?",alocacao),"inicio")).equals(novaPartida.minus(Duration.ofMinutes(25))), "Atraso atualiza agenda de recurso");`  
Verificação deste cenário: banco.consultar(c->Dados.instante(Sql.registro(c,"SELECT inicio FROM alocao\_recurso WHERE id=?",alocacao),"inicio")).equals(novaPartida.minus(Duration.ofMinutes(25))), "Atraso atualiza agenda de recurso". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.  
  
Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.  
  
Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: alocao\_recurso. Parâmetros ligados em ordem: alocao.
- 250 `verificar(banco.consultar(c->Dados.instante(Sql.registro(c,"SELECT inicio FROM agenda_aeronave WHERE voo_id=?",semVendas),"inicio")).equals(novaPartida.minus(Duration.ofMinutes(25))), "Atraso atualiza agenda de aeronave");`  
Verificação deste cenário: banco.consultar(c->Dados.instante(Sql.registro(c,"SELECT inicio FROM agenda\_aeronave WHERE voo\_id=?",semVendas),"inicio")).equals(novaPartida.minus(Duration.ofMinutes(25))), "Atraso atualiza agenda de aeronave". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.  
  
Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.  
  
Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: agenda\_aeronave. Parâmetros ligados em ordem: semVendas.
- 251 `verificar(relatorios.resumoAtrasos(admin).get("maximo").compareTo(new BigDecimal("20"))>=0,"Relatorio de atrasos calculado");`  
Verificação deste cenário: relatorios.resumoAtrasos(admin).get("maximo").compareTo(new BigDecimal("20"))>=0, "Relatorio de atrasos calculado". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.  
  
new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 252 `long pista=cad(TipoCadastro.RECURSO,"aeroporto_id",origem,"tipo","PISTA","codigo","CURTA","situacao","DISPONIVEL","comprimento_m","500");`  
Declara variável local pista do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.



```

253         rejeitar(()->operacao.alocar(admin,semVendas,pista,"PARTIDA",novaPartida,novaPartida.plusSeconds(6
0)), "Pista incompatível rejeitada");
Verificação deste cenário: ()->operacao.alocar(admin,semVendas,pista,"PARTIDA",novaPartida,novaPartida.plusSeconds(60)),
"Pista incompatível rejeitada". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste
falhar.

254         for(String tipo:new String[]{"POSICAO","ESTEIRA","BALCAO"})
cad(TipoCadastro.RECURSO,"aeroporto_id",origem,"tipo",tipo,"codigo","FICTICIO","situacao","DISPONIVEL");
Percorre cada elemento da coleção/vetor: for(String tipo:new String[]{"POSICAO","ESTEIRA","BALCAO"}). return, break ou
exceção podem encerrar esse caminho; continue passa à próxima volta.

new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

255         long impedimento=operacao.interditar(admin,pista,base,base.plusSeconds(60),"Interdicao
FICTICIA");operacao.liberarAlocacao(admin,impedimento);
Declara variável local impedimento do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
atual.

256         verificar(contar("SELECT COUNT(*) FROM alocao_recurso WHERE id="+impedimento+" AND NOT
ativa")==1,"Interdicao encerrada preserva registro");
Verificação deste cenário: contar("SELECT COUNT(*) FROM alocao_recurso WHERE id="+impedimento+" AND NOT
ativa")==1, "Interdicao encerrada preserva registro". A operação/condição deve resultar em verdadeiro; divergência faz o
programa de teste falhar.

257         long manutencao=operacao.manutencao(admin,novoAviao("MX",1),admin.funcionario().id(),"INSPECAO","F
ICTICIA",base.minusSeconds(60),base.plusSeconds(60),BigDecimal.ZERO);
Declara variável local manutencao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
atual.

258         operacao.situacaoManutencao(admin,manutencao,"EM_EXECUCAO");operacao.situacaoManutencao(admin,manu
tencao,"CONCLUIDA");
Chama OperacaoServico.situacaoManutencao: AGENDADA vai a EM_EXECUCAO/CANCELADA; EM_EXECUCAO vai a
CONCLUIDA. Ao iniciar exige janela e marca aeronave em manutenção. Ao concluir estende agenda até fim real e encerra sem
esconder conflito.

259         verificar(texto("SELECT situacao FROM manutencao_aeronave WHERE
id="+manutencao).equals("CONCLUIDA"),"Manutencao completa preserva historico");
Verificação deste cenário: texto("SELECT situacao FROM manutencao_aeronave WHERE
id="+manutencao).equals("CONCLUIDA"), "Manutencao completa preserva historico". A operação/condição deve resultar em
verdadeiro; divergência faz o programa de teste falhar.

260         long antes=contar("SELECT COUNT(*) FROM manutencao_aeronave");
Declara variável local antes do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

261         rejeitar(()->operacao.manutencao(admin,troca,admin.funcionario().id(),"PREVENTIVA","FICTICIA",nova
Partida,novaChegada,BigDecimal.ZERO),"Manutencao nao sobrepoe voo");verificar(contar("SELECT COUNT(*) FROM
manutencao_aeronave")==antes,"Manutencao rejeitada desfaz registros");
Verificação deste cenário: ()->operacao.manutencao(admin,troca,admin.funcionario().id(),"PREVENTIVA","FICTICIA",novaPartida
,novaChegada,BigDecimal.ZERO), "Manutencao nao sobrepoe voo". A operação/condição deve terminar lançando a falha
esperada; divergência faz o programa de teste falhar.

Verificação deste cenário: contar("SELECT COUNT(*) FROM manutencao_aeronave")==antes, "Manutencao rejeitada desfaz
registros". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

262         for(String tipo:new String[]{"ABASTECIMENTO","LIMPEZA","CATERING","BAGAGEM","REBOQUE"}) {
Percorre cada elemento da coleção/vetor: for(String tipo:new
String[]{"ABASTECIMENTO","LIMPEZA","CATERING","BAGAGEM","REBOQUE"}). return, break ou exceção podem encerrar
esse caminho; continue passa à próxima volta.

new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

263         long solo=operacao.servicoSolo(admin,semVendas,origem,admin.funcionario().id(),tipo,BigDecimal
.ONE,"un",BigDecimal.ZERO);
Declara variável local solo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

264         operacao.situacaoSolo(admin,solo,"EM_EXECUCAO");operacao.situacaoSolo(admin,solo,"CONCLUIDO");
Chama OperacaoServico.situacaoSolo: Só transições previstas. Grava início se ainda ausente e fim ao concluir/cancelar; não
apaga serviço.

265     }
266     verificar(contar("SELECT COUNT(*) FROM servico_solo WHERE voo_id="+semVendas+" AND
situacao='CONCLUIDO'")==5,"Cinco tipos de servico de solo integrados");
Verificação deste cenário: contar("SELECT COUNT(*) FROM servico_solo WHERE voo_id="+semVendas+" AND
situacao='CONCLUIDO'")==5, "Cinco tipos de servico de solo integrados". A operação/condição deve resultar em verdadeiro;
divergência faz o programa de teste falhar.

```

- 267 `long ocorrencia=operacao.ocorrencia(admin,origem,semVendas,"TECNICA","BAIXA","FICTICIA");operacao. encerrarOcorrencia(admin,ocorrencia);`  
Declara variável local ocorrencia do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 268 `verificar(contar("SELECT COUNT(*) FROM ocorrencia_operacional WHERE id="+ocorrencia+" AND encerrada_em IS NOT NULL")==1,"Ocorrencia aberta e encerrada");`  
Verificação deste cenário: contar("SELECT COUNT(\*) FROM ocorrencia\_operacional WHERE id="+ocorrencia+" AND encerrada\_em IS NOT NULL")==1, "Ocorrencia aberta e encerrada". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 269 `for(TipoRelatorio tipo:TipoRelatorio.values()) { relatorios.consultar(admin,tipo,1);relatorios.exportar(admin,tipo,true); }`  
Percorre cada elemento da coleção/vetor: for(TipoRelatorio tipo:TipoRelatorio.values()). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.
- 270 `verificar(!relatorios.painel(admin,origem,inicio.atZone(ZoneOffset.UTC).toLocalDate(),true).isEmpty y(),"Painel de partidas por aeroporto");`  
Verificação deste cenário: !relatorios.painel(admin,origem,inicio.atZone(ZoneOffset.UTC).toLocalDate(),true).isEmpty(), "Painel de partidas por aeroporto". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 271 `verificar(!relatorios.painel(admin,destino,fim.atZone(ZoneOffset.UTC).toLocalDate(),false).isEmpty (),"Painel de chegadas por aeroporto");`  
Verificação deste cenário: !relatorios.painel(admin,destino,fim.atZone(ZoneOffset.UTC).toLocalDate(),false).isEmpty(), "Painel de chegadas por aeroporto". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 272 `bagagensEmConexao(reserva,primeiro,segundo,inicioVolta,base);`  
Chama IntegracaoTeste.bagagensEmConexao: Cria bagagens independentes por trecho e confirma extravio/retirada/nova etiqueta sem perder o vínculo do voo.
- 273 `var demo=new DadosFicticiosServico(banco,autorizacao,auditoria,religio);var dados=demo.criar(admin,"D"+sufixo.substring(0,7));`  
Declara variável local demo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
Declara variável local dados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
new constrói DadosFicticiosServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 274 `verificar(dados.equals(demo.criar(admin,"D"+sufixo.substring(0,7))), "Demonstracao nao duplica dados");`  
Verificação deste cenário: dados.equals(demo.criar(admin,"D"+sufixo.substring(0,7))), "Demonstracao nao duplica dados". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 275 `long demoVoo=Dados.id(dados,"voo"),demoPax=Dados.id(dados,"passageiro"),demoTarifa=Dados.id(dados,"tarifa");`  
Declara variável local demoVoo, demoPax, demoTarifa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 276 `long demoReserva=reservas.reservar(admin,demoPax,List.of(new PedidoTrecho(demoPax,demoVoo,demoTarifa,1,null,BigDecimal.ZERO)),15);`  
Declara variável local demoReserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 277 `for(int i=0;i<25;i++) financeiro.pagar(admin,demoReserva,"recusa-"+sufixo+"-"+i,"CARTAO",new BigDecimal("120"),"RECUSADO");`  
Laço controlado por inicialização, condição e atualização: for(int i=0;i<25;i++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.  
  
Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 278 `Path csv=relatorios.exportar(admin,TipoRelatorio.PAGAMENTOS,true);`  
Declara variável local csv do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 279 `verificar(Files.readAllLines(csv).size()==contar("SELECT COUNT(*) FROM pagamento")+1,"Exportacao completa ultrapassa primeira pagina");`  
Verificação deste cenário: Files.readAllLines(csv).size()==contar("SELECT COUNT(\*) FROM pagamento")+1, "Exportacao completa ultrapassa primeira pagina". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
280 financeiro.pagar(admin,demoReserva,"demo-"+sufixo,"PIX",new BigDecimal("120"),"APROVADO");
```

Chama FinanceiroServico.pagar: Valida chave/forma/valor/resultado. Uma chave existente precisa ter os mesmos dados; pendente pode ser processado posteriormente; já processado retorna mesmo id somente com resultado igual. Nova chave cria pagamento; aprovação pode confirmar reserva.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
281 long demoItem=numero("SELECT id FROM item_reserva WHERE reserva_id="+demoReserva);
```

Declara variável local demoItem do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
282 voos.transicao(admin,demoVoo,SituacaoVoo.CHECKIN_ABERTO,"Teste");atendimento.checkin(admin,demoItem,Dados.id(dados,"documento"));
```

Chama VooServico.transicao: Expira pendências, reúne reservas/voos relacionados e bloqueia em ordem. Valida transição enum. Cada próximo estado tem efeitos próprios: janela/equipe/portão, ausência na partida, liberação na conclusão ou cancelamento com devolução e ocorrência. Atualiza o voo e audita.

```
283 long mala=atendimento.despachar(admin,demoItem,"CANC-"+sufixo,new BigDecimal("25"),"DESPACHADA");atendimento.cancelarCheckin(admin,demoItem);
```

Declara variável local mala do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
284 verificar(contar("SELECT COUNT(*) FROM reembolso WHERE reserva_id="+demoReserva)==0,"Cancelar excesso nao pago nao devolve dinheiro da passagem");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM reembolso WHERE reserva\_id="+demoReserva)==0, "Cancelar excesso nao pago nao devolve dinheiro da passagem". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
285 atendimento.checkin(admin,demoItem,Dados.id(dados,"documento"));
```

Chama AtendimentoServico.checkin: Exige item confirmado e voo CHECKIN\_ABERTO na janela, pessoa ativa, documento apropriado/válido, bilhete emitido, assento não bloqueado e cobertura. Check-in ativo retorna id existente; cancelado é renovado por upsert com novo código.

```
286 Instant saida=banco.consultar(c->Dados.instante(Sql.registro(c,"SELECT partida_prevista FROM voo WHERE id=?",demoVoo),"partida_prevista"));
```

Declara variável local saida do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. Parâmetros ligados em ordem: demoVoo.

```
287 relógio.definir(saida.minus(Duration.ofMinutes(30)));voos.transicao(admin,demoVoo,SituacaoVoo.EMBARQUE,"Teste");
```

Invoca relógio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
288 relógio.definir(saida);voos.transicao(admin,demoVoo,SituacaoVoo.EM_VOO,"Teste");
```

Invoca relógio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
289 verificar(texto("SELECT situacao FROM item_reserva WHERE id="+demoItem).equals("NAO_COMPARECEU"),"Fechamento identifica passageiro ausente");
```

Verificação deste cenário: texto("SELECT situacao FROM item\_reserva WHERE id="+demoItem).equals("NAO\_COMPARECEU"), "Fechamento identifica passageiro ausente". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
290 cancelamentoAposEmbarque();
```

Chama IntegracaoTeste.cancelamentoAposEmbarque: Embarca passageiro, impede cancelamento voluntário e decolagem em manutenção, depois cancela voo antes da partida com histórico e devolução.

```
291 falhasEDeadlock(primeiro,segundo);
```

Chama IntegracaoTeste.falhasEDeadlock: Injeta SQLException com SQLSTATE 08006 após INSERT para comprovar rollback. Duas threads bloqueiam voos em ordem oposta para provocar deadlock e repetição integral.

```
292 testarTerminal();
```

Chama IntegracaoTeste.testarTerminal: Inicia outro processo Java, envia login/opções por stdin e confere saída de menus/mapa.

```
293 }
```

```
294
```

## bagagensEmConexao - L295 a L307

Cria bagagens independentes por trecho e confirma extravio/retirada/nova etiqueta sem perder o vínculo do voo.

Entradas declaradas: (long reserva,long primeiro,long segundo,Instant volta,Instant base).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203).

Chamadas observadas no corpo: cad, numero, voos.transicao, atendimento.checkin, atendimento.despachar, new BigDecimal, atendimento.rastrear, verificar, texto("SELECT situacao FROM bagagem WHERE id="+primeiraMala).equals, texto, relatorios.bagagem(admin,primeiraTag).size, relatorios.bagagem, relógio.definir, volta.minus, Duration.ofHours.

```

295     private static void bagagensEmConexao(long reserva,long primeiro,long segundo,Instant volta,Instant
base) {
    Início da declaração de bagagensEmConexao. Os parâmetros entram pela chamada; o corpo só executa quando esse
    método/construtor é invocado.

296         long doc=cad(TipoCadastro.DOCUMENTO,"pessoa_id",outroPassageiro,"pais_emissor_id",pais,"tipo","PAS
SAPORTE","numero","CONEXAO-"+sufixo,"validade","2099-01-01");
        Declara variável local doc do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

297         long itemId=numero("SELECT id FROM item_reserva WHERE reserva_id="+reserva+" AND
passageiro_id="+outroPassageiro+" AND voo_id="+primeiro);
        Declara variável local itemId do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

298         long segundoItem=numero("SELECT id FROM item_reserva WHERE reserva_id="+reserva+" AND
passageiro_id="+outroPassageiro+" AND voo_id="+segundo);
        Declara variável local segundoItem do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

299         voos.transicao(admin,primeiro,SituacaoVoo.CHECKIN_ABERTO,"Conexao
FICTICIA");atendimento.checkin(admin,itemId,doc);
        Chama VooServico.transicao: Expira pendências, reúne reservas/voos relacionados e bloqueia em ordem. Valida transição enum.
        Cada próximo estado tem efeitos próprios: janela/equipe/portão, ausência na partida, liberação na conclusão ou cancelamento
        com devolução e ocorrência. Atualiza o voo e audita.

300         String primeiraTag="CON1-"+sufixo;long
primeiraMala=atendimento.despachar(admin,itemId,primeiraTag,new BigDecimal("22"),"DESPACHADA");
        Declara variável local primeiraTag do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

        Declara variável local primeiraMala do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

        new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si
        só.

301         atendimento.rastrear(admin,primeiraTag,origem,"EXTRAVIADA","Extravio
FICTICIO");atendimento.rastrear(admin,primeiraTag,origem,"RETIRADA","Recuperada FICTICIA");
        Chama AtendimentoServico.rastrear: Confere transição no mapa BAGAGEM, local do evento, chegada para descarregar/entregar
        e cobertura para carregar. RETIRADA solicita devolução da taxa; atualiza e registra histórico.

302         verificar(texto("SELECT situacao FROM bagagem WHERE id="+primeiraMala).equals("RETIRADA") &&
relatorios.bagagem(admin,primeiraTag).size()==3,"Extravio e retirada preservam rastreo");
        Verificação deste cenário: texto("SELECT situacao FROM bagagem WHERE id="+primeiraMala).equals("RETIRADA") &&
relatorios.bagagem(admin,primeiraTag).size()==3, "Extravio e retirada preservam rastreo". A operação/condição deve resultar em
        verdadeiro; divergência faz o programa de teste falhar.

303         relógio.definir(volta.minus(Duration.ofHours(2)));voos.transicao(admin,segundo,SituacaoVoo.CHECKIN
_ABERTO,"Conexao FICTICIA");atendimento.checkin(admin,segundoItem,doc);
        Invoca relógio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
        método para localizar o corpo no projeto.

304         long segundaMala=atendimento.despachar(admin,segundoItem,"CON2-"+sufixo,new
BigDecimal("22"),"DESPACHADA");
        Declara variável local segundaMala do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

        new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si
        só.

305         verificar(numero("SELECT voo_id FROM bagagem b JOIN check_in ci ON ci.id=b.checkin_id WHERE
b.id="+segundaMala)==segundo && segundaMala!=primeiraMala,"Despacho em conexao recebe nova etiqueta e
vinculo ao trecho");
        Verificação deste cenário: numero("SELECT voo_id FROM bagagem b JOIN check_in ci ON ci.id=b.checkin_id WHERE
b.id="+segundaMala)==segundo && segundaMala!=primeiraMala, "Despacho em conexao recebe nova etiqueta e vinculo ao
        trecho". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

306         relógio.definir(base);
        Invoca relógio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do
        método para localizar o corpo no projeto.

```

```

307     }
308

```

## cancelamentoAposEmbarque - L309 a L332

Embarca passageiro, impede cancelamento voluntário e decolagem em manutenção, depois cancela voo antes da partida com histórico e devolução.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203).

Chamadas observadas no corpo: new DadosFicticiosServico(banco,autorizacao,auditoria,religio).criar, new DadosFicticiosServico, sufixo.substring, Dados.id, reservas.reservar, List.of, new PedidoTrecho, financeiro.pagar, new BigDecimal, numero, voos.transicao, atendimento.checkin, banco.consultar, Dados.instante, Sql.registro, operacao.manutencao, admin.funcionario().id, admin.funcionario, relgio.instant().minusSeconds, relgio.instant, relgio.instant().plusSeconds, operacao.situacaoManutencao, relgio.definir, saida.minus, Duration.ofMinutes, atendimento.embarcar, atendimento.cartao(admin,item).get("codigo\_cartao").toString, atendimento.cartao(admin,item).get, atendimento.cartao, rejeitar, reservas.cancelar, verificar, texto("SELECT situacao FROM item\_reserva WHERE id="+item).equals, texto, texto("SELECT situacao FROM bilhete WHERE item\_id="+item).equals, contar, texto("SELECT situacao FROM aeronave WHERE id="+aviao).equals.

```

309     private static void cancelamentoAposEmbarque() {

```

Início da declaração de cancelamentoAposEmbarque. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```

310         var dados=new
        DadosFicticiosServico(banco,autorizacao,auditoria,religio).criar(admin,"C"+sufixo.substring(0,7));

```

Declara variável local dados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói DadosFicticiosServico: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

311         long flight=Dados.id(dados,"voo"),pax=Dados.id(dados,"passageiro");

```

Declara variável local flight, pax do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```

312         long reserva=reservas.reservar(admin,pax,List.of(new
        PedidoTrecho(pax,flight,Dados.id(dados,"tarifa"),1,null,BigDecimal.ZERO)),15);

```

Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

313         financeiro.pagar(admin,reserva,"emb-cancel-"+sufixo,"PIX",new BigDecimal("120"),"APROVADO");

```

Chama FinanceiroServico.pagar: Valida chave/forma/valor/resultado. Uma chave existente precisa ter os mesmos dados; pendente pode ser processado posteriormente; já processado retorna mesmo id somente com resultado igual. Nova chave cria pagamento; aprovação pode confirmar reserva.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

314         long item=numero("SELECT id FROM item_reserva WHERE reserva_id="+reserva);

```

Instrução idêntica à de IntegracaoTeste:L111. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

315         voos.transicao(admin,flight,SituacaoVoo.CHECKIN_ABERTO,"Teste");atendimento.checkin(admin,item,Dad
        os.id(dados,"documento"));

```

Chama VooServico.transicao: Expira pendências, reúne reservas/voos relacionados e bloqueia em ordem. Valida transição enum. Cada próximo estado tem efeitos próprios: janela/equipe/portão, ausência na partida, liberação na conclusão ou cancelamento com devolução e ocorrência. Atualiza o voo e audita.

```

316         Instant saida=banco.consultar(c->Dados.instante(Sql.registro(c,"SELECT partida_prevista FROM voo
        WHERE id=?",flight),"partida_prevista"));

```

Declara variável local saida do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. Parâmetros ligados em ordem: flight.

```

317         long aviao=numero("SELECT aeronave_id FROM voo WHERE id="+flight);

```

Declara variável local aviao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.



- 318 `long manutencao=operacao.manutencao(admin,aviao,admin.funcionario().id(),"CORRETIVA","Atraso FICTICIO",religio.instant().minusSeconds(60),religio.instant().plusSeconds(60),BigDecimal.ZERO);`  
Declara variável local manutencao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 319 `operacao.situacaoManutencao(admin,manutencao,"EM_EXECUCAO");`  
Chama OperacaoServico.situacaoManutencao: AGENDADA vai a EM\_EXECUCAO/CANCELADA; EM\_EXECUCAO vai a CONCLUIDA. Ao iniciar exige janela e marca aeronave em manutenção. Ao concluir estende agenda até fim real e encerra sem esconder conflito.
- 320 `religio.definir(saida.minus(Duration.ofMinutes(30)));voos.transicao(admin,flight,SituacaoVoo.EMBARQUE,"Teste");`  
Invoca religio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
- 321 `atendimento.embarcar(admin,atendimento.cartao(admin,item).get("codigo_cartao").toString(),Dados.id(dados,"alocacao"));`  
Chama AtendimentoServico.embarcar: Procura item pelo código do cartão, bloqueia e exige janela, situação EMBARQUE, bilhete e ocupação ativos, portão correto/disponível, alocação vigente, bagagens aptas e saldo coberto. Grava embarque e marca item/bilhete UTILIZADO.
- 322 `rejeitar()->reservas.cancelar(admin,reserva,null,"Teste"),"Passageiro nao cancela item ja embarcado");`  
Verificação deste cenário: `()->reservas.cancelar(admin,reserva,null,"Teste"), "Passageiro nao cancela item ja embarcado".` A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.
- 323 `religio.definir(saida.minus(Duration.ofMinutes(10)));`  
Invoca religio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
- 324 `rejeitar()->voos.transicao(admin,flight,SituacaoVoo.EM_VOO,"Teste"),"Manutencao em execucao impede decolagem");`  
Verificação deste cenário: `()->voos.transicao(admin,flight,SituacaoVoo.EM_VOO,"Teste"), "Manutencao em execucao impede decolagem".` A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.
- 325 `rejeitar()->operacao.situacaoManutencao(admin,manutencao,"CONCLUIDA"),"Manutencao atrasada nao oculta conflito com agenda de voo");`  
Verificação deste cenário: `()->operacao.situacaoManutencao(admin,manutencao,"CONCLUIDA"), "Manutencao atrasada nao oculta conflito com agenda de voo".` A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.
- 326 `voos.transicao(admin,flight,SituacaoVoo.CANCELADO,"Cancelamento apos embarque FICTICIO");`  
Chama VooServico.transicao: Expira pendências, reúne reservas/voos relacionados e bloqueia em ordem. Valida transição enum. Cada próximo estado tem efeitos próprios: janela/equipe/portão, ausência na partida, liberação na conclusão ou cancelamento com devolução e ocorrência. Atualiza o voo e audita.
- 327 `verificar(texto("SELECT situacao FROM item_reserva WHERE id="+item).equals("CANCELADO") && texto("SELECT situacao FROM bilhete WHERE item_id="+item).equals("CANCELADO"),"Cancelamento antes da partida invalida passagem ja embarcada");`  
Verificação deste cenário: `texto("SELECT situacao FROM item_reserva WHERE id="+item).equals("CANCELADO") && texto("SELECT situacao FROM bilhete WHERE item_id="+item).equals("CANCELADO"), "Cancelamento antes da partida invalida passagem ja embarcada".` A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 328 `verificar(contar("SELECT COUNT(*) FROM embarque e JOIN check_in ci ON ci.id=e.checkin_id WHERE ci.item_id="+item)==1,"Cancelamento preserva registro de embarque");`  
Verificação deste cenário: `contar("SELECT COUNT(*) FROM embarque e JOIN check_in ci ON ci.id=e.checkin_id WHERE ci.item_id="+item)==1, "Cancelamento preserva registro de embarque".` A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 329 `verificar(numero("SELECT SUM(valor) FROM reembolso WHERE reserva_id="+reserva)==120 && contar("SELECT COUNT(*) FROM ocorrencia_operacional WHERE voo_id="+flight)==1,"Cancelamento gera devolucao integral e ocorrencia");`  
Verificação deste cenário: `numero("SELECT SUM(valor) FROM reembolso WHERE reserva_id="+reserva)==120 && contar("SELECT COUNT(*) FROM ocorrencia_operacional WHERE voo_id="+flight)==1, "Cancelamento gera devolucao integral e ocorrencia".` A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 330 `operacao.situacaoManutencao(admin,manutencao,"CONCLUIDA");`  
Chama OperacaoServico.situacaoManutencao: AGENDADA vai a EM\_EXECUCAO/CANCELADA; EM\_EXECUCAO vai a CONCLUIDA. Ao iniciar exige janela e marca aeronave em manutenção. Ao concluir estende agenda até fim real e encerra sem esconder conflito.
- 331 `verificar(texto("SELECT situacao FROM aeronave WHERE id="+aviao).equals("ATIVA"),"Manutencao conclui apos resolucão do conflito");`  
Verificação deste cenário: `texto("SELECT situacao FROM aeronave WHERE id="+aviao).equals("ATIVA"), "Manutencao conclui apos resolucão do conflito".` A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
332     }
333
```

## falhasEDeadlock - L334 a L353

Injeta SQLException com SQLSTATE 08006 após INSERT para comprovar rollback. Duas threads bloqueiam voos em ordem oposta para provocar deadlock e repetição integral.

Entradas declaradas: (long primeiro,long segundo).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203).

Chamadas observadas no corpo: contar, rejeitar, banco.transacao, Sql.executar, new java.sql.SQLException, verificar, new CountDownLatch, new java.util.concurrent.atomic.AtomicInteger, Executors.newFixedThreadPool, new ArrayList<Future<Boolean>>, tarefas.add, executor.submit, banco.transacaoRepetivel, rodada.incrementAndGet, tentativas.incrementAndGet, Sql.registro, barreira.countDown, barreira.await, new AssertionError, Thread.currentThread().interrupt, Thread.currentThread, new RuntimeException, tarefa.get, tentativas.get.

```
334     private static void falhasEDeadlock(long primeiro,long segundo) throws Exception {
        Início da declaração de falhasEDeadlock. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.
```

```
335         long antes=contar("SELECT COUNT(*) FROM evento_auditoria");
        Declara variável local antes do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
```

```
336         rejeitar()->banco.transacao(c->{Sql.executar(c,"INSERT INTO evento_auditoria(acao,entidade)
VALUES ('TESTE_ROLLBACK','teste')");throw new java.sql.SQLException("Falha injetada de
conexao","08006");});,"Falha JDBC e traduzida");
        Verificação deste cenário: ()->banco.transacao(c->{Sql.executar(c,"INSERT INTO evento_auditoria(acao,entidade) VALUES
('TESTE_ROLLBACK','teste')");throw new java.sql.SQLException("Falha injetada de conexao","08006");});,"Falha JDBC e
traduzida". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.
```

Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.

Executa comando de alteração e devolve int com a quantidade de linhas afetadas. Tabelas mencionadas: evento\_auditoria.

Lança new java.sql.SQLException("Falha injetada de conexao","08006"). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói java.sql.SQLException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
337         verificar(contar("SELECT COUNT(*) FROM evento_auditoria")==antes,"Falha JDBC desfaz alteracoes
anteriores");
        Verificação deste cenário: contar("SELECT COUNT(*) FROM evento_auditoria")==antes, "Falha JDBC desfaz alteracoes
anteriores". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
```

```
338         var barreira=new CountDownLatch(2);var tentativas=new java.util.concurrent.atomic.AtomicInteger();
        Declara variável local barreira com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        Declara variável local tentativas com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        new constrói CountDownLatch, java.util.concurrent.atomic.AtomicInteger: aloca a instância e executa seu construtor com os
        argumentos mostrados; não insere no banco por si só.
```

```
339         try(var executor=Executors.newFixedThreadPool(2)) {
        Instrução idêntica à de IntegracaoTeste:L170. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.
```

```
340         var tarefas=new ArrayList<Future<Boolean>>();
        Declara variável local tarefas com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        new constrói ArrayList: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
```

```
341         for(boolean ordem:new boolean[]{true,false}) {
        Percorre cada elemento da coleção/vetor: for(boolean ordem:new boolean[]{true,false}). return, break ou exceção podem encerrar
        esse caminho; continue passa à próxima volta.

        new constrói boolean: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
```

```
342         var rodada=new java.util.concurrent.atomic.AtomicInteger();
        Declara variável local rodada com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.
```



new constrói java.util.concurrent.atomic.AtomicInteger: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
343         tarefas.add(executor.submit(()->banco.transacaoRepetivel(c->{
```

Executa uma transação completa com até três tentativas totais somente para deadlock/serialização (40P01/40001). O callback pode executar novamente.

```
344         int atual=rodada.incrementAndGet();tentativas.incrementAndGet();
```

Declara variável local atual do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
345         Sql.registro(c,"SELECT id FROM voo WHERE id=? FOR UPDATE",ordem?primeiro:segundo);
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: ordem?primeiro:segundo.

Escolha de valor: ordem. true escolhe primeiro; false escolhe segundo. Só a alternativa escolhida é avaliada.

```
346         if(atual==1) { barreira.countDown();try{if(!barreira.await(10,TimeUnit.SECONDS)) throw
new AssertionError("Barreira de deadlock");}catch(InterruptedException
e){Thread.currentThread().interrupt();throw new RuntimeException(e);} }
```

Decisão específica: (atual==1). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Decisão específica: (!barreira.await(10,TimeUnit.SECONDS)). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new AssertionError("Barreira de deadlock"). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

Trata a(s) exceção(ões) declarada(s) em catch(InterruptedException e). O tratamento mostrado abaixo decide se avisa, repete ou relança.

Lança new RuntimeException(e). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói AssertionError, RuntimeException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
347         Sql.registro(c,"SELECT id FROM voo WHERE id=? FOR
UPDATE",ordem?segundo:primeiro);return true;
```

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: voo. O FOR UPDATE mantém lock das linhas até commit/rollback. Parâmetros ligados em ordem: ordem?segundo:primeiro.

Escolha de valor: ordem. true escolhe segundo; false escolhe primeiro. Só a alternativa escolhida é avaliada.

Retorno: entrega true ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
348         }));
```

Fecha o bloco anterior e completa/continua a expressão ou abre o próximo bloco indicado nesta linha.

```
349         }
350         for(var tarefa:tarefas) verificar(tarefa.get(40,TimeUnit.SECONDS),"Transacao recuperada de
deadlock real");
```

Percorre cada elemento da coleção/vetor: for(var tarefa:tarefas). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Verificação deste cenário: tarefa.get(40,TimeUnit.SECONDS), "Transacao recuperada de deadlock real". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
351         }
352         verificar(tentativas.get()>=3,"Deadlock provocou repeticao da transacao completa");
```

Verificação deste cenário: tentativas.get()>=3, "Deadlock provocou repeticao da transacao completa". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
353     }
354 }
```

## testarTerminal - L355 a L362

Inicia outro processo Java, envia login/opções por stdin e confere saída de menus/mapa.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.cenariosAdicionais (L203).

Chamadas observadas no corpo: new ProcessBuilder, Path.of(System.getProperty("java.home"),"bin","java.exe").toString, Path.of, System.getProperty, processo.environment().put, processo.environment, processo.redirectErrorStream, processo.start, filho.outputWriter, entrada.write, new String, filho.getInputStream().readAllBytes, filho.getInputStream, verificar, filho.waitFor, filho.exitValue, saida.contains.

```

355     private static void testarTerminal() throws Exception {
        Início da declaração de testarTerminal. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

356         var processo=new ProcessBuilder(Path.of(System.getProperty("java.home"), "bin", "java.exe").toString
        (),"-Dfile.encoding=UTF-8","-cp","target/classes;lib/*","br.edu.aeroporto.Main");
        Declara variável local processo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        new constrói ProcessBuilder: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

357         processo.environment().put("AEROPORTO_DB_URL","jdbc:postgresql://127.0.0.1:55439/sistema_aeroporto
        ");processo.environment().put("AEROPORTO_DB_USUARIO","aeroporto_teste");processo.environment().put("AEROPORTO_DB_SENHA","FICTICIO-cluster-trust");
        Chama processo.environment().put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.

358         processo.redirectErrorStream(true);var filho=processo.start();
        Invoca processo.redirectErrorStream com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

        Declara variável local filho com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

359         try(var entrada=filho.outputWriter()) {
        entrada.write("teste_admin\nSenhaFicticia!2026\n16\n4\n"+voo+"\n17\n2\n0\n"); }
        Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

360         String saida= new
        String(filho.getInputStream().readAllBytes(),java.nio.charset.StandardCharsets.UTF_8);
        Declara variável local saida do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

361         verificar(filho.waitFor(30,TimeUnit.SECONDS) && filho.exitValue()==0 &&
        saida.contains("Importacao") && saida.contains("1A:O"),"Login, menus e matriz executados por entrada real do terminal");
        Verificação deste cenário: filho.waitFor(30,TimeUnit.SECONDS) && filho.exitValue()==0 && saida.contains("Importacao") && saida.contains("1A:O"), "Login, menus e matriz executados por entrada real do terminal". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

362     }
363

```

## novoAviao - L364 a L367

Cria aeronave e assentos físicos fictícios; devolve id.

Entradas declaradas: (String prefixo,int capacidade).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.regrasComplementares (L369).

Chamadas observadas no corpo: cad.

```

364     private static long novoAviao(String prefixo,int capacidade) {
        Início da declaração de novoAviao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

365         long id=cad(TipoCadastro.AERONAVE,"modelo_id",modelo,"companhia_id",companhia,"matricula",prefixo+
        "-"+sufixo,"situacao","ATIVA","fabricacao_ano",2025);
        Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

366         for(int i=0;i<capacidade;i++) { int fila=i/2+1;String coluna=i%2==0?"A":"B";cad(TipoCadastro.ASSENTO,"aeronave_id",id,"codigo",fila+coluna,"fila",fila,"coluna",coluna,"classe","ECONOMICA","saida_emergencia",false); }return id;
        Laço controlado por inicialização, condição e atualização: for(int i=0;i<capacidade;i++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

        Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        Declara variável local fila do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        Declara variável local coluna do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

        Escolha de valor: i%2==0. true escolhe "A"; false escolhe "B". Só a alternativa escolhida é avaliada.

        Retorno: entrega id ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```

```
367     }
368 }
```

## regrasComplementares - L369 a L416

Direciona limites de descanso/local/jornada, continuidade/margem, pagamento pendente, peça/peso, equipe ausente, inativação e rascunho.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: relógio.instant, cad, LocalDate.now().minusDays, LocalDate.now, base.plus, Duration.ofHours, inicio.plus, rejeitar, operacao.escalar, fim.plus, verificar, contar, novoAviao, partida.plus, planejamento.programar, programacao, chegada.plus, numero, Duration.ofMinutes, new ProgramacaoVoo, curto.numero, curto.partida, curto.chegada, curto.checkinAbre, curto.checkinFecha, curto.embarqueAbre, curto.embarqueFecha, retorno.numero, retorno.partida, retorno.chegada, retorno.checkinAbre, retorno.checkinFecha, retorno.embarqueAbre, retorno.embarqueFecha, base.plusSeconds, planejamento.tarifa, new Tarifa, new BigDecimal, reservas.reservar, List.of, new PedidoTrecho, financeiro.pagar, financeiro.cancelarPagamento, texto("SELECT situacao FROM pagamento WHERE id="+cancelar).equals, texto, relógio.definir, partida.minus, voos.transicao, atendimento.checkin, atendimento.despachar, new LinkedHashMap<>, banco.consultar, Sql.registro, valores.put, cadastros.salvar, relógio.instant().plus, proxima.plus, testarRascunho.

```
369     private static void regrasComplementares() throws Exception {
        Início da declaração de regrasComplementares. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.

370         Instant base=relógio.instant();
        Instrução idêntica à de IntegracaoTeste:L204. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

371         long cargo=cad(TipoCadastro.CARGO,"nome","Equipe SOLO FICTICIA "+sufixo,"area","OPERACAO");
        Declara variável local cargo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

372         long pessoa=cad(TipoCadastro.PESSOA,"nome","Equipe SOLO FICTICIA
        "+sufixo,"nascimento","1985-01-01");
        Declara variável local pessoa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

373         cad(TipoCadastro.FUNCIONARIO,"pessoa_id",pessoa,"cargo_id",cargo,"aeroporto_base_id",origem,"matri
        cula","SOLO-"+sufixo,"admissao",LocalDate.now().minusDays(1));
        Chama IntegracaoTeste.cad: Monta entrada do cadastro e chama serviço, devolvendo id.

374         Instant inicio=base.plus(Duration.ofHours(2)),fim=inicio.plus(Duration.ofHours(2));
        Declara variável local inicio, fim do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco
        atual.

375         rejeitar(()->operacao.escalar(admin,pessoa,destino,null,"SOLO",inicio,fim),"Primeira jornada
        inicia na base do funcionario");
        Verificação deste cenário: ()->operacao.escalar(admin,pessoa,destino,null,"SOLO",inicio,fim), "Primeira jornada inicia na base do
        funcionario". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

376         operacao.escalar(admin,pessoa,origem,null,"SOLO",inicio,fim);
        Chama OperacaoServico.escalar: Aceita funções do catálogo, monta Map com dados da jornada e valida antes de INSERT. Voo
        pode ser null para funções de solo.

377         rejeitar(()->operacao.escalar(admin,pessoa,origem,null,"SOLO",fim.plus(Duration.ofHours(11)),fim.p
        lus(Duration.ofHours(13))), "Descanso inferior a 12h rejeitado");
        Verificação deste cenário:
        ()->operacao.escalar(admin,pessoa,origem,null,"SOLO",fim.plus(Duration.ofHours(11)),fim.plus(Duration.ofHours(13))),
        "Descanso inferior a 12 h rejeitado". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa
        de teste falhar.

378         rejeitar(()->operacao.escalar(admin,pessoa,destino,null,"SOLO",fim.plus(Duration.ofHours(13)),fim.
        plus(Duration.ofHours(15))), "Jornada seguinte exige deslocamento continuo");
        Verificação deste cenário:
        ()->operacao.escalar(admin,pessoa,destino,null,"SOLO",fim.plus(Duration.ofHours(13)),fim.plus(Duration.ofHours(15))), "Jornada
        seguinte exige deslocamento continuo". A operação/condição deve terminar lançando a falha esperada; divergência faz o
        programa de teste falhar.

379         operacao.escalar(admin,pessoa,origem,null,"SOLO",fim.plus(Duration.ofHours(13)),fim.plus(Duration.
        ofHours(15)));
        Chama OperacaoServico.escalar: Aceita funções do catálogo, monta Map com dados da jornada e valida antes de INSERT. Voo
        pode ser null para funções de solo.
```

```

380      verificar(contar("SELECT COUNT(*) FROM escala_funcionario WHERE
funcionario_id="+pessoa)==2,"Jornadas com descanso e local corretos persistidas");
Verificação deste cenário: contar("SELECT COUNT(*) FROM escala_funcionario WHERE funcionario_id="+pessoa)==2,
"Jornadas com descanso e local corretos persistidas". A operação/condição deve resultar em verdadeiro; divergência faz o
programa de teste falhar.

381      rejeitar(()->operacao.escalar(admin,pessoa,origem,null,"SOLO",fim.plus(Duration.ofHours(30)),fim.p
lus(Duration.ofHours(45))), "Jornada superior a 14h rejeitada");
Verificação deste cenário:
()->operacao.escalar(admin,pessoa,origem,null,"SOLO",fim.plus(Duration.ofHours(30)),fim.plus(Duration.ofHours(45))), "Jornada
superior a 14 h rejeitada". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste
falhar.

382      long aviao=novoAviao("RV",2);Instant
partida=base.plus(Duration.ofHours(5)), chegada=partida.plus(Duration.ofHours(2));
Declara variável local aviao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
Declara variável local partida, chegada do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o
bloco atual.

383      long primeiro=planejamento.programar(admin,programacao(aviao,partida,chegada));
Declara variável local primeiro do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

384      rejeitar(()->planejamento.programar(admin,programacao(aviao,chegada.plus(Duration.ofHours(3)),cheg
ada.plus(Duration.ofHours(5))), "Aeronave nao reaparece na origem sem deslocamento");
Verificação deste cenário:
()->planejamento.programar(admin,programacao(aviao,chegada.plus(Duration.ofHours(3)),chegada.plus(Duration.ofHours(5))),
"Aeronave nao reaparece na origem sem deslocamento". A operação/condição deve terminar lançando a falha esperada;
divergência faz o programa de teste falhar.

385      long reversa=numero("SELECT id FROM rota WHERE origem_id="+destino+" AND destino_id="+origem+"
LIMIT 1");
Declara variável local reversa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

386      var
curto=programacao(aviao,chegada.plus(Duration.ofMinutes(60)),chegada.plus(Duration.ofHours(3)));
Declara variável local curto com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
escopo é o bloco atual.

387      rejeitar(()->planejamento.programar(admin,new ProgramacaoVoo(companhia,reversa,aviao,curto.numero(
),curto.partida(),curto.chegada(),curto.checkinAbre(),curto.checkinFecha(),curto.embarqueAbre(),curto.emba
rqueFecha()), "Intervalo da aeronave inferior a 75 minutos rejeitado");
Verificação deste cenário: ()->planejamento.programar(admin,new ProgramacaoVoo(companhia,reversa,aviao,curto.numero(),cur
to.partida(),curto.chegada(),curto.checkinAbre(),curto.checkinFecha(),curto.embarqueAbre(),curto.embarqueFecha()), "Intervalo
da aeronave inferior a 75 minutos rejeitado". A operação/condição deve terminar lançando a falha esperada; divergência faz o
programa de teste falhar.
new constrói ProgramacaoVoo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco
por si só.

388      var
retorno=programacao(aviao,chegada.plus(Duration.ofHours(3)),chegada.plus(Duration.ofHours(5)));
Declara variável local retorno com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
escopo é o bloco atual.

389      long segundo=planejamento.programar(admin,new ProgramacaoVoo(companhia,reversa,aviao,retorno.numer
o(),retorno.partida(),retorno.chegada(),retorno.checkinAbre(),retorno.checkinFecha(),retorno.embarqueAbre(
),retorno.embarqueFecha()));
Declara variável local segundo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
new constrói ProgramacaoVoo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco
por si só.

390      verificar(contar("SELECT COUNT(*) FROM voo WHERE aeronave_id="+aviao)==2,"Aeronave conserva
continuidade em ida e volta");
Verificação deste cenário: contar("SELECT COUNT(*) FROM voo WHERE aeronave_id="+aviao)==2, "Aeronave conserva
continuidade em ida e volta". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

391      rejeitar(()->planejamento.programar(admin,programacao(aviao,base.plusSeconds(60),base.plus(Duratio
n.ofHours(2))), "Insercao antes de voo futuro respeita proximo aeroporto");
Verificação deste cenário:
()->planejamento.programar(admin,programacao(aviao,base.plusSeconds(60),base.plus(Duration.ofHours(2))), "Insercao antes
de voo futuro respeita proximo aeroporto". A operação/condição deve terminar lançando a falha esperada; divergência faz o
programa de teste falhar.

```

- 392 `long tarifa=planejamento.tarifa(admin,primeiro,null,new Tarifa("COMPLEMENTAR","ECONOMICA",new BigDecimal("100"),BigDecimal.ZERO,new BigDecimal("23"),1,true,BigDecimal.ZERO),true);`  
 Declara variável local tarifa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
 new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 393 `long reserva=reservas.reservar(admin,passageiro,List.of(new PedidoTrecho(passageiro,primeiro,tarifa,1,null,BigDecimal.ZERO)),15);`  
 Declara variável local reserva do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
 new constrói PedidoTrecho: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 394 `long pendente=financeiro.pagar(admin,reserva,"pend-"+sufixo,"PIX",new BigDecimal("100"),"PENDENTE");`  
 Declara variável local pendente do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
 new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 395 `verificar(contar("SELECT COUNT(*) FROM bilhete b JOIN item_reserva i ON i.id=b.item_id WHERE i.reserva_id="+reserva)==0,"Pagamento pendente nao confirma nem emite bilhete");`  
 Verificação deste cenário: `contar("SELECT COUNT(*) FROM bilhete b JOIN item_reserva i ON i.id=b.item_id WHERE i.reserva_id="+reserva)==0, "Pagamento pendente nao confirma nem emite bilhete"`. A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 396 `long cancelar=financeiro.pagar(admin,reserva,"pend-cancel-"+sufixo,"PIX",BigDecimal.ONE,"PENDENTE");financeiro.cancelarPagamento(admin,cancelar);`  
 Declara variável local cancelar do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 397 `verificar(texto("SELECT situacao FROM pagamento WHERE id="+cancelar).equals("CANCELADO"),"Pagamento pendente cancelado preserva registro");`  
 Verificação deste cenário: `texto("SELECT situacao FROM pagamento WHERE id="+cancelar).equals("CANCELADO"), "Pagamento pendente cancelado preserva registro"`. A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 398 `verificar(financeiro.pagar(admin,reserva,"pend-"+sufixo,"PIX",new BigDecimal("100"),"APROVADO")==pendente,"Aprovacao reutiliza pagamento pendente idempotente");`  
 Verificação deste cenário: `financeiro.pagar(admin,reserva,"pend-"+sufixo,"PIX",new BigDecimal("100"),"APROVADO")==pendente, "Aprovacao reutiliza pagamento pendente idempotente"`. A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.  
 new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 399 `rejeitar(()->planejamento.tarifa(admin,primeiro,tarifa,new Tarifa("COMPLEMENTAR","EXECUTIVA",new BigDecimal("100"),BigDecimal.ZERO,new BigDecimal("23"),1,true,BigDecimal.ZERO),true),"Classe de tarifa vendida preservada");`  
 Verificação deste cenário: `()->planejamento.tarifa(admin,primeiro,tarifa,new Tarifa("COMPLEMENTAR","EXECUTIVA",new BigDecimal("100"),BigDecimal.ZERO,new BigDecimal("23"),1,true,BigDecimal.ZERO),true), "Classe de tarifa vendida preservada"`. A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.  
 new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 400 `religio.definir(partida.minus(Duration.ofHours(3)));voos.transicao(admin,primeiro,SituacaoVoo.CHEC KIN_ABERTO,"Teste");`  
 Invoca religio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
- 401 `long item=numero("SELECT id FROM item_reserva WHERE reserva_id="+reserva);atendimento.checkin(admin,item,documento);`  
 Declara variável local item do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 402 `atendimento.despachar(admin,item,"PEC1-"+sufixo,new BigDecimal("22"),"DESPACHADA");`  
 Chama AtendimentoServico.despachar: Valida peso/categoria e etiqueta; requer janela e check-in aberto. Repetir etiqueta exige mesmos dados. Calcula somente incremento de excesso, cria bagagem/evento e retorna id.  
 new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 403 `long segunda=atendimento.despachar(admin,item,"PEC2-"+sufixo,BigDecimal.ONE,"DESPACHADA");`  
 Declara variável local segunda do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

- 404 `verificar(numero("SELECT taxa_excesso FROM bagagem WHERE id="+segunda)==100,"Peca adicional cobra taxa sem duplicar franquia de peso");`  
Verificação deste cenário: numero("SELECT taxa\_excesso FROM bagagem WHERE id="+segunda)==100, "Peca adicional cobra taxa sem duplicar franquia de peso". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 405 `rejeitar(()->atendimento.despachar(admin,item,"PESO-"+sufixo,new BigDecimal("33"),"DESPACHADA"),"Limite de peso por peca despachada respeitado");`  
Verificação deste cenário: ()->atendimento.despachar(admin,item,"PESO-"+sufixo,new BigDecimal("33"),"DESPACHADA"), "Limite de peso por peca despachada respeitado". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.  
  
new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 406 `religio.definir(partida.minus(Duration.ofMinutes(30)));`  
Invoca religio.definir com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.
- 407 `rejeitar(()->voos.transicao(admin,primeiro,SituacaoVoo.EMBARQUE,"Teste"),"Tripulacao minima ausente impede abertura de embarque");`  
Verificação deste cenário: ()->voos.transicao(admin,primeiro,SituacaoVoo.EMBARQUE,"Teste"), "Tripulacao minima ausente impede abertura de embarque". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.
- 408 `voos.transicao(admin,primeiro,SituacaoVoo.CANCELADO,"Teste");`  
Chama VooServico.transicao: Expira pendências, reúne reservas/voos relacionados e bloqueia em ordem. Valida transição enum. Cada próximo estado tem efeitos próprios: janela/equipe/portão, ausência na partida, liberação na conclusão ou cancelamento com devolução e ocorrência. Atualiza o voo e audita.
- 409 `long livre=novoAviao("IN",1);var valores=new LinkedHashMap<>(banco.consultar(c->Sql.registro(c,"SELECT modelo_id,companhia_id,matricula,situacao,fabricacao_ano FROM aeronave WHERE id=?",livre)));`  
Declara variável local livre do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
Declara variável local valores com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.  
  
Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: aeronave. Parâmetros ligados em ordem: livre.  
  
new constrói LinkedHashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.
- 410 `valores.put("fabricacao_ano",2024);cadastros.salvar(admin,TipoCadastro.AERONAVE,livre,valores);valores.put("situacao","INATIVA");cadastros.salvar(admin,TipoCadastro.AERONAVE,livre,valores);`  
Chama valores.put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.
- 411 `verificar(numero("SELECT fabricacao_ano FROM aeronave WHERE id="+livre)==2024 && contar("SELECT COUNT(*) FROM assento_aeronave WHERE aeronave_id="+livre)==1,"Edicao e inativacao da aeronave preservam configuracao");`  
Verificação deste cenário: numero("SELECT fabricacao\_ano FROM aeronave WHERE id="+livre)==2024 && contar("SELECT COUNT(\*) FROM assento\_aeronave WHERE aeronave\_id="+livre)==1, "Edicao e inativacao da aeronave preservam configuracao". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.
- 412 `rejeitar(()->planejamento.programar(admin,programacao(livre,religio.instant().plus(Duration.ofHours(3)),religio.instant().plus(Duration.ofHours(5))),"Aeronave inativa nao admite voo");`  
Verificação deste cenário: ()->planejamento.programar(admin,programacao(livre,religio.instant().plus(Duration.ofHours(3)),religio.instant().plus(Duration.ofHours(5))), "Aeronave inativa nao admite voo". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.
- 413 `long vazio=novoAviao("RZ",2);Instant proxima=religio.instant().plus(Duration.ofHours(3));long flight=planejamento.programar(admin,programacao(vazio,proxima,proxima.plus(Duration.ofHours(2))));`  
Declara variável local vazio do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
Declara variável local proxima do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
Declara variável local flight do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.
- 414 `long fare=planejamento.tarifa(admin,flight,null,new Tarifa("RASCUNHO","ECONOMICA",new BigDecimal("100"),BigDecimal.ZERO,BigDecimal.ZERO,0,true,BigDecimal.ZERO),true);`  
Declara variável local fare do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.  
  
new constrói Tarifa, BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.



```
415     testarRascunho(flight,fare);
```

Chama IntegracaoTeste.testarRascunho: Envia comandos reais de add/list/edit/remove/clear/gravar ao processo e confere a única reserva e trecho final no banco.

```
416     }
417
```

## testarRascunho - L418 a L429

Envia comandos reais de add/list/edit/remove/clear/gravar ao processo e confere a única reserva e trecho final no banco.

Entradas declaradas: (long voo,long tarifa).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.regrasComplementares (L369).

Chamadas observadas no corpo: contar, new ProcessBuilder, Path.of(System.getProperty("java.home"), "bin", "java.exe").toString, Path.of, System.getProperty, processo.environment().put, processo.environment, processo.redirectErrorStream, processo.start, filho.outputWriter, entrada.write, new String, filho.getInputStream().readAllBytes, filho.getInputStream, verificar, filho.waitFor, filho.exitValue, saida.contains.

```
418     private static void testarRascunho(long voo,long tarifa) throws Exception {
```

Início da declaração de testarRascunho. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
419         long antes=contar("SELECT COUNT(*) FROM reserva");
```

Declara variável local antes do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
420         String trecho=passageiro+"\n"+voo+"\n"+tarifa+"\n1\n\n0\n";
```

Declara variável local trecho do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
421         String outro=outroPassageiro+"\n"+voo+"\n"+tarifa+"\n1\n\n0\n";
```

Declara variável local outro do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
422         var processo=new ProcessBuilder(Path.of(System.getProperty("java.home"), "bin", "java.exe").toString
(), "-Dfile.encoding=UTF-8", "-cp", "target/classes;lib/*", "br.edu.aeroporto.Main");
```

Instrução idêntica à de IntegracaoTeste:L356. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
423         processo.environment().put("AEROPORTO_DB_URL", "jdbc:postgresql://127.0.0.1:55439/sistema_aeroporto
");processo.environment().put("AEROPORTO_DB_USUARIO", "aeroporto_teste");processo.environment().put("AEROP
ORTO_DB_SENHA", "FICTICIO-cluster-trust");
```

Instrução idêntica à de IntegracaoTeste:L357. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
424         processo.redirectErrorStream(true);var filho=processo.start();
```

Instrução idêntica à de IntegracaoTeste:L358. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
425         try(var entrada=filho.outputWriter()) { entrada.write("teste_admin\nSenhaFicticia!2026\n12\n1\n"+p
assageiro+"\n1\n"+trecho+"4\n1\n2\n1\n"+outro+"3\n1\n1\n"+trecho+"5\n1\n"+outro+"6\n15\n0\n"); }
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

```
426         String saida=new
String(filho.getInputStream().readAllBytes(), java.nio.charset.StandardCharsets.UTF_8);
```

Declara variável local saida do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
427         verificar(filho.waitFor(30,TimeUnit.SECONDS) && filho.exitValue()==0 && saida.contains("Reserva:
") && contar("SELECT COUNT(*) FROM reserva")==antes+1, "ArrayList: adicionar, listar pagina, editar,
remover, limpar e gravar pelo terminal");
```

Verificação deste cenário: filho.waitFor(30,TimeUnit.SECONDS) && filho.exitValue()==0 && saida.contains("Reserva: ") && contar("SELECT COUNT(\*) FROM reserva")==antes+1, "ArrayList: adicionar, listar pagina, editar, remover, limpar e gravar pelo terminal". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
428         verificar(contar("SELECT COUNT(*) FROM item_reserva WHERE voo_id="+voo+" AND
passageiro_id="+outroPassageiro)==1, "Rascunho grava somente o trecho final selecionado");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM item\_reserva WHERE voo\_id="+voo+" AND passageiro\_id="+outroPassageiro)==1, "Rascunho grava somente o trecho final selecionado". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
429     }
430
```



## administracaoElmportacao - L431 a L458

Testa senha antiga/nova, permissão de relatório privado, cargo/funcionário inativados, importação válida e rollback em duplicado, migração repetível.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: numero, administracao.acesso, Set.of, new AutenticacaoServico, new UsuarioJdbc, auth.entrar, "SenhaFicticia!2026".toCharArray, administracao.alterarSenha, "NovaSenhaFicticia!2026".toCharArray, rejeitar, verificar, auth.entrar("consulta\_" + sufixo, "NovaSenhaFicticia!2026".toCharArray()).usuarioId, relatorios.exportar, new RuntimeException, cad, new LinkedHashMap<>, banco.consultar, Sql.registro, cargos.put, cadastros.salvar, contar, mapa, LocalDate.now().minusDays, LocalDate.now, funcionario.put, new ArquivosServico, Path.of, cpf, Files.writeString, arquivos.importarPassageiros, banco.transacao, c.createStatement, st.execute, Files.readString.

|     |                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 431 | private static void administracaoElmportacao() throws Exception {                                                                                                                              | Início da declaração de administracaoElmportacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 432 | long usuario=numero("SELECT id FROM usuario_sistema WHERE funcionario_id="+passageiro);                                                                                                        | Declara variável local usuario do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| 433 | administracao.acesso(admin,usuario,true,Set.of("CONSULTA"));                                                                                                                                   | Chama AdministracaoServico.acesso: Usa lock de acesso, atualiza atividade e substitui vínculos de perfis. Consulta se sobrou administrador ativo; se não, toda a mudança sofre rollback.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 434 | var auth=new AutenticacaoServico(banco,new UsuarioJdbc(),auditoria);var<br>sessao=auth.entrar("consulta_" + sufixo, "SenhaFicticia!2026".toCharArray());                                       | Declara variável local auth com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>Declara variável local sessao com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.<br><br>new constrói AutenticacaoServico, UsuarioJdbc: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.                                                                                                                                                                                                                                                                                                                                                                |
| 435 | administracao.alterarSenha(sessao,"SenhaFicticia!2026".toCharArray(),"NovaSenhaFicticia!2026".toCh<br>arArray());                                                                              | Chama AdministracaoServico.alterarSenha: Valida nova senha, bloqueia o próprio usuário, confere senha atual, grava hash novo e audita. Zera atual e nova no finally.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 436 | rejeitar()->auth.entrar("consulta_" + sufixo, "SenhaFicticia!2026".toCharArray()), "Senha antiga nao<br>autentica apos alteracao");                                                            | Verificação deste cenário: ()->auth.entrar("consulta_" + sufixo, "SenhaFicticia!2026".toCharArray()), "Senha antiga nao autentica apos alteracao". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| 437 | verificar(auth.entrar("consulta_" + sufixo, "NovaSenhaFicticia!2026".toCharArray()).usuarioId()==usua<br>rio, "Alteracao de senha autenticada");                                               | Verificação deste cenário: auth.entrar("consulta_" + sufixo, "NovaSenhaFicticia!2026".toCharArray()).usuarioId()==usuario, "Alteracao de senha autenticada". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| 438 | rejeitar()->{try{relatorios.exportar(sessao,TipoRelatorio.MANIFESTO,true);}catch(java.io.IOException<br>e){throw new RuntimeException(e);}}, "Perfil consulta nao exporta manifesto privado"); | Verificação deste cenário: ()->{try{relatorios.exportar(sessao,TipoRelatorio.MANIFESTO,true);}catch(java.io.IOException e){throw new RuntimeException(e);}}, "Perfil consulta nao exporta manifesto privado". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.<br><br>Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.<br><br>Trata a(s) exceção(ões) declarada(s) em catch(java.io.IOException e). O tratamento mostrado abaixo decide se avisa, repete ou relança.<br><br>Lança new RuntimeException(e). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.<br><br>new constrói RuntimeException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só. |
| 439 | long cargo=cad(TipoCadastro.CARGO,"nome","Cargo encerrado FICTICIO "+"sufixo,"area","ATENDIMENTO");                                                                                            | Declara variável local cargo do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |

```
440      var cargos=new LinkedHashMap<>(banco.consultar(c->Sql.registro(c,"SELECT nome,area,ativo FROM
cargo WHERE
id=?",cargo)));cargos.put("ativo",false);cadastros.salvar(admin,TipoCadastro.CARGO,cargo,cargos);
```

Declara variável local cargos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: cargo. Parâmetros ligados em ordem: cargo.

new constrói LinkedHashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
441      verificar(contar("SELECT COUNT(*) FROM cargo WHERE id="+cargo+" AND NOT ativo")==1,"Cargo editado
e inativado sem exclusao");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM cargo WHERE id="+cargo+" AND NOT ativo")==1, "Cargo editado e inativado sem exclusao". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
442      long pessoa=cad(TipoCadastro.PESSOA,"nome","Funcionario desligado FICTICIO
"+sufixo,"nascimento","1980-01-01");
```

Declara variável local pessoa do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
443      cargos.put("ativo",true);cadastros.salvar(admin,TipoCadastro.CARGO,cargo,cargos);
```

Chama cargos.put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.

```
444      var funcionario=mapa("pessoa_id",pessoa,"cargo_id",cargo,"aeroporto_base_id",origem,"companhia_id"
,null,"matricula","END-"+sufixo,"admissao",LocalDate.now().minusDays(1),"desligamento",null);
```

Declara variável local funcionario com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
445      cadastros.salvar(admin,TipoCadastro.FUNCIONARIO,null,funcionario);funcionario.put("desligamento",L
ocalDate.now());cadastros.salvar(admin,TipoCadastro.FUNCIONARIO,pessoa,funcionario);
```

Chama CadastroServico.salvar: Rejeita chaves desconhecidas, converte todos os campos pelo catálogo, serializa operação, obtém dados anteriores quando id existe, valida regras do tipo, salva e audita; retorna id.

```
446      verificar(contar("SELECT COUNT(*) FROM funcionario WHERE pessoa_id="+pessoa+" AND desligamento IS
NOT NULL")==1,"Funcionario inativado por desligamento preserva identidade");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM funcionario WHERE pessoa\_id="+pessoa+" AND desligamento IS NOT NULL")==1, "Funcionario inativado por desligamento preserva identidade". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
447      var arquivos=new ArquivosServico(banco,autorizacao,auditoria,religio);
```

Instrução idêntica à de IntegracaoTeste:L154. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
448      Path entrada=Path.of("artifacts","valida-"+sufixo+".txt");String cpf=cpf();
```

Declara variável local entrada do tipo Path. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local cpf do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
449      Files.writeString(entrada,"nome\t\nascimento\t\tcpf\ttemail\ttelefone\nFICTICIO
IMPORTADO\t1990-01-01\t"+cpf+"\timportado@example.invalid\t\n");
```

Invoca Files.writeString com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
450      verificar(arquivos.importarPassageiros(admin,entrada)==1,"Importacao valida persiste passageiro e
auditoria");
```

Verificação deste cenário: arquivos.importarPassageiros(admin,entrada)==1, "Importacao valida persiste passageiro e auditoria". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
451      long antes=contar("SELECT COUNT(*) FROM pessoa");
```

Declara variável local antes do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
452      Files.writeString(entrada,"nome\t\nascimento\t\tcpf\ttemail\ttelefone\nNOVO
FICTICIO\t1990-01-01\t"+cpf()+"\t\t\nDUPLICADO FICTICIO\t1990-01-01\t"+cpf()+"\t\t\n");
```

Invoca Files.writeString com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
453      rejeitar()->{try{arquivos.importarPassageiros(admin,entrada);}catch(java.io.IOException e){throw
new RuntimeException(e);}}, "Importacao com documento existente falha no banco");
```

Verificação deste cenário: ()->{try{arquivos.importarPassageiros(admin,entrada);}catch(java.io.IOException e){throw new RuntimeException(e);}}, "Importacao com documento existente falha no banco". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Trata a(s) exceção(ões) declarada(s) em catch(java.io.IOException e). O tratamento mostrado abaixo decide se avisa, repete ou relança.

Lança new RuntimeException(e). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói RuntimeException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
454         verificar(contar("SELECT COUNT(*) FROM pessoa")==antes,"Importacao com falha no segundo registro
reverte o primeiro");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM pessoa")==antes, "Importacao com falha no segundo registro reverte o primeiro". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
455         long antesDaMigracao=contar("SELECT COUNT(*) FROM pessoa");
```

Declara variável local antesDaMigracao do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
456         banco.transacao(c->{try(var st=c.createStatement()){st.execute(Files.readString(Path.of("sql/migra
coes/001_cadastros_ativos.sql")));}catch(java.io.IOException e){throw new RuntimeException(e);}return
null;});
```

Executa o callback como unidade de gravação. O resultado só sai após commit; falha provoca rollback.

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

Trata a(s) exceção(ões) declarada(s) em catch(java.io.IOException e). O tratamento mostrado abaixo decide se avisa, repete ou relança.

Lança new RuntimeException(e). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

return null encerra esta execução entregando ausência de referência. Nos callbacks de transação sem resultado, satisfaz o contrato genérico; confira se o retorno pertence à lambda ou ao método externo.

new constrói RuntimeException: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
457         verificar(contar("SELECT COUNT(*) FROM pessoa")==antesDaMigracao,"Migracao repetida preserva dados
existentes");
```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM pessoa")==antesDaMigracao, "Migracao repetida preserva dados existentes". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
458     }
459 }
```

## cadastrosRestantes - L460 a L480

Cadastra/edita país pelo menu, edita rota, rejeita inativação com voo ativo e verifica preservação histórica após inativar rota livre.

Entradas declaradas: ().

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.main (L45).

Chamadas observadas no corpo: executarTerminal, numero, verificar, criacao.contains, texto("SELECT nome FROM pais WHERE id="+id).equals, texto, edicao.contains, new LinkedHashMap<>, banco.consultar, Sql.registro, valores.put, new BigDecimal, cadastros.salvar, rejeitar, cad, icao, novoAviao, relógio.instant().plus, relógio.instant, Duration.ofDays, programacao, inicio.plus, Duration.ofHours, new ProgramacaoVoo, p.numero, p.partida, p.chegada, p.checkinAbre, p.checkinFecha, p.embarqueAbre, p.embarqueFecha, planejamento.programar, voos.transicao, mapa, contar.

```
460     private static void cadastrosRestantes() throws Exception {
```

Início da declaração de cadastrosRestantes. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
461         String codigo="QZ",nome="País MENU FICTICIO "+sufixo;
```

Declara variável local codigo, nome do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
462         String criacao=executarTerminal("9\n1\n2\n"+codigo+"\n"+nome+"\n0\n");
```

Declara variável local criacao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
463         long id=numero("SELECT id FROM pais WHERE codigo='QZ'");
```

Declara variável local id do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
464      verificar(id>0 && criacao.contains("Cadastro salvo:"),"Pais cadastrado pelo menu real do terminal");
```

Verificação deste cenário: id>0 && criacao.contains("Cadastro salvo:"), "Pais cadastrado pelo menu real do terminal". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
465      String edicao=executarTerminal("9\n1\n3\n"+id+"\n"+codigo+"\n"+nome+" EDITADO\n9\n1\n1\n1\n0\n");
```

Declara variável local edicao do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
466      verificar(texto("SELECT nome FROM pais WHERE id="+id).equals(nome+" EDITADO") && edicao.contains(nome+" EDITADO"),"Pais editado e consultado pelo menu real do terminal");
```

Verificação deste cenário: texto("SELECT nome FROM pais WHERE id="+id).equals(nome+" EDITADO") && edicao.contains(nome+" EDITADO"), "Pais editado e consultado pelo menu real do terminal". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
467      var valores=new LinkedHashMap<>(banco.consultar(c->Sql.registro(c,"SELECT origem_id,destino_id,distancia_km,ativa FROM rota WHERE id=?",rota)));
```

Declara variável local valores com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Consulta uma linha como Map<String,Object>; ausência é recusada e mais de uma linha também falha. Tabelas mencionadas: rota. Parâmetros ligados em ordem: rota.

new constrói LinkedHashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
468      valores.put("distancia_km",new BigDecimal("1234"));cadastros.salvar(admin,TipoCadastro.ROTA,rota,valores);
```

Chama valores.put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
469      verificar(numero("SELECT distancia_km FROM rota WHERE id="+rota)==1234,"Rota editada preserva identidade");
```

Verificação deste cenário: numero("SELECT distancia\_km FROM rota WHERE id="+rota)==1234, "Rota editada preserva identidade". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```
470      valores.put("ativa",false);rejeitar(()->cadastros.salvar(admin,TipoCadastro.ROTA,rota,valores),"Rota com voos ativos nao pode ser inativada");
```

Chama valores.put: Associa chave e valor no Map e devolve o valor anterior, que pode ser null.

Verificação deste cenário: ()->cadastros.salvar(admin,TipoCadastro.ROTA,rota,valores), "Rota com voos ativos nao pode ser inativada". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

```
471      long cidade=numero("SELECT cidade_id FROM aeroporto WHERE id="+origem);
```

Declara variável local cidade do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
472      long novoDestino=cad(TipoCadastro.AEROPORTO,"cidade_id",cidade,"codigo_icao",icao(),"nome","Destino ROTA FICTICIO "+sufixo,"fuso_horario","America/Sao_Paulo");
```

Declara variável local novoDestino do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
473      long novaRota=cad(TipoCadastro.ROTA,"origem_id",origem,"destino_id",novoDestino,"distancia_km","1000");
```

Declara variável local novaRota do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
474      long livre=novoAviao("RF",2);Instant inicio=relogio.instant().plus(Duration.ofDays(2));var p=programacao(livre,inicio,inicio.plus(Duration.ofHours(2)));
```

Declara variável local livre do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local inicio do tipo Instant. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local p com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
475      var nova=new ProgramacaoVoo(companhia,novaRota,livre,p.numero(),p.partida(),p.chegada(),p.checkinAbre(),p.checkinFecha(),p.embarqueAbre(),p.embarqueFecha());
```

Declara variável local nova com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói ProgramacaoVoo: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

476         long
        historico=planejamento.programar(admin,nova);voos.transicao(admin,historico,SituacaoVoo.CANCELADO,"Teste
de inativacao de rota");

```

Declara variável local historico do tipo long. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```

477         var novos=mapa("origem_id",origem,"destino_id",novoDestino,"distancia_km",new
BigDecimal("1000"),"ativa",false);cadastros.salvar(admin,TipoCadastro.ROTA,novaRota,novos);

```

Declara variável local novos com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói BigDecimal: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

478         verificar(contar("SELECT COUNT(*) FROM rota WHERE id="+novaRota+" AND NOT ativa")==1 &&
numero("SELECT rota_id FROM voo WHERE id="+historico)==novaRota,"Rota inativada preserva voos
historicos");

```

Verificação deste cenário: contar("SELECT COUNT(\*) FROM rota WHERE id="+novaRota+" AND NOT ativa")==1 && numero("SELECT rota\_id FROM voo WHERE id="+historico)==novaRota, "Rota inativada preserva voos historicos". A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

```

479         rejeitar(()->planejamento.programar(admin,nova),"Rota inativa nao admite novos voos");

```

Verificação deste cenário: ()->planejamento.programar(admin,nova), "Rota inativa nao admite novos voos". A operação/condição deve terminar lançando a falha esperada; divergência faz o programa de teste falhar.

```

480     }

```

```

481

```

## executarTerminal - L482 a L493

Lê saída em executor enquanto escreve entrada; timeout de 30 s destrói filho travado; devolve texto apenas com saída bem-sucedida.

Entradas declaradas: (String comandos).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: IntegracaoTeste.cadastrosRestantes (L460).

Chamadas observadas no corpo: new ProcessBuilder, Path.of(System.getProperty("java.home"),"bin","java.exe").toString, Path.of, System.getProperty, processo.environment().put, processo.environment, processo.redirectErrorStream, processo.start, Executors.newSingleThreadExecutor, leitor.submit, new String, filho.getInputStream().readAllBytes, filho.getInputStream, filho.outputWriter, entrada.write, filho.waitFor, filho.destroyForcibly, new AssertionError, filho.exitValue, leitura.get.

```

482     private static String executarTerminal(String comandos) throws Exception {
        Início da declaração de executarTerminal. Os parâmetros entram pela chamada; o corpo só executa quando esse
        método/construtor é invocado.

```

```

483         var processo=new ProcessBuilder(Path.of(System.getProperty("java.home"),"bin","java.exe").toString
(),"-Dfile.encoding=UTF-8","-cp","target/classes;lib/*","br.edu.aeroporto.Main");
        Instrução idêntica à de IntegracaoTeste:L356. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

```

484         processo.environment().put("AEROPORTO_DB_URL","jdbc:postgresql://127.0.0.1:55439/sistema_aeroporto
");processo.environment().put("AEROPORTO_DB_USUARIO","aeroporto_teste");processo.environment().put("AEROPOR
TO_DB_SENHA","FICTICIO-cluster-trust");
        Instrução idêntica à de IntegracaoTeste:L357. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

```

485         processo.redirectErrorStream(true);var filho=processo.start();
        Instrução idêntica à de IntegracaoTeste:L358. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta
        execução. A posição e o contexto do método acima mostram quando ela ocorre.

```

```

486         try(var leitor=Executors.newSingleThreadExecutor()) {
        Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à
        criação.

```

```

487         var leitura=leitor.submit(()->new
String(filho.getInputStream().readAllBytes(),java.nio.charset.StandardCharsets.UTF_8));
        Declara variável local leitura com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o
        escopo é o bloco atual.

        new constrói String: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```

```

488         try(var entrada=filho.outputWriter()) {
        entrada.write("teste_admin\nSenhaFicticia!2026\n"+comandos); }
        Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à
        criação.

```

```
489         if(!filho.waitFor(30,TimeUnit.SECONDS)) { filho.destroyForcibly();throw new
AssertionError("Terminal excedeu 30 segundos"); }
```

Decisão específica: (!filho.waitFor(30,TimeUnit.SECONDS)). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

Lança new AssertionError("Terminal excedeu 30 segundos"). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói AssertionError: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
490         if(filho.exitValue()!=0) throw new AssertionError("Terminal encerrou com erro");
```

Decisão específica: (filho.exitValue()!=0). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new AssertionError("Terminal encerrou com erro"). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

new constrói AssertionError: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
491         return leitura.get(5,TimeUnit.SECONDS);
```

Retorno: entrega leitura.get(5,TimeUnit.SECONDS) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Chama leitura.get: Lê pelo índice de lista/vetor ou pela chave de Map, conforme o tipo do receptor.

```
492     }
```

```
493 }
```

```
494
```

## cpf - L495 a L498

Gera nove dígitos aleatórios e calcula dois verificadores; é massa de teste, não consulta pessoa real.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElImportacao (L431).

Chamadas observadas no corpo: new java.security.SecureRandom, new StringBuilder, codigo.append, r.nextInt, codigo.charAt, codigo.toString.

```
495     private static String cpf() {
```

Início da declaração de cpf. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
496         var r=new java.security.SecureRandom();var codigo=new StringBuilder();for(int i=0;i<9;i++)
codigo.append(r.nextInt(10));
```

Declara variável local r com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local codigo com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Laço controlado por inicialização, condição e atualização: for(int i=0;i<9;i++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói java.security.SecureRandom, StringBuilder: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
497         for(int tamanho=9;tamanho<=10;tamanho++){int soma=0;for(int i=0;i<tamanho;i++)
soma+=(codigo.charAt(i)-'0')*(tamanho+1-i);int
digito=11-soma%11;codigo.append(digito>=10?0:digito);}return codigo.toString();
```

Laço controlado por inicialização, condição e atualização: for(int tamanho=9;tamanho<=10;tamanho++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local tamanho do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local soma do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Laço controlado por inicialização, condição e atualização: for(int i=0;i<tamanho;i++). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Atualiza soma usando += e o resultado de (codigo.charAt(i)-'0')\*(tamanho+1-i). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

Declara variável local digito do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Escolha de valor: digito>=10. true escolhe 0; false escolhe digito. Só a alternativa escolhida é avaliada.



Retorno: entrega `codigo.toString()` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
498     }
499 }
```

## texto - L500 a L500

Lê primeira coluna SQL como String para uma assertiva.

Entradas declaradas: (String sql).

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: `IntegracaoTeste.bagagensEmConexao` (L295); `IntegracaoTeste.cadastrosRestantes` (L460); `IntegracaoTeste.cancelamentoAposEmbarque` (L309); `IntegracaoTeste.cenariosAdicionais` (L203); `IntegracaoTeste.regrasComplementares` (L369).

Chamadas observadas no corpo: `banco.consultar`, `Sql.unico(c,sql,r->r.getString(1)).orElse`, `Sql.unico`, `r.getString`.

```
500     private static String texto(String sql) { return
banco.consultar(c->Sql.unico(c,sql,r->r.getString(1)).orElse("")); }
```

Início da declaração de texto. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega `banco.consultar(c->Sql.unico(c,sql,r->r.getString(1)).orElse(""))` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado.

```
501 }
```

## programacao - L502 a L502

Gera record de programação com número único e janelas relativas ao início/fim.

Entradas declaradas: (long aviao,Instant inicio,Instant fim).

Saída declarada: ProgramacaoVoo - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: `IntegracaoTeste.cadastros` (L62); `IntegracaoTeste.cadastrosRestantes` (L460); `IntegracaoTeste.cancelamentos` (L190); `IntegracaoTeste.cenariosAdicionais` (L203); `IntegracaoTeste.concorrencia` (L164); `IntegracaoTeste.regrasComplementares` (L369).

Chamadas observadas no corpo: `new ProgramacaoVoo`, `sufixo.substring(0,5).toUpperCase`, `sufixo.substring`, `String.format`, `inicio.minus`, `Duration.ofHours`, `Duration.ofMinutes`.

```
502     private static ProgramacaoVoo programacao(long aviao,Instant inicio,Instant fim) { return new Programa
caoVoo(companhia,rota,aviao,sufixo.substring(0,5).toUpperCase(Locale.ROOT)+String.format("%03d",numeroVoo+
+),inicio,fim,inicio.minus(Duration.ofHours(4)),inicio.minus(Duration.ofMinutes(45)),inicio.minus(Duration
.ofMinutes(40)),inicio.minus(Duration.ofMinutes(10))); }
```

Início da declaração de programacao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega `new ProgramacaoVoo(companhia,rota,aviao,sufixo.substring(0,5).toUpperCase(Locale.ROOT)+String.format("%03d",numeroVoo++),inicio,fim,inicio.minus(Duration.ofHours(4)),inicio.minus(Duration.ofMinutes(45)),inicio.minus(D...)` ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

`new` constrói `ProgramacaoVoo`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

## icao - L503 a L509

Tenta códigos aleatórios e confere se já usados na massa.

Entradas declaradas: ().

Saída declarada: String - texto imutável.

Possíveis chamadores identificados: `IntegracaoTeste.cadastros` (L62); `IntegracaoTeste.cadastrosRestantes` (L460).

Chamadas observadas no corpo: `new StringBuilder`, `new java.security.SecureRandom().ints(3,0,26).forEach`, `new java.security.SecureRandom().ints`, `new java.security.SecureRandom`, `b.append`, `b.toString`, `banco.consultar`, `Sql.listar(c,"SELECT id FROM aeroporto WHERE codigo_icao=? UNION ALL SELECT id FROM companhia_aerea WHERE codigo_icao=?")`, `r->r.getLong(1)`, `codigo`, `codigo.substring(1)`, `isEmpty`, `Sql.listar`, `r.getLong`, `codigo.substring`.

```
503     private static String icao() {
```

Início da declaração de icao. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.



```
504         while(true) {
```

Repete enquanto a condição permanecer verdadeira: while(true). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

```
505         StringBuilder b=new StringBuilder("X");new
java.security.SecureRandom().ints(3,0,26).forEach(n->b.append((char)('A'+n)));String codigo=b.toString();
```

Declara variável local b do tipo StringBuilder. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Declara variável local codigo do tipo String. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

new constrói StringBuilder, java.security.SecureRandom: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
506         boolean usado=banco.consultar(c->!Sql.listar(c,"SELECT id FROM aeroporto WHERE codigo_icao=?
UNION ALL SELECT id FROM companhia_aerea WHERE
codigo_icao=?",r->r.getLong(1),codigo,codigo.substring(1)).isEmpty());
```

Declara variável local usado do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Consulta uma lista, aplicando o mapeador a cada linha, e devolve List<T>. Tabelas mencionadas: aeroporto, companhia\_aerea. Parâmetros ligados em ordem: codigo, codigo.substring(1).

```
507         if(!usado) return codigo;
```

Decisão específica: (!usado). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo encerra a execução atual com return.

Retorno: entrega codigo ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

```
508     }
```

```
509 }
```

## cad - L510 a L512

Monta entrada do cadastro e chama serviço, devolvendo id.

Entradas declaradas: (TipoCadastro tipo,Object... valores).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); IntegracaoTeste.bagagensEmConexao (L295); IntegracaoTeste.cadastros (L62); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.novoAviao (L364); IntegracaoTeste.regrasComplementares (L369).

Chamadas observadas no corpo: new LinkedHashMap<String,Object>, tipo.campos, dados.put, campo.nome, campo.tipo().equals, campo.tipo, dados.putAll, mapa, cadastros.salvar.

```
510     private static long cad(TipoCadastro tipo,Object... valores) {
```

Início da declaração de cad. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

```
511         var dados=new LinkedHashMap<String,Object>();for(var campo:tipo.campos())
dados.put(campo.nome(),campo.tipo().equals("b")?true:null);dados.putAll(mapa(valores));return
cadastros.salvar(admin,tipo,null,dados);
```

Declara variável local dados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Percorre cada elemento da coleção/vetor: for(var campo:tipo.campos()). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Escolha de valor: campo.tipo().equals("b"). true escolhe true; false escolhe null. Só a alternativa escolhida é avaliada.

Retorno: entrega cadastros.salvar(admin,tipo,null,dados) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói LinkedHashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
512     }
```

## mapa - L513 a L513

Transforma pares chave/valor em LinkedHashMap; completa campos opcionais conforme o helper.

Entradas declaradas: (Object... valores).

Saída declarada: Map<String,Object> - valor/coleção/objeto compatível com o tipo declarado, salvo interrupção por exceção.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElImportacao (L431); IntegracaoTeste.cad (L510); IntegracaoTeste.cadastros (L62); IntegracaoTeste.cadastrosRestantes (L460).

Chamadas observadas no corpo: new LinkedHashMap<String,Object>, dados.put, valores[i].toString.

```
513     private static Map<String,Object> mapa(Object... valores) { var dados=new
      LinkedHashMap<String,Object>();for(int i=0;i<valores.length;i+=2)
      dados.put(valores[i].toString(),valores[i+1]);return dados; }
```

Início da declaração de mapa. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Declara variável local dados com tipo inferido pelo inicializador. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Laço controlado por inicialização, condição e atualização: for(int i=0;i<valores.length;i+=2). return, break ou exceção podem encerrar esse caminho; continue passa à próxima volta.

Declara variável local i do tipo int. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

Atualiza i usando += e o resultado de 2. Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

Retorno: entrega dados ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

new constrói LinkedHashMap: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

## contar - L514 a L514

Obtém valor agregado como long e converte para contagem int.

Entradas declaradas: (String sql).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); IntegracaoTeste.arquivos (L153); IntegracaoTeste.atendimento (L127); IntegracaoTeste.cadastros (L62); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.comercial (L109); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.falhasEDeadlock (L334); IntegracaoTeste.inicializar (L50); IntegracaoTeste.regrasComplementares (L369); IntegracaoTeste.testarRascunho (L418).

Chamadas observadas no corpo: numero.

```
514 private static long contar(String sql) { return numero(sql); }
```

Início da declaração de contar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega numero(sql) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

## numero - L515 a L515

Obtém primeira coluna SQL como long.

Entradas declaradas: (String sql).

Saída declarada: long - inteiro primitivo de 64 bits; neste contexto representa id, contador ou valor inteiro identificado pelo nome.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); IntegracaoTeste.atendimento (L127); IntegracaoTeste.bagagensEmConexao (L295); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.comercial (L109); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.contar (L514); IntegracaoTeste.regrasComplementares (L369).

Chamadas observadas no corpo: banco.consultar, Sql.unico(c,sql,r->r.getLong(1)).orElse, Sql.unico, r.getLong.

```
515 private static long numero(String sql) { return
    banco.consultar(c->Sql.unico(c,sql,r->r.getLong(1)).orElse(0L)); }
```

Início da declaração de numero. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Retorno: entrega banco.consultar(c->Sql.unico(c,sql,r->r.getLong(1)).orElse(0L)) ao chamador desta execução; nenhuma instrução posterior desse mesmo corpo executa por esse caminho.

Executa o callback com Connection somente leitura e isolamento repeatable-read. A função recebida produz o resultado da consulta.

Consulta no máximo uma linha; entrega Optional e usa o mapeador recebido para definir o tipo do resultado.

## verificar - L516 a L516

Condição falsa lança AssertionError; verdadeira incrementa verificacoes e imprime OK.

Entradas declaradas: (boolean condicao,String nome).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: IntegracaoTeste.administracaoElimportacao (L431); IntegracaoTeste.arquivos (L153); IntegracaoTeste.atendimento (L127); IntegracaoTeste.bagagensEmConexao (L295); IntegracaoTeste.cadastros (L62); IntegracaoTeste.cadastrosRestantes (L460); IntegracaoTeste.cancelamentoAposEmbarque (L309); IntegracaoTeste.cancelamentos (L190); IntegracaoTeste.cenariosAdicionais (L203); IntegracaoTeste.comercial (L109); IntegracaoTeste.concorrencia (L164); IntegracaoTeste.falhasEDeadlock (L334); IntegracaoTeste.inicializar (L50); IntegracaoTeste.regrasComplementares (L369); IntegracaoTeste.rejeitar (L517); IntegracaoTeste.testarRascunho (L418); IntegracaoTeste.testarTerminal (L355).

Chamadas observadas no corpo: new AssertionError, System.out.println.

```
516 private static void verificar(boolean condicao,String nome) { verificacoes++;if(!condicao) throw new
    AssertionError(nome);System.out.println("OK "+nome); }
```

Início da declaração de verificar. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Decisão específica: (!condicao). Se for true, executa o ramo seguinte; se for false, pula esse ramo. Aqui o ramo interrompe a operação lançando a exceção.

Lança new AssertionError(nome). Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar rollback.

`new` constrói `AssertionError`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

## rejeitar - L517 a L517

Executa operação que deve lançar regra/persistência; se ocorrer sucesso inesperado, lança `AssertionError`.

Entradas declaradas: (`Runnable acao`, `String nome`).

Saída declarada: `void` - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Possíveis chamadores identificados: `IntegracaoTeste.administracaoElimportacao` (L431); `IntegracaoTeste.arquivos` (L153); `IntegracaoTeste.atendimento` (L127); `IntegracaoTeste.cadastros` (L62); `IntegracaoTeste.cadastrosRestantes` (L460); `IntegracaoTeste.cancelamentoAposEmbarque` (L309); `IntegracaoTeste.cancelamentos` (L190); `IntegracaoTeste.cenariosAdicionais` (L203); `IntegracaoTeste.comercial` (L109); `IntegracaoTeste.concorrencia` (L164); `IntegracaoTeste.falhasEDeadlock` (L334); `IntegracaoTeste.inicializar` (L50); `IntegracaoTeste.regrasComplementares` (L369).

Chamadas observadas no corpo: `acao.run`, `verificar`, `new AssertionError`.

```
517     private static void rejeitar(Runnable acao,String nome) { try{acao.run();}catch(RegraNegocioException
| PersistenciaException esperado){verificar(true,nome);return;}throw new AssertionError(nome); }
```

Início da declaração de `rejeitar`. Os parâmetros entram pela chamada; o corpo só executa quando esse método/construtor é invocado.

Delimita a operação protegida. As capturas seguintes tratam falhas desta execução.

Trata a(s) exceção(ões) declarada(s) em `catch(RegraNegocioException | PersistenciaException esperado)`. O tratamento mostrado abaixo decide se avisa, repete ou relança.

Verificação deste cenário: `true`, `nome`. A operação/condição deve resultar em verdadeiro; divergência faz o programa de teste falhar.

`return` encerra o caminho atual sem entregar valor (`void`).

Lança `new AssertionError(nome)`. Interrompe o caminho normal; a exceção sobe até uma captura compatível e pode provocar `rollback`.

`new` constrói `AssertionError`: aloca a instância e executa seu construtor com os argumentos mostrados; não insere no banco por si só.

```
518 }
```

## PrepararBancoTeste

scripts/PrepararBancoTeste.java

Programa auxiliar do script de teste: cria o banco do cluster isolado se necessário e aplica schema/migração, depois conta as 41 tabelas.

23 linhas | 1 métodos/construtores declarados | SHA-256: a5ecd4283fc878cf...

```
1 import java.nio.file.Files;
2 import java.nio.file.Path;
3 import java.sql.DriverManager;
4
5 public class PrepararBancoTeste {
```

Declara PrepararBancoTeste como tipo com campos e métodos.

### main - L6 a L22

Conecta apenas ao cluster da porta 55439, cria sistema\_aeroporto se não existir, instala schema se pessoa estiver ausente, aplica migração e conta tabelas.

Entradas declaradas: (String[] args).

Saída declarada: void - não entrega valor ao chamador; pode executar efeitos e lançar exceções.

Sem chamador direto resolvido nesta análise. Pode ser ponto de entrada, contrato, construtor, callback, sobrescrita ou chamada por referência/interface.

Chamadas observadas no corpo: DriverManager.getConnection, c.createStatement, s.executeQuery, r.next, r.getBoolean, s.executeUpdate, s.execute, Files.readString, Path.of, System.out.println, r.getInt.

```
6 public static void main(String[] args) throws Exception {
```

Instrução idêntica à de IntegracaoTeste:L45. Aplica aqui a mesma mecânica, dentro deste bloco e com as variáveis locais desta execução. A posição e o contexto do método acima mostram quando ela ocorre.

```
7 // Esta porta pertence exclusivamente ao cluster descartavel de artifacts.
```

Comentário/documentação do autor; descreve intenção ou convenção e não executa.

```
8 try(var
c=DriverManager.getConnection("jdbc:postgresql://127.0.0.1:55439/postgres","aeroporto_teste","");var
s=c.createStatement()) {
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

```
9 try(var r=s.executeQuery("SELECT EXISTS (SELECT 1 FROM pg_database WHERE
datname='sistema_aeroporto')")) {
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

```
10 r.next();if(!r.getBoolean(1)) s.executeUpdate("CREATE DATABASE sistema_aeroporto");
```

Chama r.next: Avança o cursor ResultSet; true indica que há uma linha para ler.

Decisão específica: (!r.getBoolean(1)). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
11 }
```

```
12 }
```

```
13 try(var c=DriverManager.getConnection("jdbc:postgresql://127.0.0.1:55439/sistema_aeroporto","aerop
orto_teste","");var s=c.createStatement()) {
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

```
14 boolean existe;
```

Declara variável local existe do tipo boolean. O valor é obtido pela expressão de inicialização mostrada; o escopo é o bloco atual.

```
15 try(var r=s.executeQuery("SELECT to_regclass('aeroporto.pessoa') IS NOT NULL")) {
r.next();existe=r.getBoolean(1); }
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

Atualiza existe usando = e o resultado de r.getBoolean(1). Atribuição modifica a variável/célula indicada; não é um teste de igualdade.

```
16 if(!existe) s.execute(Files.readString(Path.of("sql/criar_banco.sql")));
```

Decisão específica: (!existe). Se for true, executa o ramo seguinte; se for false, pula esse ramo.

```
17         s.execute(Files.readString(Path.of("sql/migracoes/001_cadastrados_ativos.sql")));
```

Invoca s.execute com os argumentos mostrados. O receptor/contrato determina a implementação; veja as chamadas do método para localizar o corpo no projeto.

```
18         try(var r=s.executeQuery("SELECT COUNT(*) FROM information_schema.tables WHERE  
table_schema='aeroporto' AND table_type='BASE TABLE'")) {
```

Delimita a operação protegida. Os recursos entre parênteses são fechados automaticamente ao final, em ordem inversa à criação.

```
19         r.next();System.out.println("Tabelas no PostgreSQL de teste: "+r.getInt(1));
```

Chama r.next: Avança o cursor ResultSet; true indica que há uma linha para ler.

```
20     }
```

```
21 }
```

```
22 }
```

```
23 }
```

# Web: como a interface e a API se conectam

## O significado de web

web/ contém a interface executada no navegador. Um arquivo .jsx mistura expressões JavaScript com marcação JSX transformada pelo Vite. Componentes retornam uma descrição da interface; React atualiza o DOM. O servidor em server/ executa JavaScript no Node, trata HTTP e consulta PostgreSQL pelo pacote pg.

## Componentes, props e estado

Um componente como Flights ou ReservationForm é uma função que recebe props. Props são entradas fornecidas pelo componente pai. useState conserva estado entre renderizações; chamar o setter agenda atualização. useEffect executa sincronização depois da renderização e pode devolver limpeza. onClick={funcao} entrega o manipulador: a função roda no clique. {condicao && <Elemento/>} e expressões ternárias escolhem partes da tela.

## JSX e className

class em Java declara uma classe. className em JSX define classes CSS de um elemento visual. <Modal> é um componente; <div> é uma tag do navegador. key ajuda React a reconhecer itens de uma lista; não é o id persistido automaticamente. preventDefault impede a navegação padrão do formulário. Controlar um campo com value/onChange mantém seu valor no estado.

## async, await e retornos

Uma função async devolve Promise. await aguarda a operação dentro da função; não congela toda a interface. api(path, options) chama fetch, lê JSON, verifica response.ok e devolve os dados; nos erros lança Error com status/fields. No download, a resposta vira Blob, uma URL temporária e um link acionado para salvar; a URL é liberada depois.

## HTTP, rota e JSON

GET lê; POST cria ou aciona uma operação; PUT edita no contrato usado aqui. /api/reservations/:id/checkin tem um parâmetro de caminho; ?date=... tem parâmetro de consulta. JSON serializa valores entre cliente e servidor. Um código HTTP representa o resultado da requisição; é diferente do enum de situação de voo. Express registra manipuladores e middleware na ordem declarada.

## Sessão e autorização web

server/app.mjs mantém sessões e tentativas de login em Maps do processo. A sessão dura oito horas e o cookie é HttpOnly com SameSite Strict; reiniciar esse processo perde as sessões em memória. Origin e token CSRF são verificados nas operações protegidas. O perfil escolhe permissões. Ocultar um botão pela interface melhora a experiência; a autorização no servidor é que controla a operação recebida.

## Validação, transação e edição concorrente

validation.mjs define schemas Zod, incluindo validação de CPF. operations.mjs verifica relações/regras antes de escrever e usa as transações de database.mjs. A versão do registro ajuda a rejeitar edição sobre uma versão antiga. AsyncLocalStorage mantém o cliente da transação nas consultas daquela execução. O Pool reutiliza conexões; os schemas são selecionados de forma controlada.

## Diferenças reais em relação ao Java

A reserva web usa um fluxo simplificado com confirmação e assento, sem reproduzir integralmente os pagamentos e as 41 tabelas da aplicação Java. Os estados web e Java têm nomes e transições diferentes. A interface mostra dados do schema aerohub, não os DTOs Java. Para mudar uma regra nos dois produtos, é necessário avaliar os dois caminhos de código.

## Tratamento de erro visual

ErrorBoundary em main.jsx é uma classe React que trata falhas de renderização descendentes. Não substitui o try/catch de uma requisição assíncrona. App e formulários recebem resultados, exibem mensagens, estados de carregamento e erros de campos. O servidor transforma validação em resposta 422, recusas em status de negócio e duplicidades em 409 quando aplicável.



## Caminho completo de uma reserva web

ReservationForm valida campos e chama a operação fornecida por App. App envia POST /api/reservations por api(). fetch inclui cookie e token CSRF. Express autentica, confere permissão e valida o JSON. createReservation em operations.mjs usa a transação PostgreSQL, confere voo/passageiro/assento e grava reserva/auditoria. A resposta JSON volta a App, que atualiza os dados e mostra o resultado. Se a API lança Error no cliente, a tela apresenta a mensagem sem fingir sucesso.

## web/main.jsx

Monta a árvore React no elemento root, importa fontes/CSS e define ErrorBoundary para falhas de renderização.

Arquivo com 40 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## web/App.jsx

Coordena autenticação, sessão, navegação por hash, atualização do estado remoto, filtros, abertura de formulários e operações de reserva/check-in/cancelamento/exportação.

| Linha | Função/componente | Parâmetros visíveis |
|-------|-------------------|---------------------|
| 120   | route             | Nenhum              |
| 123   | Brand             | { dark = false }    |
| 139   | Login             | { onLogin }         |
| 145   | submit            | e, credentials      |
| 253   | App               | Nenhum              |
| 326   | hash              | Nenhum              |
| 339   | outside           | e                   |
| 346   | navigate          | id                  |
| 353   | edit              | entity, row         |
| 354   | create            | entity              |
| 355   | openFlight        | row                 |
| 356   | reserve           | flight, passenger   |
| 357   | save              | entity, body, id    |
| 368   | exportData        | entity              |
| 376   | checkin           | row                 |
| 389   | cancel            | row                 |
| 404   | logout            | Nenhum              |
| 454   | openSearch        | result              |

Arquivo com 837 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## web/api.js

Contrato HTTP compartilhado: CSRF, envio JSON, conversão de erro e download CSV.

| Linha | Função/componente | Parâmetros visíveis |
|-------|-------------------|---------------------|
| 2     | setCsrft          | value               |
| 5     | api               | path, options = {}  |
| 21    | download          | table, date         |

## api.js: leitura de cada instrução

```

1 let csrf = '';
  Variável de módulo guarda o token CSRF atual.

2 export function setCsrft(value) {
  Exporta o setter; App pode atualizar o token após autenticação.

3   csrf = value;
  Atribui o valor recebido ao estado do módulo.

4 }

5 export async function api(path, options = {}) {
  Função async; options assume objeto vazio se omitido.

6   const response = await fetch(`/api${path}`, {
  Inicia fetch no prefixo /api e aguarda a resposta HTTP.

7     credentials: 'same-origin',
  Envia credenciais do mesmo site, incluindo o cookie da sessão.

8     ...options,
  Copia opções fornecidas pelo chamador; a ordem de spread permite substituições.

9     headers: { 'Content-Type': 'application/json', 'X-CSRF-Token': csrf, ...options.headers },
  Compõe cabeçalhos JSON/CSRF e permite os cabeçalhos adicionais.

10    ...(options.body ? { body: JSON.stringify(options.body) } : {}),
  Se body existir, serializa o objeto para JSON; caso contrário acrescenta objeto vazio.

11  });
12  const data = await response.json();
  Aguarda o corpo JSON. Falha de parse também rejeita a Promise.

13  if (!response.ok) {
  response.ok é false fora da faixa de sucesso HTTP.

14    const error = new Error(data.message || 'Não foi possível concluir a operação.');
```

Cria Error com a mensagem da API ou texto padrão.

```

15    error.status = response.status;
  Anexa status HTTP para o tratamento da tela.

16    error.fields = data.fields;
  Anexa os erros de campos para a tela associar cada mensagem.

17    throw error;
  Rejeita a execução lançando o erro; não retorna sucesso.

18  }
19  return data;
  Devolve os dados JSON ao chamador que aguardou api().

20 }

21 export async function download(table, date) {
  Exporta a operação assíncrona de download.

22   const response = await fetch(`/api/export/${table}?date=${date}`, { credentials: 'same-origin' });
  Consulta o endpoint CSV com data e cookie.

23   if (!response.ok) throw new Error((await response.json()).message);
  Se a resposta falhar, lê a mensagem JSON e lança Error.

24   const url = URL.createObjectURL(await response.blob());
  Converte resposta em Blob e cria uma URL temporária para ele.

25   const link = document.createElement('a');
```

Cria um elemento de link na memória do DOM.

```
26  link.href = url;
    Define a origem do arquivo no link.

27  link.download = `aerohub-${table}-${date}.csv`;
    Define o nome sugerido para salvar o CSV.

28  link.click();
    Dispara o download acionando o link.

29  setTimeout(() => URL.revokeObjectURL(url), 1000);
    Libera a URL temporária após um segundo; evita manter esse recurso indefinidamente.

30 }
```

Arquivo com 30 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

web/model.js

Rótulos e permissões, horários/datas locais, normalização de busca, índices dos dados e alertas derivados.

| Linha | Função/componente | Parâmetros visíveis |
|-------|-------------------|---------------------|
| 29    | can               | user, entity        |
| 30    | time              | value               |
| 38    | dateLabel         | value               |
| 44    | dateTime          | value               |
| 54    | localDay          | value               |
| 56    | inputTime         | value               |
| 58    | fromInputTime     | value               |
| 60    | initials          | name                |
| 66    | normalize         | value               |
| 71    | activeFlight      | f                   |
| 72    | seatName          | index               |
| 73    | effective         | f                   |
| 74    | indexState        | state               |
| 80    | operationalAlerts | state, date         |

Arquivo com 134 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

web/Dashboard.jsx

Compõe indicadores, painéis e resumo operacional a partir dos dados recebidos.

| Linha | Função/componente | Parâmetros visíveis                                             |
|-------|-------------------|-----------------------------------------------------------------|
| 17    | Dashboard         | { state, index, date, alerts, onFlight, navigate, tab, setTab } |

Arquivo com 281 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## web/Views.jsx

Componentes das páginas Flights, Passengers, Airlines, Aircraft, Gates, Alerts, History e Reports, com filtros e ações de cada cadastro.

| Linha | Função/componente | Parâmetros visíveis                                        |
|-------|-------------------|------------------------------------------------------------|
| 40    | FlightsView       | { state, index, user, date, onFlight, onCreate, onExport } |
| 47    | filter            | setter                                                     |
| 364   | AirlinesView      | { state, index, user, onEdit, onCreate, onHistory }        |
| 455   | AircraftView      | { state, index, user, onEdit, onCreate }                   |
| 571   | GatesView         | { state, index, user, date, onEdit, onCreate, onFlight }   |
| 679   | AlertsView        | { alerts, onOpen }                                         |
| 711   | HistoryView       | { state }                                                  |
| 764   | ReportsView       | { state, index, date, user, onExport }                     |

Arquivo com 854 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## web/forms.jsx

EntityForm e telas FlightDetail, ReservationForm, BoardingPass e ConfirmDialog. Converte campos, envia operações e mostra validações.

| Linha | Função/componente | Parâmetros visíveis                                               |
|-------|-------------------|-------------------------------------------------------------------|
| 25    | EntityForm        | { entity, row, state, index, user, onSave, onClose }              |
| 80    | set               | key, value                                                        |
| 81    | input             | key, type = 'text', extra = {}                                    |
| 90    | select            | key, options                                                      |
| 99    | options           | rows                                                              |
| 100   | field             | key, label, content, wide = false                                 |
| 110   | submit            | e                                                                 |
| 406   | FlightDetail      | { flight: f, state, index, user, onClose, onEdit, onReservation } |
| 508   | ReservationForm   | { state, index, onSave, onClose, flight, passenger }              |
| 523   | submit            | e                                                                 |
| 649   | BoardingPass      | { reservation: r, index, onClose }                                |
| 746   | ConfirmDialog     | { title, detail, onConfirm, onClose }                             |
| 749   | confirm           | Nenhum                                                            |

Arquivo com 780 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## web/components.jsx

Peças compartilhadas: botões, badges, avatar, modal, campo, busca, paginação, estados vazios/carregamento e tabela de voos.

| Linha | Função/componente | Parâmetros visíveis                                           |
|-------|-------------------|---------------------------------------------------------------|
| 14    | IconButton        | { icon: Icon, label, ...props }                               |
| 21    | Badge             | { status }                                                    |
| 29    | AirlineMark       | { airline, small = false }                                    |
| 39    | Avatar            | { name, small = false }                                       |
| 54    | Loading           | Nenhum                                                        |
| 62    | Modal             | { title, subtitle, children, onClose, wide = false }          |
| 68    | handler           | event                                                         |
| 120   | Field             | { label, error, children, wide = false }                      |
| 141   | SearchInput       | { value, onChange, placeholder = 'Buscar...' }                |
| 155   | Pagination        | { page, setPage, total, size = 10 }                           |
| 184   | FlightTable       | { flights, index, onOpen, compact = false, showDate = false } |

Arquivo com 258 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## web/styles.css

Define aparência, espaçamento, cores, responsividade e impressão; não altera regras persistidas.

Arquivo com 3425 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## server/index.mjs

Abre o banco, cria a aplicação Express, integra Vite em desenvolvimento ou dist em produção, inicia escuta e fecha recursos nos sinais.

Arquivo com 33 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## server/app.mjs

Define middleware, sessões/login, permissões e rotas da API. Delega regras a operations e traduz falhas em respostas HTTP.

| Linha | Função/componente | Parâmetros visíveis |
|-------|-------------------|---------------------|
| 14    | createApp         | db                  |

## Rotas registradas

| Linha | Verbo | Caminho                       |
|-------|-------|-------------------------------|
| 43    | POST  | /api/login                    |
| 93    | GET   | /api/session                  |
| 94    | POST  | /api/logout                   |
| 99    | GET   | /api/state                    |
| 100   | POST  | /api/reservations             |
| 103   | POST  | /api/reservations/:id/checkin |
| 106   | POST  | /api/reservations/:id/cancel  |
| 109   | GET   | /api/export/:table            |
| 116   | POST  | /api/:table                   |
| 119   | PUT   | /api/:table/:id               |

Arquivo com 142 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## server/database.mjs

Configuração PostgreSQL, Pool, cliente transacional por AsyncLocalStorage, seleção de schema e inicialização opcional da massa demo.

| Linha | Função/componente | Parâmetros visíveis             |
|-------|-------------------|---------------------------------|
| 6     | today             | Nenhum                          |
| 8     | localTime         | day, hour                       |
| 9     | passwordHash      | password, salt = randomBytes(16 |
| 32    | parameters        | sql                             |
| 39    | localConnection   | Nenhum                          |
| 127   | transaction       | db, action                      |

Arquivo com 147 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## server/operations.mjs

Autoriza, lê o estado, valida relações e datas, salva entidades com versão, reserva/check-in/cancela e exporta CSV.

| Linha | Função/componente | Parâmetros visíveis                        |
|-------|-------------------|--------------------------------------------|
| 22    | ms                | value                                      |
| 23    | flightTime        | flight                                     |
| 24    | gateOverlap       | a, b                                       |
| 34    | authorize         | user, capability                           |
| 41    | audit             | db, user, action, entity, entityId, detail |

| Linha | Função/componente | Parâmetros visíveis           |
|-------|-------------------|-------------------------------|
| 52    | record            | db, table, id                 |
| 58    | readState         | db, user                      |
| 80    | validateBusiness  | db, table, item, existing     |
| 219   | saveEntity        | db, user, table, raw, id      |
| 274   | seatIndex         | seat                          |
| 279   | createReservation | db, user, raw                 |
| 338   | updateReservation | db, user, id, action, version |
| 397   | csvExport         | db, user, table, day          |
| 425   | quote             | value                         |

Arquivo com 437 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## server/validation.mjs

Schemas de entrada Zod e cálculo de CPF. Rejeita formato/tipo antes da regra relacional.

| Linha | Função/componente | Parâmetros visíveis |
|-------|-------------------|---------------------|
| 3     | text              | min = 2, max = 120  |
| 117   | validCpf          | value               |

Arquivo com 126 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## server/errors.mjs

BusinessError carrega status/código; ensure lança essa falha quando uma condição não é atendida.

| Linha | Função/componente | Parâmetros visíveis              |
|-------|-------------------|----------------------------------|
| 8     | ensure            | condition, message, status = 409 |

Arquivo com 10 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.

## server/demo.mjs

Gera dados fictícios para demonstração em schema isolado quando solicitado. Não é cadastro real de uma companhia.

| Linha | Função/componente | Parâmetros visíveis |
|-------|-------------------|---------------------|
| 3     | seed              | db                  |

Arquivo com 202 linhas. A análise linha por linha integral deste guia é a dos arquivos Java; este capítulo identifica as funções web e explica seu papel no fluxo e na linguagem.



# Banco PostgreSQL, arquivos e ferramentas

## sql/criar\_banco.sql

Cria o schema aeroporto e o modelo relacional Java. pessoa é identidade; passageiro e funcionario são papéis. voo se conecta à companhia, rota e aeronave. inventario\_assento\_voo copia a configuração física por voo; ocupacao\_assento mantém uso/histórico. reserva agrega item\_reserva; pagamento/reembolso e bilhete/checkin/bagagem registram fases específicas.

| Linha SQL | Tabela                |
|-----------|-----------------------|
| 35        | pais                  |
| 37        | pais                  |
| 44        | cidade                |
| 46        | cidade                |
| 55        | aeroporto             |
| 57        | aeroporto             |
| 68        | terminal              |
| 70        | terminal              |
| 80        | recurso_aeroportuario |
| 82        | recurso_aeroportuario |
| 99        | companhia_aerea       |
| 101       | companhia_aerea       |
| 111       | pessoa                |
| 113       | pessoa                |
| 123       | documento_pessoa      |
| 125       | documento_pessoa      |
| 136       | contato_pessoa        |
| 138       | contato_pessoa        |
| 153       | passageiro            |
| 155       | passageiro            |
| 162       | cargo                 |
| 164       | cargo                 |
| 171       | funcionario           |
| 173       | funcionario           |
| 185       | modelo_aeronave       |
| 187       | modelo_aeronave       |

| Linha SQL | Tabela                 |
|-----------|------------------------|
| 197       | habilitacao_tripulante |
| 199       | habilitacao_tripulante |
| 210       | usuario_sistema        |
| 212       | usuario_sistema        |
| 223       | perfil_acesso          |
| 225       | perfil_acesso          |
| 232       | usuario_perfil         |
| 234       | usuario_perfil         |
| 241       | aeronave               |
| 243       | aeronave               |
| 253       | assento_aeronave       |
| 255       | assento_aeronave       |
| 268       | rota                   |
| 270       | rota                   |
| 279       | voo                    |
| 281       | voo                    |
| 310       | inventario_assento_voo |
| 312       | inventario_assento_voo |
| 326       | manutencao_aeronave    |
| 328       | manutencao_aeronave    |
| 344       | agenda_aeronave        |
| 346       | agenda_aeronave        |
| 361       | alocacao_recurso       |
| 363       | alocacao_recurso       |
| 378       | escala_funcionario     |
| 380       | escala_funcionario     |
| 395       | tarifa_voo             |
| 397       | tarifa_voo             |
| 413       | reserva                |
| 415       | reserva                |
| 428       | item_reserva           |

| Linha SQL | Tabela                 |
|-----------|------------------------|
| 430       | item_reserva           |
| 461       | ocupacao_assento       |
| 463       | ocupacao_assento       |
| 487       | bilhete                |
| 489       | bilhete                |
| 499       | pagamento              |
| 501       | pagamento              |
| 515       | reembolso              |
| 517       | reembolso              |
| 531       | check_in               |
| 533       | check_in               |
| 551       | bagagem                |
| 553       | bagagem                |
| 565       | evento_bagagem         |
| 567       | evento_bagagem         |
| 578       | embarque               |
| 580       | embarque               |
| 592       | historico_voo          |
| 594       | historico_voo          |
| 607       | ocorrencia_operacional |
| 609       | ocorrencia_operacional |
| 623       | servico_solo           |
| 625       | servico_solo           |
| 641       | evento_auditoria       |
| 643       | evento_auditoria       |

## sql/web.sql

Cria as nove tabelas do modelo web no schema configurado. Esse modelo é próprio de aerohub; não migra automaticamente as tabelas do modelo Java. Índices, restrições e transações complementam validações da API.

| Linha SQL | Tabela   |
|-----------|----------|
| 2         | users    |
| 6         | airlines |

| Linha SQL | Tabela       |
|-----------|--------------|
| 10        | terminals    |
| 13        | gates        |
| 17        | aircraft     |
| 22        | flights      |
| 30        | passengers   |
| 35        | reservations |
| 43        | audit        |

## sql/migracoes/001\_cadastros\_ativos.sql

Migração aditiva: altera estruturas/regras sem reconstruir todo o banco. O Main pode aplicá-la com --migrar, e a inicialização comum verifica se a coluna indicadora existe. Execute no banco apropriado, depois de revisar a configuração.

## As palavras SQL usadas pelo código

### SELECT, JOIN e filtros

SELECT escolhe colunas/expressões; FROM escolhe origem; JOIN combina registros pela condição ON. LEFT JOIN preserva a linha da esquerda mesmo sem correspondência. WHERE filtra antes da agregação; GROUP BY agrupa; HAVING filtra grupos; ORDER BY ordena; LIMIT/OFFSET paginam. COUNT/SUM/MIN/MAX agregam. COALESCE escolhe o primeiro valor não nulo; EXISTS verifica presença.

### Escrita e retorno

INSERT cria; UPDATE modifica as linhas do WHERE; DELETE remove, onde usado. RETURNING entrega colunas da linha afetada ao JDBC. ON CONFLICT especifica tratamento de colisão. Marcadores ? do PreparedStatement recebem valores separados do texto SQL. Identificadores de tabelas/colunas precisam de catálogo controlado: TipoCadastro impede que a entrada do usuário seja um nome arbitrário.

### Restrições e histórico

PRIMARY KEY identifica uma linha; FOREIGN KEY mantém referência válida; UNIQUE evita duplicidade; CHECK restringe valores; NOT NULL impede ausência. Índices parciais restringem só as linhas que atendem ao predicado, como ocupação ativa. Exclusões por intervalo protegem agendas com btree\_gist. Triggers executam rotinas no banco, inclusive proteção de histórico, independentemente de o chamador ser o menu.

### Estados e nomes

SQLSTATE é um código de erro do PostgreSQL/JDBC, como 23505 para duplicidade. Não é SituacaoVoo. Schema é um espaço de nomes dentro do banco, não o banco inteiro. currentSchema em BancoDados coloca aeroporto/public no caminho de busca. Timestamps são normalizados a microssegundos; apresentação local depende do fuso do aeroporto/operação.

## Arquivos que completam a execução

| Arquivo/pasta                         | Para que serve                                                                                                                       |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| compilar.ps1                          | Seleciona Java, confere/baixa o driver JDBC, lista fontes e executa <code>javac --release 21</code> em <code>target/classes</code> . |
| executar.ps1                          | Monta o classpath e inicia Main, encaminhando inicialização, migração ou diagnóstico.                                                |
| testar-java.ps1                       | Compila/roda as verificações locais e, com opção de integração, os cenários PostgreSQL.                                              |
| testar-postgres.ps1                   | Cria cluster isolado na porta 55439, prepara schema, executa integração e opcionalmente web/navegador; encerra o cluster no finally. |
| scripts/configurar-java.ps1           | Localiza/configura o JDK usado pelos scripts da sessão.                                                                              |
| scripts/sync-demo.mjs                 | Sincroniza valores da massa fictícia original, preservando registros que já foram editados.                                          |
| scripts/visual-check.mjs              | Auxilia a conferência visual da interface por automação.                                                                             |
| pom.xml                               | Configuração Maven de compilação Java e dependência PostgreSQL JDBC.                                                                 |
| package.json / package-lock.json      | Declaram scripts/dependências Node; o lock fixa a resolução instalada.                                                               |
| playwright.config.mjs                 | Configura testes reais do navegador, servidor e encerramento do banco de teste.                                                      |
| tests/*.test.mjs                      | Testes Node da API e operações. <code>tests/helpers/postgres.mjs</code> fornece banco isolado.                                       |
| tests/e2e/airport.spec.mjs            | Fluxos da interface com navegador: login, reserva, check-in, cadastros e validações.                                                 |
| index.html / public/                  | Ponto HTML de montagem e recursos estáticos, como foto e favicon.                                                                    |
| config/application.properties.example | Exemplo da configuração local Java; o arquivo real contém valores do ambiente.                                                       |
| docs/                                 | Modelagem, dicionário, tópicos, diagramas, operação, apresentação e checklist.                                                       |
| lib/                                  | Driver JDBC usado no classpath.                                                                                                      |
| target/ e dist/                       | Resultados de compilação Java e construção web, respectivamente.                                                                     |
| logs/ e relatorios/                   | Saídas geradas em execução, conforme configuração/ações.                                                                             |
| artifacts/ e test-results/            | Ferramentas/evidências de teste e resultados de navegador; não são o fonte das regras.                                               |
| tmp/pdfs/ e output/pdf/               | Materiais de elaboração deste guia e o PDF entregue.                                                                                 |

## Como os testes demonstram as conexões

UnidadeTeste roda sem PostgreSQL e concentra invariantes locais. IntegracaoTeste usa dependências reais, um Clock controlável e dados fictícios no banco isolado. As funções `verificar/rejeitar` substituem um framework de asserções: falso ou sucesso inesperado lança `AssertionError`. `ExecutorService/Future/barreiras` permitem provocar disputas e observar resultados. `ProcessBuilder` abre outro processo Java para testar as entradas e saídas do terminal. O relatório descreve esses testes; não declara uma nova execução deles nesta tarefa de documentação.

### Cuidado ao interpretar as saídas

Passar um teste demonstra o cenário representado por aquele teste, não todas as entradas possíveis. Dados de demonstração e pagamentos simulados não comprovam integração externa. A lista de cobertura abaixo comprova que o fonte Java inteiro foi incluído no guia; ela não é um índice de cobertura de execução de testes.

# Índice de métodos e conferência de cobertura

Incluídos 67 arquivos, 4314 linhas físicas e 325 métodos/construtores declarados. Cada linha original tem registro de inclusão. Não foram encontrados erros sintáticos pelo analisador de estrutura usado para a documentação. Isso não substitui javac nem os testes.

## MenuOperacional

src/main/java/br/edu/aeroporto/cli/MenuOperacional.java

Linhas 187; métodos/construtores 18; SHA-256 completo: 5fd94e29e53908112c6626b27684f9ef252a8d42cf2690c0bde1f9aa803acba1

| Método/construtor | Linhas  | Saída declarada |
|-------------------|---------|-----------------|
| MenuOperacional   | 27-33   | Construtor      |
| executar          | 35-42   | void            |
| passageiros       | 44-48   | void            |
| cadastros         | 50-59   | void            |
| perfis            | 61-61   | Set<String>     |
| acesso            | 63-70   | void            |
| programacao       | 72-75   | ProgramacaoVoo  |
| planejamento      | 77-91   | void            |
| trecho            | 93-93   | PedidoTrecho    |
| reservas          | 95-102  | void            |
| rascunho          | 104-125 | void            |
| financeiro        | 127-136 | void            |
| atendimento       | 138-148 | void            |
| operacao          | 150-165 | void            |
| relatorios        | 167-175 | void            |
| arquivos          | 177-183 | void            |
| escolher          | 185-185 | T               |
| mostrar           | 186-186 | void            |

## MenuPrincipal

src/main/java/br/edu/aeroporto/cli/MenuPrincipal.java

Linhas 168; métodos/construtores 10; SHA-256 completo: c21cb107a14d2811e939d2e9f540a4bc0aa413005cef24d5b0ce3fe7623b29fd

| Método/construtor   | Linhas  | Saída declarada |
|---------------------|---------|-----------------|
| MenuPrincipal       | 34-43   | Construtor      |
| executar            | 45-98   | void            |
| cadastrarPassageiro | 100-106 | void            |

| Método/construtor | Linhas  | Saída declarada |
|-------------------|---------|-----------------|
| buscarPassageiros | 108-114 | void            |
| mostrar           | 116-119 | void            |
| consultarVoos     | 121-125 | List<VooResumo> |
| mostrarVoos       | 127-136 | void            |
| mostrarAssentos   | 138-145 | void            |
| mostrarReserva    | 147-156 | void            |
| exportarVoos      | 158-167 | void            |

## Terminal

src/main/java/br/edu/aeroporto/cli/Terminal.java

Linhas 107; métodos/construtores 11; SHA-256 completo: cb829cf5ec6f967cbae8d1667d686c33066e2c3f567ec277e44e215855cb3371

| Método/construtor | Linhas  | Saída declarada      |
|-------------------|---------|----------------------|
| ler               | 12-14   | String               |
| lerBruto          | 16-25   | String               |
| senha             | 27-35   | char[]               |
| inteiro           | 37-47   | int                  |
| id                | 49-59   | long                 |
| data              | 61-69   | LocalDate            |
| instante          | 71-76   | java.time.Instant    |
| decimal           | 78-84   | java.math.BigDecimal |
| idOpcional        | 86-92   | Long                 |
| sim               | 94-101  | boolean              |
| close             | 103-104 | void                 |

## Configuracao

src/main/java/br/edu/aeroporto/config/Configuracao.java

Linhas 42; métodos/construtores 3; SHA-256 completo: 96003a4b26ca7ddef5f39088889d645ab5df702cbb009e9aae3bd57d637ef7a

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| carregar          | 13-31  | Configuracao    |
| valor             | 33-36  | String          |
| toString          | 38-41  | String          |

## CadastroPassageiro

src/main/java/br/edu/aeroporto/dominio/CadastroPassageiro.java



Linhas 20; métodos/construtores 1; SHA-256 completo: c25f936692b5c65cc50bad6a9c2c0428dc6caa0487f81f3a8dd4ebfedaebec6

| Método/construtor  | Linhas | Saída declarada |
|--------------------|--------|-----------------|
| CadastroPassageiro | 8-19   | Construtor      |

## Dados

src/main/java/br/edu/aeroporto dominio/Dados.java

Linhas 51; métodos/construtores 8; SHA-256 completo: 907d60090795b49caf275a1d4a5e62eb2c8c74d7cb45e300c923d0d52cf15a02

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| Dados             | 12-12  | Construtor      |
| id                | 14-18  | long            |
| texto             | 20-22  | String          |
| dinheiro          | 24-27  | BigDecimal      |
| valor             | 29-34  | BigDecimal      |
| instante          | 36-42  | Instant         |
| exigir            | 44-46  | void            |
| codigo            | 48-50  | String          |

## Funcionario

src/main/java/br/edu/aeroporto dominio/Funcionario.java

Linhas 15; métodos/construtores 2; SHA-256 completo: 04e4d9070246966320d0581753cf7ac448a1e2b296731b48be2be4597f3eb902

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| Funcionario       | 8-11   | Construtor      |
| identificacao     | 13-14  | String          |

## MapaAssentos

src/main/java/br/edu/aeroporto dominio/MapaAssentos.java

Linhas 37; métodos/construtores 5; SHA-256 completo: 37a7ac035c02069411d881a422d7e0d83df0be9eb4024054c37a936a127d928d

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| MapaAssentos      | 12-24  | Construtor      |
| filas             | 26-26  | int[]           |
| colunas           | 27-27  | String[]        |
| celulas           | 28-32  | String[][]      |
| mesmoMapa         | 34-36  | boolean         |

## Passageiro

src/main/java/br/edu/aeroporto dominio/Passageiro.java

Linhas 17; métodos/construtores 3; SHA-256 completo: c9da87d583289d7b3f826bb31fe9cb5e60609b5e24165a11ddb74db7740bb508

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| Passageiro        | 8-11   | Construtor      |
| codigoCliente     | 13-13  | String          |
| identificacao     | 15-16  | String          |

## PedidoTrecho

src/main/java/br/edu/aeroporto/dominio/PedidoTrecho.java

Linhas 11; métodos/construtores 1; SHA-256 completo: e379e5763c52e87b491cd10a3507acabd026aee53e78f6801573303b5939b141

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| PedidoTrecho      | 6-10   | Construtor      |

## Pessoa

src/main/java/br/edu/aeroporto/dominio/Pessoa.java

Linhas 37; métodos/construtores 9; SHA-256 completo: 745d08f1c5715a9353601390101a811cdf97e17798dc4c59a7f69315bdb31a78

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| Pessoa            | 12-18  | Construtor      |
| id                | 20-20  | long            |
| nome              | 21-21  | String          |
| nascimento        | 22-22  | LocalDate       |
| ativa             | 23-23  | boolean         |
| identificacao     | 24-24  | String          |
| equals            | 26-30  | boolean         |
| hashCode          | 32-33  | int             |
| toString          | 35-36  | String          |

## PrimeiroAcesso

src/main/java/br/edu/aeroporto/dominio/PrimeiroAcesso.java

Linhas 18; métodos/construtores 1; SHA-256 completo: c5d301f4fe85bd2195cbd96cb6fdbf8c61e19921caf5cae2ffa9d42a67658f59

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| PrimeiroAcesso    | 9-17   | Construtor      |

## ProgramacaoVoo

src/main/java/br/edu/aeroporto/dominio/ProgramacaoVoo.java

Linhas 25; métodos/construtores 2; SHA-256 completo: d6e191ae01673e111b7deed4f2942618fabb9012e2dfa060f82cdb7c7cd4c462

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| ProgramacaoVoo    | 7-19   | Construtor      |

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| validarHorarios   | 21-24  | void            |

## Sessao

src/main/java/br/edu/aeroporto/dominio/Sessao.java

Linhas 10; métodos/construtores 1; SHA-256 completo: 17fa279266d978df09ec88b8b58ef5b85563c310f95b4e0db6a236835695ee7e

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| Sessao            | 6-8    | Construtor      |

## SituacaoReserva

src/main/java/br/edu/aeroporto/dominio/SituacaoReserva.java

Linhas 5; métodos/construtores 0; SHA-256 completo: b759d75e0ad66f4be37fb0daee1efd93626c2a46e0a68d05ea3c5627aedd5d9

Não possui métodos/construtores explícitos no fonte; constantes/componentes e membros gerados são explicados na leitura do tipo.

## SituacaoVoo

src/main/java/br/edu/aeroporto/dominio/SituacaoVoo.java

Linhas 17; métodos/construtores 1; SHA-256 completo: 56bd5f33b3751d56a759c6432fc1646c93d022d1346c150f0c881f3d363b28ff

| Método/construtor    | Linhas | Saída declarada |
|----------------------|--------|-----------------|
| permiteTransicaoPara | 6-16   | boolean         |

## Tarifa

src/main/java/br/edu/aeroporto/dominio/Tarifa.java

Linhas 15; métodos/construtores 1; SHA-256 completo: 1a66897a938bcb4cd235c1b79c0607e5cc84b959eec9e8a236e90859336f8c43

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| Tarifa            | 8-14   | Construtor      |

## TipoCadastro

src/main/java/br/edu/aeroporto/dominio/TipoCadastro.java

Linhas 67; métodos/construtores 5; SHA-256 completo: 848a4ebf8c17a109bec70e090f16df3b5906b04ebfa8ddedb2a79f323ac46e72

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| TipoCadastro      | 32-39  | Construtor      |
| tabela            | 41-41  | String          |
| chave             | 42-42  | String          |
| campos            | 43-43  | List<Campo>     |
| converter         | 46-65  | Object          |

## Validacao

src/main/java/br/edu/aeroporto/dominio/Validacao.java

Linhas 44; métodos/construtores 5; SHA-256 completo: 2a9e749cb9208f27f290af93eaaa94a3ac89ebc0b9ef31332fbfef1fc5873d95

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| Validacao         | 8-8    | Construtor      |
| texto             | 10-18  | String          |
| nascimento        | 20-23  | LocalDate       |
| login             | 25-27  | String          |
| cpf               | 29-43  | String          |

## AssentoResumo

src/main/java/br/edu/aeroporto/dto/AssentoResumo.java

Linhas 5; métodos/construtores 1; SHA-256 completo: 9376e27ecb7b9635f534c86cd512c2d1d1369d53b0561e050991e500b658f518

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| livre             | 4-4    | boolean         |

## ReservaResumo

src/main/java/br/edu/aeroporto/dto/ReservaResumo.java

Linhas 18; métodos/construtores 2; SHA-256 completo: 1213a7b76236b1a66d450c3f00966c042fd19430011585e8a8db5bf41581e3c7

| Método/construtor      | Linhas | Saída declarada |
|------------------------|--------|-----------------|
| ReservaResumo          | 10-10  | Construtor      |
| totalOriginalPassagens | 15-17  | BigDecimal      |

## TipoRelatorio

src/main/java/br/edu/aeroporto/dto/TipoRelatorio.java

Linhas 52; métodos/construtores 3; SHA-256 completo: f4528e8d3fc635e4a306c17b5014d21cca75d42f57e5b76f726921ac97343358

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| TipoRelatorio     | 49-49  | Construtor      |
| sql               | 50-50  | String          |
| restrito          | 51-51  | boolean         |

## VooResumo

src/main/java/br/edu/aeroporto/dto/VooResumo.java

Linhas 15; métodos/construtores 1; SHA-256 completo: 109bd7f06e4d43328889146df6145ad4892be8cf81c522d2f19db8648ff4a9d4

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| atrasado          | 11-14  | boolean         |

## PersistenciaException

src/main/java/br/edu/aeroporto/excecao/PersistenciaException.java

Linhas 25; métodos/construtores 2; SHA-256 completo: e97addea6511d99dd238e897843b46c8df42d33573707b8aed3a99e4756ad764

| Método/construtor     | Linhas | Saída declarada |
|-----------------------|--------|-----------------|
| PersistenciaException | 6-8    | Construtor      |
| mensagem              | 10-24  | String          |

## RegraNegocioException

src/main/java/br/edu/aeroporto/excecao/RegraNegocioException.java

Linhas 7; métodos/construtores 1; SHA-256 completo: d1649821d6e52a6f9f2c7250501d86bc494160cdc891b9eb30a6735d410a9d4a

| Método/construtor     | Linhas | Saída declarada |
|-----------------------|--------|-----------------|
| RegraNegocioException | 4-6    | Construtor      |

## AuditoriaJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/AuditoriaJdbc.java

Linhas 19; métodos/construtores 2; SHA-256 completo: 67f62f8933c5e5d51c2d5fa0946d4ca58bec03ad3b8a0f6eb074391f1477820f

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| registrar         | 7-13   | void            |
| definirAutorDoVoo | 15-18  | void            |

## AutorizacaoJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/AutorizacaoJdbc.java

Linhas 57; métodos/construtores 6; SHA-256 completo: 84ad8999f6b072cb229c5b90c570d04e52ede70b66dda5b6960b6b7c36afb9a1

| Método/construtor   | Linhas | Saída declarada |
|---------------------|--------|-----------------|
| exigirConsulta      | 9-11   | void            |
| exigirAtendimento   | 13-15  | void            |
| exigirAdministrador | 17-19  | void            |
| exigirOperacao      | 21-23  | void            |
| exigirPerfil        | 25-37  | void            |
| exigir              | 39-56  | void            |

## BancoDados

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/BancoDados.java

Linhas 69; métodos/construtores 6; SHA-256 completo: dcac25b2bb912b88dfaf5cbd59b6b3eec76bc8c0468e6426d13cb1099a8d1387

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| BancoDados        | 13-15  | Construtor      |
| consultar         | 17-19  | T               |
| transacao         | 21-23  | T               |

| Método/construtor  | Linhas | Saída declarada |
|--------------------|--------|-----------------|
| transacaoRepetivel | 25-34  | T               |
| executar           | 36-56  | T               |
| abrir              | 58-68  | Connection      |

## BloqueiosJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/BloqueiosJdbc.java

Linhas 34; métodos/construtores 5; SHA-256 completo: f95714cf776b6a556724d665134f1819a36854e58f6bb93a311b2ce9d132c3fe

| Método/construtor | Linhas | Saída declarada     |
|-------------------|--------|---------------------|
| BloqueiosJdbc     | 11-11  | Construtor          |
| voo               | 13-15  | Map<String, Object> |
| voos              | 17-19  | void                |
| reserva           | 21-27  | Map<String, Object> |
| item              | 29-33  | Map<String, Object> |

## CadastroJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/CadastroJdbc.java

Linhas 35; métodos/construtores 3; SHA-256 completo: 70bfc2be68578eb656e7382fad108f43d4efdfca74e7ceae478f68d57c4c541

| Método/construtor | Linhas | Saída declarada           |
|-------------------|--------|---------------------------|
| salvar            | 12-26  | long                      |
| listar            | 28-30  | List<Map<String, Object>> |
| buscar            | 32-34  | Map<String, Object>       |

## ConsultasJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/ConsultasJdbc.java

Linhas 72; métodos/construtores 3; SHA-256 completo: ae1aaef99d691854fc2260887710c9c107acdee1a0e2e399ccc1ebacd47a5a6

| Método/construtor | Linhas | Saída declarada         |
|-------------------|--------|-------------------------|
| voos              | 18-38  | List<VooResumo>         |
| assentos          | 40-50  | List<AssentoResumo>     |
| reserva           | 52-71  | Optional<ReservaResumo> |

## FinanceiroJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/FinanceiroJdbc.java

Linhas 41; métodos/construtores 3; SHA-256 completo: 1cb5ccf7b0eaa164a6a84be503772237b8b7776b13e4ae054141f1f3e40c2ec1

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| devido            | 10-17  | BigDecimal      |

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| recebido          | 19-23  | BigDecimal      |
| confirmar         | 25-40  | void            |

## InicializacaoJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/InicializacaoJdbc.java

Linhas 63; métodos/construtores 1; SHA-256 completo: c0168dd7ee02b8097e346767ee501c9bfd02341d3ae7448622cb3d9b47f759f0

| Método/construtor  | Linhas | Saída declarada |
|--------------------|--------|-----------------|
| criarAdministrador | 12-62  | long            |

## OperacaoJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/OperacaoJdbc.java

Linhas 82; métodos/construtores 4; SHA-256 completo: 0b23184733675c1e98f38290dfaacc12bed91fb5777c7f97dd99ccea4bed3324

| Método/construtor   | Linhas | Saída declarada |
|---------------------|--------|-----------------|
| validarEscalasDoVoo | 14-18  | void            |
| validarEscala       | 20-55  | void            |
| validarTripulacao   | 57-65  | void            |
| validarRecurso      | 67-81  | void            |

## OperacaoSql

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/OperacaoSql.java

Linhas 9; métodos/construtores 1; SHA-256 completo: 5a0cc21df3fa43f652595d5a67ba5114eb29166f19d284aa10940a36ae740135

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| executar          | 8-8    | T               |

## PassageiroJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/PassageiroJdbc.java

Linhas 81; métodos/construtores 7; SHA-256 completo: 21da02d0438c93ed95d97cb657a61879001a410915f184bdc6d9a07e8de676ce

| Método/construtor | Linhas | Saída declarada      |
|-------------------|--------|----------------------|
| vincular          | 21-27  | Passageiro           |
| editar            | 29-35  | Passageiro           |
| cadastrar         | 37-54  | Passageiro           |
| buscar            | 56-59  | Optional<Passageiro> |
| buscar            | 61-68  | List<Passageiro>     |
| mapear            | 70-73  | Passageiro           |
| salvarContato     | 75-80  | void                 |



## PlanejamentoJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/PlanejamentoJdbc.java

Linhas 97; métodos/construtores 7; SHA-256 completo: 55fe312bf66c25050386fe86a459638c17ced03e66b24189d779f6be89083c64

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| serializar        | 11-13  | void            |
| validarFrota      | 15-41  | void            |
| programar         | 43-52  | long            |
| inventario        | 54-60  | void            |
| tarifa            | 62-74  | long            |
| agendas           | 76-91  | void            |
| efetivo           | 93-96  | Instant         |

## ReservaJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/ReservaJdbc.java

Linhas 171; métodos/construtores 9; SHA-256 completo: a90fd363edc6a589d6c816c6b41bd4fd99fc1cd5a19d888207ab7a96b460b937

| Método/construtor  | Linhas  | Saída declarada |
|--------------------|---------|-----------------|
| reservar           | 15-67   | long            |
| novoltem           | 69-74   | long            |
| janela             | 76-79   | Instant         |
| recalcular         | 81-91   | void            |
| cancelarItem       | 93-108  | BigDecimal      |
| invalidar          | 110-118 | void            |
| solicitarDevolucao | 120-133 | void            |
| limiteDevolucao    | 135-146 | BigDecimal      |
| expirar            | 148-170 | int             |

## Sql

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/Sql.java

Linhas 88; métodos/construtores 9; SHA-256 completo: 6c3b68520c5ef24a3e047d7fdc3f41da22d7e7f7f605b9b8296680ed68c78b4a

| Método/construtor | Linhas | Saída declarada     |
|-------------------|--------|---------------------|
| Sql               | 18-18  | Construtor          |
| linha             | 20-27  | Map<String, Object> |
| registro          | 29-32  | Map<String, Object> |
| mapear            | 36-36  | T                   |
| listar            | 39-47  | List<T>             |

| Método/construtor | Linhas | Saída declarada   |
|-------------------|--------|-------------------|
| unico             | 49-58  | Optional<T>       |
| inserir           | 60-63  | long              |
| executar          | 65-69  | int               |
| preparar          | 71-87  | PreparedStatement |

## UsuarioJdbc

src/main/java/br/edu/aeroporto/infraestrutura/jdbc/UsuarioJdbc.java

Linhas 38; métodos/construtores 2; SHA-256 completo: bf7d6eebf13d12b3ef619ba91b76f05154f2a1ac97c82c37d7188e5b5f9b8218

| Método/construtor | Linhas | Saída declarada       |
|-------------------|--------|-----------------------|
| buscarAtivo       | 13-32  | Optional<Credenciais> |
| existeUsuario     | 34-37  | boolean               |

## Main

src/main/java/br/edu/aeroporto/Main.java

Linhas 176; métodos/construtores 5; SHA-256 completo: 535b2289cbf1647c9fe072497922e0807e0937dfa6b285868dbb7578f223e560

| Método/construtor | Linhas  | Saída declarada |
|-------------------|---------|-----------------|
| Main              | 53-53   | Construtor      |
| main              | 55-141  | void            |
| entrar            | 143-154 | Sessao          |
| inicializar       | 156-163 | void            |
| configurarLog     | 165-175 | void            |

## ExportadorCsv

src/main/java/br/edu/aeroporto/relatorio/ExportadorCsv.java

Linhas 31; métodos/construtores 3; SHA-256 completo: 4a4d8d30276e2d952723bc09775afb191f878e7b67f3d123ac2a329fb3296d06

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| extensao          | 9-10   | String          |
| escrever          | 12-24  | void            |
| campo             | 26-30  | String          |

## ExportadorTxt

src/main/java/br/edu/aeroporto/relatorio/ExportadorTxt.java

Linhas 26; métodos/construtores 2; SHA-256 completo: 3bfd3386d2eb9475567d06bef75303757055d77554399bb7352e4f66e713890

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| extensao          | 12-13  | String          |

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| escrever          | 15-25  | void            |

## ExportadorVoos

src/main/java/br/edu/aeroporto/relatorio/ExportadorVoos.java

Linhas 11; métodos/construtores 2; SHA-256 completo: 52ae54dc9822e84787f99445ae9a9806f80528f66c79c54f106ebee28f379d0a

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| extensao          | 9-9    | String          |
| escrever          | 10-10  | void            |

## ConsultasRepositorio

src/main/java/br/edu/aeroporto/repositorio/ConsultasRepositorio.java

Linhas 16; métodos/construtores 3; SHA-256 completo: c0d1689b61bf55fa6688f4aedeae19f149efbacce0ed1190ea34ecc45342cfd0

| Método/construtor | Linhas | Saída declarada         |
|-------------------|--------|-------------------------|
| voos              | 13-13  | List<VooResumo>         |
| assentos          | 14-14  | List<AssentoResumo>     |
| reserva           | 15-15  | Optional<ReservaResumo> |

## PassageiroRepositorio

src/main/java/br/edu/aeroporto/repositorio/PassageiroRepositorio.java

Linhas 17; métodos/construtores 5; SHA-256 completo: 6456b29d1ec60d8e896d82bd1941f72d4cd24a1e1e70fa755477d5a26c2a05fa

| Método/construtor | Linhas | Saída declarada      |
|-------------------|--------|----------------------|
| vincular          | 11-11  | Passageiro           |
| editar            | 12-13  | Passageiro           |
| cadastrar         | 14-14  | Passageiro           |
| buscar            | 15-15  | Optional<Passageiro> |
| buscar            | 16-16  | List<Passageiro>     |

## UsuarioRepositorio

src/main/java/br/edu/aeroporto/repositorio/UsuarioRepositorio.java

Linhas 16; métodos/construtores 3; SHA-256 completo: 8071b9287da7e4697b853ca0df43a03180d0fc82ca726ce74d8b60c93fdbd850

| Método/construtor | Linhas | Saída declarada       |
|-------------------|--------|-----------------------|
| toString          | 10-11  | String                |
| buscarAtivo       | 14-14  | Optional<Credenciais> |
| existeUsuario     | 15-15  | boolean               |

## Senhas

src/main/java/br/edu/aeroporto/seguranca/Senhas.java

Linhas 57; métodos/construtores 4; SHA-256 completo: 1c1f0b71ce8b2a9aaf74584374f48c3a70e9f2351ae32c4e0d8d50a87d022b98

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| Senhas            | 15-15  | Construtor      |
| gerar             | 17-27  | String          |
| conferir          | 29-45  | boolean         |
| derivar           | 47-56  | byte[]          |

## AdministracaoServico

src/main/java/br/edu/aeroporto/servico/AdministracaoServico.java

Linhas 83; métodos/construtores 7; SHA-256 completo: 2fb9f3b89785f5c765bae253dc40b3586210fe8977c2b1bbd1298f3df37e2d66

| Método/construtor    | Linhas | Saída declarada          |
|----------------------|--------|--------------------------|
| AdministracaoServico | 17-19  | Construtor               |
| usuarios             | 21-30  | List<Map<String,Object>> |
| criarUsuario         | 32-45  | long                     |
| acesso               | 47-65  | void                     |
| alterarSenha         | 67-79  | void                     |
| validarSenha         | 81-81  | void                     |
| validarPerfis        | 82-82  | void                     |

## ArquivosServico

src/main/java/br/edu/aeroporto/servico/ArquivosServico.java

Linhas 77; métodos/construtores 5; SHA-256 completo: cad6fe0b9ff60819296b7635c1e31a3de91ef10022b6d897d13b4777a0bf9328

| Método/construtor   | Linhas | Saída declarada |
|---------------------|--------|-----------------|
| ArquivosServico     | 20-22  | Construtor      |
| importarPassageiros | 24-52  | int             |
| listar              | 54-58  | List<Path>      |
| copiarRelatorio     | 60-71  | Path            |
| hash                | 73-76  | byte[]          |

## AtendimentoServico

src/main/java/br/edu/aeroporto/servico/AtendimentoServico.java

Linhas 161; métodos/construtores 10; SHA-256 completo: fc956da919e23ff818c59b50525b04f95336b51ab32a1fe71792bed1d89fc646

| Método/construtor  | Linhas  | Saída declarada    |
|--------------------|---------|--------------------|
| AtendimentoServico | 28-30   | Construtor         |
| checkin            | 32-58   | long               |
| cancelarCheckin    | 60-74   | void               |
| cartao             | 76-86   | Map<String,Object> |
| despachar          | 88-104  | long               |
| excesso            | 106-109 | BigDecimal         |
| rastrear           | 111-130 | void               |
| embarcar           | 132-151 | long               |
| evento             | 153-156 | void               |
| janela             | 158-160 | void               |

## AutenticacaoServico

src/main/java/br/edu/aeroporto/servico/AutenticacaoServico.java

Linhas 46; métodos/construtores 3; SHA-256 completo: 6bbda729c6efc2a5a1fc6352b19602cde4d5053931a60a74f3c99d72432b5ad2

| Método/construtor   | Linhas | Saída declarada |
|---------------------|--------|-----------------|
| AutenticacaoServico | 17-21  | Construtor      |
| existeUsuario       | 23-25  | boolean         |
| entrar              | 27-45  | Sessao          |

## CadastroServico

src/main/java/br/edu/aeroporto/servico/CadastroServico.java

Linhas 135; métodos/construtores 6; SHA-256 completo: 932fea6819fad4f3086486dff4b57659d924d1e11680eebf3c34986bfef081c

| Método/construtor | Linhas  | Saída declarada           |
|-------------------|---------|---------------------------|
| CadastroServico   | 21-23   | Construtor                |
| listar            | 25-28   | List<Map<String, Object>> |
| salvar            | 30-46   | long                      |
| permitir          | 48-51   | void                      |
| validar           | 53-130  | void                      |
| existe            | 132-134 | boolean                   |

## ConsultaServico

src/main/java/br/edu/aeroporto/servico/ConsultaServico.java

Linhas 52; métodos/construtores 4; SHA-256 completo: 35f0a56cb83741436ce706d08b6a98c369080655b5b282c10d2339fb26a02298

| Método/construtor | Linhas | Saída declarada     |
|-------------------|--------|---------------------|
| ConsultaServico   | 22-26  | Construtor          |
| voos              | 28-35  | List<VooResumo>     |
| assentos          | 37-43  | List<AssentoResumo> |
| reserva           | 45-51  | ReservaResumo       |

## DadosFicticiosServico

src/main/java/br/edu/aeroporto/servico/DadosFicticiosServico.java

Linhas 80; métodos/construtores 5; SHA-256 completo: 2f437a8cf3c652ad5f3600ea187a65745f3954c06889fcd951d0612d0b8368e1

| Método/construtor     | Linhas | Saída declarada    |
|-----------------------|--------|--------------------|
| DadosFicticiosServico | 15-17  | Construtor         |
| criar                 | 19-21  | Map<String,Object> |
| criar                 | 23-68  | Map<String,Object> |
| icao                  | 70-70  | String             |
| codigoLivre           | 72-79  | String             |

## FinanceiroServico

src/main/java/br/edu/aeroporto/servico/FinanceiroServico.java

Linhas 101; métodos/construtores 9; SHA-256 completo: 5f514d0bf90e498b7cdf3761cc8b95fa50bf14e0063d54b4d40fc280f029c33f

| Método/construtor  | Linhas | Saída declarada |
|--------------------|--------|-----------------|
| FinanceiroServico  | 18-20  | Construtor      |
| pagar              | 22-39  | long            |
| validarReserva     | 41-45  | void            |
| processar          | 47-53  | void            |
| cancelarPagamento  | 55-61  | void            |
| confirmar          | 63-65  | void            |
| saldo              | 67-69  | BigDecimal      |
| solicitarReembolso | 71-87  | long            |
| processarReembolso | 89-100 | void            |

## InicializacaoServico

src/main/java/br/edu/aeroporto/servico/InicializacaoServico.java

Linhas 42; métodos/construtores 2; SHA-256 completo: 154aa473fffaafd7adf5122dd2a4082ef5d2bcc29074de75358cdb91186a5f8d

| Método/construtor    | Linhas | Saída declarada |
|----------------------|--------|-----------------|
| InicializacaoServico | 20-25  | Construtor      |

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| criar             | 27-41  | void            |

## OperacaoServico

src/main/java/br/edu/aeroporto/servico/OperacaoServico.java

Linhas 173; métodos/construtores 14; SHA-256 completo: 58aba7c7caa438e58b24082f87f7bee78023c5d4e9d76accac5ca96bccfad15c

| Método/construtor  | Linhas  | Saída declarada |
|--------------------|---------|-----------------|
| OperacaoServico    | 20-22   | Construtor      |
| alocar             | 24-31   | long            |
| interditar         | 33-41   | long            |
| liberarAlocacao    | 43-54   | void            |
| habilitar          | 56-68   | long            |
| escalar            | 70-79   | long            |
| liberarEscala      | 81-92   | void            |
| manutencao         | 94-105  | long            |
| situacaoManutencao | 107-128 | void            |
| servicoSolo        | 130-142 | long            |
| situacaoSolo       | 144-153 | void            |
| ocorrencia         | 155-164 | long            |
| encerrarOcorrencia | 166-168 | void            |
| funcionarioAtivo   | 170-172 | void            |

## PassageiroServico

src/main/java/br/edu/aeroporto/servico/PassageiroServico.java

Linhas 88; métodos/construtores 6; SHA-256 completo: 12d86eb08ea3b44002111a6f364b37d8654a9b653163b318151779253c2ed789

| Método/construtor | Linhas | Saída declarada  |
|-------------------|--------|------------------|
| PassageiroServico | 25-32  | Construtor       |
| cadastrar         | 34-42  | Passageiro       |
| vincular          | 44-53  | Passageiro       |
| editar            | 55-69  | Passageiro       |
| buscar            | 71-78  | Passageiro       |
| buscar            | 80-87  | List<Passageiro> |

## PlanejamentoServico

src/main/java/br/edu/aeroporto/servico/PlanejamentoServico.java

Linhas 110; métodos/construtores 7; SHA-256 completo: 70e7bab8623fd085d3f9356abf8e9a8ad02d4077660bfe18f0d25a39f834862f

| Método/construtor   | Linhas | Saída declarada |
|---------------------|--------|-----------------|
| PlanejamentoServico | 16-18  | Construtor      |
| programar           | 20-28  | long            |
| tarifa              | 30-38  | long            |
| bloquearAssento     | 40-50  | void            |
| reprogramar         | 52-69  | void            |
| horarios            | 71-91  | void            |
| trocarAeronave      | 93-109 | void            |

## RelatoriosCompletosServico

src/main/java/br/edu/aeroporto/servico/RelatoriosCompletosServico.java

Linhas 104; métodos/construtores 9; SHA-256 completo: 3f3026aa89c28df3b54f47b80759f4b1b101acee71efde5e15918078fdf9b36a

| Método/construtor          | Linhas | Saída declarada                  |
|----------------------------|--------|----------------------------------|
| RelatoriosCompletosServico | 19-21  | Construtor                       |
| permitir                   | 23-27  | void                             |
| consultar                  | 29-32  | List<Map<String,Object>>         |
| painel                     | 34-42  | List<Map<String,Object>>         |
| mapa                       | 44-49  | MapaAssentos                     |
| bagagem                    | 51-54  | List<Map<String,Object>>         |
| resumoAtrasos              | 56-66  | Map<String,java.math.BigDecimal> |
| exportar                   | 68-93  | Path                             |
| escrever                   | 95-103 | void                             |

## RelatorioServico

src/main/java/br/edu/aeroporto/servico/RelatorioServico.java

Linhas 29; métodos/construtores 1; SHA-256 completo: c85fd3ec3dbca1a00961ba8947dace054147c0ed0123845f12660608e58986df

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| exportar          | 14-28  | Path            |

## ReservaServico

src/main/java/br/edu/aeroporto/servico/ReservaServico.java

Linhas 66; métodos/construtores 6; SHA-256 completo: a87255bb63188175b1cc2879e970df745efb4a2b464695c9958b5c427471096f

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| ReservaServico    | 18-20  | Construtor      |



| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| expirarPendencias | 22-22  | int             |
| expirar           | 24-24  | int             |
| reservar          | 26-34  | long            |
| cancelar          | 36-47  | void            |
| remarcar          | 49-65  | long            |

## VooServico

src/main/java/br/edu/aeroporto/servico/VooServico.java

Linhas 73; métodos/construtores 2; SHA-256 completo: 2e4086f381b32d9cc3b6a2d74b707a3c59e04c66f5a2a7bdd57b31095a024d06

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| VooServico        | 18-20  | Construtor      |
| transicao         | 22-72  | void            |

## IntegracaoTeste

src/test/java/br/edu/aeroporto/IntegracaoTeste.java

Linhas 518; métodos/construtores 33; SHA-256 completo: 37f33cf89e2a1ec37b8a0248f35edb7f54b3e2a481d87cca9c6d2f6f671e36a5

| Método/construtor        | Linhas  | Saída declarada |
|--------------------------|---------|-----------------|
| getZone                  | 19-19   | Zoneld          |
| withZone                 | 20-20   | Clock           |
| instant                  | 21-21   | Instant         |
| definir                  | 22-22   | void            |
| main                     | 45-48   | void            |
| inicializar              | 50-60   | void            |
| cadastros                | 62-107  | void            |
| comercial                | 109-125 | void            |
| atendimento              | 127-151 | void            |
| arquivos                 | 153-162 | void            |
| concorrenca              | 164-188 | void            |
| cancelamentos            | 190-201 | void            |
| cenariosAdicionais       | 203-293 | void            |
| bagagensEmConexao        | 295-307 | void            |
| cancelamentoAposEmbarque | 309-332 | void            |
| falhasEDeadlock          | 334-353 | void            |

| Método/construtor         | Linhas  | Saída declarada    |
|---------------------------|---------|--------------------|
| testarTerminal            | 355-362 | void               |
| novoAviao                 | 364-367 | long               |
| regrasComplementares      | 369-416 | void               |
| testarRascunho            | 418-429 | void               |
| administracaoElImportacao | 431-458 | void               |
| cadastrosRestantes        | 460-480 | void               |
| executarTerminal          | 482-493 | String             |
| cpf                       | 495-498 | String             |
| texto                     | 500-500 | String             |
| programacao               | 502-502 | ProgramacaoVoo     |
| icao                      | 503-509 | String             |
| cad                       | 510-512 | long               |
| mapa                      | 513-513 | Map<String,Object> |
| contar                    | 514-514 | long               |
| numero                    | 515-515 | long               |
| verificar                 | 516-516 | void               |
| rejeitar                  | 517-517 | void               |

## UnidadeTeste

src/test/java/br/edu/aeroporto/UnidadeTeste.java

Linhas 32; métodos/construtores 3; SHA-256 completo: 6673212555cabbab14c60fb52bccdb3e203f406473efb1316039c392e85279c

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| main              | 10-28  | void            |
| assertiva         | 30-30  | void            |
| falha             | 31-31  | void            |

## PrepararBancoTeste

scripts/PrepararBancoTeste.java

Linhas 23; métodos/construtores 1; SHA-256 completo: a5ecd4283fc878cfd5d983377c862ad8530b4949d71886eac3cc4bfed0946edb

| Método/construtor | Linhas | Saída declarada |
|-------------------|--------|-----------------|
| main              | 6-22   | void            |

# Roteiro de estudo e referências

## Primeiro: leia uma declaração

Abra Pessoa e identifique private, final e long. Encontre a diferença entre campo id, parâmetro id e método id(). Explique por que this.id = id não é recursão. Depois compare com Terminal.idOpcional, que retorna Long.

## Depois: acompanhe um retorno

Parta de MenuPrincipal.cadastrarPassageiro. Marque a construção de CadastroPassageiro, chamada do serviço, transação, inserção SQL e criação de Passageiro. Siga o valor que volta até printf. Identifique o ponto exato em que commit ocorre.

## Em seguida: acompanhe uma recusa

Escolha a regra de conexão de 60 minutos em ReservaJdbc. Escreva quais dados entram na condição, qual exceção ocorre se não forem aceitos e por que a reserva não pode ficar parcialmente persistida. Compare com o teste que espera rollback.

## Depois: compare caminhos

Compare a reserva Java com createReservation web. Identifique os schemas e estados usados por cada aplicação. Trace o JSON da interface e o DTO do menu: ambos transportam dados, mas pertencem a caminhos diferentes.

## Por último: explique sem decorar

Para cada método, responda: quem pode chamá-lo; o que entra; quais condições permitem avançar; que dependência usa; que efeito produz; o que devolve; como falha. Volte ao código quando uma resposta parecer genérica demais. Os números de linha e o sumário permitem conferir cada afirmação.

# Documentação oficial para aprofundar

## Java Language Specification 21 - Classes

Regras formais de classes, membros, construtores, herança e records.

## Java 21 API - Connection

Contrato de commit, rollback, isolamento e fechamento da conexão JDBC.

## Java 21 API - Optional

Contrato das operações de presença, ausência e extração.

## React - Your First Component

Componentes e retorno de interface em JSX.

A fonte principal deste guia é o próprio código do projeto. As referências oficiais acima complementam os conceitos gerais; as regras de negócio foram descritas a partir dos arquivos, sem atribuir a elas caráter de norma externa.